

Reference Manual

CKS32F103x8 and CKS32F103xB advanced ARM®-based 32-bit MCUs

Introduction

This reference manual is addressed to application developers.

It provides complete information on how to use the CKS32F103xx microcontroller memory and peripherals.

For information on the Arm® Cortex®-M3 core, refer to the *CKS32F103xx Cortex®-M3 programming manual*.

Related documents

- CKS32F103xx datasheet
- CKS32F10xxx Cortex®-M3 programming manual

CKS copyright

Contents

1. Documentation conventions	1
1.1 List of abbreviations for registers	1
1.2 Peripheral availability	1
2. Memory and bus architecture	2
2.1 System architecture	2
2.2 Memory organization	3
2.3 Memory map	3
2.3.1 Embedded SRAM	4
2.3.2 Bit banding	4
2.3.3 Embedded Flash memory	5
2.4 Boot configuration	6
3. CRC calculation unit	8
3.1 CRC introduction	8
3.2 CRC main features	8
3.3 CRC functional description	8
3.4 CRC registers	9
3.4.1 Data register (CRC_DR)	9
3.4.2 Independent data register (CRC_IDR)	9
3.4.3 Control register (CRC_CR)	9
3.4.4 CRC register map	10
4. Power control (PWR)	11
4.1 Power supplies	11
4.1.1 Independent A/D and D/A converter supply and reference voltage	11
4.1.2 Battery backup domain	12
4.1.3 Voltage regulator	12
4.2 Power supply supervisor	13
4.2.1 Power on reset (POR)/power down reset (PDR)	13
4.2.2 Programmable voltage detector (PVD)	13
4.3 Low-power modes	14
4.3.1 Slowing down system clocks	15
4.3.2 Peripheral clock gating	15
4.3.3 Sleep mode	15
4.3.4 Stop mode	16
4.3.5 Standby mode	17
4.3.6 Auto-wakeup (AWU) from low-power mode	18
4.4 Power control registers	18
4.4.1 Power control register (PWR_CR)	18
4.4.2 Power control/status register (PWR_CSR)	19
4.4.3 PWR register map	20
5. Backup registers (BKP)	21
5.1 BKP introduction	21
5.2 BKP main features	21
5.3 BKP functional description	21
5.3.1 Tamper detection	21
5.3.2 RTC calibration	22
5.4 BKP registers	22
5.4.1 Backup data register x (BKP_DRx) (x = 1 ... 10)	22
5.4.2 RTC clock calibration register (BKP_RTCCR)	22
5.4.3 Backup control register (BKP_CR)	23
5.4.4 Backup control/status register (BKP_CSR)	23
5.4.5 BKP register map	24
6. Reset and clock control (RCC)	32

6.1	Reset	32
6.1.1	System reset	32
6.1.2	Power reset	32
6.1.3	Backup domain reset	33
6.2	Clocks	33
6.2.1	HSE clock	35
6.2.2	HSI clock	36
6.2.3	PLL	36
6.2.4	LSE clock	36
6.2.5	LSI clock	37
6.2.6	System clock (SYSCLK) selection	37
6.2.7	Clock security system (CSS)	37
6.2.8	RTC clock	37
6.2.9	Watchdog clock	38
6.2.10	Clock-out capability	38
6.3	RCC registers	38
6.3.1	Clock control register (RCC_CR)	38
6.3.2	Clock configuration register (RCC_CFGR)	39
6.3.3	Clock interrupt register (RCC_CIR)	41
6.3.4	APB2 peripheral reset register (RCC_APB2RSTR)	43
6.3.5	APB1 peripheral reset register (RCC_APB1RSTR)	44
6.3.6	AHB peripheral clock enable register (RCC_AHBENR)	45
6.3.7	APB2 peripheral clock enable register (RCC_APB2ENR)	45
6.3.8	APB1 peripheral clock enable register (RCC_APB1ENR)	46
6.3.9	Backup domain control register (RCC_BDCR)	48
6.3.10	Control/status register (RCC_CSR)	48
6.3.11	RCC register map	49
7	General-purpose and alternate-function I/Os (GPIOs and AFIOs)	52
7.1	GPIO functional description	52
7.1.1	General-purpose I/O (GPIO)	54
7.1.2	Atomic bit set or reset	54
7.1.3	External interrupt/wakeup lines	54
7.1.4	Alternate functions (AF)	54
7.1.5	Software remapping of I/O alternate functions	55
7.1.6	GPIO locking mechanism	55
7.1.7	Input configuration	55
7.1.8	Output configuration	56
7.1.9	Alternate function configuration	57
7.1.10	Analog configuration	57
7.1.11	GPIO configurations for device peripherals	58
7.2	GPIO registers	59
7.2.1	Port configuration register low (GPIOx_CRL) (x=A..E)	59
7.2.2	Port configuration register high (GPIOx_CRH) (x=A..E)	60
7.2.3	Port input data register (GPIOx_IDR) (x=A..E)	61
7.2.4	Port output data register (GPIOx_ODR) (x=A..E)	61
7.2.5	Port bit set/reset register (GPIOx_BSRR) (x=A..E)	61
7.2.6	Port bit reset register (GPIOx_BRR) (x=A..E)	62
7.2.7	Port configuration lock register (GPIOx_LCKR) (x=A..E)	62
7.3	Alternate function I/O and debug configuration (AFIO)	62
7.3.1	Using OSC32_IN/OSC32_OUT pins as GPIO ports PC14/PC15	63
7.3.2	Using OSC_IN/OSC_OUT pins as GPIO ports PD0/PD1	63
7.3.3	CAN alternate function remapping	63
7.3.4	JTAG/SWD alternate function remapping	63
7.3.5	Timer alternate function remapping	64
7.3.6	USART alternate function remapping	65

7.3.7	I ² C1 alternate function remapping.....	65
7.3.8	SPI1 alternate function remapping.....	66
7.4	AFIO registers	66
7.4.1	Event control register (AFIO_EVCR)	66
7.4.2	AF remap and debug I/O configuration register (AFIO_MAPR)	67
7.4.3	External interrupt configuration register 1 (AFIO_EXTICR1).....	68
7.4.4	External interrupt configuration register 2 (AFIO_EXTICR2).....	69
7.4.5	External interrupt configuration register 3 (AFIO_EXTICR3).....	69
7.4.6	External interrupt configuration register 4 (AFIO_EXTICR4).....	69
7.5	GPIO and AFIO register maps.....	71
8	Interrupts and events.....	74
8.1	Nested vectored interrupt controller (NVIC).....	74
8.1.1	SysTick calibration value register	74
8.1.2	Interrupt and exception vectors	74
8.2	External interrupt/event controller (EXTI)	75
8.2.1	Main features.....	75
8.2.2	Block diagram	76
8.2.3	Wakeup event management.....	76
8.2.4	Functional description.....	77
8.2.5	External interrupt/event line mapping.....	77
8.3	EXTI registers.....	78
8.3.1	Interrupt mask register (EXTI_IMR)	79
8.3.2	Event mask register (EXTI_EMR).....	79
8.3.3	Rising trigger selection register (EXTI_RTSTR).....	79
8.3.4	Falling trigger selection register (EXTI_FTSTR)	79
8.3.5	Software interrupt event register (EXTI_SWIER).....	80
8.3.6	Pending register (EXTI_PR)	80
8.3.7	EXTI register map.....	81
9	Direct memory access controller (DMA)	83
9.1	DMA introduction	83
9.2	DMA main features	83
9.3	DMA functional description	84
9.3.1	DMA transactions.....	84
9.3.2	Arbiter.....	85
9.3.3	DMA channels	85
9.3.4	Programmable data width, data alignment and endians	86
9.3.5	Error management.....	87
9.3.6	Interrupts	88
9.3.7	DMA request mapping.....	88
9.4	DMA registers	90
9.4.1	DMA interrupt status register (DMA_ISR)	90
9.4.2	DMA interrupt flag clear register (DMA_IFCR).....	90
9.4.3	DMA channel x configuration register (DMA_CCRx)(x = 1...7).....	91
9.4.4	DMA channel x number of data register (DMA_CNDTRx)(x = 1...7).....	92
9.4.5	DMA channel x peripheral address register (DMA_CPARx)(x = 1...7).....	92
9.4.6	DMA channel x memory address register (DMA_CMARx)(x = 1...7)	93
9.4.7	DMA register map	93
10	Analog-to-digital converter (ADC)	100
10.1	ADC introduction	100
10.2	ADC main features	100
10.3	ADC functional description	100
10.3.1	ADC on-off control	102
10.3.2	ADC clock	102
10.3.3	Channel selection	102
10.3.4	Single conversion mode	102

10.3.5	Continuous conversion mode.....	103
10.3.6	Timing diagram.....	103
10.3.7	Analog watchdog	104
10.3.8	Scan mode	105
10.3.9	Injected channel management	105
10.3.10	Discontinuous mode	106
10.4	Calibration	107
10.5	Data alignment.....	107
10.6	Channel-by-channel programmable sample time	107
10.7	Conversion on external trigger.....	108
10.8	DMA request	109
10.9	Dual ADC mode	109
10.9.1	Injected simultaneous mode	110
10.9.2	Regular simultaneous mode	111
10.9.3	Fast interleaved mode	111
10.9.4	Slow interleaved mode.....	112
10.9.5	Alternate trigger mode	113
10.9.6	Independent mode	113
10.9.7	Combined regular/injected simultaneous mode	114
10.9.8	Combined regular simultaneous + alternate trigger mode	114
10.9.9	Combined injected simultaneous + interleaved	115
10.10	Temperature sensor.....	115
10.11	ADC interrupts.....	116
10.12	ADC registers	117
10.12.1	ADC status register (ADC_SR).....	117
10.12.2	ADC control register 1 (ADC_CR1)	117
10.12.3	ADC control register 2 (ADC_CR2).....	119
10.12.4	ADC sample time register 1 (ADC_SMPR1).....	121
10.12.5	ADC sample time register 2 (ADC_SMPR2).....	121
10.12.6	ADC injected channel data offset register x (ADC_JOFRx) (x=1..4)	122
10.12.7	ADC watchdog high threshold register (ADC_HTR)	122
10.12.8	ADC watchdog low threshold register (ADC_LTR)	122
10.12.9	ADC regular sequence register 1 (ADC_SQR1)	122
10.12.10	ADC regular sequence register 2 (ADC_SQR2)	123
10.12.11	ADC regular sequence register 3 (ADC_SQR3)	123
10.12.12	ADC injected sequence register (ADC_JSQR)	124
10.12.13	ADC injected data register x (ADC_JDRx) (x= 1..4).....	124
10.12.14	ADC regular data register (ADC_DR).....	124
10.12.15	ADC register map	126
11	Advanced-control timer (TIM1).....	130
11.1	TIM1 introduction	130
11.2	TIM1 main features	130
11.3	TIM1 functional description	131
11.3.1	Time-base unit.....	131
11.3.2	Counter modes	133
11.3.3	Repetition counter	141
11.3.4	Clock selection.....	142
11.3.5	Capture/compare channels	145
11.3.6	Input capture mode.....	147
11.3.7	PWM input mode	147
11.3.8	Forced output mode	148
11.3.9	Output compare mode	148
11.3.10	PWM mode.....	149
11.3.11	Complementary outputs and dead-time insertion	151
11.3.12	Using the break function.....	153

11.3.13	Clearing the OCxREF signal on an external event	154
11.3.14	6-step PWM generation	155
11.3.15	One-pulse mode	156
11.3.16	Encoder interface mode	158
11.3.17	Timer input XOR function	159
11.3.18	Interfacing with Hall sensors	160
11.3.19	TIM1 and external trigger synchronization	161
11.3.20	Timer synchronization	164
11.3.21	Debug mode	164
11.4	TIM1 registers	164
11.4.1	TIM1 control register 1 (TIM1_CR1)	164
11.4.2	TIM1 control register 2 (TIM1_CR2)	165
11.4.3	TIM1 slave mode control register (TIM1_SMCR)	166
11.4.4	TIM1 DMA/interrupt enable register (TIM1_DIER)	168
11.4.5	TIM1 status register (TIM1_SR)	169
11.4.6	TIM1 event generation register (TIM1_EGR)	170
11.4.7	TIM1 capture/compare mode register 1 (TIM1_CCMR1)	171
11.4.8	TIM1 capture/compare mode register 2 (TIM1_CCMR2)	173
11.4.9	TIM1 capture/compare enable register (TIM1_CCER)	174
11.4.10	TIM1 counter (TIM1_CNT)	175
11.4.11	TIM1 prescaler (TIM1_PSC)	176
11.4.12	TIM1 auto-reload register (TIM1_ARR)	176
11.4.13	TIM1 repetition counter register (TIM1_RCR)	176
11.4.14	TIM1 capture/compare register 1 (TIM1_CCR1)	176
11.4.15	TIM1 capture/compare register 2 (TIM1_CCR2)	177
11.4.16	TIM1 capture/compare register 3 (TIM1_CCR3)	177
11.4.17	TIM1 capture/compare register 4 (TIM1_CCR4)	177
11.4.18	TIM1 break and dead-time register (TIM1_BDTR)	178
11.4.19	TIM1 DMA control register (TIM1_DCR)	179
11.4.20	TIM1 DMA address for full transfer (TIM1_DMAR)	179
11.4.21	TIM1 register map	181
12	General-purpose timers (TIMx)	185
12.1	TIMx introduction	185
12.2	TIMx main features	185
12.3	TIMx functional description	186
12.3.1	Time-base unit	186
12.3.2	Counter modes	188
12.3.3	Clock selection	196
12.3.4	Capture/compare channels	199
12.3.5	Input capture mode	200
12.3.6	PWM input mode	201
12.3.7	Forced output mode	202
12.3.8	Output compare mode	202
12.3.9	PWM mode	203
12.3.10	One-pulse mode	205
12.3.11	Clearing the OCxREF signal on an external event	207
12.3.12	Encoder interface mode	207
12.3.13	Timer input XOR function	209
12.3.14	Timers and external trigger synchronization	210
12.3.15	Timer synchronization	212
12.3.16	Debug mode	216
12.4	TIMx registers	216
12.4.1	TIMx control register 1 (TIMx_CR1)	216
12.4.2	TIMx control register 2 (TIMx_CR2)	217
12.4.3	TIMx slave mode control register (TIMx_SMCR)	218

12.4.4	TIMx DMA/Interrupt enable register (TIMx_DIER)	219
12.4.5	TIMx status register (TIMx_SR)	220
12.4.6	TIMx event generation register (TIMx_EGR)	221
12.4.7	TIMx capture/compare mode register 1 (TIMx_CCMR1)	222
12.4.8	TIMx capture/compare mode register 2 (TIMx_CCMR2)	224
12.4.9	TIMx capture/compare enable register (TIMx_CCER)	224
12.4.10	TIMx counter (TIMx_CNT)	225
12.4.11	TIMx prescaler (TIMx_PSC)	225
12.4.12	TIMx auto-reload register (TIMx_ARR)	226
12.4.13	TIMx capture/compare register 1 (TIMx_CCR1)	226
12.4.14	TIMx capture/compare register 2 (TIMx_CCR2)	226
12.4.15	TIMx capture/compare register 3 (TIMx_CCR3)	227
12.4.16	TIMx capture/compare register 4 (TIMx_CCR4)	227
12.4.17	TIMx DMA control register (TIMx_DCR)	227
12.4.18	TIMx DMA address for full transfer (TIMx_DMAR)	228
12.4.19	TIMx register map	228
13	Real-time clock (RTC)	232
13.1	RTC introduction	232
13.2	RTC main features	232
13.3	RTC functional description	232
13.3.1	Overview	232
13.3.2	Resetting RTC registers	233
13.3.3	Reading RTC registers	233
13.3.4	Configuring RTC registers	234
13.3.5	RTC flag assertion	234
13.4	RTC registers	235
13.4.1	RTC control register high (RTC_CRH)	235
13.4.2	RTC control register low (RTC_CRL)	235
13.4.3	RTC prescaler load register (RTC_PRLH / RTC_PRL)	236
13.4.4	RTC prescaler divider register (RTC_DIVH / RTC_DIVL)	237
13.4.5	RTC counter register (RTC_CNTH / RTC_CNTL)	237
13.4.6	RTC alarm register high (RTC_ALRH/RTC_ALRL)	238
13.4.7	RTC register map	239
14	Independent watchdog (IWDG)	241
14.1	IWDG introduction	241
14.2	IWDG main features	241
14.3	IWDG functional description	241
14.3.1	Hardware watchdog	241
14.3.2	Register access protection	241
14.3.3	Debug mode	242
14.4	IWDG registers	242
14.4.1	Key register (IWDG_KR)	242
14.4.2	Prescaler register (IWDG_PR)	243
14.4.3	Reload register (IWDG_RLR)	243
14.4.4	Status register (IWDG_SR)	244
14.4.5	IWDG register map	245
15	Window watchdog (WWDG)	246
15.1	WWDG introduction	246
15.2	WWDG main features	246
15.3	WWDG functional description	246
15.4	How to program the watchdog timeout	247
15.5	Debug mode	248
15.6	WWDG registers	248
15.6.1	Control register (WWDG_CR)	249
15.6.2	Configuration register (WWDG_CFR)	249

15.6.3	Status register (WWDG_SR)	249
15.6.4	WWDG register map	251
16	Universal serial bus full-speed device interface(USB)	252
16.1	USB introduction	252
16.2	USB main features	252
16.3	USB functional description	253
16.3.1	Description of USB blocks	254
16.4	Programming considerations	255
16.4.1	Generic USB device programming	255
16.4.2	System and power-on reset	255
16.4.3	Double-buffered endpoints	260
16.4.4	Isochronous transfers	261
16.4.5	Suspend/Resume events	262
16.5	USB registers	263
16.5.1	Common registers	264
16.5.2	Endpoint-specific registers	268
16.5.3	Buffer descriptor table	270
16.5.4	USB register map	272
17	Controller area network (bxCAN)	276
17.1	bxCAN introduction	276
17.2	bxCAN main features	276
17.3	bxCAN general description	277
17.3.1	CAN 2.0B active core	277
17.3.2	Control, status and configuration registers	277
17.3.3	Tx mailboxes	278
17.3.4	Acceptance filters	278
17.4	bxCAN operating modes	278
17.4.1	Initialization mode	278
17.4.2	Normal mode	279
17.4.3	Sleep mode (low power)	279
17.5	Test mode	280
17.5.1	Silent mode	280
17.5.2	Loop back mode	280
17.5.3	Loop back combined with silent mode	281
17.6	Debug mode	281
17.7	bxCAN functional description	281
17.7.1	Transmission handling	281
17.7.2	Time triggered communication mode	283
17.7.3	Reception handling	283
17.7.4	Identifier filtering	285
17.7.5	Message storage	288
17.7.6	Error management	289
17.7.7	Bit timing	290
17.8	bxCAN interrupts	292
17.9	CAN registers	294
17.9.1	Register access protection	294
17.9.2	CAN control and status registers	294
17.9.3	CAN mailbox registers	301
17.9.4	CAN filter registers	304
17.9.5	bxCAN register map	307
18	Serial peripheral interface (SPI)	315
18.1	SPI introduction	315
18.2	SPI main features	315
18.2.1	SPI features	315
18.3	SPI functional description	315

18.3.1	General description	315
18.3.2	Configuring the SPI in slave mode	319
18.3.3	Configuring the SPI in master mode	319
18.3.4	Configuring the SPI for half-duplex communication.....	320
18.3.5	Data transmission and reception procedures	321
18.3.6	CRC calculation	325
18.3.7	Status flags	326
18.3.8	Disabling the SPI	327
18.3.9	SPI communication using DMA (direct memory addressing)	327
18.3.10	Error flags	328
18.3.11	SPI interrupts	329
18.4	SPI registers	329
18.4.1	SPI control register 1 (SPI_CR1).....	329
18.4.2	SPI control register 2 (SPI_CR2).....	330
18.4.3	SPI status register (SPI_SR).....	331
18.4.4	SPI data register (SPI_DR)	332
18.4.5	SPI CRC polynomial register (SPI_CRCPR)	332
18.4.6	SPI RX CRC register (SPI_RXCRCR).....	332
18.4.7	SPI TX CRC register (SPI_TXCRCR)	333
18.4.8	SPI register map.....	333
19	Inter-integrated circuit (I2C) interface.....	335
19.1	I2C introduction.....	335
19.2	I2C main features.....	335
19.3	I2C functional description	336
19.3.1	Mode selection	336
19.3.2	I2C slave mode.....	337
19.3.3	I2C master mode	339
19.3.4	Error conditions.....	342
19.3.5	SDA/SCL line control.....	343
19.3.6	SMBus	343
19.3.7	DMA requests	345
19.3.8	Packet error checking (PEC).....	346
19.4	I2C interrupts	347
19.5	I2C debug mode.....	348
19.6	I2C registers.....	348
19.6.1	I2C Control register 1 (I2C_CR1).....	348
19.6.2	I2C Control register 2 (I2C_CR2).....	350
19.6.3	I2C Own address register 1 (I2C_OAR1)	351
19.6.4	I2C Own address register 2 (I2C_OAR2)	351
19.6.5	I2C Data register (I2C_DR)	351
19.6.6	I2C Status register 1 (I2C_SR1).....	352
19.6.7	I2C Status register 2 (I2C_SR2).....	354
19.6.8	I2C Clock control register (I2C_CCR)	354
19.6.9	I2C TRISE register (I2C_TRISE).....	355
19.6.10	I2C register map	356
20	Universal synchronous asynchronous receiver transmitter (USART)	358
20.1	USART introduction.....	358
20.2	USART main features.....	358
20.3	USART functional description	359
20.3.1	USART character description	360
20.3.2	Transmitter	361
20.3.3	Receiver	363
20.3.4	Fractional baud rate generation.....	366
20.3.5	USART receiver's tolerance to clock deviation.....	367
20.3.6	Multiprocessor communication.....	368

20.3.7	Parity control.....	369
20.3.8	LIN (local interconnection network) mode.....	369
20.3.9	USART synchronous mode.....	372
20.3.10	Single-wire half-duplex communication	374
20.3.11	Smartcard.....	375
20.3.12	IrDA SIR ENDEC block	376
20.3.13	Continuous communication using DMA	378
20.3.14	Hardware flow control.....	380
20.4	USART interrupts	381
20.5	USART mode configuration.....	382
20.6	USART registers.....	382
20.6.1	Status register (USART_SR).....	383
20.6.2	Data register (USART_DR)	384
20.6.3	Baud rate register (USART_BRR).....	385
20.6.4	Control register 1 (USART_CR1).....	385
20.6.5	Control register 2 (USART_CR2)	386
20.6.6	Control register 3 (USART_CR3)	388
20.6.7	Guard time and prescaler register (USART_GTPR).....	389
20.6.8	USART register map	390
21	Device electronic signature	392
21.1	Memory size registers.....	392
21.1.1	Flash size register	392
21.2	Unique device ID register (96 bits).....	392
22	Debug support (DBG)	394
22.1	Overview.....	394
22.2	Reference Arm [®] documentation	395
22.3	SWJ debug port (serial wire and JTAG).....	395
22.3.1	Mechanism to select the JTAG-DP or the SW-DP	396
22.4	Pinout and debug port pins	396
22.4.1	SWJ debug port pins	396
22.4.2	Flexible SWJ-DP pin assignment.....	396
22.4.3	Internal pull-up and pull-down on JTAG pins.....	397
22.4.4	Using serial wire and releasing the unused debug pins as GPIOs.....	398
22.5	CKS32F103xx JTAG TAP connection.....	398
22.6	ID codes and locking mechanism	399
22.6.1	MCU device ID code	399
22.6.2	Boundary scan TAP.....	400
22.6.3	Cortex [®] -M3 TAP.....	400
22.6.4	Cortex-M3 JEDEC-106 ID code.....	400
22.7	JTAG debug port	400
22.8	SW debug port	401
22.8.1	SW protocol introduction.....	401
22.8.2	SW protocol sequence.....	401
22.8.3	SW-DP state machine (reset, idle states, ID code)	402
22.8.4	DP and AP read/write accesses	402
22.8.5	SW-DP registers.....	403
22.8.6	SW-AP registers.....	403
22.9	AHB-AP (AHB access port) - valid for both JTAG-DP and SW-DP.....	403
22.10	Core debug.....	404
22.11	Capability of the debugger host to connect under system reset.....	404
22.12	FPB (Flash patch breakpoint)	405
22.13	DWT (data watchpoint trigger).....	405
22.14	ITM (instrumentation trace macrocell).....	405
22.14.1	General description.....	405

22.14.2	Time stamp packets, synchronization and overflow packets.....	406
22.15	ETM (Embedded trace macrocell).....	407
22.15.1	ETM general description	407
22.15.2	ETM signal protocol and packet types.....	407
22.15.3	Main ETM registers.....	407
22.15.4	ETM configuration example.....	408
22.16	MCU debug component (DBGMCU).....	408
22.16.1	Debug support for low-power modes	408
22.16.2	Debug support for timers, watchdog, bxCAN and I2C	408
22.16.3	Debug MCU configuration register	409
22.17	TPIU (trace port interface unit)	410
22.17.1	Introduction	410
22.17.2	TRACE pin assignment	411
22.17.3	TPUI formatter	412
22.17.4	TPUI frame synchronization packets.....	413
22.17.5	Transmission of the synchronization frame packet	413
22.17.6	Synchronous mode	413
22.17.7	Asynchronous mode	413
22.17.8	TRACECLKIN connection inside the CKS32F103xx	414
22.17.9	TPIU registers.....	414
22.17.10	Example of configuration	415
22.18	DBG register map	415

CKS copyright

1. Documentation conventions

1.1 List of abbreviations for registers

The following abbreviations are used in register descriptions:

read / write (rw)	Software can read and write to these bits.
read-only (r)	Software can only read these bits.
write-only (w)	Software can only write to this bit. Reading the bit returns the reset value.
read/clear (rc_w1)	Software can read as well as clear this bit by writing 1. Writing '0' has no effect on the bit value.
read / clear (rc_w0)	Software can read as well as clear this bit by writing 0. Writing '1' has no effect on the bit value.
read / clear by read (rc_r)	Software can read this bit. Reading this bit automatically clears it to '0'. Writing '0' has no effect on the bit value.
read / set (rs)	Software can read as well as set this bit. Writing '0' has no effect on the bit value.
read-only write trigger (rt_w)	Software can read this bit. Writing '0' or '1' triggers an event but has no effect on the bit value.
toggle (t)	Software can only toggle this bit by writing '1'. Writing '0' has no effect.
Reserved(Res.)	Reserved bit, must be kept at reset value.

1.2 Peripheral availability

For peripheral availability and number across all CKS32F103xx sales types, refer to the CKS32F103xx datasheet.

2. Memory and bus architecture

2.1 System architecture

The main system consists of:

- Three masters:
 - Cortex[®]-M3 core DCode bus (D-bus) and System bus (S-bus)
 - general-purpose DMA
- Three slaves:
 - Internal SRAM
 - Internal Flash memory
 - AHB to APBx (APB1 or APB2), which connect all the APB peripherals

These are interconnected using a multilayer AHB bus architecture as shown in Figure 1

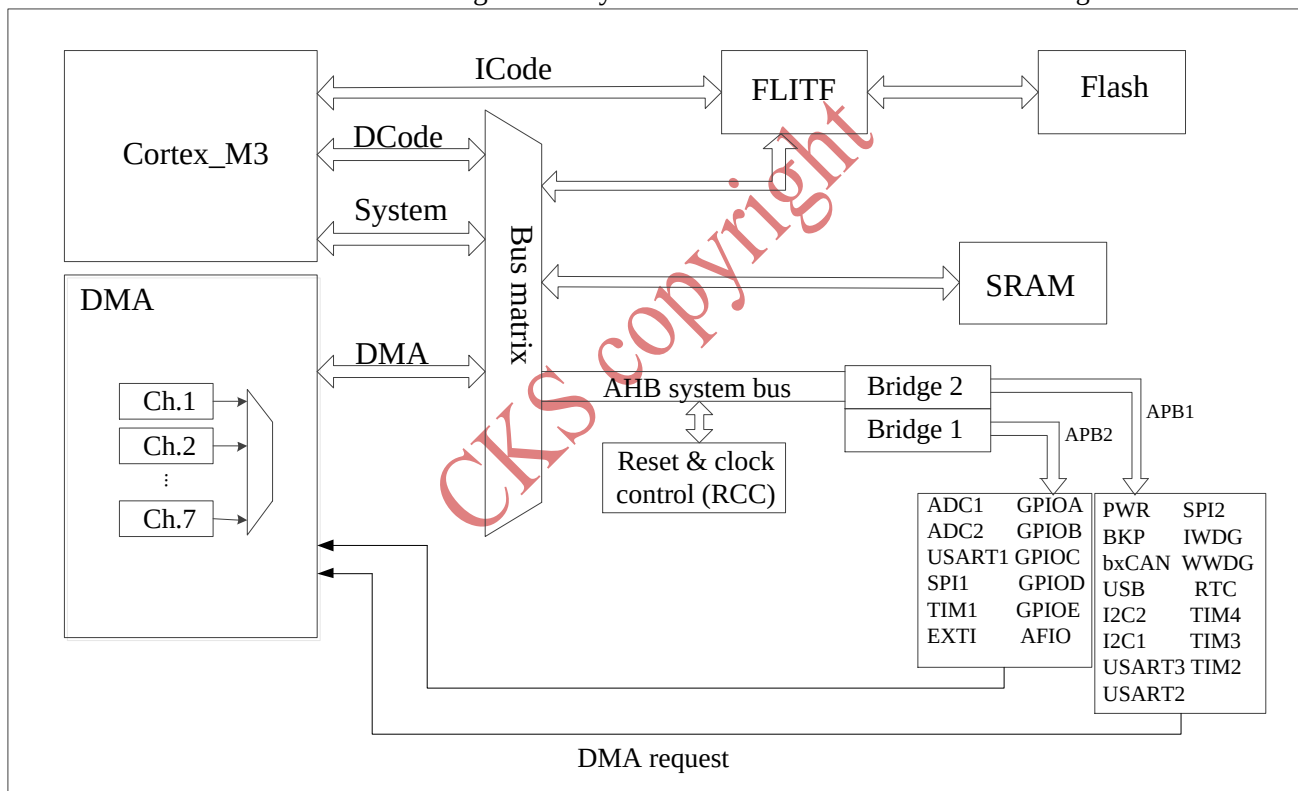


Figure 1 System architecture

ICode bus

This bus connects the Instruction bus of the Cortex[®]-M3 core to the Flash memory instruction interface. Prefetching is performed on this bus.

DCode bus

This bus connects the DCode bus (literal load and debug access) of the Cortex[®]-M3 core to the Flash memory Data interface.

System bus

This bus connects the system bus of the Cortex[®]-M3 core (peripherals bus) to a BusMatrix which manages the arbitration between the core and the DMA.

DMA bus

This bus connects the AHB master interface of the DMA to the BusMatrix which manages the access of CPU DCode and DMA to SRAM, Flash memory and peripherals.

BusMatrix

The BusMatrix manages the access arbitration between the core system bus and the DMA master bus. The arbitration uses a Round Robin algorithm. In connectivity line devices, the BusMatrix is composed of three masters (CPU DCode, System bus, DMA1 bus) and three slaves (FLITF, SRAM and AHB2APB bridges).

AHB peripherals are connected on system bus through a BusMatrix to allow DMA access.

AHB/APB bridges (APB)

The two AHB/APB bridges provide full synchronous connections between the AHB and the 2 APB buses. APB1 is limited to 36 MHz, APB2 operates at full speed (up to 72 MHz depending on the device).

Refer to Table 1 for the address mapping of the peripherals connected to each bridge.

After each device reset, all peripheral clocks are disabled (except for the SRAM and FLITF). Before using a peripheral you have to enable its clock in the RCC_AHBENR, RCC_APB2ENR or RCC_APB1ENR register.

Note: When a 16- or 8-bit access is performed on an APB register, the access is transformed into a 32-bit access: the bridge duplicates the 16- or 8-bit data to feed the 32-bit vector

2.2 Memory organization

Program memory, data memory, registers and I/O ports are organized within the same linear 4-Gbyte address space.

The bytes are coded in memory in Little Endian format. The lowest numbered byte in a word is considered the word's least significant byte and the highest numbered byte the most significant.

For the detailed mapping of peripheral registers refer to the related sections.

The addressable memory space is divided into 8 main blocks, each of 512 MB.

All the memory areas that are not allocated to on-chip memories and peripherals are considered "Reserved"). Refer to the Memory map figure in the corresponding product datasheet.

2.3 Memory map

See the datasheet corresponding to your device for a comprehensive diagram of the memory map. Table 1 gives the boundary addresses of the peripherals available in all CKS32F103xx devices.

Table 1 Register boundary addresses

Boundary address	Peripheral	Bus	Register map
0x4002 3000 – 0x4002 33FF	CRC	AHB	Section 3.4.4
0x4002 2000 – 0x4002 23FF	Flash memory interface		
0x4002 1400 – 0x4002 1FFF	Reserved		
0x4002 1000 – 0x4002 13FF	Reset and clock control (RCC)		Section 6.3.11
0x4002 0400 – 0x4002 0FFF	Reserved		
0x4002 0000 – 0x4002 03FF	DMA		Section 9.4.7
0x4001 3C00 – 0x4000 FFFF	Reserved	APB2	
0x4001 3800 – 0x4001 3BFF	USART1		Section 20.6.8
0x4001 3400 - 0x4001 37FF	Reserved		
0x4001 3000 – 0x4001 33FF	SPI1		Section 18.4
0x4001 2C00 – 0x4001 2FFF	TIM1 timer		Section 11.4.21
0x4001 2800 – 0x4001 2BFF	ADC2		Section 10.12.15

0x4001 2400 – 0x4001 27FF	ADC1		Section 10.12.15
0x4001 1C00 – 0x4001 23FF	Reserved		
0x4001 1800 – 0x4001 1BFF	GPIO Port E		Section 7.5
0x4001 1400 – 0x4001 17FF	GPIO Port D		Section 7.5
0x4001 1000 – 0x4001 13FF	GPIO Port C		Section 7.5
0x4001 0C00 – 0x4001 0FFF	GPIO Port B		Section 7.5
0x4001 0800 – 0x4001 0BFF	GPIO Port A		Section 7.5
0x4001 0400 – 0x4001 07FF	EXTI		Section 8.3.7
0x4001 0000 – 0x4001 03FF	AFIO		Section 7.5
0x4000 7400 – 0x4000 FFFF	Reserved		
0x4000 7000 – 0x4000 73FF	Power control (PWR)		Section 4.4.3
0x4000 6C00 – 0x4000 6FFF	Backup registers (BKP)		Section 5.4.5
0x4000 6800 – 0x4000 6BFF	Reserved		
0x4000 6400 – 0x4000 67FF	bxCAN1		Section 17.9.5
0x4000 6000 – 0x4000 63FF	Shared USB/CAN SRAM 512 bytes		
0x4000 5C00 – 0x4000 5FFF	USB device FS registers		Section 16.5.4
0x4000 5800 – 0x4000 5BFF	I2C2		Section 19.6.10
0x4000 5400 – 0x4000 57FF	I2C1		Section 19.6.10
0x4000 4C00 – 0x4000 53FF	Reserved	APB1	
0x4000 4800 – 0x4000 4BFF	USART3		Section 20.6.8
0x4000 4400 – 0x4000 47FF	USART2		Section 20.6.8
0x4000 3C00 – 0x4000 3FFF	Reserved		
0x4000 3800 – 0x4000 3BFF	SPI2		Section 18.4
0x4000 3400 – 0x4000 37FF	Reserved		
0x4000 3000 – 0x4000 33FF	Independent watchdog (IWDG)		Section 14.4.5
0x4000 2C00 – 0x4000 2FFF	Window watchdog (WWDG)		Section 15.6.4
0x4000 2800 – 0x4000 2BFF	RTC		Section 13.4.7
0x4000 0C00 – 0x4000 27FF	Reserved		
0x4000 0800 – 0x4000 0BFF	TIM4 timer		Section 12.4.19
0x4000 0400 – 0x4000 07FF	TIM3 timer		Section 12.4.19
0x4000 0000 – 0x4000 03FF	TIM2 timer		Section 12.4.19

2.3.1 Embedded SRAM

The CKS32F103xx features up to 96 Kbytes of static SRAM. It can be accessed as bytes, half-words (16 bits) or full words (32 bits). The SRAM start address is 0x2000 0000.

2.3.2 Bit banding

The Cortex®-M3 memory map includes two bit-band regions. These regions map each word in an alias region of memory to a bit in a bit-band region of memory. Writing to a word in the alias region has the same effect as a read-modify-write operation on the targeted bit in the bit-band region.

In the CKS32F103xx both peripheral registers and SRAM are mapped in a bit-band region. This allows single bit-band write and read operations to be performed. The operations are only available for Cortex®-M3 accesses, not from other bus masters (e.g. DMA).

A mapping formula shows how to reference each word in the alias region to a corresponding bit in the bit-band region. The mapping formula is:

$$\text{bit_word_addr} = \text{bit_band_base} + (\text{byte_offset} \times 32) + (\text{bit_number} \times 4)$$

where:

bit_word_addr is the address of the word in the alias memory region that maps to the targeted bit.

bit_band_base is the starting address of the alias region

byte_offsets is the number of the byte in the bit-band region that contains the targeted bit

bit_number is the bit position (0-7) of the targeted bit.

Example:

The following example shows how to map bit 2 of the byte located at SRAM address 0x20000300 in the alias region:

$$0x22006008 = 0x22000000 + (0x300 \times 32) + (2 \times 4).$$

Writing to address 0x22006008 has the same effect as a read-modify-write operation on bit 2 of the byte at SRAM address 0x20000300.

Reading address 0x22006008 returns the value (0x01 or 0x00) of bit 2 of the byte at SRAM address 0x20000300 (0x01: bit set; 0x00: bit reset).

For more information on Bit-Banding refer to the *Cortex®-M3 Technical Reference Manual*.

2.3.3 Embedded Flash memory

The high-performance Flash memory module has the following key features:

- Up to 128K Memory organization: the Flash memory is organized as a main block and an information block:
 - Main memory block of size:
 - up to 16 Kb × 64 bits divided into 128 pages of 1 Kbyte each for medium-density devices (Table 2).
 - Information block of size : 256×64 bits (Table 2).

The Flash memory interface (FLITF) features:

- Read interface with prefetch buffer (2x64-bit words)
- Option byte Loader
- Flash Program / Erase operation
- Read / Write protection

Table 2 Flash module organization

Block	Name	Base addresses	Size (bytes)
Main memory	Page 0	0x0800 0000 – 0x0800 03FF	1K
	Page 1	0x0800 0400 – 0x0800 07FF	1K
	Page 2	0x0800 0800 – 0x0800 0BFF	1K
	Page 3	0x0800 0C00 – 0x0800 0FFF	1K
	Page 4	0x0800 1000 – 0x0800 13FF	1K
			
	Page 127	0x0801 FC00 – 0x0801 FFFF	1K
Information block	System memory	0x1FFF F000 – 0x1FFF F7FF	2K
	Option Bytes	0x1FFF F800 – 0x1FFF F80F	16
Flash memory interface registers	FLASH_ACR	0x4002 2000 – 0x4002 2003	4
	FLASH_KEYR	0x4002 2004 – 0x4002 2007	4
	FLASH_OPTKEYR	0x4002 2008 – 0x4002 200B	4
	FLASH_SR	0x4002 200C – 0x4002 200F	4
	FLASH_CR	0x4002 2010 – 0x4002 2013	4
	FLASH_AR	0x4002 2014 – 0x4002 2017	4
	Reserved	0x4002 2018 – 0x4002 201B	4
	FLASH_OBR	0x4002 201C – 0x4002 201F	4
	FLASH_WRPR	0x4002 2020 – 0x4002 2023	4

Reading the Flash memory

Flash memory instructions and data access are performed through the AHB bus. The prefetch block is used for instruction fetches through the ICode bus. Arbitration is performed in the Flash memory interface, and priority is given to data access on the DCode bus.

Read accesses can be performed with the following configuration options:

- Latency: number of wait states for a read operation programmed on-the-fly
- Prefetch buffer (2 x 64-bit blocks): it is enabled after reset; a whole block can be replaced with a single read from the Flash memory as the size of the block matches the bandwidth of the Flash memory. Thanks to the prefetch buffer, faster CPU execution is possible as the CPU fetches one word at a time with the next word readily available in the prefetch buffer
- Half cycle: for power optimization

Note:

1. These options should be used in accordance with the Flash memory access time. The wait states represent the ratio of the SYSCCLK (system clock) period to the Flash memory access time:

zero wait state, if $0 < \text{SYSCCLK} \leq 24 \text{ MHz}$

one wait state, if $24 \text{ MHz} < \text{SYSCCLK} \leq 48 \text{ MHz}$

two wait states, if $48 \text{ MHz} < \text{SYSCCLK} \leq 72 \text{ MHz}$

2. Half cycle configuration is not available in combination with a prescaler on the AHB. The system clock (SYSCCLK) should be equal to the HCLK clock. This feature can therefore be used only with a low-frequency clock of 8 MHz or less. It can be generated from the HSI or the HSE but not from the PLL.

3. The prefetch buffer must be kept on when using a prescaler different from 1 on the AHB clock.

4. The prefetch buffer must be switched on/off only when SYSCCLK is lower than 24 MHz and no prescaler is applied on the AHB clock (SYSCCLK must be equal to HCLK). The prefetch buffer is usually switched on/off during the initialization routine, while the microcontroller is running on the internal 8 MHz RC (HSI) oscillator.

5. Using DMA: DMA accesses Flash memory on the DCode bus and has priority over ICode instructions. The DMA provides one free cycle after each transfer. Some instructions can be performed together with DMA transfer.

Programming and erasing the Flash memory

The Flash memory can be programmed 16 bits (half words) at a time.

The Flash memory erase operation can be performed at page level or on the whole Flash area (mass-erase). The mass-erase does not affect the information blocks.

To ensure that there is no over-programming, the Flash Programming and Erase Controller blocks are clocked by a fixed clock.

The End of write operation (programming or erasing) can trigger an interrupt. This interrupt can be used to exit from WFI mode, only if the FLITF clock is enabled. Otherwise, the interrupt is served only after an exit from WFI.

2.4 Boot configuration

In the CKS32F103xx, 3 different boot modes can be selected through BOOT[1:0] pins as shown in Table 3.

Table 3 Boot modes

Boot mode selection pins		Boot mode	Aliasing
BOOT1	BOOT0		
X	0	Main Flash memory	Main Flash memory is selected as boot space
0	1	System memory	System memory is selected as boot space
1	1	Embedded SRAM	Embedded SRAM is selected as boot space

The values on the BOOT pins are latched on the 4th rising edge of SYSCCLK after a reset. It is up to the user to set the BOOT1 and BOOT0 pins after Reset to select the required boot mode.

The BOOT pins are also re-sampled when exiting from Standby mode. Consequently they must be kept in the required Boot mode configuration in Standby mode. After this startup delay has elapsed, the CPU fetches the top-of-stack value from address 0x0000 0000, then starts code execution from the boot memory starting from 0x0000 0004.

Due to its fixed memory map, the code area starts from address 0x0000 0000 (accessed through the ICode/DCode buses) while the data area (SRAM) starts from address 0x2000 0000 (accessed through the system bus). The Cortex[®]-M3 CPU always fetches the reset vector on the ICode bus,

which implies to have the boot space available only in the code area (typically, Flash memory). CKS32F103xx microcontrollers implement a special mechanism to be able to boot also from SRAM and not only from main Flash memory and System memory.

Depending on the selected boot mode, main Flash memory, system memory or SRAM is accessible as follows:

- Boot from main Flash memory: the main Flash memory is aliased in the boot memory space (0x0000 0000), but still accessible from its original memory space (0x800 0000). In other words, the Flash memory contents can be accessed starting from address 0x0000 0000 or 0x800 0000.
- Boot from system memory: the system memory is aliased in the boot memory space (0x0000 0000), but still accessible from its original memory space (0x1FFF B000 in connectivity line devices, 0x1FFF F000 in other devices).
- Boot from the embedded SRAM: SRAM is accessible only at address 0x2000 0000.

Note: When booting from SRAM, in the application initialization code, you have to relocate the vector table in SRAM using the NVIC exception table and offset register.

Embedded boot loader

The embedded boot loader is located in the System memory, programmed during production. It is used to reprogram the Flash memory with one of the available serial interfaces:

- In CKS32F10xx devices the boot loader is activated through the USART1 interface.

CKS copyright

3. CRC calculation unit

3.1 CRC introduction

The CRC (cyclic redundancy check) calculation unit is used to get a CRC code from a 32-bit data word and a fixed generator polynomial.

Among other applications, CRC-based techniques are used to verify data transmission or storage integrity. In the scope of the EN/IEC 60335-1 standard, they offer a means of verifying the Flash memory integrity. The CRC calculation unit helps compute a signature of the software during runtime, to be compared with a reference signature generated at link-time and stored at a given memory location.

3.2 CRC main features

- Uses CRC-32 (Ethernet) polynomial: 0x4C11DB7

$$- X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^4 + X^2 + X + 1$$
- Single input/output 32-bit data register
- CRC computation done in 4 AHB clock cycles (HCLK)
- General-purpose 8-bit register (can be used for temporary storage)

The block diagram is shown in Figure 2

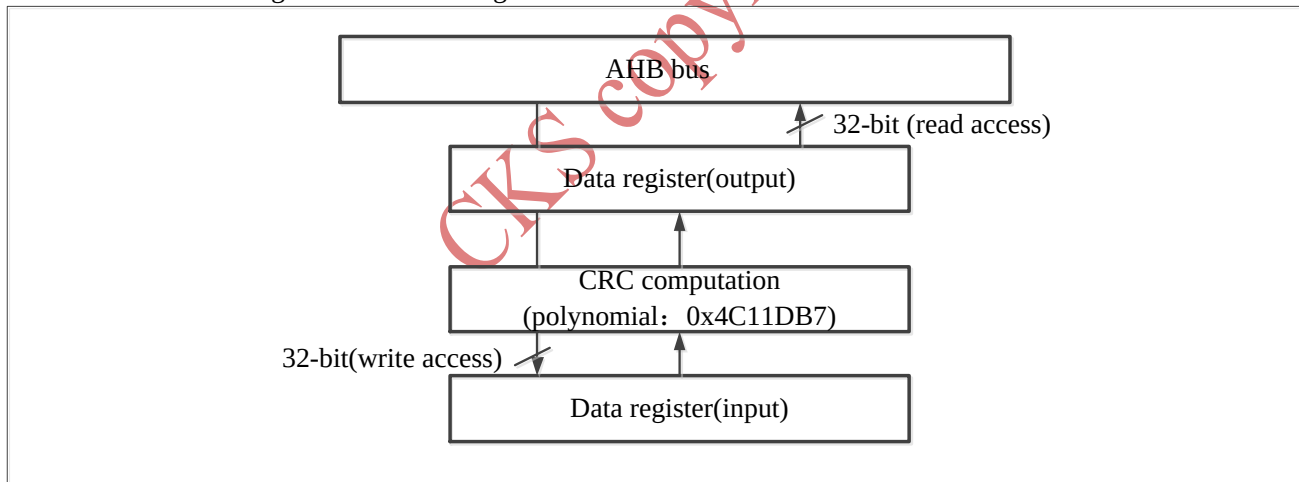


Figure 2 CRC calculation unit block diagram

3.3 CRC functional description

The CRC calculation unit mainly consists of a single 32-bit data register, which:

- is used as an input register to enter new data in the CRC calculator (when writing into the register)
- holds the result of the previous CRC calculation (when reading the register)

Each write operation into the data register creates a combination of the previous CRC value and the new one (CRC computation is done on the whole 32-bit data word, and not byte per byte).

The write operation is stalled until the end of the CRC computation, thus allowing back-to-back write accesses or consecutive write and read accesses.

The CRC calculator can be reset to 0xFFFF FFFF with the RESET control bit in the CRC_CR register. This operation does not affect the contents of the CRC_IDR register.

3.4 CRC registers

The CRC calculation unit contains two data registers and a control register.

3.4.1 Data register (CRC_DR)

Address offset : 0x00

Reset value: 0xFFFF FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31: 0		Data register bits Used as an input register when writing new data into the CRC calculator. Holds the previous CRC calculation result when it is read.													

3.4.2 Independent data register (CRC_IDR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								IDR[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw
Bits 31: 8		Reserved, must be kept at reset value.													
Bits 7:0		General-purpose 8-bit data register bits Can be used as a temporary storage location for one byte. This register is not affected by CRC resets generated by the RESET bit in the CRC_CR register.													

3.4.3 Control register (CRC_CR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															RESET
															w
位 31: 1		Reserved, must be kept at reset value.													
位 0		RESET bit Resets the CRC calculation unit and sets the data register to 0xFFFF FFFF. This bit can only be set, it is automatically cleared by hardware.													

3.4.4 CRC register map

The following table provides the CRC register map and reset values.

Table 4 CRC calculation unit register map and reset values

Offset	Register	31~24	23~16	15~8	7	6	5	4	3	2	1	0
0x00	CRC_DR	Data register 0xFFFF FFFF										
	Reset value											
0x04	CRC_IDR	Reserved			Independent data register 0x00							
	Reset value											
0x08	CRC_CR	Reserved										RESET 0
	Reset value											

CKS copyright

4 Power control (PWR)

4.1 Power supplies

The device requires a 2.0-to-3.6 V operating voltage supply (V_{DD}). An embedded regulator is used to supply the internal 1.5 V digital power.

The real-time clock (RTC) and backup registers can be powered from the V_{BAT} voltage when the main V_{DD} supply is powered off.

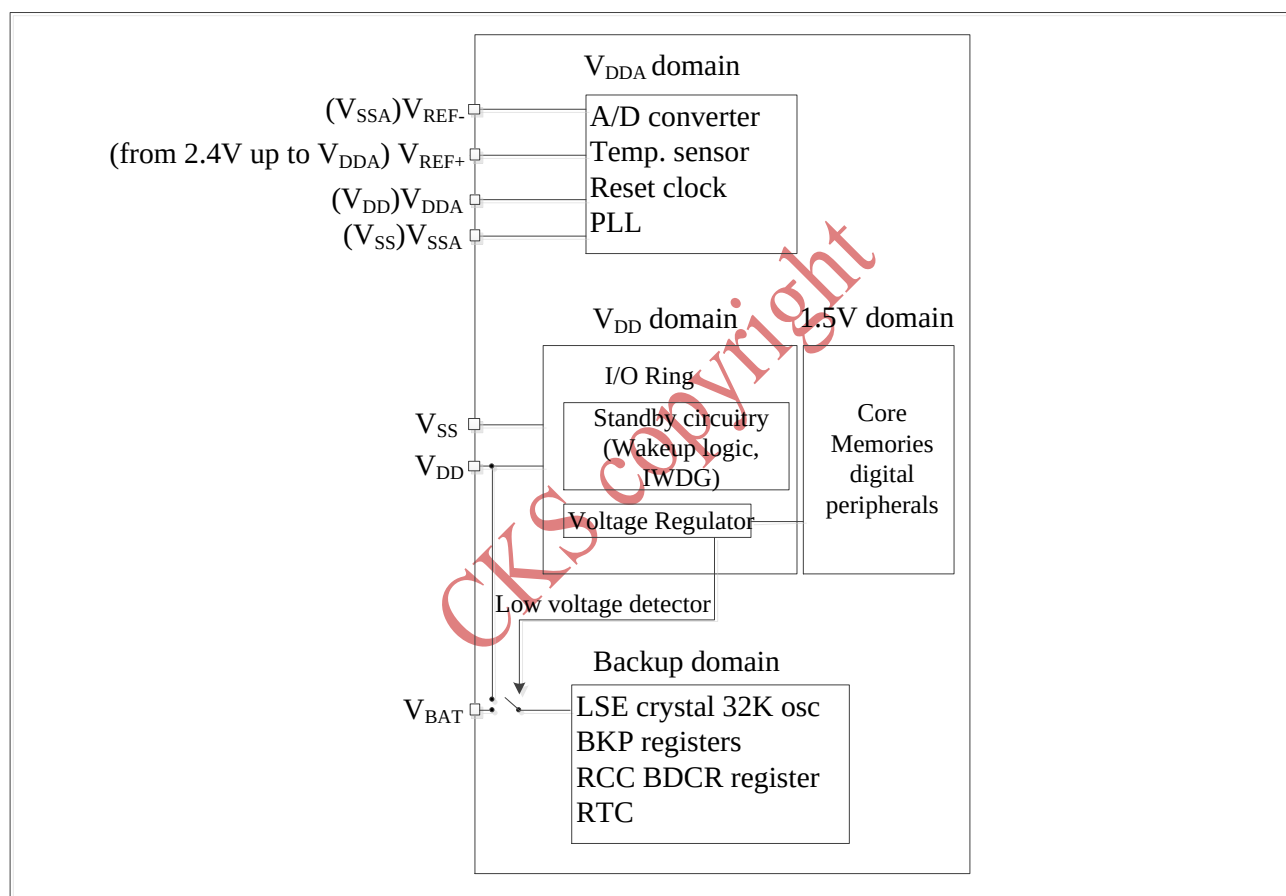


Figure 3 Power supply overview

1. V_{DDA} and V_{SSA} must be connected to V_{DD} and V_{SS} , respectively.

4.1.1 Independent A/D and D/A converter supply and reference voltage

To improve conversion accuracy, the ADC and the DAC have an independent power supply which can be separately filtered and shielded from noise on the PCB.

- The ADC and DAC voltage supply input is available on a separate V_{DDA} pin.
- An isolated supply ground connection is provided on pin V_{SSA} .

When available (according to package), V_{REF-} must be tied to V_{SSA} .

On 100-pin

To ensure a better accuracy on low-voltage inputs and outputs, the user can connect a separate external reference voltage on VREF+. VREF+ is the highest voltage, represented by the full scale value, for an analog input (ADC) or output (DAC) signal. The voltage on VREF+ can range from 2.4 V to VDDA.

On 64-pin packages and packages with less pins

The VREF+ and VREF- pins are not available, they are internally connected to the ADC voltage supply (VDDA) and ground (VSSA).

4.1.2 Battery backup domain

To retain the content of the Backup registers and supply the RTC function when VDD is turned off, VBAT pin can be connected to an optional standby voltage supplied by a battery or by another source. The VBAT pin powers the RTC unit, the LSE oscillator and the PC13 to PC15 IOs, allowing the RTC to operate even when the main digital supply (VDD) is turned off. The switch to the VBAT supply is controlled by the Power Down Reset embedded in the Reset block.

Warning: During tRSTTEMPO (temporization at VDD startup) or after a PDR is detected, the power switch between VBAT and VDD remains connected to VBAT.

During the startup phase, if VDD is established in less than tRSTTEMPO (Refer to the datasheet for the value of tRSTTEMPO) and $VDD > VBAT + 0.6\text{ V}$, a current may be injected into VBAT through an internal diode connected between VDD and the power switch (VBAT). If the power supply/battery connected to the VBAT pin cannot support this current injection, it is strongly recommended to connect an external low-drop diode between this power supply and the VBAT pin.

If no external battery is used in the application, it is recommended to connect VBAT externally to VDD with a 100 nF external ceramic decoupling capacitor.

When the backup domain is supplied by VDD (analog switch connected to VDD), the following functions are available:

- PC14 and PC15 can be used as either GPIO or LSE pins
- PC13 can be used as GPIO, TAMPER pin, RTC Calibration Clock, RTC Alarm or second output (refer to Section 5)

Note: Due to the fact that the switch only sinks a limited amount of current (3 mA), the use of GPIOs PC13 to PC15 in output mode is restricted: the speed has to be limited to 2 MHz with a maximum load of 30 pF and these IOs must not be used as a current source (e.g. to drive a LED).

When the backup domain is supplied by VBAT (analog switch connected to VBAT because VDD is not present), the following functions are available:

- PC14 and PC15 can be used as LSE pins only
- PC13 can be used as TAMPER pin, RTC Alarm or Second output (refer to Section 5.4.2))

4.1.3 Voltage regulator

The voltage regulator is always enabled after Reset. It works in three different modes depending on the application modes

- In Run mode, the regulator supplies full power to the 1.5 V domain (core, memories and digital peripherals).
- In Stop mode the regulator supplies low-power to the 1.5 V domain, preserving contents of registers and SRAM.

- In Standby Mode, the regulator is powered off. The contents of the registers and SRAM are lost except for the Standby circuitry and the Backup Domain.

4.2 Power supply supervisor

4.2.1 Power on reset (POR)/power down reset (PDR)

The device has an integrated POR/PDR circuitry that allows proper operation starting from/down to 2 V. The device remains in Reset mode when V_{DD}/V_{DDA} is below a specified threshold, $V_{POR/PDR}$, without the need for an external reset circuit. For more details concerning the power on/power down reset threshold, refer to the electrical characteristics of the datasheet.

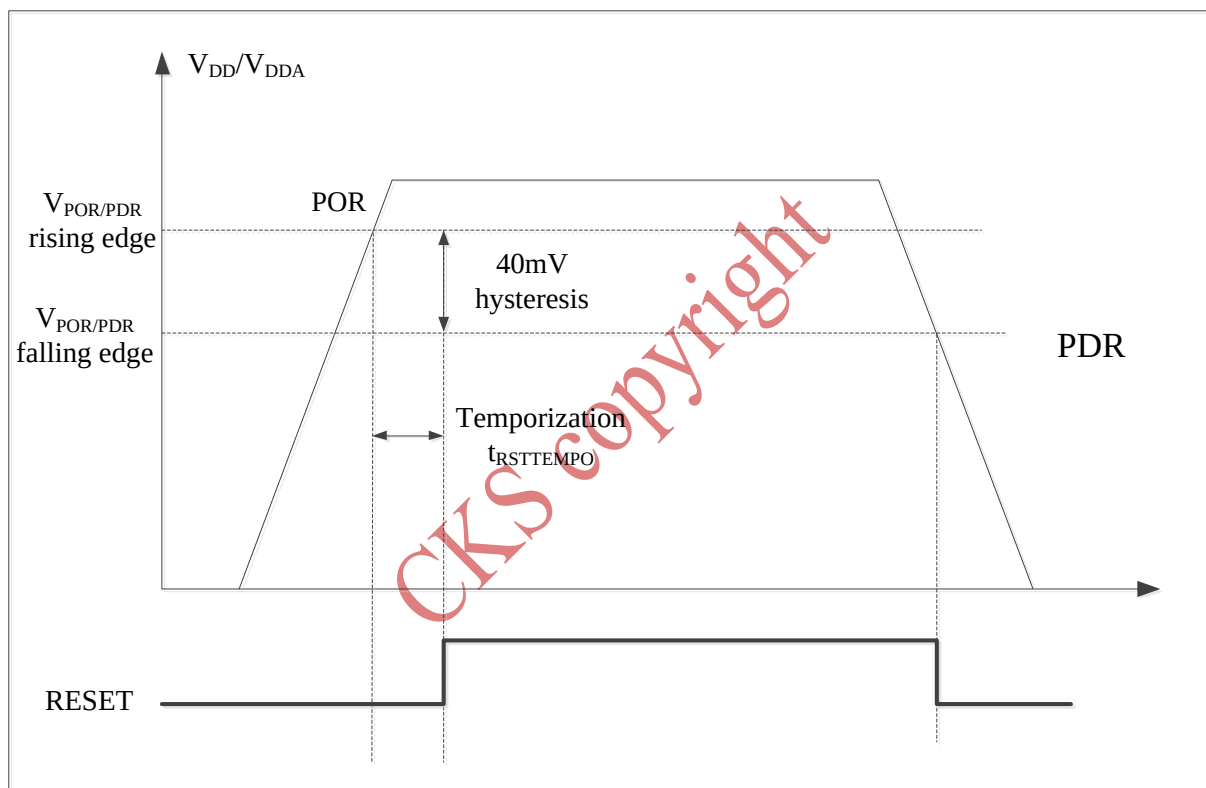


Figure 4 Power on reset/power down reset waveform

4.2.2 Programmable voltage detector (PVD)

The PVD can be used to monitor the V_{DD}/V_{DDA} power supply by comparing it to a threshold selected by the PLS[2:0] bits in the *Power control register (PWR_CR)*.

The PVD is enabled by setting the PVDE bit.

A PVDO flag is available, in the *Power control/status register (PWR_CSR)*, to indicate if V_{DD}/V_{DDA} is higher or lower than the PVD threshold. This event is internally connected to the EXTI line16 and can generate an interrupt if enabled through the EXTI registers. The PVD output interrupt can be generated when V_{DD}/V_{DDA} drops below the PVD threshold and/or when V_{DD}/V_{DDA} rises above the PVD

threshold depending on EXTI line16 rising/falling edge configuration. As an example the service routine could perform emergency shutdown tasks.

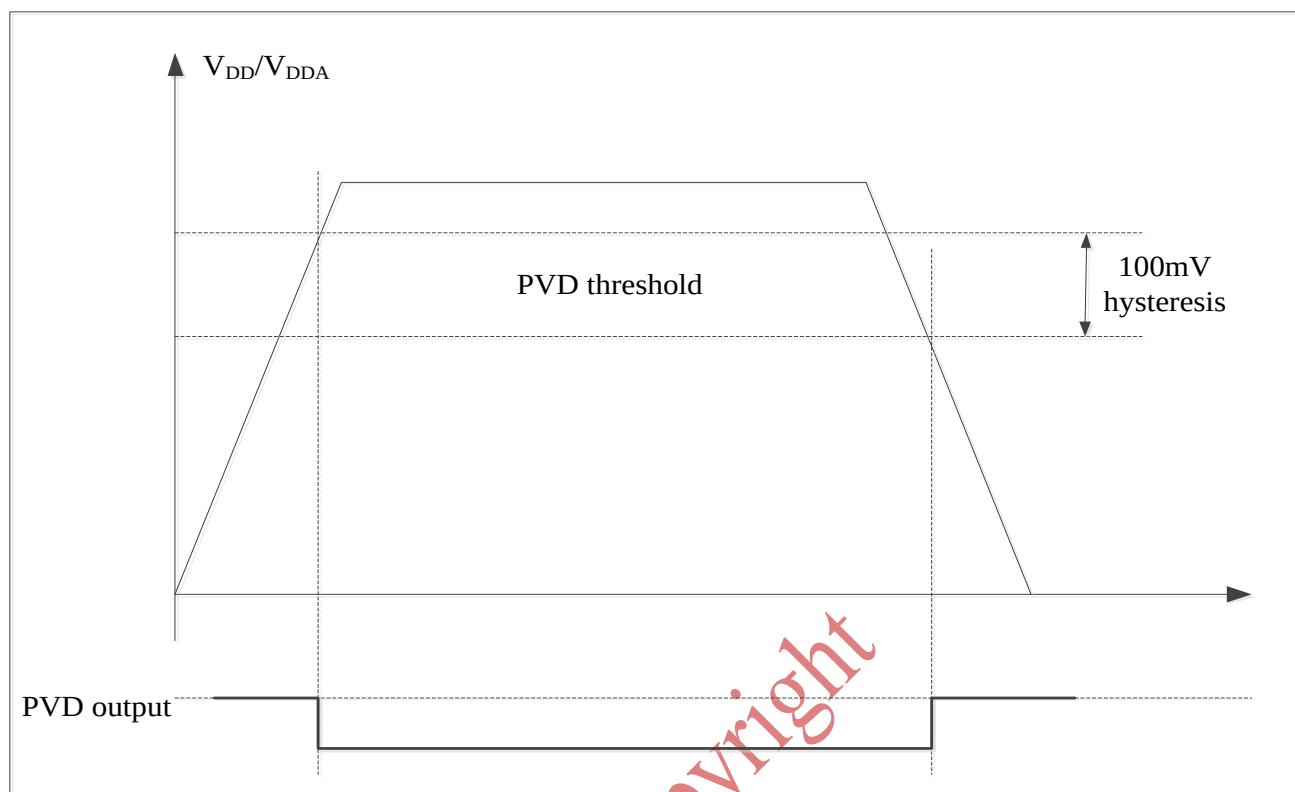


Figure 5 PVD thresholds

4.3 Low-power modes

By default, the microcontroller is in Run mode after a system or a power Reset. Several low-power modes are available to save power when the CPU does not need to be kept running, for example when waiting for an external event. It is up to the user to select the mode that gives the best compromise between low-power consumption, short startup time and available wakeup sources.

The CKS32F103xx devices feature three low-power modes:

- Sleep mode (CPU clock off, all peripherals including Cortex[®]-M3 core peripherals like NVIC, SysTick, etc. are kept running)
- Stop mode (all clocks are stopped)
- Standby mode (1.5V domain powered-off)

In addition, the power consumption in Run mode can be reduced by one of the following means:
Slowing down the system clocks.

Gating the clocks to the APB and AHB peripherals when they are unused.

Table 5 Low-power mode summary

Mode name	Entry	Wakeup	Effect on 1.5V domain clocks	Effect on V _{DD} domain clocks	Voltage regulator
Sleep (Sleep now or Sleep-on -exit)	WFI	Any interrupt	CPU clock OFF no effect on other clocks or analog clock sources	None	ON
	WFE	Wakeup event			
Stop	PDDS and LPDS bits + SLEEPDEEP bit + WFI or WFE	Any EXTI line (configured in the EXTI registers)	All 1.5V domain clocks OFF	HSI and HSE oscillators OFF	ON or in low-power mode (depends on Power)

					<i>control register (PWR_CR)</i>
Standby	PDDS bit + SLEEPDEEP bit + WFI or WFE	WKUP pin rising edge, RTC alarm, external reset in NRST pin, IWDG reset			OFF

4.3.1 Slowing down system clocks

In Run mode the speed of the system clocks (SYSCLK, HCLK, PCLK1, PCLK2) can be reduced by programming the prescaler registers. These prescalers can also be used to slow down peripherals before entering Sleep mode.

For more details refer to *Section 7.3.2*.

4.3.2 Peripheral clock gating

In Run mode, the HCLK and PCLKx for individual peripherals and memories can be stopped at any time to reduce power consumption.

To further reduce power consumption in Sleep mode the peripheral clocks can be disabled prior to executing the WFI or WFE instructions.

Peripheral clock gating is controlled by the *AHB peripheral clock enable register (RCC_AHBENR)*, *APB1 peripheral clock enable register (RCC_APB1ENR)* and *APB2 peripheral clock enable register (RCC_APB2ENR)*.

4.3.3 Sleep mode

Entering Sleep mode

The Sleep mode is entered by executing the WFI (Wait For Interrupt) or WFE (Wait for Event) instructions. Two options are available to select the Sleep mode entry mechanism, depending on the SLEEPONEXIT bit in the Cortex®-M3 System Control register:

Sleep-now: if the SLEEPONEXIT bit is cleared, the MCU enters Sleep mode as soon as WFI or WFE instruction is executed.

Sleep-on-exit: if the SLEEPONEXIT bit is set, the MCU enters Sleep mode as soon as it exits the lowest priority ISR.

In the Sleep mode, all I/O pins keep the same state as in the Run mode.

Refer to Table 6 and Table 7 for details on how to enter Sleep mode.

Exiting Sleep mode

If the WFI instruction is used to enter Sleep mode, any peripheral interrupt acknowledged by the nested vectored interrupt controller (NVIC) can wake up the device from Sleep mode.

If the WFE instruction is used to enter Sleep mode, the MCU exits Sleep mode as soon as an event occurs.

The wakeup event can be generated either by:

- enabling an interrupt in the peripheral control register but not in the NVIC, and enabling the SEVONPEND bit in the Cortex®-M3 System Control register. When the MCU resumes from WFE, the peripheral interrupt pending bit and the peripheral NVIC IRQ channel pending bit (in the NVIC interrupt clear pending register) have to be cleared.
- or configuring an external or internal EXTI line in event mode. When the CPU resumes from WFE, it is not necessary to clear the peripheral interrupt pending bit or the NVIC IRQ channel pending bit as the pending bit corresponding to the event line is not set.

This mode offers the lowest wakeup time as no time is wasted in interrupt entry/exit.

Refer to Table 6 and Table 7 for more details on how to exit Sleep mode.

Table 6 Sleep-now

Sleep-now mode	Description
Mode entry	WFI (Wait for Interrupt) or WFE (Wait for Event) while: – SLEEPDEEP = 0 and – SLEEPONEXIT = 0 Refer to the Cortex®-M3 System Control register.
Mode exit	If WFI was used for entry: Interrupt: Refer to <i>Section 9.1.2: Interrupt and exception vectors</i> If WFE was used for entry Wakeup event: Refer to <i>Section 9.2.3: Wakeup event management</i>
Wakeup latency	None

Table 7 Sleep-on-exit

Sleep-on-exit	Description
Mode entry	WFI (wait for interrupt) while: – SLEEPDEEP = 0 and – SLEEPONEXIT = 1 Refer to the Cortex®-M3 System Control register.
Mode exit	Interrupt: refer to <i>Section 9.1.2: Interrupt and exception vectors</i> .
Wakeup latency	None

4.3.4 Stop mode

The Stop mode is based on the Cortex®-M3 deepsleep mode combined with peripheral clock gating. The voltage regulator can be configured either in normal or low-power mode. In Stop mode, all clocks in the 1.5 V domain are stopped, the PLL, the HSI and the HSE RC oscillators are disabled. SRAM and register contents are preserved.

In the Stop mode, all I/O pins keep the same state as in the Run mode.

Entering Stop mode

Refer to Table 8 for details on how to enter the Stop mode.

To further reduce power consumption in Stop mode, the internal voltage regulator can be put in low-power mode. This is configured by the LPDS bit of the *Power control register (PWR_CR)*.

If Flash memory programming is ongoing, the Stop mode entry is delayed until the memory access is finished.

If an access to the APB domain is ongoing, The Stop mode entry is delayed until the APB access is finished.

In Stop mode, the following features can be selected by programming individual control bits:

- Independent watchdog (IWDG): the IWDG is started by writing to its Key register or by hardware option. Once started it cannot be stopped except by a Reset. See *Section 19.3: IWDG functional description*.
- Real-time clock (RTC): this is configured by the RTCEN bit in the *Backup domain control register (RCC_BDCR)*
- Internal RC oscillator (LSI RC): this is configured by the LSION bit in the *Control/status register (RCC_CSR)*.
- External 32.768 kHz oscillator (LSE OSC): this is configured by the LSEON bit in the *Backup domain control register (RCC_BDCR)*.

The ADC or DAC can also consume power during the Stop mode, unless they are disabled before entering it. To disable them, the ADON bit in the ADC_CR2 register and the ENx bit in the DAC_CR register must both be written to 0.

Exiting Stop mode

Refer to Table 8 for more details on how to exit Stop mode.

When exiting Stop mode by issuing an interrupt or a wakeup event, the HSI RC oscillator is selected as system clock.

When the voltage regulator operates in low-power mode, an additional startup delay is incurred when waking up from Stop mode. By keeping the internal regulator ON during Stop mode, the consumption is higher although the startup time is reduced.

Table 8 Stop mode

Stop mode	Description
Mode entry	WFI (Wait for Interrupt) or WFE (Wait for Event) while: – Set SLEEPDEEP bit in Cortex[®]-M3 System Control register – Clear PDDS bit in Power Control register (PWR_CR) Select the voltage regulator mode by configuring LPDS bit in PWR_CR Note: To enter Stop mode, all EXTI Line pending bits (in <i>Pending register (EXTI_PR)</i>), all peripheral interrupt pending bits, and RTC Alarm flag must be reset. Otherwise, the Stop mode entry procedure is ignored and program execution continues.
Mode exit	If WFI was used for entry: Any EXTI Line configured in Interrupt mode (the corresponding EXTI Interrupt vector must be enabled in the NVIC). Refer to <i>Section 8.1.2: Interrupt and exception vectors</i>. If WFE was used for entry: Any EXTI Line configured in event mode. Refer to <i>Section 8.2.3: Wakeup event management</i>
Wakeup latency	HSI RC wakeup time + regulator wakeup time from Low-power mode

4.3.5 Standby mode

The Standby mode allows to achieve the lowest power consumption. It is based on the Cortex[®]-M3 deepsleep mode, with the voltage regulator disabled. The 1.5 V domain is consequently powered off. The PLL, the HSI oscillator and the HSE oscillator are also switched off. SRAM and register contents are lost except for registers in the Backup domain and Standby circuitry (see Figure 3).

Entering Standby mode

Refer to Table 9 for more details on how to enter Standby mode.

In Standby mode, the following features can be selected by programming individual control bits:

- Independent watchdog (IWDG): the IWDG is started by writing to its Key register or by hardware option. Once started it cannot be stopped except by a reset. See *Section 14.3: IWDG functional description*.
- Real-time clock (RTC): this is configured by the RTCEN bit in the Backup domain control register (RCC_BDCR).
- Internal RC oscillator (LSI RC): this is configured by the LSION bit in the Control/status register (RCC_CSR).
- External 32.768 kHz oscillator (LSE OSC): this is configured by the LSEON bit in the Backup domain control register (RCC_BDCR).

Exiting Standby mode

The microcontroller exits the Standby mode when an external reset (NRST pin), an IWDG reset, a rising edge on the WKUP pin or the rising edge of an RTC alarm occurs (see Figure 131). All registers are reset after wakeup from Standby except for *Power control/status register (PWR_CSR)*.

After waking up from Standby mode, program execution restarts in the same way as after a Reset (boot pins sampling, vector reset is fetched, etc.). The SBF status flag in the *Power control/status register (PWR_CSR)* indicates that the MCU was in Standby mode.

Refer to Table 9 for more details on how to exit Standby mode.

Table 9 Standby mode

Standby mode	Description
Mode entry	WFI (Wait for Interrupt) or WFE (Wait for Event) while:

	– Set SLEEPDEEP in Cortex[®]-M3 System Control register – Set PDDS bit in Power Control register (PWR_CR) – Clear WUF bit in Power Control/Status register (PWR_CSR)
Mode exit	WKUP pin rising edge, RTC alarm event's rising edge, external Reset in NRST pin, IWDG Reset.
Wakeup latency	Reset phase

I/O states in Standby mode

In Standby mode, all I/O pins are high impedance except:

- Reset pad (still available)
- TAMPER pin if configured for tamper or calibration out
- WKUP pin, if enabled

Debug mode

By default, the debug connection is lost if the application puts the MCU in Stop or Standby mode while the debug features are used. This is due to the fact that the Cortex[®]-M3 core is no longer clocked. However, by setting some configuration bits in the DBGMCU_CR register, the software can be debugged even when using the low-power modes extensively. For more details, refer to *Section 22.16.1: Debug support for low-power modes*.

4.3.6 Auto-wakeup (AWU) from low-power mode

The RTC can be used to wakeup the MCU from low-power mode without depending on an external interrupt (Auto-wakeup mode). The RTC provides a programmable time base for waking up from Stop or Standby mode at regular intervals. For this purpose, two of the three alternative RTC clock sources can be selected by programming the RTCSEL[1:0] bits in the *Backup domain control register (RCC_BDCR)*:

- Low-power 32.768 kHz external crystal oscillator (LSE OSC).
This clock source provides a precise time base with very low-power consumption (less than 1 µA added consumption in typical conditions)
- Low-power internal RC Oscillator (LSI RC)
This clock source has the advantage of saving the cost of the 32.768 kHz crystal. This internal RC Oscillator is designed to add minimum power consumption.

To wakeup from Stop mode with an RTC alarm event, it is necessary to:

- Configure the EXTI Line 17 to be sensitive to rising edge
- Configure the RTC to generate the RTC alarm

To wakeup from Standby mode, there is no need to configure the EXTI Line 17.

4.4 Power control registers

The peripheral registers can be accessed by half-words (16-bit) or words (32-bit).

4.4.1 Power control register (PWR_CR)

Address offset: 0x00

Reset value: 0x0000 0000 (reset by wakeup from Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								DBP	PLS[2:0]		PVDE	CSBF	CWUF	PDDS	LPDS
								rw	rw	rw	rw	rw	rc_wl	rc_wl	rw
Bits 31: 9								Reserved, must be kept at reset value.							

Bit 8	DBP: Disable backup domain write protection. In reset state, the RTC and backup registers are protected against parasitic write access. This bit must be set to enable write access to these registers. 0: Access to RTC and Backup registers disabled 1: Access to RTC and Backup registers enabled <i>Note: If the HSE divided by 128 is used as the RTC clock, this bit must remain set to 1.</i>
Bits 7:5	PLS[2:0]: PVD level selection. These bits are written by software to select the voltage threshold detected by the Power Voltage Detector 000: 2.2V 001: 2.3V 010: 2.4V 011: 2.5V 100: 2.6V 101: 2.7V 110: 2.8V 111: 2.9V <i>Note: Refer to the electrical characteristics of the datasheet for more details.</i>
Bit 4	PVDE: Power voltage detector enable. This bit is set and cleared by software. 0: PVD disabled 1: PVD enabled
Bit 3	CSBF: Clear standby flag. This bit is always read as 0. 0: No effect 1: Clear the SBF Standby Flag (write).
Bit 2	CWUF: Clear wakeup flag This bit is always read as 0. 0: No effect 1: Clear the WUF Wakeup Flag after 2 System clock cycles. (write)
Bit 1	PDDS: Power down deepsleep. This bit is set and cleared by software. It works together with the LPDS bit. 0: Enter Stop mode when the CPU enters Deepsleep. The regulator status depends on the LPDS bit. 1: Enter Standby mode when the CPU enters Deepsleep.
Bit 0	LPDS: Low-power deepsleep This bit is set and cleared by software. It works together with the PDDS bit. 0: Voltage regulator on during Stop mode 1: Voltage regulator in low-power mode during Stop mode

4.4.2 Power control/status register (PWR_CSR)

Address offset: 0x04

Reset value: 0x0000 0000 (not reset by wakeup from Standby mode)

Additional APB cycles are needed to read this register versus a standard APB read.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16			
Reserved																		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Reserved							EWUP	Reserved					PVDO	SBF	WUF			
							rw	r									r	r
Bits 31: 9							Reserved, must be kept at reset value.											
Bit 8							EWUP: Enable WKUP pin This bit is set and cleared by software. 0: WKUP pin is used for general purpose I/O. An event on the WKUP pin does not wakeup the device from Standby mode. 1: WKUP pin is used for wakeup from Standby mode and forced in input pull down configuration (rising edge on WKUP pin wakes-up the system from Standby mode). <i>Note: This bit is reset by a system Reset.</i>											

Bits 7:3	Reserved, must be kept at reset value.
Bit 2	PVDO: PVD output This bit is set and cleared by hardware. It is valid only if PVD is enabled by the PVDE bit. 0: V_{DD}/V_{DDA} is higher than the PVD threshold selected with the PLS[2:0] bits. 1: V_{DD}/V_{DDA} is lower than the PVD threshold selected with the PLS[2:0] bits. <i>Note: The PVD is stopped by Standby mode. For this reason, this bit is equal to 0 after Standby or reset until the PVDE bit is set.</i>
Bit 1	SBF: Standby flag This bit is set by hardware and cleared only by a POR/PDR (power on reset/power down reset) or by setting the CSBF bit in the <i>Power control register (PWR_CR)</i> 0: Device has not been in Standby mode 1: Device has been in Standby mode
Bit 0	WUF: Wakeup flag This bit is set by hardware and cleared by hardware, by a system reset or by setting the CWUF bit in the <i>Power control register (PWR_CR)</i> 0: No wakeup event occurred 1: A wakeup event was received from the WKUP pin or from the RTC alarm <i>Note: An additional wakeup event is detected if the WKUP pin is enabled (by setting the EWUP bit) when the WKUP pin level is already high.</i>

4.4.3 PWR register map

The following table summarizes the PWR registers.

Table 10 PWR register map and reset values

004h		000h		Offset
Reset value	PWR_SCR	Reset value	PWR_CR	Register
Reserved	Reserved	Reserved	Reserved	31
				30
				29
				28
				27
				26
				25
				24
				23
				22
				21
				20
				19
				18
				17
				16
0	EWUP	Reserved	DBP	15
				14
				13
				12
				11
				10
				9
				8
				7
				6
0	Reserved	Reserved	PLS [2:0]	5
				4
				3
				2
				1
0	PVD0	0	CWUF	0
0	SBF	0	PDDS	0
0	WUTE	0	IBDS	0

Refer to Table 1 for the register boundary addresses.

5. Backup registers (BKP)

5.1 BKP introduction

The backup registers are forty two 16-bit registers for storing 84 bytes of user application data. They are implemented in the backup domain that remains powered on by V_{BAT} when the V_{DD} power is switched off. They are not reset when the device wakes up from Standby mode or by a system reset or power reset. In addition, the BKP control registers are used to manage the Tamper detection feature and RTC calibration.

After reset, access to the Backup registers and RTC is disabled and the Backup domain (BKP) is protected against possible parasitic write access. To enable access to the Backup registers and the RTC, proceed as follows:

- enable the power and backup interface clocks by setting the PWREN and BKPEN bits in the RCC_APB1ENR register
- set the DBP bit the Power Control Register (PWR_CR) to enable access to the Backup registers and RTC.

5.2 BKP main features

- 20-byte data registers
- Status/control register for managing tamper detection with interrupt capability
- Calibration register for storing the RTC calibration value
- Possibility to output the RTC Calibration Clock, RTC Alarm pulse or Second pulse on TAMPER pin PC13 (when this pin is not used for tamper detection)

5.3 BKP functional description

5.3.1 Tamper detection

The TAMPER pin generates a Tamper detection event when the pin changes from 0 to 1 or from 1 to 0 depending on the TPAL bit in the *Backup control register (BKP_CR)*. A tamper detection event resets all data backup registers.

However to avoid losing Tamper events, the signal used for edge detection is logically ANDed with the Tamper enable in order to detect a Tamper event in case it occurs before the TAMPER pin is enabled.

- **When TPAL=0:** If the TAMPER pin is already high before it is enabled (by setting TPE bit), an extra Tamper event is detected as soon as the TAMPER pin is enabled (while there was no rising edge on the TAMPER pin after TPE was set)
- **When TPAL=1:** If the TAMPER pin is already low before it is enabled (by setting the TPE bit), an extra Tamper event is detected as soon as the TAMPER pin is enabled (while there was no falling edge on the TAMPER pin after TPE was set)

By setting the TPIE bit in the BKP_CSR register, an interrupt is generated when a Tamper detection event occurs

After a Tamper event has been detected and cleared, the TAMPER pin should be disabled and then re-enabled with TPE before writing to the backup data registers (BKP_DRx) again. This prevents software from writing to the backup data registers (BKP_DRx), while the TAMPER pin value still indicates a Tamper detection. This is equivalent to a level detection on the TAMPER pin.

Note: Tamper detection is still active when V_{DD} power is switched off. To avoid unwanted resetting of the data backup registers, the TAMPER pin should be externally tied to the correct level

5.3.2 RTC calibration

For measurement purposes, the RTC clock with a frequency divided by 64 can be output on the TAMPER pin. This is enabled by setting the CCO bit in the *RTC clock calibration register (BKP_RTCCR)*. The clock can be slowed down by up to 121 ppm by configuring CAL[6:0] bits.

5.4 BKP registers

Refer to Section 2.2 for a list of abbreviations used in register descriptions.
The peripheral registers can be accessed by half-words (16-bit) or words (32-bit).

5.4.1 Backup data register x (BKP_DRx) (x = 1 ... 10)

Address offset: 0x04 to 0x28, 0x40 to 0xBC

Reset value: 0x0000 0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
D[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 15: 0		D[15:0] Backup data These bits can be written with user data. <i>Note: The BKP_DRx registers are not reset by a System reset or Power reset or when the device wakes up from Standby mode. They are reset by a Backup Domain reset or by a TAMPER pin event (if the TAMPER pin function is activated).</i>													

5.4.2 RTC clock calibration register (BKP_RTCCR)

Address offset: 0x2C

Reset value: 0x0000 0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						ASOS	ASOE	CCO	CAL[6: 0]						
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 15: 8		Reserved, must be kept at reset value.													
Bit 9		ASOS: Alarm or second output selection When the ASOE bit is set, the ASOS bit can be used to select whether the signal output on the TAMPER pin is the RTC Second pulse signal or the Alarm pulse signal: 0: RTC Alarm pulse output selected 1: RTC Second pulse output selected <i>Note: This bit is reset only by a Backup domain reset.</i>													
Bit 8		ASOE: Alarm or second output enable Setting this bit outputs either the RTC Alarm pulse signal or the Second pulse signal on the TAMPER pin depending on the ASOS bit. The output pulse duration is one RTC clock period. The TAMPER pin must not be enabled while the ASOE bit is set. <i>Note: This bit is reset only by a Backup domain reset.</i>													
Bit 7		CCO: Calibration clock output 0: No effect 1: Setting this bit outputs the RTC clock with a frequency divided by 64 on the TAMPER pin. The TAMPER pin must not be enabled while the CCO bit is set in order to avoid unwanted Tamper													

	detection. <i>Note: This bit is reset when the V_{DD} supply is powered off.</i>
Bits6:0	CAL[6:0]: Calibration value This value indicates the number of clock pulses that will be ignored every 2^{20} clock pulses. This allows the calibration of the RTC, slowing down the clock by steps of $1000000/2^{20}$ PPM. The clock of the RTC can be slowed down from 0 to 121PPM.

5.4.3 Backup control register (BKP_CR)

Address offset: 0x30

Reset value: 0x0000 0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														TPAL	TPE
														rw	rw
Bits 15: 2	Reserved, must be kept at reset value.														
Bit 1	TPAL: TAMPER pin active level 0: A high level on the TAMPER pin resets all data backup registers (if TPE bit is set). 1: A low level on the TAMPER pin resets all data backup registers (if TPE bit is set).														
Bit 0	TPE: TAMPER pin enable 0: The TAMPER pin is free for general purpose I/O 1: Tamper alternate I/O function is activated.														

Note: Setting the TPAL and TPE bits at the same time is always safe, however resetting both at the same time can generate a spurious Tamper event. For this reason it is recommended to change the TPAL bit only when the TPE bit is reset.

5.4.4 Backup control/status register (BKP_CSR)

Address offset: 0x34

Reset value: 0x0000 0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						TIF	TEF	Reserved					TPIE	CTI	CTE
						r	r						rw	w	w
Bits 15: 10	Reserved, must be kept at reset value.														
Bit 9	TIF: Tamper interrupt flag This bit is set by hardware when a Tamper event is detected and the TPIE bit is set. It is cleared by writing 1 to the CTI bit (also clears the interrupt). It is also cleared if the TPIE bit is reset. 0: No Tamper interrupt 1: A Tamper interrupt occurred <i>Note: This bit is reset only by a system reset and wakeup from Standby mode.</i>														
Bit 8	TEF: Tamper event flag This bit is set by hardware when a Tamper event is detected. It is cleared by writing 1 to the CTE bit. 0: No Tamper event 1: A Tamper event occurred <i>Note: A Tamper event resets all the BKP_DRx registers. They are held in reset as long as the TEF bit is set. If a write to the BKP_DRx registers is performed while this bit is set, the value will not be stored</i>														
Bits 7:3	Reserved, must be kept at reset value.														
Bit 2	TPIE: TAMPER pin interrupt enable 0: Tamper interrupt disabled 1: Tamper interrupt enabled (the TPE bit must also be set in the BKP_CR register <i>Note: A Tamper interrupt does not wake up the core from low-power modes. This bit is reset only by a system reset and wakeup from Standby mode.</i>														
Bit 1	CTI: Clear tamper interrupt This bit is write only, and is always read as 0. 0: No effect 1: Clear the Tamper interrupt and the TIF Tamper interrupt flag.														
Bit 0	CTE: Clear tamper event This bit is write only, and is always read as 0 0: No effect 1: Reset the TEF Tamper event flag (and the Tamper detector)														

5.4.5 BKP register map

BKP registers are mapped as 16-bit addressable registers as described in the table below:

Table 11 BKP register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
000h	Reserved																																
004h	BKP_DR1	Reserved																D[15:0]															
	Rest value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
008h	BKP_DR2	Reserved																D[15:0]															
	Rest value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
00Ch	BKP_DR3	Reserved																D[15:0]															
	Rest value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
010h	BKP_DR4	Reserved																D[15:0]															
	Rest value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
014h	BKP_DR5	Reserved																D[15:0]															

	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
018h	BKP_DR6	Reserved	D[15:0]																
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
01Ch	BKP_DR7	Reserved	D[15:0]																
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
020h	BKP_DR8	Reserved	D[15:0]																
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
024h	BKP_DR9	Reserved	D[15:0]																
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
028h	BKP_DR10	Reserved	D[15:0]																
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
02Ch	BKP_RTCCT	Reserved											ASOS	ASOE	CC0	CAL[6:0]			

	Rest value											0	0	0	0	0	0	0	0	0	0
030h	RTC_CR	Reserved																	TPAL	TPE	
	Rest value																		0	0	
034h	RTC_CSR	Reserved										TIF	TFE	Reserved					TPIE	CTI	CTE
	Rest value											0	0						0	0	0
038h	Reserved																				
03Ch	Reserved																				
040h	BKP_DR11	Reserved										D[15:0]									
	Rest value											0	0	0	0	0	0	0	0	0	0
044h	BKP_DR12	Reserved										D[15:0]									
	Rest value											0	0	0	0	0	0	0	0	0	0
048h	BKP_DR13	Reserved										D[15:0]									
	Rest value											0	0	0	0	0	0	0	0	0	0

04Ch	BKP_DR14	Reserved	D[15:0]															
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
050h	BKP_DR15	Reserved	D[15:0]															
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
054h	BKP_DR16	Reserved	D[15:0]															
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
058h	BKP_DR17	Reserved	D[15:0]															
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
05Ch	BKP_DR18	Reserved	D[15:0]															
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
060h	BKP_DR19	Reserved	D[15:0]															
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

064h	BKP_DR20	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
068h	BKP_DR21	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
06Ch	BKP_DR22	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
070h	BKP_DR23	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
074h	BKP_DR24	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
078h	BKP_DR25	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0

07Ch	BKP_DR26	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
080h	BKP_DR27	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
084h	BKP_DR28	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
088h	BKP_DR29	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
08Ch	BKP_DR30	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
090h	BKP_DR31	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0

094h	BKP_DR32	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
098h	BKP_DR33	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
09Ch	BKP_DR34	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
0A0h	BKP_DR35	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
0A4h	BKP_DR36	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
0A8h	BKP_DR37	Reserved	D[15:0]													
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0

0ACh	BKP_DR38	Reserved	D[15:0]															
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0B0h	BKP_DR39	Reserved	D[15:0]															
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0B4h	BKP_DR40	Reserved	D[15:0]															
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0B8h	BKP_DR41	Reserved	D[15:0]															
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0BCh	BKP_DR42	Reserved	D[15:0]															
	Rest value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Refer to Table 1 for the register boundary addresses.

6 Reset and clock control (RCC)

6.1 Reset

There are three types of reset, defined as system reset, power reset and backup domain reset.

6.1.1 System reset

A system reset sets all registers to their reset values except the reset flags in the clock controller CSR register and the registers in the Backup domain (see Figure 3).

A system reset is generated when one of the following events occurs:

1. A low level on the NRST pin (external reset)
2. Window watchdog end of count condition (WWDG reset)
3. Independent watchdog end of count condition (IWDG reset)
4. A software reset (SW reset) (see *Software reset*)
5. Low-power management reset (see *Low-power management reset*)

The reset source can be identified by checking the reset flags in the Control/Status register, RCC_CSR (see Section 6.3.10: *Control/status register (RCC_CSR)*).

Software reset

The SYSRESETREQ bit in Cortex®-M3 Application Interrupt and Reset Control Register must be set to force a software reset on the device. Refer to the CKS32F103xx Cortex®-M3 programming manual for more details.

Low-power management reset

There are two ways to generate a low-power management reset:

1. Reset generated when entering Standby mode:
This type of reset is enabled by resetting nRST_STDBY bit in User Option Bytes. In this case, whenever a Standby mode entry sequence is successfully executed, the device is reset instead of entering Standby mode.
2. Reset when entering Stop mode:
This type of reset is enabled by resetting nRST_STOP bit in User Option Bytes. In this case, whenever a Stop mode entry sequence is successfully executed, the device is reset instead of entering Stop mode.

For further information on the User Option Bytes, refer to the CKS32F10xxx Flash programming manual.

6.1.2 Power reset

A power reset is generated when one of the following events occurs:

1. Power-on/power-down reset (POR/PDR reset)
2. When exiting Standby mode

A power reset sets all registers to their reset values except the Backup domain (see Figure 3).

These sources act on the NRST pin and it is always kept low during the delay phase. The RESET service routine vector is fixed at address 0x0000_0004 in the memory map.

The system reset signal provided to the device is output on the NRST pin. The pulse generator guarantees

a minimum reset pulse duration of 20 μ s for each reset source (external or internal reset). In case of an external reset, the reset pulse is generated while the NRST pin is asserted low.

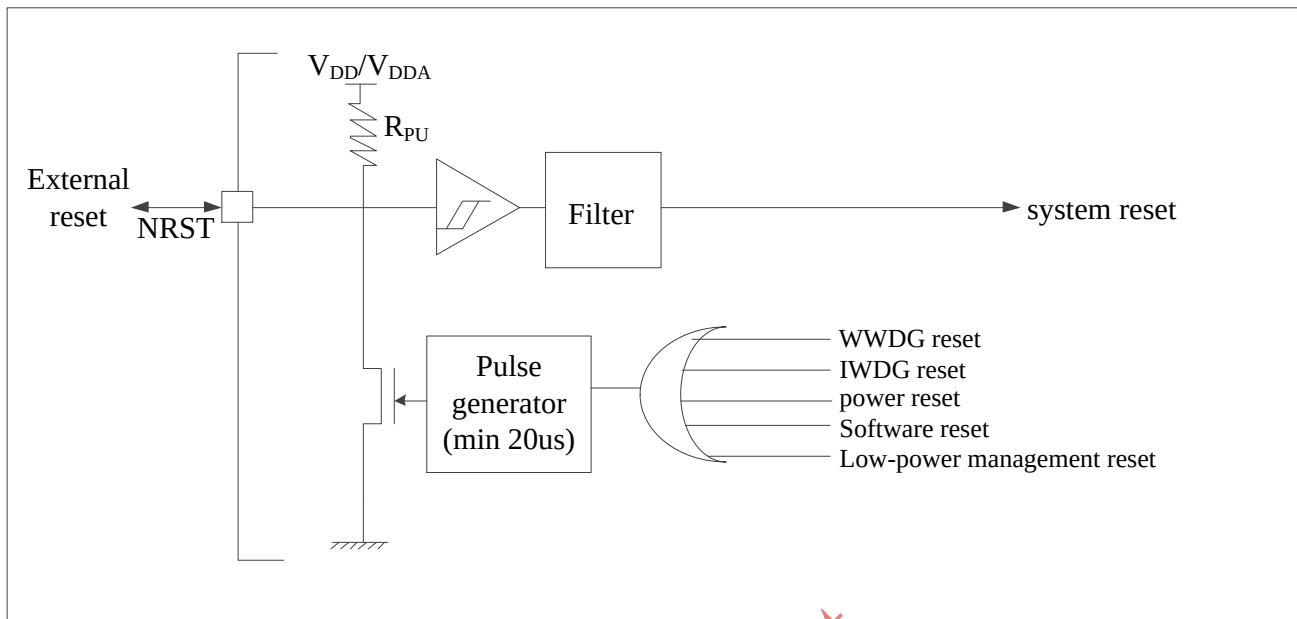


Figure 6 Simplified diagram of the reset circuit

6.1.3 Backup domain reset

The backup domain has two specific resets that affect only the backup domain (see Figure 3). A backup domain reset is generated when one of the following events occurs:

1. Software reset, triggered by setting the BDRST bit in the *Backup domain control register* (RCC_BDCR).
2. VDD or VBAT power on, if both supplies have previously been powered off.

6.2 Clocks

Three different clock sources can be used to drive the system clock (SYSCLK):

- HSI oscillator clock
- HSE oscillator clock
- PLL clock

The devices have the following two secondary clock sources:

- 40 kHz low speed internal RC (LSI RC), which drives the independent watchdog and optionally the RTC used for Auto-wakeup from Stop/Standby mode.
- 32.768 kHz low speed external crystal (LSE crystal), which optionally drives the real-time clock (RTCCLK)

Each clock source can be switched on or off independently when it is not used, to optimize power consumption.

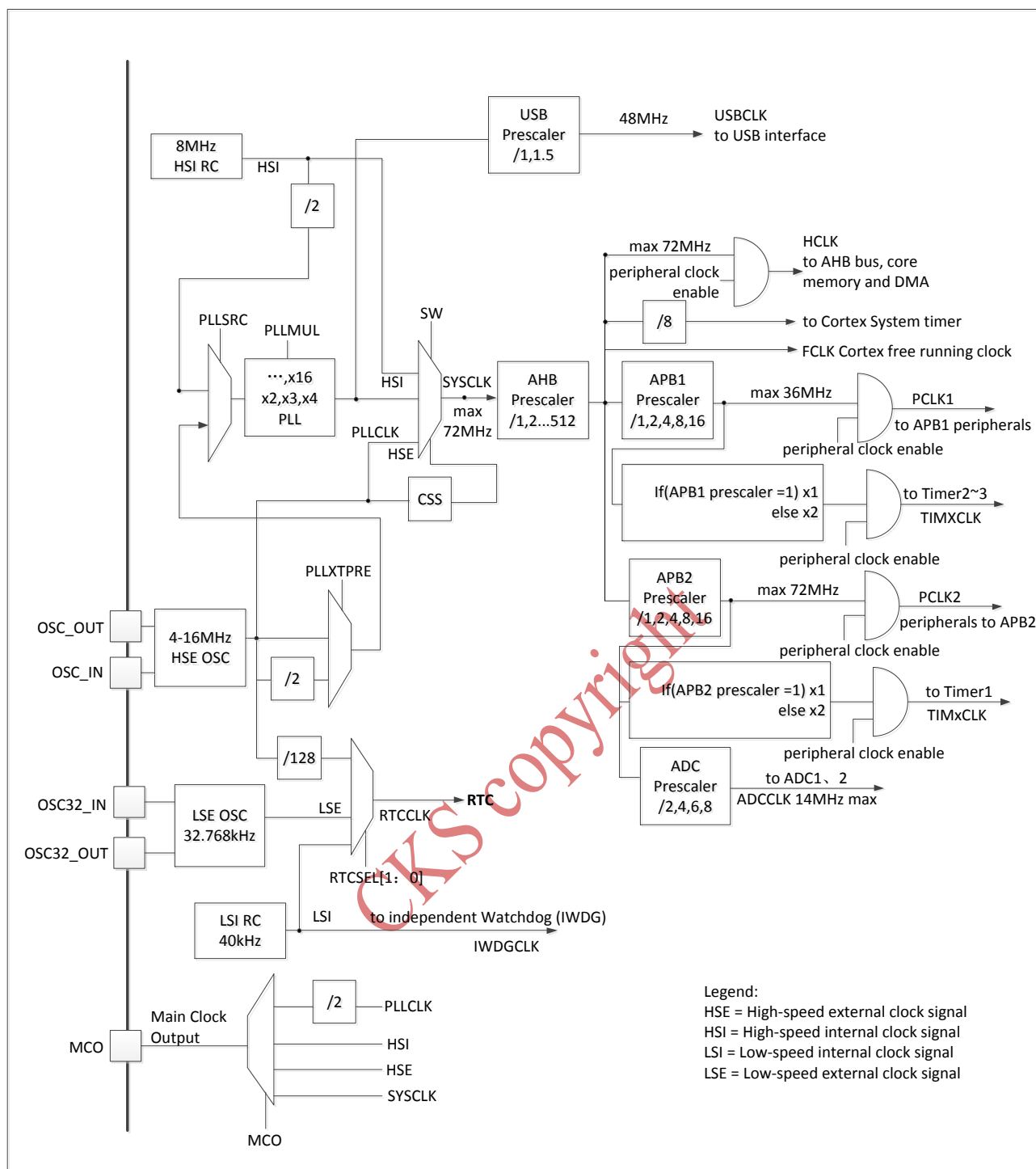


Figure 7 Clock tree

1. When the HSI is used as a PLL clock input, the maximum system clock frequency that can be achieved is 64 MHz.
2. For full details about the internal and external clock source characteristics refer to the “Electrical characteristics” section in your device datasheet
 Several prescalers allow the configuration of the AHB frequency, the high speed APB (APB2) and the low speed APB (APB1) domains. The maximum frequency of the AHB and the APB2 domains is 72 MHz. The maximum allowed frequency of the APB1 domain is 36 MHz. The SDIO AHB interface is clocked with a fixed frequency equal to HCLK/2.
 The RCC feeds the Cortex® System Timer (SysTick) external clock with the AHB clock (HCLK) divided by 8. The SysTick can work either with this clock or with the Cortex® clock (HCLK), configurable in the SysTick Control and Status Register. The ADCs are clocked by the clock of the High Speed domain (APB2) divided by 2, 4, 6 or 8.
 The Flash memory programming interface clock (FLITFCLK) is always the HSI clock.

The timer clock frequencies are automatically fixed by hardware. There are two cases:

1. if the APB prescaler is 1, the timer clock frequencies are set to the same frequency as that of the APB domain to which the timers are connected.
2. otherwise, they are set to twice ($\times 2$) the frequency of the APB domain to which the timers are connected.

FCLK acts as Cortex[®]-M3's free-running clock. For more details refer to Arm[®] Cortex-M3 r1p1 *Technical Reference Manual (TRM)*.

6.2.1 HSE clock

The high speed external clock signal (HSE) can be generated from two possible clock sources:

- HSE external crystal/ceramic resonator
- HSE user external clock

The resonator and the load capacitors have to be placed as close as possible to the oscillator pins in order to minimize output distortion and startup stabilization time. The loading capacitance values must be adjusted according to the selected oscillator.

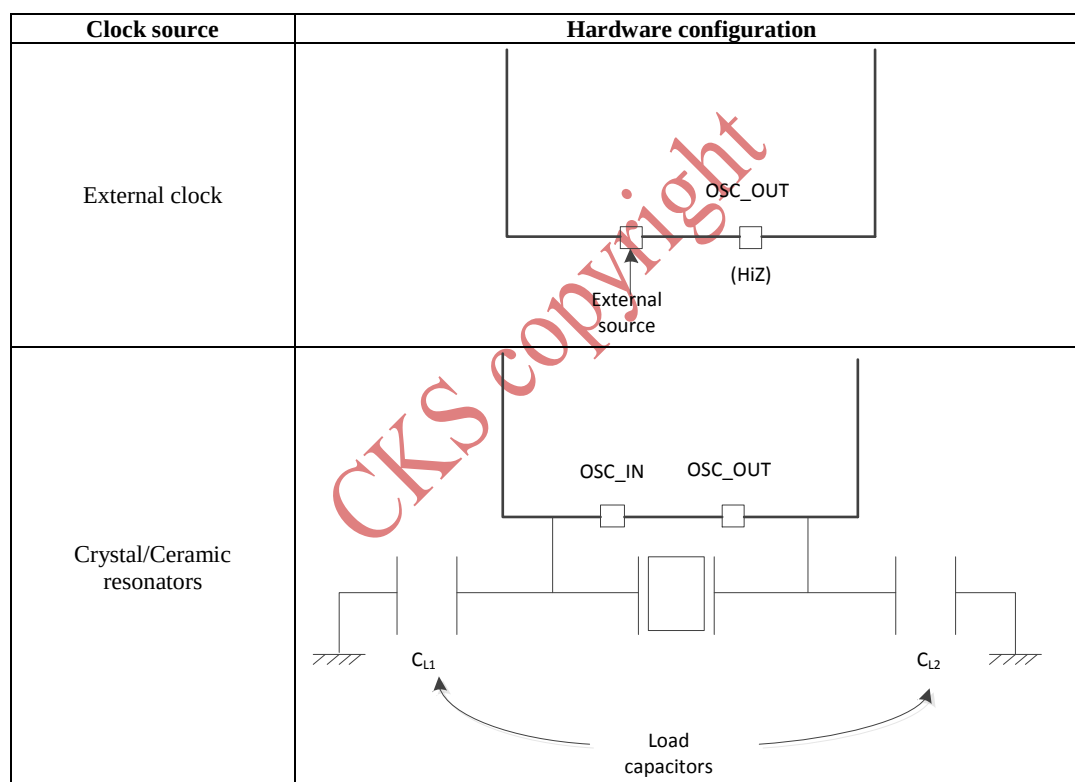


Figure 8 HSE/LSE clock sources

External source (HSE bypass)

In this mode, an external clock source must be provided. It can have a frequency of up to 25 MHz. You select this mode by setting the HSEBYP and HSEON bits in the *Clock control register (RCC_CR)*. The external clock signal (square, sinus or triangle) with $\sim 50\%$ duty cycle has to drive the OSC_IN pin while the OSC_OUT pin should be left hi-Z. See Figure 8.

External crystal/ceramic resonator (HSE crystal)

The 4 to 16 MHz external oscillator has the advantage of producing a very accurate rate on the main clock.

The associated hardware configuration is shown in Figure 8. Refer to the electrical characteristics section of the *datasheet* for more details.

The HSERDY flag in the *Clock control register (RCC_CR)* indicates if the high-speed external oscillator is stable or not. At startup, the clock is not released until this bit is set by hardware. An interrupt can be generated if enabled in the *Clock interrupt register (RCC_CIR)*.

The HSE Crystal can be switched on and off using the HSEON bit in the *Clock control register (RCC_CR)*.

6.2.2 HSI clock

The HSI clock signal is generated from an internal 8 MHz RC Oscillator and can be used directly as a system clock or divided by 2 to be used as PLL input.

The HSI RC oscillator has the advantage of providing a clock source at low cost (no external components). It also has a faster startup time than the HSE crystal oscillator however, even with calibration the frequency is less accurate than an external crystal oscillator or ceramic resonator.

Calibration

RC oscillator frequencies can vary from one chip to another due to manufacturing process variations, this is why each device is factory calibrated for 1% accuracy at $T_A=25\text{ }^{\circ}\text{C}$. After reset, the factory calibration value is loaded in the HSICAL[7:0] bits in the *Clock control register (RCC_CR)*.

If the application is subject to voltage or temperature variations this may affect the RC oscillator speed. You can trim the HSI frequency in the application using the HSITRIM[4:0] bits in the *Clock control register (RCC_CR)*.

The HSI RDY flag in the *Clock control register (RCC_CR)* indicates if the HSI RC is stable or not. At startup, the HSI RC output clock is not released until this bit is set by hardware.

The HSI RC can be switched on and off using the HSION bit in the *Clock control register (RCC_CR)*.

The HSI signal can also be used as a backup source (Auxiliary clock) if the HSE crystal oscillator fails. Refer to Section 6.2.7: Clock security system (CSS).

6.2.3 PLL

The internal PLL can be used to multiply the HSI RC output or HSE crystal output clock frequency. Refer to Figure 7 and *Clock control register (RCC_CR)*.

The PLL configuration (selection of HSI oscillator divided by 2 or HSE oscillator for PLL input clock, and multiplication factor) must be done before enabling the PLL. Once the PLL enabled, these parameters cannot be changed.

An interrupt can be generated when the PLL is ready if enabled in the *Clock interrupt register (RCC_CIR)*.

If the USB interface is used in the application, the PLL must be programmed to output 48 or 72 MHz. This is needed to provide a 48 MHz USBCLK.

6.2.4 LSE clock

The LSE crystal is a 32.768 kHz Low Speed External crystal or ceramic resonator. It has the advantage providing a low-power but highly accurate clock source to the real-time clock peripheral (RTC) for clock/calendar or other timing functions.

The LSE crystal is switched on and off using the LSEON bit in *Backup domain control register (RCC_BDCR)*.

The LSE RDY flag in the *Backup domain control register (RCC_BDCR)* indicates if the LSE crystal is stable or not. At startup, the LSE crystal output clock signal is not released until this bit is set by hardware. An interrupt can be generated if enabled in the *Clock interrupt register (RCC_CIR)*.

External source (LSE bypass)

In this mode, an external clock source must be provided. It can have a frequency of up to 1 MHz. You select this mode by setting the LSEBYP and LSEON bits in the *Backup domain control register*

(*RCC_BDCR*). The external clock signal (square, sinus or triangle) with ~50% duty cycle has to drive the OSC32_IN pin while the OSC32_OUT pin should be left Hi-Z. See Figure 8.

6.2.5 LSI clock

The LSI RC acts as an low-power clock source that can be kept running in Stop and Standby mode for the independent watchdog (IWDG) and Auto-wakeup unit (AWU). The clock frequency is around 40 kHz (between 30 kHz and 60 kHz). For more details, refer to the electrical characteristics section of the datasheets.

The LSI RC can be switched on and off using the LSION bit in the *Control/status register (RCC_CSR)*. The LSIRDY flag in the *Control/status register (RCC_CSR)* indicates if the low-speed internal oscillator is stable or not. At startup, the clock is not released until this bit is set by hardware. An interrupt can be generated if enabled in the *Clock interrupt register (RCC_CIR)*.

6.2.6 System clock (SYSCLK) selection

After a system reset, the HSI oscillator is selected as system clock. When a clock source is used directly or through the PLL as system clock, it is not possible to stop it.

A switch from one clock source to another occurs only if the target clock source is ready (clock stable after startup delay or PLL locked). If a clock source which is not yet ready is selected, the switch will occur when the clock source will be ready. Status bits in the *Clock control register (RCC_CR)* indicate which clock(s) is (are) ready and which clock is currently used as system clock.

6.2.7 Clock security system (CSS)

Clock Security System can be activated by software. In this case, the clock detector is enabled after the HSE oscillator startup delay, and disabled when this oscillator is stopped.

If a failure is detected on the HSE clock, the HSE oscillator is automatically disabled, a clock failure event is sent to the break input of the advanced-control timer (TIM1) and an interrupt is generated to inform the software about the failure (Clock Security System Interrupt CSSI), allowing the MCU to perform rescue operations. The CSSI is linked to the Cortex®-M3 NMI (Non-Maskable Interrupt) exception vector.

Note: Once the CSS is enabled and if the HSE clock fails, the CSS interrupt occurs and an NMI is automatically generated. The NMI will be executed indefinitely unless the CSS interrupt pending bit is cleared. As a consequence, in the NMI ISR user must clear the CSS interrupt by setting the CSSC bit in the *Clock interrupt register (RCC_CIR)*.

If the HSE oscillator is used directly or indirectly as the system clock (indirectly means: it is used as PLL input clock, and the PLL clock is used as system clock), a detected failure causes a switch of the system clock to the HSI oscillator and the disabling of the HSE oscillator. If the HSE clock (divided or not) is the clock entry of the PLL used as system clock when the failure occurs, the PLL is disabled too.

6.2.8 RTC clock

The RTCCLK clock source can be either the HSE/128, LSE or LSI clocks. This is selected by programming the RTCSEL[1:0] bits in the *Backup domain control register (RCC_BDCR)*. This selection cannot be modified without resetting the Backup domain.

The LSE clock is in the Backup domain, whereas the HSE and LSI clocks are not. Consequently:

- If LSE is selected as RTC clock:
 - The RTC continues to work even if the V_{DD} supply is switched off, provided the V_{BAT} supply is maintained.
- If LSI is selected as Auto-Wakeup unit (AWU) clock:

- The AWU state is not guaranteed if the V_{DD} supply is powered off.
- If the HSE clock divided by 128 is used as the RTC clock:
 - The RTC state is not guaranteed if the V_{DD} supply is powered off or if the internal voltage regulator is powered off (removing power from the 1.5 V domain).
 - The DPB bit (disable backup domain write protection) in the Power controller register must be set to 1 (refer to *Section 4.4.1: Power control register (PWR_CR)*).

6.2.9 Watchdog clock

If the Independent watchdog (IWDG) is started by either hardware option or software access, the LSI oscillator is forced ON and cannot be disabled. After the LSI oscillator temporization, the clock is provided to the IWDG.

6.2.10 Clock-out capability

The microcontroller clock output (MCO) capability allows the clock to be output onto the external MCO pin. The configuration registers of the corresponding GPIO port must be programmed in alternate function mode. One of 4 clock signals can be selected as the MCO clock.

- SYSCLK
- HIS
- HSE
- PLL clock divided by 2

The selection is controlled by the MCO[2:0] bits of the *Clock configuration register (RCC_CFGR)*.

6.3 RCC registers

Refer to *Section 1.1* for a list of abbreviations used in register descriptions.

6.3.1 Clock control register (RCC_CR)

Address offset: 0x00

Reset value: 0x000 XX83, where X is undefined.

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved						PLL RDY	PLLON	Reserved				CSS ON	HSE BYP	HSE RDY	HSE ON
						r	rw					rw	rw	r	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSICAL[7: 0]								HSITRIM[4:0]				Res	HSI RDY	HSION	
r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw		r	rw
Bits 31: 26		Reserved, must be kept at reset value.													
Bit 25		PLLRDY: PLL clock ready flag Set by hardware to indicate that the PLL is locked. 0: PLL unlocked 1: PLL locked													
Bit 24		PLLON: PLL enable Set and cleared by software to enable PLL. Cleared by hardware when entering Stop or Standby mode. This bit can not be reset if the PLL clock is used as system clock or is selected to become the system clock. 0: PLL OFF 1: PLL ON													

Bits 23:20	Reserved, must be kept at reset value.
Bit 19	CSSON: Clock security system enable Set and cleared by software to enable the clock security system. When CSSON is set, the clock detector is enabled by hardware when the HSE oscillator is ready, and disabled by hardware if a HSE clock failure is detected. 0: Clock detector OFF 1: Clock detector ON (Clock detector ON if the HSE oscillator is ready, OFF if not).
Bit 18	HSEBYP: External high-speed clock bypass Set and cleared by software to bypass the oscillator with an external clock. The external clock must be enabled with the HSEON bit set, to be used by the device. The HSEBYP bit can be written only if the HSE oscillator is disabled. 0: external 4-16 MHz oscillator not bypassed 1: external 4-16 MHz oscillator bypassed with external clock
Bit 17	HSERDY: External high-speed clock ready flag Set by hardware to indicate that the HSE oscillator is stable. This bit needs 6 cycles of the HSE oscillator clock to fall down after HSEON reset. 0: HSE oscillator not ready 1: HSE oscillator ready
Bit 16	HSEON: HSE clock enable Set and cleared by software. Cleared by hardware to stop the HSE oscillator when entering Stop or Standby mode. This bit cannot be reset if the HSE oscillator is used directly or indirectly as the system clock. 0: HSE oscillator OFF 1: HSE oscillator ON
Bits 15:8	HSICAL[7:0]: Internal high-speed clock calibration These bits are initialized automatically at startup.
Bits 7:3	HSITRIM[4:0]: Internal high-speed clock trimming These bits provide an additional user-programmable trimming value that is added to the HSICAL[7:0] bits. It can be programmed to adjust to variations in voltage and temperature that influence the frequency of the internal HSI RC. The default value is 16, which, when added to the HSICAL value, should trim the HSI to 8 MHz $\pm$ 1%. The trimming step (F_{hsitrim}) is around 40 kHz between two consecutive HSICAL steps.
Bit 2	Reserved, must be kept at reset value.
Bit 1	HSIRDY: Internal high-speed clock ready flag Set by hardware to indicate that internal 8 MHz RC oscillator is stable. After the HSION bit is cleared, HSIRDY goes low after 6 internal 8 MHz RC oscillator clock cycles. 0: internal 8 MHz RC oscillator not ready 1: internal 8 MHz RC oscillator ready
Bit 0	HSION: Internal high-speed clock enable Set and cleared by software. Set by hardware to force the internal 8 MHz RC oscillator ON when leaving Stop or Standby mode or in case of failure of the external 4-16 MHz oscillator used directly or indirectly as system clock. This bit cannot be reset if the internal 8 MHz RC is used directly or indirectly as system clock or is selected to become the system clock. 0: internal 8 MHz RC oscillator OFF 1: internal 8 MHz RC oscillator ON

6.3.2 Clock configuration register (RCC_CFGR)

Address offset: 0x04

Reset value: 0x0000 0000

Access: $0 \leq \text{wait state} \leq 2$, word, half-word and byte access 1 or 2 wait states inserted only if the access occurs during clock source switch.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved					MCO[2:0]			Res	USB PRE	PLLMUL[3:0]				PLL XTPRE	PLL SRC
15	14	13	12	11	rw	rw	rw	7	rw	rw	rw	rw	rw	rw	rw
ADCPRE[1:0]		PPRE2[2:0]			PPRE1[2:0]			HPRE[3:0]				SWS[1:0]		SW[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	r	r	rw	rw
Bits 31: 27				Reserved, must be kept at reset value.											

Bits 26:24	MCO: Microcontroller clock output Set and cleared by software. 0xx: No clock 100: System clock (SYSCLK) selected 101: HSI clock selected 110: HSE clock selected 111: PLL clock divided by 2 selected <i>Note: This clock output may have some truncated cycles at startup or during MCO clock source switching.</i> <i>When the System Clock is selected to output to the MCO pin, make sure that this clock does not exceed 50 MHz (the maximum IO speed).</i>
Bit 22	USBPRE: USB prescaler Set and cleared by software to generate 48 MHz USB clock. This bit must be valid before enabling the USB clock in the RCC_APB1ENR register. This bit can't be reset if the USB clock is enabled. 0: PLL clock is divided by 1.5 1: PLL clock is not divided
Bits 21:18	PLLMUL: PLL multiplication factor These bits are written by software to define the PLL multiplication factor. These bits can be written only when PLL is disabled. Caution: The PLL output frequency must not exceed 72 MHz. 0000: PLL input clock x 2 0001: PLL input clock x 3 0010: PLL input clock x 4 0011: PLL input clock x 5 0100: PLL input clock x 6 0101: PLL input clock x 7 0110: PLL input clock x 8 0111: PLL input clock x 9 1000: PLL input clock x 10 1001: PLL input clock x 11 1010: PLL input clock x 12 1011: PLL input clock x 13 1100: PLL input clock x 14 1101: PLL input clock x 15 1110: PLL input clock x 16 1111: PLL input clock x 16
Bit 17	PLLXTPRE: HSE divider for PLL entry Set and cleared by software to divide HSE before PLL entry. This bit can be written only when PLL is disabled. 0: HSE clock not divided 1: HSE clock divided by 2
Bit 16	PLLSRC: PLL entry clock source Set and cleared by software to select PLL clock source. This bit can be written only when PLL is disabled. 0: HSI oscillator clock / 2 selected as PLL input clock 1: HSE oscillator clock selected as PLL input clock
Bits 15:14	ADCPRE: ADC prescaler Set and cleared by software to select the frequency of the clock to the ADCs. 00: PCLK2 divided by 2 01: PCLK2 divided by 4 10: PCLK2 divided by 6 11: PCLK2 divided by 8
Bits 13:11	PPRE2: APB high-speed prescaler (APB2) Set and cleared by software to control the division factor of the APB high-speed clock (PCLK2). 0xx: HCLK not divided 100: HCLK divided by 2 101: HCLK divided by 4 110: HCLK divided by 8 111: HCLK divided by 16
Bits 10:8	PPRE1: APB low-speed prescaler (APB1) Set and cleared by software to control the division factor of the APB low-speed clock (PCLK1). Warning: the software has to set correctly these bits to not exceed 36 MHz on this domain. 0xx: HCLK not divided

	100: HCLK divided by 2 101: HCLK divided by 4 110: HCLK divided by 8 111: HCLK divided by 16
Bits 7:4	HPRE: AHB prescaler Set and cleared by software to control the division factor of the AHB clock. 0xxx: SYSCLK not divided 1000: SYSCLK divided by 2 1001: SYSCLK divided by 4 1010: SYSCLK divided by 8 1011: SYSCLK divided by 16 1100: SYSCLK divided by 64 1101: SYSCLK divided by 128 1110: SYSCLK divided by 256 1111: SYSCLK divided by 512 <i>Note: The prefetch buffer must be kept on when using a prescaler different from 1 on the AHB clock. Refer to Reading the Flash memory section for more details.</i>
Bits 3:2	SWS: System clock switch status Set and cleared by hardware to indicate which clock source is used as system clock. 00: HSI oscillator used as system clock 01: HSE oscillator used as system clock 10: PLL used as system clock 11: not applicable
Bits 1:0	SW: System clock switch Set and cleared by software to select SYSCLK source. Set by hardware to force HSI selection when leaving Stop and Standby mode or in case of failure of the HSE oscillator used directly or indirectly as system clock (if the Clock Security System is enabled). 00: HSI selected as system clock 01: HSE selected as system clock 10: PLL selected as system clock 11: not allowed

6.3.3 Clock interrupt register (RCC_CIR)

Address offset: 0x08

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								CSSC	Reserved	PLL RDYC	HSE RDYC	HSI RDYC	LSE RDYC	LSI RDYC	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	PLL RDYIE	HSE RDYIE	HSI RDYIE	LSE RDYIE	LSI RDYIE	CSSF	Reserved	PLL RDYF	HSE RDYF	HSI RDYF	LSE RDYF	LSI RDYF			
Bits 31: 24		Reserved, must be kept at reset value.													
Bit 23		CSSC: Clock security system interrupt clear This bit is set by software to clear the CSSF flag. 0: No effect 1: Clear CSSF flag													
Bits 22: 21		Reserved, must be kept at reset value.													
Bit 20		PLLRDYC: PLL ready interrupt clear This bit is set by software to clear the PLLRDYF flag. 0: No effect 1: PLLRDYF cleared													
Bit 19		HSERDYC: HSE ready interrupt clear This bit is set by software to clear the HSERDYF flag. 0: No effect													

	1: HSERDYF cleared
Bit 18	HSIRDYC : HSI ready interrupt clear This bit is set software to clear the HSIRDYF flag. 0: No effect 1: HSIRDYF cleared
Bit 17	LSERDYC : LSE ready interrupt clear This bit is set by software to clear the LSERDYF flag. 0: No effect 1: LSERDYF cleared
Bit 16	LSIRDYC : LSI ready interrupt clear This bit is set by software to clear the LSIRDYF flag. 0: No effect 1: LSIRDYF cleared
Bits 15:13	Reserved, must be kept at reset value.
Bit 12	PLLRDYIE : PLL ready interrupt enable Set and cleared by software to enable/disable interrupt caused by PLL lock. 0: PLL lock interrupt disabled 1: PLL lock interrupt enabled
Bit 11	HSERDYIE : HSE ready interrupt enable Set and cleared by software to enable/disable interrupt caused by the external 4-16 MHz oscillator stabilization. 0: HSE ready interrupt disabled 1: HSE ready interrupt enabled
Bit 10	HSIRDYIE : HSI ready interrupt enable Set and cleared by software to enable/disable interrupt caused by the internal 8 MHz RC oscillator stabilization. 0: HSI ready interrupt disabled 1: HSI ready interrupt enabled
Bit 9	LSERDYIE : LSE ready interrupt enable Set and cleared by software to enable/disable interrupt caused by the external 32 kHz oscillator stabilization. 0: LSE ready interrupt disabled 1: LSE ready interrupt enabled
Bit 8	LSIRDYIE : LSI ready interrupt enable Set and cleared by software to enable/disable interrupt caused by internal RC 40 kHz oscillator stabilization. 0: LSI ready interrupt disabled 1: LSI ready interrupt enabled
Bit 7	CSSF : Clock security system interrupt flag Set by hardware when a failure is detected in the external 4-16 MHz oscillator. Cleared by software setting the CSSC bit. 0: No clock security interrupt caused by HSE clock failure 1: Clock security interrupt caused by HSE clock failure
Bits 6:5	Reserved, must be kept at reset value.
Bit 4	PLLRDYF : PLL ready interrupt flag Set by hardware when the PLL locks and PLLRDYIE is set. Cleared by software setting the PLLRDYC bit. 0: No clock ready interrupt caused by PLL lock 1: Clock ready interrupt caused by PLL lock
Bit 3	HSERDYF : HSE ready interrupt flag Set by hardware when External High Speed clock becomes stable and HSERDYIE is set. Cleared by software setting the HSERDYC bit. 0: No clock ready interrupt caused by the external 4-16 MHz oscillator 1: Clock ready interrupt caused by the external 4-16 MHz oscillator
Bit 2	HSIRDYF : HSI ready interrupt flag Set by hardware when the Internal High Speed clock becomes stable and HSIRDYIE is set. Cleared by software setting the HSIRDYC bit. 0: No clock ready interrupt caused by the internal 8 MHz RC oscillator 1: Clock ready interrupt caused by the internal 8 MHz RC oscillator
Bit 1	LSERDYF : LSE ready interrupt flag Set by hardware when the External Low Speed clock becomes stable and LSERDYIE is set. Cleared by software setting the LSERDYC bit. 0: No clock ready interrupt caused by the external 32 kHz oscillator 1: Clock ready interrupt caused by the external 32 kHz oscillator
Bit 0	LSIRDYF : LSI ready interrupt flag Set by hardware when the internal low speed clock becomes stable and LSIRDYIE is set. Cleared by software setting the LSIRDYC bit. 0: No clock ready interrupt caused by the internal RC 40 kHz oscillator

	1: Clock ready interrupt caused by the internal RC 40 kHz oscillator
--	--

6.3.4 APB2 peripheral reset register (RCC_APB2RSTR)

Address offset: 0x0C

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	USART1 RST	Res	SPI1 RST	TIM1 RST	ADC2 RST	ADC1 RST	Reserved		IOPE RST	IOPD RST	IOPC RST	IOPB RST	IOPA RST	Res	AFIO RST
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	res	rw
Bits 31: 15		Reserved, must be kept at reset value.													
Bit 14		USART1RST: USART1 reset Set and cleared by software. 0: No effect 1: Reset USART1													
Bit 13		Reserved, must be kept at reset value.													
Bit 12		SPI1RST: SPI1 reset Set and cleared by software. 0: No effect 1: Reset SPI1													
Bit 11		TIM1RST: TIM1 timer reset Set and cleared by software. 0: No effect 1: Reset TIM1 timer													
Bit 10		ADC2RST: ADC 2 interface reset Set and cleared by software. 0: No effect 1: Reset ADC 2 interface													
Bit 9		ADC1RST: ADC 1 interface reset Set and cleared by software. 0: No effect 1: Reset ADC 1 interface													
Bits 8:7		Reserved, must be kept at reset value.													
Bit 6		IOPERST: IO port E reset Set and cleared by software. 0: No effect 1: Reset IO port E													
Bit 5		IOPDRST: IO port D reset Set and cleared by software. 0: No effect 1: Reset IO port D													
Bit 4		IOPCRST: IO port C reset Set and cleared by software. 0: No effect 1: Reset IO port C													
Bit 3		IOPBRST: IO port B reset Set and cleared by software. 0: No effect 1: Reset IO port B													
Bit 2		IOPARST: IO port A reset Set and cleared by software. 0: No effect 1: Reset IO port A													
Bit 1		Reserved, must be kept at reset value.													
Bit 0		AFIORST: Alternate function IO reset Set and cleared by software. 0: No effect 1: Reset Alternate Function													

6.3.5 APB1 peripheral reset register (RCC_APB1RSTR)

Address offset: 0x10

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved			PWR RST	BKP RST	Res	CAN RST	Res	USB RST	I2C2 RST	I2C1 RST	Reserved		UART3 RST	UART2 RST	Res
rw			rw	rw		rw		rw	rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	SPI2 RST	Reserved		WWDG RST	Reserved								TIM4 RST	TIM3 RST	TIM2 RST
rw	rw	rw		rw	rw								rw	rw	rw

Bits 31:29	Reserved, must be kept at reset value.
Bit 28	PWRRST : Power interface reset Set and cleared by software. 0: No effect 1: Reset power interface
Bit 27	BKPRST : Backup interface reset Set and cleared by software. 0: No effect 1: Reset backup interface
Bit 26	Reserved, must be kept at reset value.
Bit 25	CANRST : CAN reset Set and cleared by software. 0: No effect 1: Reset CAN
Bit 24	Reserved, always read as 0.
Bit 23	USBRST : USB reset Set and cleared by software. 0: No effect 1: Reset USB
Bit 22	I2C2RST : I2C2 reset Set and cleared by software. 0: No effect 1: Reset I2C2
Bit 21	I2C1RST : I2C1 reset Set and cleared by software. 0: No effect 1: Reset I2C1
Bits 20:19	Reserved, must be kept at reset value.
Bit 18	USART3RST : USART3 reset Set and cleared by software. 0: No effect 1: Reset USART3
Bit 17	USART2RST : USART2 reset Set and cleared by software. 0: No effect 1: Reset USART2
Bits 16:15	Reserved, must be kept at reset value.
Bit 14	SPI2RST : SPI2 reset Set and cleared by software. 0: No effect 1: Reset SPI2
Bits 13:12	Reserved, must be kept at reset value.
Bit 11	WWDGRST : Window watchdog reset Set and cleared by software. 0: No effect 1: Reset window watchdog
Bits 10:3	Reserved, must be kept at reset value.
Bit 2	TIM4RST : TIM4 timer reset Set and cleared by software. 0: No effect

	1: Reset TIM4
Bit 1	TIM3RST: TIM3 timer reset Set and cleared by software. 0: No effect 1: Reset TIM3
Bit 0	TIM2RST: TIM2 timer reset Set and cleared by software 0: No effect 1: Reset TIM2

6.3.6 AHB peripheral clock enable register (RCC_AHBENR)

Address offset: 0x14

Reset value: 0x0000 0014

Access: no wait state, word, half-word and byte access

Note: When the peripheral clock is not active, the peripheral register values may not be readable by software and the returned value is always 0x0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16		
Reserved																	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Reserved									CRC EN	Res	FLITF EN	Res	SRAM EN	Res	DMA EN		
rw			rw			rw			rw			rw			rw		
Bits 31:7		Reserved, must be kept at reset value.															
Bit 6		CRCEN: CRC clock enable Set and cleared by software. 0: CRC clock disabled 1: CRC clock enabled															
Bit 5		Reserved, must be kept at reset value.															
Bit 4		FLITFEN: FLITF clock enable Set and cleared by software to disable/enable FLITF clock during Sleep mode. 0: FLITF clock disabled during Sleep mode 1: FLITF clock enabled during Sleep mode															
Bit 3		Reserved, must be kept at reset value.															
Bit 2		SRAMEN: SRAM interface clock enable Set and cleared by software to disable/enable SRAM interface clock during Sleep mode. 0: SRAM interface clock disabled during Sleep mode. 1: SRAM interface clock enabled during Sleep mode															
Bit 1		Reserved, must be kept at reset value.															
Bit 0		DMAEN: DMA clock enable Set and cleared by software. 0: DMA clock disabled; 1: DMA clock enabled.															

6.3.7 APB2 peripheral clock enable register (RCC_APB2ENR)

Address: 0x18

Reset value: 0x0000 0000

Access: word, half-word and byte access

No wait states, except if the access occurs while an access to a peripheral in the APB2 domain is on going. In this case, wait states are inserted until the access to APB2 peripheral is finished.

Note: When the peripheral clock is not active, the peripheral register values may not be readable by software and the returned value is always 0x0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Res	USART1 EN	Res	SPI1 EN	TIM1 EN	ADC2 EN	ADC1 EN	Reserved	IOPE EN	IOPD EN	IOPC EN	IOPB EN	IOPA EN	Res	AFIO EN
	rw		rw	rw	rw	rw		rw	rw	rw	rw	rw		rw
Bits 31:15	Reserved, must be kept at reset value.													
Bit 14	USART1EN: USART1 clock enable Set and cleared by software. 0: USART1 clock disabled 1: USART1 clock enabled													
Bit 13	Reserved, must be kept at reset value.													
Bit 12	SPI1EN: SPI1 clock enable Set and cleared by software 0: SPI1 clock disabled 1: SPI1 clock enabled													
Bit 11	TIM1EN: TIM1 timer clock enable Set and cleared by software. 0: TIM1 timer clock disabled 1: TIM1 timer clock enabled													
Bit 10	ADC2EN: ADC 2 interface clock enable Set and cleared by software. 0: ADC 2 interface clock disabled 1: ADC 2 interface clock enabled													
Bit 9	ADC1EN: ADC 1 interface clock enable Set and cleared by software. 0: ADC 1 interface disabled 1: ADC 1 interface clock enabled													
Bits 8: 7	Reserved, must be kept at reset value.													
Bit 6	IOPEEN: IO port E clock enable Set and cleared by software. 0: IO port E clock disabled 1: IO port E clock enabled													
Bit 5	IOPDEN: IO port D clock enable Set and cleared by software. 0: IO port D clock disabled 1: IO port D clock enabled													
Bit 4	IOPCEN: IO port C clock enable Set and cleared by software. 0: IO port C clock disabled 1: IO port C clock enabled													
Bit 3	IOPBEN: IO port B clock enable Set and cleared by software. 0: IO port B clock disabled 1: IO port B clock enabled													
Bit 2	IOPAEN: IO port A clock enable Set and cleared by software. 0: IO port A clock disabled 1: IO port A clock enabled													
Bit 1	Reserved, must be kept at reset value.													
Bit 0	AFIOEN: Alternate function IO clock enable Set and cleared by software. 0: Alternate Function IO clock disabled 1: Alternate Function IO clock enabled													

6.3.8 APB1 peripheral clock enable register (RCC_APB1ENR)

Address: 0x1C

Reset value: 0x0000 0000

Access: word, half-word and byte access

No wait state, except if the access occurs while an access to a peripheral on APB1 domain is on going. In this case, wait states are inserted until this access to APB1 peripheral is finished.

Note: When the peripheral clock is not active, the peripheral register values may not be readable by software and the returned value is always 0x0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Reserved			PWR EN	BKP EN	Res	CAN EN	Res	USB EN	I2C2 EN	I2C1 EN	Reserved		UART3 EN	UART2 EN	Res	
15	14	13	rw 12	rw 11	10	rw 9	8	rw 7	rw 6	rw 5	4	3	rw 2	rw 1	0	
Res	SPI2 EN	Reserved		WWDG EN	Reserved								TIM4 EN	TIM3 EN	TIM2 EN	
rw			rw			rw			rw			rw			rw	rw

Bits 31:29	Reserved, must be kept at reset value.
Bit 28	PWREN: Power interface clock enable Set and cleared by software. 0: Power interface clock disabled 1: Power interface clock enabled
Bit 27	BKPEN: Backup interface clock enable Set and cleared by software. 0: Backup interface clock disabled 1: Backup interface clock enabled
Bit 26	Reserved, must be kept at reset value.
Bit 25	CANEN: CAN clock enable Set and cleared by software. 0: CAN clock disabled 1: CAN clock enabled
Bit 24	Reserved, always read as 0.
Bit 23	USBEN: USB clock enable Set and cleared by software. 0: USB clock disabled 1: USB clock enabled
Bit 22	I2C2EN: I2C2 clock enable Set and cleared by software. 0: I2C2 clock disabled 1: I2C2 clock enabled
Bit 21	I2C1EN: I2C1 clock enable Set and cleared by software. 0: I2C1 clock disabled 1: I2C1 clock enabled
Bits 20: 19	Reserved, must be kept at reset value.
Bit 18	USART3EN: USART3 clock enable Set and cleared by software. 0: USART3 clock disabled 1: USART3 clock enabled
Bit 17	USART2EN: USART2 clock enable Set and cleared by software. 0: USART2 clock disabled 1: USART2 clock enabled
Bits 16: 15	Reserved, always read as 0.
Bit 14	SPI2EN: SPI2 clock enable Set and cleared by software. 0: SPI2 clock disabled 1: SPI2 clock enabled
Bits 13:12	Reserved, always read as 0.
Bit 11	WWDGEN: Window watchdog clock enable Set and cleared by software. 0: Window watchdog clock disabled 1: Window watchdog clock enabled
Bits 10:3	Reserved, must be kept at reset value
Bit 2	TIM4EN: TIM4 timer clock enable Set and cleared by software. 0: TIM4 clock disabled 1: TIM4 clock enabled
Bit 1	TIM3EN: TIM3 timer clock enable Set and cleared by software. 0: TIM3 clock disabled 1: TIM3 clock enabled
Bit 0	TIM2EN: TIM2 timer clock enable Set and cleared by software. 0: TIM2 clock disabled

	1: TIM2 clock enabled
--	-----------------------

6.3.9 Backup domain control register (RCC_BDCR)

Address offset: 0x20

Reset value: 0x0000 0000, reset by Backup domain Reset

Access: $0 \leq \text{wait state} \leq 3$, word, half-word and byte access

Wait states are inserted in case of successive accesses to this register.

Note: The LSEON, LSEBYP, RTCSEL and RTCEN bits of the Backup domain control register (RCC_BDCR) are in the Backup domain. As a result, after Reset, these bits are write-protected and the DBP bit in the Power control register (PWR_CR) has to be set before these can be modified. Refer to Section 5.1: Power control (PWR) for further information. These bits are only reset after a Backup domain Reset (see Section 6.1.3: Backup domain reset). Any internal or external Reset will not have any effect on these bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
Reserved															BDR ST	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	rw
RTC EN	Reserved					RTCSEL[1:0]		Reserved					LSE BYP	LSE RDY	LSE ON	rw
rw																rw
Bits 31:17		Reserved, must be kept at reset value														rw
Bit 16		BDRST: Backup domain software reset Set and cleared by software. 0: Reset not activated 1: Resets the entire Backup domain														rw
Bit 15		RTCEN: RTC clock enable Set and cleared by software. 0: RTC clock disabled 1: RTC clock enabled														rw
Bits 14:10		Reserved, must be kept at reset value														rw
Bits 9:8		RTCSEL[1:0]: RTC clock source selection Set by software to select the clock source for the RTC. Once the RTC clock source has been selected, it cannot be changed anymore unless the Backup domain is reset. The BDRST bit can be used to reset them. 00: No clock 01: LSE oscillator clock used as RTC clock 10: LSI oscillator clock used as RTC clock 11: HSE oscillator clock divided by 128 used as RTC clock														rw
Bits 7:3		Reserved, must be kept at reset value														rw
Bit 2		LSEBYP: External low-speed oscillator bypass Set and cleared by software to bypass oscillator in debug mode. This bit can be written only when the external 32 kHz oscillator is disabled. 0: LSE oscillator not bypassed 1: LSE oscillator bypassed														rw
Bit 1		LSERDY: External low-speed oscillator ready Set and cleared by hardware to indicate when the external 32 kHz oscillator is stable. After the LSEON bit is cleared, LSERDY goes low after 6 external low-speed oscillator clock cycles. 0: External 32 kHz oscillator not ready 1: External 32 kHz oscillator ready														rw
Bit 0		LSEON: External low-speed oscillator enable Set and cleared by software. 0: External 32 kHz oscillator OFF 1: External 32 kHz oscillator ON														rw

6.3.10 Control/status register (RCC_CSR)

Address: 0x24

Reset value: 0x0C00 0000, reset by system Reset, except reset flags by power Reset only.

Access: $0 \leq \text{wait state} \leq 3$, word, half-word and byte access

Wait states are inserted in case of successive accesses to this register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPWR RSTF	WWDG RSTF	IWDG RSTF	SFT RSTF	POR RSTF	PIN RSTF	Res	PMVF	Reserved							
rw	rw	rw	rw	rw	rw		rw								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														LSI RDY	LSI ON
													r		rw

Bit 31	LPWRRSTF: Low-power reset flag Set by hardware when a Low-power management reset occurs. Cleared by writing to the RMVF bit. 0: No Low-power management reset occurred 1: Low-power management reset occurred For further information on Low-power management reset, refer to <i>Low-power management reset</i> .
Bit 30	WWDGRSTF: Window watchdog reset flag Set by hardware when a window watchdog reset occurs. Cleared by writing to the RMVF bit. 0: No window watchdog reset occurred 1: Window watchdog reset occurred
Bit 29	IWDGRSTF: Independent watchdog reset flag Set by hardware when an independent watchdog reset from V _{DD} domain occurs. Cleared by writing to the RMVF bit. 0: No watchdog reset occurred 1: Watchdog reset occurred
Bit 28	SFTRSTF: Software reset flag Set by hardware when a software reset occurs. Cleared by writing to the RMVF bit. 0: No software reset occurred 1: Software reset occurred
Bit 27	PORRSTF: POR/PDR reset flag Set by hardware when a POR/PDR reset occurs. Cleared by writing to the RMVF bit. 0: No POR/PDR reset occurred 1: POR/PDR reset occurred
Bit 26	PINRSTF: PIN reset flag Set by hardware when a reset from the NRST pin occurs. Cleared by writing to the RMVF bit. 0: No reset from NRST pin occurred 1: Reset from NRST pin occurred
Bit 25	Reserved, must be kept at reset value.
Bit 24	RMVF: Remove reset flag Set by software to clear the reset flags. 0: No effect 1: Clear the reset flags
Bits 23:2	Reserved, must be kept at reset value.
Bit 1	LSIRDY: Internal low-speed oscillator ready Set and cleared by hardware to indicate when the internal RC 40 kHz oscillator is stable. After the LSION bit is cleared, LSIRDY goes low after 3 internal RC 40 kHz oscillator clock cycles. 0: Internal RC 40 kHz oscillator not ready 1: Internal RC 40 kHz oscillator ready
Bit 0	LSION: Internal low-speed oscillator enable Set and cleared by software. 0: Internal RC 40 kHz oscillator OFF 1: Internal RC 40 kHz oscillator ON

6.3.11 RCC register map

The following table gives the RCC register map and the reset values.

Table 12 RCC register map and reset values

010h		00Ch		008h		004h		000h		Offset							
Reset value	RCC_APB1RST	Reset value	RCC_APB2RSTR	Reset value	RCC_CIR	Reset value	RCC_CFGR	Reset value	RCC_CR	Register							
Reserved		Reserved						Reserved		31							
															30		
	0															29	
PWRRST		Reserved						Reserved			28						
BKPRST																27	
																	26
Reserved		Reserved						Reserved			25						
																	24
0																	23
CANRST		Reserved						Reserved			22						
																	21
0																	20
Reserved		Reserved						Reserved			19						
																	18
0																	17
UART3RST		Reserved						Reserved			16						
UART2RST																	15
0																	14
Reserved		Reserved						Reserved			13						
																	12
0																	11
WWDGRST		Reserved						Reserved			10						
																	9
0																	8
Reserved		Reserved						Reserved			7						
																	6
	0																5
Reserved		Reserved						Reserved			4						
																	3
	0																2
TIM4RST		Reserved						Reserved			1						
TIM3RST																	0
0																	

024h		01Ch		020h		018h		014h							
Reset value	RCC_CSR	Reset value	RCC_BDCR	Reset value	RCC_APB1ENR	Reset value	RCC_APB2ENR	Reset value	RCC_AHBENR						
0	LPWRRSTF	Reserved				Reserved	Reserved	Reserved	Reserved						
0	WWDGRSTF														
0	IWDGRSTF														
0	SFTRSTF														
1	PORRSTF														
1	PINRSTF														
	Reserved														
0	RMVVF														
Reserved						BDRST				0	Reserved	0	USART1EN	0	CRGEN
						RTCEN				0	Reserved	0	SPI1EN	Reserved	
		Reserved		Reserved		0	TIM1EN	1	FLITFEN						
						0	ADC2EN	Reserved							
		RTC SEL [1:0]		Reserved		0	ADC1EN	1	SRAMEN						
						Reserved		0	IOPBEN	Reserved					
		Reserved		Reserved		0	IOPAEN	1	Reserved						
						Reserved		0	IOPEEN	Reserved					
						Reserved		0	IOPDEN	Reserved					
		Reserved		Reserved		Reserved		0	IOPCEN	1	FLITFEN				
Reserved								0	IOPBEN	Reserved					
Reserved								0	IOPAEN	Reserved					
0	LSIRDY	0	LSEBYP	0	TIM4EN	0	Reserved	0	Reserved						
0	LSION	0	LSERDYF	0	TIM3EN	0	APIOEN	0	DMAN						

Refer to Table 1 for the register boundary addresses.

7 General-purpose and alternate-function I/Os (GPIOs and AFIOs)

7.1 GPIO functional description

Each of the general-purpose I/O ports has two 32-bit configuration registers (GPIOx_CRL, GPIOx_CRH), two 32-bit data registers (GPIOx_IDR, GPIOx_ODR), a 32-bit set/reset register (GPIOx_BSRR), a 16-bit reset register (GPIOx_BRR) and a 32-bit locking register (GPIOx_LCKR).

Subject to the specific hardware characteristics of each I/O port listed in the *datasheet*, each port bit of the General Purpose IO (GPIO) Ports, can be individually configured by software in several modes:

- Input floating
- Input pull-up
- Input-pull-down
- Analog
- Output open-drain
- Output push-pull

Each I/O port bit is freely programmable, however the I/O port registers have to be accessed as 32-bit words (half-word or byte accesses are not allowed). The purpose of the GPIOx_BSRR and GPIOx_BRR registers is to allow atomic read/modify accesses to any of the GPIO registers. This way, there is no risk that an IRQ occurs between the read and the modify access.

Figure 9 shows the basic structure of an I/O Port bit.

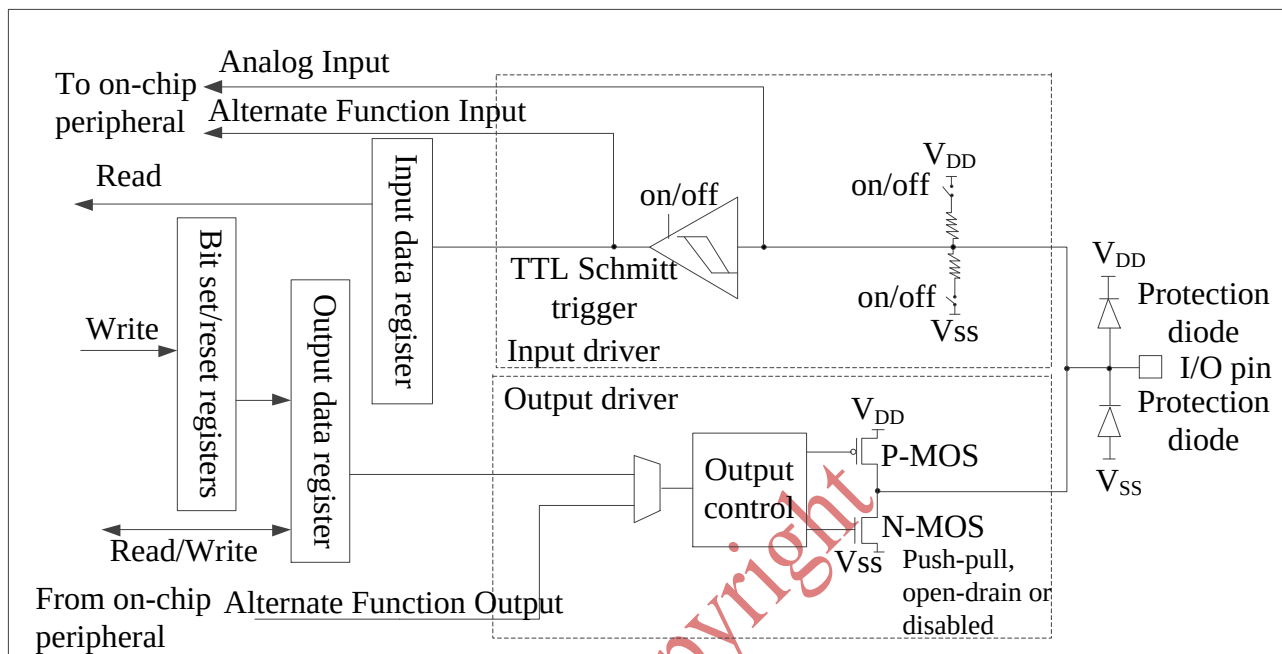


Figure 9 Basic structure of a standard I/O port bit

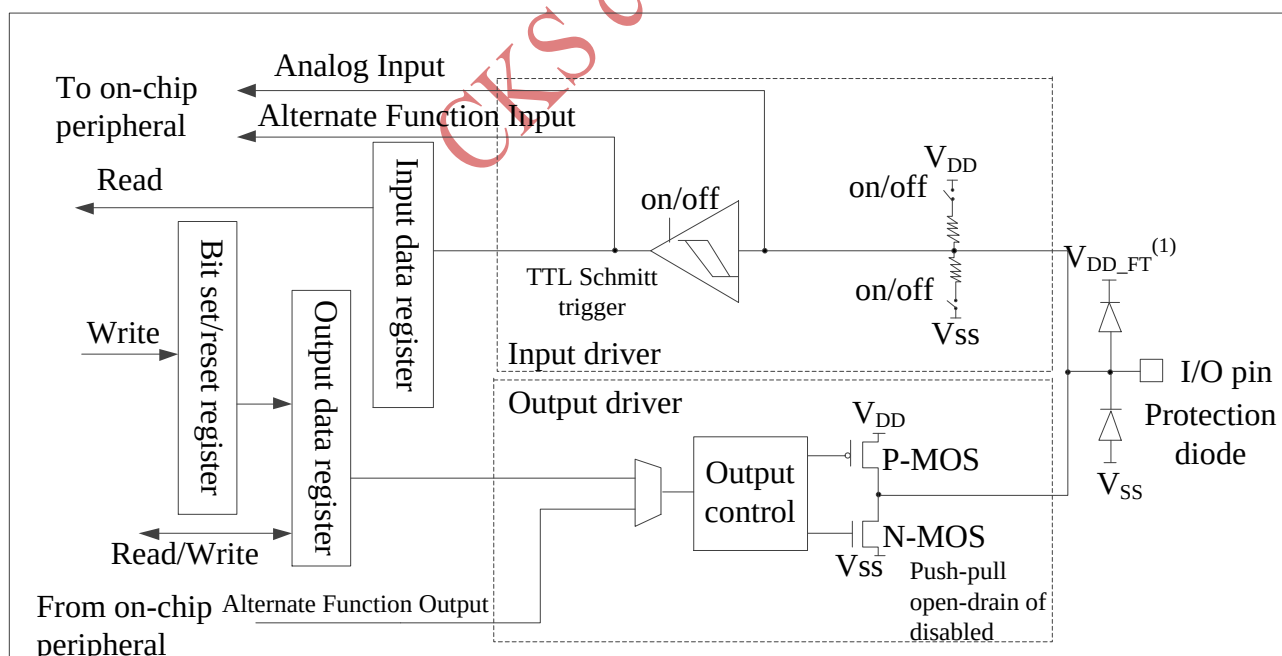


Figure 10 Basic structure of a 5-Volt tolerant I/O port bit

(1) VDD_FT is a potential specific to 5-Volt tolerant I/Os, and different from VDD.

Table 13 Port bit configuration table

Configuration mode		CNF1	CNF0	MODE1	MODE0	PxODR register
General	Push-Pull	0	0	01		0 or 1

purpose output	Open-Drain		1	10 11 see 错误!未找到引用源。	0 or 1
Alternate Function output	Push-Pull	1	0		Don't care
	Open-Drain		1		Don't care
Input	Analog	0	0	00	Don't care
	Input floating		1		Don't care
	Input pull-down	1	0		0
	Input pull-up				1

Table 14 Output MODE bits

MODE[1:0]	Meaning
00	Reserved
01	Maximum output speed 10 MHz
10	Maximum output speed 2 MHz
11	Maximum output speed 50 MHz

7.1.1 General-purpose I/O (GPIO)

During and just after reset, the alternate functions are not active and the I/O ports are configured in Input Floating mode (CNF_x[1:0]=01b, MODEx[1:0]=00b).

The JTAG pins are in input PU/PD after reset:

- PA15: JTDI in PU
- PA14: JTCK in PD
- PA13: JTMS in PU
- PB4: NJTRST in PU

When configured as output, the value written to the Output Data register (GPIO_x_ODR) is output on the I/O pin. It is possible to use the output driver in Push-Pull mode or Open-Drain mode (only the N-MOS is activated when outputting 0).

The Input Data register (GPIO_x_IDR) captures the data present on the I/O pin at every APB2 clock cycle. All GPIO pins have an internal weak pull-up and weak pull-down that can be activated or not when configured as input.

7.1.2 Atomic bit set or reset

There is no need for the software to disable interrupts when programming the GPIO_x_ODR at bit level: it is possible to modify only one or several bits in a single atomic APB2 write access.

This is achieved by programming to '1' the Bit Set/Reset Register (GPIO_x_BSRR, or for reset only GPIO_x_BRR) to select the bits to modify. The unselected bits will not be modified.

7.1.3 External interrupt/wakeup lines

All ports have external interrupt capability. To use external interrupt lines, the port must be configured in input mode. For more information on external interrupts, refer to:

- Section 8.2: External interrupt/event controller (EXTI)
- Section 8.2.3: Wakeup event management.

7.1.4 Alternate functions (AF)

It is necessary to program the Port Bit Configuration Register before using a default alternate function.

- For alternate function inputs, the port must be configured in Input mode (floating, pull-up or pull-down) and the input pin must be driven externally.

Note: It is also possible to emulate the AFI input pin by software by programming the GPIO controller. In this case, the port should be configured in Alternate Function Output mode. And obviously, the corresponding port should not be driven externally as it will be driven by the software using the GPIO controller.

- For alternate function outputs, the port must be configured in Alternate Function Output mode (Push-Pull or Open-Drain).

● For bidirectional Alternate Functions, the port bit must be configured in Alternate Function Output mode (Push-Pull or Open-Drain). In this case the input driver is configured in input floating mode. If a port bit is configured as Alternate Function Output, this disconnects the output register and connects the pin to the output signal of an on-chip peripheral.

If software configures a GPIO pin as Alternate Function Output, but peripheral is not activated, its output is not specified.

7.1.5 Software remapping of I/O alternate functions

To optimize the number of peripheral I/O functions for different device packages, it is possible to remap some alternate functions to some other pins. This is achieved by software, by programming the corresponding registers (refer to *AFIO registers*). In that case, the alternate functions are no longer mapped to their original assignments.

7.1.6 GPIO locking mechanism

The locking mechanism allows the IO configuration to be frozen. When the LOCK sequence has been applied on a port bit, it is no longer possible to modify the value of the port bit until the next reset.

7.1.7 Input configuration

When the I/O Port is programmed as Input:

- The Output Buffer is disabled
- The Schmitt Trigger Input is activated
- The weak pull-up and pull-down resistors are activated or not depending on input configuration (pull-up, pull-down or floating):
- The data present on the I/O pin is sampled into the Input Data Register every APB2 clock cycle
- A read access to the Input Data Register obtains the I/O State.

Figure 11 shows the Input Configuration of the I/O Port bit.

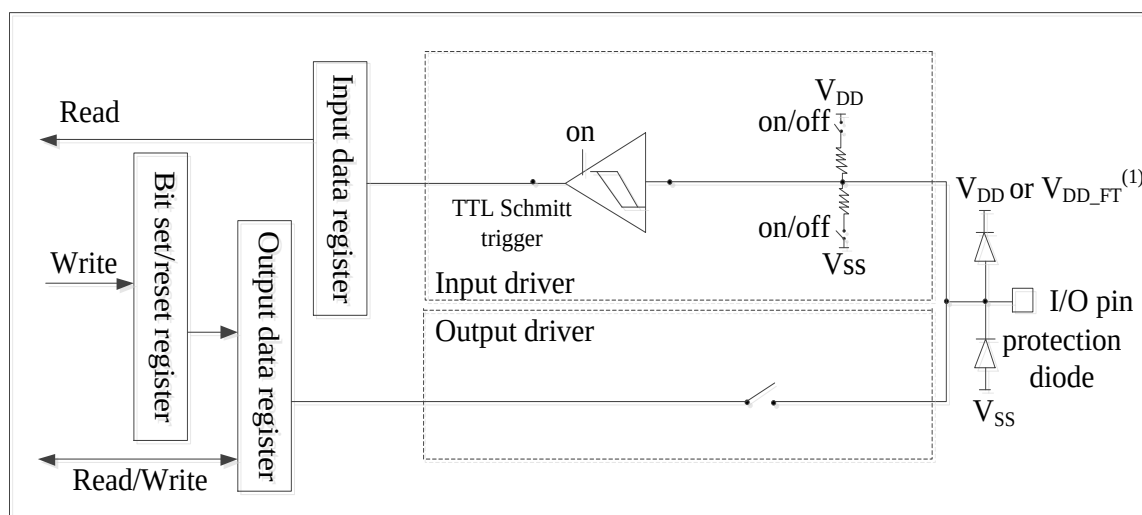


Figure 11 Input floating/pull up/pull down configurations

(1) V_{DD_FT} is a potential specific to 5-Volt tolerant I/Os, and different from V_{DD}

7.1.8 Output configuration

When the I/O Port is programmed as Output:

- The Output Buffer is enabled:
 - Open Drain Mode: A “0” in the Output register activates the N-MOS while a “1” in the Output register leaves the port in Hi-Z (the P-MOS is never activated)
 - Push-Pull Mode: A “0” in the Output register activates the N-MOS while a “1” in the Output register activates the P-MOS
- The Schmitt Trigger Input is activated.
- The weak pull-up and pull-down resistors are disabled.
- The data present on the I/O pin is sampled into the Input Data Register every APB2 clock cycle
- A read access to the Input Data Register gets the I/O state in open drain mode
- A read access to the Output Data register gets the last written value in Push-Pull mode

Figure 12 shows the Output configuration of the I/O Port bit.

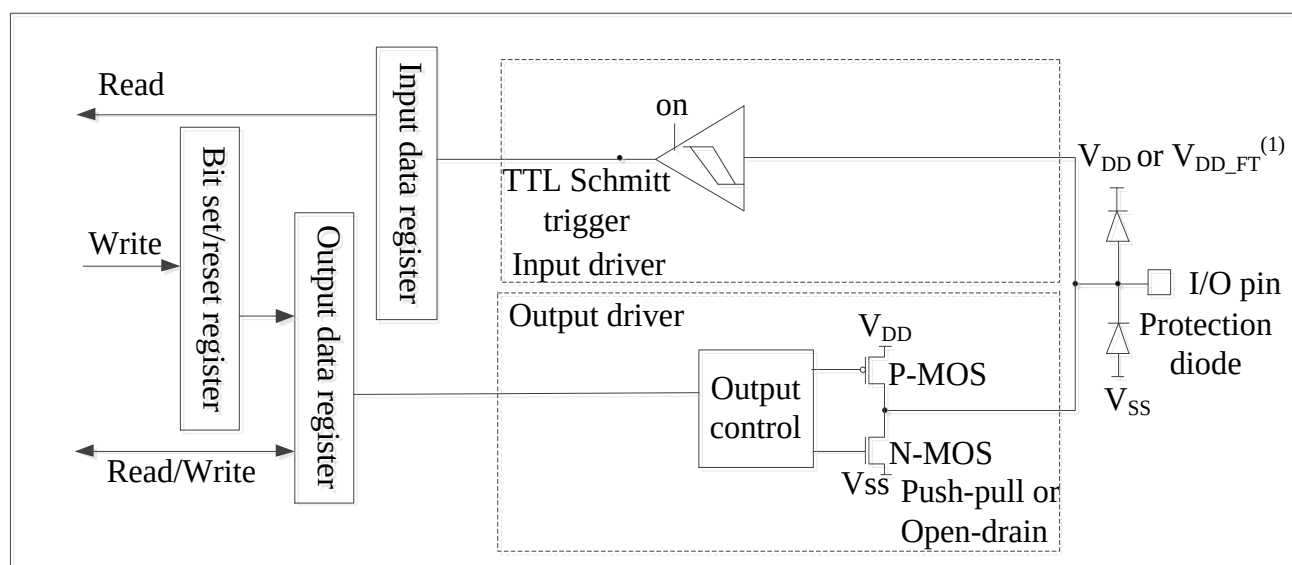


Figure 12 Output configuration

(1) V_{DD_FT} is a potential specific to 5-Volt tolerant I/Os, and different from V_{DD}

7.1.9 Alternate function configuration

When the I/O Port is programmed as Alternate Function:

- The Output Buffer is turned on in Open Drain or Push-Pull configuration
- The Output Buffer is driven by the signal coming from the peripheral (alternate function out)
- The Schmitt Trigger Input is activated
- The weak pull-up and pull-down resistors are disabled.
- The data present on the I/O pin is sampled into the Input Data Register every APB2 clock cycle
- A read access to the Input Data Register gets the I/O state in open drain mode
- A read access to the Output Data register gets the last written value in Push-Pull mode

Figure 13 shows the Alternate Function Configuration of the I/O Port bit. Also, refer to *Section 7.4: AFIO registers* for further information.

A set of Alternate Function I/O registers allows the user to remap some alternate functions to different pins. Refer to *Section 7.3: Alternate function I/O and debug configuration (AFIO)*.

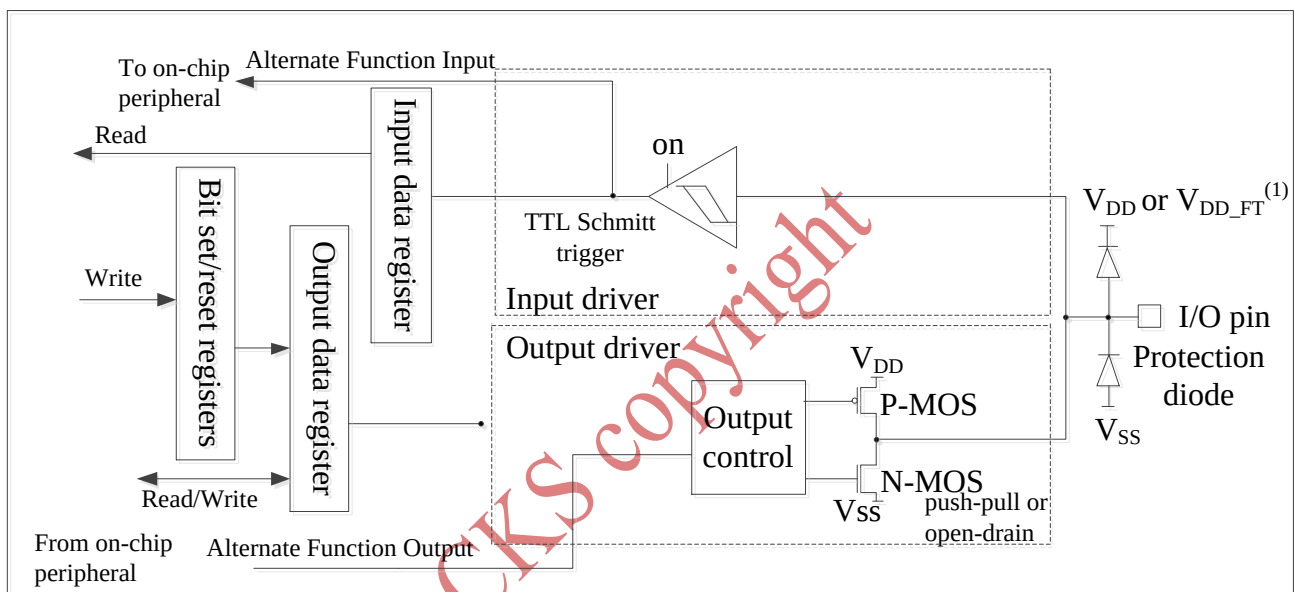


Figure 13 Alternate function configuration

(1) V_{DD_FT} is a potential specific to 5-Volt tolerant I/Os, and different from V_{DD} .

7.1.10 Analog configuration

When the I/O Port is programmed as Analog configuration:

- The Output Buffer is disabled.
- The Schmitt Trigger Input is de-activated providing zero consumption for every analog value of the I/O pin. The output of the Schmitt Trigger is forced to a constant value (0).
- The weak pull-up and pull-down resistors are disabled.
- Read access to the Input Data Register gets the value "0".

Figure 14 shows the high impedance-analog configuration of the I/O Port bit.

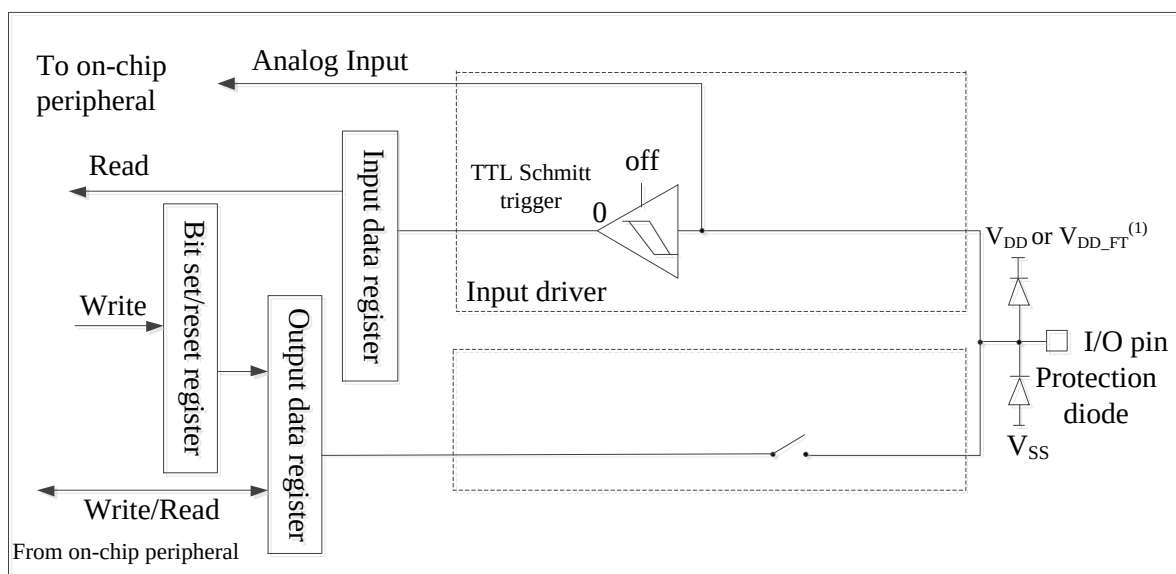


Figure 14 High impedance-analog configuration

(1) V_{DD_FT} is a potential specific to 5-Volt tolerant I/Os, and different from V_{DD} .

7.1.11 GPIO configurations for device peripherals

Table 15 to Table 23 give the GPIO configurations of the device peripherals.

Table 15 Advanced timers TIM1

TIM1 pinout	Configuration	GPIO configuration
TIM1_CHx	Input capture channel x	Input floating
	Output compare channel x	Alternate function push-pull
TIM1_CHxN	Complementary output channel x	Alternate function push-pull
TIM1_BKIN	Break input	Input floating
TIM1_ETR	External trigger timer input	Input floating

Table 16 General-purpose timers TIM2/3/4

TIM2/3/4 pinout	Configuration	GPIO configuration
TIM2/3/4_CHx	Input capture channel x	Input floating
	Output compare channel x	Alternate function push-pull
TIM2/3/4_ETR	External trigger timer input	Input floating

Table 17 USARTs

USART pinout	Configuration	GPIO configuration
USARTx_TX	Full duplex	Alternate function push-pull
	Half duplex synchronous mode	Alternate function push-pull
USARTx_RX	Full duplex	Input floating / Input pull-up
	Half duplex synchronous mode	Not used. Can be used as a general IO
USARTx_CK	Synchronous mode	Alternate function push-pull
USARTx_RTS	Hardware flow control	Alternate function push-pull
USARTx_CTS	Hardware flow control	Input floating/ Input pull-up

Table 18 SPI

SPI pinout	Configuration	GPIO configuration
SPIx_SCK	Master	Alternate function push-pull
	Slave	Input floating

SPIx_MOSI	Full duplex / master	Alternate function push-pull
	Full duplex / slave	Input floating / Input pull-up
	Simplex bidirectional data wire / master	Alternate function push-pull
	Simplex bidirectional data wire/ slave	Not used. Can be used as a GPIO
SPIx_MISO	Full duplex / master	Input floating / Input pull-up
	Full duplex / slave (point to point)	Alternate function push-pull
	Full duplex / slave (multi-slave)	Alternate function open drain
	Simplex bidirectional data wire / master	Not used. Can be used as a GPIO
SPIx_NSS	Hardware master /slave	Input floating/ Input pull-up / Input pull-down
	Hardware master/ NSS output enabled	Alternate function push-pull
	Software	Not used. Can be used as a GPIO

Table 19 I2C

I2C pinout	Configuration	GPIO configuration
I2Cx_SCL	I2C clock	Alternate function open drain
I2Cx_SDA	I2C Data I/O	Alternate function open drain

Table 20 bxCAN

BxCAN pinout	GPIO configuration
CAN_TX	Alternate function push-pull
CAN_RX	Input floating / Input pull-up

Table 21 USB

USB pinout	GPIO configuration
USB_DM / USB_DP	As soon as the USB is enabled, these pins are automatically connected to the USB internal transceiver.

The GPIO configuration of the ADC inputs should be analog.

Table 22 ADC

ADC pin	GPIO configuration
ADC	Analog

Table 23 Other IOs

Pins	Alternate function	GPIO configuration
TAMPER-RTC	RTC output	Forced by hardware when configuring the BKP_CR and BKP_RTCCR registers
	Tamper event input	
MCO	Clock output	Alternate function push-pull
EXTI input lines	External input interrupts	Input floating / input pull-up / input pull-down

7.2 GPIO registers

Refer to *Section 1* for a list of abbreviations used in register descriptions.
The peripheral registers have to be accessed by words (32-bit).

7.2.1 Port configuration register low (GPIOx_CRL) (x=A..E)

Address offset: 0x00

Reset value: 0x4444 4444

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16

CNF7[1:0]	MODE7[1:0]	CNF6[1:0]	MODE6[1:0]	CNF5[1:0]	MODE5[1:0]	CNF4[1:0]	MODE4[1:0]
rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw
15 14	13 12	11 10	9 8	7 6	5 4	3 2	1 0
CNF3[1:0]	MODE3[1:0]	CNF2[1:0]	MODE2[1:0]	CNF1[1:0]	MODE1[1:0]	CNF0[1:0]	MODE0[1:0]
rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw
Bits 31: 30, 27:26 , 23:22, 19:18, 15:14, 11:10, 7:6, 3:2		CNFy[1:0]: Port x configuration bits (y= 0 .. 7) These bits are written by software to configure the corresponding I/O port. Refer to Table 13 In input mode (MODE[1:0]=00): 00: Analog mode 01: Floating input (reset state) 10: Input with pull-up / pull-down 11: Reserved In output mode (MODE[1:0] > 00): 00: General purpose output push-pull 01: General purpose output Open-drain 10: Alternate function output Push-pull 11: Alternate function output Open-drain					
Bits 29:28, 25:24, 21:20, 17:16, 13:12, 9:8, 5:4, 1:0		MODEy[1:0]: Port x mode bits (y= 0 .. 7) These bits are written by software to configure the corresponding I/O port. Refer to Table 13 00: Input mode (reset state) 01: Output mode, max speed 10 MHz. 10: Output mode, max speed 2 MHz 11: Output mode, max speed 50 MHz.					

7.2.2 Port configuration register high (GPIOx_CRH) (x=A..E)

Address offset: 0x04

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]	MODE15[1:0]	CNF14[1:0]	MODE14[1:0]	CNF13[1:0]	MODE13[1:0]	CNF12[1:0]	MODE12[1:0]	CNF11[1:0]	MODE11[1:0]	CNF10[1:0]	MODE10[1:0]	CNF9[1:0]	MODE9[1:0]	CNF8[1:0]	MODE8[1:0]
rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw
15 14	13 12	11 10	9 8	7 6	5 4	3 2	1 0	CNF11[1:0]	MODE11[1:0]	CNF10[1:0]	MODE10[1:0]	CNF9[1:0]	MODE9[1:0]	CNF8[1:0]	MODE8[1:0]
rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw
Bits 31: 30, 27:26 , 23:22, 19:18, 15:14, 11:10, 7:6, 3:2		CNFy[1:0]: Port x configuration bits (y= 8 .. 15) These bits are written by software to configure the corresponding I/O port. Refer to Table 13 In input mode (MODE[1:0]=00): 00: Analog mode 01: Floating input (reset state) 10: Input with pull-up / pull-down 11: Reserved In output mode (MODE[1:0] > 00): 00: General purpose output push-pull 01: General purpose output Open-drain 10: Alternate function output Push-pull 11: Alternate function output Open-drain													
Bits 29:28, 25:24, 21:20, 17:16, 13:12, 9:8, 5:4, 1:0		MODEy[1:0]: Port x mode bits (y= 8 .. 15) These bits are written by software to configure the corresponding I/O port. Refer to Table 13 00: Input mode (reset state) 01: Output mode, max speed 10 MHz 10: Output mode, max speed 2 MHz. 11: Output mode, max speed 50 MHz.													

7.2.3 Port input data register (GPIOx_IDR) (x=A..E)

Address offset: 0x08

Reset value: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR	IDR	IDR	IDR	IDR	IDR	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
15	14	13	12	11	10										
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 31: 16		Reserved, must be kept at reset value.													
Bits 15:0		IDRy : Port input data (y= 0 .. 15) These bits are read only and can be accessed in Word mode only. They contain the input value of the corresponding I/O port.													

7.2.4 Port output data register (GPIOx_ODR) (x=A..E)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR	ODR	ODR	ODR	ODR	ODR	ODR	ODR	ODR	ODR	ODR	ODR	ODR	ODR	ODR	ODR
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31: 16		Reserved, must be kept at reset value.													
Bits 15:0		ODRy : Port output data (y= 0 .. 15) These bits can be read and written by software and can be accessed in Word mode only. <i>Note: For atomic bit set/reset, the ODR bits can be individually set and cleared by writing to the GPIOx_BSRR register (x = A .. E).</i>													

7.2.5 Port bit set/reset register (GPIOx_BSRR) (x=A..E)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
Bits 31: 16		BRy : Port x Reset bit y (y= 0 .. 15) These bits are write-only and can be accessed in Word mode only. 0: No action on the corresponding ODRx bit 1: Reset the corresponding ODRx bit <i>Note: If both BSx and BRx are set, BSx has priority.</i>													
Bits 15:0		BSy : Port x Set bit y (y= 0 .. 15) These bits are write-only and can be accessed in Word mode only. 0: No action on the corresponding ODRx bit 1: Set the corresponding ODRx bit													

7.2.6 Port bit reset register (GPIOx_BRR) (x=A..E)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
Bits 31: 16				Reserved											
Bits 15:0				BRy: Port x Reset bit y (y= 0 .. 15) These bits are write-only and can be accessed in Word mode only. 0: No action on the corresponding ODRx bit 1: Reset the corresponding ODRx bit											

7.2.7 Port configuration lock register (GPIOx_LCKR) (x=A..E)

This register is used to lock the configuration of the port bits when a correct write sequence is applied to bit 16 (LCKK). The value of bits [15:0] is used to lock the configuration of the GPIO. During the write sequence, the value of LCKR[15:0] must not change. When the LOCK sequence has been applied on a port bit it is no longer possible to modify the value of the port bit until the next reset. Each lock bit freezes the corresponding 4 bits of the control register (CRL, CRH).

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															LCKK
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LCK	LCK	LCK	LCK	LCK	LCK	LCK	LCK	LCK	LCK	LCK	LCK	LCK	LCK	LCK	LCK
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31: 17				Reserved											
Bit 16				LCKK[16]: Lock key This bit can be read anytime. It can only be modified using the Lock Key Writing Sequence. 0: Port configuration lock key not active 1: Port configuration lock key active. GPIOx_LCKR register is locked until the next reset. LOCK key writing sequence: Write 1 Write 0 Write 1 Read 0 Read 1 (this read is optional but confirms that the lock is active) <i>Note: During the LOCK Key Writing sequence, the value of LCK[15:0] must not change.</i> Any error in the lock sequence will abort the lock.											
Bits 15:0				LCKy: Port x Lock bit y (y= 0 .. 15) These bits are read write but can only be written when the LCKK bit is 0. 0: Port configuration not locked 1: Port configuration locked.											

7.3 Alternate function I/O and debug configuration (AFIO)

To optimize the number of peripherals available for the 64-pin or the 100-pin package, it is possible to remap some alternate functions to some other pins. This is achieved by software, by programming the *AF remap and debug I/O configuration register (AFIO_MAPR)*. In this case, the alternate functions are no longer mapped to their original assignments.

7.3.1 Using OSC32_IN/OSC32_OUT pins as GPIO ports PC14/PC15

The LSE oscillator pins OSC32_IN and OSC32_OUT can be used as general-purpose I/O PC14 and PC15, respectively, when the LSE oscillator is off. The LSE has priority over the GP I/Os function.

Note: The PC14/PC15 GPIO functionality is lost when the 1.5 V domain is powered off (by entering standby mode) or when the backup domain is supplied by V_{BAT} (V_{DD} no more supplied). In this case the I/Os are set in analog mode.
Refer to the note on I/O usage restrictions in Section 4.1.2: Battery backup domain.

7.3.2 Using OSC_IN/OSC_OUT pins as GPIO ports PD0/PD1

The HSE oscillator pins OSC_IN/OSC_OUT can be used as general-purpose I/O PD0/PD1 by programming the PD01_REMAP bit in the AF remap and debug I/O configuration register (AFIO_MAPR).

This remap is available only on 36-, 48- and 64-pin packages (PD0 and PD1 are available on 100-pin packages, no need for remapping).

Note: The external interrupt/event function is not remapped. PD0 and PD1 cannot be used for external interrupt/event generation on 36-, 48- and 64-pin packages.

7.3.3 CAN alternate function remapping

The CAN signals can be mapped on Port A, Port B or Port D as shown in Table 24. For port D, remapping is not possible in devices delivered in 36-, 48- and 64-pin packages.

Table 24 CAN alternate function remapping

Alternate function	CAN_REMAP[1:0]="00"	CAN_REMAP[1:0]="10" ⁽¹⁾	CAN_REMAP[1:0]="11" ⁽²⁾
CAN_RX	PA11	PB8	PD0
CAN_TX	PA12	PB9	PD1

1. Remap not available on 36-pin package

2. This remapping is available only on 100-pin packages, when PD0 and PD1 are not remapped on OSC-IN and OSC-OUT.

7.3.4 JTAG/SWD alternate function remapping

The debug interface signals are mapped on the GPIO ports as shown in Table 25.

Table 25 Debug interface signals

Alternate function	GPIO port
JTMS / SWDIO	PA13
JTCK / SWCLK	PA14
JTDI	PA15
JTDO / TRACESWO	PB3
NJTRST	PB4
TRACECK	PE2
TRACED0	PE3

TRACED1	PE4
TRACED2	PE5
TRACED3	PE6

To optimize the number of free GPIOs during debugging, this mapping can be configured in different ways by programming the SWJ_CFG[2:0] bits in the *AF remap and debug I/O configuration register (AFIO_MAPR)*. Refer to Table 26.

Table 26 Debug port mapping

SWJ_CFG [2:0]	Available debug ports	SWJ I/O pin assigned				
		PA13/ JTMS/ SWDIO	PA14/ JTCK/ SWCLK	PA15/ JTDI	PB3/ JTDO/ TRACESWO	PB4/ NJTRST
000	Full SWJ (JTAG-DP + SW-DP) (Reset state)	X	X	X	X	X
001	Full SWJ (JTAG-DP + SW-DP) but without NJTRST	X	X	X	x	Free
010	JTAG-DP Disabled and SW-DP Enabled	X	X	Free	Free ⁽¹⁾	Free
100	JTAG-DP Disabled and SW-DP Disabled	Free	Free	Free	Free	Free
Other	Forbidden	-	-	-	-	-

1. Released only if not using asynchronous trace

7.3.5 Timer alternate function remapping

Timer 4 channels 1 to 4 can be remapped from Port B to Port D. Other timer remapping possibilities are listed in Table 28 to Table 30. Refer to *AF remap and debug I/O configuration register (AFIO_MAPR)*.

Table 27 TIM4 alternate function remapping

Alternate function	TIM4_REMAP = 0	TIM4_REMAP = 1 ⁽¹⁾
TIM4_CH1	PB6	PD12
TIM4_CH2	PB7	PD13
TIM4_CH3	PB8	PD14
TIM4_CH4	PB9	PD15

1. Remap available only for 100-pin and for 144-pin package.

Table 28 TIM3 alternate function remapping

Alternate function	TIM3_REMAP[1:0] = 00 (no remap)	TIM3_REMAP[1:0] = 10 (partial remap)	TIM3_REMAP[1:0] = 11 (full remap) ⁽¹⁾
TIM3_CH1	PA6	PB4	PC6
TIM3_CH2	PA7	PB5	PC7
TIM3_CH3	PB0		PC8
TIM3_CH4	PB1		PC9

1. Remap available only for 64-pin and 100-pin packages

Table 29 TIM2 alternate function remapping

Alternate function	TIM2_REMAP[1:0] = 00 (no remap)	TIM2_REMAP[1:0] = 01 (partial remap)	TIM2_REMAP[1:0] = 10 (partial remap) ⁽¹⁾	TIM2_REMAP[1:0] = 11 (full remap) ⁽¹⁾
TIM2_CH1_ETR ⁽²⁾	PA0	PA15	PA0	PA15
TIM2_CH2	PA1	PB3	PA1	PB3
TIM2_CH3	PA2		PB10	

TIM2_CH4	PA3	PB11
----------	-----	------

1. Remap not available on 36-pin package
2. TIM_CH1 and TIM_ETR share the same pin but cannot be used at the same time (which is why we have this notation: TIM2_CH1_ETR)

Table 30 TIM1 alternate function remapping

Alternate function	TIM1_REMAP[1:0] = 00 (no remap)	TIM1_REMAP[1:0] = 10 (partial remap)	TIM1_REMAP[1:0] = 11 (full remap) ⁽¹⁾
TIM1_ETR	PA12		PE7
TIM1_CH1	PA8		PE9
TIM1_CH2	PA9		PE11
TIM1_CH3	PA10		PE13
TIM1_CH4	PA11		PE14
TIM1_BKIN	PB12 ⁽²⁾	PA6	PE15
TIM1_CH1N	PB13 ⁽²⁾	PA7	PE8
TIM1_CH2N	PB14 ⁽²⁾	PA8	PE10
TIM1_CH3N	PB15 ⁽²⁾	PA9	PE12

1. Remap available only for 100-pin packages.
2. Remap not available on 36-pin package

7.3.6 USART alternate function remapping

Refer to *AF remap and debug I/O configuration register (AFIO_MAPR)*.

Table 31 USART3 remapping

Alternate function	USART3_REMAP[1:0] = 00 (no remap)	USART3_REMAP[1:0] = 01 (partial remap) ⁽¹⁾	USART3_REMAP[1:0] = 11 (full remap) ⁽²⁾
USART3_TX	PB10	PC10	PD8
USART3_RX	PB11	PC11	PD9
USART3_CK	PB12	PC12	PD10
USART3_CTS	PB13		PD11
USART3_RTS	PB14		PD12

1. Remap available only for 64-pin and 100-pin packages
2. Remap available only for 100-pin packages

Table 32 USART2 remapping

Alternate function	USART2_REMAP = 0	USART2_REMAP = 1(1)
USART2_CTS	PA0	PD3
USART2_RTS	PA1	PD4
USART2_TX	PA2	PD5
USART2_RX	PA3	PD6
USART2_CK	PA4	PD7

1. Remap available only for 100-pin packages.

Table 33 USART1 remapping

Alternate function	USART1_REMAP=0	USART1_REMAP=1
USART1_TX	PA9	PB6
USART1_RX	PA10	PB7

7.3.7 I²C1 alternate function remapping

Refer to *AF remap and debug I/O configuration register (AFIO_MAPR)*

Table 34 I2C1 remapping

Alternate function	I2C1_REMAP = 0	I2C1_REMAP = 1 ⁽¹⁾
I2C1_SCL	PB6	PB8
I2C1_SDA	PB7	PB9

1. Remap not available on 36-pin package

7.3.8 SPI1 alternate function remapping

Refer to *AF remap and debug I/O configuration register (AFIO_MAPR)*

Table 35 SPI1 remapping

Alternate function	I2C1_REMAP = 0	I2C1_REMAP = 1
SPI1_NSS	PA4	PA15
SPI1_SCK	PA5	PB3
SPI1_MISO	PA6	PB4
SPI1_MOSI	PA7	PB5

7.4 AFIO registers

Refer to *Section 1* for a list of abbreviations used in register descriptions.

Note: To read/write the *AFIO_EVCR*, *AFIO_MAPR* and *AFIO_EXTICRX* registers, the *AFIO* clock should first be enabled. Refer to *Section 6.3.7: APB2 peripheral clock enable register (RCC_APB2ENR)*.

The peripheral registers have to be accessed by words (32-bit).

7.4.1 Event control register (AFIO_EVCR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								EVOE	PORT[2:0]			PIN[3:0]			
Bits 31: 8								Reserved							
Bit 7								EVOE: Event output enable Set and cleared by software. When set the EVENTOUT Cortex [®] output is connected to the I/O selected by the PORT[2:0] and PIN[3:0] bits.							
Bits 6: 4								PORT[2:0]: Port selection Set and cleared by software. Select the port used to output the Cortex [®] EVENTOUT signal. <i>Note: The EVENTOUT signal output capability is not extended to ports PF and PG.</i> 000: PA selected 001: PB selected 010: PC selected 011: PD selected 100: PE selected							
Bits 3: 0								PIN[3:0]: Pin selection (x = A .. E) Set and cleared by software. Select the pin used to output the Cortex [®] EVENTOUT signal. 0000: Px0 selected 0001: Px1 selected 0010: Px2 selected 0011: Px3 selected							

1111: Px15 selected

7.4.2 AF remap and debug I/O configuration register (AFIO_MAPR)

Address offset: 0x04

Reset value: 0x0000 0000

Memory map and bit definitions

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved					SWJ_CFG[2:0]		Reserved				ADC2_ETRGREG_REMAP	ADC2_ETRGINJ_REMAP	ADC1_ETRGREG_REMAP	ADC1_ETRGINJ_REMAP	Res
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PD01_REMAP	CAN_REMAP	TIM4_REMAP	TIM3_REMAP	TIM2_REMAP	TIM1_REMAP	USART3_REMAP	USART2_REMAP	USART1_REMAP	I2C1_REMAP	SPI1_REMAP					
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31: 27					Reserved										
Bits 26:24					SWJ_CFG[2:0]: Serial wire JTAG configuration These bits are write-only (when read, the value is undefined). They are used to configure the SWJ and trace alternate function I/Os. The SWJ (Serial Wire JTAG) supports JTAG or SWD access to the Cortex® debug port. The default state after reset is SWJ ON without trace. This allows JTAG or SW mode to be enabled by sending a specific sequence on the JTMS / JTCK pin. 000: Full SWJ (JTAG-DP + SW-DP): Reset State 001: Full SWJ (JTAG-DP + SW-DP) but without NJTRST 010: JTAG-DP Disabled and SW-DP Enabled 100: JTAG-DP Disabled and SW-DP Disabled Other combinations: no effect										
Bits 23:21					Reserved										
Bit 20					ADC2_ETRGREG_REMAP: ADC 2 external trigger regular conversion remapping Set and cleared by software. This bit controls the trigger input connected to ADC2 external trigger regular conversion. When this bit is reset, the ADC2 external trigger regular conversion is connected to EXTI11.										
Bit 19					ADC2_ETRGINJ_REMAP: ADC 2 external trigger injected conversion remapping Set and cleared by software. This bit controls the trigger input connected to ADC2 external trigger injected conversion. When this bit is reset, the ADC2 external trigger injected conversion is connected to EXTI15.										
Bit 18					ADC1_ETRGREG_REMAP: ADC 1 external trigger regular conversion remapping Set and cleared by software. This bit controls the trigger input connected to ADC1 External trigger regular conversion. When reset the ADC1 External trigger regular conversion is connected to EXTI11.										
Bit 17					ADC1_ETRGINJ_REMAP: ADC 1 External trigger injected conversion remapping Set and cleared by software. This bit controls the trigger input connected to ADC1 External trigger injected conversion. When reset the ADC1 External trigger injected conversion is connected to EXTI15.										
Bit 16					Reserved										
Bit 15					PD01_REMAP: Port D0/Port D1 mapping on OSC_IN/OSC_OUT This bit is set and cleared by software. It controls the mapping of PD0 and PD1 GPIO functionality. When the HSE oscillator is not used (application running on internal 8 MHz RC) PD0 and PD1 can be mapped on OSC_IN and OSC_OUT. This is available only on 36-, 48- and 64-pin packages (PD0 and PD1 are available on 100-pin packages, no need for remapping). 0: No remapping of PD0 and PD1 1: PD0 remapped on OSC_IN, PD1 remapped on OSC_OUT										
Bits 14:13					CAN_REMAP[1:0]: CAN alternate function remapping These bits are set and cleared by software. They control the mapping of alternate functions CAN_RX and CAN_TX in devices with a single CAN interface. 00: CAN_RX mapped to PA11, CAN_TX mapped to PA12 01: Not used 10: CAN_RX mapped to PB8, CAN_TX mapped to PB9 (not available on 36-pin package) 11: CAN_RX mapped to PD0, CAN_TX mapped to PD1										
Bit 12					TIM4_REMAP: TIM4 remapping This bit is set and cleared by software. It controls the mapping of TIM4 channels 1 to 4 onto										

	the GPIO ports 0: No remap (TIM4_CH1/PB6, TIM4_CH2/PB7, TIM4_CH3/PB8, TIM4_CH4/PB9) 1: Full remap (TIM4_CH1/PD12, TIM4_CH2/PD13, TIM4_CH3/PD14, TIM4_CH4/PD15) <i>Note: TIM4_ETR on PE0 is not re-mapped.</i>
Bits 11:10	TIM3_REMAP[1:0]: TIM3 remapping These bits are set and cleared by software. They control the mapping of TIM3 channels 1 to 4 on the GPIO ports. 00: No remap (CH1/PA6, CH2/PA7, CH3/PB0, CH4/PB1) 01: Not used 10: Partial remap (CH1/PB4, CH2/PB5, CH3/PB0, CH4/PB1) 11: Full remap (CH1/PC6, CH2/PC7, CH3/PC8, CH4/PC9) <i>Note: TIM3_ETR on PE0 is not re-mapped.</i>
Bits 9:8	TIM2_REMAP[1:0]: TIM2 remapping These bits are set and cleared by software. They control the mapping of TIM2 channels 1 to 4 and external trigger (ETR) on the GPIO ports. 00: No remap (CH1/ETR/PA0, CH2/PA1, CH3/PA2, CH4/PA3) 01: Partial remap (CH1/ETR/PA15, CH2/PB3, CH3/PA2, CH4/PA3) 10: Partial remap (CH1/ETR/PA0, CH2/PA1, CH3/PB10, CH4/PB11) 11: Full remap (CH1/ETR/PA15, CH2/PB3, CH3/PB10, CH4/PB11)
Bits 7:6	TIM1_REMAP[1:0]: TIM1 remapping These bits are set and cleared by software. They control the mapping of TIM1 channels 1 to 4, 1N to 3N, external trigger (ETR) and Break input (BKIN) on the GPIO ports. 00: No remap (ETR/PA12, CH1/PA8, CH2/PA9, CH3/PA10, CH4/PA11, BKIN/PB12, CH1N/PB13, CH2N/PB14, CH3N/PB15) 01: Partial remap (ETR/PA12, CH1/PA8, CH2/PA9, CH3/PA10, CH4/PA11, BKIN/PA6, CH1N/PA7, CH2N/PB0, CH3N/PB1) 10: not used 11: Full remap (ETR/PE7, CH1/PE9, CH2/PE11, CH3/PE13, CH4/PE14, BKIN/PE15, CH1N/PE8, CH2N/PE10, CH3N/PE12)
Bits 5:4	USART3_REMAP[1:0]: USART3 remapping These bits are set and cleared by software. They control the mapping of USART3 CTS, RTS,CK,TX and RX alternate functions on the GPIO ports. 00: No remap (TX/PB10, RX/PB11, CK/PB12, CTS/PB13, RTS/PB14) 01: Partial remap (TX/PC10, RX/PC11, CK/PC12, CTS/PB13, RTS/PB14) 10: not used 11: Full remap (TX/PD8, RX/PD9, CK/PD10, CTS/PD11, RTS/PD12)
Bit 3	USART2_REMAP: USART2 remapping This bit is set and cleared by software. It controls the mapping of USART2 CTS, RTS,CK,TX and RX alternate functions on the GPIO ports. 0: No remap (CTS/PA0, RTS/PA1, TX/PA2, RX/PA3, CK/PA4) 1: Remap (CTS/PD3, RTS/PD4, TX/PD5, RX/PD6, CK/PD7)
Bit 2	USART1_REMAP: USART1 remapping This bit is set and cleared by software. It controls the mapping of USART1 TX and RX alternate functions on the GPIO ports. 0: No remap (TX/PA9, RX/PA10) 1: Remap (TX/PB6, RX/PB7)
Bit 1	I2C1_REMAP: I2C1 remapping This bit is set and cleared by software. It controls the mapping of I2C1 SCL and SDA alternate functions on the GPIO ports. 0: No remap (SCL/PB6, SDA/PB7) 1: Remap (SCL/PB8, SDA/PB9)
Bit 0	SPI1_REMAP: SPI1 remapping This bit is set and cleared by software. It controls the mapping of SPI1 NSS, SCK, MISO, MOSI alternate functions on the GPIO ports. 0: No remap (NSS/PA4, SCK/PA5, MISO/PA6, MOSI/PA7) 1: Remap (NSS/PA15, SCK/PB3, MISO/PB4, MOSI/PB5)

7.4.3 External interrupt configuration register 1 (AFIO_EXTICR1)

Address offset: 0x08

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:16				Reserved											
Bits 15: 0				EXTIx[3:0]: EXTI x configuration (x= 0 to 3) These bits are written by software to select the source input for EXTIx external interrupt. Refer to <i>Section 8.2.5: External interrupt/event line mapping</i> 0000: PA[x] pin 0001: PB[x] pin 0010: PC[x] pin 0011: PD[x] pin 0100: PE[x] pin											

7.4.4 External interrupt configuration register 2 (AFIO_EXTICR2)

Address offset: 0x0C

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI7[3:0]				EXTI6[3:0]				EXTI5[3:0]				EXTI4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:16				Reserved											
Bits 15: 0				EXTIx[3:0]: EXTI x configuration (x= 4 to 7) These bits are written by software to select the source input for EXTIx external interrupt. 0000: PA[x] pin 0001: PB[x] pin 0010: PC[x] pin 0011: PD[x] pin 0100: PE[x] pin											

7.4.5 External interrupt configuration register 3 (AFIO_EXTICR3)

Address offset: 0x10

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI11[3:0]				EXTI10[3:0]				EXTI9[3:0]				EXTI8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:16				Reserved											
Bits 15: 0				EXTIx[3:0]: EXTI x configuration (x= 8 to 11) These bits are written by software to select the source input for EXTIx external interrupt. 0000: PA[x] pin 0001: PB[x] pin 0010: PC[x] pin 0011: PD[x] pin 0100: PE[x] pin											

7.4.6 External interrupt configuration register 4 (AFIO_EXTICR4)

Address offset: 0x14

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI15[3:0]				EXTI14[3:0]				EXTI13[3:0]				EXTI12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:16				Reserved											
Bits 15: 0				EXTIx[3:0]: EXTI x configuration (x= 12 to 15) These bits are written by software to select the source input for EXTIx external interrupt. 0000: PA[x] pin 0001: PB[x] pin 0010: PC[x] pin 0011: PD[x] pin 0100: PE[x] pin											

CKS copyright

7.5 GPIO and AFIO register maps

The following tables give the GPIO and AFIO register map and the reset values.
Refer to Table 1 for the register boundary addresses.

Table 36 GPIO register map and reset values

00Ch		008h		004h		000h		Offset
reset value	GPIOx_ODR	reset value	GPIOx_IDR	reset value	GPIOx_CRH	reset value	GPIOx_CRL	Register
Reserved	Reserved	Reserved	Reserved	0	CNF15[1:0]	0	CNF7[1:0]	31
				1		1		30
				0	MODE15[1:0]	0	MODE7[1:0]	29
				1		1		28
				0	CNF14[1:0]	0	CNF6[1:0]	27
				1		1		26
				0	MODE14[1:0]	0	MODE6[1:0]	25
				1		1		24
				0	CNF13[1:0]	0	CNF5[1:0]	23
				1		1		22
				0	MODE13[1:0]	0	MODE5[1:0]	21
				1		1		20
				0	CNF12[1:0]	0	CNF4[1:0]	19
				1		1		18
				0	MODE12[1:0]	0	MODE4[1:0]	17
				1		1		16
0	CNF11[1:0]	0	CNF3[1:0]	15				
1		1		14				
0	MODE11[1:0]	0	MODE3[1:0]	13				
1		1		12				
0	CNF10[1:0]	0	CNF2[1:0]	11				
1		1		10				
0	MODE10[1:0]	0	MODE2[1:0]	9				
1		1		8				
0	CNF9[1:0]	0	CNF1[1:0]	7				
1		1		6				
0	MODE9[1:0]	0	MODE1[1:0]	5				
1		1		4				
0	CNF8[1:0]	0	CNF0[1:0]	3				
1		1		2				
0	MODE8[1:0]	0	MODE0[1:0]	1				
1		1		0				

010h	GPIOx_BSRR	BR[15:0]																BSR[15:0]																
	reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
014h	GPIOx_BRR	Reserved																BR[15:0]																
	reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
018h	GPIOx_ODR	Reserved																LCKK	LCK[15:0]															
	reset value																		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 37 AFIO register map and reset values

Offset	Register	Offset	Register
004h	AFIO_MAPR	31	
		30	
		29	
		28	
		27	
		26	
		25	
		24	
		23	
		22	
		21	
		20	
		19	
		18	
		17	
		16	
		15	
		14	
		13	
		12	
		11	
		10	
		9	
		8	
		7	
		6	
		5	
		4	
		3	
		2	
		1	
		0	

004h	AFIO_MAPR	AFIO_EVCRL	Reserved	31
		reset value		30
				29
				28
				27
				26
				25
				24
				23
				22
				21
				20
				19
				18
				17
				16
				15
				14
				13
				12
				11
				10
				9
				8
				7
				6
				5
				4
				3
				2
				1
				0

004h	AFIO_MAPR	AFIO_EVCRL	Reserved	31
		reset value		30
				29
				28
				27
				26
				25
				24
				23
				22
				21
				20
				19
				18
				17
				16
				15
				14
				13
				12
				11
				10
				9
				8
				7
				6
				5
				4
				3
				2
				1
				0

004h	AFIO_MAPR	AFIO_EVCRL	Reserved	31
		reset value		30
				29
				28
				27
				26
				25
				24
				23
				22
				21
				20
				19
				18
				17
				16
				15
				14
				13
				12
				11
				10
				9
				8
				7
				6
				5
				4
				3
				2
				1
				0

004h	AFIO_MAPR	AFIO_EVCRL	Reserved	31
		reset value		30
				29
				28
				27
				26
				25
				24
				23
				22
				21
				20
				19
				18
				17
				16
				15
				14
				13
				12
				11
				10
				9
				8
				7
				6
				5
				4
				3
				2
				1
				0

004h	AFIO_MAPR	AFIO_EVCRL	Reserved	31
		reset value		30
				29
				28
				27
				26
				25
				24
				23
				22
				21
				20
				19
				18
				17
				16
				15
				14
				13
				12
				11
				10
				9
				8
				7
				6
				5
				4
				3
				2
				1
				0

004h	AFIO_MAPR	AFIO_EVCRL	Reserved	31
		reset value		30
				29
				28
				27
				26
				25
				24
				23
				22
				21
				20
				19
				18
				17
				16
				15
				14
				13
				12
				11
				10
				9
				8
				7
				6
				5
				4
				3
				2
				1
				0

004h	AFIO_MAPR	AFIO_EVCRL	Reserved	31
		reset value		30
				29
				28
				27
				26
				25
				24
				23
				22
				21
				20
				19
				18
				17
				16
				15
				14
				13
				12
				11
				10
				9
				8
				7
				6
				5
				4
				3
				2
				1
				0

004h	AFIO_MAPR	AFIO_EVCRL	Reserved	31
		reset value		30
				29
				28
				27
				26
				25
				24
				23
				22
				21
				20</

	reset value		0	0	0		0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
008h	AFIO_EXTICR1	Reserved										EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
	reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
00Ch	AFIO_EXTICR2	Reserved										EXTI7[3:0]				EXTI6[3:0]				EXTI5[3:0]				EXTI4[3:0]			
	reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
010h	AFIO_EXTICR3	Reserved										EXTI11[3:0]				EXTI10[3:0]				EXTI9[3:0]				EXTI8[3:0]			
	reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
014h	AFIO_EXTICR4	Reserved										EXTI15[3:0]				EXTI14[3:0]				EXTI13[3:0]				EXTI12[3:0]			
	reset value											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

8 Interrupts and events

8.1 Nested vectored interrupt controller (NVIC)

Features

- 43 (not including the sixteen Cortex[®]-M3 interrupt lines)
- 16 programmable priority levels (4 bits of interrupt priority are used)
- Low-latency exception and interrupt handling
- Power management control
- Implementation of System Control Registers

The NVIC and the processor core interface are closely coupled, which enables low latency interrupt processing and efficient processing of late arriving interrupts.

All interrupts including the core exceptions are managed by the NVIC.

8.1.1 SysTick calibration value register

The SysTick calibration value is set to 9000, which gives a reference time base of 1 ms with the SysTick clock set to 9 MHz (max HCLK/8).

8.1.2 Interrupt and exception vectors

Table 38 Vector table for connectivity line devices

Position	Priority	Type of priority	Acronym	Description	Address
	-	-	-	Reserved	0x0000_0000
	-3	fixed	Reset	Reset	0x0000_0004
	-2	fixed	NMI	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000_0008
	-1	fixed	HardFault	All class of fault	0x0000_000C
	0	settable	MemManage	Memory management	0x0000_0010
	1	settable	BusFault	Pre-fetch fault, memory access fault	0x0000_0014
	2	settable	UsageFault	Undefined instruction or illegal state	0x0000_0018
	-	-	-	Reserved	0x0000_001C ~0x0000_002B
	3	settable	SVCall	System service call via SWI instruction	0x0000_002C
	4	settable	DebugMonitor	Debug Monitor	0x0000_0030
	-	-	-	Reserved	0x0000_0034
	5	settable	PendSV	Pendable request for system service	0x0000_0038
	6	settable	SysTick	System tick timer	0x0000_003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000_0040
1	8	settable	PVD	PVD through EXTI Line detection interrupt	0x0000_0044
2	9	settable	TAMPER	Tamper interrupt	0x0000_0048
3	10	settable	RTC	RTC global interrupt	0x0000_004C
4	11	settable	FLASH	Flash global interrupt	0x0000_0050
5	12	settable	RCC	RCC global interrupt	0x0000_0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000_0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000_005C

8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000_0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000_0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000_0068
11	18	settable	DMA_Channel 1	DMA1 Channel1 global interrupt	0x0000_006C
12	19	settable	DMA_Channel 2	DMA1 Channel2 global interrupt	0x0000_0070
13	20	settable	DMA_Channel 3	DMA1 Channel3 global interrupt	0x0000_0074
14	21	settable	DMA_Channel 4	DMA1 Channel4 global interrupt	0x0000_0078
15	22	settable	DMA_Channel 5	DMA1 Channel5 global interrupt	0x0000_007C
16	23	settable	DMA_Channel 6	DMA1 Channel6 global interrupt	0x0000_0080
17	24	settable	DMA_Channel 7	DMA1 Channel7 global interrupt	0x0000_0084
18	25	settable	ADC1_2	ADC1 and ADC2 global interrupt	0x0000_0088
19	26	settable	CAN_TX	CAN TX interrupts	0x0000_008C
20	27	settable	CAN_RX0	CAN RX0 interrupts	0x0000_0090
21	28	settable	CAN_RX1	CAN RX1 interrupt	0x0000_0094
22	29	settable	CAN_SCE	CAN SCE interrupt	0x0000_0098
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000_009C
24	31	settable	TIM1_BRK	TIM1 Break interrupt	0x0000_00A0
25	32	settable	TIM1_UP	TIM1 Update interrupt	0x0000_00A4
26	33	settable	TIM1_TRG_COM	TIM1 Trigger and Commutation interrupts	0x0000_00A8
27	34	settable	TIM1_CC	TIM1 Capture Compare interrupt	0x0000_00AC
28	35	settable	TIM2	TIM2 global interrupt	0x0000_00B0
29	36	settable	TIM3	TIM3 global interrupt	0x0000_00B4
30	37	settable	TIM4	TIM4 global interrupt	0x0000_00B8
31	38	settable	I2C1_EV	I ² C1 event interrupt	0x0000_00BC
32	39	settable	I2C1_ER	I ² C1 error interrupt	0x0000_00C0
33	40	settable	I2C2_EV	I ² C2 event interrupt	0x0000_00C4
34	41	settable	I2C2_ER	I ² C2 error interrupt	0x0000_00C8
35	42	settable	SPI1	SPI1 global interrupt	0x0000_00CC
36	43	settable	SPI2	SPI2 global interrupt	0x0000_00D0
37	44	settable	USART1	USART1 global interrupt	0x0000_00D4
38	45	settable	USART2	USART2 global interrupt	0x0000_00D8
39	46	settable	USART3	USART3 global interrupt	0x0000_00DC
40	47	settable	EXTI15_10	EXTI Line[15:10] interrupts	0x0000_00E0
41	48	settable	RTCAlarm	RTC alarm through EXTI line interrupt	0x0000_00E4
42	49	settable	OTG_FS_WKUP	USB On-The-Go FS Wakeup through EXTI line interrupt	0x0000_00E8

8.2 External interrupt/event controller (EXTI)

The external interrupt/event controller consists of 19 edge detectors in other devices for generating event/interrupt requests. Each input line can be independently configured to select the type (event or interrupt) and the corresponding trigger event (rising or falling or both). Each line can also masked independently. A pending register maintains the status line of the interrupt requests.

8.2.1 Main features

The EXTI controller main features are the following:

- Independent trigger and mask on each interrupt/event line
- Dedicated status bit for each interrupt line
- Generation of up to 19 software event/interrupt requests
- Detection of external signal with pulse width lower than APB2 clock period. Refer to the electrical characteristics section of the datasheet for details on this parameter.

8.2.2 Block diagram

The block diagram is shown in Figure 15

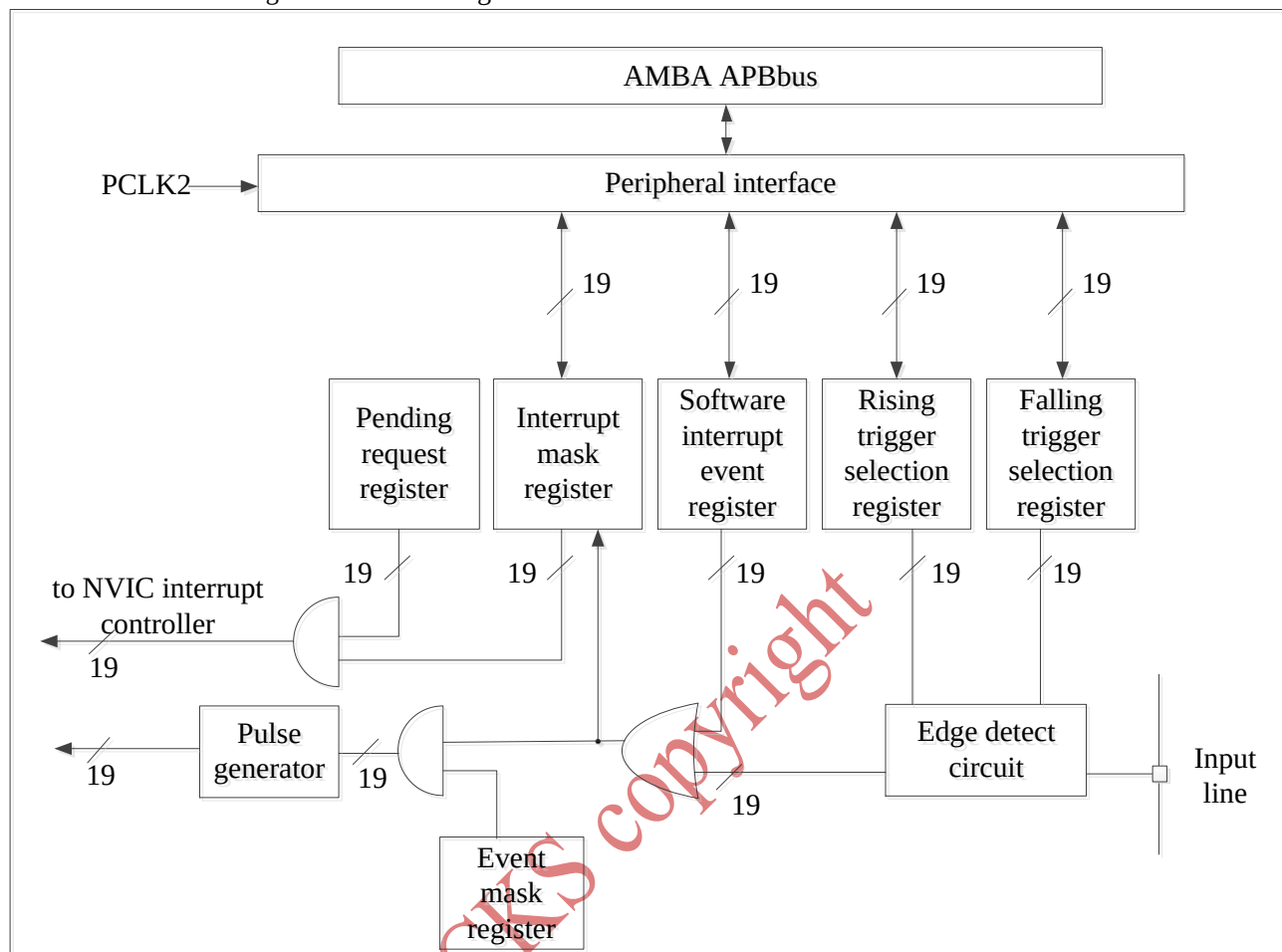


Figure 15 External interrupt/event controller block diagram

8.2.3 Wakeup event management

The CKS32F103xx is able to handle external or internal events in order to wake up the core (WFE). The wakeup event can be generated either by:

- enabling an interrupt in the peripheral control register but not in the NVIC, and enabling the SEVONPEND bit in the Cortex[®]-M3 System Control register. When the MCU resumes from WFE, the peripheral interrupt pending bit and the peripheral NVIC IRQ channel pending bit (in the NVIC interrupt clear pending register) have to be cleared.
- or configuring an external or internal EXTI line in event mode. When the CPU resumes from WFE, it is not necessary to clear the peripheral interrupt pending bit or the NVIC IRQ channel pending bit as the pending bit corresponding to the event line is not set.

In connectivity line devices, Ethernet wakeup events also have the WFE wakeup capability.

To use an external line as a wakeup event, refer to *Section 8.2.4: Functional description*.

8.2.4 Functional description

To generate the interrupt, the interrupt line should be configured and enabled. This is done by programming the two trigger registers with the desired edge detection and by enabling the interrupt request by writing a '1' to the corresponding bit in the interrupt mask register. When the selected edge occurs on the external interrupt line, an interrupt request is generated. The pending bit corresponding to the interrupt line is also set. This request is reset by writing a '1' in the pending register.

To generate the event, the event line should be configured and enabled. This is done by programming the two trigger registers with the desired edge detection and by enabling the event request by writing a '1' to the corresponding bit in the event mask register. When the selected edge occurs on the event line, an event pulse is generated. The pending bit corresponding to the event line is not set.

An interrupt/event request can also be generated by software by writing a '1' in the software interrupt/event register.

Hardware interrupt selection

To configure the 19 lines as interrupt sources, use the following procedure:

- Configure the mask bits of the 19 Interrupt lines (EXTI_IMR)
- Configure the Trigger Selection bits of the Interrupt lines (EXTI_RTISR and EXTI_FTSR)
- Configure the enable and mask bits that control the NVIC IRQ channel mapped to the External Interrupt Controller (EXTI) so that an interrupt coming from one of the 19 lines can be correctly acknowledged.

Hardware event selection

To configure the 19 lines as event sources, use the following procedure:

Configure the mask bits of the 19 Event lines (EXTI_EMR)

Configure the Trigger Selection bits of the Event lines (EXTI_RTISR and EXTI_FTSR)

Software interrupt/event selection

The 19 lines can be configured as software interrupt/event lines. The following is the procedure to generate a software interrupt.

- Configure the mask bits of the 19 Interrupt/Event lines (EXTI_IMR, EXTI_EMR)
- Set the required bit of the software interrupt register (EXTI_SWIER)

8.2.5 External interrupt/event line mapping

The 80 GPIOs are connected to the 16 external interrupt/event lines in the following manner:

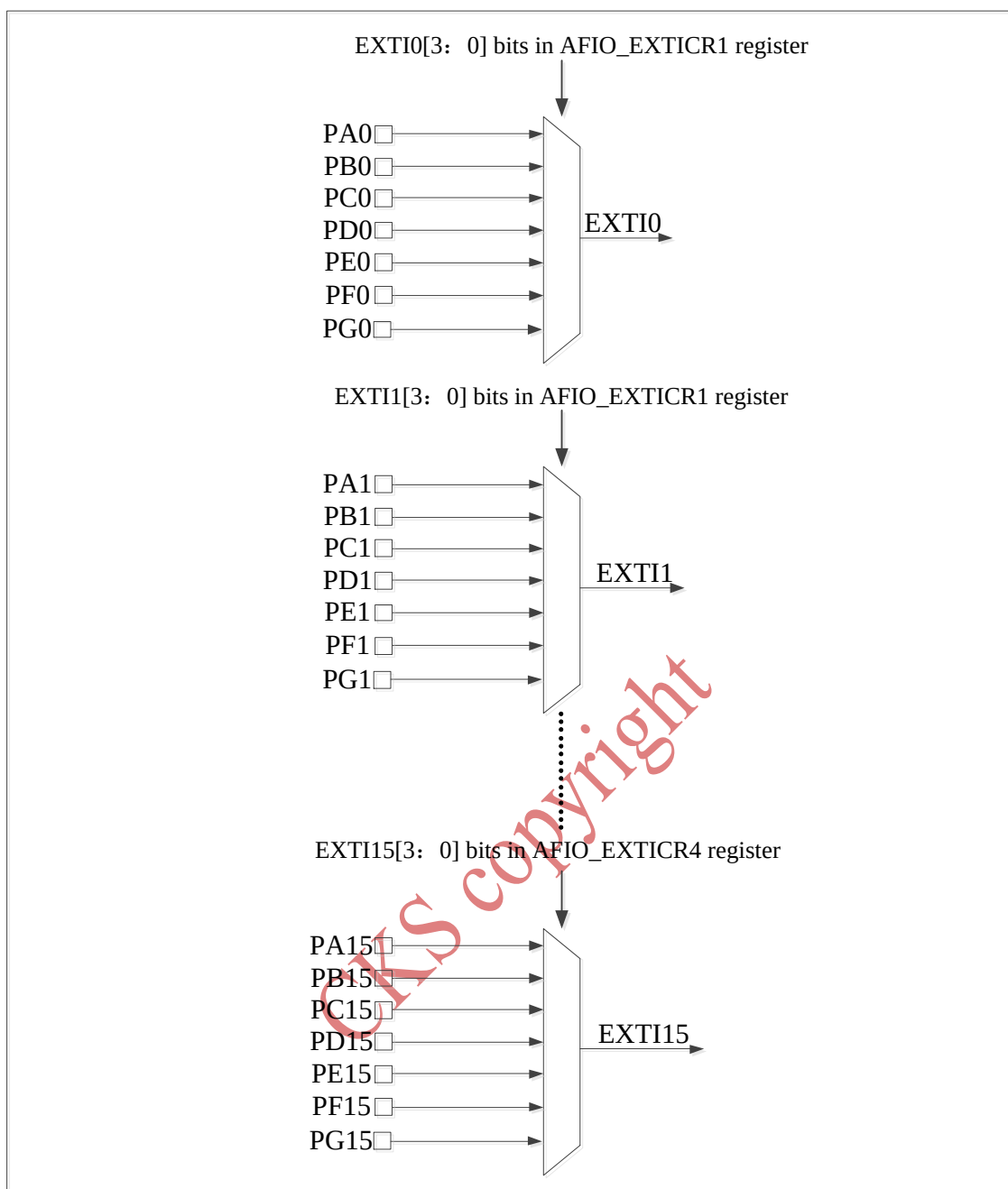


Figure 16 External interrupt/event GPIO mapping

1. To configure the AFIO_EXTICRx for the mapping of external interrupt/event lines onto GPIOs, the AFIO clock should first be enabled. Refer to *Section 6.3.7: APB2 peripheral clock enable register (RCC_APB2ENR)*.

The three other EXTI lines are connected as follows:

- EXTI line 16 is connected to the PVD output
- EXTI line 17 is connected to the RTC Alarm event
- EXTI line 18 is connected to the USB Wakeup event

8.3 EXTI registers

Refer to *Section 1.1* for a list of abbreviations used in register descriptions. The peripheral registers have to be accessed by words (32-bit).

8.3.1 Interrupt mask register (EXTI_IMR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved													MR18	MR17	MR16
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:19		Reserved, must be kept at reset value (0)													
Bits 18: 0		MRx: Interrupt Mask on line x 0: Interrupt request from Line x is masked 1: Interrupt request from Line x is not masked													

8.3.2 Event mask register (EXTI_EMR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved													MR18	MR17	MR16
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:19		Reserved, must be kept at reset value (0)													
Bits 18: 0		MRx: Event mask on line x 0: Event request from Line x is masked 1: Event request from Line x is not masked													

8.3.3 Rising trigger selection register (EXTI_RTSTR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved													TR18	TR17	TR16
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:19		Reserved, must be kept at reset value (0)													
Bits 18: 0		TRx: Rising trigger event configuration bit of line x 0: Rising trigger disabled (for Event and Interrupt) for input line 1: Rising trigger enabled (for Event and Interrupt) for input line.													

Note: The external wakeup lines are edge triggered, no glitches must be generated on these lines.

If a rising edge on external interrupt line occurs during writing of EXTI_RTSTR register, the pending bit will not be set.

Rising and Falling edge triggers can be set for the same interrupt line. In this configuration, both generate a trigger condition.

8.3.4 Falling trigger selection register (EXTI_FTSR)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved													TR18	TR17	TR16
													rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:19		Reserved, must be kept at reset value (0)													
Bits 18: 0		TRx: Falling trigger event configuration bit of line x 0: Falling trigger disabled (for Event and Interrupt) for input line 1: Falling trigger enabled (for Event and Interrupt) for input line													

Note: The external wakeup lines are edge triggered, no glitches must be generated on these lines.
 If a falling edge on external interrupt line occurs during writing of EXTI_FTSR register, the pending bit will not be set.
 Rising and Falling edge triggers can be set for the same interrupt line. In this configuration, both generate a trigger condition.

8.3.5 Software interrupt event register (EXTI_SWIER)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved													SW IER 18	SW IER 17	SW IER 16
													rw 2	rw 1	rw 0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SW IER 15	SW IER 14	SW IER 13	SW IER 12	SW IER 11	SW IER 10	SW IER 9	SW IER 8	SW IER 7	SW IER 6	SW IER 5	SW IER 4	SW IER 3	SW IER 2	SW IER 1	SW IER 0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:19		Reserved, must be kept at reset value (0)													
Bits 18: 0		SWIERx: Software interrupt on line x If the interrupt is enabled on this line in the EXTI_IMR, writing a '1' to this bit when it is set to '0' sets the corresponding pending bit in EXTI_PR resulting in an interrupt request generation. This bit is cleared by clearing the corresponding bit of EXTI_PR (by writing a 1 into the bit). <i>Note: Bit 19 used in connectivity line devices and is reserved otherwise.</i>													

8.3.6 Pending register (EXTI_PR)

Address offset: 0x14

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved													PR18	PR17	PR16
													rc wl 2	rc wl 1	rc wl 0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc wl	rc wl	rc wl	rc wl	rc wl	rc wl	rc wl	rc wl	rc wl	rc wl	rc wl	rc wl	rc wl	rc wl	rc wl	rc wl
Bits 31:19		Reserved, must be kept at reset value (0)													
Bits 18: 0		PRx: Pending bit 0: No trigger request occurred 1: selected trigger request occurred This bit is set when the selected edge event arrives on the external interrupt line. This bit is cleared by writing a '1' into the bit.													

8.3.7 EXTI register map

The following table gives the EXTI register map and the reset values.

Table 39 External interrupt/event controller register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
000h	EXTI_IMR	Reserved														MR[18:0]																	
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
004h	EXTI_EMR	Reserved														MR[18:0]																	
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
008h	EXTI_RTSTR	Reserved														TR[18:0]																	
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
00Ch	EXTI_FTSTR	Reserved														TR[18:0]																	
	Reset value															0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
010h	EXTI_SWIER	Reserved														SWIER[18:0]																	

	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
014h	EXTI_PR	Reserved	PR[18:0]																			
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Refer to Table 1 for the register boundary addresses.

CKS copyright

9 Direct memory access controller (DMA)

9.1 DMA introduction

Direct memory access (DMA) is used in order to provide high-speed data transfer between peripherals and memory as well as memory to memory. Data can be quickly moved by DMA without any CPU actions. This keeps CPU resources free for other operations.

The two DMA controllers have 7 channels, each dedicated to managing memory access requests from one or more peripherals. It has an arbiter for handling the priority between DMA requests.

9.2 DMA main features

- 7 independently configurable channels (requests)
- Each of the 7 channels is connected to dedicated hardware DMA requests, software trigger is also supported on each channel. This configuration is done by software.
- Priorities between requests from channels of one DMA are software programmable (4 levels consisting of *very high*, *high*, *medium*, *low*) or hardware in case of equality (request 0 has priority over request 1, etc.)
- Independent source and destination transfer size (byte, half word, word), emulating packing and unpacking. Source/destination addresses must be aligned on the data size.
- Support for circular buffer management
- 3 event flags (DMA Half Transfer, DMA Transfer complete and DMA TransferError) logically ORed together in a single interrupt request for each channel
- Memory-to-memory transfer
- Peripheral-to-memory and memory-to-peripheral, and peripheral-to-peripheral transfers
- Access to Flash, SRAM, APB1, APB2 and AHB peripherals as source and destination
- Programmable number of data to be transferred: up to 65536

The block diagram is shown in Figure 17.

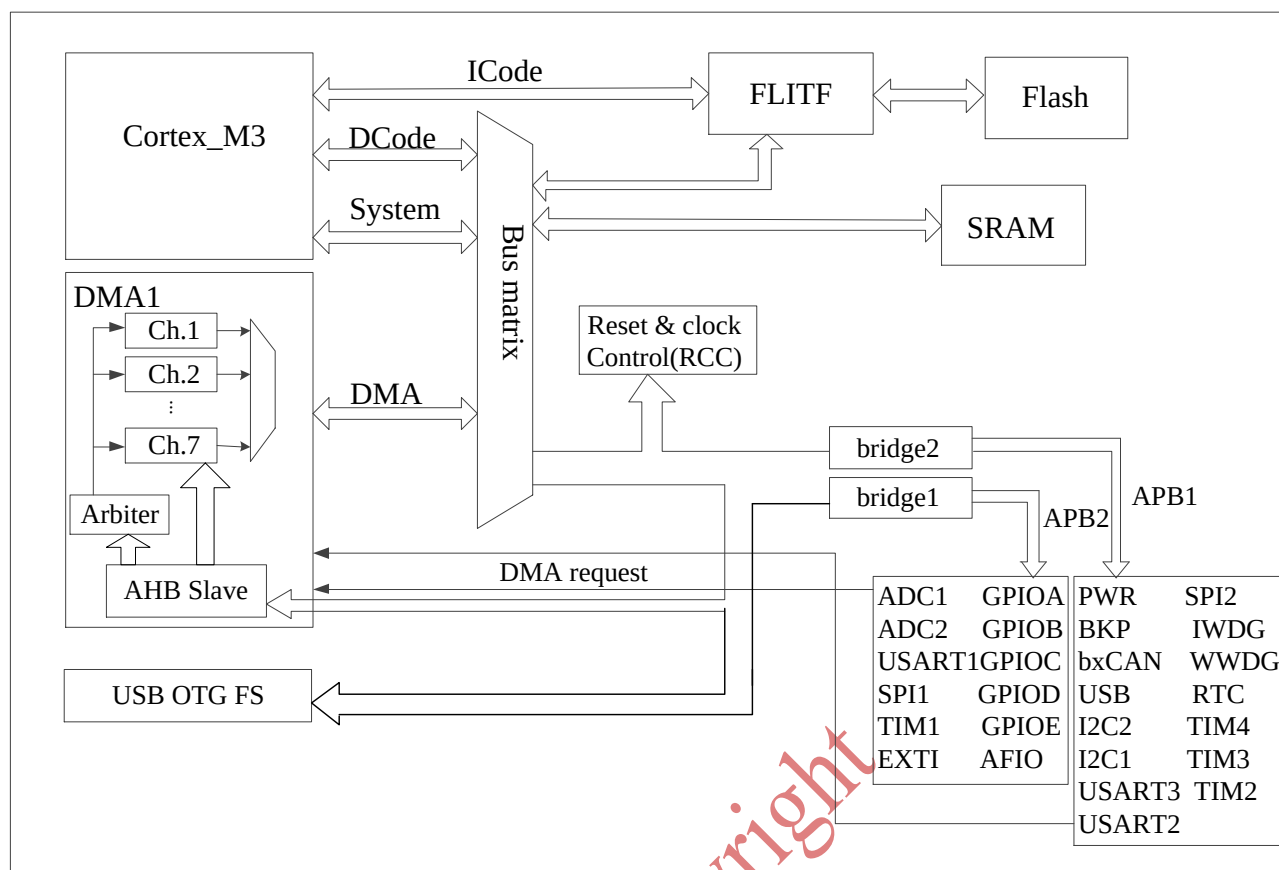


Figure 17 DMA block diagram in connectivity line devices

9.3 DMA functional description

The DMA controller performs direct memory transfer by sharing the system bus with the Cortex[®]-M3 core. The DMA request may stop the CPU access to the system bus for some bus cycles, when the CPU and DMA are targeting the same destination (memory or peripheral). The bus matrix implements round-robin scheduling, thus ensuring at least half of the system bus bandwidth (both to memory and peripheral) for the CPU.

9.3.1 DMA transactions

After an event, the peripheral sends a request signal to the DMA Controller. The DMA controller serves the request depending on the channel priorities. As soon as the DMA Controller accesses the peripheral, an Acknowledge is sent to the peripheral by the DMA Controller. The peripheral releases its request as soon as it gets the Acknowledge from the DMA Controller. Once the request is deasserted by the peripheral, the DMA Controller release the Acknowledge. If there are more requests, the peripheral can initiate the next transaction.

In summary, each DMA transfer consists of three operations:

- The loading of data from the peripheral data register or a location in memory addressed through an internal current peripheral/memory address register. The start address used for the first transfer is the base peripheral/memory address programmed in the DMA_CPARx or DMA_CMARx register.
- The storage of the data loaded to the peripheral data register or a location in memory addressed through an internal current peripheral/memory address register. The start address used for the first transfer is the base peripheral/memory address programmed in the DMA_CPARx or DMA_CMARx register.

- The post-decrementing of the DMA_CNDTRx register, which contains the number of transactions that have still to be performed.

9.3.2 Arbiter

The arbiter manages the channel requests based on their priority and launches the peripheral/memory access sequences.

The priorities are managed in two stages:

- Software: each channel priority can be configured in the DMA_CCRx register. There are four levels:
 - Very high priority
 - High priority
 - Medium priority
 - Low priority
- Hardware: if 2 requests have the same software priority level, the channel with the lowest number will get priority versus the channel with the highest number. For example, channel 2 gets priority over channel 4.

9.3.3 DMA channels

Each channel can handle DMA transfer between a peripheral register located at a fixed address and a memory address. The amount of data to be transferred (up to 65535) is programmable. The register which contains the amount of data items to be transferred is decremented after each transaction.

Programmable data sizes

Transfer data sizes of the peripheral and memory are fully programmable through the PSIZE and MSIZE bits in the DMA_CCRx register.

Pointer incrementation

Peripheral and memory pointers can optionally be automatically post-incremented after each transaction depending on the PINC and MINC bits in the DMA_CCRx register. If incremented mode is enabled, the address of the next transfer will be the address of the previous one incremented by 1, 2 or 4 depending on the chosen data size. The first transfer address is the one programmed in the DMA_CPARx/DMA_CMARx registers. During transfer operations, these registers keep the initially programmed value. The current transfer addresses (in the current internal peripheral/memory address register) are not accessible by software.

If the channel is configured in noncircular mode, no DMA request is served after the last transfer (that is once the number of data items to be transferred has reached zero). In order to reload a new number of data items to be transferred into the DMA_CNDTRx register, the DMA channel must be disabled.

In circular mode, after the last transfer, the DMA_CNDTRx register is automatically reloaded with the initially programmed value. The current internal address registers are reloaded with the base address values from the DMA_CPARx/DMA_CMARx registers.

Channel configuration procedure

The following sequence should be followed to configure a DMA channelx (where x is the channel number).

1. Set the peripheral register address in the DMA_CPARx register. The data will be moved from/ to this address to/ from the memory after the peripheral event.
2. Set the memory address in the DMA_CMARx register. The data will be written to or read from this memory after the peripheral event.
3. Configure the total number of data to be transferred in the DMA_CNDTRx register. After each peripheral event, this value will be decremented.
4. Configure the channel priority using the PL[1:0] bits in the DMA_CCRxregister.
5. Configure data transfer direction, circular mode, peripheral & memory incremented mode, peripheral & memory data size, and interrupt after half and/or full transfer in the DMA_CCRx register.
6. Activate the channel by setting the ENABLE bit in the DMA_CCRx register.

As soon as the channel is enabled, it can serve any DMA request from the peripheral connected on the channel.

Once half of the bytes are transferred, the half-transfer flag (HTIF) is set and an interrupt is generated if the Half-Transfer Interrupt Enable bit (HTIE) is set. At the end of the transfer, the Transfer Complete Flag (TCIF) is set and an interrupt is generated if the Transfer Complete Interrupt Enable bit (TCIE) is set.

Circular mode

Circular mode is available to handle circular buffers and continuous data flows (e.g. ADC scan mode). This feature can be enabled using the CIRC bit in the DMA_CCRx register. When circular mode is activated, the number of data to be transferred is automatically reloaded with the initial value programmed during the channel configuration phase, and the DMA requests continue to be served.

Memory-to-memory mode

The DMA channels can also work without being triggered by a request from a peripheral. This mode is called Memory to Memory mode.

If the MEM2MEM bit in the DMA_CCRx register is set, then the channel initiates transfers as soon as it is enabled by software by setting the Enable bit (EN) in the DMA_CCRx register. The transfer stops once the DMA_CNDTRx register reaches zero. Memory to Memory mode may not be used at the same time as Circular mode.

9.3.4 Programmable data width, data alignment and endians

When PSIZE and MSIZE are not equal, the DMA performs some data alignments as described in Table 40

Table 40 Programmable data width and endian behavior (when bits PINC = MINC = 1)

Source port width	Destination port width	Number of data items to transfer	Source content: address / data	Transfer operations	Destination content: address / data
8	8	4	0x0 / B0 0x1 / B1 0x2 / B2 0x3 / B3	1: READ B0[7:0] @0x0 then WRITE B0[7:0] @0x0 2: READ B1[7:0] @0x1 then WRITE B1[7:0] @0x1 3: READ B2[7:0] @0x2 then WRITE B2[7:0] @0x2 4: READ B3[7:0] @0x3 then WRITE B3[7:0] @0x3	0x0 / B0 0x1 / B1 0x2 / B2 0x3 / B3
8	16	4	0x0 / B0 0x1 / B1 0x2 / B2 0x3 / B3	1: READ B0[7:0] @0x0 then WRITE 00B0[15:0] @0x0 2: READ B1[7:0] @0x1 then WRITE 00B1[15:0] @0x2 3: READ B3[7:0] @0x2 then WRITE 00B2[15:0] @0x4 4: READ B4[7:0] @0x3 then WRITE 00B3[15:0] @0x6	0x0 / 00B0 0x2 / 00B1 0x4 / 00B2 0x6 / 00B3
8	32	4	0x0 / B0 0x1 / B1 0x2 / B2 0x3 / B3	1: READ B0[7:0] @0x0 then WRITE 000000B0[31:0] @0x0 2: READ B1[7:0] @0x1 then WRITE 000000B1[31:0] @0x4 3: READ B3[7:0] @0x2 then WRITE 000000B2[31:0] @0x8 4: READ B4[7:0] @0x3 then WRITE 000000B3[31:0] @0xC	0x0 / 000000B0 0x4 / 000000B1 0x8 / 000000B2 0xC / 000000B3
16	8	4	0x0 / B1B0 0x2 / B3B2 0x4 / B5B4 0x6 / B7B6	1: READ B1B0[15:0] @0x0 then WRITE B0[7:0] @0x0 2: READ B3B2[15:0] @0x2 then WRITE B2[7:0] @0x1 3: READ B5B4[15:0] @0x4 then WRITE B4[7:0] @0x2 4: READ B7B6[15:0] @0x6 then WRITE B6[7:0] @0x3	0x0 / B0 0x1 / B2 0x2 / B4 0x3 / B6
16	16	4	0x0 / B1B0 0x2 / B3B2 0x4 / B5B4 0x6 / B7B6	1: READ B1B0[15:0] @0x0 then WRITE B1B0[15:0] @0x0 2: READ B3B2[15:0] @0x2 then WRITE B3B2[15:0] @0x2 3: READ B5B4[15:0] @0x4 then WRITE B5B4[15:0] @0x4 4: READ B7B6[15:0] @0x6 then WRITE B7B6[15:0] @0x6	0x0 / B1B0 0x2 / B3B2 0x4 / B5B4 0x6 / B7B6
16	32	4	0x0 / B1B0 0x2 / B3B2 0x4 / B5B4 0x6 / B7B6	1: READ B1B0[15:0] @0x0 then WRITE 0000B1B0[31:0] @0x0 2: READ B3B2[15:0] @0x2 then WRITE 0000B3B2[31:0] @0x4 3: READ B5B4[15:0] @0x4 then WRITE 0000B5B4[31:0] @0x8 4: READ B7B6[15:0] @0x6 then WRITE 0000B7B6[31:0] @0xC	0x0 / 0000B1B0 0x4 / 0000B3B2 0x8 / 0000B5B4 0xC / 0000B7B6

32	8	4	0x0 / B3B2B1B0 0x4 / B7B6B5B4 0x8 / BBBAB9B8 0xC / BFBEBDBC	1: READ B3B2B1B0[31:0] @0x0 then WRITE B0[7:0] @0x0 2: READ B7B6B5B4[31:0] @0x4 then WRITE B4[7:0] @0x1 3: READ BBBAB9B8[31:0] @0x8 then WRITE B8[7:0] @0x2 4: READ BFBEBDBC[31:0] @0xC then WRITE BC[7:0] @0x3	0x0 / B0 0x1 / B4 0x2 / B8 0x3 / BC
32	16	4	0x0 / B3B2B1B0 0x4 / B7B6B5B4 0x8 / BBBAB9B8 0xC / BFBEBDBC	1: READ B3B2B1B0[31:0] @0x0 then WRITE B1B0[7:0] @0x0 2: READ B7B6B5B4[31:0] @0x4 then WRITE B5B4[7:0] @0x1 3: READ BBBAB9B8[31:0] @0x8 then WRITE B9B8[7:0] @0x2 4: READ BFBEBDBC[31:0] @0xC then WRITE BDBC[7:0] @0x3	0x0 / B1B0 0x2 / B5B4 0x4 / B9B8 0x6 / BDBC
32	32	4	0x0 / B3B2B1B0 0x4 / B7B6B5B4 0x8 / BBBAB9B8 0xC / BFBEBDBC	1: READ B3B2B1B0[31:0] @0x0 then WRITE B3B2B1B0[31:0] @0x0 2: READ B7B6B5B4[31:0] @0x4 then WRITE B7B6B5B4[31:0] @0x4 3: READ BBBAB9B8[31:0] @0x8 then WRITE BBBAB9B8[31:0] @0x8 4: READ BFBEBDBC[31:0] @0xC then WRITE BFBEBDBC[31:0] @0xC	0x0 / B3B2B1B0 0x4 / B7B6B5B4 0x8 / BBBAB9B8 0xC / BFBEBDBC

Addressing an AHB peripheral that does not support byte or halfword write operations

When the DMA initiates an AHB byte or halfword write operation, the data are duplicated on the unused lanes of the HWDATA[31:0] bus. So when the used AHB slave peripheral does not support byte or halfword write operations (when HSIZE is not used by the peripheral) *and* does not generate any error, the DMA writes the 32 HWDATA bits as shown in the two examples below:

- To write the halfword “0xABCD”, the DMA sets the HWDATA bus to “0xABCDABCD” with HSIZE = HalfWord
- To write the byte “0xAB”, the DMA sets the HWDATA bus to “0xABABABAB” with HSIZE = Byte

Assuming that the AHB/APB bridge is an AHB 32-bit slave peripheral that does not take the HSIZE data into account, it will transform any AHB byte or halfword operation into a 32-bit APB operation in the following manner:

- an AHB byte write operation of the data “0xB0” to 0x0 (or to 0x1, 0x2 or 0x3) will be converted to an APB word write operation of the data “0xB0B0B0B0” to 0x0
- an AHB halfword write operation of the data “0xB1B0” to 0x0 (or to 0x2) will be converted to an APB word write operation of the data “0xB1B0B1B0” to 0x0

For instance, to write the APB backup registers (16-bit registers aligned to a 32-bit address boundary), the memory source size (MSIZE) must be configured to “16-bit” and the peripheral destination size (PSIZE) to “32-bit”.

9.3.5 Error management

A DMA transfer error can be generated by reading from or writing to a reserved address space. When a DMA transfer error occurs during a DMA read or a write access, the faulty channel is automatically disabled through a hardware clear of its EN bit in the corresponding Channel configuration register (DMA_CCRx). The channel's transfer error interrupt flag (TEIF) in the DMA_IFR register is set and an interrupt is generated if the transfer error interrupt enable bit (TEIE) in the DMA_CCRx register is set.

9.3.6 Interrupts

An interrupt can be produced on a Half-transfer, Transfer complete or Transfer error for each DMA channel. Separate interrupt enable bits are available for flexibility.

Table 41 DMA interrupt requests

Interrupt event	Event flag	Enable Control bit
Half-transfer	HTIF	HTIE
Transfer complete	TCIF	TCIE
Transfer error	TEIF	TEIE

9.3.7 DMA request mapping

DMA controller

The 7 requests from the peripherals (TIMx[1,2,3,4], ADC1, SPI1, SPI/I2S2, I2Cx[1,2] and USARTx[1,2,3]) are simply logically ORed before entering the DMA1, this means that only one request must be enabled at a time. Refer to Figure 18.

The peripheral DMA requests can be independently activated/de-activated by programming the DMA control bit in the registers of the corresponding peripheral.

CKS copyright

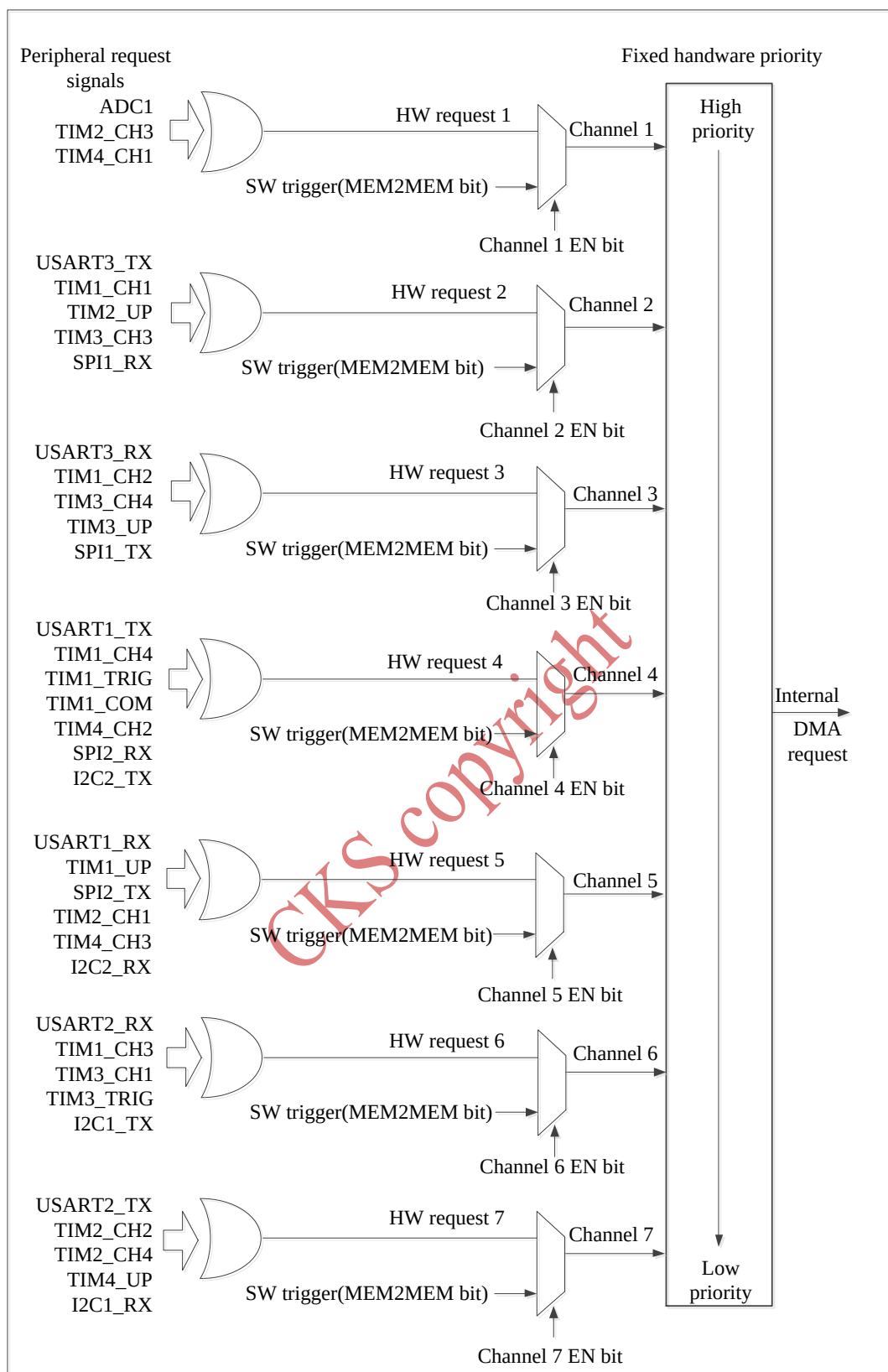


Figure 18 DMA request mapping

Table 42 lists the DMA requests for each channel.

Table 42 Summary of DMA requests for each channel

Peripherals	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Channel 6	Channel 7
ADC1	ADC1						
SPI		SPI1_RX	SPI1_TX	SPI2_RX	SPI2_TX		

USART		USART3_TX	USART3_RX	USART1_TX	USART1_RX	USART2_RX	USART2_TX
I2C				I2C2_TX	I2C2_RX	I2C1_TX	I2C1_RX
TIM1		TIM1_CH1	TIM1_CH2	TIM1_TX4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	
TIM2	TIM2_CH3	TIM2_UP			TIM2_CH1		TIM2_CH2 TIM2_CH4
TIM3		TIM3_CH3	TIM3_CH4 TIM3_UP			TIM3_CH1 TIM3_TRIG	
TIM4	TIM4_CH1			TIM4_CH2	TIM4_CH3		TIM4_UP

9.4 DMA registers

Refer to *Section 1.1* for a list of abbreviations used in register descriptions.

The peripheral registers can be accessed by bytes (8-bit), half-words (16-bit) or words (32-bit).

9.4.1 DMA interrupt status register (DMA_ISR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				TEIF 7	HTIF 7	TCIF 7	GIF 7	TEIF 6	HTIF 6	TCIF 6	GIF 6	TEIF 5	HTIF 5	TCIF 5	GIF 5
Reserved				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TEIF 4	HTIF 4	TCIF 4	GIF 4	TEIF 3	HTIF 3	TCIF 3	GIF 3	TEIF 2	HTIF 2	TCIF 2	GIF 2	TEIF 1	HTIF 1	TCIF 1	GIF 1
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 31:28				Reserved, must be kept at reset value.											
Bits 27, 23, 19, 15, 11, 7, 3				TEIFx : Channel x transfer error flag (x = 1 ..7) This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_IFCR register. 0: No transfer error (TE) on channel x 1: A transfer error (TE) occurred on channel x											
Bits 26, 22, 18, 14, 10, 6, 2				HTIFx : Channel x half transfer flag (x = 1 ..7) This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_IFCR register. 0: No half transfer (HT) event on channel x 1: A half transfer (HT) event occurred on channel x											
Bits 25, 21, 17, 13, 9, 5, 1				TCIFx : Channel x transfer complete flag (x = 1 ..7) This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_IFCR register. 0: No transfer complete (TC) event on channel x 1: A transfer complete (TC) event occurred on channel x											
Bits 24, 20, 16, 12, 8, 4, 0				GIFx : Channel x global interrupt flag (x = 1 ..7) This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_IFCR register. 0: No TE, HT or TC event on channel x 1: A TE, HT or TC event occurred on channel x											

9.4.2 DMA interrupt flag clear register (DMA_IFCR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CTEI F7	CHTI F7	CTCI F7	CGI F7	CTEI F6	CHTI F6	CTCI F6	CGI F6	CTEI F5	CHTI F5	CTCI F5	CGI F5

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CTEI F4	CHTI F4	CTCI F4	CGI F4	CTEI F3	CHTI F3	CTCI F3	CGI F3	CTEI F2	CHTI F2	CTCI F2	CGI F2	CTEI F1	CHTI F1	CTCI F1	CGI F1
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:28				Reserved, must be kept at reset value.											
Bits 27, 23, 19, 15, 11, 7, 3				CTEIFx: Channel x transfer error clear (x = 1 ..7) This bit is set and cleared by software. 0: No effect 1: Clears the corresponding TEIF flag in the DMA_ISR register											
Bits 26, 22, 18, 14, 10, 6, 2				CHTIFx: Channel x half transfer clear (x = 1 ..7) This bit is set and cleared by software. 0: No effect 1: Clears the corresponding HTIF flag in the DMA_ISR register											
Bits 25, 21, 17, 13, 9, 5, 1				CTCIFx: Channel x transfer complete clear (x = 1 ..7) This bit is set and cleared by software. 0: No effect 1: Clears the corresponding TCIF flag in the DMA_ISR register											
Bits 24, 20, 16, 12, 8, 4, 0				CGIFx: Channel x global interrupt clear (x = 1 ..7) This bit is set and cleared by software. 0: No effect 1: Clears the GIF, TEIF, HTIF and TCIF flags in the DMA_ISR register											

9.4.3 DMA channel x configuration register (DMA_CCRx)(x = 1...7)

Address offset: 0x08 + 20 x (channel number – 1)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	MEM2 MEM	PL[1:0]	MSIZE[1:0]	PSIZE[1:0]	MINC	PINC	CIRC	DIR	TEIE	HTIE	TCIE	EN			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:15				Reserved, must be kept at reset value											
Bit 14				MEM2MEM: Memory to memory mode This bit is set and cleared by software. 0: Memory to memory mode disabled 1: Memory to memory mode enabled											
Bits 13:12				PL[1:0]: Channel priority level These bits are set and cleared by software. 00: Low 01: Medium 10: High 11: Very high											
Bits 11:10				MSIZE[1:0]: Memory size These bits are set and cleared by software 00: 8-bits 01: 16-bits 10: 32-bits 11: Reserved											
Bits 9:8				PSIZE[1:0]: Peripheral size These bits are set and cleared by software. 00: 8-bits 01: 16-bits 10: 32-bits 11: Reserved											
Bit 7				MINC: Memory increment mode This bit is set and cleared by software. 0: Memory increment mode disabled 1: Memory increment mode enabled											
Bit 6				PINC: Peripheral increment mode This bit is set and cleared by software.											

	0: Peripheral increment mode disabled 1: Peripheral increment mode enabled
Bit 5	CIRC: Circular mode This bit is set and cleared by software 0: Circular mode disabled 1: Circular mode enabled
Bit 4	DIR: Data transfer direction This bit is set and cleared by software. 0: Read from peripheral 1: Read from memory
Bit 3	TEIE: Transfer error interrupt enable This bit is set and cleared by software. 0: TE interrupt disabled 1: TE interrupt enabled
Bit 2	HTIE: Half transfer interrupt enable This bit is set and cleared by software. 0: HT interrupt disabled 1: HT interrupt enabled
Bit 1	TCIE: Transfer complete interrupt enable This bit is set and cleared by software. 0: TC interrupt disabled 1: TC interrupt enabled
Bit 0	EN: Channel enable This bit is set and cleared by software. 0: Channel disabled 1: Channel enabled

9.4.4 DMA channel x number of data register (DMA_CNDTRx)(x = 1...7)

Address offset: 0x0C + 20 x (channel number – 1)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NDT[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 31:16		Reserved, must be kept at reset value.													
Bits 15:0		NDT[15:0]: Number of data to transfer Number of data to be transferred (0 up to 65535). This register can only be written when the channel is disabled. Once the channel is enabled, this register is read-only, indicating the remaining bytes to be transmitted. This register decrements after each DMA transfer. Once the transfer is completed, this register can either stay at zero or be reloaded automatically by the value previously programmed if the channel is configured in auto-reload mode. If this register is zero, no transaction can be served whether the channel is enabled or not.													

9.4.5 DMA channel x peripheral address register (DMA_CPARx)(x = 1...7)

Address offset: 0x10 + 20 x (channel number – 1)

Reset value: 0x0000 0000

This register must *not* be written when the channel is enabled.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PA[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:0				PA[31:0]: Peripheral address Base address of the peripheral data register from/to which the data will be read/written. When PSIZE is 01 (16-bit), the PA[0] bit is ignored. Access is automatically aligned to a half- word address. When PSIZE is 10 (32-bit), PA[1:0] are ignored. Access is automatically aligned to a word address.											

9.4.6 DMA channel x memory address register (DMA_CMARx)(x = 1...7)

Address offset: 0x14 + 20 x (channel number – 1)

Reset value: 0x0000 0000

This register must *not* be written when the channel is enabled.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MA[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:0				MA[31:0]: Memory address Base address of the memory area from/to which the data will be read/written. When MSIZE is 01 (16-bit), the MA[0] bit is ignored. Access is automatically aligned to a half-word address. When MSIZE is 10 (32-bit), MA[1:0] are ignored. Access is automatically aligned to a word address.											

9.4.7 DMA register map

The following table gives the DMA register map and the reset values.

Table 43 DMA register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
000h	DMA_ISR	Reserved																															
	Reset value	0																															
004h	DMA_IFCR	Reserved																															
		Reserved																															
		CTEIF7	CHTIF7	CTCIF7	CGIF7	CTEIF6	CHTIF6	CTCIF6	CGIF6	CTEIF5	CHTIF5	CTCIF5	CGIF5	CTEIF4	CHTIF4	CTCIF4	CGIF4	CTEIF3	CHTIF3	CTCIF3	CGIF3	CTEIF2	CHTIF2	CTCIF2	CGIF2	CTEIF1	CHTIF1	CTCIF1	CGIF1				

	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
--	-------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
020h	DNA_CNDTR2	Reserved																NDT[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
024h	DNA_CPAR2	PA[31:0]																															
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
028h	DNA_CMAR2	MA[31:0]																															
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
02Ch	Reserved																																
030h	DNA_CCR3	Reserved																MEM2MEM	PL [1:0]		MSIZE [1:0]		PSIZE [1:0]		MINC	PINC	CIRC	DIR	TEIE	HTIE	TCIE	EN	
	Reset value																		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
034h	DNA_CNDTR3	Reserved																NDT[15:0]															

	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
038h	DNA_CPAR3	PA[31:0]																																	
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
03Ch	DNA_CMAR3	MA[31:0]																																	
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
040h	Reserved																																		
044h	DNA_CCR4	Reserved																MEM2MEM	PL [1:0]		MSIZE [1:0]		PSIZE [1:0]		MINC	PINC	CIRC	DIR	TEIE	HTIE	TCIE	EN			
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
048h	DNA_CNDTR4	Reserved																NDT[15:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
04Ch	DNA_CPAR4	PA[31:0]																																	

	Reset value																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
--	-------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

[illegible]

[illegible]

Refer to Table 1 for the register boundary addresses.

10 Analog-to-digital converter (ADC)

10.1 ADC introduction

The 12-bit ADC is a successive approximation analog-to-digital converter. It has up to 18 multiplexed channels allowing it measure signals from sixteen external and two internal sources. A/D conversion of the various channels can be performed in single, continuous, scan or discontinuous mode. The result of the ADC is stored in a left-aligned or right-aligned 16-bit data register.

The analog watchdog feature allows the application to detect if the input voltage goes outside the user-defined high or low thresholds.

The ADC input clock is generated from the PCLK2 clock divided by a prescaler and it must not exceed 14 MHz, refer to Figure 7 .

10.2 ADC main features

- 12-bit resolution
- Interrupt generation at End of Conversion, End of Injected conversion and Analog watchdog event
- Single and continuous conversion modes
- Scan mode for automatic conversion of channel 0 to channel 'n'
- Self-calibration
- Data alignment with in-built data coherency
- Channel by channel programmable sampling time
- External trigger option for both regular and injected conversion
- Discontinuous mode
- Dual mode (on devices with 2 ADCs or more)
- ADC conversion time: 1 μ s at 56 MHz (1.17 μ s at 72 MHz)
- ADC supply requirement: 2.4 V to 3.6 V
- ADC input range: $V_{REF-} \leq V_{IN} \leq V_{REF+}$
- DMA request generation during regular channel conversion

The block diagram of the ADC is shown in Figure 19.

Note: V_{REF-} , if available (depending on package), must be tied to V_{SSA} .

10.3 ADC functional description

Figure 19 shows a single ADC block diagram and Table 44 gives the ADC pin description.

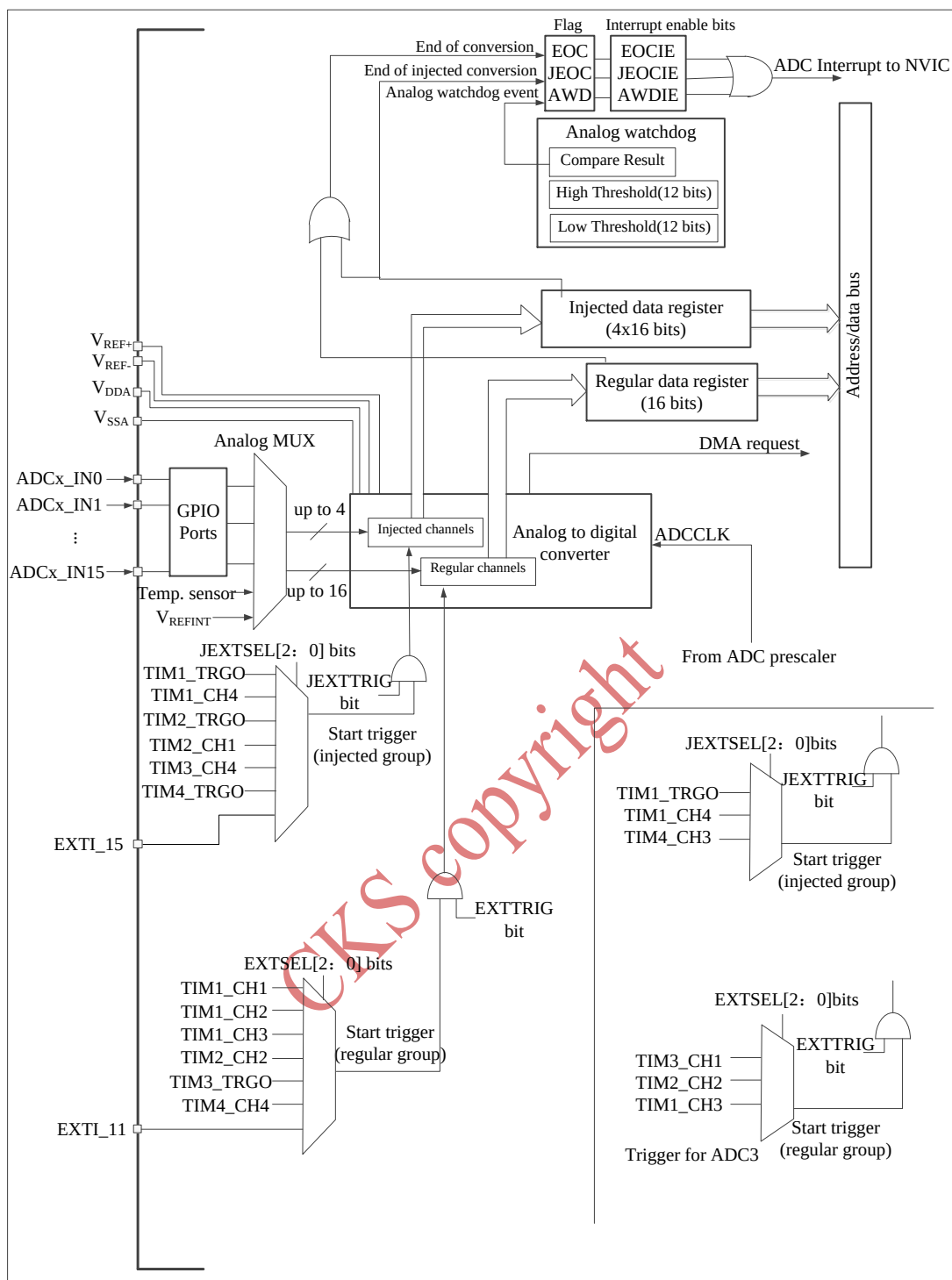


Figure 19 Single ADC block diagram

Table 44 ADC pins

Name	Signal type	Remarks
VREF+	Input, analog reference positive	The higher/positive reference voltage for the ADC, $2.4\text{ V} \leq V_{\text{REF}+} \leq V_{\text{DDA}}$
VDDA ⁽¹⁾	Input, analog supply	Analog power supply equal to VDD and $2.4\text{ V} \leq V_{\text{DDA}} \leq V_{\text{DD}}(3.6\text{ V})$
VREF-	Input, analog reference negative	The lower/negative reference voltage for the ADC, $V_{\text{REF}-} = V_{\text{SSA}}$
VSSA ⁽¹⁾	Input, analog supply ground	Ground for analog power supply equal to VSS
ADCx_IN[15:0]	Analog signals	Up to 16 analog channels

1. VDDA and VSSA have to be connected to VDD and VSS, respectively

10.3.1 ADC on-off control

The ADC can be powered-on by setting the ADON bit in the ADC_CR2 register. When the ADON bit is set for the first time, it wakes up the ADC from Power Down mode.

Conversion starts when ADON bit is set for a second time by software after ADC power-up time (t_{STAB}).

The conversion can be stopped, and the ADC put in power down mode by resetting the ADON bit. In this mode the ADC consumes almost no power (only a few μA).

10.3.2 ADC clock

The ADCCLK clock provided by the Clock Controller is synchronous with the PCLK2 (APB2 clock). The RCC controller has a dedicated programmable prescaler for the ADC clock reset and clock control (RCC) for more details.

10.3.3 Channel selection

There are 16 multiplexed channels. It is possible to organize the conversions in two groups: regular and injected. A group consists of a sequence of conversions which can be done on any channel and in any order. For instance, it is possible to do the conversion in the following order: Ch3, Ch8, Ch2, Ch2, Ch0, Ch2, Ch2, Ch15

- The **regular group** is composed of up to 16 conversions. The regular channels and their order in the conversion sequence must be selected in the ADC_SQRx registers. The total number of conversions in the regular group must be written in the L[3:0] bits in the ADC_SQR1 register.
- The **injected group** is composed of up to 4 conversions. The injected channels and their order in the conversion sequence must be selected in the ADC_JSQR register. The total number of conversions in the injected group must be written in the L[1:0] bits in the ADC_JSQR register.

If the ADC_SQRx or ADC_JSQR registers are modified during a conversion, the current conversion is reset and a new start pulse is sent to the ADC to convert the new chosen group.

Temperature sensor/VREFINT internal channels

The Temperature sensor is connected to channel ADCx_IN16 and the internal reference voltage VREFINT is connected to ADCx_IN17. These two internal channels can be selected and converted as injected or regular channels.

Note: The sensor and VREFINT are only available on the master ADC1 peripheral.

10.3.4 Single conversion mode

In Single conversion mode the ADC does one conversion. This mode is started either by setting the ADON bit in the ADC_CR2 register (for a regular channel only) or by external trigger (for a regular or injected channel), while the CONT bit is 0.

Once the conversion of the selected channel is complete:

- If a regular channel was converted:
 - The converted data is stored in the 16-bit ADC_DR register
 - The EOC (End Of Conversion) flag is set
 - and an interrupt is generated if the EOCIE is set.
- If an injected channel was converted:
 - The converted data is stored in the 16-bit ADC_DRJ1 register
 - The JEOP (End Of Conversion Injected) flag is set
 - and an interrupt is generated if the JEOPIE bit is set.

The ADC is then stopped.

10.3.5 Continuous conversion mode

In continuous conversion mode ADC starts another conversion as soon as it finishes one. This mode is started either by external trigger or by setting the ADON bit in the ADC_CR2 register, while the CONT bit is 1.

After each conversion:

- If a regular channel was converted:
 - The converted data is stored in the 16-bit ADC_DR register
 - The EOC (End Of Conversion) flag is set
 - An interrupt is generated if the EOCIE is set.
- If an injected channel was converted:
 - The converted data is stored in the 16-bit ADC_DRJ1 register
 - The JEOC (End Of Conversion Injected) flag is set.
 - An interrupt is generated if the JEOCIE bit is set.

10.3.6 Timing diagram

As shown in *Figure 20*, the ADC needs a stabilization time of t_{STAB} before it starts converting accurately. After the start of ADC conversion and after 14 clock cycles, the EOC flag is set and the 16-bit ADC Data register contains the result of the conversion.

CKS copyright

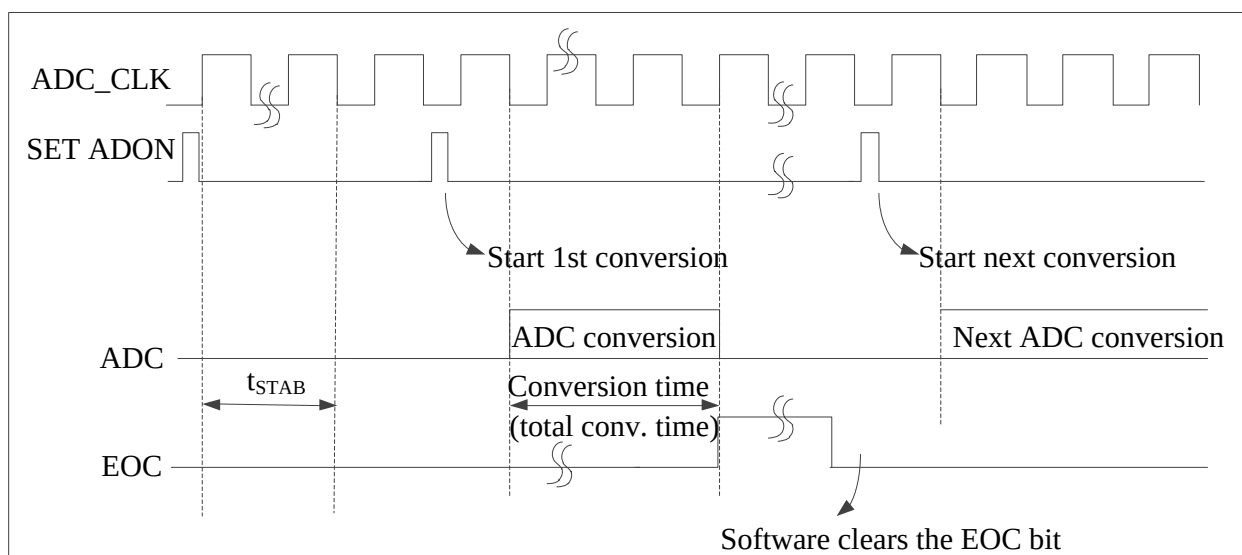


Figure 20 Timing diagram

10.3.7 Analog watchdog

The AWD analog watchdog status bit is set if the analog voltage converted by the ADC is below a low threshold or above a high threshold. These thresholds are programmed in the 12 least significant bits of the ADC_HTR and ADC_LTR 16-bit registers. An interrupt can be enabled by using the AWDIE bit in the ADC_CR1 register.

The threshold value is independent of the alignment selected by the ALIGN bit in the ADC_CR2 register. The comparison is done before the alignment (see *Section 10.5*).

The analog watchdog can be enabled on one or more channels by configuring the ADC_CR1 register as shown in Table 45

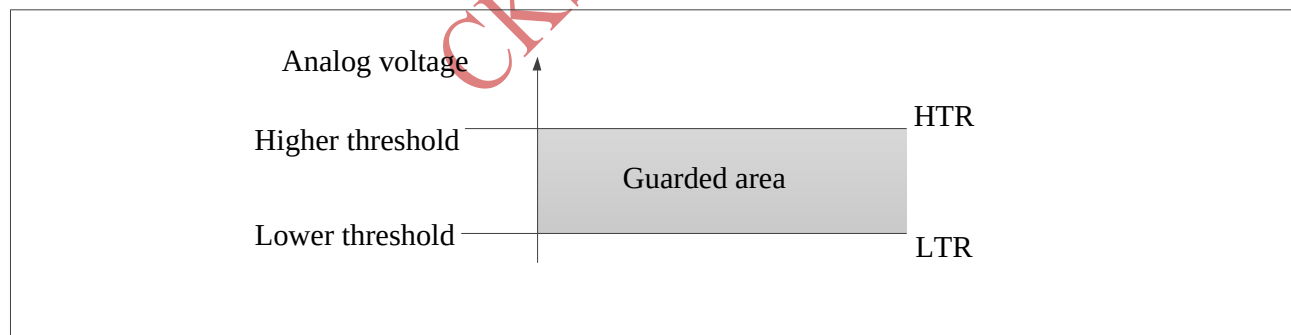


Figure 21 Analog watchdog guarded area

Table 45 Analog watchdog channel selection

Channels to be guarded by analog watchdog	ADC_CR1 register control bits		
	AWDSGL bit	AWDEN bit	JAWDEN bit
None	x	0	0
All injected channels	0	0	1
All regular channels	0	1	0
All regular and injected channels	0	1	1
Single ⁽¹⁾ injected channel	1	0	1
Single ⁽¹⁾ regular channel	1	1	0
Single ⁽¹⁾ regular or injected channel	1	1	1

(1) Selected by AWDCH[4:0] bits

10.3.8 Scan mode

This mode is used to scan a group of analog channels.

Scan mode can be selected by setting the SCAN bit in the ADC_CR1 register. Once this bit is set, ADC scans all the channels selected in the ADC_SQRx registers (for regular channels) or in the ADC_JSQR (for injected channels). A single conversion is performed for each channel of the group. After each end of conversion the next channel of the group is converted automatically. If the CONT bit is set, conversion does not stop at the last selected group channel but continues again from the first selected group channel. When using scan mode, DMA bit must be set and the direct memory access controller is used to transfer the converted data of regular group channels to SRAM after each update of the ADC_DR register. The injected channel converted data is always stored in the ADC_JDRx registers.

10.3.9 Injected channel management

Triggered injection

To use triggered injection, the JAUTO bit must be cleared and SCAN bit must be set in the ADC_CR1 register.

1. Start conversion of a group of regular channels either by external trigger or by setting the ADON bit in the ADC_CR2 register.
2. If an external injected trigger occurs during the regular group channel conversion, the current conversion is reset and the injected channel sequence is converted in Scan once mode.
3. Then, the regular group channel conversion is resumed from the last interrupted regular conversion. If a regular event occurs during an injected conversion, it doesn't interrupt it but the regular sequence is executed at the end of the injected sequence. Figure 22 shows the timing diagram.

Note: When using triggered injection, the interval between trigger events must be longer than the injection sequence. For instance, if the sequence length is 28 ADC clock cycles (that is two conversions with a 1.5 clock-period sampling time), the minimum interval between triggers must be 29 ADC clock cycles.

Auto-injection

If the JAUTO bit is set, then the injected group channels are automatically converted after the regular group channels. This can be used to convert a sequence of up to 20 conversions programmed in the ADC_SQRx and ADC_JSQR registers.

In this mode, external trigger on injected channels must be disabled.

If the CONT bit is also set in addition to the JAUTO bit, regular channels followed by injected channels are continuously converted.

For ADC clock prescalers ranging from 4 to 8, a delay of 1 ADC clock period is automatically inserted when switching from regular to injected sequence (respectively injected to regular). When the ADC clock prescaler is set to 2, the delay is 2 ADC clock periods.

Note: It is not possible to use both auto-injected and discontinuous modes simultaneously.

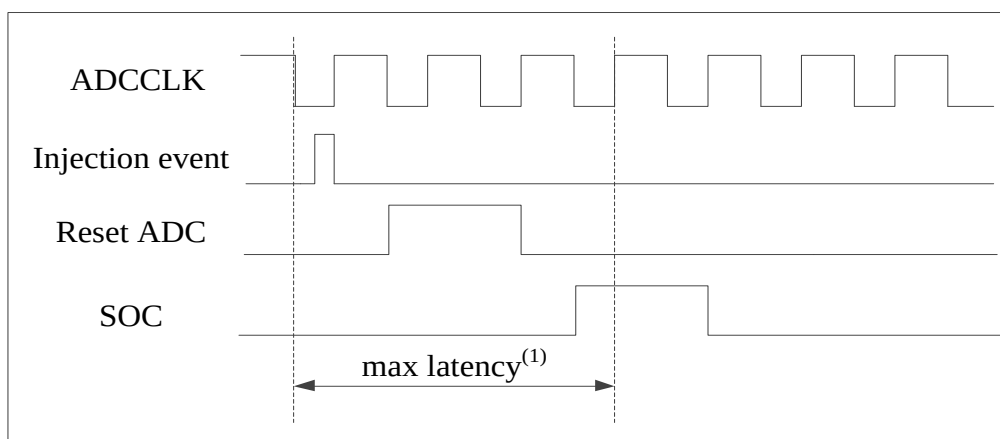


Figure 22 Injected conversion latency

(1) The maximum latency value can be found in the electrical characteristics of the CKS32F103x8 and CKS32F103xB datasheets

10.3.10 Discontinuous mode

Regular group

This mode is enabled by setting the DISCEN bit in the ADC_CR1 register. It can be used to convert a short sequence of n conversions ($n \leq 8$) which is a part of the sequence of conversions selected in the ADC_SQRx registers. The value of n is specified by writing to the DISCNUM[2:0] bits in the ADC_CR1 register.

When an external trigger occurs, it starts the next n conversions selected in the ADC_SQRx registers until all the conversions in the sequence are done. The total sequence length is defined by the L[3:0] bits in the ADC_SQR1 register.

Example:

$n = 3$, channels to be converted = 0, 1, 2, 3, 6, 7, 9, 10

1st trigger: sequence converted 0, 1, 2. An EOC event is generated at each conversion

2nd trigger: sequence converted 3, 6, 7. An EOC event is generated at each conversion

3rd trigger: sequence converted 9, 10. An EOC event is generated at each conversion

4th trigger: sequence converted 0, 1, 2. An EOC event is generated at each conversion

Note: When a regular group is converted in discontinuous mode, no rollover will occur. When all sub groups are converted, the next trigger starts conversion of the first sub-group.

In the example above, the 4th trigger reconverts the 1st sub-group channels 0, 1 and 2.

Injected group

This mode is enabled by setting the JDISCEN bit in the ADC_CR1 register. It can be used to convert the sequence selected in the ADC_JSQR register, channel by channel, after an external trigger event.

When an external trigger occurs, it starts the next channel conversions selected in the ADC_JSQR registers until all the conversions in the sequence are done. The total sequence length is defined by the JL[1:0] bits in the ADC_JSQR register.

Example:

$n = 1$, channels to be converted = 1, 2, 3

1st trigger: channel 1 converted

2nd trigger: channel 2 converted

3rd trigger: channel 3 converted and EOC and JEOC events generated

4th trigger: channel 1

Note: When all injected channels are converted, the next trigger starts the conversion of the first injected channel. In the example above, the 4th trigger reconverts the 1st injected channel 1.

It is not possible to use both auto-injected and discontinuous modes simultaneously.

The user must avoid setting discontinuous mode for both regular and injected groups together.

Discontinuous mode must be enabled only for one group conversion.

10.4 Calibration

The ADC has an built-in self calibration mode. Calibration significantly reduces accuracy errors due to internal capacitor bank variations. During calibration, an error-correction code (digital word) is calculated for each capacitor, and during all subsequent conversions, the error contribution of each capacitor is removed using this code.

Calibration is started by setting the CAL bit in the ADC_CR2 register. Once calibration is over, the CAL bit is reset by hardware and normal conversion can be performed. It is recommended to calibrate the ADC once at power-on. The calibration codes are stored in the ADC_DR as soon as the calibration phase ends.

Note: *It is recommended to perform a calibration after each power-up. Before starting a calibration, the ADC must have been in power-on state (ADON bit = '1') for at least two ADC clock cycles.*

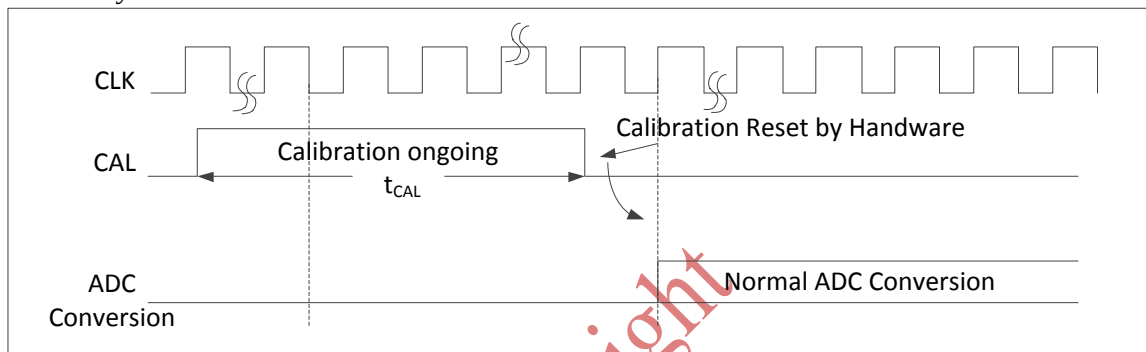


Figure 23 Calibration timing diagram

10.5 Data alignment

ALIGN bit in the ADC_CR2 register selects the alignment of data stored after conversion. Data can be left or right aligned as shown in Figure 24 and Figure 25

The injected group channels converted data value is decreased by the user-defined offset written in the ADC_JOFRx registers so the result can be a negative value. The SEXT bit is the extended sign value. For regular group channels no offset is subtracted so only twelve bits are significant.

Injected group

SEXT	SEXT	SEXT	SEXT	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
------	------	------	------	-----	-----	----	----	----	----	----	----	----	----	----	----

Regular group

0	0	0	0	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
---	---	---	---	-----	-----	----	----	----	----	----	----	----	----	----	----

Figure 24 Right alignment of data

Injected group

SEXT	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	0	0	0
------	-----	-----	----	----	----	----	----	----	----	----	----	----	---	---	---

Regular group

D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	0	0	0	0
-----	-----	----	----	----	----	----	----	----	----	----	----	---	---	---	---

Figure 25 Left alignment of data

10.6 Channel-by-channel programmable sample time

ADC samples the input voltage for a number of ADC_CLK cycles which can be modified using the SMP[2:0] bits in the ADC_SMPR1 and ADC_SMPR2 registers. Each channel can be sampled with a different sample time.

The total conversion time is calculated as follows:

$T_{conv} = \text{Sampling time} + 12.5 \text{ cycles}$

Example:

With an ADCCLK = 14 MHz and a sampling time of 1.5 cycles:

$T_{conv} = 1.5 + 12.5 = 14 \text{ cycles} = 1 \mu\text{s}$

10.7 Conversion on external trigger

Conversion can be triggered by an external event (e.g. timer capture, EXTI line). If the EXTTRIG control bit is set then external events are able to trigger a conversion. The EXTSEL[2:0] and JEXTSEL[2:0] control bits allow the application to select which out of 8 possible events can trigger conversion for the regular and injected groups.

Note: When an external trigger is selected for ADC regular or injected conversion, only the rising edge of the signal can start the conversion.

Table 46 External trigger for regular channels for ADC1 and ADC2

Source	Type	EXTSEL[2:0]
TIM1_CC1 event	Internal signal from on-chip timers	000
TIM1_CC2 event		001
TIM1_CC3 event		010
TIM2_CC2 event		011
TIM3_TRGO event		100
TIM4_CC4 event		101
EXTI line 11 ⁽¹⁾	External pin/Internal signal from on-chip timers	110
SWSTART	Software control bit	111

- The selection of the external trigger EXTI line11 event for regular channels is done through configuration bits ADC1_ETRGREG_REMAP and ADC2_ETRGREG_REMAP for ADC1 and ADC2, respectively.

Table 47 External trigger for injected channels for ADC1 and ADC2

Source	Connection type	JEXTSEL[2:0]
TIM1_TRGO event	Internal signal from on-chip timers	000
TIM1_CC4 event		001
TIM2_TRGO event		010
TIM2_CC1 event		011
TIM3_CC4 event		100
TIM4_TRGO event		101
EXTI line 15 ⁽¹⁾	External pin/Internal signal from on-chip timers	110
JSWSTART	Software control bit	111

- The selection of the external trigger EXTI line15 event for injected channels is done through configuration bits ADC1_ETRGINJ_REMAP and ADC2_ETRGINJ_REMAP for ADC1 and ADC2, respectively.

Table 48 External trigger for regular channels for ADC3

Source	Connection type	EXTSEL[2:0]
TIM3_CC1 event	Internal signal from on-chip timers	000
TIM2_CC3 event		001
TIM1_CC3 event		010
SWSTART	Software control bit	111

Table 49 External trigger for injected channels for ADC3

Source	Connection type	JEXTSEL[2:0]
TIM1_TRGO event	Internal signal from on-chip timers	000
TIM1_CC4 event		001
TIM4_CC3 event		010
JSWSTART	Software control bit	111

The software source trigger events can be generated by setting a bit in a register (SWSTART and JSWSTART in ADC_CR2).

A regular group conversion can be interrupted by an injected trigger.

10.8 DMA request

Since converted regular channels value are stored in a unique data register, it is necessary to use DMA for conversion of more than one regular channel. This avoids the loss of data already stored in the ADC_DR register.

Only the end of conversion of a regular channel generates a DMA request, which allows the transfer of its converted data from the ADC_DR register to the destination location selected by the user.

Note: Only ADC1 and ADC3 have this DMA capability. ADC2-converted data can be transferred in dual ADC mode using DMA thanks to master ADC1.

10.9 Dual ADC mode

In devices with two ADCs or more, dual ADC mode can be used (see *Figure 26*).

In dual ADC mode the start of conversion is triggered alternately or simultaneously by the ADC1 master to the ADC2 slave, depending on the mode selected by the DUALMOD[2:0] bits in the ADC1_CR1 register.

Note: In dual mode, when configuring conversion to be triggered by an external event, the user must set the trigger for the master only and set a software trigger for the slave to prevent spurious triggers to start unwanted slave conversion. However, external triggers must be enabled on both master and slave ADCs.

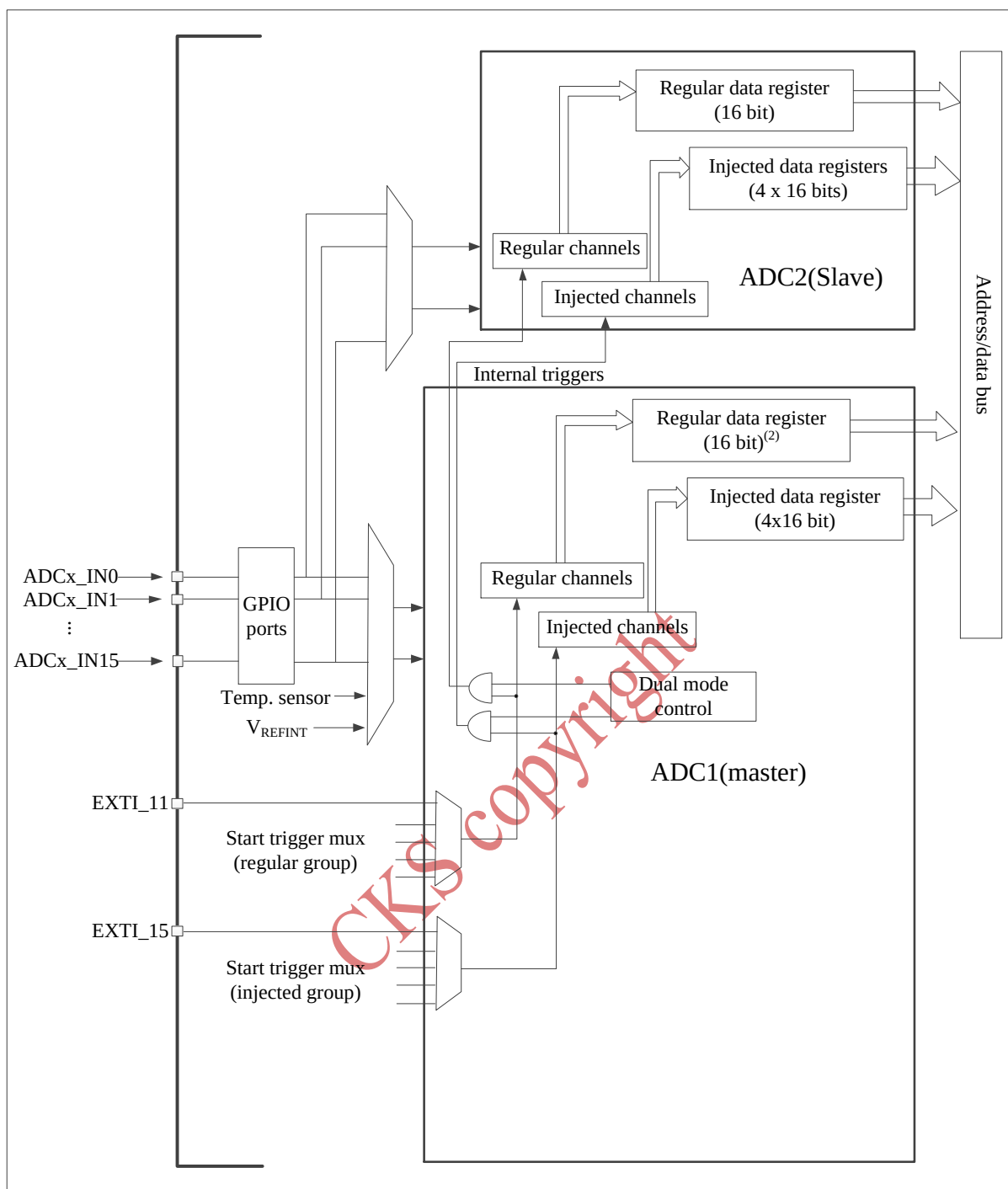
The following six possible modes are implemented:

- Injected simultaneous mode
- Regular simultaneous mode
- Fast interleaved mode
- Slow interleaved mode
- Alternate trigger mode
- Independent mode

It is also possible to use the previous modes combined in the following ways:

- Injected simultaneous mode + Regular simultaneous mode
- Regular simultaneous mode + Alternate trigger mode
- Injected simultaneous mode + Interleaved mode

Note: In dual ADC mode, to read the slave converted data on the master data register, the DMA bit must be enabled even if it is not used to transfer converted regular channel data.

Figure 26 Dual ADC block diagram⁽¹⁾

1. External triggers are present on ADC2 but are not shown for the purposes of this diagram
2. In some dual ADC modes, the ADC1 data register (ADC1_DR) contains both ADC1 and ADC2 regular converted data over the entire 32 bits.

10.9.1 Injected simultaneous mode

This mode converts an injected channel group. The source of external trigger comes from the injected group mux of ADC1 (selected by the JEXTSEL[2:0] bits in the ADC1_CR2 register). A simultaneous trigger is provided to ADC2.

Note: Do not convert the same channel on the two ADCs (no overlapping sampling times for the two ADCs when converting the same channel).

At the end of conversion event on ADC1 or ADC2:

- The converted data is stored in the ADC_JDRx registers of each ADC interface.
- An JEOP interrupt is generated (if enabled on one of the two ADC interfaces) when the ADC1/ADC2 injected channels are all converted.

Note: In simultaneous mode, exactly the same sampling time should be configured for the two channels that will be sampled simultaneously by ADC

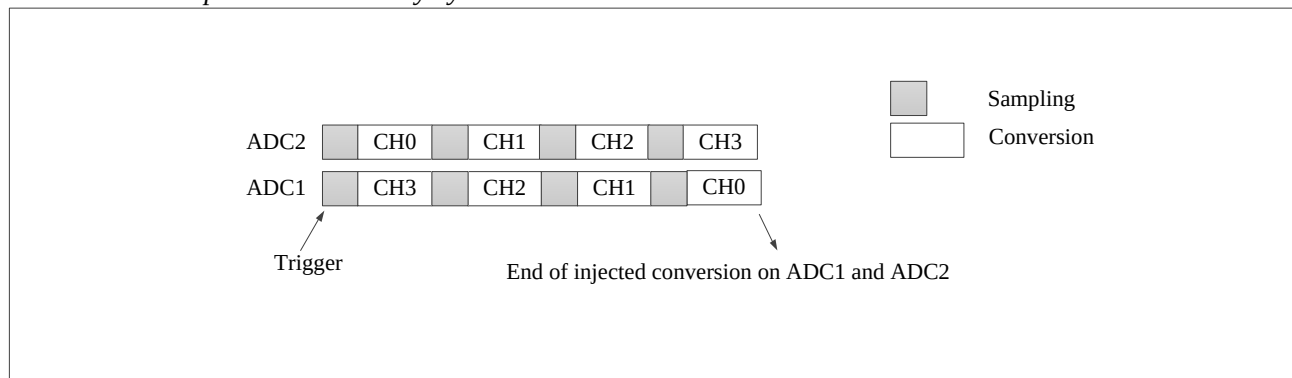


Figure 27 Injected simultaneous mode on 4 channels

10.9.2 Regular simultaneous mode

This mode is performed on a regular channel group. The source of the external trigger comes from the regular group mux of ADC1 (selected by the EXTSEL[2:0] bits in the ADC1_CR2 register). A simultaneous trigger is provided to the ADC2.

Note: Do not convert the same channel on the two ADCs (no overlapping sampling times for the two ADCs when converting the same channel).

At the end of conversion event on ADC1 or ADC2:

- A 32-bit DMA transfer request is generated (if DMA bit is set) which transfers to SRAM the ADC1_DR 32-bit register containing the ADC2 converted data in the upper halfword and the ADC1 converted data in the lower halfword.
- An EOC interrupt is generated (if enabled on one of the two ADC interfaces) when ADC1/ADC2 regular channels are all converted.

Note: In regular simultaneous mode, exactly the same sampling time should be configured for the two channels that will be sampled simultaneously by ADC.

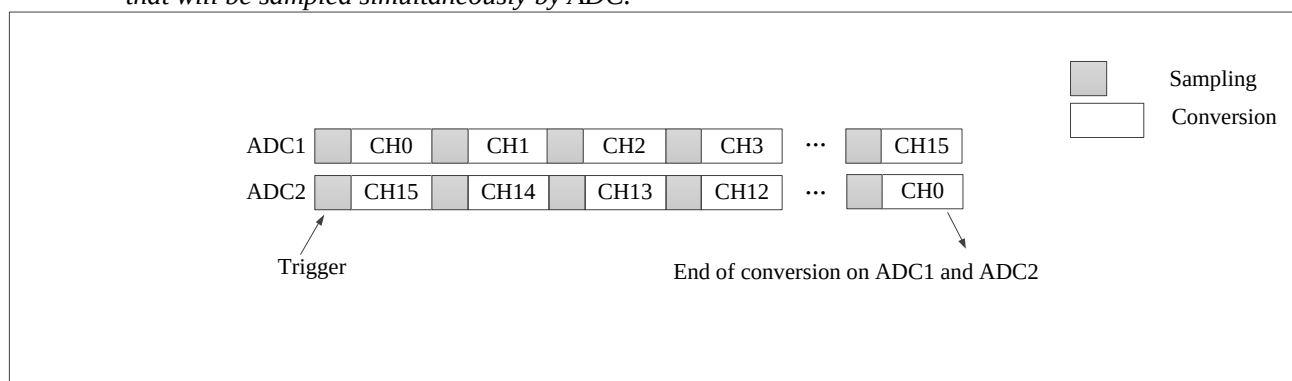


Figure 28 Regular simultaneous mode on 16 channels

10.9.3 Fast interleaved mode

This mode can be started only on a regular channel group (usually one channel). The source of external trigger comes from the regular channel mux of ADC1. After an external trigger occurs:

- ADC2 starts immediately and

- ADC1 starts after a delay of 7 ADC clock cycles.

If CONT bit is set on both ADC1 and ADC2 the selected regular channels of both ADCs are continuously converted.

After an EOC interrupt is generated by ADC1 (if enabled through the EOCIE bit) a 32-bit DMA transfer request is generated (if the DMA bit is set) which transfers to SRAM the ADC1_DR 32-bit register containing the ADC2 converted data in the upper halfword and the ADC1 converted data in the lower halfword.

Note: The maximum sampling time allowed is $<7 \text{ ADCCLK}$ cycles to avoid the overlap between ADC sampling phases in the event that they convert the same channel.

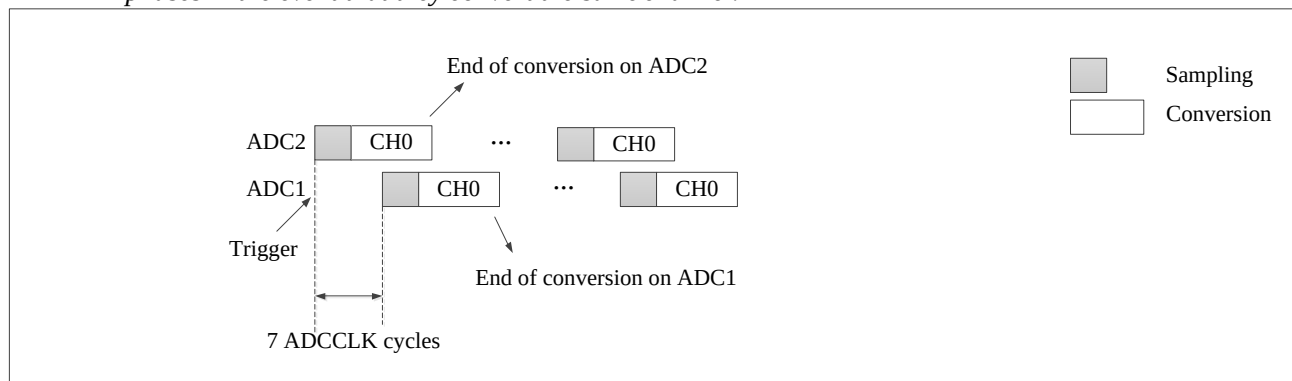


Figure 29 Fast interleaved mode on 1 channel in continuous conversion mode

10.9.4 Slow interleaved mode

This mode can be started only on a regular channel group (only one channel). The source of external trigger comes from regular channel mux of ADC1. After external trigger occurs:

- ADC2 starts immediately and
- ADC1 starts after a delay of 14 ADC clock cycles.
- ADC2 starts after a second delay of 14 ADC cycles, and so on.

Note: The maximum sampling time allowed is $<14 \text{ ADCCLK}$ cycles to avoid an overlap with the next conversion.

After an EOC interrupt is generated by ADC1 (if enabled through the EOCIE bit) a 32-bit DMA transfer request is generated (if the DMA bit is set) which transfers to SRAM the ADC1_DR 32-bit register containing the ADC2 converted data in the upper halfword and the ADC1 converted data in the lower halfword.

A new ADC2 start is automatically generated after 28 ADC clock cycles

CONT bit can not be set in the mode since it continuously converts the selected regular channel.

Note: The application must ensure that no external trigger for injected channel occurs when interleaved mode is enabled.

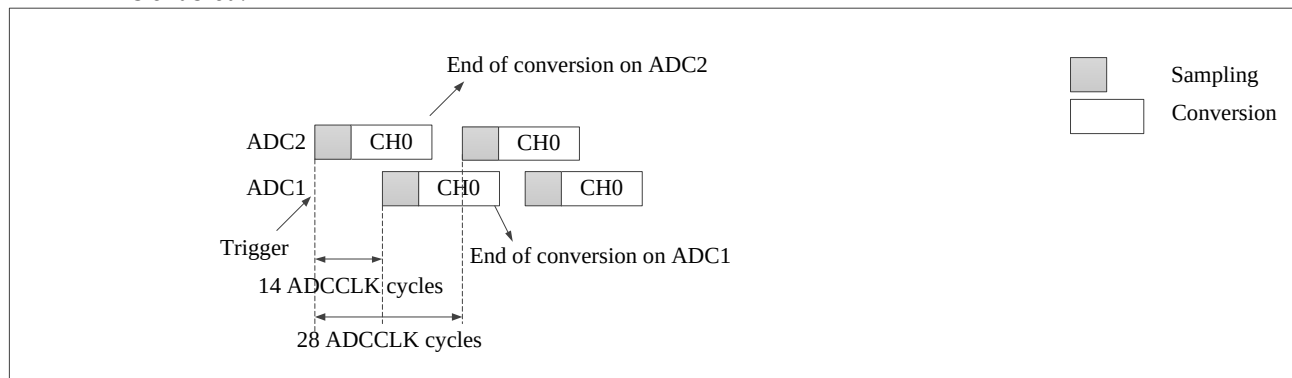


Figure 30 Slow interleaved mode on 1 channel

10.9.5 Alternate trigger mode

This mode can be started only on an injected channel group. The source of external trigger comes from the injected group mux of ADC1.

- When the 1st trigger occurs, all injected group channels in ADC1 are converted.
- When the 2nd trigger arrives, all injected group channels in ADC2 are converted.
- and so on.

A JEOC interrupt, if enabled, is generated after all injected group channels of ADC1 are converted.

A JEOC interrupt, if enabled, is generated after all injected group channels of ADC2 are converted.

If another external trigger occurs after all injected group channels have been converted then the alternate trigger process restarts by converting ADC1 injected group channels.

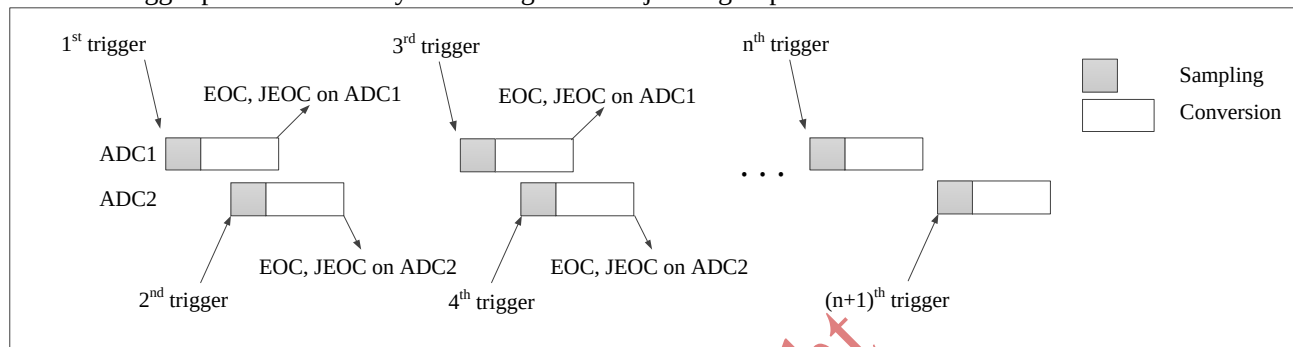


Figure 31 Alternate trigger: injected channel group of each ADC

If the injected discontinuous mode is enabled for both ADC1 and ADC2:

- When the 1st trigger occurs, the first injected channel in ADC1 is converted.
- When the 2nd trigger arrives, the first injected channel in ADC2 are converted.
- and so on....

A JEOC interrupt, if enabled, is generated after all injected group channels of ADC1 are converted.

A JEOC interrupt, if enabled, is generated after all injected group channels of ADC2 are converted.

If another external trigger occurs after all injected group channels have been converted then the alternate trigger process restarts.

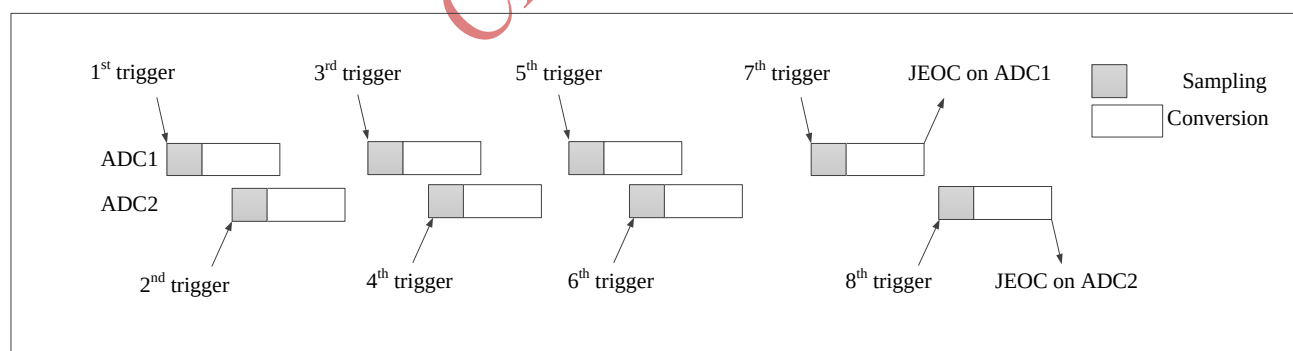


Figure 32 Alternate trigger: 4 injected channels (each ADC) in discontinuous model

10.9.6 Independent mode

In this mode the dual ADC synchronization is bypassed and each ADC interfaces works independently.

10.9.7 Combined regular/injected simultaneous mode

It is possible to interrupt simultaneous conversion of a regular group to start simultaneous conversion of an injected group.

Note: In combined regular/injected simultaneous mode, exactly the same sampling time should be configured for the two channels that will be sampled simultaneously by ADC.

10.9.8 Combined regular simultaneous + alternate trigger mode

It is possible to interrupt regular group simultaneous conversion to start alternate trigger conversion of an injected group. Figure 33 shows the behavior of an alternate trigger interrupting a regular simultaneous conversion.

The injected alternate conversion is immediately started after the injected event arrives. If regular conversion is already running, in order to ensure synchronization after the injected conversion, the regular conversion of both (master/slave) ADCs is stopped and resumed synchronously at the end of the injected conversion.

Note: In combined regular simultaneous + alternate trigger mode, exactly the same sampling time should be configured for the two channels that will be sampled simultaneously by ADC.

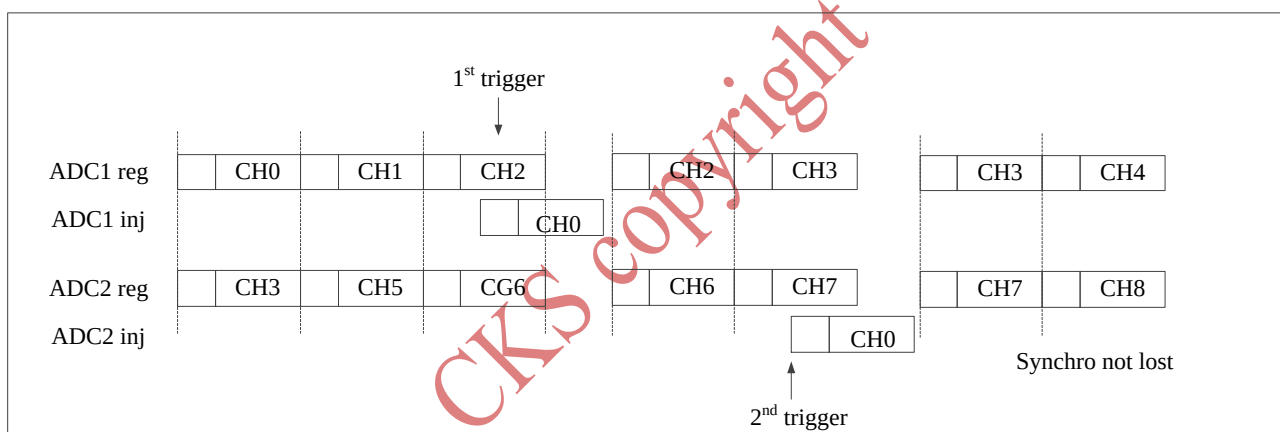


Figure 33 Alternate + Regular simultaneous

If a trigger occurs during an injected conversion that has interrupted a regular conversion, it will be ignored. Figure 34 shows the behavior in this case (the second trigger is ignored).

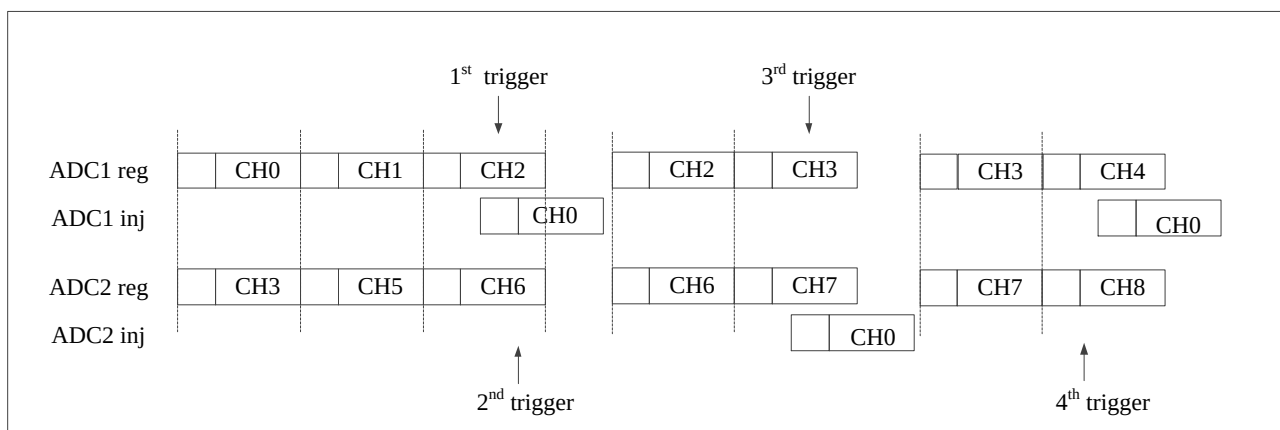


Figure 34 Case of trigger occurring during injected conversion

10.9.9 Combined injected simultaneous + interleaved

It is possible to interrupt an interleaved conversion with an injected event. In this case the interleaved conversion is interrupted and the injected conversion starts, at the end of the injected sequence the interleaved conversion is resumed. Figure 35 shows the behavior using an example.

Note: When the ADC clock prescaler is set to 4, the interleaved mode does not recover with evenly spaced sampling periods: the sampling interval is 8 ADC clock periods followed by 6 ADC clock periods, instead of 7 clock periods followed by 7 clock periods.

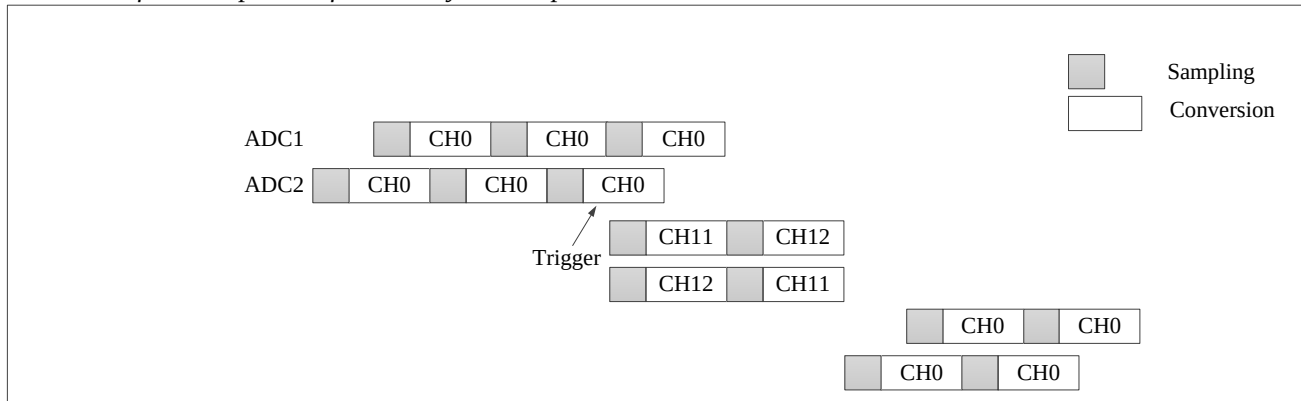


Figure 35 Interleaved single channel with injected sequence CH11, CH12

10.10 Temperature sensor

The temperature sensor can be used to measure the ambient temperature (T_A) of the device.

The temperature sensor is internally connected to the ADCx_IN16 input channel which is used to convert the sensor output voltage into a digital value. The recommended sampling time for the temperature sensor is 17.1 μ s.

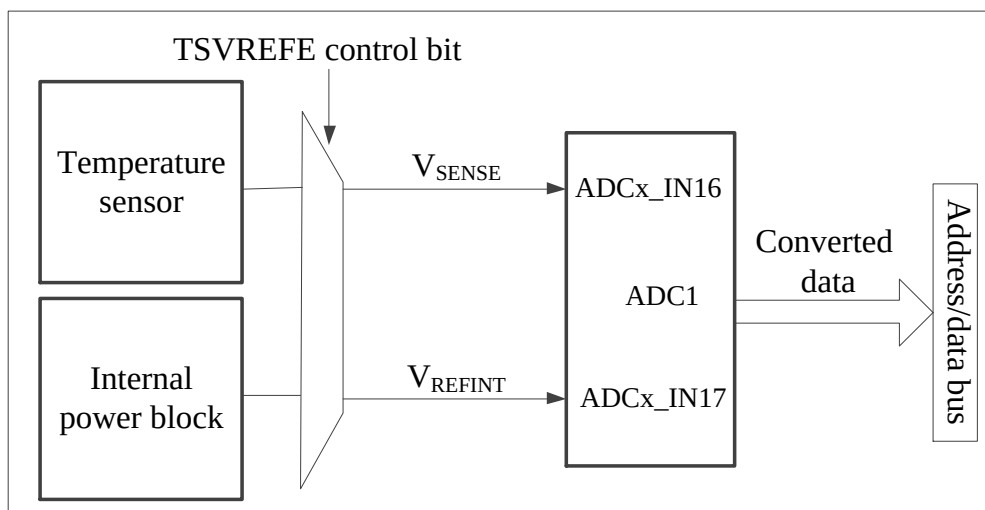
The block diagram of the temperature sensor is shown in Figure 36.

When not in use, this sensor can be put in power down mode.

Note: The TSVREFE bit must be set to enable both internal channels: ADCx_IN16 (temperature sensor) and ADCx_IN17 (V_{REFINT}) conversion.

The temperature sensor output voltage changes linearly with temperature. The offset of this line varies from chip to chip due to process variation (up to 45 $^{\circ}$ C from one chip to another).

The internal temperature sensor is more suited to applications that detect temperature variations instead of absolute temperatures. If accurate temperature readings are needed, an external temperature sensor part should be used.

Figure 36 Temperature sensor and V_{REFINT} channel block diagram

Reading the temperature

To use the sensor:

1. Select the ADCx_IN16 input channel.
2. Select a sample time of 17.1 μ s
3. Set the TSVREFE bit in the ADC control register 2 (ADC_CR2) to wake up the temperature sensor from power down mode.
4. Start the ADC conversion by setting the ADON bit (or by external trigger).
5. Read the resulting V_{SENSE} data in the ADC data register
6. Obtain the temperature using the following formula:

$$\text{Temperature (in } ^\circ\text{C)} = \{(V_{25} - V_{\text{SENSE}}) / \text{Avg_Slope}\} + 25.$$

Where,

V₂₅ = V_{SENSE} value for 25 $^\circ$ C and

Avg_Slope = Average Slope for curve between Temperature vs. V_{SENSE} (given in mV/ $^\circ$ C or μ V/ $^\circ$ C).

Refer to the Electrical characteristics section for the actual values of V₂₅ and Avg_Slope.

Note: The sensor has a startup time after waking from power down mode before it can output V_{SENSE} at the correct level. The ADC also has a startup time after power-on, so to minimize the delay, the ADON and TSVREFE bits should be set at the same time.

10.11 ADC interrupts

An interrupt can be produced on end of conversion for regular and injected groups and when the analog watchdog status bit is set. Separate interrupt enable bits are available for flexibility.

Note: ADC1 and ADC2 interrupts are mapped onto the same interrupt vector. ADC3 interrupts are mapped onto a separate interrupt vector.

Two other flags are present in the ADC_SR register, but there is no interrupt associated with them:

- JSTRT (Start of conversion for injected group channels)
- STRT (Start of conversion for regular group channels)

Table 50 ADC interrupts

Interrupt event	Event flag	Enable Control bit
End of conversion regular group	EOC	EOCIE
End of conversion injected group	JEOC	JEOCIE
Analog watchdog status bit is set	AWD	AWDIE

10.12 ADC registers

Refer to *Section 1.1* for a list of abbreviations used in register descriptions.
The peripheral registers have to be accessed by words (32-bit).

10.12.1 ADC status register (ADC_SR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved											STRT	JSTRT	JEOC	EOC	AWD
											rc w0	rc w0	rc w0	rc w0	rc w0
Bits 31:5		Reserved, must be kept at reset value.													
Bit 4		STRT: Regular channel Start flag This bit is set by hardware when regular channel conversion starts. It is cleared by software. 0: No regular channel conversion started 1: Regular channel conversion has started													
Bit 3		JSTRT: Injected channel Start flag This bit is set by hardware when injected channel group conversion starts. It is cleared by software. 0: No injected group conversion started 1: Injected group conversion has started													
Bit 2		JEOC: Injected channel end of conversion This bit is set by hardware at the end of all injected group channel conversion. It is cleared by software. 0: Conversion is not complete 1: Conversion complete													
Bit 1		EOC: End of conversion This bit is set by hardware at the end of a group channel conversion (regular or injected). It is cleared by software or by reading the ADC_DR. 0: Conversion is not complete 1: Conversion complete													
Bit 0		AWD: Analog watchdog flag This bit is set by hardware when the converted voltage crosses the values programmed in the ADC_LTR and ADC_HTR registers. It is cleared by software. 0: No Analog watchdog event occurred 1: Analog watchdog event occurred													

10.12.2 ADC control register 1 (ADC_CR1)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								AWD EN	JAWD EN	Reserved		DUALMOD[3: 0]			
								rw	rw			rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DISCNUM[2:0]		JDICS EN	DICS EN	JAUTO	AWD SGL	SCAN	JEOC IE	AWDIE	EOCIE	AWDCH[4:0]					
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:24		Reserved, must be kept at reset value.													
Bit 23		AWDEN: Analog watchdog enable on regular channels This bit is set/reset by software. 0: Analog watchdog disabled on regular channels													

	1: Analog watchdog enabled on regular channels
Bit 22	JAWDEN: Analog watchdog enable on injected channels This bit is set/reset by software. 0: Analog watchdog disabled on injected channels 1: Analog watchdog enabled on injected channels
Bits 21:20	Reserved, must be kept at reset value.
Bits 19:16	DUALMOD[3:0]: Dual mode selection These bits are written by software to select the operating mode. 0000: Independent mode. 0001: Combined regular simultaneous + injected simultaneous mode 0010: Combined regular simultaneous + alternate trigger mode 0011: Combined injected simultaneous + fast interleaved mode 0100: Combined injected simultaneous + slow Interleaved mode 0101: Injected simultaneous mode only 0110: Regular simultaneous mode only 0111: Fast interleaved mode only 1000: Slow interleaved mode only 1001: Alternate trigger mode only <i>Note: These bits are reserved in ADC2 and ADC3. In dual mode, a change of channel configuration generates a restart that can produce a loss of synchronization. It is recommended to disable dual mode before any configuration change.</i>
Bits 15:13	DISCNUM[2:0]: Discontinuous mode channel count These bits are written by software to define the number of regular channels to be converted in discontinuous mode, after receiving an external trigger. 000: 1 channel 001: 2 channels 111: 8 channels
Bit 12	JDISCEN: Discontinuous mode on injected channels This bit set and cleared by software to enable/disable discontinuous mode on injected group channels 0: Discontinuous mode on injected channels disabled 1: Discontinuous mode on injected channels enabled
Bit 11	DISCEN: Discontinuous mode on regular channels This bit set and cleared by software to enable/disable Discontinuous mode on regular channels. 0: Discontinuous mode on regular channels disabled 1: Discontinuous mode on regular channels enabled
Bit 10	JAUTO: Automatic Injected Group conversion This bit set and cleared by software to enable/disable automatic injected group conversion after regular group conversion. 0: Automatic injected group conversion disabled 1: Automatic injected group conversion enabled
Bit 9	AWDSGL: Enable the watchdog on a single channel in scan mode This bit set and cleared by software to enable/disable the analog watchdog on the channel identified by the AWDCH[4:0] bits. 0: Analog watchdog enabled on all channels 1: Analog watchdog enabled on a single channel
Bit 8	SCAN: Scan mode This bit is set and cleared by software to enable/disable Scan mode. In Scan mode, the inputs selected through the ADC_SQRx or ADC_JSQRx registers are converted. 0: Scan mode disabled 1: Scan mode enabled <i>Note: An EOC or JEOC interrupt is generated only on the end of conversion of the last channel if the corresponding EOCIE or JEOCIE bit is set.</i>
Bit 7	JEOCIE: Interrupt enable for injected channels This bit is set and cleared by software to enable/disable the end of conversion interrupt for injected channels. 0: JEOC interrupt disabled 1: JEOC interrupt enabled. An interrupt is generated when the JEOC bit is set.
Bit 6	AWDIE: Analog watchdog interrupt enable This bit is set and cleared by software to enable/disable the analog watchdog interrupt. 0: Analog watchdog interrupt disabled 1: Analog watchdog interrupt enabled
Bit 5	EOCIE: Interrupt enable for EOC This bit is set and cleared by software to enable/disable the End of Conversion interrupt. 0: EOC interrupt disabled 1: EOC interrupt enabled. An interrupt is generated when the EOC bit is set.

Bits 4:0	AWDCH[4:0]: Analog watchdog channel select bits These bits are set and cleared by software. They select the input channel to be guarded by the Analog watchdog. 00000: ADC analog Channel0 00001: ADC analog Channel1 01111: ADC analog Channel15 10000: ADC analog Channel16 10001: ADC analog Channel17 Other values reserved. <i>Note: ADC1 analog Channel16 and Channel17 are internally connected to the temperature sensor and to V_{REFINT}, respectively.</i> <i>ADC2 analog inputs Channel16 and Channel17 are internally connected to V_{SS}.</i> <i>ADC3 analog inputs Channel9, Channel14, Channel15, Channel16 and Channel17 are connected to V_{SS}.</i>
----------	---

10.12.3 ADC control register 2 (ADC_CR2)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16				
Reserved								TS WREFE	SW START	JSW START	EXT TRIG	EXTSEL[2: 0]			Res				
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
JEXT TRIG	JEXTSEL [2: 0]			ALIGN	Res		DMA	Reserved				RST CAL	CAL	CONT	ADON				
rw	rw	rw	rw	rw			rw					rw	rw	rw	rw				
Bits 31:24		Reserved, must be kept at reset value.																	
Bit 23		TSVREFE: Temperature sensor and V _{REFINT} enable This bit is set and cleared by software to enable/disable the temperature sensor and V _{REFINT} channel. In devices with dual ADCs this bit is present only in ADC1. 0: Temperature sensor and V _{REFINT} channel disabled 1: Temperature sensor and V _{REFINT} channel enabled																	
Bit 22		SWSTART: Start conversion of regular channels This bit is set by software to start conversion and cleared by hardware as soon as conversion starts. It starts a conversion of a group of regular channels if SWSTART is selected as trigger event by the EXTSEL[2:0] bits. 0: Reset state 1: Starts conversion of regular channels																	
Bit 21		JSWSTART: Start conversion of injected channels This bit is set by software and cleared by software or by hardware as soon as the conversion starts. It starts a conversion of a group of injected channels (if JSWSTART is selected as trigger event by the JEXTSEL[2:0] bits). 0: Reset state 1: Starts conversion of injected channels																	
Bit 20		EXTTRIG: External trigger conversion mode for regular channels This bit is set and cleared by software to enable/disable the external trigger used to start conversion of a regular channel group. 0: Conversion on external event disabled 1: Conversion on external event enabled																	
Bits 19:17		EXTSEL[2:0]: External event select for regular group These bits select the external event used to trigger the start of conversion of a regular group: For ADC1 and ADC2, the assigned triggers are: 000: Timer 1 CC1 event 001: Timer 1 CC2 event 010: Timer 1 CC3 event 011: Timer 2 CC2 event 100: Timer 3 TRGO event 101: Timer 4 CC4 event 110: EXTI line 11 event																	

	111: SWSTART For ADC3, the assigned triggers are: 000: Timer 3 CC1 event 001: Timer 2 CC3 event 010: Timer 1 CC3 event 111: SWSTART
Bit 16	Reserved, must be kept at reset value.
Bit 15	JEXTTRIG : External trigger conversion mode for injected channels This bit is set and cleared by software to enable/disable the external trigger used to start conversion of an injected channel group. 0: Conversion on external event disabled 1: Conversion on external event enabled
Bits 14:12	JEXTSEL[2:0] : External event select for injected group These bits select the external event used to trigger the start of conversion of an injected group: For ADC1 and ADC2 the assigned triggers are: 000: Timer 1 TRGO event 001: Timer 1 CC4 event 010: Timer 2 TRGO event 011: Timer 2 CC1 event 100: Timer 3 CC4 event 101: Timer 4 TRGO event 110: EXTI line15 event 111: JSWSTART For ADC3 the assigned triggers are: 000: Timer 1 TRGO event 001: Timer 1 CC4 event 010: Timer 4 CC3 event 111: JSWSTART
Bit 11	ALIGN : Data alignment This bit is set and cleared by software. Refer to Figure 24 and Figure 25. 0: Right Alignment 1: Left Alignment
Bits 10:9	Reserved, must be kept at reset value.
Bit 8	DMA : Direct memory access mode This bit is set and cleared by software. Refer to the DMA controller chapter for more details. 0: DMA mode disabled 1: DMA mode enabled Only ADC1 and ADC3 can generate a DMA request.
Bits 7:4	Reserved, must be kept at reset value.
Bit 3	RSTCAL : Reset calibration This bit is set by software and cleared by hardware. It is cleared after the calibration registers are initialized. 0: Calibration register initialized. 1: Initialize calibration register. <i>Note: If RSTCAL is set when conversion is ongoing, additional cycles are required to clear the calibration registers.</i>
Bit 2	CAL : A/D Calibration This bit is set by software to start the calibration. It is reset by hardware after calibration is complete. 0: Calibration completed 1: Enable calibration
Bit 1	CONT : Continuous conversion This bit is set and cleared by software. If set conversion takes place continuously till this bit is reset. 0: Single conversion mode 1: Continuous conversion mode
Bit 0	ADON : A/D converter ON / OFF This bit is set and cleared by software. If this bit holds a value of zero and a 1 is written to it then it wakes up the ADC from Power Down state. Conversion starts when this bit holds a value of 1 and a 1 is written to it. The application should allow a delay of t_{STAB} between power up and start of conversion. Refer to figure 20. 0: Disable ADC conversion/calibration and go to power down mode. 1: Enable ADC and to start conversion <i>Note: If any other bit in this register apart from ADON is changed at the same time, then conversion is not triggered. This is to prevent triggering an erroneous conversion.</i>

10.12.4 ADC sample time register 1 (ADC_SMPR1)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								SMP17[2:0]			SMP16[2:0]			SMP15[2:1]	
15	14	13	12	11	10	9	8	rw	rw	rw	rw	rw	rw	rw	rw
SMP15_0	SMP14[2:0]			SMP13[2:0]			SMP12[2:0]			SMP11[2:0]			SMP10[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:24		Reserved, must be kept at reset value.													
Bits 23:0		SMPx[2:0]: Channel x Sample time selection These bits are written by software to select the sample time individually for each channel. During sample cycles channel selection bits must remain unchanged. 000: 1.5 cycles 001: 7.5 cycles 010: 13.5 cycles 011: 28.5 cycles 100: 41.5 cycles 101: 55.5 cycles 110: 71.5 cycles 111: 239.5 cycles <i>Note: ADC1 analog Channel16 and Channel 17 are internally connected to the temperature sensor and to V_{REFINT}, respectively.</i> <i>ADC2 analog input Channel16 and Channel17 are internally connected to V_{SS}.</i> <i>ADC3 analog inputs Channel14, Channel15, Channel16 and Channel17 are connected to V_{SS}.</i>													

10.12.5 ADC sample time register 2 (ADC_SMPR2)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SMP9[2:0]			SMP8[2:0]			SMP7[2:0]			SMP6[2:0]			SMP5[2:1]	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMP5_0	SMP4[2:0]			SMP3[2:0]			SMP2[2:0]			SMP1[2:0]			SMP0[2:0]		
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
		Bits 31:30		Reserved, must be kept at reset value.											
		Bits 23:0		SMPx[2:0]: Channel x Sample time selection These bits are written by software to select the sample time individually for each channel. During sample cycles channel selection bits must remain unchanged. 000: 1.5 cycles 001: 7.5 cycles 010: 13.5 cycles 011: 28.5 cycles 100: 41.5 cycles 101: 55.5 cycles 110: 71.5 cycles 111: 239.5 cycles <i>Note: ADC3 analog input Channel9 is connected to V_{SS}.</i>											

10.12.6 ADC injected channel data offset register x (ADC_JOFRx)

(x=1..4)

Address offset: 0x14-0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				JOFFSETx[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:12				Reserved, must be kept at reset value.											
Bits 11:0				JOFFSETx[11:0]: Data offset for injected channel x These bits are written by software to define the offset to be subtracted from the raw converted data when converting injected channels. The conversion result can be read from in the ADC_JDRx registers.											

10.12.7 ADC watchdog high threshold register (ADC_HTR)

Address offset: 0x24

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				HT[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:12				Reserved, must be kept at reset value.											
Bits 11:0				HT[11:0]: Analog watchdog high threshold These bits are written by software to define the high threshold for the analog watchdog.											

10.12.8 ADC watchdog low threshold register (ADC_LTR)

Address offset: 0x28

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				LTR[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:12				Reserved, must be kept at reset value.											
Bits 11:0				LT[11:0]: Analog watchdog low threshold These bits are written by software to define the low threshold for the analog watchdog.											

10.12.9 ADC regular sequence register 1 (ADC_SQR1)

Address offset: 0x2C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								L[3:0]				SQ16[4:1]			
								rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ16 _0	SQ15[4:0]					SQ14[4:0]					SQ13[4:0]				
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 31:24		Reserved, must be kept at reset value.													
Bits 23:20		L[3:0] : Regular channel sequence length These bits are written by software to define the total number of conversions in the regular channel conversion sequence. 0000: 1 conversion 0001: 2 conversions 1111: 16 conversions													
Bits 19:15		SQ16[4:0] : 16th conversion in regular sequence These bits are written by software with the channel number (0..17) assigned as the 16th in the conversion sequence.													
Bits 14:10		SQ15[4:0] : 15th conversion in regular sequence													
Bits 9:5		SQ14[4:0] : 14th conversion in regular sequence													
Bits 4:0		SQ13[4:0] : 13th conversion in regular sequence													

10.12.10 ADC regular sequence register 2 (ADC_SQR2)

Address offset: 0x30

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SQ12[4:0]					SQ11[4:0]					SQ10[4:1]			
		rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ10 _0	SQ9[4:0]					SQ8[4:0]					SQ7[4:0]				
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 31:30		Reserved, must be kept at reset value.													
Bits 29:25		SQ12[4:0] : 12th conversion in regular sequence These bits are written by software with the channel number (0..17) assigned as the 12th in the sequence to be converted.													
Bits 24:20		SQ11[4:0] : 11th conversion in regular sequence													
Bits 19:15		SQ10[4:0] : 10th conversion in regular sequence													
Bits 14:10		SQ9[4:0] : 9th conversion in regular sequence													
Bits 9:5		SQ8[4:0] : 8th conversion in regular sequence													
Bits 4:0		SQ7[4:0] : 7th conversion in regular sequence													

10.12.11 ADC regular sequence register 3 (ADC_SQR3)

Address offset: 0x34

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		SQ6[4:0]					SQ5[4:0]					SQ4[4:1]			
		rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ4 _0	SQ3[4:0]					SQ2[4:0]					SQ1[4:0]				
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 31:30		Reserved, must be kept at reset value.													
Bits 29:25		SQ6[4:0] : 6th conversion in regular sequence These bits are written by software with the channel number (0..17) assigned as the 6th in the sequence to be converted.													
Bits 24:20		SQ5[4:0] : 5th conversion in regular sequence													
Bits 19:15		SQ4[4:0] : 4th conversion in regular sequence													
Bits 14:10		SQ3[4:0] : 3rd conversion in regular sequence													
Bits 9:5		SQ2[4:0] : 2nd conversion in regular sequence													

Bits 4:0	SQ1[4:0] : 1st conversion in regular sequence
----------	--

10.12.12 ADC injected sequence register (ADC_JSQR)

Address offset: 0x38

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved										JL[3:0]		JSQ4[4:1]			
15	14	13	12	11	10	9	8	7	6	rw	rw	rw	rw	rw	rw
JSQ4_0	JSQ3[4:0]					JSQ2[4:0]					5	4	3	2	1
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:22		Reserved, must be kept at reset value.													
Bits 21:20		JL[1:0] : Injected sequence length These bits are written by software to define the total number of conversions in the injected channel conversion sequence. 00: 1 conversion 01: 2 conversions 10: 3 conversions 11: 4 conversions													
Bits 19:15		JSQ4[4:0] : 4th conversion in injected sequence (when JL[1:0] = 3) These bits are written by software with the channel number (0..17) assigned as the 4th in the sequence to be converted. <i>Note: Unlike a regular conversion sequence, if JL[1:0] length is less than four, the channels are converted in a sequence starting from (4-JL). Example: ADC_JSQR[21:0] = 10 00011 00011 00111 00010 means that a scan conversion will convert the following channel sequence: 7, 3, 3. (not 2, 7, 3)</i>													
Bits 14:10		JSQ3[4:0] : 3rd conversion in injected sequence (when JL[1:0] = 3)													
Bits 9:5		JSQ2[4:0] : 2nd conversion in injected sequence (when JL[1:0] = 3)													
Bits 4:0		SQ1[4:0] : 1st conversion in injected sequence (when JL[1:0] = 3)													

10.12.13 ADC injected data register x (ADC_JDRx) (x= 1..4)

Address offset: 0x3C – 0x48

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JDATA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 31:16		Reserved, must be kept at reset value.													
Bits 21:20		JDATA[15:0] : Injected data These bits are read only. They contain the conversion result from injected channel x. The data is left or right-aligned as shown in Figure 24 and Figure 25.													

10.12.14 ADC regular data register (ADC_DR)

Address offset: 0x4C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADC2DATA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16	ADC2DATA[15:0]: ADC2 data In ADC1: In dual mode, these bits contain the regular data of ADC2. Refer to <i>Section 10.9: Dual ADC mode</i> . In ADC2 and ADC3: these bits are not used.
Bits 15:0	DATA[15:0]: Regular data These bits are read only. They contain the conversion result from the regular channels. The data is left or right-aligned as shown in Figure 24 and Figure 25.

CKS copyright

10.12.15 ADC register map

The following table summarizes the ADC registers.

Table 51 ADC register map and reset values

Offset	00h		04h		08h		0Ch		10h
Register	Reset value	ADC_SR	Reset value	ADC_CR1	Reset value	ADC_CR2	Reset value	ADC_SMPR1	ADC_SMPR2
31	Reserved				Reserved	Reserved	Reserved	Sample time bits SMPx_x	Sample time bits SMPx_x
30									
29									
28									
27									
26									
25									
24									
23									
22									
21									
20									
19									
18									
17									
16									
15									
14									
13									
12									
11									
10									
9									
8									
7									
6									
5									
4	0	STRT	AWDCH [4:0]	Reserved	Reserved	Reserved	Reserved	Reserved	Reserved
3	0	JSTRT							
2	0	JEOC							
1	0	EOC							
0	0	AWD							

	Reset value	00																																			
14h	ADC_CR1	Reserved																								JOFFSET1[11:0]											
	Reset value																									00000000000000											
18h	ADC_CR2	Reserved																								JOFFSET2[11:0]											
	Reset value																									00000000000000											
1Ch	ADC_CR3	Reserved																								JOFFSET3[11:0]											
	Reset value																									00000000000000											
20h	ADC_CR4	Reserved																								JOFFSET4[11:0]											
	Reset value																									00000000000000											
2Ch	ADC_SQR1	Reserved												L[3:0]				SQXx_x conversion in regular sequence bits																			
	Reset value													00																							
30h	ADC_SQR2	Reserved	SQXx_x conversion in regular sequence bits																																		

[illegible]

[illegible]

Refer to Table 1 for the register boundary addresses.

11 Advanced-control timer (TIM1)

11.1 TIM1 introduction

The advanced-control timer (TIM1) consist of a 16-bit auto-reload counter driven by a programmable prescaler.

It may be used for a variety of purposes, including measuring the pulse lengths of input signals (input capture) or generating output waveforms (output compare, PWM, complementary PWM with dead-time insertion).

Pulse lengths and waveform periods can be modulated from a few microseconds to several milliseconds using the timer prescaler and the RCC clock controller prescalers.

The advanced-control (TIM1) and general-purpose (TIMx) timers are completely independent, and do not share any resources. They can be synchronized together as described in *Section 11.3.20*.

11.2 TIM1 main features

TIM1 timer features include:

- 16-bit up, down, up/down auto-reload counter.
- 16-bit programmable prescaler allowing dividing (also “on the fly”) the counter clock frequency either by any factor between 1 and 65536.
- Up to 4 independent channels for:
 - Input Capture
 - Output Compare
 - PWM generation (Edge and Center-aligned Mode)
 - One-pulse mode output
- Complementary outputs with programmable dead-time
- Synchronization circuit to control the timer with external signals and to interconnect several timers together.
- Repetition counter to update the timer registers only after a given number of cycles of the counter.
- Break input to put the timer’s output signals in reset state or in a known state.
- Interrupt/DMA generation on the following events:
 - Update: counter overflow/underflow, counter initialization (by software or internal/external trigger)
 - Trigger event (counter start, stop, initialization or count by internal/external trigger)
 - Input capture
 - Output compare
 - Break input
- Supports incremental (quadrature) encoder and hall-sensor circuitry for positioning purposes
- Trigger input for external clock or cycle-by-cycle current management

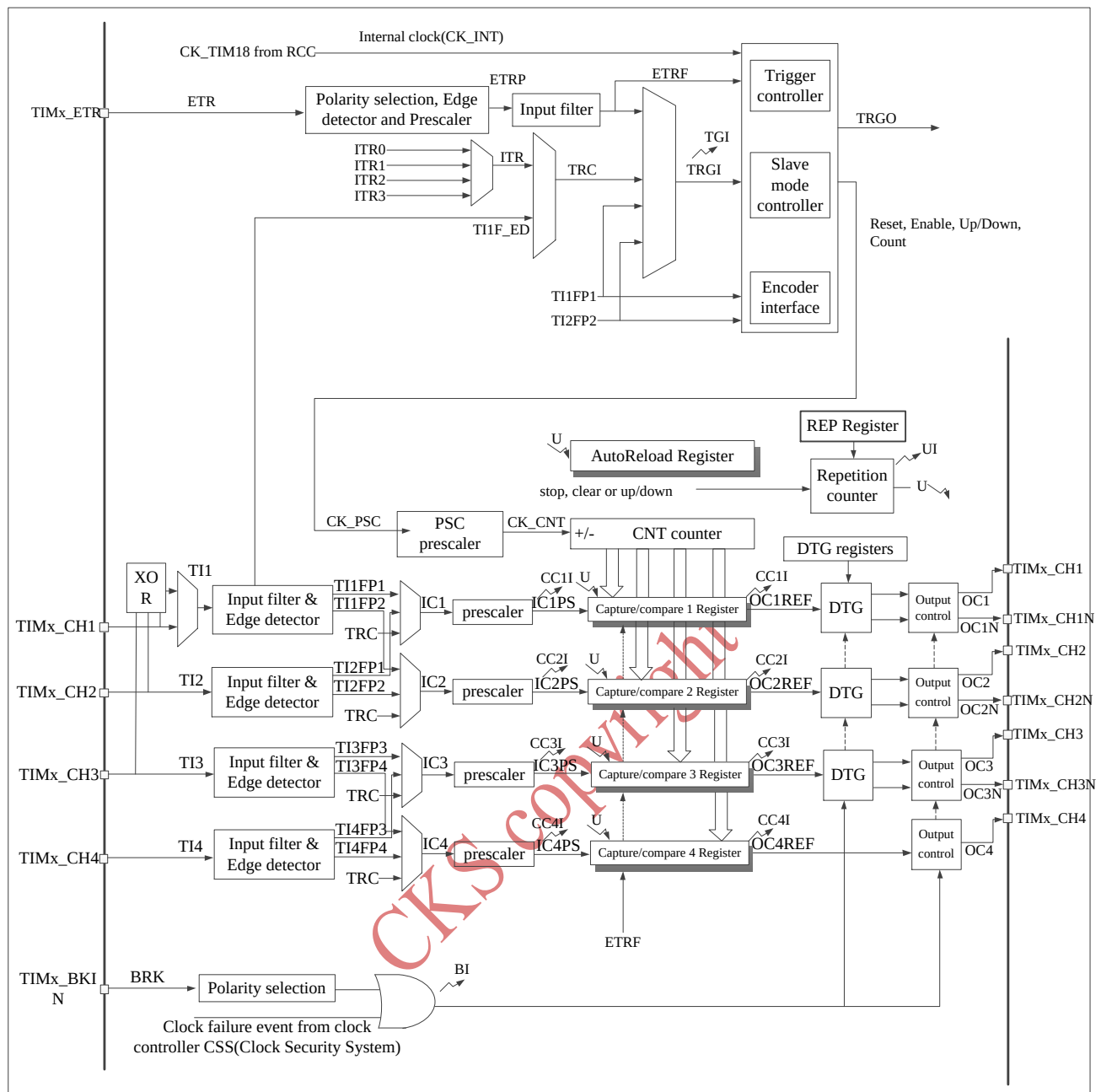


Figure 37 Advanced-control timer block diagram

11.3 TIM1 functional description

11.3.1 Time-base unit

The main block of the programmable advanced-control timer is a 16-bit counter with its related auto-reload register. The counter can count up, down or both up and down. The counter clock can be divided by a prescaler.

The counter, the auto-reload register and the prescaler register can be written or read by software. This is true even when the counter is running.

The time-base unit includes:

- Counter register (TIM1_CNT)
- Prescaler register (TIM1_PSC)
- Auto-reload register (TIM1_ARR)
- Repetition counter register (TIM1_RCR)

The auto-reload register is preloaded. Writing to or reading from the auto-reload register accesses the preload register. The content of the preload register are transferred into the shadow register permanently or at each update event (UEV), depending on the auto-reload preload enable bit (ARPE) in TIM1_CR1 register. The update event is sent when the counter reaches the overflow (or underflow when downcounting) and if the UDIS bit equals 0 in the TIM1_CR1 register. It can also be generated by software. The generation of the update event is described in detailed for each configuration.

The counter is clocked by the prescaler output CK_CNT, which is enabled only when the counter enable bit (CEN) in TIM1_CR1 register is set (refer also to the slave mode controller description to get more details on counter enabling).

Note that the counter starts counting 1 clock cycle after setting the CEN bit in the TIM1_CR1 register.

Prescaler description

The prescaler can divide the counter clock frequency by any factor between 1 and 65536. It is based on a 16-bit counter controlled through a 16-bit register (in the TIM1_PSC register). It can be changed on the fly as this control register is buffered. The new prescaler ratio is taken into account at the next update event.

Figure 38 and *Figure 39* give some examples of the counter behavior when the prescaler ratio is changed on the fly:

CKS copyright

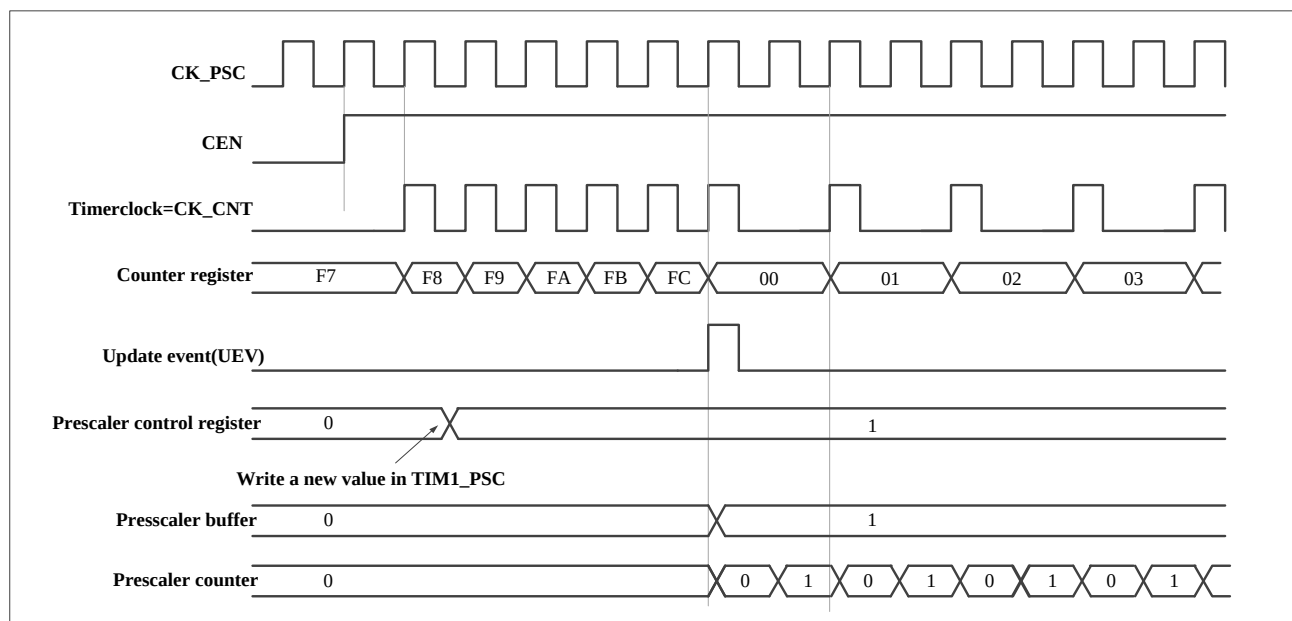


Figure 38 Counter timing diagram with prescaler division change from 1 to 2

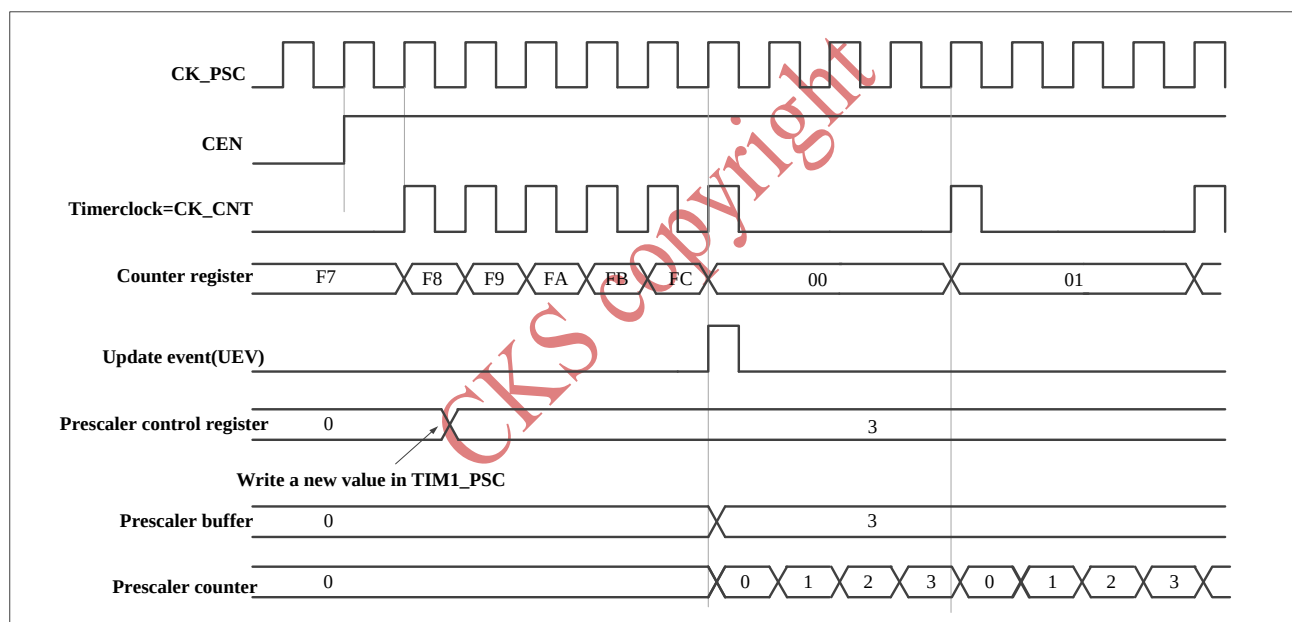


Figure 39 Counter timing diagram with prescaler division change from 1 to 4

11.3.2 Counter modes

Upcounting mode

In upcounting mode, the counter counts from 0 to the auto-reload value (content of the TIM1_ARR register), then restarts from 0 and generates a counter overflow event.

If the repetition counter is used, the update event (UEV) is generated after upcounting is repeated for the number of times programmed in the repetition counter register plus one (TIM1_RCR+1). Else the update event is generated at each counter overflow.

Setting the UG bit in the TIM1_EGR register (by software or by using the slave mode controller) also generates an update event.

The UEV event can be disabled by software by setting the UDIS bit in the TIM1_CR1 register. This is to avoid updating the shadow registers while writing new values in the preload registers. Then no update

event occurs until the UDIS bit has been written to 0. However, the counter restarts from 0, as well as the counter of the prescaler (but the prescale rate does not change). In addition, if the URS bit (update request selection) in TIM1_CR1 register is set, setting the UG bit generates an update event UEV but without setting the UIF flag (thus no interrupt or DMA request is sent). This is to avoid generating both update and capture interrupts when clearing the counter on the capture event.

When an update event occurs, all the registers are updated and the update flag (UIF bit in TIM1_SR register) is set (depending on the URS bit):

- The repetition counter is reloaded with the content of TIM1_RCR register,
- The auto-reload shadow register is updated with the preload value (TIM1_ARR),
- The buffer of the prescaler is reloaded with the preload value (content of the TIM1_PSC register).

The following figures show some examples of the counter behavior for different clock frequencies when TIM1_ARR=0x36.

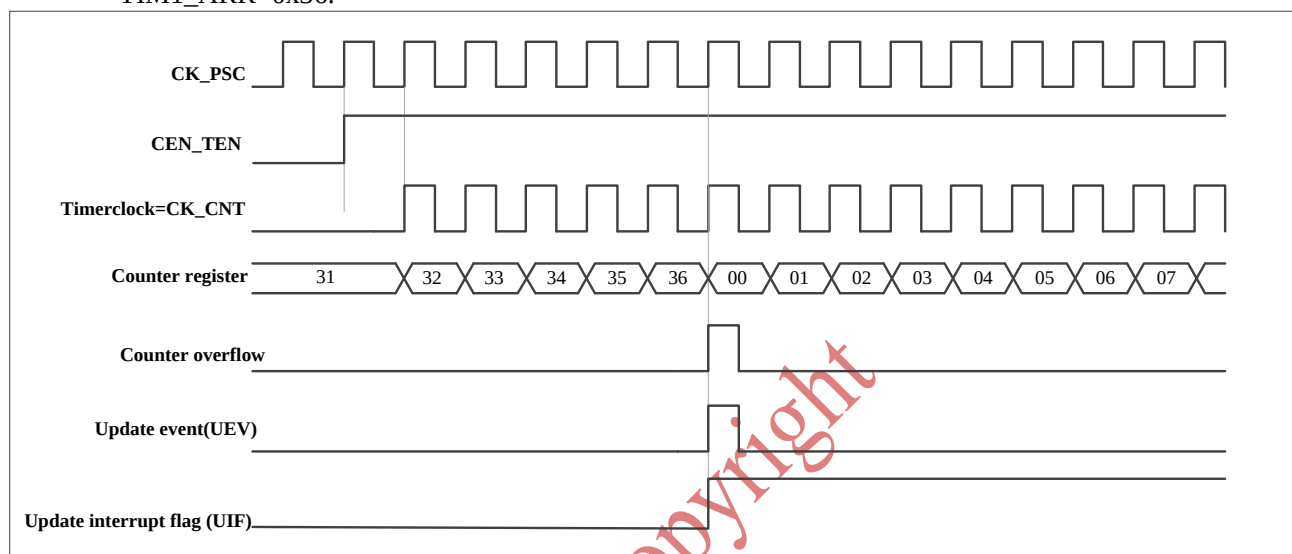


Figure 40 Counter timing diagram, internal clock divided by 1

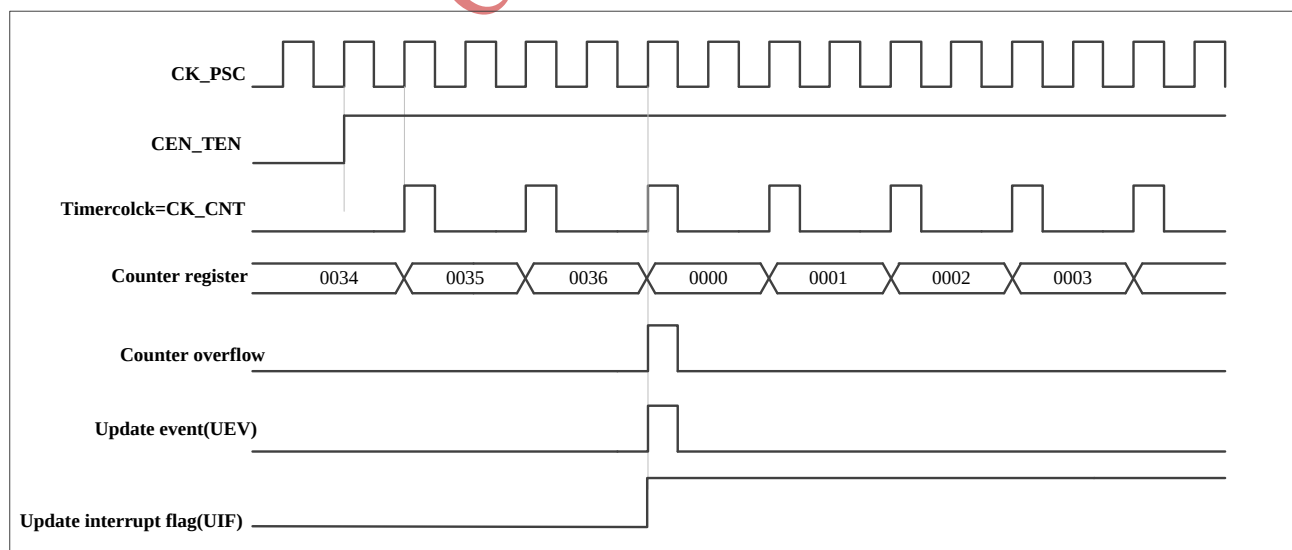


Figure 41 Counter timing diagram, internal clock divided by 2

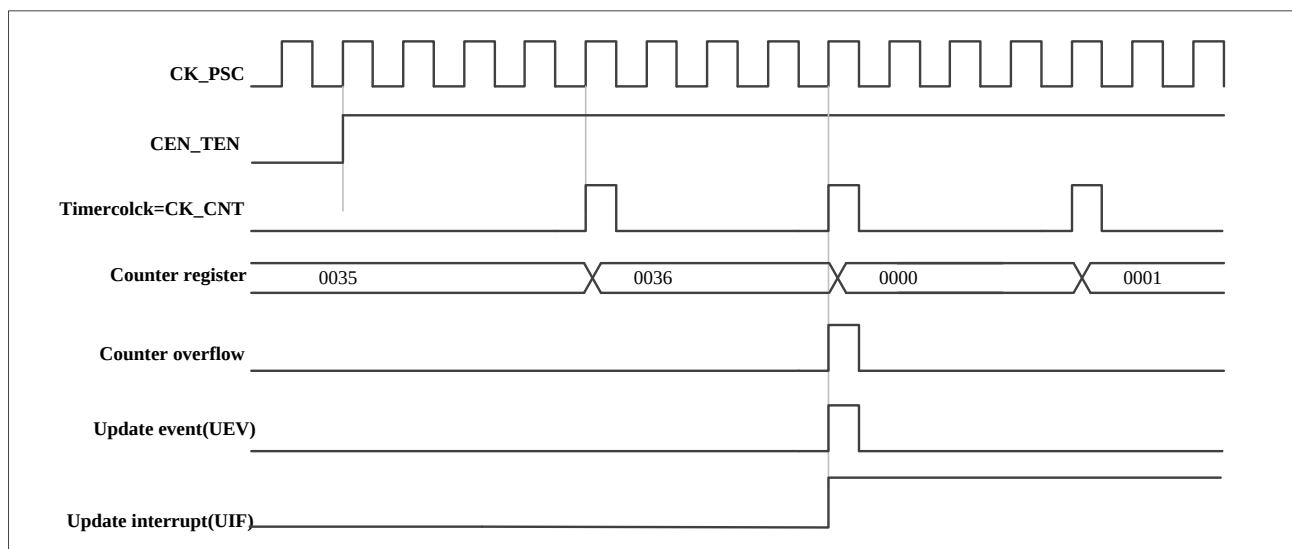


Figure 42 Counter timing diagram, internal clock divided by 4

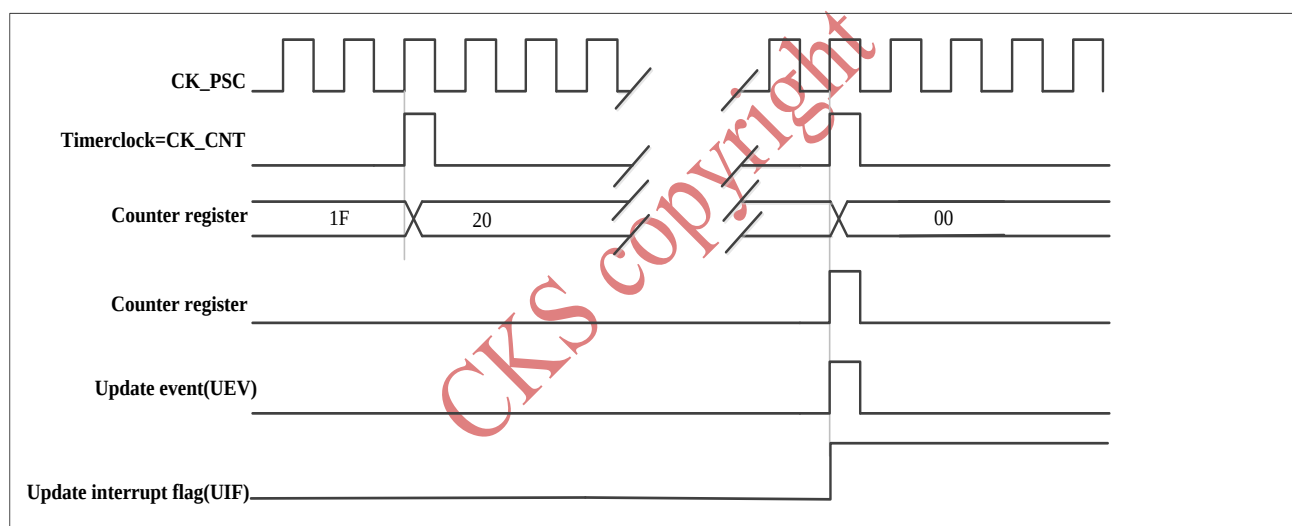


Figure 43 Counter timing diagram, internal clock divided by N

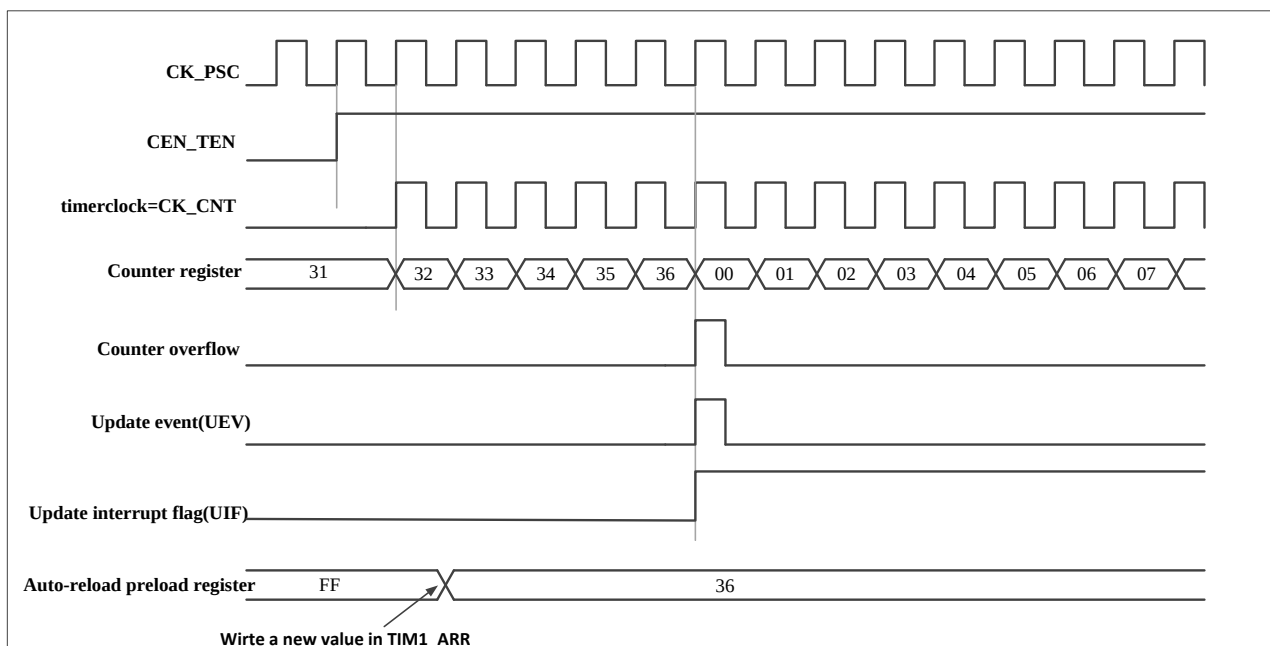


Figure 44 Counter timing diagram, update event when ARPE=0 (TIMx_ARR not preloaded)

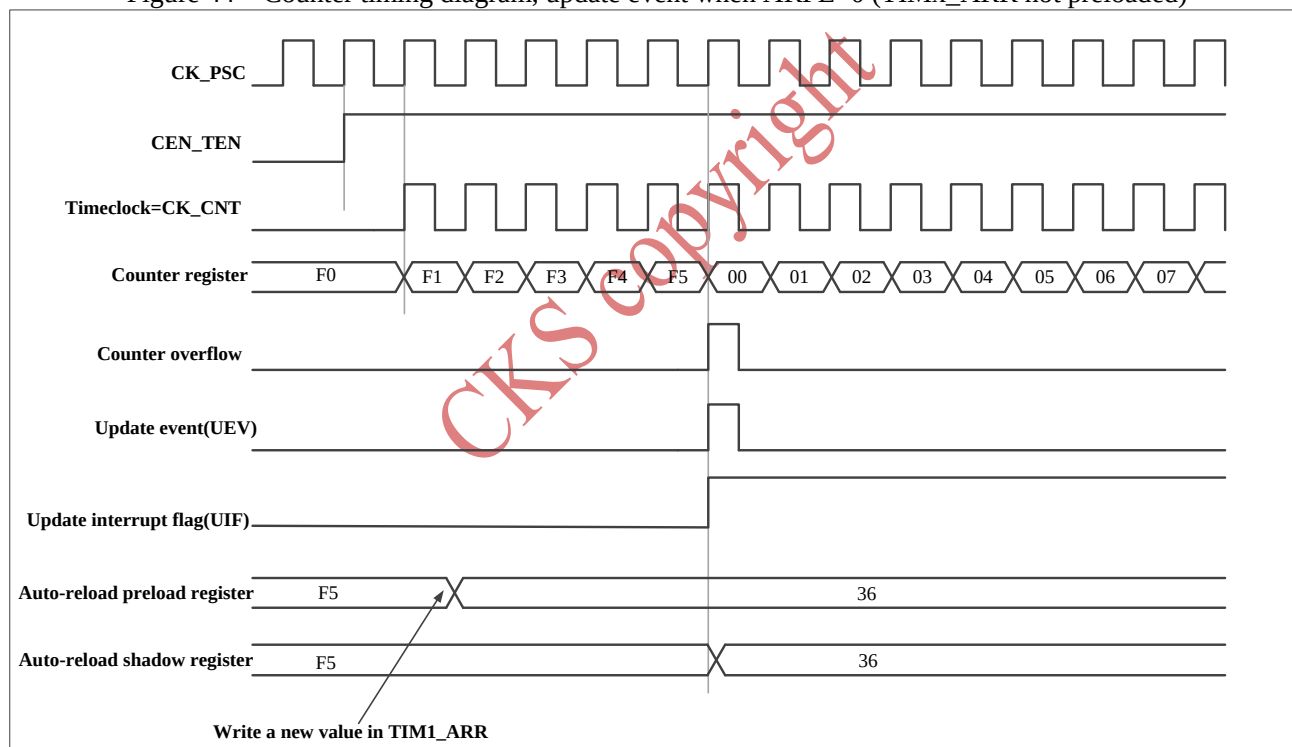


Figure 45 Counter timing diagram, update event when ARPE=1 (TIMx_ARR preloaded)

Downcounting mode

In downcounting mode, the counter counts from the auto-reload value (content of the TIM1_ARR register) down to 0, then restarts from the auto-reload value and generates a counter underflow event. If the repetition counter is used, the update event (UEV) is generated after downcounting is repeated for the number of times programmed in the repetition counter register plus one (TIM1_RCR+1). Else the update event is generated at each counter underflow.

Setting the UG bit in the TIM1_EGR register (by software or by using the slave mode controller) also generates an update event.

The UEV update event can be disabled by software by setting the UDIS bit in TIM1_CR1 register. This is to avoid updating the shadow registers while writing new values in the preload registers. Then no update event occurs until UDIS bit has been written to 0. However, the counter restarts from the current auto-reload value, whereas the counter of the prescaler restarts from 0 (but the prescale rate doesn't change).

In addition, if the URS bit (update request selection) in TIM1_CR1 register is set, setting the UG bit generates an update event UEV but without setting the UIF flag (thus no interrupt or DMA request is sent). This is to avoid generating both update and capture interrupts when clearing the counter on the capture event.

When an update event occurs, all the registers are updated and the update flag (UIF bit in TIM1_SR register) is set (depending on the URS bit):

- The repetition counter is reloaded with the content of TIM1_RCR register
- The buffer of the prescaler is reloaded with the preload value (content of the TIM1_PSC register)
- The auto-reload active register is updated with the preload value (content of the TIM1_ARR register).
Note that the auto-reload is updated before the counter is reloaded, so that the next period is the expected one

The following figures show some examples of the counter behavior for different clock frequencies when TIM1_ARR=0x36.

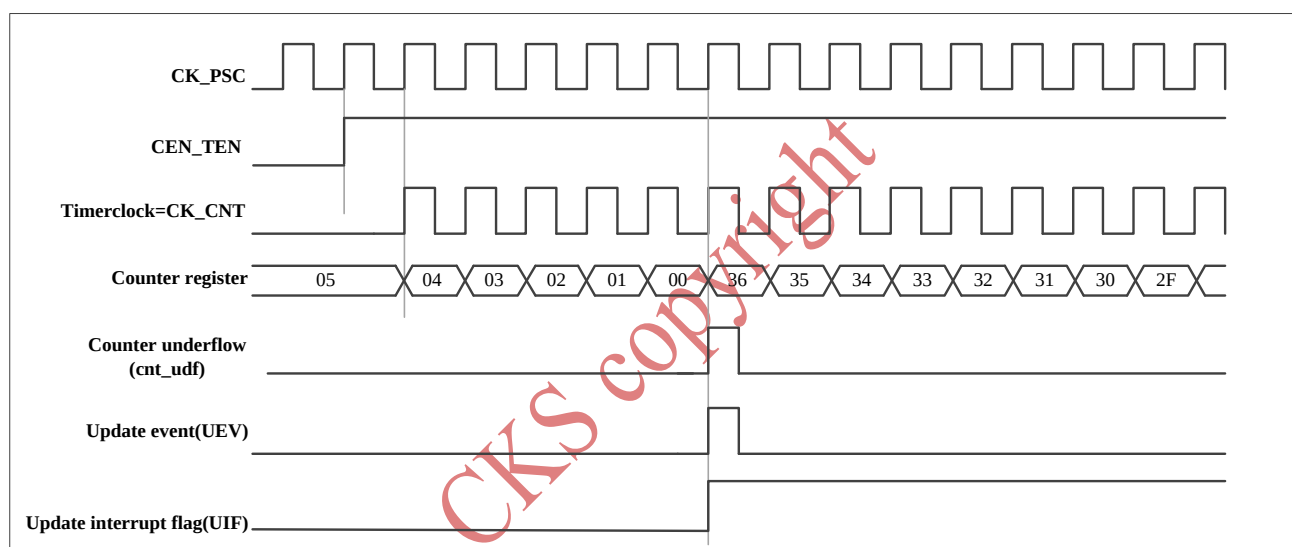


Figure 46 Counter timing diagram, internal clock divided by 1

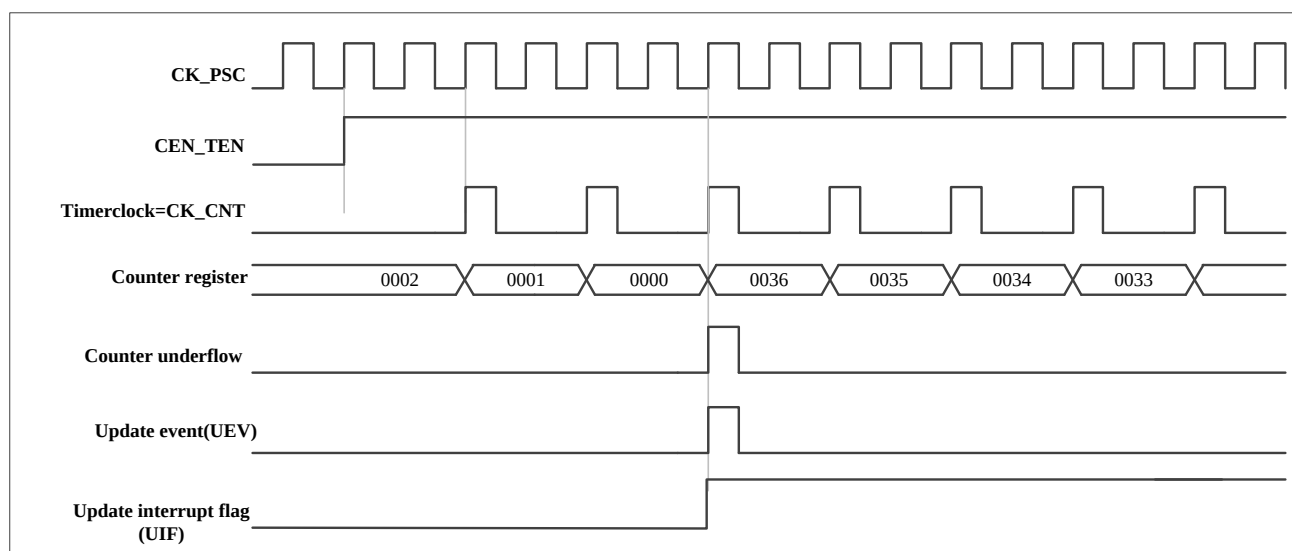


Figure 47 Counter timing diagram, internal clock divided by 2

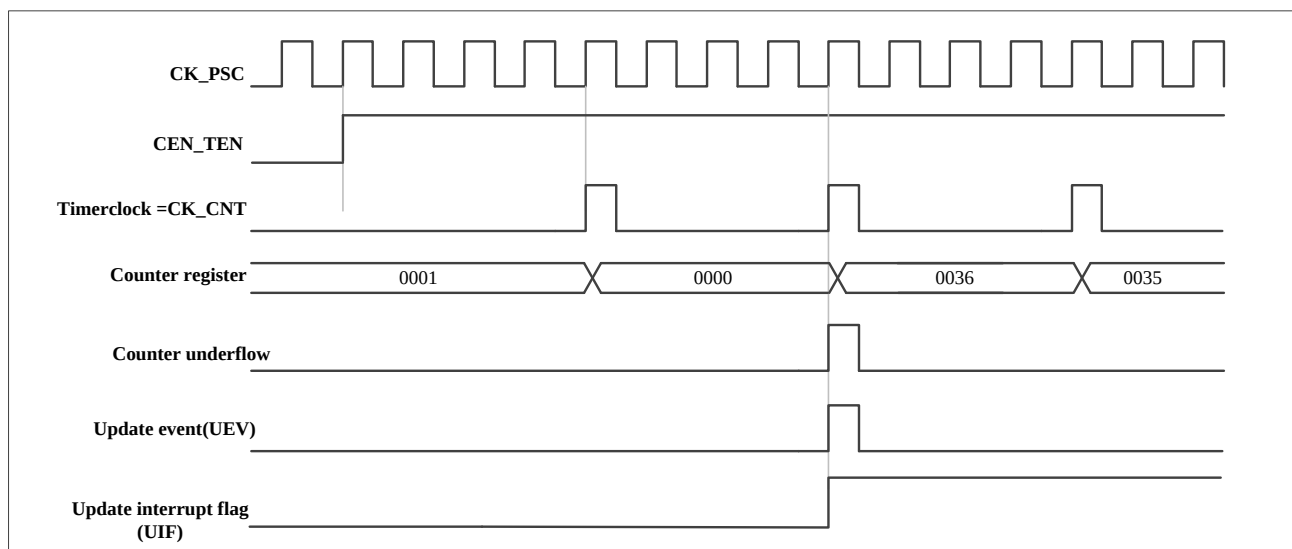


Figure 48 Counter timing diagram, internal clock divided by 4

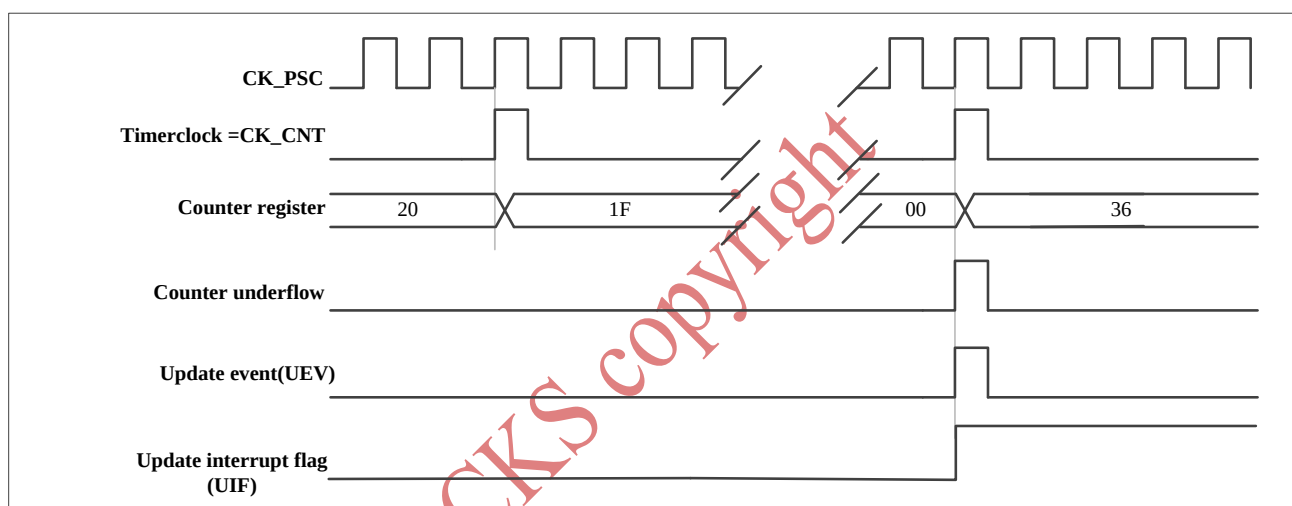


Figure 49 Counter timing diagram, internal clock divided by N

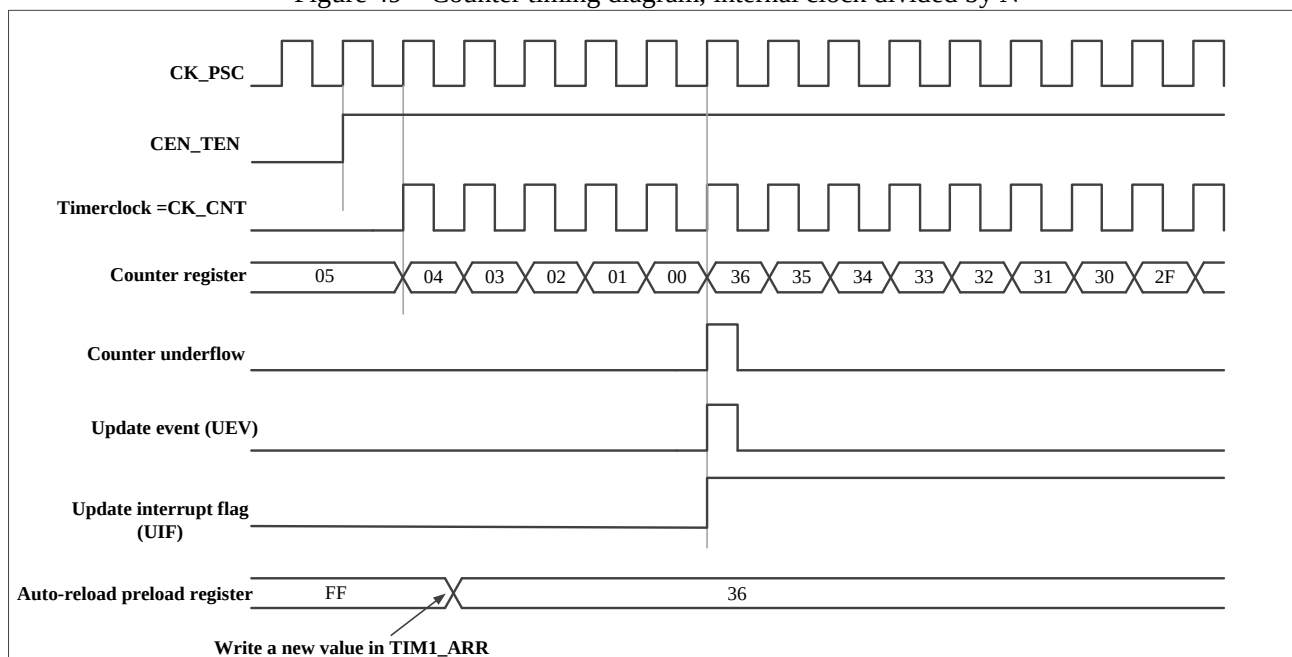


Figure 50 Counter timing diagram, update event when repetition counter is not used

Center-aligned mode (up/down counting)

In center-aligned mode, the counter counts from 0 to the auto-reload value (content of the TIMx_ARR register) – 1, generates a counter overflow event, then counts from the auto-reload value down to 1 and generates a counter underflow event. Then it restarts counting from 0.

In this mode, the DIR direction bit in the TIM1_CR1 register cannot be written. It is updated by hardware and gives the current direction of the counter.

The update event can be generated at each counter overflow and at each counter underflow or by setting the UG bit in the TIM1_EGR register (by software or by using the slave mode controller) also generates an update event. In this case, the counter restarts counting from 0, as well as the counter of the prescaler.

The UEV update event can be disabled by software by setting the UDIS bit in the TIM1_CR1 register. This is to avoid updating the shadow registers while writing new values in the preload registers. Then no update event occurs until UDIS bit has been written to 0. However, the counter continues counting up and down, based on the current auto-reload value.

In addition, if the URS bit (update request selection) in TIM1_CR1 register is set, setting the UG bit generates an UEV update event but without setting the UIF flag (thus no interrupt or DMA request is sent). This is to avoid generating both update and capture interrupts when clearing the counter on the capture event.

When an update event occurs, all the registers are updated and the update flag (UIF bit in TIM1_SR register) is set (depending on the URS bit):

- The repetition counter is reloaded with the content of TIM1_RCR register
- The buffer of the prescaler is reloaded with the preload value (content of the TIM1_PSC register)
- The auto-reload active register is updated with the preload value (content of the TIM1_ARR register). Note that if the update source is a counter overflow, the auto-reload is updated before the counter is reloaded, so that the next period is the expected one (the counter is loaded with the new value).

The following figures show some examples of the counter behavior for different clock frequencies.

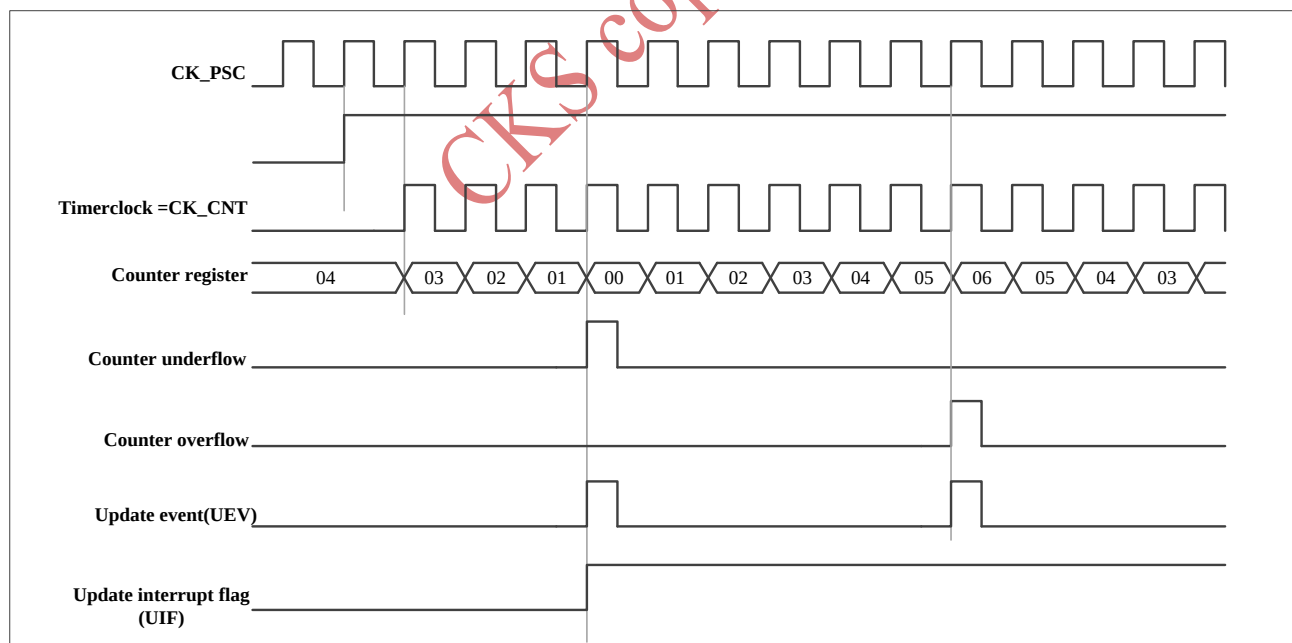


Figure 51 Counter timing diagram, internal clock divided by 1, TIMx_ARR = 0x6

1. Here, center-aligned mode 1 is used (for more details refer to *Section 14.4: TIM1 registers*).

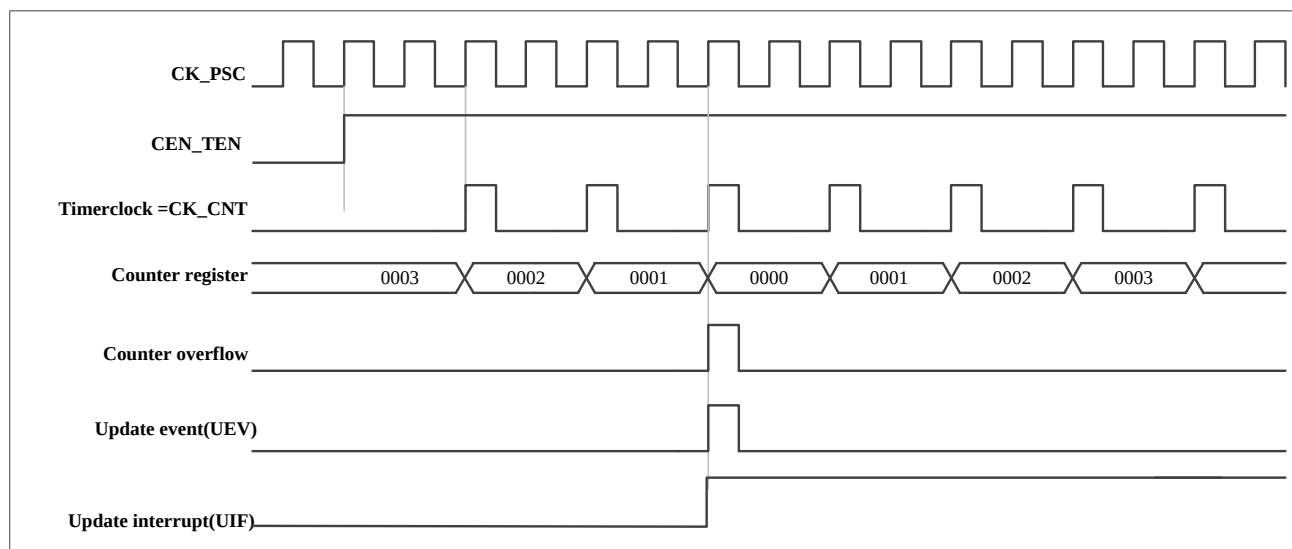


Figure 52 Counter timing diagram, internal clock divided by 2

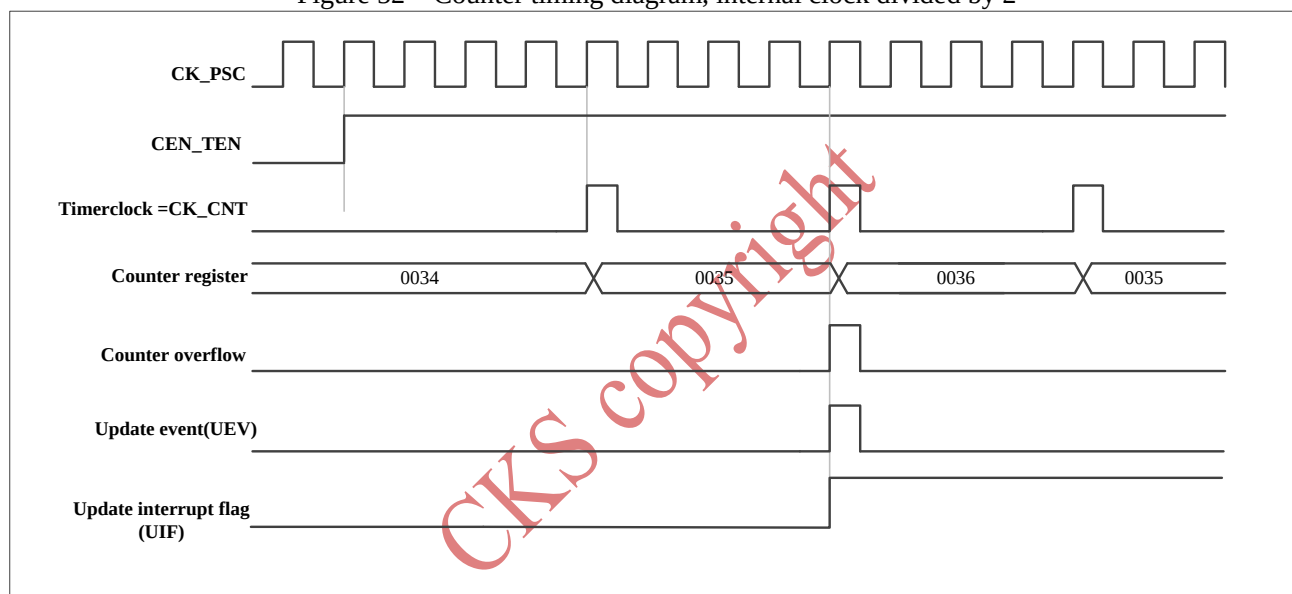


Figure 53 Counter timing diagram, internal clock divided by 4, TIMx_ARR=0x36

1. Center-aligned mode 2 or 3 is used with an UIF on overflow.

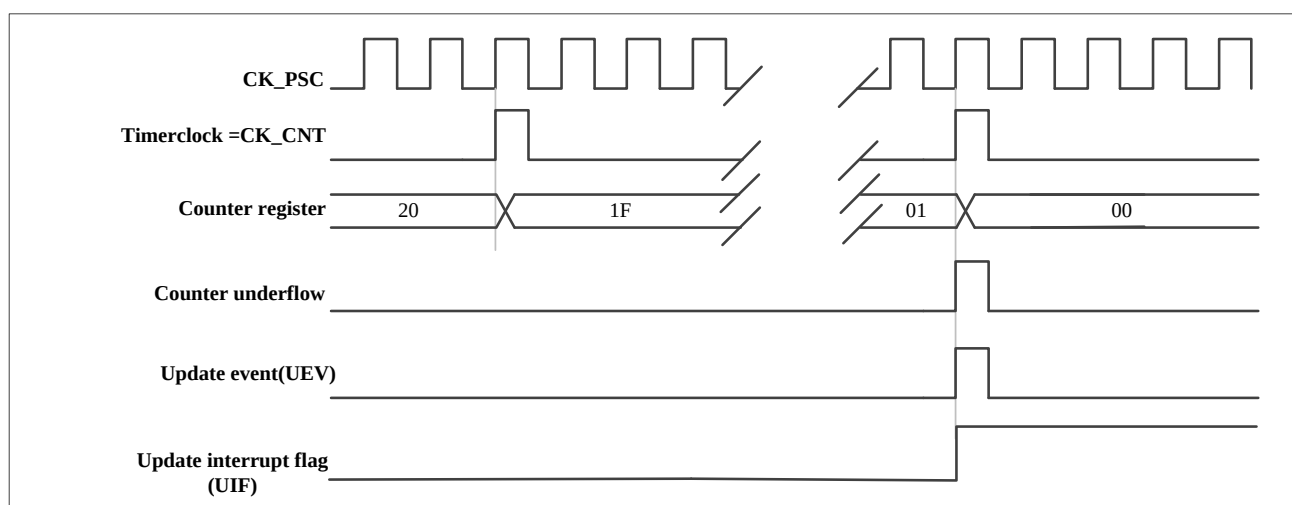


Figure 54 Counter timing diagram, internal clock divided by N

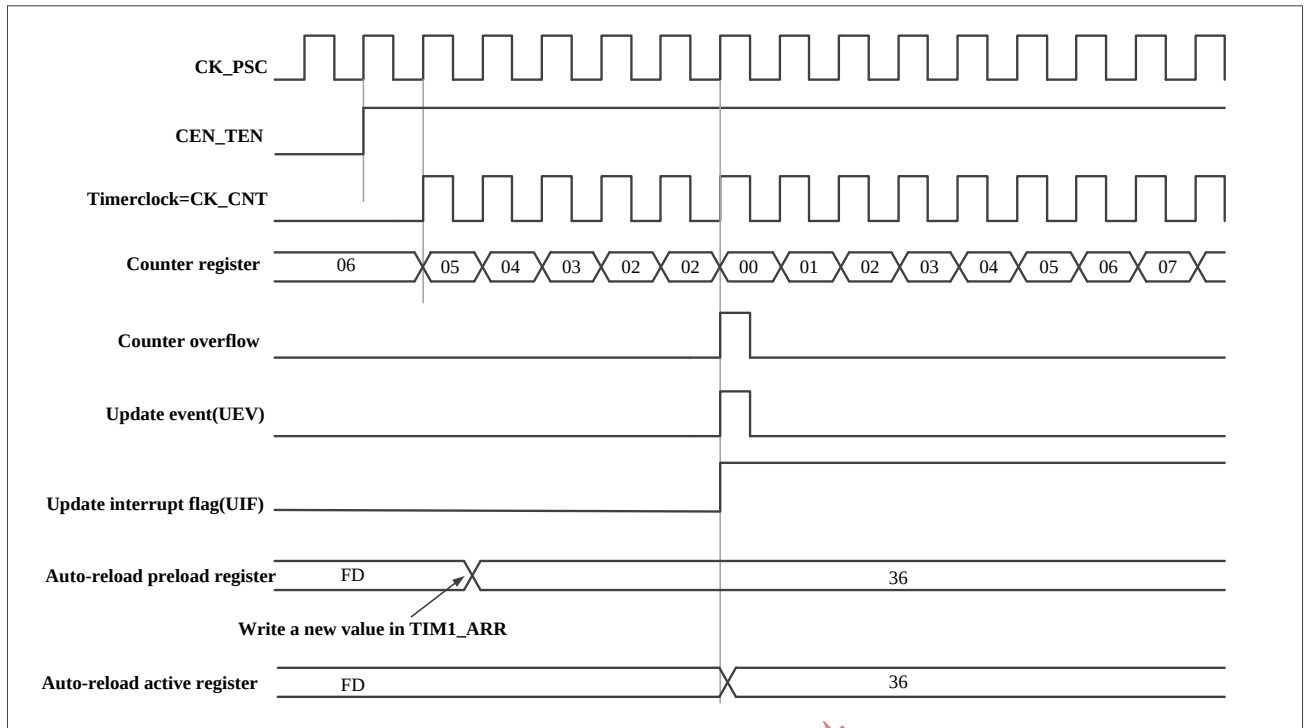


Figure 55 Counter timing diagram, update event with ARPE=1 (counter underflow)

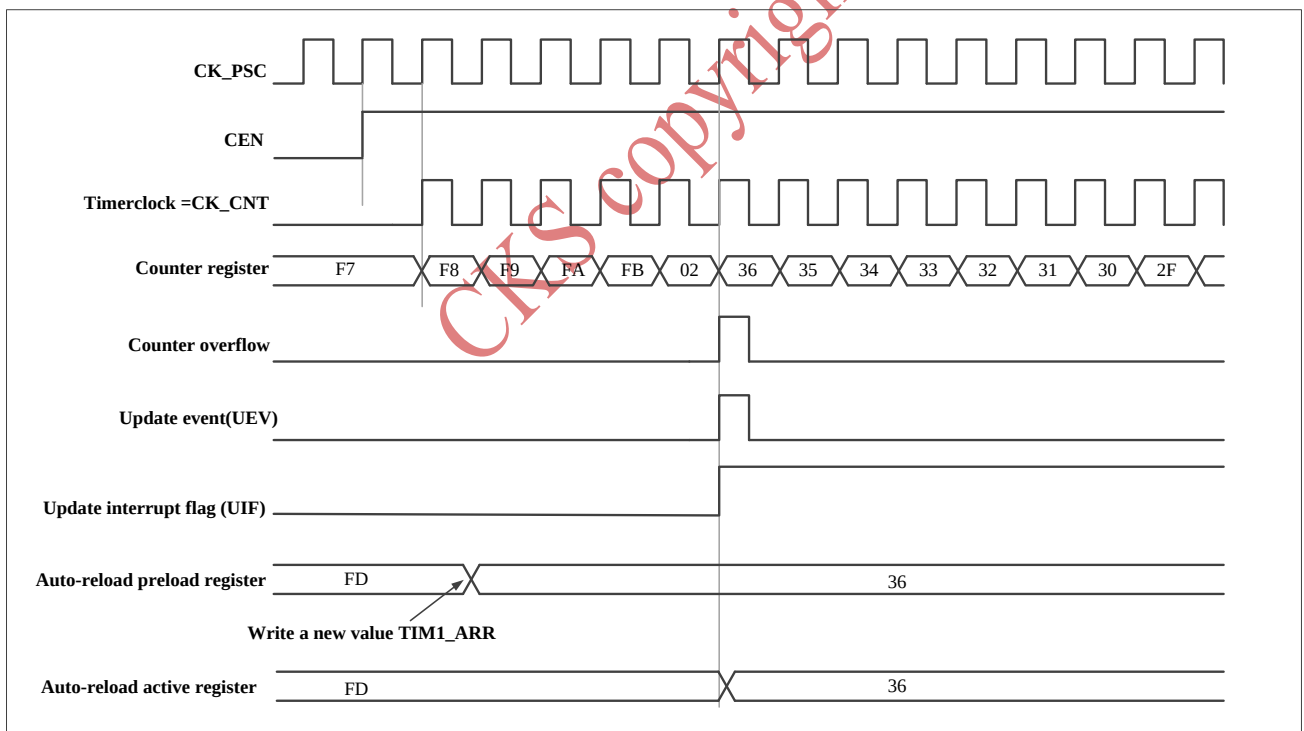


Figure 56 Counter timing diagram, Update event with ARPE=1 (counter overflow)

11.3.3 Repetition counter

Section 11.3.1: Time-base unit describes how the update event (UEV) is generated with respect to the counter overflows/underflows. It is actually generated only when the repetition counter has reached zero. This can be useful when generating PWM signals.

This means that data are transferred from the preload registers to the shadow registers (TIM1_ARR auto-reload register, TIM1_PSC prescaler register, but also TIM1_CCRx capture/compare registers in compare mode) every N+1 counter overflows or underflows, where N is the value in the TIM1_RCR repetition counter register.

The repetition counter is decremented:

- At each counter overflow in upcounting mode,
- At each counter underflow in downcounting mode,
- At each counter overflow and at each counter underflow in center-aligned mode. Although this limits the maximum number of repetition to 128 PWM cycles, it makes it possible to update the duty cycle twice per PWM period. When refreshing compare registers only once per PWM period in center-aligned mode, maximum resolution is $2 \times T_{\text{clk}}$, due to the symmetry of the pattern.

The repetition counter is an auto-reload type; the repetition rate is maintained as defined by the TIMx_RCR register value (refer to Figure 57). When the update event is generated by software (by setting the UG bit in TIM1_EGR register) or by hardware through the slave mode controller, it occurs immediately whatever the value of the repetition counter is and the repetition counter is reloaded with the content of the TIM1_RCR register.

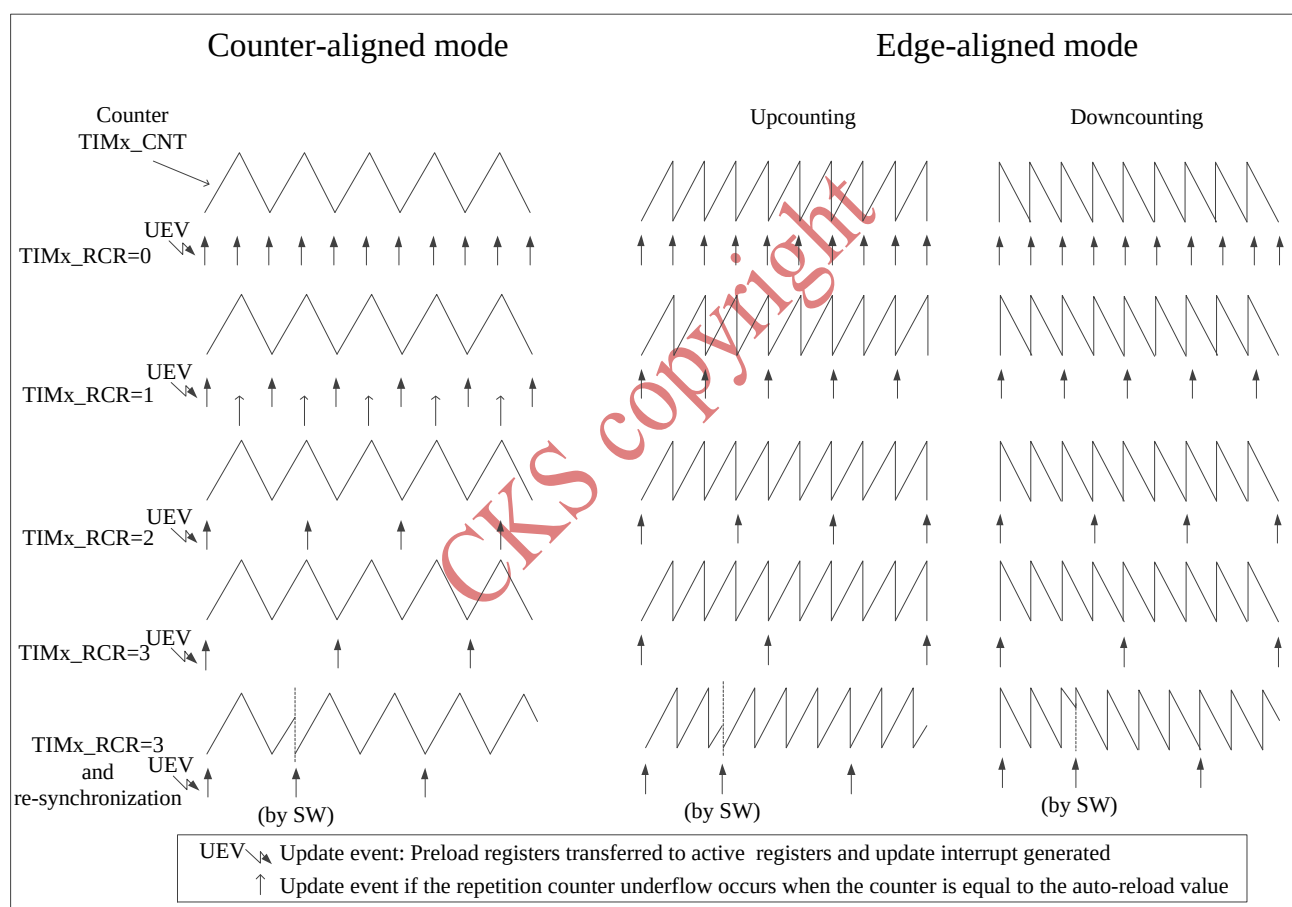


Figure 57 Update rate examples depending on mode and TIMx_RCR register settings

11.3.4 Clock selection

The counter clock can be provided by the following clock sources:

- Internal clock (CK_INT)
- External clock mode1: external input pin
- External clock mode2: external trigger input ETR
- Internal trigger inputs (ITRx): using one timer as prescaler for another timer, for example, the user can configure Timer 1 to act as a prescaler for Timer 2. Refer to next section for more details.

Internal clock source (CK_INT)

If the slave mode controller is disabled (SMS=000), then the CEN, DIR (in the TIM1_CR1 register) and UG bits (in the TIM1_EGR register) are actual control bits and can be changed only by software (except UG which remains cleared automatically). As soon as the CEN bit is written to 1, the prescaler is clocked by the internal clock CK_INT.

Figure 58 shows the behavior of the control circuit and the upcounter in normal mode, without prescaler.

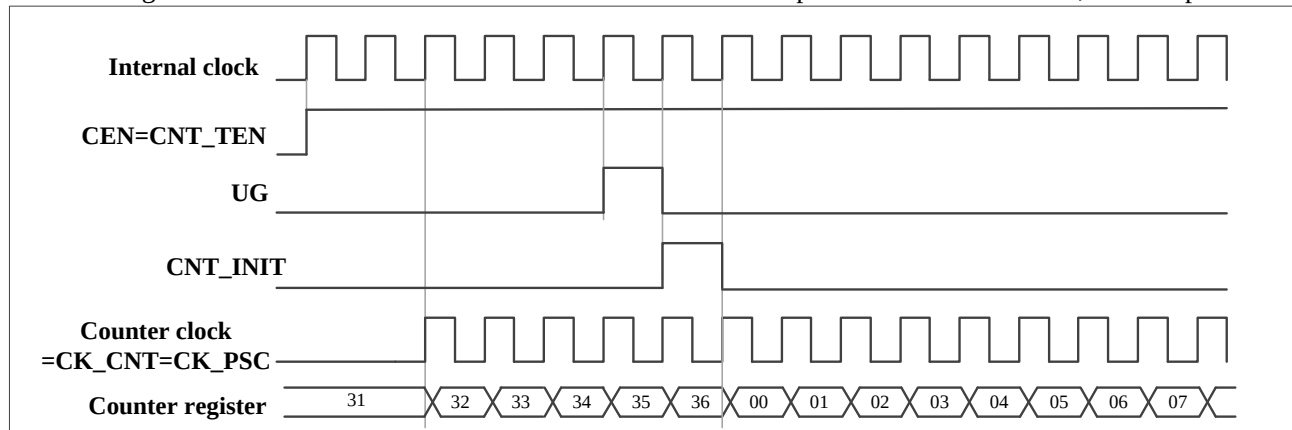


Figure 58 Control circuit in normal mode, internal clock divided by 1

External clock source mode 1

This mode is selected when SMS=111 in the TIM1_SMCR register. The counter can count at each rising or falling edge on a selected input.

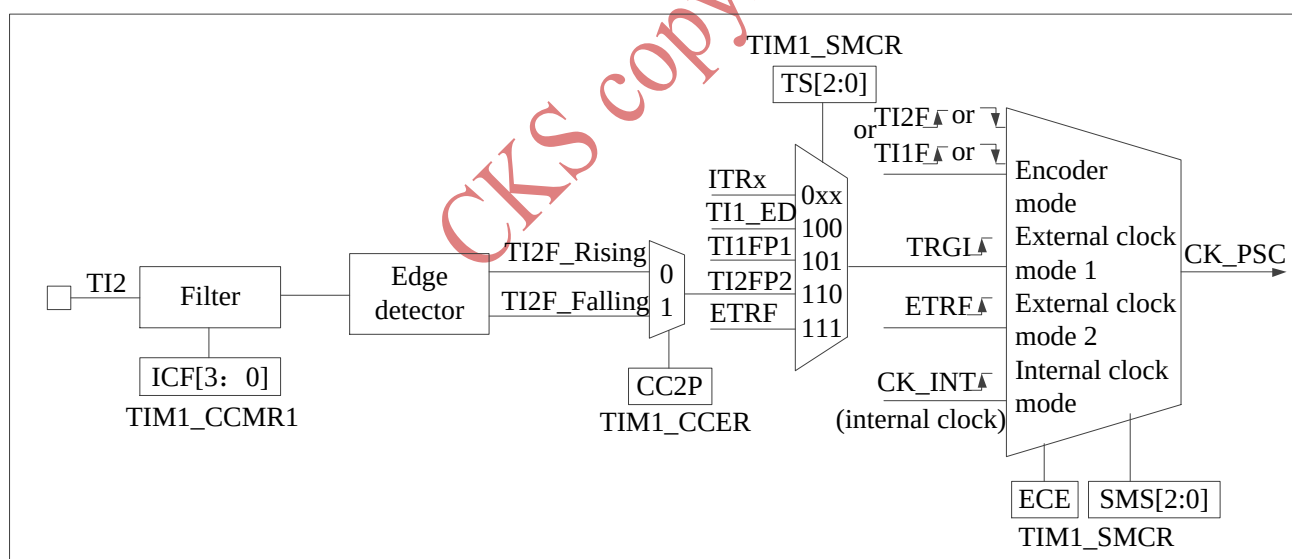


Figure 59 TI2 external clock connection example

For example, to configure the upcounter to count in response to a rising edge on the TI2 input, use the following procedure:

Configure channel 2 to detect rising edges on the TI2 input by writing CC2S = '01' in the TIM1_CCMR1 register.

Configure the input filter duration by writing the IC2F[3:0] bits in the TIM1_CCMR1 register (if no filter is needed, keep IC2F=0000).

Select rising edge polarity by writing CC2P=0 in the TIM1_CCER register.

Configure the timer in external clock mode 1 by writing SMS=111 in the TIM1_SMCR register.

Select TI2 as the trigger input source by writing TS=110 in the TIM1_SMCR register.

Enable the counter by writing CEN=1 in the TIM1_CR1 register.

Note: The capture prescaler is not used for triggering, so the user does not need to configure it. When a rising edge occurs on TI2, the counter counts once and the TIF flag is set. The delay between the rising edge on TI2 and the actual clock of the counter is due to the resynchronization circuit on TI2 input.

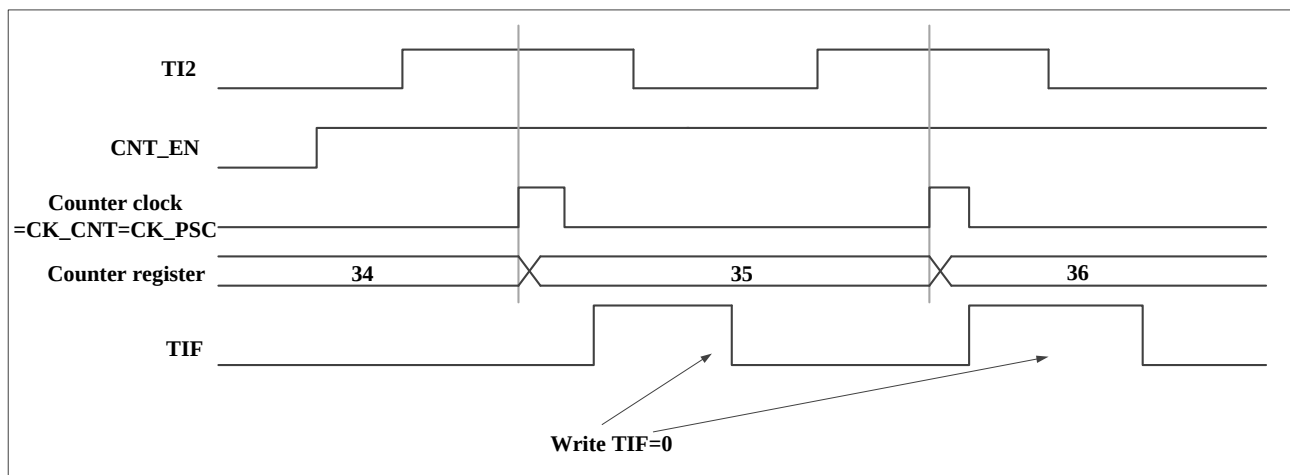


Figure 60 Control circuit in external clock mode 1

External clock source mode 2

This mode is selected by writing ECE=1 in the TIM1_SMCR register. The counter can count at each rising or falling edge on the external trigger input ETR. Figure 61 gives an overview of the external trigger input block.

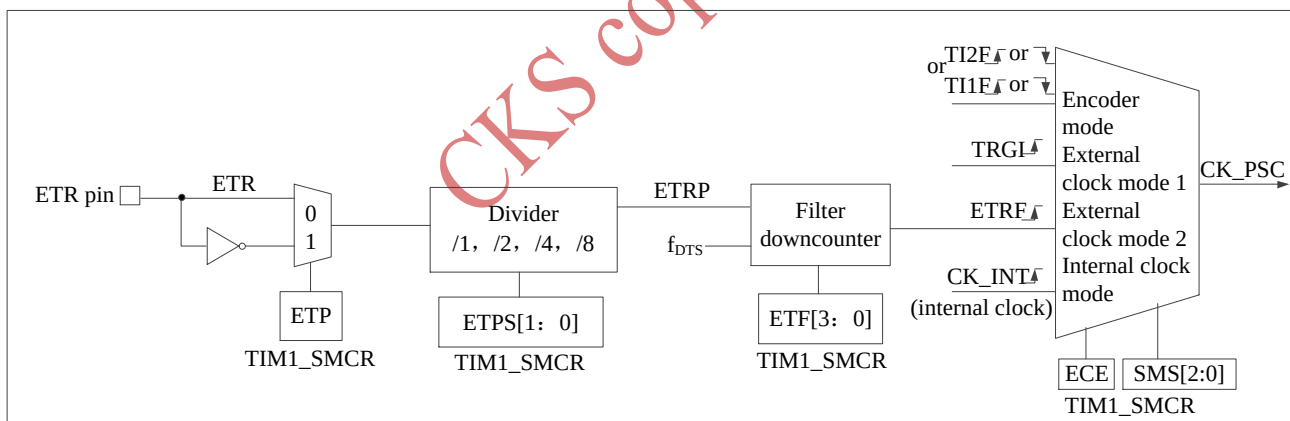


Figure 61 External trigger input block

For example, to configure the upcounter to count each 2 rising edges on ETR, use the following procedure:

As no filter is needed in this example, write ETF[3:0]=0000 in the TIM1_SMCR register.

Set the prescaler by writing ETPS[1:0]=01 in the TIM1_SMCR register

Select rising edge detection on the ETR pin by writing ETP=0 in the TIM1_SMCR register

Enable external clock mode 2 by writing ECE=1 in the TIM1_SMCR register.

Enable the counter by writing CEN=1 in the TIM1_CR1 register.

The counter counts once each 2 ETR rising edges.

The delay between the rising edge on ETR and the actual clock of the counter is due to the resynchronization circuit on the ETRP signal.

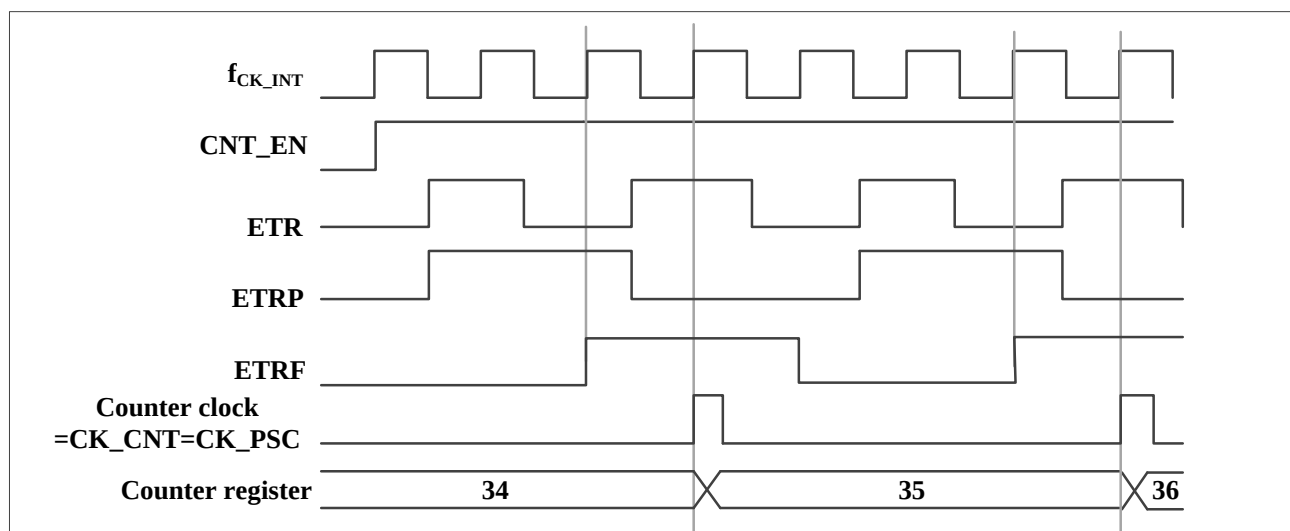


Figure 62 Control circuit in external clock mode 2

11.3.5 Capture/compare channels

Each Capture/Compare channel is built around a capture/compare register (including a shadow register), a input stage for capture (with digital filter, multiplexing and prescaler) and an output stage (with comparator and output control). Figure 63 to Figure 66 give an overview of one Capture/Compare channel.

The input stage samples the corresponding TIx input to generate a filtered signal $TIxF$. Then, an edge detector with polarity selection generates a signal ($TIxFPx$) which can be used as trigger input by the slave mode controller or as the capture command. It is prescaled before the capture register ($ICxPS$).

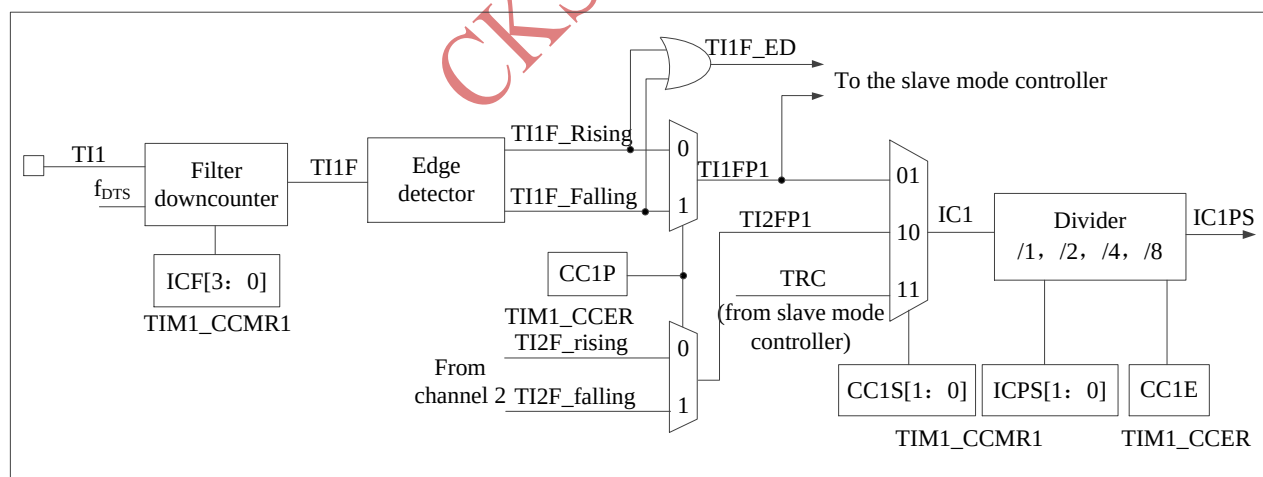


Figure 63 Capture/compare channel (example: channel 1 input stage)

The output stage generates an intermediate waveform that is then used for reference: $OCxRef$ (active high). The polarity acts at the end of the chain.

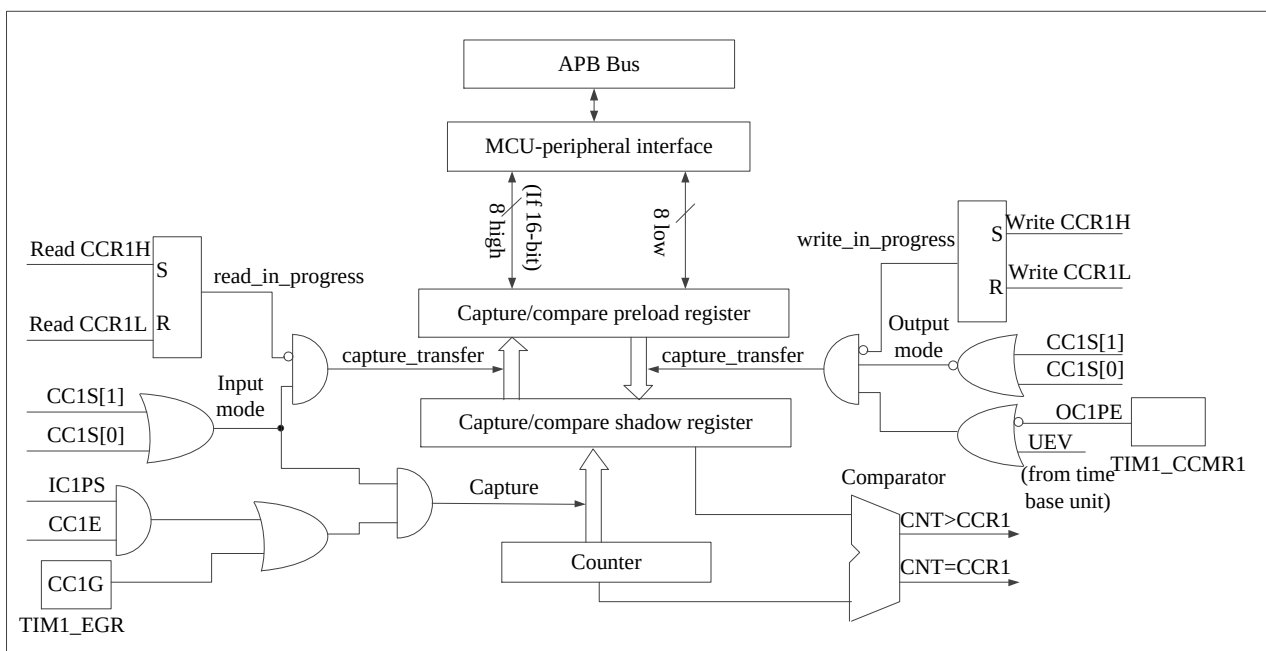


Figure 64 Capture/compare channel 1 main circuit

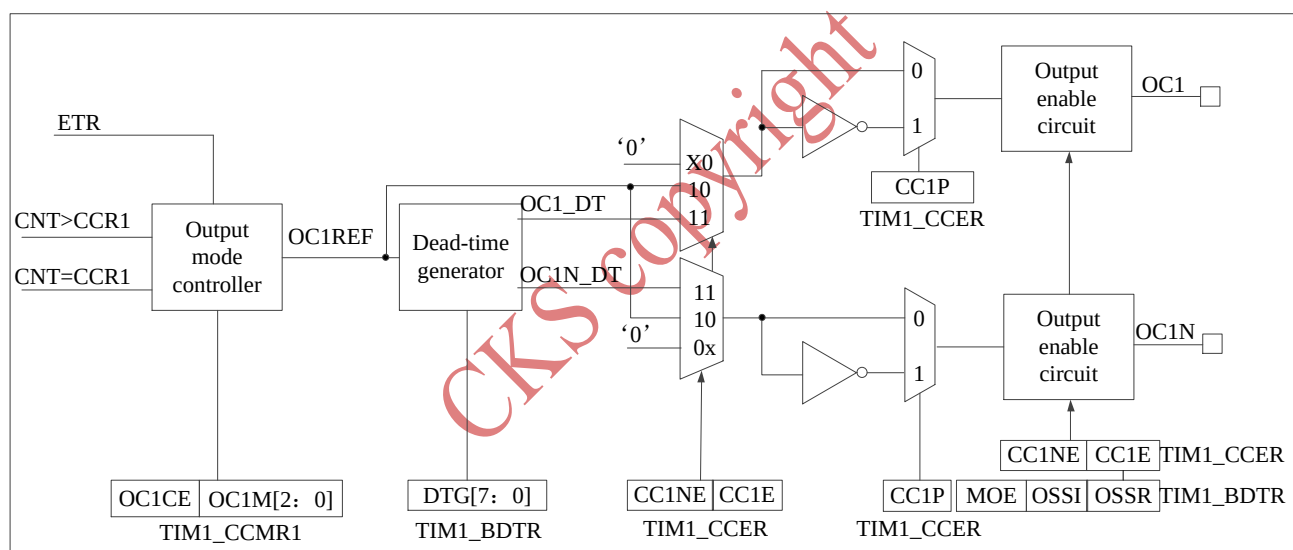


Figure 65 Output stage of capture/compare channel (channel 1 to 3)

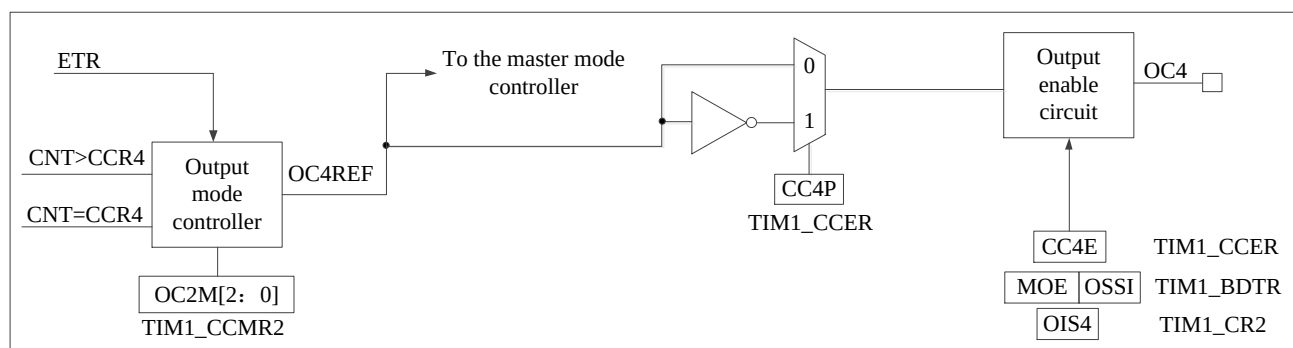


Figure 66 Output stage of capture/compare channel (channel 4)

The capture/compare block is made of one preload register and one shadow register. Write and read always access the preload register.

In capture mode, captures are actually done in the shadow register, which is copied into the preload register.

In compare mode, the content of the preload register is copied into the shadow register which is compared to the counter.

11.3.6 Input capture mode

In Input capture mode, the Capture/Compare Registers (TIM1_CCRx) are used to latch the value of the counter after a transition detected by the corresponding ICx signal. When a capture occurs, the corresponding CCxIF flag (TIM1_SR register) is set and an interrupt or a DMA request can be sent if they are enabled. If a capture occurs while the CCxIF flag was already high, then the over-capture flag CCxOF (TIM1_SR register) is set. CCxIF can be cleared by software by writing it to '0' or by reading the captured data stored in the TIM1_CCRx register. CCxOF is cleared when written to '0'.

The following example shows how to capture the counter value in TIM1_CCR1 when TI1 input rises. To do this, use the following procedure:

- Select the active input: TIM1_CCR1 must be linked to the TI1 input, so write the CC1S bits to 01 in the TIM1_CCMR1 register. As soon as CC1S becomes different from 00, the channel is configured in input and the TIM1_CCR1 register becomes read-only.
- Program the needed input filter duration with respect to the signal connected to the timer (by programming ICxIF bits in the TIM1_CCMRx register if the input is a TIX input). Let's imagine that, when toggling, the input signal is not stable during at most five internal clock cycles. We must program a filter duration longer than these five clock cycles. We can validate a transition on TI1 when 8 consecutive samples with the new level have been detected (sampled at fDTS frequency). Then write IC1F bits to 0011 in the TIM1_CCMR1 register.
- Select the edge of the active transition on the TI1 channel by writing CC1P bit to 0 in the TIM1_CCER register (rising edge in this case).
- Program the input prescaler. In our example, we wish the capture to be performed at each valid transition, so the prescaler is disabled (write IC1PS bits to '00' in the TIM1_CCMR1 register).
- Enable capture from the counter into the capture register by setting the CC1E bit in the TIM1_CCER register.
- If needed, enable the related interrupt request by setting the CC1IE bit in the TIM1_DIER register, and/or the DMA request by setting the CC1DE bit in the TIM1_DIER register.

When an input capture occurs:

- The TIM1_CCR1 register gets the value of the counter on the active transition.
- CC1IF flag is set (interrupt flag). CC1OF is also set if at least two consecutive captures occurred whereas the flag was not cleared.
- An interrupt is generated depending on the CC1IE bit.
- A DMA request is generated depending on the CC1DE bit.

In order to handle the overcapture, it is recommended to read the data before the overcapture flag. This is to avoid missing an overcapture which could happen after reading the flag and before reading the data.

Note: IC interrupt and/or DMA requests can be generated by software by setting the corresponding CCxG bit in the TIM1_EGR register.

11.3.7 PWM input mode

This mode is a particular case of input capture mode. The procedure is the same except:

- Two ICx signals are mapped on the same TIX input.
- These 2 ICx signals are active on edges with opposite polarity.
- One of the two TIXFP signals is selected as trigger input and the slave mode controller is configured in reset mode.

For example, user can measure the period (in TIM1_CCR1 register) and the duty cycle (in TIM1_CCR2 register) of the PWM applied on TI1 using the following procedure (depending on CK_INT frequency and prescaler value):

- Select the active input for TIM1_CCR1: write the CC1S bits to 01 in the TIM1_CCMR1 register (TI1 selected).
- Select the active polarity for TI1FP1 (used both for capture in TIM1_CCR1 and counter clear): write the CC1P bit to '0' (active on rising edge).
- Select the active input for TIM1_CCR2: write the CC2S bits to 10 in the TIM1_CCMR1 register (TI1 selected).
- Select the active polarity for TI1FP2 (used for capture in TIM1_CCR2): write the CC2P bit to '1' (active on falling edge).
- Select the valid trigger input: write the TS bits to 101 in the TIMx_SMCR register (TI1FP1 selected).
- Configure the slave mode controller in reset mode: write the SMS bits to 100 in the TIM1_SMCR register.
- Enable the captures: write the CC1E and CC2E bits to '1' in the TIM1_CCER register.

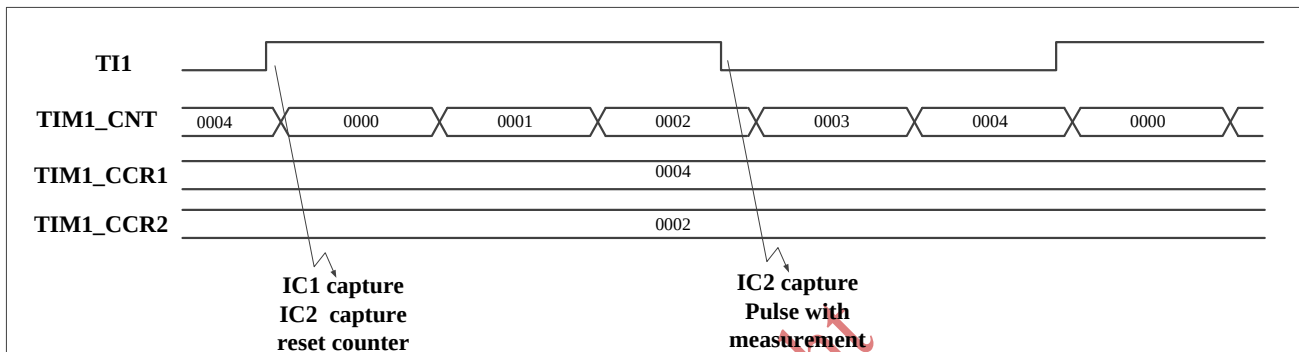


Figure 67 PWM input mode timing

1. The PWM input mode can be used only with the TIMx_CH1/TIMx_CH2 signals due to the fact that only, TI1FP1 and TI2FP2 are connected to the slave mode controller.

11.3.8 Forced output mode

In output mode (CCxS bits = 00 in the TIM1_CCMRx register), each output compare signal (OCxREF and then OCx/OCxN) can be forced to active or inactive level directly by software, independently of any comparison between the output compare register and the counter.

To force an output compare signal (OCXREF/OCx) to its active level, the user just needs to write 101 in the OCxM bits in the corresponding TIMx_CCMRx register. Thus OCXREF is forced high (OCxREF is always active high) and OCx get opposite value to CCxP polarity bit.

For example: CCxP=0 (OCx active high) => OCx is forced to high level.

The OCxREF signal can be forced low by writing the OCxM bits to 100 in the TIM1_CCMRx register.

Anyway, the comparison between the TIM1_CCRx shadow register and the counter is still performed and allows the flag to be set. Interrupt and DMA requests can be sent accordingly. This is described in the output compare mode section below.

11.3.9 Output compare mode

This function is used to control an output waveform or indicating when a period of time has elapsed. When a match is found between the capture/compare register and the counter, the output compare function:

- Assigns the corresponding output pin to a programmable value defined by the output compare mode (OCxM bits in the TIM1_CCMRx register) and the output polarity (CCxP bit in the TIM1_CCER register). The output pin can keep its level (OCXM=000), be set active (OCXM=001), be set inactive (OCXM=010) or can toggle (OCXM=011) on match.
- Sets a flag in the interrupt status register (CCxIF bit in the TIM1_SR register).
- Generates an interrupt if the corresponding interrupt mask is set (CCXIE bit in the TIM1_DIER register).

- Sends a DMA request if the corresponding enable bit is set (CCxDE bit in the TIM1_DIER register, CCDS bit in the TIM1_CR2 register for the DMA request selection).

The TIM1_CCRx registers can be programmed with or without preload registers using the OCxPE bit in the TIM1_CCMRx register.

In output compare mode, the update event UEV has no effect on OCxREF and OCx output. The timing resolution is one count of the counter. Output compare mode can also be used to output a single pulse (in One Pulse mode).

Procedure:

1. Select the counter clock (internal, external, prescaler).
2. Write the desired data in the TIM1_ARR and TIM1_CCRx registers.
3. Set the CCxIE bit if an interrupt request is to be generated.
4. Select the output mode. For example:
 - Write OCxM = 011 to toggle OCx output pin when CNT matches CCRx
 - Write OCxPE = 0 to disable preload register
 - Write CCxP = 0 to select active high polarity
 - Write CCxE = 1 to enable the output
5. Enable the counter by setting the CEN bit in the TIM1_CR1 register.

The TIM1_CCRx register can be updated at any time by software to control the output waveform, provided that the preload register is not enabled (OCxPE=0', else TIM1_CCRx shadow register is updated only at the next update event UEV). An example is given in Figure 68.

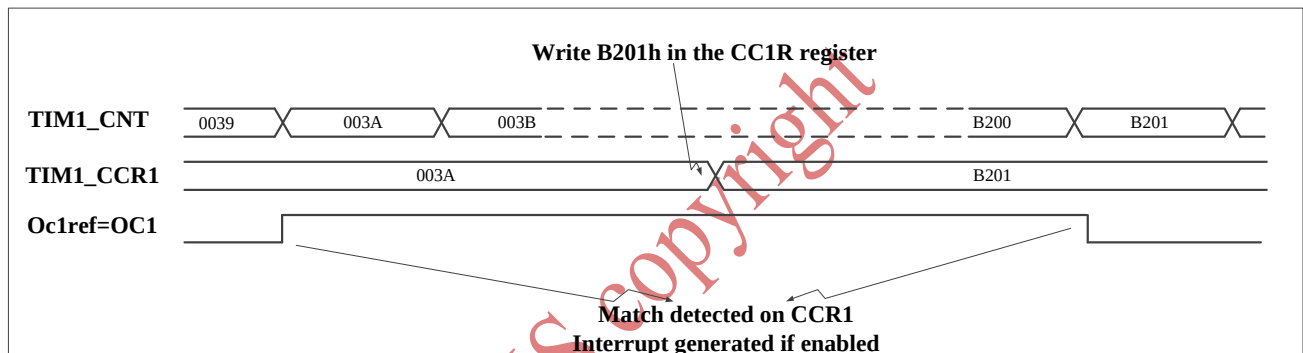


Figure 68. Output compare mode, toggle on OC1

11.3.10 PWM mode

Pulse Width Modulation mode allows generating a signal with a frequency determined by the value of the TIM1_ARR register and a duty cycle determined by the value of the TIM1_CCRx register.

The PWM mode can be selected independently on each channel (one PWM per OCx output) by writing '110' (PWM mode 1) or '111' (PWM mode 2) in the OCxM bits in the TIM1_CCMRx register. The corresponding preload register must be enabled by setting the OCxPE bit in the TIM1_CCMRx register, and eventually the auto-reload preload register (in upcounting or center-aligned modes) by setting the ARPE bit in the TIM1_CR1 register.

As the preload registers are transferred to the shadow registers only when an update event occurs, before starting the counter, the user must initialize all the registers by setting the UG bit in the TIM1_EGR register.

OCx polarity is software programmable using the CCxP bit in the TIM1_CCER register. It can be programmed as active high or active low. OCx output is enabled by a combination of the CCxE, CCxNE, MOE, OSSI and OSSR bits (TIM1_CCER and TIM1_BDTR registers). Refer to the TIM1_CCER register description for more details.

In PWM mode (1 or 2), TIM1_CNT and TIM1_CCRx are always compared to determine whether $TIM1_CCR_x \leq TIM1_CNT$ or $TIM1_CNT \leq TIM1_CCR_x$ (depending on the direction of the counter).

The timer is able to generate PWM in edge-aligned mode or center-aligned mode depending on the CMS bits in the TIM1_CR1 register.

PWM edge-aligned mode

- Upcounting configuration

Upcounting is active when the DIR bit in the TIM1_CR1 register is low. Refer to *Upcounting mode*.

In the following example, we consider PWM mode 1. The reference PWM signal OCxREF is high as long as TIM1_CNT < TIM1_CCRx else it becomes low. If the compare value in TIM1_CCRx is greater than the auto-reload value (in TIM1_ARR) then OCxREF is held at '1'. If the compare value is 0 then OCxRef is held at '0'.

Figure 69 shows some edge-aligned PWM waveforms in an example where TIM1_ARR=8.

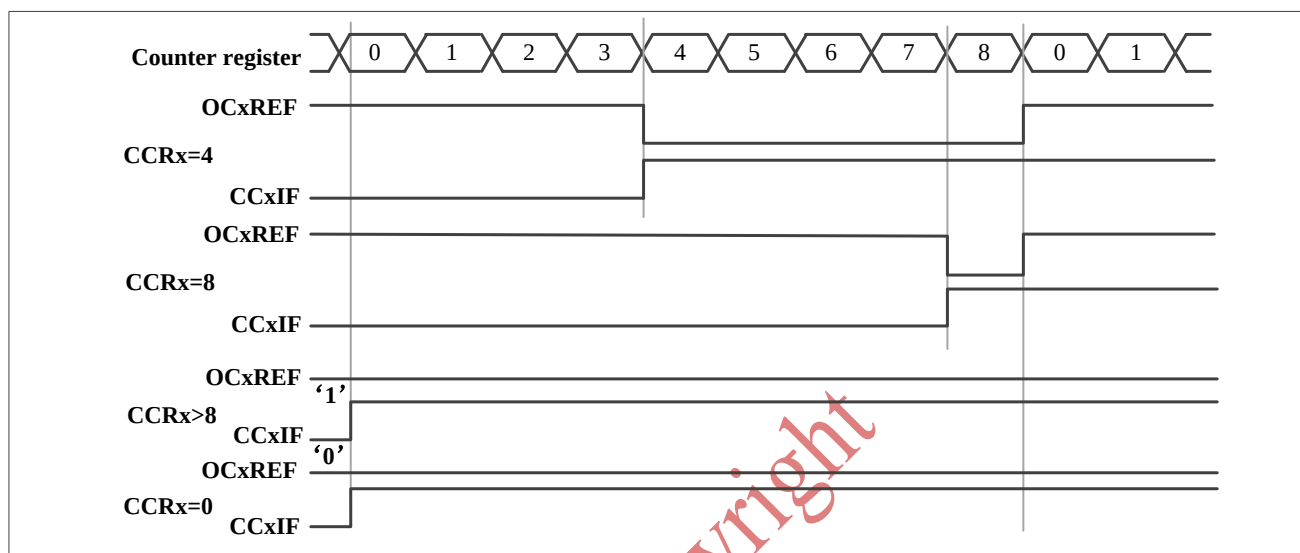


Figure 69 Edge-aligned PWM waveforms (ARR=8)

- Downcounting configuration

Downcounting is active when DIR bit in TIM1_CR1 register is high. Refer to *Downcounting mode*. In PWM mode 1, the reference signal OCxRef is low as long as TIM1_CNT > TIM1_CCRx else it becomes high. If the compare value in TIM1_CCRx is greater than the auto-reload value in TIM1_ARR, then OCxREF is held at '1'. 0% PWM is not possible in this mode.

PWM center-aligned mode

Center-aligned mode is active when the CMS bits in TIM1_CR1 register are different from '00' (all the remaining configurations having the same effect on the OCxRef/OCx signals). The compare flag is set when the counter counts up, when it counts down or both when it counts up and down depending on the CMS bits configuration. The direction bit (DIR) in the TIM1_CR1 register is updated by hardware and must not be changed by software. Refer to *Center-aligned mode (up/down counting)*.

Figure 70 shows some center-aligned PWM waveforms in an example where:

- TIM1_ARR=8
- PWM mode is the PWM mode 1
- The flag is set when the counter counts down corresponding to the center-aligned mode 1 selected for CMS=01 in TIM1_CR1 register.

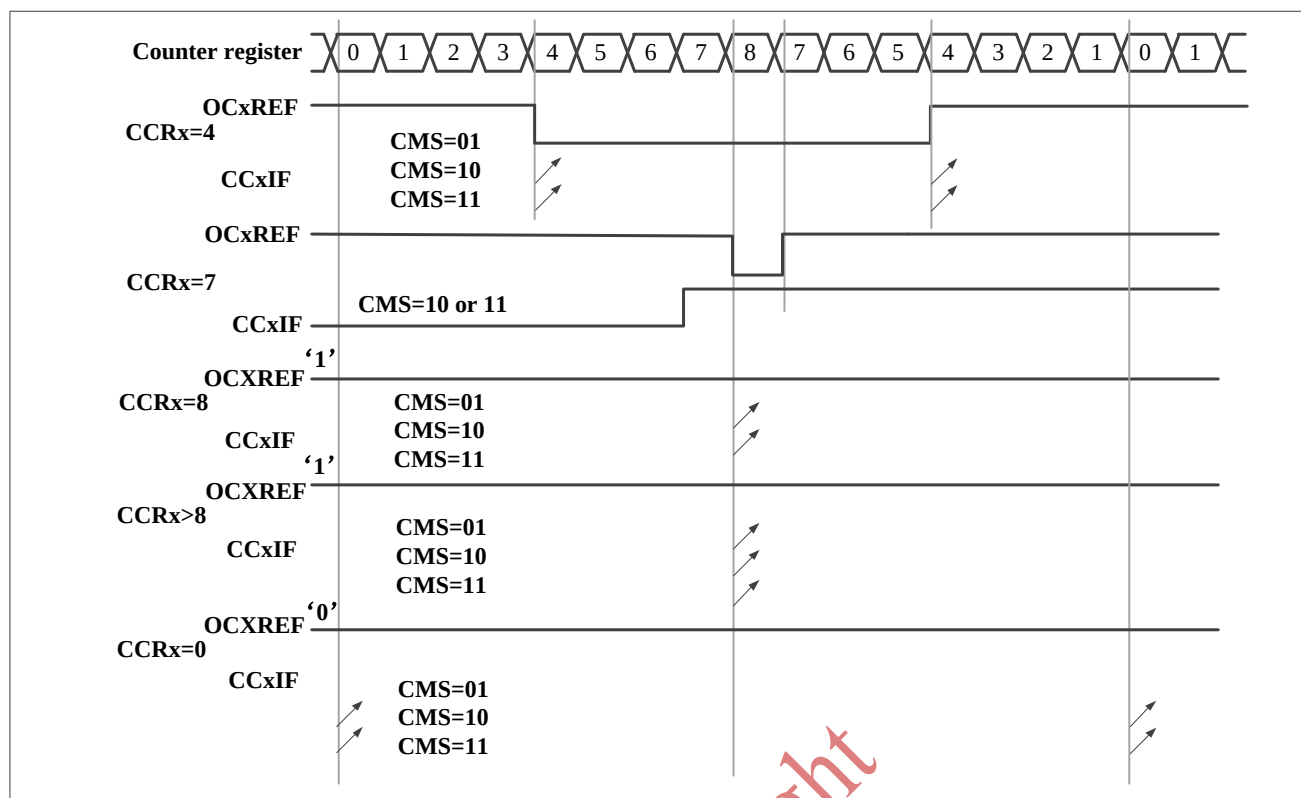


Figure 70 Center-aligned PWM waveforms (ARR=8)

Hints on using center-aligned mode:

- When starting in center-aligned mode, the current up-down configuration is used. It means that the counter counts up or down depending on the value written in the DIR bit in the TIM1_CR1 register. Moreover, the DIR and CMS bits must not be changed at the same time by the software.
- Writing to the counter while running in center-aligned mode is not recommended as it can lead to unexpected results. In particular:
 - The direction is not updated if the user writes a value in the counter greater than the auto-reload value (TIM1_CNT > TIM1_ARR). For example, if the counter was counting up, it will continue to count up.
 - The direction is updated if the user writes 0 or writes the TIM1_ARR value in the counter but no Update Event UEV is generated.
- The safest way to use center-aligned mode is to generate an update by software (setting the UG bit in the TIM1_EGR register) just before starting the counter and not to write the counter while it is running.

11.3.11 Complementary outputs and dead-time insertion

The advanced-control timer (TIM1) can output two complementary signals and manage the switching-off and the switching-on instants of the outputs.

This time is generally known as dead-time and it has to be adjusted depending on the devices connected to the outputs and their characteristics (intrinsic delays of level-shifters, delays due to power switches...) User can select the polarity of the outputs (main output OCx or complementary OCxN) independently for each output. This is done by writing to the CCxP and CCxNP bits in the TIM1_CCER register.

The complementary signals OCx and OCxN are activated by a combination of several control bits: the CCxE and CCxNE bits in the TIM1_CCER register and the MOE, OISx, OISxN, OSS1 and OSSR bits in the TIM1_BDTR and TIM1_CR2 registers. Refer to Table 54 for more details. In particular, the dead-time is activated when switching to the IDLE state (MOE falling down to 0).

Dead-time insertion is enabled by setting both CCxE and CCxNE bits, and the MOE bit if the break circuit is present. DTG[7:0] bits of the TIMx_BDTR register are used to control the dead-time generation for all channels. From a reference waveform OCxREF, it generates 2 outputs OCx and OCxN. If OCx and OCxN are active high:

- The OCx output signal is the same as the reference signal except for the rising edge, which is delayed relative to the reference rising edge.
- The OCxN output signal is the opposite of the reference signal except for the rising edge, which is delayed relative to the reference falling edge.

If the delay is greater than the width of the active output (OCx or OCxN) then the corresponding pulse is not generated.

The following figures show the relationships between the output signals of the dead-time generator and the reference signal OCxREF. (we suppose CCxP=0, CCxNP=0, MOE=1, CCxE=1 and CCxNE=1 in these examples)

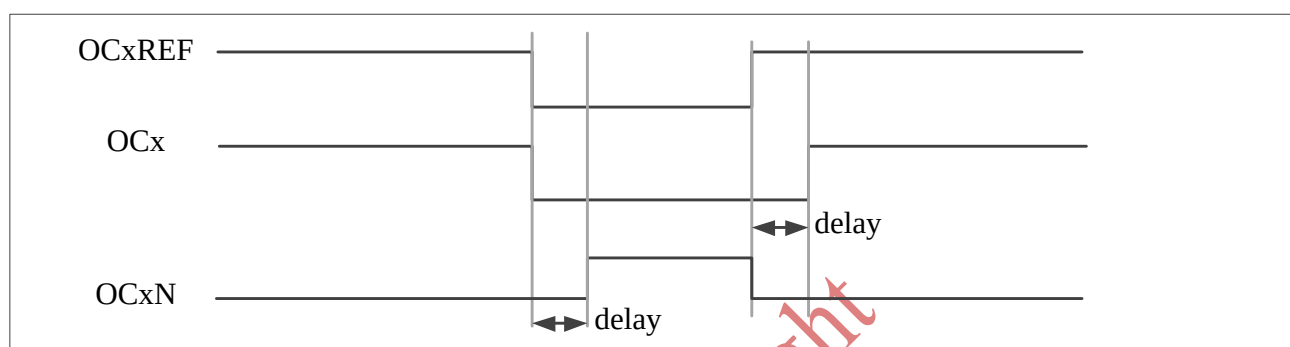


Figure 71 Complementary output with dead-time insertion

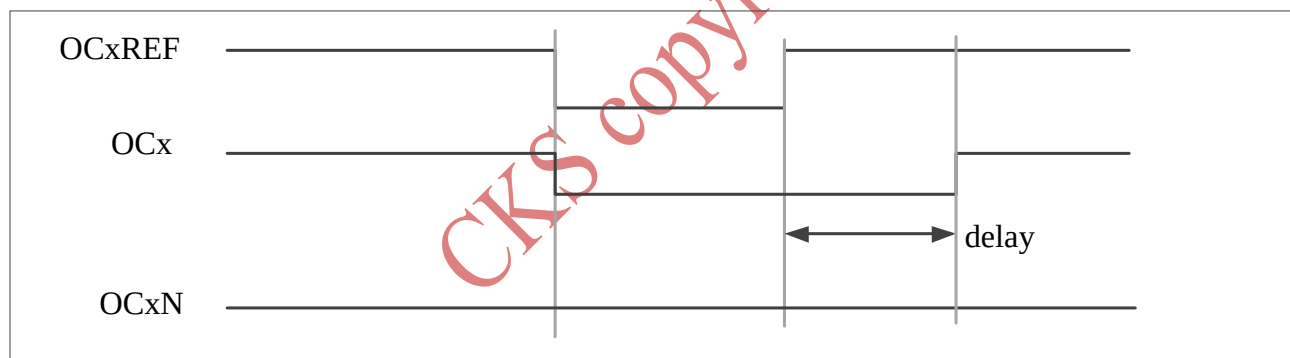


Figure 72 Dead-time waveforms with delay greater than the negative pulse

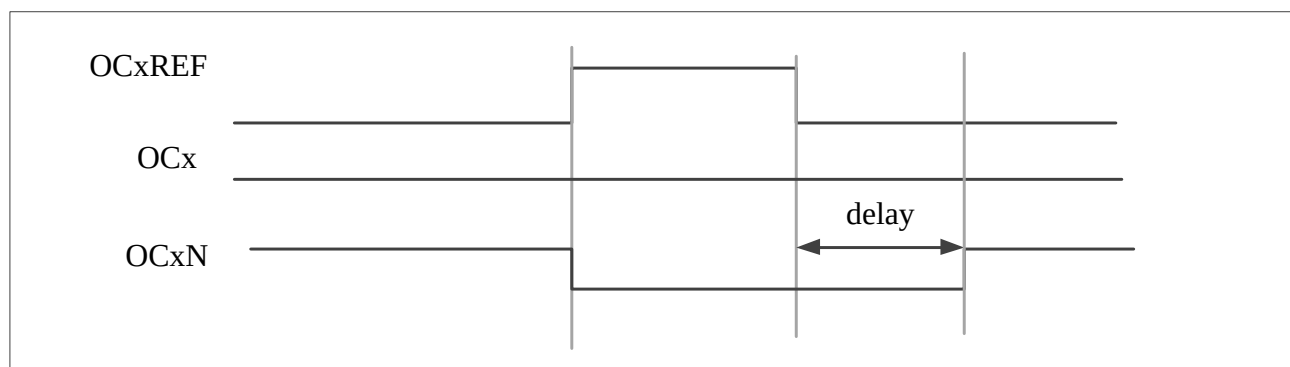


Figure 73 Dead-time waveforms with delay greater than the positive pulse

The dead-time delay is the same for each of the channels and is programmable with the DTG bits in the TIM1_BDTR register. Refer to *Section 11.4.18: TIM1 break and dead-time register (TIM1_BDTR)* for delay calculation.

Re-directing OCxREF to OCx or OCxN

In output mode (forced, output compare or PWM), OCxREF can be re-directed to the OCx output or to OCxN output by configuring the CCxE and CCxNE bits in the TIM1_CCER register.

This allows the user to send a specific waveform (such as PWM or static active level) on one output while the complementary remains at its inactive level. Other possibilities are to have both outputs at inactive level or both outputs active and complementary with dead-time.

Note: When only OCxN is enabled (CCxE=0, CCxNE=1), it is not complemented and becomes active as soon as OCxREF is high. For example, if CCxNP=0 then OCxN=OCxRef. On the other hand, when both OCx and OCxN are enabled (CCxE=CCxNE=1) OCx becomes active when OCxREF is high whereas OCxN is complemented and becomes active when OCxREF is low.

11.3.12 Using the break function

When using the break function, the output enable signals and inactive levels are modified according to additional control bits (MOE, OSSI and OSSR bits in the TIMx_BDTR register, OISx and OISxN bits in the TIM1_CR2 register). In any case, the OCx and OCxN outputs cannot be set both to active level at a given time. Refer to Table 54 for more details.

The break source can be either the break input pin or a clock failure event, generated by the Clock Security System (CSS), from the Reset Clock Controller. For further information on the Clock Security System, refer to Section 6.2.7: Clock security system (CSS).

When exiting from reset, the break circuit is disabled and the MOE bit is low. User can enable the break function by setting the BKE bit in the TIM1_BDTR register. The break input polarity can be selected by configuring the BKP bit in the same register. BKE and BKP can be modified at the same time. When the BKE and BKP bits are written, a delay of 1 APB clock cycle is applied before the writing is effective. Consequently, it is necessary to wait 1 APB clock period to correctly read back the bit after the write operation.

Because MOE falling edge can be asynchronous, a resynchronization circuit has been inserted between the actual signal (acting on the outputs) and the synchronous control bit (accessed in the TIM1_BDTR register). It results in some delays between the asynchronous and the synchronous signals. In particular, if MOE is written to 1 whereas it was low, a delay (dummy instruction) must be inserted before reading it correctly. This is because the user writes an asynchronous signal, but reads a synchronous signal.

When a break occurs (selected level on the break input):

- The MOE bit is cleared asynchronously, putting the outputs in inactive state, idle state or in reset state (selected by the OSSI bit). This feature functions even if the MCU oscillator is off.
- Each output channel is driven with the level programmed in the OISx bit in the TIM1_CR2 register as soon as MOE=0. If OSSI=0 then the timer releases the enable output else the enable output remains high.
- When complementary outputs are used:
 - The outputs are first put in reset state inactive state (depending on the polarity). This is done asynchronously so that it works even if no clock is provided to the timer.
 - If the timer clock is still present, then the dead-time generator is reactivated in order to drive the outputs with the level programmed in the OISx and OISxN bits after a dead-time. Even in this case, OCx and OCxN cannot be driven to their active level together. Note that because of the resynchronization on MOE, the dead-time duration is a bit longer than usual (around 2 ck_tim clock cycles).
 - If OSSI=0 then the timer releases the enable outputs else the enable outputs remain or become high as soon as one of the CCxE or CCxNE bits is high.
- The break status flag (BIF bit in the TIM1_SR register) is set. An interrupt can be generated if the BIE bit in the TIM1_DIER register is set. A DMA request can be sent if the BDE bit in the TIM1_DIER register is set.
- If the AOE bit in the TIM1_BDTR register is set, the MOE bit is automatically set again at the next update event UEV. This can be used to perform a regulation, for instance. Else, MOE remains low until it is written to '1' again. In this case, it can be used for security and the break input can be connected to an alarm from power drivers, thermal sensors or any security components.

Note: The break inputs is acting on level. Thus, the MOE cannot be set while the break input is active (neither automatically nor by software). In the meantime, the status flag BIF cannot be cleared.

The break can be generated by the BRK input which has a programmable polarity and an enable bit BKE in the TIM1_BDTR Register.

In addition to the break input and the output management, a write protection has been implemented inside the break circuit to safeguard the application. It allows freezing the configuration of several parameters (dead-time duration, OCx/OCxN polarities and state when disabled, OCxM configurations, break enable and polarity). The user can choose from three levels of protection selected by the LOCK bits in the TIM1_BDTR register. Refer to *Section 11.4.18: TIM1 break and dead-time register (TIM1_BDTR)*. The LOCK bits can be written only once after an MCU reset.

Figure 74 shows an example of behavior of the outputs in response to a break.

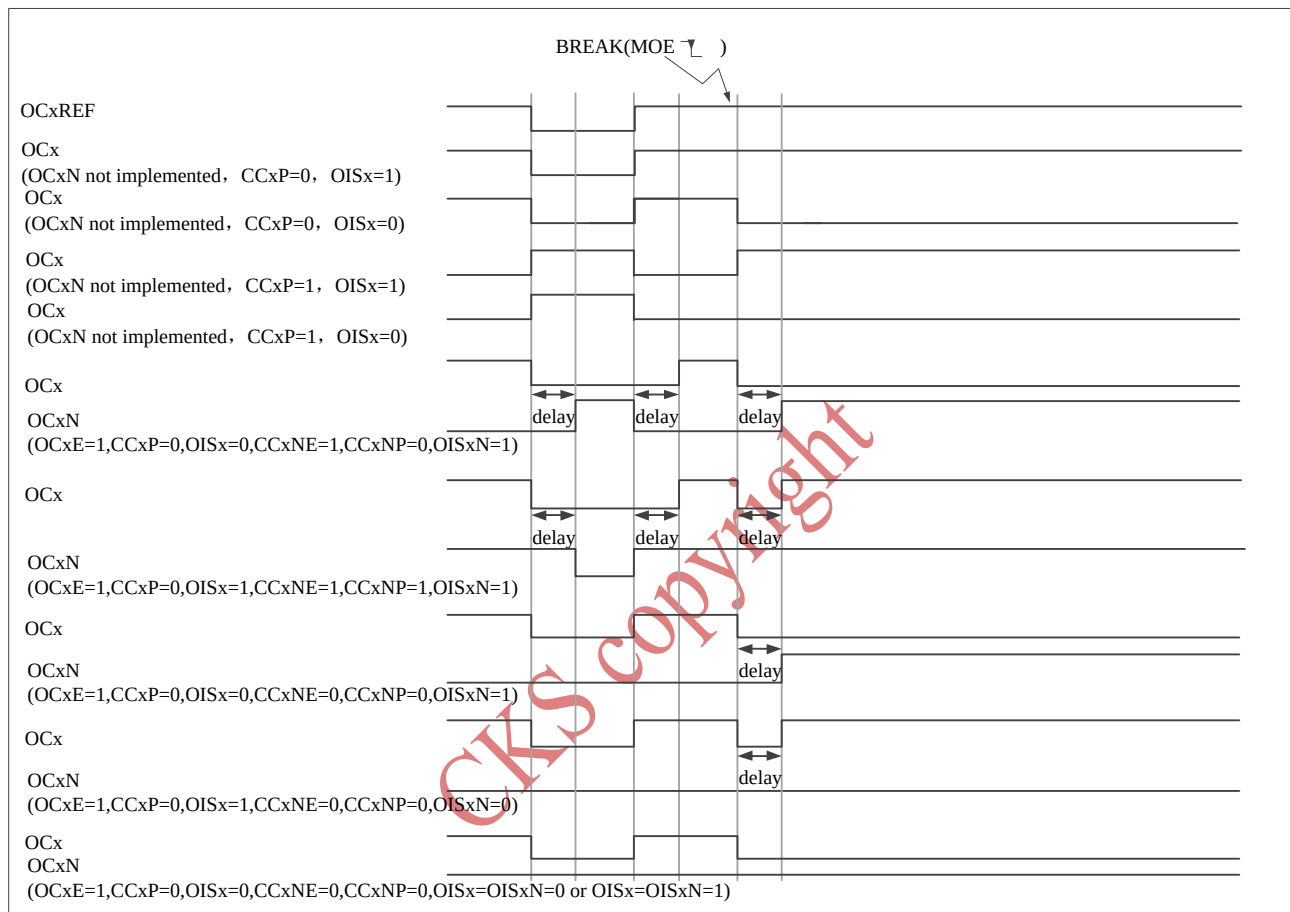


Figure 74 Output behavior in response to a break

11.3.13 Clearing the OCxREF signal on an external event

The OCxREF signal for a given channel can be driven Low by applying a High level to the ETRF input (OCxCE enable bit of the corresponding TIM1_CCMRx register set to '1'). The OCxREF signal remains Low until the next update event, UEV, occurs.

This function can only be used in output compare and PWM modes, and does not work in forced mode.

For example, the ETR signal can be connected to the output of a comparator to be used for current handling. In this case, the ETR must be configured as follow:

1. The External Trigger Prescaler should be kept off: bits ETPS[1:0] of the TIM1_SMCR register set to '00'.
2. The external clock mode 2 must be disabled: bit ECE of the TIM1_SMCR register set to '0'.
3. The External Trigger Polarity (ETP) and the External Trigger Filter (ETF) can be configured according to the user needs.

Figure 75 shows the behavior of the OCxREF signal when the ETRF Input becomes High, for both values of the enable bit OCxCE. In this example, the timer TIM1 is programmed in PWM mode.

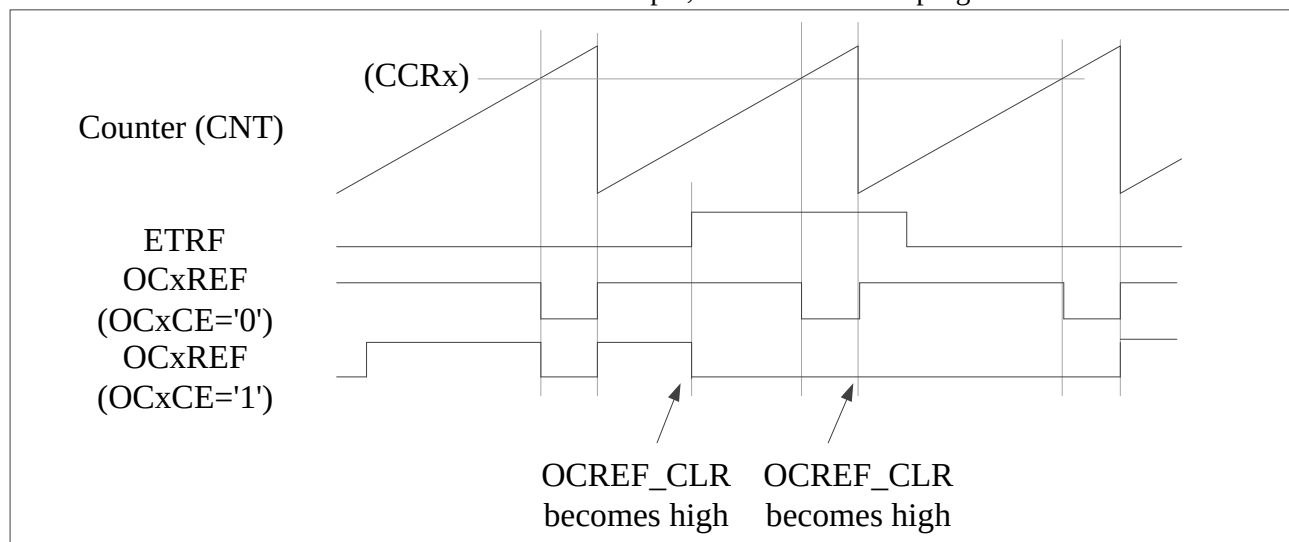


Figure 75 Clearing TIM1 OCxREF

11.3.14 6-step PWM generation

When complementary outputs are used on a channel, preload bits are available on the OCxM, CCxE and CCxNE bits. The preload bits are transferred to the shadow bits at the COM commutation event. The user can thus program in advance the configuration for the next step and change the configuration of all the channels at the same time. COM can be generated by software by setting the COM bit in the TIM1_EGR register or by hardware (on TRGI rising edge).

A flag is set when the COM event occurs (COMIF bit in the TIM1_SR register), which can generate an interrupt (if the COMIE bit is set in the TIM1_DIER register) or a DMA request (if the COMDE bit is set in the TIM1_DIER register).

Figure 76 describes the behavior of the OCx and OCxN outputs when a COM event occurs, in 3 different examples of programmed configurations.

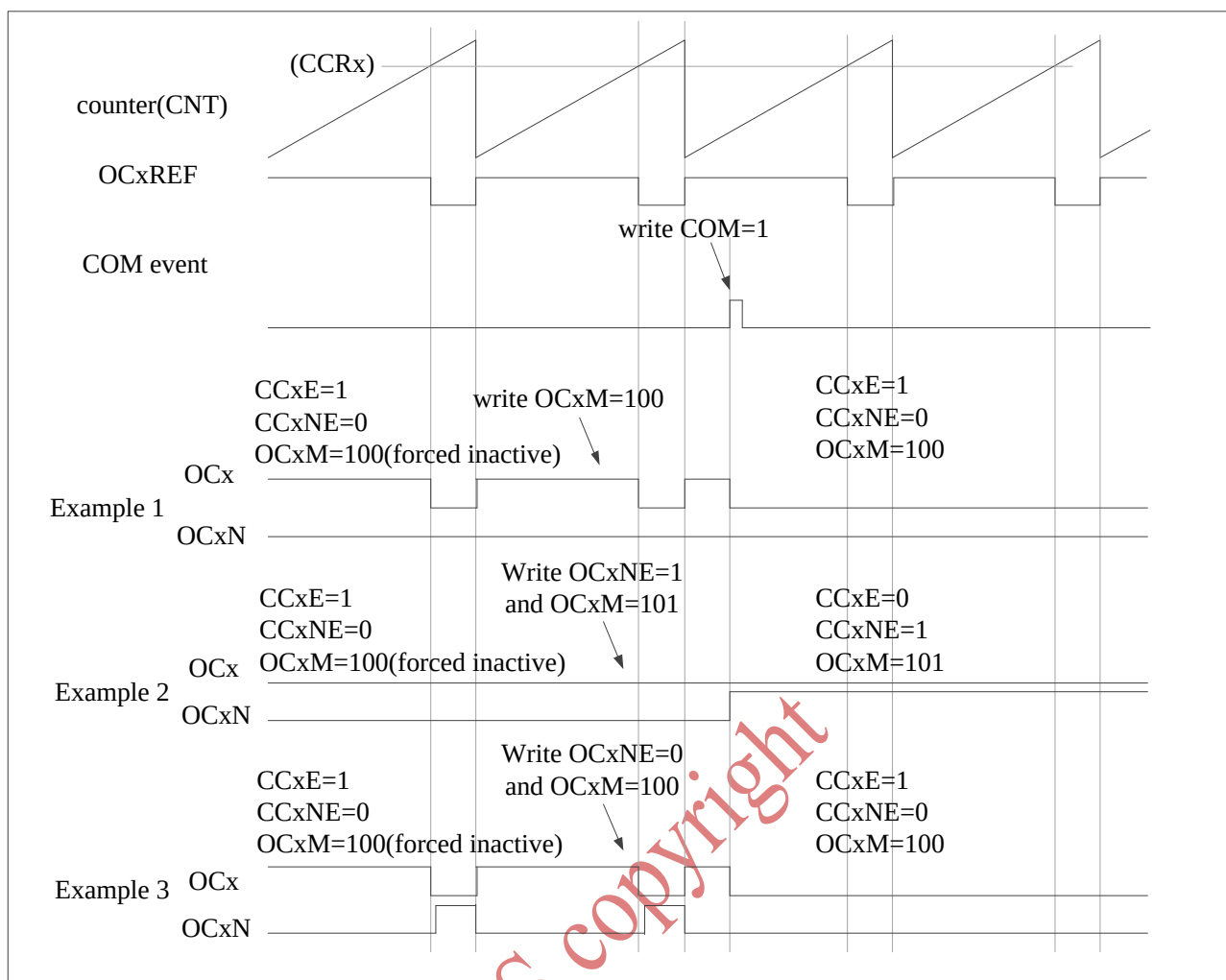


Figure 76 6-step generation, COM example (OSSR=1)

11.3.15 One-pulse mode

One-pulse mode (OPM) is a particular case of the previous modes. It allows the counter to be started in response to a stimulus and to generate a pulse with a programmable length after a programmable delay.

Starting the counter can be controlled through the slave mode controller. Generating the waveform can be done in output compare mode or PWM mode. Select One-pulse mode by setting the OPM bit in the TIM1_CR1 register. This makes the counter stop automatically at the next update event UEV.

A pulse can be correctly generated only if the compare value is different from the counter initial value. Before starting (when the timer is waiting for the trigger), the configuration must be:

- In upcounting: $CNT < CCRx \leq ARR$ (in particular, $0 < CCRx$)
- In downcounting: $CNT > CCRx$

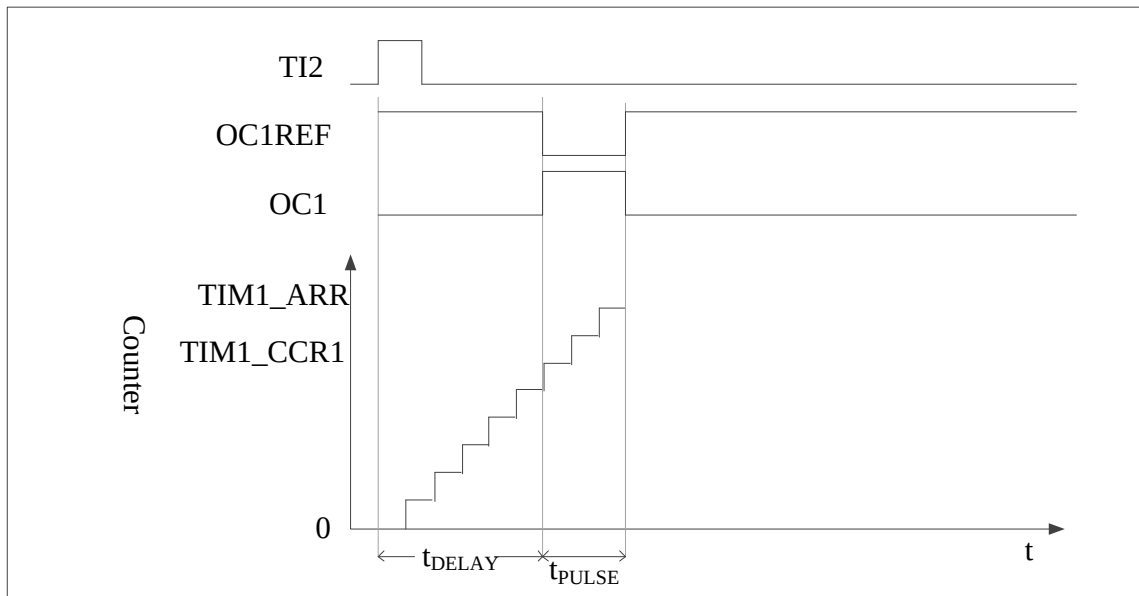


Figure 77 Example of one pulse mode

For example the user may want to generate a positive pulse on OC1 with a length of t_{PULSE} and after a delay of t_{DELAY} as soon as a positive edge is detected on the TI2 input pin.

Let's use TI2FP2 as trigger 1:

- Map TI2FP2 to TI2 by writing CC2S='01' in the TIM1_CCMR1 register.
- TI2FP2 must detect a rising edge, write CC2P='0' in the TIM1_CCER register.
- Configure TI2FP2 as trigger for the slave mode controller (TRGI) by writing TS='110' in the TIM1_SMCR register.
- TI2FP2 is used to start the counter by writing SMS to '110' in the TIM1_SMCR register (trigger mode). The OPM waveform is defined by writing the compare registers (taking into account the clock frequency and the counter prescaler).
- The t_{DELAY} is defined by the value written in the TIM1_CCR1 register.
- The t_{PULSE} is defined by the difference between the auto-reload value and the compare value (TIM1_ARR - TIM1_CCR1).
- Let us say the user wants to build a waveform with a transition from '0' to '1' when a compare match occurs and a transition from '1' to '0' when the counter reaches the auto-reload value. To do this, enable PWM mode 2 by writing OC1M=111 in the TIM1_CCMR1 register. The user can optionally enable the preload registers by writing OC1PE='1' in the TIM1_CCMR1 register and ARPE in the TIM1_CR1 register. In this case the compare value must be written in the TIM1_CCR1 register, the auto-reload value in the TIM1_ARR register, generate an update by setting the UG bit and wait for external trigger event on TI2. CC1P is written to '0' in this example.

In our example, the DIR and CMS bits in the TIM1_CR1 register should be low.

The user only wants one pulse (Single mode), so '1' must be written in the OPM bit in the TIM1_CR1 register to stop the counter at the next update event (when the counter rolls over from the auto-reload value back to 0). When OPM bit in the TIM1_CR1 register is set to '0', so the Repetitive Mode is selected.

Particular case: OCx fast enable:

In One-pulse mode, the edge detection on TIx input set the CEN bit which enables the counter. Then the comparison between the counter and the compare value makes the output toggle. But several clock cycles are needed for these operations and it limits the minimum delay $t_{DELAY\ min}$ we can get. If the user wants to output a waveform with the minimum delay, the OCxFE bit in the TIM1_CCMRx register must be set. Then OCxRef (and OCx) are forced in response to the stimulus,

without taking in account the comparison. Its new level is the same as if a compare match had occurred. OCxFE acts only if the channel is configured in PWM1 or PWM2 mode.

11.3.16 Encoder interface mode

To select Encoder Interface mode write SMS='001' in the TIM1_SMCR register if the counter is counting on TI2 edges only, SMS='010' if it is counting on TI1 edges only and SMS='011' if it is counting on both TI1 and TI2 edges.

Select the TI1 and TI2 polarity by programming the CC1P and CC2P bits in the TIMx_CCER register. When needed, the user can program the input filter as well.

The two inputs TI1 and TI2 are used to interface to an incremental encoder. Refer to Table 52. The counter is clocked by each valid transition on TI1FP1 or TI2FP2 (TI1 and TI2 after input filter and polarity selection, TI1FP1=TI1 if not filtered and not inverted, TI2FP2=TI2 if not filtered and not inverted) assuming that it is enabled (CEN bit in TIM1_CR1 register written to '1'). The sequence of transitions of the two inputs is evaluated and generates count pulses as well as the direction signal. Depending on the sequence the counter counts up or down, the DIR bit in the TIM1_CR1 register is modified by hardware accordingly. The DIR bit is calculated at each transition on any input (TI1 or TI2), whatever the counter is counting on TI1 only, TI2 only or both TI1 and TI2.

Encoder interface mode acts simply as an external clock with direction selection. This means that the counter just counts continuously between 0 and the auto-reload value in the TIM1_ARR register (0 to ARR or ARR down to 0 depending on the direction). So user must configure TIM1_ARR before starting, in the same way, the capture, compare, prescaler, repetition counter, trigger output features continue to work as normal. Encoder mode and External clock mode 2 are not compatible and must not be selected together.

In this mode, the counter is modified automatically following the speed and the direction of the incremental encoder and its content, therefore, always represents the encoder's position. The count direction correspond to the rotation direction of the connected sensor. Table 52 summarizes the possible combinations, assuming TI1 and TI2 do not switch at the same time.

Table 52 Counting direction versus encoder signals

Active edge	Level on opposite signal (TI1FP1 for TI2, TI2FP2 for TI1)	TI1FP1 signal		TI2FP2 signal	
		Rising	Falling	Rising	Falling
Counting on TI1 only	High	Down	Up	No Count	No Count
	Low	Up	Down	No Count	No Count
Counting on TI2 only	High	No Count	No Count	Up	Down
	Low	No Count	No Count	Down	Up
Counting on TI1 and TI2	High	Down	Up	Up	Down
	Low	Up	Down	Down	Up

An external incremental encoder can be connected directly to the MCU without external interface logic. However, comparators are normally be used to convert the encoder's differential outputs to digital signals. This greatly increases noise immunity. The third encoder output which indicate the mechanical zero position, may be connected to an external interrupt input and trigger a counter reset. Figure 78 gives an example of counter operation, showing count signal generation and direction control. It also shows how input jitter is compensated where both edges are selected. This might occur if the sensor is positioned near to one of the switching points. For this example we assume that the configuration is the following:

- CC1S='01' (TIMx_CCMR1 register, TI1FP1 mapped on TI1).
- CC2S='01' (TIMx_CCMR2 register, TI1FP2 mapped on TI2).
- CC1P='0', and IC1F = '0000' (TIMx_CCER register, TI1FP1 non-inverted, TI1FP1=TI1).
- CC2P='0', and IC2F = '0000' (TIMx_CCER register, TI1FP2 non-inverted, TI1FP2= TI2).
- SMS='011' (TIMx_SMCR register, both inputs are active on both rising and falling edges).

- CEN='1' (TIMx_CR1 register, Counter enabled).

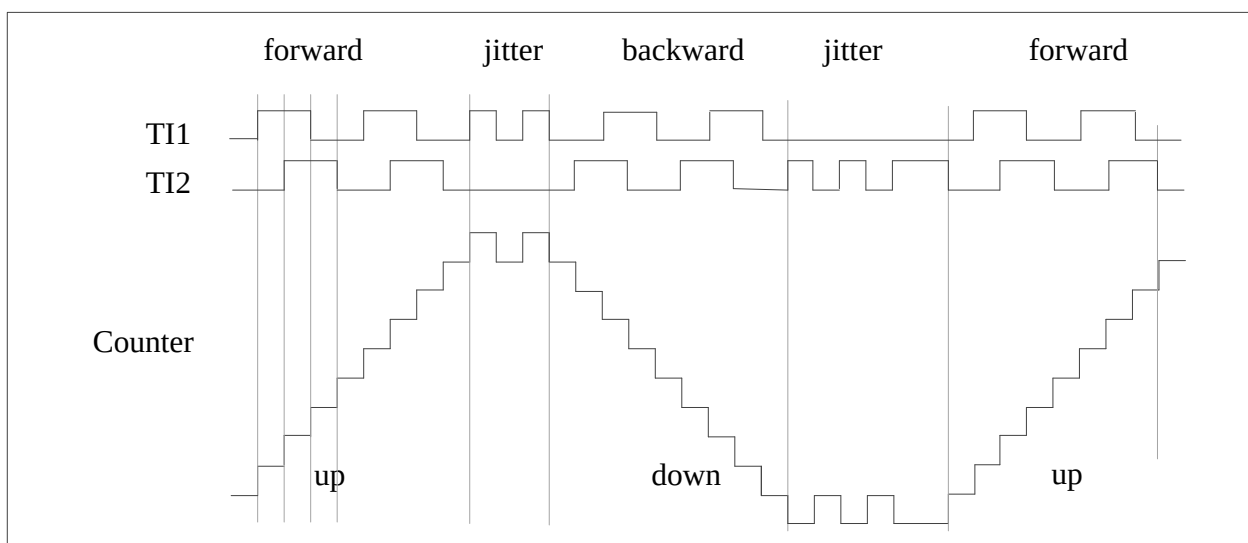


Figure 78 Example of counter operation in encoder interface mode

Figure 79 gives an example of counter behavior when TI1FP1 polarity is inverted (same configuration as above except CC1P='1').

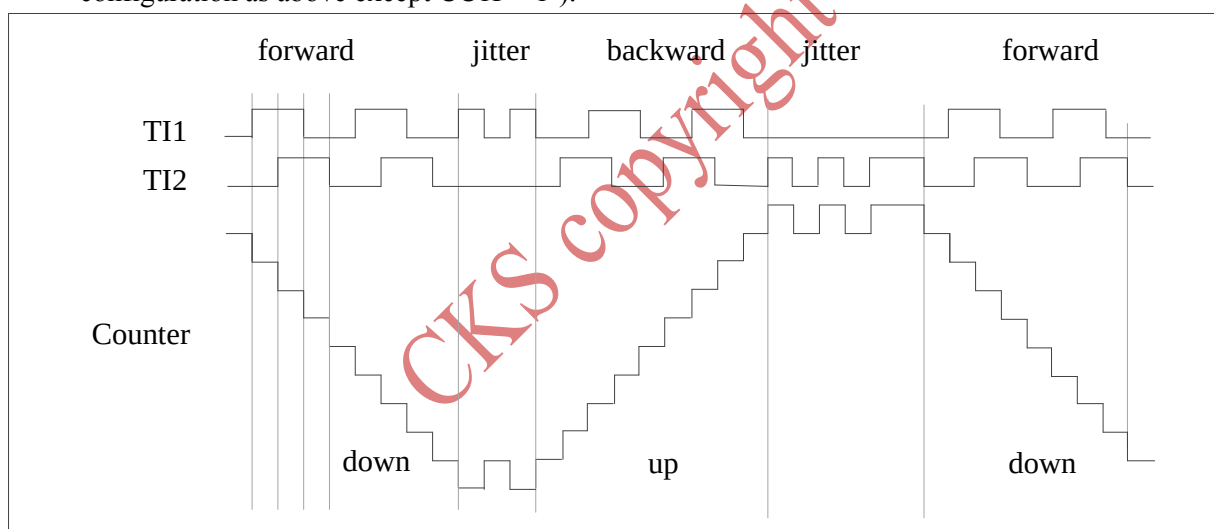


Figure 79 Example of encoder interface mode with TI1FP1 polarity inverted

The timer, when configured in Encoder Interface mode provides information on the sensor's current position. The user can obtain dynamic information (speed, acceleration, deceleration) by measuring the period between two encoder events using a second timer configured in capture mode. The output of the encoder which indicates the mechanical zero can be used for this purpose. Depending on the time between two events, the counter can also be read at regular times. This can be done by latching the counter value into a third input capture register if available (then the capture signal must be periodic and can be generated by another timer). When available, it is also possible to read its value through a DMA request generated by a real-time clock.

11.3.17 Timer input XOR function

The TI1S bit in the TIM1_CR2 register, allows the input filter of channel 1 to be connected to the output of a XOR gate, combining the three input pins TIM1_CH1, TIM1_CH2 and TIM1_CH3.

The XOR output can be used with all the timer input functions such as trigger or input capture. An example of this feature used to interface Hall sensors is given in Section 11.3.18

11.3.18 Interfacing with Hall sensors

This is done using the advanced-control timer (TIM1) to generate PWM signals to drive the motor and another timer TIMx (TIM2, TIM3 or TIM4) referred to as “interfacing timer” in Figure 80. The “interfacing timer” captures the 3 timer input pins (CC1, CC2, CC3) connected through a XOR to the TI1 input channel (selected by setting the TI1S bit in the TIM1_CR2 register).

The slave mode controller is configured in reset mode; the slave input is TI1F_ED. Thus, each time one of the 3 inputs toggles, the counter restarts counting from 0. This creates a time base triggered by any change on the Hall inputs.

On the “interfacing timer”, capture/compare channel 1 is configured in capture mode, capture signal is TRC (see Figure 76). The captured value, which corresponds to the time elapsed between 2 changes on the inputs, gives information about motor speed.

The “interfacing timer” can be used in output mode to generate a pulse which changes the configuration of the channels of the advanced-control timer (TIM1) (by triggering a COM event). The TIM1 timer is used to generate PWM signals to drive the motor. To do this, the interfacing timer channel must be programmed so that a positive pulse is generated after a programmed delay (in output compare or PWM mode). This pulse is sent to the advanced-control timer (TIM1) through the TRGO output.

Example: the user wants to change the PWM configuration of the advanced-control timer TIM1 after a programmed delay each time a change occurs on the Hall inputs connected to one of the TIM1 timers.

- Configure 3 timer inputs ORed to the TI1 input channel by writing the TI1S bit in the TIM1_CR2 register to ‘1’.
- Program the time base: write the TIM1_ARR to the max value (the counter must be cleared by the TI1 change. Set the prescaler to get a maximum counter period longer than the time between 2 changes on the sensors,
- Program channel 1 in capture mode (TRC selected): write the CC1S bits in the TIM1_CCMR1 register to ‘11’. The user can also program the digital filter if needed,
- Program channel 2 in PWM 2 mode with the desired delay: write the OC2M bits to ‘111’ and the CC2S bits to ‘00’ in the TIM1_CCMR1 register,
- Select OC2REF as trigger output on TRGO: write the MMS bits in the TIM1_CR2 register to ‘101’.

In the advanced-control timer TIM1, the right ITR input must be selected as trigger input, the timer is programmed to generate PWM signals, the capture/compare control signals are preloaded (CCPC=1 in the TIM1_CR2 register) and the COM event is controlled by the trigger input (CCUS=1 in the TIM1_CR2 register). The PWM control bits (CCxE, OCxM) are written after a COM event for the next step (this can be done in an interrupt subroutine generated by the rising edge of OC2REF). Figure 80 describes this example.

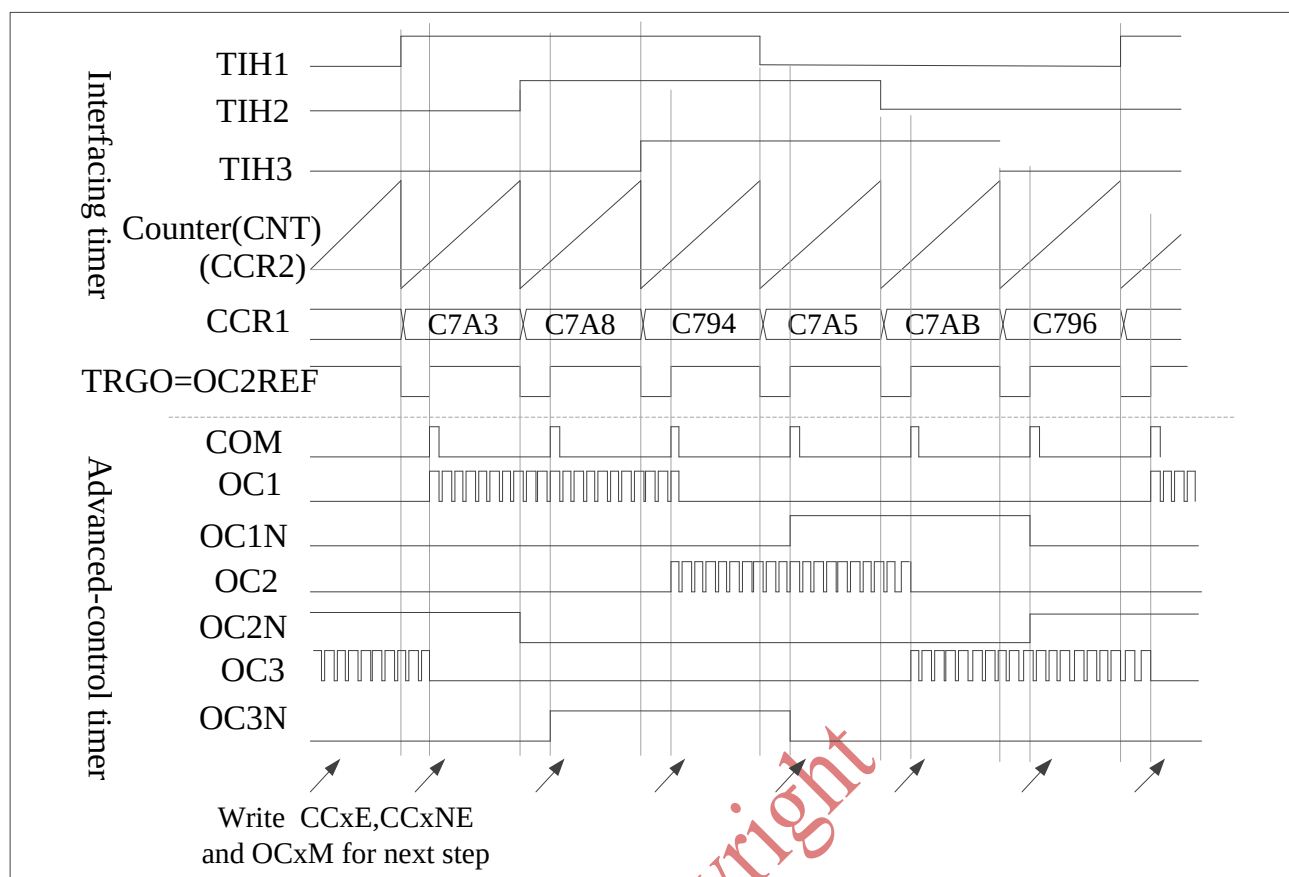


Figure 80 Example of Hall sensor interface

11.3.19 TIM1 and external trigger synchronization

The TIM1 timer can be synchronized with an external trigger in several modes: Reset mode, Gated mode and Trigger mode.

Slave mode: Reset mode

The counter and its prescaler can be reinitialized in response to an event on a trigger input. Moreover, if the URS bit from the TIM1_CR1 register is low, an update event UEV is generated. Then all the preloaded registers (TIM1_ARR, TIM1_CCRx) are updated.

In the following example, the upcounter is cleared in response to a rising edge on TI1 input:

- Configure the channel 1 to detect rising edges on TI1. Configure the input filter duration (in this example, we don't need any filter, so we keep IC1F=0000). The capture prescaler is not used for triggering, so there's no need to configure it. The CC1S bits select the input capture source only, CC1S = 01 in the TIM1_CCMR1 register. Write CC1P=0 in TIM1_CCER register to validate the polarity (and detect rising edges only).
- Configure the timer in reset mode by writing SMS=100 in TIMx_SMCR register. Select TI1 as the input source by writing TS=101 in TIM1_SMCR register.
- Start the counter by writing CEN=1 in the TIM1_CR1 register.

The counter starts counting on the internal clock, then behaves normally until TI1 rising edge. When TI1 rises, the counter is cleared and restarts from 0. In the meantime, the trigger flag is set (TIF bit in the TIM1_SR register) and an interrupt request, or a DMA request can be sent if enabled (depending on the TIE and TDE bits in TIM1_DIER register).

The following figure shows this behavior when the auto-reload register TIM1_ARR=0x36. The delay between the rising edge on TI1 and the actual reset of the counter is due to the resynchronization circuit on TI1 input.

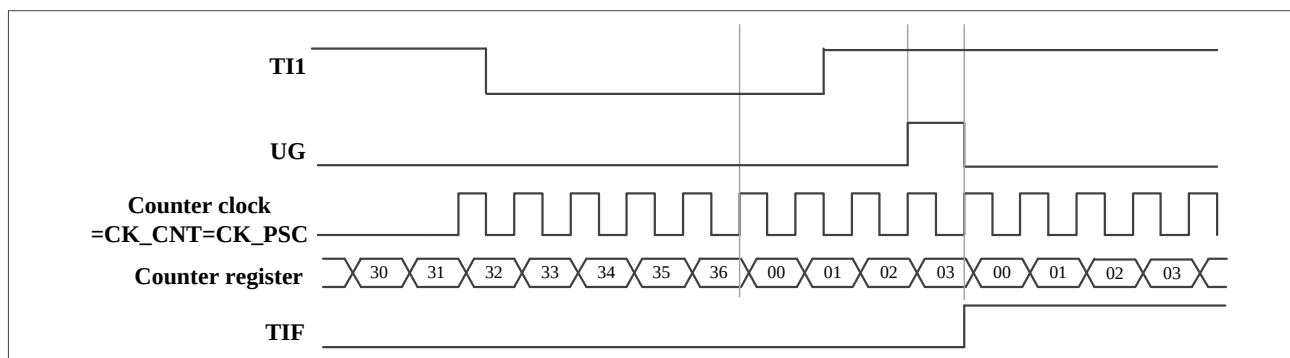


Figure 81 Control circuit in reset mode

Slave mode: Gated mode

The counter can be enabled depending on the level of a selected input.

In the following example, the upcounter counts only when TI1 input is low:

- Configure the channel 1 to detect low levels on TI1. Configure the input filter duration (in this example, we don't need any filter, so we keep IC1F=0000). The capture prescaler is not used for triggering, so the user does not need to configure it. The CC1S bits select the input capture source only, CC1S=01 in TIM1_CCMR1 register. Write CC1P=1 in TIM1_CCER register to validate the polarity (and detect low level only).
- Configure the timer in gated mode by writing SMS=101 in TIM1_SMCR register. Select TI1 as the input source by writing TS=101 in TIM1_SMCR register.
- Enable the counter by writing CEN=1 in the TIM1_CR1 register (in gated mode, the counter doesn't start if CEN=0, whatever is the trigger input level).

The counter starts counting on the internal clock as long as TI1 is low and stops as soon as TI1 becomes high. The TIF flag in the TIM1_SR register is set both when the counter starts or stops. The delay between the rising edge on TI1 and the actual stop of the counter is due to the resynchronization circuit on TI1 input.

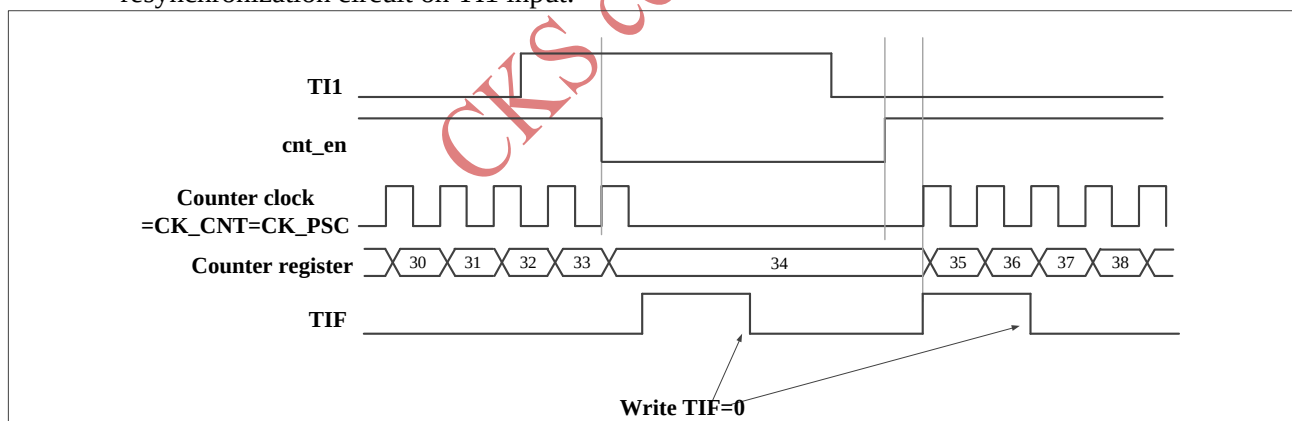


Figure 82 Control circuit in gated mode

Slave mode: Trigger mode

The counter can start in response to an event on a selected input.

In the following example, the upcounter starts in response to a rising edge on TI2 input:

- Configure the channel 2 to detect rising edges on TI2. Configure the input filter duration (in this example, we don't need any filter, so we keep IC2F=0000). The capture prescaler is not used for triggering, so there's no need to configure it. The CC2S bits are configured to select the input capture source only, CC2S=01 in TIM1_CCMR1 register. Write CC2P=1 in TIM1_CCER register to validate the polarity (and detect low level only).
- Configure the timer in trigger mode by writing SMS=110 in TIM1_SMCR register. Select TI2 as the input source by writing TS=110 in TIM1_SMCR register.

When a rising edge occurs on TI2, the counter starts counting on the internal clock and the TIF flag is set.

The delay between the rising edge on TI2 and the actual start of the counter is due to the resynchronization circuit on TI2 input.

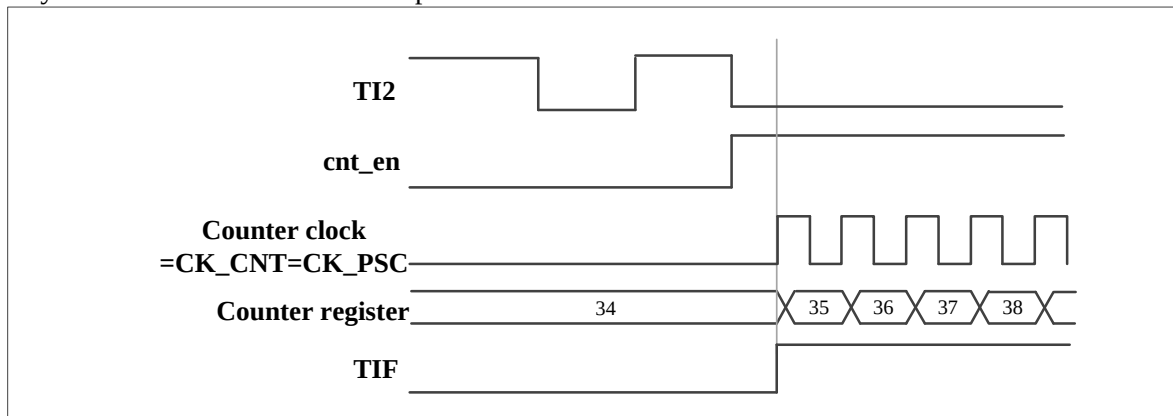


Figure 83 Control circuit in trigger mode

Slave mode: external clock mode 2 + trigger mode

The external clock mode 2 can be used in addition to another slave mode (except external clock mode 1 and encoder mode). In this case, the ETR signal is used as external clock input, and another input can be selected as trigger input (in reset mode, gated mode or trigger mode). It is recommended not to select ETR as TRGI through the TS bits of TIM1_SMCR register.

In the following example, the upcounter is incremented at each rising edge of the ETR signal as soon as a rising edge of TI1 occurs:

1. Configure the external trigger input circuit by programming the TIM1_SMCR register as follows:
 - ETF = 0000: no filter
 - ETPS = 00: prescaler disabled
 - ETP = 0: detection of rising edges on ETR and ECE=1 to enable the external clock mode 2.
2. Configure the channel 1 as follows, to detect rising edges on TI:
 - IC1F=0000: no filter.
 - The capture prescaler is not used for triggering and does not need to be configured.
 - CC1S=01 in TIM1_CCMR1 register to select only the input capture source
 - CC1P=0 in TIM1_CCER register to validate the polarity (and detect rising edge only).
3. Configure the timer in trigger mode by writing SMS=110 in TIM1_SMCR register. Select TI1 as the input source by writing TS=101 in TIM1_SMCR register.

A rising edge on TI1 enables the counter and sets the TIF flag. The counter then counts on ETR rising edges.

The delay between the rising edge of the ETR signal and the actual reset of the counter is due to the resynchronization circuit on ETRP input.

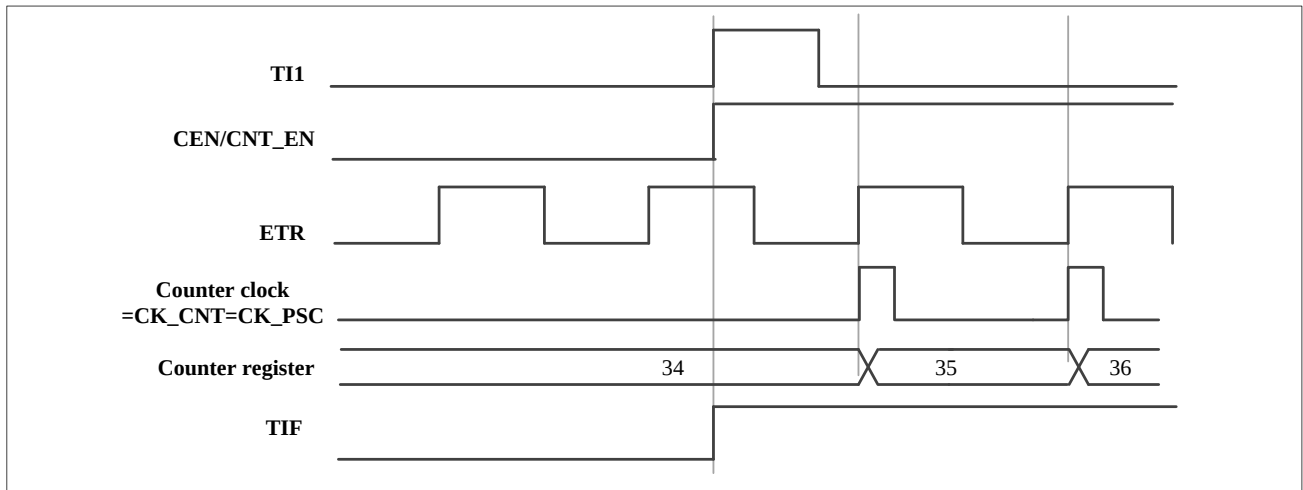


Figure 84 Control circuit in external clock mode 2 + trigger mode

11.3.20 Timer synchronization

The TIM timers are linked together internally for timer synchronization or chaining. Refer to Section 12.3.15.

11.3.21 Debug mode

When the microcontroller enters debug mode (Cortex[®]-M3 core halted), the TIM1 counter either continues to work normally or stops, depending on DBG_TIM1_STOP configuration bit in DBG module. For more details, refer to Section 22.16.2: *Debug support for timers, watchdog, bxCAN and I2C*.

11.4 TIM1 registers

Refer to Section 1 for a list of abbreviations used in register descriptions.
The peripheral registers can be accessed by half-words (16-bit) or words (32-bit).

11.4.1 TIM1 control register 1 (TIM1_CR1)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CKD[1:0]		ARPE	CMS[1:0]		DIR	OPM	URS	UDIS	CEN
						rw		rw	rw	rw	rw	rw	rw	rw	rw
Bits15:10		Reserved, must be kept at reset value.													
Bits 9:8		CKD[1:0]: Clock division This bit-field indicates the division ratio between the timer clock (CK_INT) frequency and the dead-time and sampling clock (t _{DTS})used by the dead-time generators and the digital filters (ETR, TIx), 00: t _{DTS} =t _{CK_INT} 01: t _{DTS} =2*t _{CK_INT} 10: t _{DTS} =4*t _{CK_INT} 11: Reserved, do not program this value													
Bit 7		ARPE: Auto-reload preload enable													

	0: TIM1_ARR register is not buffered 1: TIM1_ARR register is buffered
Bits 6:5	CMS[1:0]: Center-aligned mode selection 00: Edge-aligned mode. The counter counts up or down depending on the direction bit (DIR). 01: Center-aligned mode 1. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIM1_CCMRx register) are set only when the counter is counting down. 10: Center-aligned mode 2. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIM1_CCMRx register) are set only when the counter is counting up. 11: Center-aligned mode 3. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIM1_CCMRx register) are set both when the counter is counting up or down. <i>Note: It is not allowed to switch from edge-aligned mode to center-aligned mode as long as the counter is enabled (CEN=1)</i>
Bit 4	DIR: Direction 0: Counter used as upcounter 1: Counter used as downcounter <i>Note: This bit is read only when the timer is configured in Center-aligned mode or Encoder mode.</i>
Bit 3	OPM: One pulse mode 0: Counter is not stopped at update event 1: Counter stops counting at the next update event (clearing the bit CEN)
Bit 2	URS: Update request source This bit is set and cleared by software to select the UEV event sources. 0: Any of the following events generate an update interrupt or DMA request if enabled. These events can be: - Counter overflow/underflow - Setting the UG bit - Update generation through the slave mode controller 1: Only counter overflow/underflow generates an update interrupt or DMA request if enabled.
Bit 1	UDIS: Update disable This bit is set and cleared by software to enable/disable UEV event generation. 0: UEV enabled. The Update (UEV) event is generated by one of the following events: - Counter overflow/underflow - Setting the UG bit - Update generation through the slave modecontroller Buffered registers are then loaded with their preload values. 1: UEV disabled. The Update event is not generated, shadow registers keep their value (ARR, PSC, CCRx). However the counter and the prescaler are reinitialized if the UG bit is set or if a hardware reset is received from the slave mode controller.
Bit 0	CEN: Counter enable 0: Counter disabled 1: Counter enabled <i>Note: External clock, gated mode and encoder mode can work only if the CEN bit has been previously set by software. However trigger mode can set the CEN bit automatically by hardware.</i>

11.4.2 TIM1 control register 2 (TIM1_CR2)

Address offset: 0x04

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	OIS4	OIS3N	OIS3	OIS2N	OIS2	OIS1N	OIS1	TI1S	MMS[2: 0]		CCDS	CCUS	Res	CCPC	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw
Bit 15	Reserved, must be kept at reset value.														
Bit 14	OIS4: Output Idle state 4 (OC4 output). Refer to OIS1 bit.														
Bit 13	OIS3N: Output Idle state 3 (OC3N output). Refer to OIS1N bit.														
Bit 12	OIS3: Output Idle state 3 (OC3 output). Refer to OIS1 bit.														
Bit 11	OIS2N: Output Idle state 2 (OC2N output). Refer to OIS1N bit.														
Bit 10	OIS2: Output Idle state 2 (OC2 output). Refer to OIS1 bit.														

Bit 9	OIS1N : Output Idle state 1 (OC1N output) 0: OC1N=0 after a dead-time when MOE=0 1: OC1N=1 after a dead-time when MOE=0 <i>Note: This bit can not be modified as long as LOCK level 1, 2 or 3 has been programmed (LOCK bits in TIM1_BDTR register).</i>
Bit 8	OIS1 : Output Idle state 1 (OC1 output) 0: OC1=0 (after a dead-time if OC1N is implemented) when MOE=0 1: OC1=1 (after a dead-time if OC1N is implemented) when MOE=0 <i>Note: This bit can not be modified as long as LOCK level 1, 2 or 3 has been programmed (LOCK bits in TIM1_BDTR register).</i>
Bit 7	TI1S : TI1 selection 0: The TIM1_CH1 pin is connected to TI1 input 1: The TIM1_CH1, CH2 and CH3 pins are connected to the TI1 input (XOR combination)
Bits 6: 4	MMS[2:0] : Master mode selection These bits allow to select the information to be sent in master mode to slave timers for synchronization (TRGO). The combination is as follows: 000: Reset - the UG bit from the TIM1_EGR register is used as trigger output (TRGO). If the reset is generated by the trigger input (slave mode controller configured in reset mode) then the signal on TRGO is delayed compared to the actual reset. 001: Enable - the Counter Enable signal CNT_EN is used as trigger output (TRGO). It is useful to start several timers at the same time or to control a window in which a slave timer is enable. The Counter Enable signal is generated by a logic OR between CEN control bit and the trigger input when configured in gated mode. When the Counter Enable signal is controlled by the trigger input, there is a delay on TRGO, except if the master/slave mode is selected (see the MSM bit description in TIM1_SMCR register). 010: Update - The update event is selected as trigger output (TRGO). For instance a master timer can then be used as a prescaler for a slave timer. 011: Compare Pulse - The trigger output send a positive pulse when the CC1IF flag is to be set (even if it was already high), as soon as a capture or a compare match occurred. (TRGO). 100: Compare - OC1REF signal is used as trigger output (TRGO) 101: Compare - OC2REF signal is used as trigger output (TRGO) 110: Compare - OC3REF signal is used as trigger output (TRGO) 111: Compare - OC4REF signal is used as trigger output (TRGO) <i>Note: The clock of the slave timer and ADC must be enabled prior to receiving events from the master timer, and must not be changed on-the-fly while triggers are received from the master timer</i>
Bit 3	CCDS : Capture/compare DMA selection 0: CCx DMA request sent when CCx event occurs 1: CCx DMA requests sent when update event occurs
Bit 2	CCUS : Capture/compare control update selection 0: When capture/compare control bits are preloaded (CCPC=1), they are updated by setting the COMG bit only 1: When capture/compare control bits are preloaded (CCPC=1), they are updated by setting the COMG bit or when an rising edge occurs on TRGI <i>Note: This bit acts only on channels that have a complementary output.</i>
Bit 1	Reserved, must be kept at reset value.
Bit 0	CCPC : Capture/compare preloaded control 0: CCxE, CCxNE and OCxM bits are not preloaded 1: CCxE, CCxNE and OCxM bits are preloaded, after having been written, they are updated only when a commutation event (COM) occurs (COMG bit set or rising edge detected on TRGI, depending on the CCUS bit). <i>Note: This bit acts only on channels that have a complementary output.</i>

11.4.3 TIM1 slave mode control register (TIM1_SMCR)

Address offset: 0x08

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETP	ECE	ETPS[1:0]			ETF[3:0]			MSM		TS[2:0]		Res		SMS[2:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw

Bit 15	ETP: External trigger polarity This bit selects whether ETR or $\overline{\text{ETR}}$ is used for trigger operations 0: ETR is non-inverted, active at high level or rising edge. 1: ETR is inverted, active at low level or falling edge.
Bit 14	ECE: External clock enable This bit enables External clock mode 2. 0: External clock mode 2 disabled 1: External clock mode 2 enabled. The counter is clocked by any active edge on the ETRF signal. <i>Note: 1: Setting the ECE bit has the same effect as selecting external clock mode 1 with TRGI connected to ETRF (SMS=111 and TS=111).</i> <i>2: It is possible to simultaneously use external clock mode 2 with the following slave modes: reset mode, gated mode and trigger mode. Nevertheless, TRGI must not be connected to ETRF in this case (TS bits must not be 111).</i> <i>3: If external clock mode 1 and external clock mode 2 are enabled at the same time, the external clock input is ETRF.</i>
Bits 13:12	ETPS[1:0]: External trigger prescaler External trigger signal ETRP frequency must be at most 1/4 of TIMxCLK frequency. A prescaler can be enabled to reduce ETRP frequency. It is useful when inputting fast external clocks. 00: Prescaler OFF 01: ETRP frequency divided by 2 10: ETRP frequency divided by 4 11: ETRP frequency divided by 8
Bits 11:8	ETF[3:0]: External trigger filter This bit-field then defines the frequency used to sample ETRP signal and the length of the digital filter applied to ETRP. The digital filter is made of an event counter in which N consecutive events are needed to validate a transition on the output: 0000: No filter, sampling is done at f_{DTS} 0001: $f_{\text{SAMPLING}} = f_{\text{CK_INT}}$, N=2 0010: $f_{\text{SAMPLING}} = f_{\text{CK_INT}}$, N=4 0011: $f_{\text{SAMPLING}} = f_{\text{CK_INT}}$, N=8 0100: $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$, N=6 0101: $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$, N=8 0110: $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$, N=6 0111: $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$, N=8 1000: $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$, N=6 1001: $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$, N=8 1010: $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$, N=5 1011: $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$, N=6 1100: $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$, N=8 1101: $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$, N=5 1110: $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$, N=6 1111: $f_{\text{SAMPLING}} = f_{\text{DTS}}/32$, N=8
Bit 7	MSM: Master/slave mode 0: No action 1: The effect of an event on the trigger input (TRGI) is delayed to allow a perfect synchronization between the current timer and its slaves (through TRGO). It is useful if we want to synchronize several timers on a single external event.
Bits 6:4	TS[2:0]: Trigger selection This bit-field selects the trigger input to be used to synchronize the counter. 000: Internal Trigger 0 (ITR0) 001: Internal Trigger 1 (ITR1) 010: Internal Trigger 2 (ITR2) 011: Internal Trigger 3 (ITR3) 100: TI1 Edge Detector (TI1F_ED) 101: Filtered Timer Input 1 (TI1FP1) 110: Filtered Timer Input 2 (TI2FP2) 111: External Trigger input (ETRF)

	See Table 53 for more details on ITRx meaning for each Timer. <i>Note: These bits must be changed only when they are not used (e.g. when SMS=000) to avoid wrong edge detections at the transition.</i>
Bit 3	Reserved, must be kept at reset value.
Bits 2:0	SMS: Slave mode selection When external signals are selected the active edge of the trigger signal (TRGI) is linked to the polarity selected on the external input (see Input Control register and Control Register description. 000: Slave mode disabled - if CEN = '1' then the prescaler is clocked directly by the internal clock. 001: Encoder mode 1 - Counter counts up/down on TI2FP1 edge depending on TI1FP2 level. 010: Encoder mode 2 - Counter counts up/down on TI1FP2 edge depending on TI2FP1 level. 011: Encoder mode 3 - Counter counts up/down on both TI1FP1 and TI2FP2 edges depending on the level of the other input. 100: Reset Mode - Rising edge of the selected trigger input (TRGI) reinitializes the counter and generates an update of the registers. 101: Gated Mode - The counter clock is enabled when the trigger input (TRGI) is high. The counter stops (but is not reset) as soon as the trigger becomes low. Both start and stop of the counter are controlled. 110: Trigger Mode - The counter starts at a rising edge of the trigger TRGI (but it is not reset). Only the start of the counter is controlled. 111: External Clock Mode 1 - Rising edges of the selected trigger (TRGI) clock the counter. <i>Note: The gated mode must not be used if TI1F_ED is selected as the trigger input (TS='100'). Indeed, TI1F_ED outputs 1 pulse for each transition on TI1F, whereas the gated mode checks the level of the trigger signal.</i> <i>The clock of the slave timer must be enabled prior to receiving events from the master timer, and must not be changed on-the-fly while triggers are received from the master timer.</i>

Table 53 TIM1 Internal trigger connection

Slave TIM	ITR1 (TS=001)	ITR2 (TS=010)	ITR3 (TS=011)
TIM1	TIM2	TIM3	TIM4

11.4.4 TIM1 DMA/interrupt enable register (TIM1_DIER)

Address offset: 0x0C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	TDE	COM DE	CC4 DE	CC3 DE	CC2 DE	CC1 DE	UDE	BIE	TIE	COM IE	CC4 IE	CC3 IE	CC2 IE	CC1 IE	UIE
	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bit 15	Reserved, must be kept at reset value.														
Bit 14	TDE: Trigger DMA request enable 0: Trigger DMA request disabled 1: Trigger DMA request enabled														
Bit 13	COMDE: COM DMA request enable 0: COM DMA request disabled 1: COM DMA request enabled														
Bit 12	CC4DE: Capture/Compare 4 DMA request enable 0: CC4 DMA request disabled 1: CC4 DMA request enabled														
Bit 11	CC3DE: Capture/Compare 3 DMA request enable 0: CC3 DMA request disabled 1: CC3 DMA request enabled														
Bit 10	CC2DE: Capture/Compare 2 DMA request enable 0: CC2 DMA request disabled 1: CC2 DMA request enabled														
Bit 9	CC1DE: Capture/Compare 1 DMA request enable 0: CC1 DMA request disabled 1: CC1 DMA request enabled														
Bit 8	UDE: Update DMA request enable 0: Update DMA request disabled 1: Update DMA request enabled														

Bit 7	BIE : Break interrupt enable 0: Break interrupt disabled 1: Break interrupt enabled
Bit 6	TIE : Trigger interrupt enable 0: Trigger interrupt disabled 1: Trigger interrupt enabled
Bit 5	COMIE : COM interrupt enable 0: COM interrupt disabled 1: COM interrupt enabled
Bit 4	CC4IE : Capture/Compare 4 interrupt enable 0: CC4 interrupt disabled 1: CC4 interrupt enabled
Bit 3	CC3IE : Capture/Compare 3 interrupt enable 0: CC3 interrupt disabled 1: CC3 interrupt enabled
Bit 2	CC2IE : Capture/Compare 2 interrupt enable 0: CC2 interrupt disabled 1: CC2 interrupt enabled
Bit 1	CC1IE : Capture/Compare 1 interrupt enable 0: CC1 interrupt disabled 1: CC1 interrupt enabled
Bit 0	UIE : Update interrupt enable 0: Update interrupt disabled 1: Update interrupt enabled

11.4.5 TIM1 status register (TIM1_SR)

Address offset: 0x10

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		CC4 0F	CC3 0F	CC2 0F	CC1 0F	Res	BIF	TIF	COM IF	CC4 IF	CC2 IF	CC2 IF	CC1 IF	UIF	
		rc w0	rc w0	rc w0	rc w0		rc w0	rc w0	rc w0	rc w0	rc w0	rc w0	rc w0	rc w0	rc w0

Bits15: 13	Reserved, must be kept at reset value.
Bit 12	CC4OF : Capture/Compare 4 overcapture flag. Refer to CC1OF description
Bit 11	CC3OF : Capture/Compare 3 overcapture flag. Refer to CC1OF description
Bit 10	CC2OF : Capture/Compare 2 overcapture flag. Refer to CC1OF description
Bit 9	CC1OF : Capture/Compare 1 overcapture flag This flag is set by hardware only when the corresponding channel is configured in input capture mode. It is cleared by software by writing it to '0'. 0: No overcapture has been detected. 1: The counter value has been captured in TIM1_CCR1 register while CC1IF flag was already set
Bit 8	Reserved, must be kept at reset value.
Bit 7	BIF : Break interrupt flag This flag is set by hardware as soon as the break input goes active. It can be cleared by software if the break input is not active. 0: No break event occurred. 1: An active level has been detected on the break input.
Bit 6	TIF : Trigger interrupt flag This flag is set by hardware on trigger event (active edge detected on TRGI input when the slave mode controller is enabled in all modes but gated mode, both edges in case gated mode is selected). It is cleared by software. 0: No trigger event occurred. 1: Trigger interrupt pending.
Bit 5	COMIF : COM interrupt flag This flag is set by hardware on COM event (when Capture/compare Control bits - CCxE, CCxNE, OCxM - have been updated). It is cleared by software. 0: No COM event occurred. 1: COM interrupt pending.

Bit 4	CC4IF : Capture/Compare 4 interrupt flag refer to CC1IF description
Bit 3	CC3IF : Capture/Compare 3 interrupt flag refer to CC1IF description
Bit 2	CC2IF : Capture/Compare 2 interrupt flag refer to CC1IF description
Bit 1	CC1IF: Capture/Compare 1 interrupt flag If channel CC1 is configured as output: This flag is set by hardware when the counter matches the compare value, with some exception in center-aligned mode (refer to the CMS bits in the TIM1_CR1 register description). It is cleared by software. 0: No match. 1: The content of the counter TIM1_CNT matches the content of the TIM1_CCR1 register. When the contents of TIM1_CCR1 are greater than the contents of TIM1_ARR, the CC1IF bit goes high on the counter overflow (in upcounting and up/down-counting modes) or underflow (in downcounting mode) If channel CC1 is configured as input: This bit is set by hardware on a capture. It is cleared by software or by reading the TIM1_CCR1 register. 0: No input capture occurred 1: The counter value has been captured in TIM1_CCR1 register (An edge has been detected on IC1 which matches the selected polarity)
Bit 0	UIF: Update interrupt flag This bit is set by hardware on an update event. It is cleared by software. 0: No update occurred. 1: Update interrupt pending. This bit is set by hardware when the registers are updated: – At overflow or underflow regarding the repetition counter value (update if repetition counter = 0) and if the UDIS=0 in the TIM1_CR1 register. – When CNT is reinitialized by software using the UG bit in TIM1_EGR register, if URS=0 and UDIS=0 in the TIM1_CR1 register. When CNT is reinitialized by a trigger event (refer to Section 11.4.3, if URS=0 and UDIS=0 in the TIM1_CR1 register.

11.4.6 TIM1 event generation register (TIM1_EGR)

Address offset: 0x14

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								BG	TG	COMG	CC4G	CC3G	CC2G	CC1G	UG
								w	w	w	w	w	w	w	w

Bits 15: 8	Reserved, must be kept at reset value.
Bit 7	BG: Break generation This bit is set by software in order to generate an event, it is automatically cleared by hardware. 0: No action 1: A break event is generated. MOE bit is cleared and BIF flag is set. Related interrupt or DMA transfer can occur if enabled.
Bit 6	TG: Trigger generation This bit is set by software in order to generate an event, it is automatically cleared by hardware. 0: No action 1: The TIF flag is set in TIM1_SR register. Related interrupt or DMA transfer can occur if enabled.
Bit 5	COMG: Capture/Compare control update generation This bit can be set by software, it is automatically cleared by hardware 0: No action 1: When CCPC bit is set, it allows to update CCxE, CCxNE and OCxM bits <i>Note: This bit acts only on channels having a complementary output.</i>
Bit 4	CC4G : Capture/Compare 4 generation refer to CC1G description
Bit 3	CC3G : Capture/Compare 3 generation refer to CC1G description
Bit 2	CC2G : Capture/Compare 2 generation refer to CC1G description
Bit 1	CC1G: Capture/Compare 1 generation This bit is set by software in order to generate an event, it is automatically cleared by hardware. 0: No action 1: A capture/compare event is generated on channel 1:

	If channel CC1 is configured as output: CC1IF flag is set, Corresponding interrupt or DMA request is sent if enabled. If channel CC1 is configured as input: The current value of the counter is captured in TIM1_CCR1 register. The CC1IF flag is set, the corresponding interrupt or DMA request is sent if enabled. The CC1OF flag is set if the CC1IF flag was already high.
Bit 0	UG: Update generation This bit can be set by software, it is automatically cleared by hardware. 0: No action 1: Reinitialize the counter and generates an update of the registers. Note that the prescaler counter is cleared too (anyway the prescaler ratio is not affected). The counter is cleared if the center-aligned mode is selected or if DIR=0 (upcounting), else it takes the auto-reload value (TIM1_ARR) if DIR=1 (downcounting).

11.4.7 TIM1 capture/compare mode register 1 (TIM1_CCMR1)

Address offset: 0x18

Reset value: 0x0000

The channels can be used in input (capture mode) or in output (compare mode). The direction of a channel is defined by configuring the corresponding CCxS bits. All the other bits of this register have a different function in input and in output mode. For a given bit, OCxx describes its function when the channel is configured in output, ICxx describes its function when the channel is configured in input. So the user must take care that the same bit can have a different meaning for the input stage and for the output stage.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2CE	OC2M[2: 0]		OC2PE	OC2FE	CC2S		OC1CE	OC1M[2: 0]		OC1PE	OC1FE	CC1S			
IC2F[3: 0]		IC2PSC[1: 0]		[1: 0]		IC1F[3: 0]		IC1PSC[1: 0]		[1: 0]					
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Output compare mode:

Bit 15	OC2CE: Output Compare 2 clear enable
Bits 14:12	OC2M[2:0]: Output Compare 2 mode
Bit 11	OC2PE: Output Compare 2 preload enable
Bit 10	OC2FE: Output Compare 2 fast enable
Bits 9:8	CC2S[1:0]: Capture/Compare 2 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC2 channel is configured as output 01: CC2 channel is configured as input, IC2 is mapped on TI2 10: CC2 channel is configured as input, IC2 is mapped on TI1 11: CC2 channel is configured as input, IC2 is mapped on TRC. This mode is working only if an internal trigger input is selected through the TS bit (TIM1_SMCR register) <i>Note: CC2S bits are writable only when the channel is OFF (CC2E = '0' in TIM1_CCER).</i>
Bit 7	OC1CE: Output Compare 1 clear enable OC1CE: Output Compare 1 Clear Enable 0: OC1Ref is not affected by the ETRF Input 1: OC1Ref is cleared as soon as a High level is detected on ETRF input
Bits 6:4	OC1M: Output Compare 1 mode These bits define the behavior of the output reference signal OC1REF from which OC1 and OC1N are derived. OC1REF is active high whereas OC1 and OC1N active level depends on CC1P and CC1NP bits. 000: Frozen - The comparison between the output compare register TIM1_CCR1 and the counter TIM1_CNT has no effect on the outputs.(this mode is used to generate a timing base). 001: Set channel 1 to active level on match. OC1REF signal is forced high when the counter TIM1_CNT matches the capture/compare register 1 (TIM1_CCR1). 010: Set channel 1 to inactive level on match. OC1REF signal is forced low when the counter TIM1_CNT matches the capture/compare register 1 (TIM1_CCR1). 011: Toggle - OC1REF toggles when TIM1_CNT=TIM1_CCR1. 100: Force inactive level - OC1REF is forced low. 101: Force active level - OC1REF is forced high.

	110: PWM mode 1 - In upcounting, channel 1 is active as long as TIM1_CNT<TIM1_CCR1 else inactive. In downcounting, channel 1 is inactive (OC1REF='0') as long as TIM1_CNT>TIM1_CCR1 else active (OC1REF='1'). 111: PWM mode 2 - In upcounting, channel 1 is inactive as long as TIM1_CNT<TIM1_CCR1 else active. In downcounting, channel 1 is active as long as TIM1_CNT>TIM1_CCR1 else inactive. <i>Note: 1: These bits can not be modified as long as LOCK level 3 has been programmed (LOCK bits in TIM1_BDTR register) and CCIS='00' (the channel is configured in output).</i> <i>2: In PWM mode 1 or 2, the OCREF level changes only when the result of the comparison changes or when the output compare mode switches from "frozen" mode to "PWM" mode.</i>
Bit 3	OC1PE: Output Compare 1 preload enable 0: Preload register on TIM1_CCR1 disabled. TIM1_CCR1 can be written at anytime, the new value is taken in account immediately. 1: Preload register on TIM1_CCR1 enabled. Read/Write operations access the preload register. TIM1_CCR1 preload value is loaded in the active register at each update event. <i>Note: 1: These bits can not be modified as long as LOCK level 3 has been programmed (LOCK bits in TIM1_BDTR register) and CCIS='00' (the channel is configured in output).</i> <i>2: The PWM mode can be used without validating the preload register only in one pulse mode (OPM bit set in TIM1_CR1 register). Else the behavior is not guaranteed.</i>
Bit 2	OC1FE: Output Compare 1 fast enable This bit is used to accelerate the effect of an event on the trigger in input on the CC output. 0: CC1 behaves normally depending on counter and CCR1 values even when the trigger is ON. The minimum delay to activate CC1 output when an edge occurs on the trigger input is 5 clock cycles. 1: An active edge on the trigger input acts like a compare match on CC1 output. Then, OC is set to the compare level independently from the result of the comparison. Delay to sample the trigger input and to activate CC1 output is reduced to 3 clock cycles. OC1FE acts only if the channel is configured in PWM1 or PWM2 mode.
Bits 1:0	CC1S: Capture/Compare 1 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC1 channel is configured as output 01: CC1 channel is configured as input, IC1 is mapped on TI1 10: CC1 channel is configured as input, IC1 is mapped on TI2 11: CC1 channel is configured as input, IC1 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIM1_SMCR register) <i>Note: CC1S bits are writable only when the channel is OFF (CC1E = '0' in TIM1_CCER)</i>

Input capture mode

Bits 15:12	IC2F: Input capture 2 filter
Bits 11:10	IC2PSC[1:0]: Input capture 2 prescaler
Bits 9:8	CC2S: Capture/Compare 2 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC2 channel is configured as output 01: CC2 channel is configured as input, IC2 is mapped on TI2 10: CC2 channel is configured as input, IC2 is mapped on TI1 11: CC2 channel is configured as input, IC2 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIM1_SMCR register) <i>Note: CC2S bits are writable only when the channel is OFF (CC2E = '0' in TIM1_CCER).</i>
Bits 7:4	IC1F[3:0]: Input capture 1 filter This bit-field defines the frequency used to sample TI1 input and the length of the digital filter applied to TI1. The digital filter is made of an event counter in which N consecutive events are needed to validate a transition on the output: 0000: No filter, sampling is done at f_{DTS} 0001: $f_{SAMPLING}=f_{CK_INT}$, N=2 0010: $f_{SAMPLING}=f_{CK_INT}$, N=4 0011: $f_{SAMPLING}=f_{CK_INT}$, N=8 0100: $f_{SAMPLING}=f_{DTS}/2$, N=6 0101: $f_{SAMPLING}=f_{DTS}/2$, N=8 0110: $f_{SAMPLING}=f_{DTS}/4$, N=6 0111: $f_{SAMPLING}=f_{DTS}/4$, N=8 1000: $f_{SAMPLING}=f_{DTS}/8$, N=6 1001: $f_{SAMPLING}=f_{DTS}/8$, N=8

	1010: $f_{\text{SAMPLING}} = f_{\text{DTS}}/16, N=5$ 1011: $f_{\text{SAMPLING}} = f_{\text{DTS}}/16, N=6$ 1100: $f_{\text{SAMPLING}} = f_{\text{DTS}}/16, N=8$ 1101: $f_{\text{SAMPLING}} = f_{\text{DTS}}/32, N=5$ 1110: $f_{\text{SAMPLING}} = f_{\text{DTS}}/32, N=6$ 1111: $f_{\text{SAMPLING}} = f_{\text{DTS}}/32, N=8$
Bits 3:2	IC1PSC : Input capture 1 prescaler This bit-field defines the ratio of the prescaler acting on CC1 input (IC1). The prescaler is reset as soon as CC1E='0' (TIM1_CCER register). 00: no prescaler, capture is done each time an edge is detected on the capture input 01: capture is done once every 2 events 10: capture is done once every 4 events 11: capture is done once every 8 events
Bits 1:0	CC1S : Capture/Compare 1 Selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC1 channel is configured as output 01: CC1 channel is configured as input, IC1 is mapped on TI1 10: CC1 channel is configured as input, IC1 is mapped on TI2 11: CC1 channel is configured as input, IC1 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIM1_SMCR register) <i>Note: CC1S bits are writable only when the channel is OFF (CC1E = '0' in TIM1_CCER).</i>

11.4.8 TIM1 capture/compare mode register 2 (TIM1_CCMR2)

Address offset: 0x1C

Reset value: 0x0000

Refer to the above CCMR1 register description.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC4CE	OC4M[2: 0]			OC4PE	OC4FE	CC4S		OC3CE	OC3M[2: 0]			OC3PE	OC3FE	CC3S	
IC4F[3: 0]				IC4PSC[1: 0]		[1: 0]		IC3F[3: 0]				IC3PSC[1: 0]		[1: 0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Output compare mode

Bit 15	OC4CE : Output compare 4 clear enable
Bits 14:12	OC4M : Output compare 4 mode
Bit 11	OC4PE : Output compare 4 preload enable
Bit 10	OC4FE : Output compare 4 fast enable
Bits 9:8	CC4S : Capture/Compare 4 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC4 channel is configured as output 01: CC4 channel is configured as input, IC4 is mapped on TI4 10: CC4 channel is configured as input, IC4 is mapped on TI3 11: CC4 channel is configured as input, IC4 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIM1_SMCR register) <i>Note: CC4S bits are writable only when the channel is OFF (CC4E = '0' in TIM1_CCER).</i>
Bit 7	OC3CE : Output compare 3 clear enable
Bits 6:4	OC3M : Output compare 3 mode
Bit 3	OC3PE : Output compare 3 preload enable
Bit 2	OC3FE : Output compare 3 fast enable
Bits 1:0	CC3S : Capture/Compare 3 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC3 channel is configured as output 01: CC3 channel is configured as input, IC3 is mapped on TI3 10: CC3 channel is configured as input, IC3 is mapped on TI4 11: CC3 channel is configured as input, IC3 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIM1_SMCR register) <i>Note: CC3S bits are writable only when the channel is OFF (CC3E = '0' in TIM1_CCER).</i>

Input capture mode:

Bits 15:12	IC4F : Input capture 4 filter
Bits 11:10	IC4PSC : Input capture 4 prescaler
Bits 9:8	CC4S : Capture/Compare 4 selection This bit-field defines the direction of the channel (input/output) as well as the used input.

	00: CC4 channel is configured as output 01: CC4 channel is configured as input, IC4 is mapped on TI4 10: CC4 channel is configured as input, IC4 is mapped on TI3 11: CC4 channel is configured as input, IC4 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIM1_SMCR register) <i>Note: CC4S bits are writable only when the channel is OFF (CC4E = '0' in TIM1_CCER).</i>
Bits 7:4	IC3F: Input capture 3 filter
Bits 3:2	IC3PSC: Input capture 3 prescaler
Bits 1:0	CC3S: Capture/compare 3 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC3 channel is configured as output 01: CC3 channel is configured as input, IC3 is mapped on TI3 10: CC3 channel is configured as input, IC3 is mapped on TI4 11: CC3 channel is configured as input, IC3 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIM1_SMCR register) <i>Note: CC3S bits are writable only when the channel is OFF (CC3E = '0' in TIM1_CCER).</i>

11.4.9 TIM1 capture/compare enable register (TIM1_CCER)

Address offset: 0x20

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	CC 4P	CC 4E	CC 3NP	CC 3NE	CC 3P	CC 3E	CC 2NP	CC 2NE	CC 2P	CC 2E	CC 1NP	CC 1N	CC 1P	CC 1E	CC 1E
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits15:14	Reserved, must be kept at reset value.														
Bit 13	CC4P: Capture/Compare 4 output polarity refer to CC1P description														
Bit 12	CC4E: Capture/Compare 4 output enable refer to CC1E description														
Bit 11	CC3NP: Capture/Compare 3 complementary output polarity refer to CC1NP description														
Bit 10	CC3NE: Capture/Compare 3 complementary output enable refer to CC1NE description														
Bit 9	CC3P: Capture/Compare 3 output polarity refer to CC1P description														
Bit 8	CC3E: Capture/Compare 3 output enable refer to CC1E description														
Bit 7	CC2NP: Capture/Compare 2 complementary output polarity refer to CC1NP description														
Bit 6	CC2NE: Capture/Compare 2 complementary output enable refer to CC1NE description														
Bit 5	CC2P: Capture/Compare 2 output polarity refer to CC1P description														
Bit 4	CC2E: Capture/Compare 2 output enable refer to CC1E description														
Bit 3	CC1NP: Capture/Compare 1 complementary output polarity 0: OC1N active high. 1: OC1N active low. <i>Note: This bit is not writable as soon as LOCK level 2 or 3 has been programmed (LOCK bits in TIM1_BDTR register) and CC1S="00" (the channel is configured in output).</i>														
Bit 2	CC1NE: Capture/Compare 1 complementary output enable 0: Off - OC1N is not active. OC1N level is then function of MOE, OSS1, OSSR, OIS1, OIS1N and CC1E bits. 1: On - OC1N signal is output on the corresponding output pin depending on MOE, OSS1, OSSR, OIS1, OIS1N and CC1E bits.														
Bit 1	CC1P: Capture/Compare 1 output polarity CC1 channel configured as output: 0: OC1 active high 1: OC1 active low CC1 channel configured as input: This bit selects whether IC1 or IC1 is used for trigger or capture operations. 0: non-inverted: capture is done on a rising edge of IC1. When used as external trigger, IC1 is non-inverted. 1: inverted: capture is done on a falling edge of IC1. When used as external trigger, IC1 is inverted. <i>Note: This bit is not writable as soon as LOCK level 2 or 3 has been programmed (LOCK bits in TIM1_BDTR register).</i>														
Bit 0	CC1E: Capture/Compare 1 output enable CC1 channel configured as output: 0: Off - OC1 is not active. OC1 level is then function of MOE, OSS1, OSSR, OIS1, OIS1N and CC1NE bits. 1: On - OC1 signal is output on the corresponding output pin depending on MOE, OSS1, OSSR, OIS1, OIS1N and CC1NE bits.														

	CC1 channel configured as input: This bit determines if a capture of the counter value can actually be done into the input capture/compare register 1 (TIM1_CCR1) or not. 0: Capture disabled. 1: Capture enabled.
--	--

Table 54 Output control bits for complementary OCx and OCxN channels with break feature

Control bits					Output states ⁽¹⁾	
MOE bit	OSSI bit	OSSR bit	CCxE bit	CCxNE bit	OCx output state	OCxN output state
1	X	0	0	0	Output Disabled (not driven by the timer), OCx=0, OCx_EN=0	Output Disabled (not driven by the timer), OCxN=0, OCxN_EN=0
		0	0	1	Output Disabled (not driven by the timer), OCx=0, OCx_EN=0	OCxREF + Polarity OCxN= OCxREF xor CCxNP, OCxN_EN=1
		0	1	0	OCxREF + Polarity OCx=OCxREF xor CCxP, OCx_EN=1	Output Disabled (not driven by the timer) OCxN=0, OCxN_EN=0
		0	1	1	OCREF + Polarity + dead-time OCx_EN=1	Complementary to OCREF (not OCREF) + Polarity + dead-time OCxN_EN=1
		1	0	0	Output Disabled (not driven by the timer) OCx=CCxP, OCx_EN=0	Output Disabled (not driven by the timer) OCxN=CCxNP, OCxN_EN=0
		1	0	1	Off-State (output enabled with inactive state) OCx=CCxP, OCx_EN=1	OCxREF + Polarity OCxN=OCxREF xor CCxNP, OCxN_EN=1
		1	1	0	OCxREF + Polarity OCx=OCxREF xor CCxP, OCx_EN=1	Off-State (output enabled with inactive state) OCxN=CCxNP, OCxN_EN=1
		1	1	1	OCREF + Polarity + dead-time OCx_EN=1	Complementary to OCREF (not OCREF) + Polarity + dead-time OCxN_EN=1
0	0	X	0	0	Output Disabled (not driven by the timer) Asynchronously: OCx=CCxP, OCx_EN=0, OCxN=CCxNP, OCxN_EN=0 Then if the clock is present: OCx=OISx and OCxN=OISxN after a dead-time, assuming that OISx and OISxN do not correspond to OCx and OCxN both in active state.	
	0		0	1		
	0		1	0		
	0		1	1		
	1		0	0	Off-State (output enabled with inactive state) Asynchronously: OCx=CCxP, OCx_EN=1, OCxN=CCxNP, OCxN_EN=1 Then if the clock is present: OCx=OISx and OCxN=OISxN after a dead-time, assuming that OISx and OISxN do not correspond to OCx and OCxN both in active state	
	1		0	1		
	1		1	0		
	1		1	1		

1. When both outputs of a channel are not used (CCxE = CCxNE = 0), the OISx, OISxN, CCxP and CCxNP bits must be kept cleared.

Note: The state of the external I/O pins connected to the complementary OCx and OCxN channels depends on the OCx and OCxN channel state and the GPIO and AFIO registers.

11.4.10 TIM1 counter (TIM1_CNT)

Address offset: 0x24

Reset value: 0x0000

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

CNT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 15:0				CNT[15:0]: Counter value											

11.4.11 TIM1 prescaler (TIM1_PSC)

Address offset: 0x28

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 15:0				PSC[15:0]: Prescaler value The counter clock frequency (CK_CNT) is equal to $f_{CK_PSC} / (PSC[15:0] + 1)$. PSC contains the value to be loaded in the active prescaler register at each update event (including when the counter is cleared through UG bit of TIMx_EGR register or through trigger controller when configured in “reset mode”).											

11.4.12 TIM1 auto-reload register (TIM1_ARR)

Address offset: 0x2C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 15:0				ARR[15:0]: Auto-reload value ARR is the value to be loaded in the actual auto-reload register. Refer to <i>Section 11.3.1: Time-base unit</i> for more details about ARR update and behavior. The counter is blocked while the auto-reload value is null.											

11.4.13 TIM1 repetition counter register (TIM1_RCR)

Address offset: 0x30

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								REP[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw
Bits 15:8				Reserved, must be kept at reset value.											
Bits 7:0				REP[7:0]: Repetition counter value These bits allow the user to set-up the update rate of the compare registers (i.e. periodic transfers from preload to active registers) when preload registers are enable, as well as the update interrupt generation rate, if this interrupt is enable. Each time the REP_CNT related downcounter reaches zero, an update event is generated and it restarts counting from REP value. As REP_CNT is reloaded with REP value only at the repetition update event U_RC, any write to the TIM1_RCR register is not taken in account until the next repetition update event. It means in PWM mode (REP+1) corresponds to: - the number of PWM periods in edge-aligned mode - the number of half PWM period in center-aligned mode.											

11.4.14 TIM1 capture/compare register 1 (TIM1_CCR1)

Address offset: 0x34

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR1[15:0]															

rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15:0			CCR1[15:0]: Capture/Compare 1 value If channel CC1 is configured as output: CCR1 is the value to be loaded in the actual capture/compare 1 register (preload value). It is loaded permanently if the preload feature is not selected in the TIM1_CCMR1 register (bit OC1PE). Else the preload value is copied in the active capture/compare 1 register when an update event occurs. The active capture/compare register contains the value to be compared to the counter TIM1_CNT and signaled on OC1 output. If channel CC1 is configured as input: CCR1 is the counter value transferred by the last input capture 1 event (IC1). The TIM1_CCR1 register is read-only and cannot be programmed.												

11.4.15 TIM1 capture/compare register 2 (TIM1_CCR2)

Address offset: 0x38

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR2[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15:0			CCR2[15:0]: Capture/Compare 2 value If channel CC2 is configured as output: CCR2 is the value to be loaded in the actual capture/compare 2 register (preload value). It is loaded permanently if the preload feature is not selected in the TIM1_CCMR2 register (bit OC2PE). Else the preload value is copied in the active capture/compare 2 register when an update event occurs. The active capture/compare register contains the value to be compared to the counter TIM1_CNT and signalled on OC2 output. If channel CC2 is configured as input: CCR2 is the counter value transferred by the last input capture 2 event (IC2). The TIM1_CCR2 register is read-only and cannot be programmed.												

11.4.16 TIM1 capture/compare register 3 (TIM1_CCR3)

Address offset: 0x3C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR3[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15:0			CCR3[15:0]: Capture/Compare value If channel CC3 is configured as output: CCR3 is the value to be loaded in the actual capture/compare 3 register (preload value). It is loaded permanently if the preload feature is not selected in the TIM1_CCMR3 register (bit OC3PE). Else the preload value is copied in the active capture/compare 3 register when an update event occurs. The active capture/compare register contains the value to be compared to the counter TIM1_CNT and signalled on OC3 output. If channel CC3 is configured as input: CCR3 is the counter value transferred by the last input capture 3 event (IC3). The TIM1_CCR3 register is read-only and cannot be programmed.												

11.4.17 TIM1 capture/compare register 4 (TIM1_CCR4)

Address offset: 0x40

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR4[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15:0			CCR4[15:0]: Capture/Compare value If channel CC4 is configured as output:												

	CCR4 is the value to be loaded in the actual capture/compare 4 register (preload value). It is loaded permanently if the preload feature is not selected in the TIM1_CCMR4 register (bit OC4PE). Else the preload value is copied in the active capture/compare 4 register when an update event occurs. The active capture/compare register contains the value to be compared to the counter TIM1_CNT and signalled on OC4 output. If channel CC4 is configured as input: CCR4 is the counter value transferred by the last input capture 4 event (IC4). The TIM1_CCR3 register is read-only and cannot be programmed.
--	--

11.4.18 TIM1 break and dead-time register (TIM1_BDTR)

Address offset: 0x44

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MORE	AOE	BKP	BKE	OSSR	OSSI	LOCK[1: 0]		DTG[7: 0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Note: As the bits AOE, BKP, BKE, OSSI, OSSR and DTG[7:0] can be write-locked depending on the LOCK configuration, it can be necessary to configure all of them during the first write access to the TIM1_BDTR register.

Bit 15	MOE: Main output enable This bit is cleared asynchronously by hardware as soon as the break input is active. It is set by software or automatically depending on the AOE bit. It is acting only on the channels which are configured in output. 0: OC and OCN outputs are disabled or forced to idle state. 1: OC and OCN outputs are enabled if their respective enable bits are set (CCxE, CCxNE in TIM1_CCER register). See OC/OCN enable description for more details (Section 11.4.9).
Bit 14	AOE: Automatic output enable 0: MOE can be set only by software 1: MOE can be set by software or automatically at the next update event (if the break input is not be active) <i>Note: This bit can not be modified as long as LOCK level 1 has been programmed (LOCK bits in TIM1_BDTR register).</i>
Bit 13	BKP: Break polarity 0: Break input BRK is active low 1: Break input BRK is active high <i>Note: This bit can not be modified as long as LOCK level 1 has been programmed (LOCK bits in TIM1_BDTR register).</i> <i>Note: Any write operation to this bit takes a delay of 1 APB clock cycle to become effective.</i>
Bit 12	BKE: Break enable 0: Break inputs (BRK and CSS clock failure event) disabled 1: Break inputs (BRK and CSS clock failure event) enabled <i>Note: This bit cannot be modified when LOCK level 1 has been programmed (LOCK bits in TIM1_BDTR register).</i> <i>Note: Any write operation to this bit takes a delay of 1 APB clock cycle to become effective.</i>
Bit 11	OSSR: Off-state selection for Run mode This bit is used when MOE=1 on channels having a complementary output which are configured as outputs. OSSR is not implemented if no complementary output is implemented in the timer. See OC/OCN enable description for more details (Section 11.4.9). 0: When inactive, OC/OCN outputs are disabled (OC/OCN enable output signal=0). 1: When inactive, OC/OCN outputs are enabled with their inactive level as soon as CCxE=1 or CCxNE=1. Then, OC/OCN enable output signal=1 <i>Note: This bit can not be modified as soon as the LOCK level 2 has been programmed (LOCK bits in TIM1_BDTR register).</i>
Bit 10	OSSI: Off-state selection for Idle mode This bit is used when MOE=0 on channels configured as outputs. See OC/OCN enable description for more details (Section 11.4.9). 0: When inactive, OC/OCN outputs are disabled (OC/OCN enable output signal=0). 1: When inactive, OC/OCN outputs are forced first with their idle level as soon as CCxE=1 or CCxNE=1. OC/OCN enable output signal=1) <i>Note: This bit can not be modified as soon as the LOCK level 2 has been programmed (LOCK bits in TIM1_BDTR register).</i>
Bits 9:8	LOCK[1:0]: Lock configuration These bits offer a write protection against software errors.

	00: LOCK OFF - No bit is write protected. 01: LOCK Level 1 = DTG bits in TIM1_BDTR register, OISx and OISxN bits in TIM1_CR2 register and BKE/BKP/AOE bits in TIMx_BDTR register can no longer be written. 10: LOCK Level 2 = LOCK Level 1 + CC Polarity bits (CCxP/CCxNP bits in TIM1_CCER register, as long as the related channel is configured in output through the CCxS bits) as well as OSSR and OSSI bits can no longer be written. 11: LOCK Level 3 = LOCK Level 2 + CC Control bits (OCxM and OCxPE bits in TIM1_CCMRx registers, as long as the related channel is configured in output through the CCxS bits) can no longer be written. <i>Note: The LOCK bits can be written only once after the reset. Once the TIM1_BDTR register has been written, their content is frozen until the next reset.</i>
Bits 7:0	DTG[7:0]: Dead-time generator setup This bit-field defines the duration of the dead-time inserted between the complementary outputs. DT correspond to this duration. DTG[7:5]=0xx => DT=DTG[7:0]x t_{dtg} with $t_{dtg}=t_{DTS}$. DTG[7:5]=10x => DT=(64+DTG[5:0])xt_{dtg} with $T_{dtg}=2xt_{DTS}$. DTG[7:5]=110 => DT=(32+DTG[4:0])xt_{dtg} with $T_{dtg}=8xt_{DTS}$. DTG[7:5]=111 => DT=(32+DTG[4:0])xt_{dtg} with $T_{dtg}=16xt_{DTS}$. Example if $T_{DTS}=125ns$ (8MHz), dead-time possible values are: 0 to 15875 ns by 125 ns steps, 16 us to 31750 ns by 250 ns steps, 32 us to 63us by 1 us steps, 64 us to 126 us by 2 us steps <i>Note: This bit-field can not be modified as long as LOCK level 1, 2 or 3 has been programmed (LOCK bits in TIM1_BDTR register).</i>

11.4.19 TIM1 DMA control register (TIM1_DCR)

Address offset: 0x48

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			DBL[4:0]					Reserved			DBA[4:0]				
			rw	rw	rw	rw	rw				rw	rw	rw	rw	rw
Bits 15:13			Reserved, must be kept at reset value.												
Bits 12:8			DBL[4:0]: DMA burst length This 5-bit vector defines the number of DMA transfers (the timer detects a burst transfer when a read or a write access to the TIM1_DMAR register address is performed). 00000: 1 transfer 00001: 2 transfers 00010: 3 transfers ... 10001: 18 transfers												
Bits 7:5			Reserved, must be kept at reset value.												
Bits 4:0			DBA[4:0]: DMA base address This 5-bits vector defines the base-address for DMA transfers (when read/write access are done through the TIMx_DMAR address). DBA is defined as an offset starting from the address of the TIM1_CR1 register. Example: 00000: TIM1_CR1, 00001: TIM1_CR2, 00010: TIM1_SMCR, ... Example: Let us consider the following transfer: DBL = 7 transfers and DBA = TIMx_CR1. In this case the transfer is done to/from 7 registers starting from the TIM1_CR1 address.												

11.4.20 TIM1 DMA address for full transfer (TIM1_DMAR)

Address offset: 0x4C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
----	----	----	----	----	----	---	---	---	---	---	---	---	---	---	---

DMAB[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 15:0		DMAB[31:0]: DMA register for burst accesses A read or write operation to the DMAR register accesses the register located at the address (TIM1_CR1 address) + (DBA + DMA index) x 4 where TIM1_CR1 address is the address of the control register 1, DBA is the DMA base address configured in TIM1_DCR register, DMA index is automatically controlled by the DMA transfer, and ranges from 0 to DBL (DBL configured in TIM1_DCR).													

CKS copyright

11.4.21 TIM1 register map

TIM1 register is mapped as 16-bit addressable registers as described in the table below:

Table 55 TIM1 register map and reset values

Offset	Register		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
000h	TIM1_CR1	Reset value	Reserved																						CKD [1:0]		0	0	0	ARPE	0	CMS [1:0]	0	0	DIR	0	OPM	0	URS	UDIS	CFN	
004h	TIM1_CR2	Reset value	Reserved																						0	0	0	TI1S	0	MMS [2:0]	0	0	0	CCDS	0	Reserved						
008h	TIM1_SMCR	Reset value	Reserved														0	EPT	ECE	ETPS[1:0]	0	EFT [3:0]	0	MSM	0	TS [2:0]	0	0	0	Reserved	SMS [2:0]	0	0	0								
00Ch	TIM1_DIER	Reset value	Reserved														0	TDE	Reserved	0	CC4DE	0	CC3DE	0	CC2DE	0	CC1DE	0	UDE	Reserved	0	TIE	Reserved	0	CC4IE	0	CC3IE	0	CC2IE	0	CC1IE	UIF
010h	TIM1_SR	Reset value	Reserved														0	CC4OE	0	CC3OE	0	CC2OE	0	CC1OE	Reserved	0	TIF	Reserved	0	CC4IF	0	CC3IF	0	CC2IF	0	CC1IF	UIF					
014h	TIM1_EGR	Reserved																																								

	Reset value	0																0	0	0	0	0	0
018h	TIM1_CCMR1 Output Compare mode	Reserved	OC2CE	OC2M [2:0]				OC2PE	OC2FE	CC2S [1: 0]		OC1CE	OC1M [2:0]				OC1PE	OC1FE	CC1S [1:0]				
	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
	TIM1_CCMR1 Input Capture mode		IC2F [2:0]				IC2P SC [1: 0]	CC2S [1: 0]		IC1F [3:0]				IC1P SC [1: 0]		CC1S [1:0]							
	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
01Ch	TIM1_CCMR2 Output Compare mode	Reserved	OC4CE	OC4M [2:0]				OC4PE	OC4FE	CC4S [1: 0]		OC3CE	OC3M [2:0]				OC3PE	OC3FE	CC3S [1:0]				
	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
	TIM1_CCMR2 Input Capture mode		IC4F [2:0]				IC4P SC [1: 0]	CC4S [1: 0]		IC3F [3:0]				IC3P SC [1: 0]		CC3S [1:0]							
	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						
020h	TIM1_CCER	Reserved	CC4P	CC4E	CC3NP	CC3NE	CC3P	CC3E	CC2NP	CC2NE	CC2P	CC2E	CC1NP	CC1NE	CC1P	CC1E							
	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0							
024h	TIM1_CNT	Reserved	CNT[15: 0]																				

	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
028h	TIM1_PSC	Reserved	PSC[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
02Ch	TIM1_ARR	Reserved	ARR[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
030h	TIM1_RCR	Reserved												REP[7: 0]				
	Reset value													0	0	0	0	0
034h	TIM1_CCR1	Reserved	CCR1[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
038h	TIM1_CCR2	Reserved	CCR2[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
03Ch	TIM1_CCR3	Reserved	CCR3[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

040h	TIM1_CCR4	Reserved	CCR4[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
044h	TIM1_BDTR	Reserved	MOE	AOE	BKP	BKE	OSSR	OSSI	LOCK [1: 0]	DT[7: 0]								
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
048h	TIM1_DCR	Reserved	DBL[4: 0]				Reserved				DBA[4: 0]							
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
04Ch	TIM1_DMAR	Reserved	DMAB[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Refer to Table 1 for the register boundary addresses.

12 General-purpose timers (TIMx)

12.1 TIMx introduction

The general-purpose timers consist of a 16-bit auto-reload counter driven by a programmable prescaler.

They may be used for a variety of purposes, including measuring the pulse lengths of input signals (input capture) or generating output waveforms (output compare and PWM).

Pulse lengths and waveform periods can be modulated from a few microseconds to several milliseconds using the timer prescaler and the RCC clock controller prescalers.

The timers are completely independent, and do not share any resources. They can be synchronized together as described in *Section 12.3.15*.

12.2 TIMx main features

General-purpose TIMx timer features include:

- 16-bit up, down, up/down auto-reload counter.
- 16-bit programmable prescaler used to divide (also “on the fly”) the counter-clock frequency by any factor between 1 and 65536.
- Up to 4 independent channels for:
 - Input capture
 - Output compare
 - PWM generation (Edge- and Center-aligned modes)
 - One-pulse mode output
- Synchronization circuit to control the timer with external signals and to interconnect several timers.
- Interrupt/DMA generation on the following events:
 - Update: counter overflow/underflow, counter initialization (by software or internal/external trigger)
 - Trigger event (counter start, stop, initialization or count by internal/external trigger)
 - Input capture
 - Output compare
- Supports incremental (quadrature) encoder and hall-sensor circuitry for positioning purposes
- Trigger input for external clock or cycle-by-cycle current management

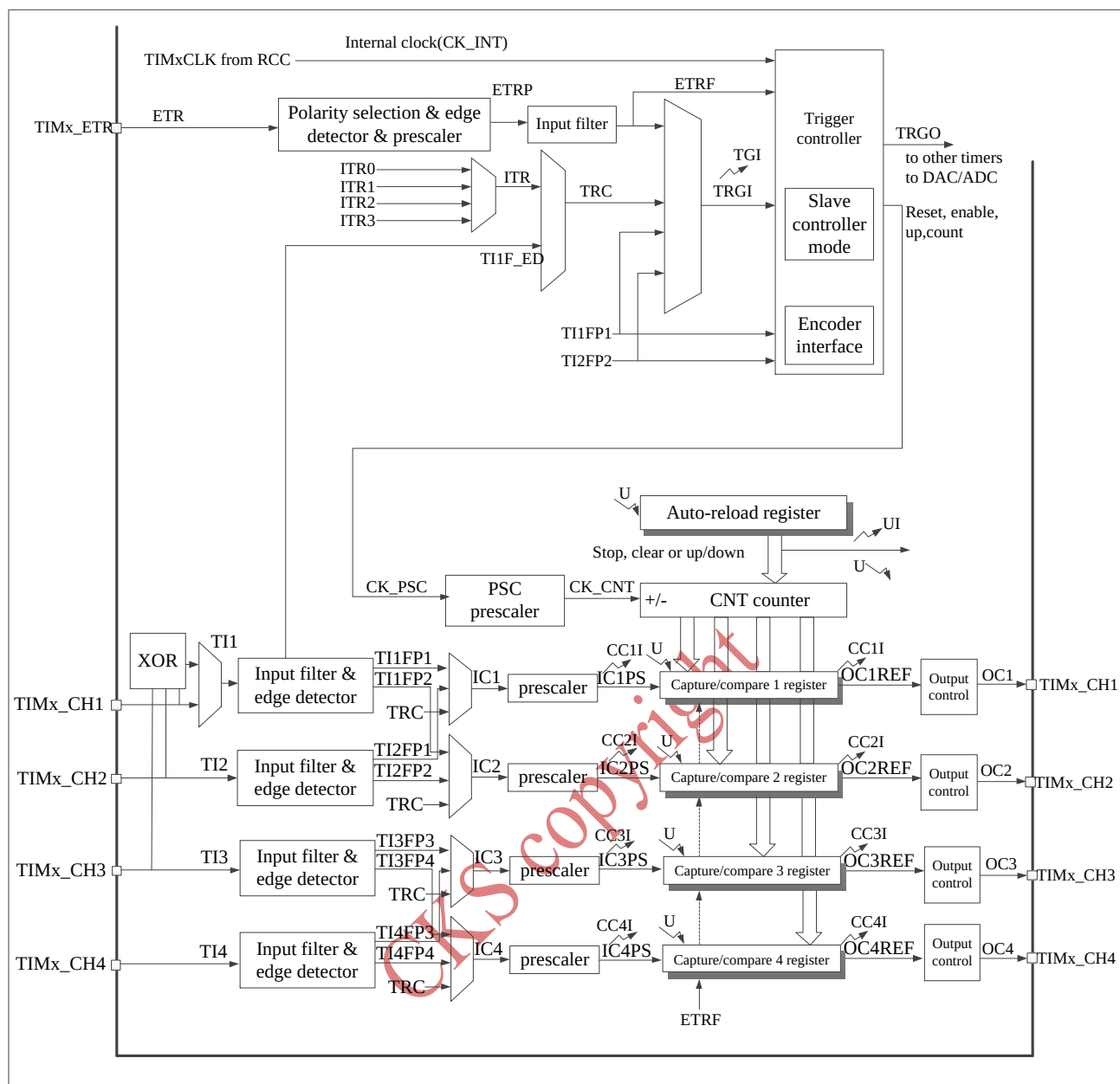


Figure 85 General-purpose timer block diagram

12.3 TIMx functional description

12.3.1 Time-base unit

The main block of the programmable timer is a 16-bit counter with its related auto-reload register. The counter can count up, down or both up and down. The counter clock can be divided by a prescaler.

The counter, the auto-reload register and the prescaler register can be written or read by software. This is true even when the counter is running.

The time-base unit includes:

- Counter Register (TIMx_CNT)
- Prescaler Register (TIMx_PSC):
- Auto-Reload Register (TIMx_ARR)

The auto-reload register is preloaded. Writing to or reading from the auto-reload register accesses the preload register. The content of the preload register are transferred into the shadow register permanently or at each update event (UEV), depending on the auto-reload preload enable bit (ARPE) in TIMx_CR1 register. The update event is sent when the counter reaches the overflow (or underflow when downcounting) and if the UDIS bit equals 0 in the TIMx_CR1 register. It can also be generated by software. The generation of the update event is described in detail for each configuration.

The counter is clocked by the prescaler output CK_CNT, which is enabled only when the counter enable bit (CEN) in TIMx_CR1 register is set (refer also to the slave mode controller description to get more details on counter enabling).

Note that the actual counter enable signal CNT_EN is set 1 clock cycle after CEN.

Prescaler description

The prescaler can divide the counter clock frequency by any factor between 1 and 65536. It is based on a 16-bit counter controlled through a 16-bit register (in the TIMx_PSC register). It can be changed on the fly as this control register is buffered. The new prescaler ratio is taken into account at the next update event.

Figure 86 and Figure 87 give some examples of the counter behavior when the prescaler ratio is changed on the fly:

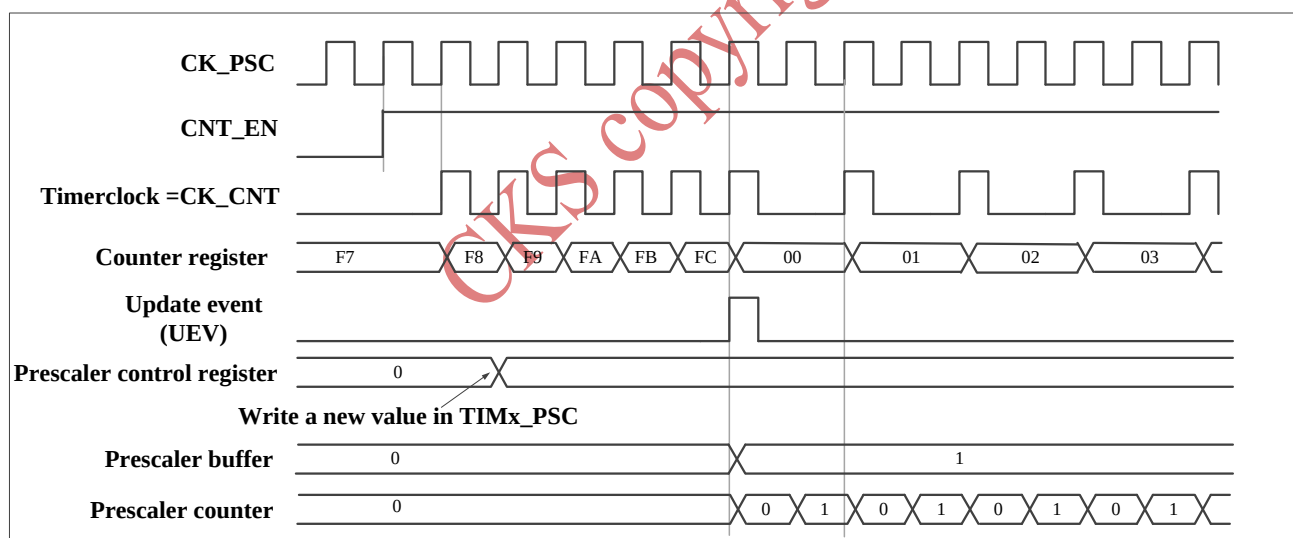


Figure 86 Counter timing diagram with prescaler division change from 1 to 2

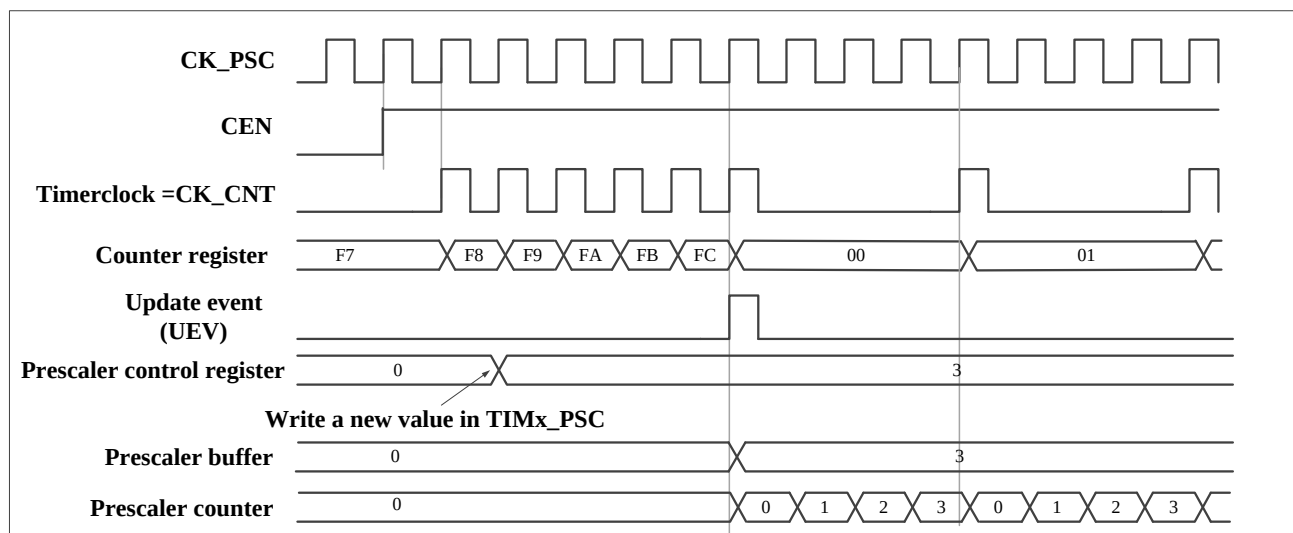


Figure 87 Counter timing diagram with prescaler division change from 1 to 4

12.3.2 Counter modes

Upcounting mode

In upcounting mode, the counter counts from 0 to the auto-reload value (content of the TIMx_ARR register), then restarts from 0 and generates a counter overflow event.

An Update event can be generated at each counter overflow or by setting the UG bit in the TIMx_EGR register (by software or by using the slave mode controller).

The UEV event can be disabled by software by setting the UDIS bit in TIMx_CR1 register. This is to avoid updating the shadow registers while writing new values in the preload registers. Then no update event occurs until the UDIS bit has been written to 0. However, the counter restarts from 0, as well as the counter of the prescaler (but the prescale rate does not change). In addition, if the URS bit (update request selection) in TIMx_CR1 register is set, setting the UG bit generates an update event UEV but without setting the UIF flag (thus no interrupt or DMA request is sent). This is to avoid generating both update and capture interrupts when clearing the counter on the capture event. When an update event occurs, all the registers are updated and the update flag (UIF bit in TIMx_SR register) is set (depending on the URS bit):

- The buffer of the prescaler is reloaded with the preload value (content of the TIMx_PSC register)
- The auto-reload shadow register is updated with the preload value (TIMx_ARR)

The following figures show some examples of the counter behavior for different clock frequencies when TIMx_ARR=0x36.

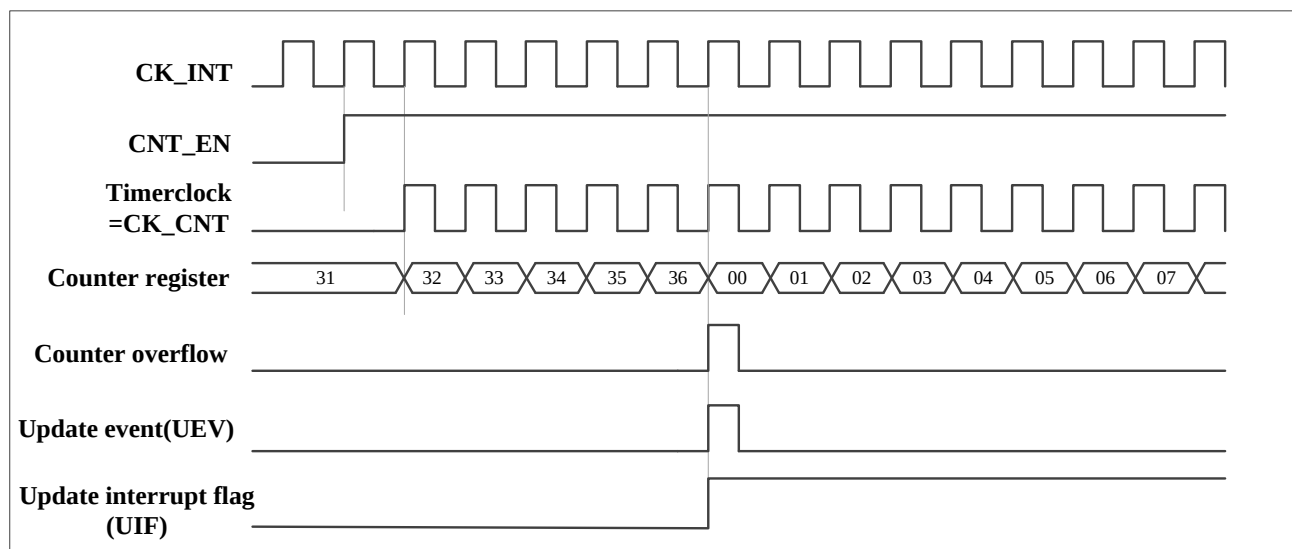


Figure 88 Counter timing diagram, internal clock divided by 1

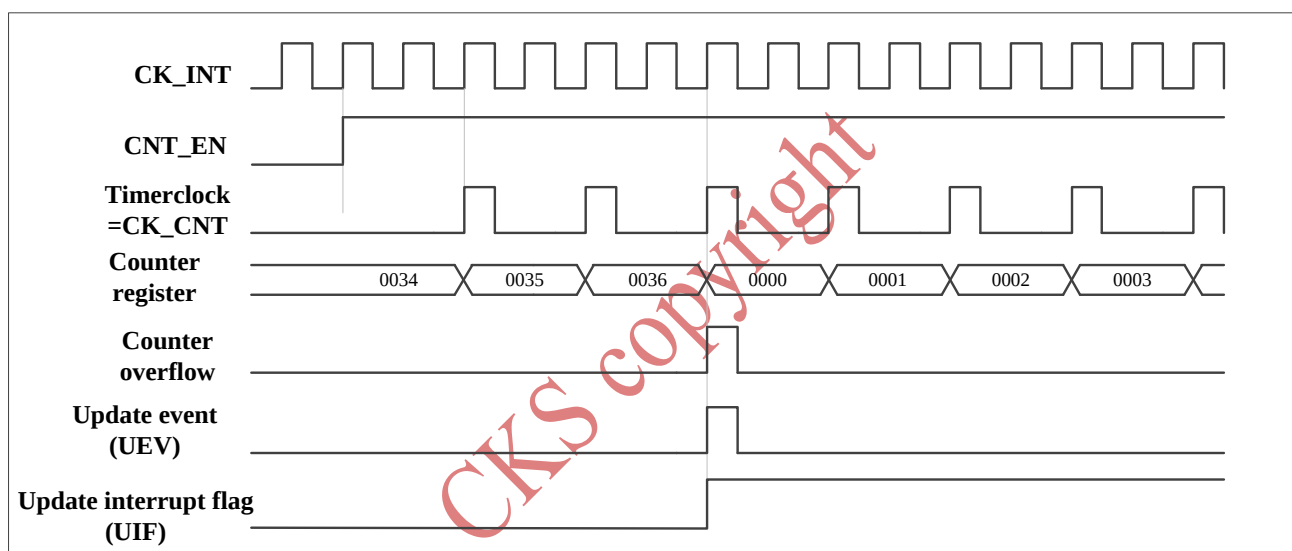


Figure 89 Counter timing diagram, internal clock divided by 2

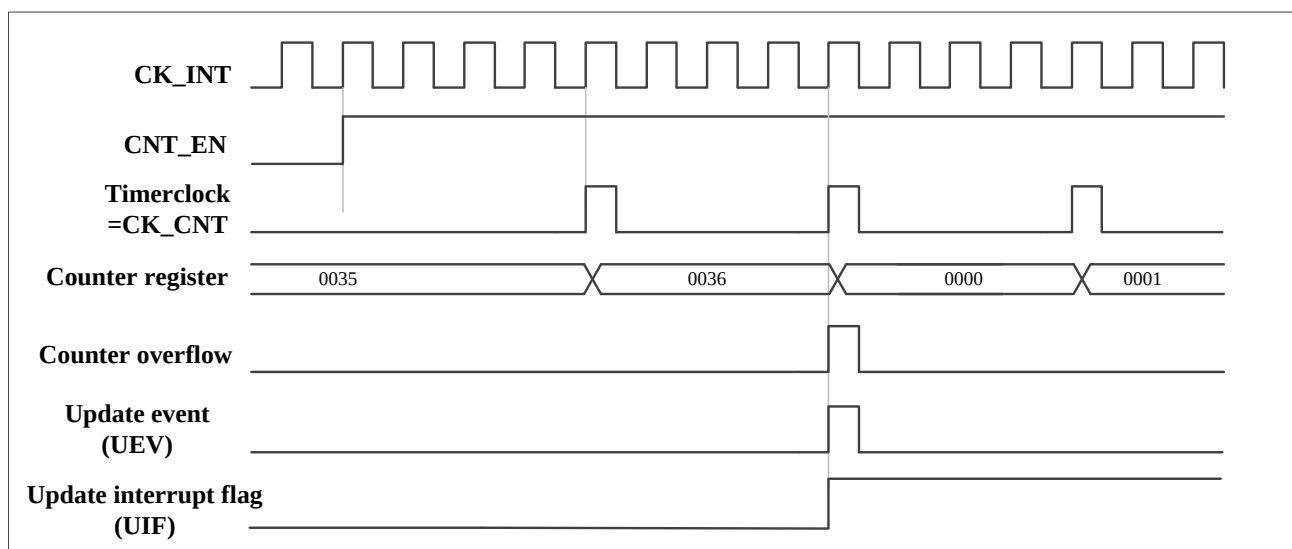


Figure 90 Counter timing diagram, internal clock divided by 4

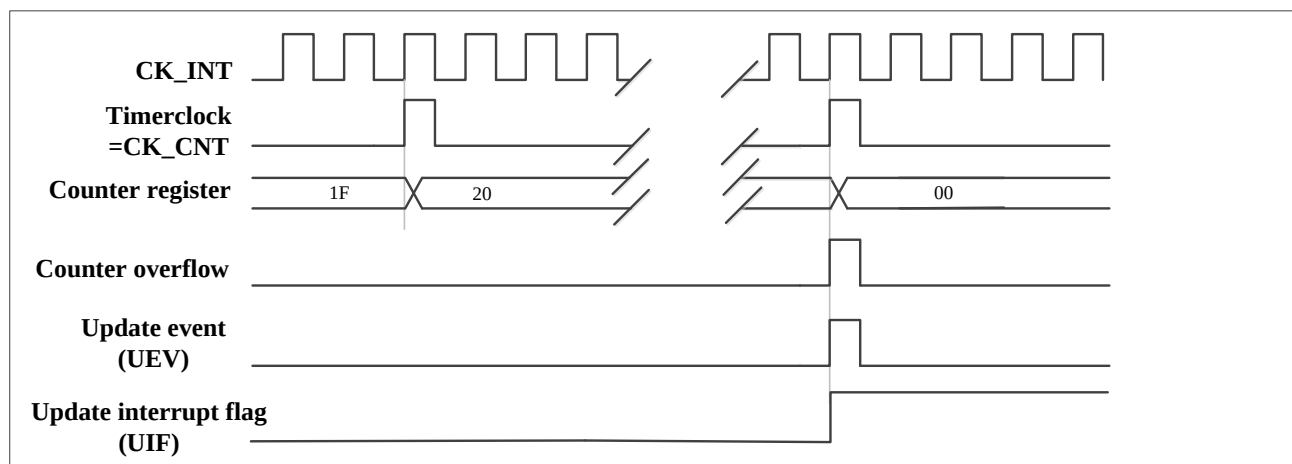


Figure 91 Counter timing diagram, internal clock divided by N

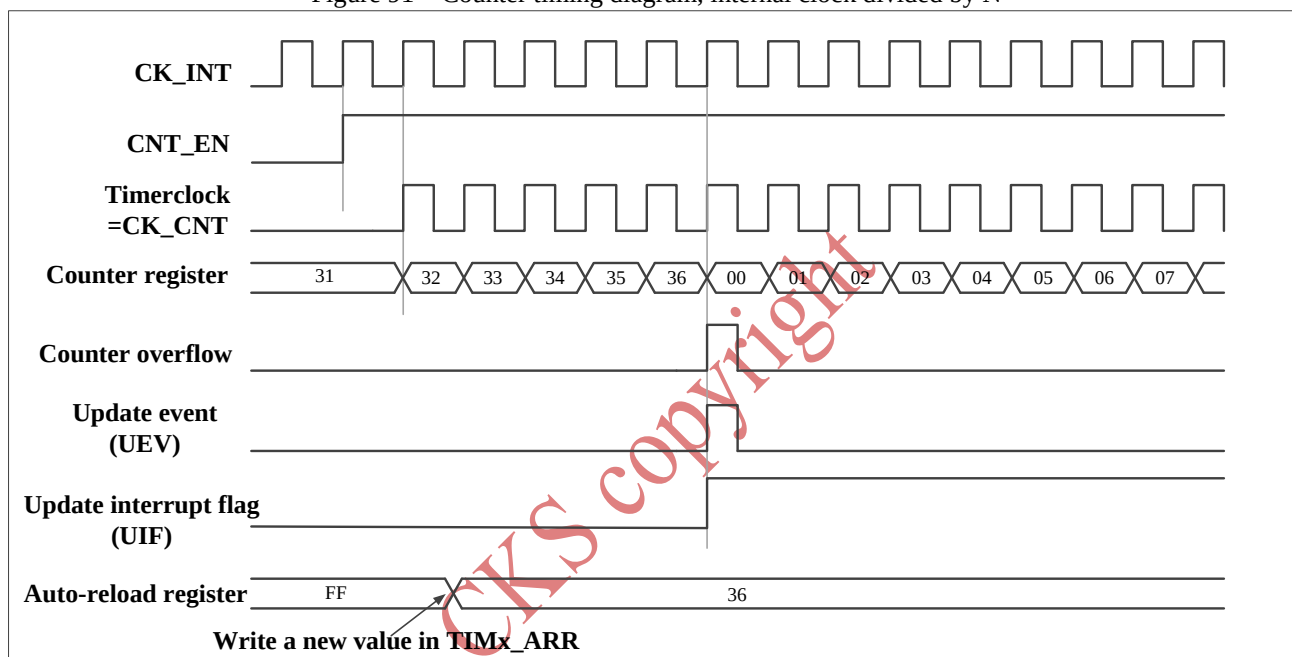


Figure 92 Counter timing diagram, Update event when ARPE=0 (TIMx_ARR not preloaded)

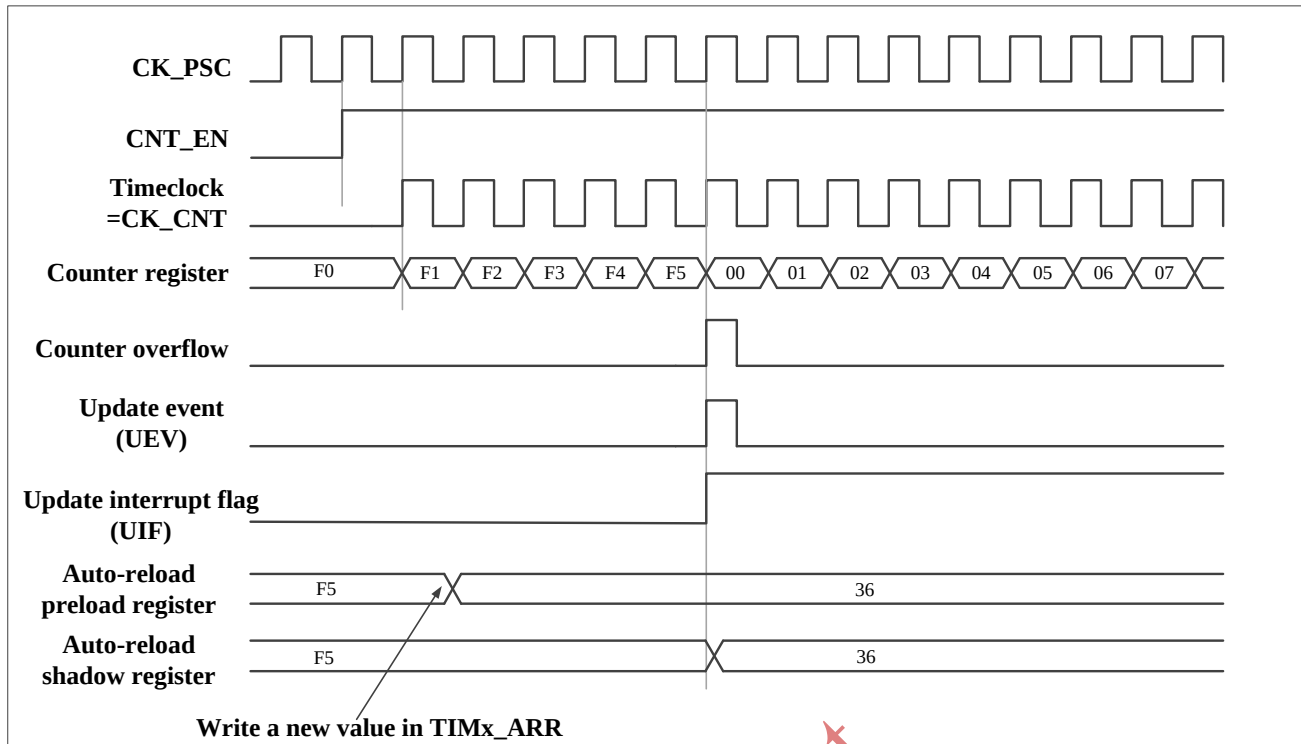


Figure 93 Counter timing diagram, Update event when ARPE=1 (TIMx_ARR preloaded)

Downcounting mode

In downcounting mode, the counter counts from the auto-reload value (content of the TIMx_ARR register) down to 0, then restarts from the auto-reload value and generates a counter underflow event. An Update event can be generated at each counter underflow or by setting the UG bit in the TIMx_EGR register (by software or by using the slave mode controller).

The UEV update event can be disabled by software by setting the UDIS bit in TIMx_CR1 register. This is to avoid updating the shadow registers while writing new values in the preload registers. Then no update event occurs until UDIS bit has been written to 0. However, the counter restarts from the current auto-reload value, whereas the counter of the prescaler restarts from 0 (but the prescale rate doesn't change).

In addition, if the URS bit (update request selection) in TIMx_CR1 register is set, setting the UG bit generates an update event UEV but without setting the UIF flag (thus no interrupt or DMA request is sent). This is to avoid generating both update and capture interrupts when clearing the counter on the capture event.

When an update event occurs, all the registers are updated and the update flag (UIF bit in TIMx_SR register) is set (depending on the URS bit):

- The buffer of the prescaler is reloaded with the preload value (content of the TIMx_PSC register).
- The auto-reload active register is updated with the preload value (content of the TIMx_ARR register).
Note that the auto-reload is updated before the counter is reloaded, so that the next period is the expected one.

The following figures show some examples of the counter behavior for different clock frequencies when TIMx_ARR=0x36.

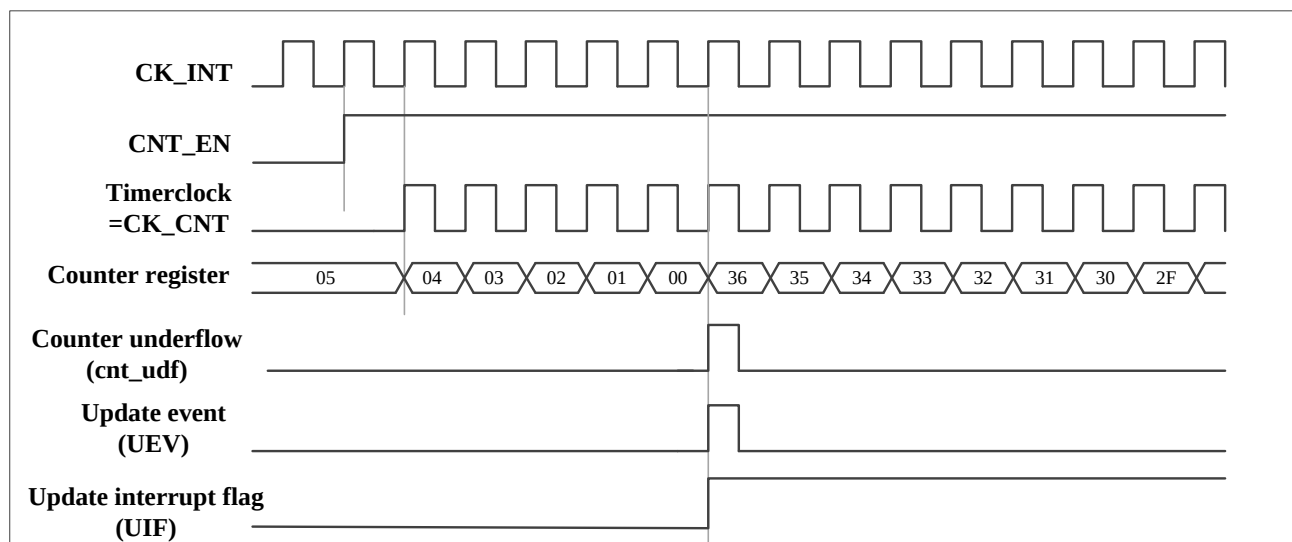


Figure 94 Counter timing diagram, internal clock divided by 1

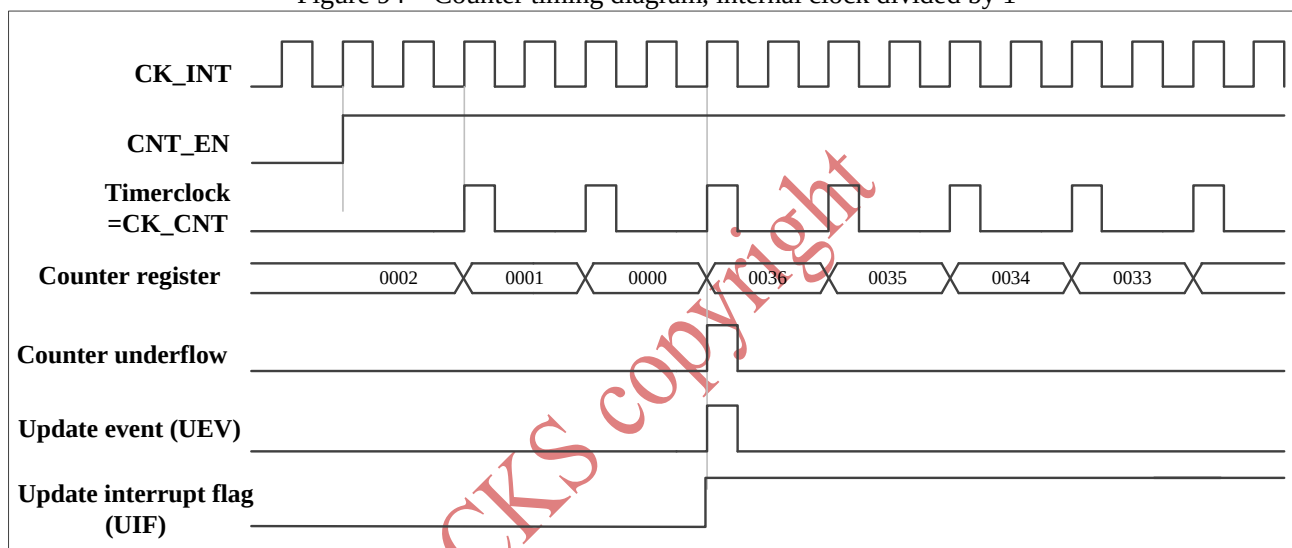


Figure 95 Counter timing diagram, internal clock divided by 2

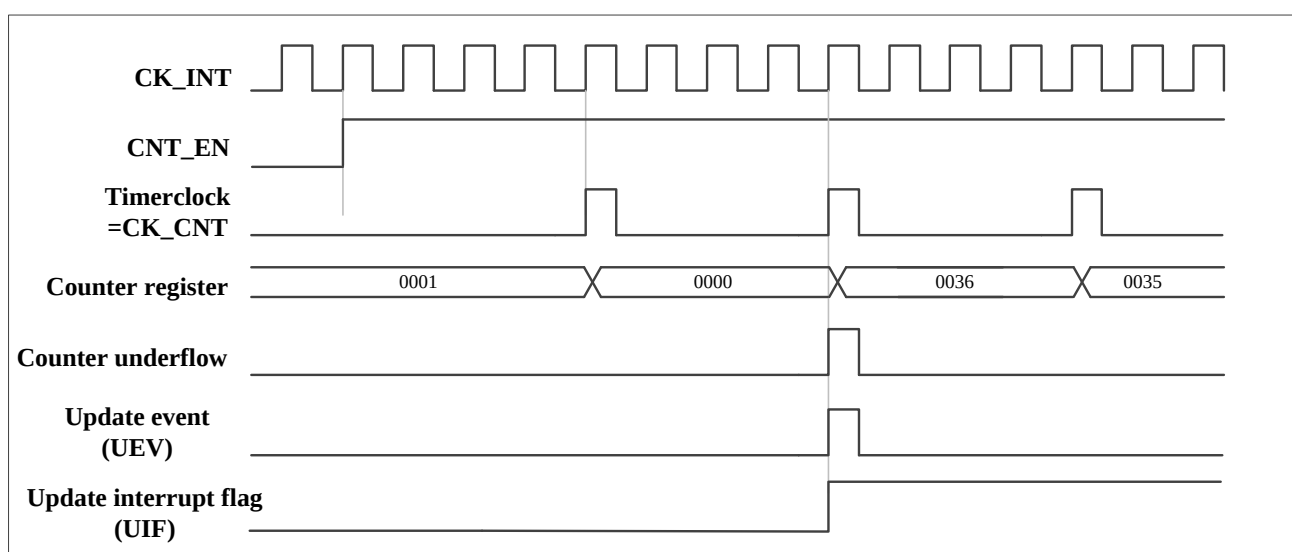


Figure 96 Counter timing diagram, internal clock divided by 4

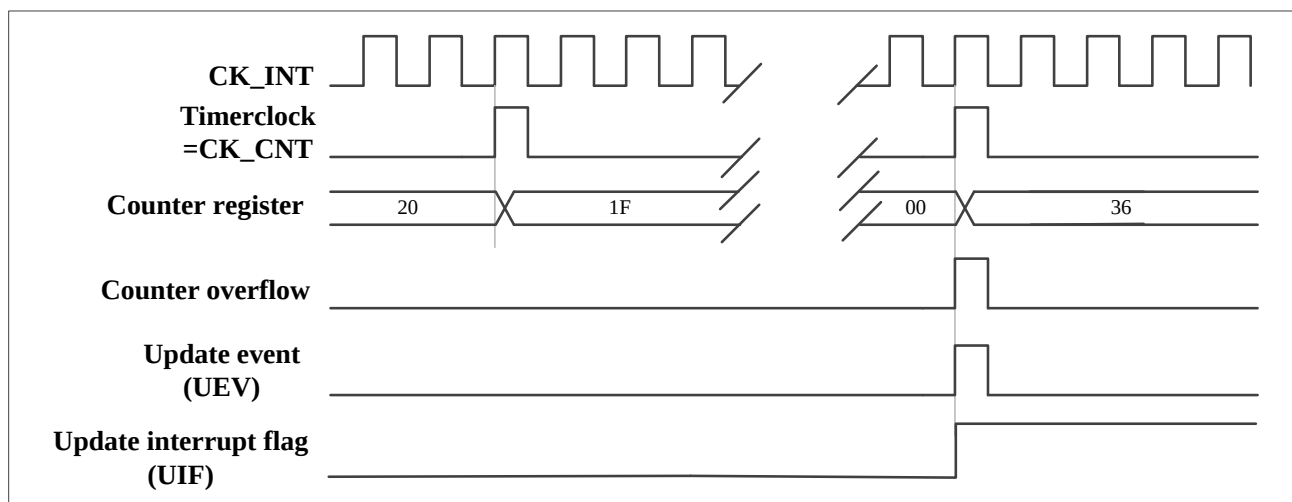


Figure 97 Counter timing diagram, internal clock divided by N

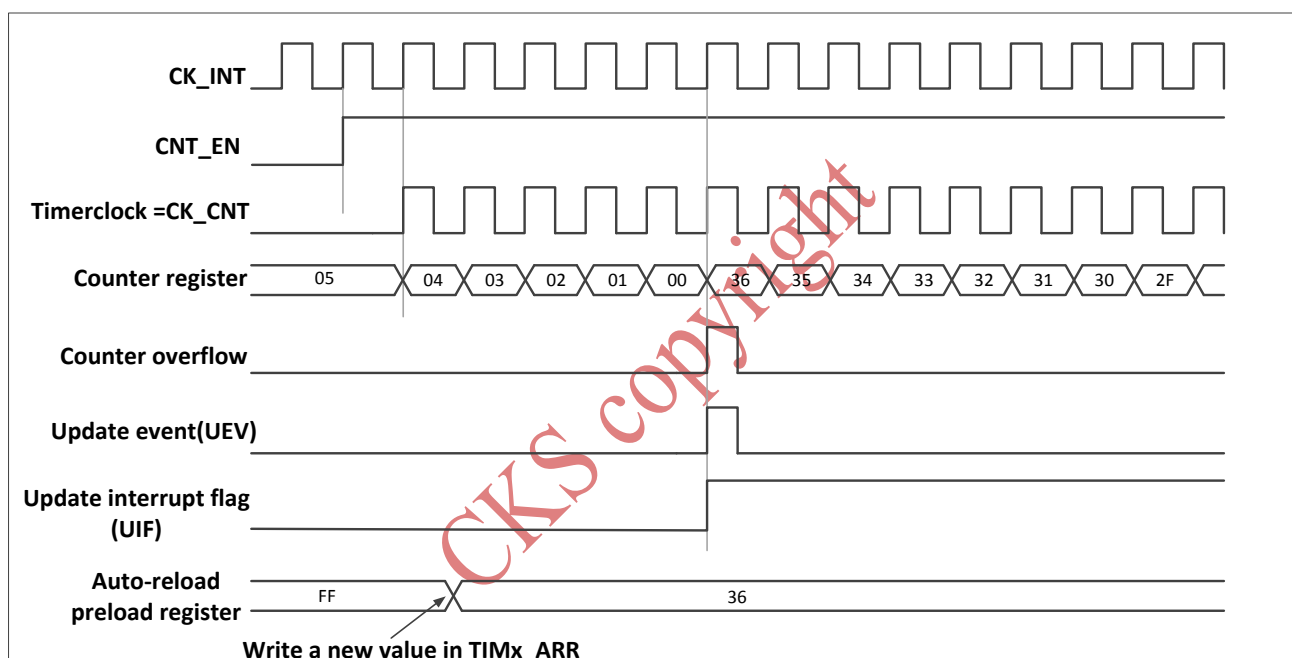


Figure 98 Counter timing diagram, Update event

Center-aligned mode (up/down counting)

In center-aligned mode, the counter counts from 0 to the auto-reload value (content of the TIMx_ARR register) – 1, generates a counter overflow event, then counts from the auto-reload value down to 1 and generates a counter underflow event. Then it restarts counting from 0.

Center-aligned mode is active when the CMS bits in TIMx_CR1 register are not equal to '00'. The Output compare interrupt flag of channels configured in output is set when: the counter counts down (Center aligned mode 1, CMS = "01"), the counter counts up (Center aligned mode 2, CMS = "10") the counter counts up and down (Center aligned mode 3, CMS = "11").

In this mode, the direction bit (DIR from TIMx_CR1 register) cannot be written. It is updated by hardware and gives the current direction of the counter.

The update event can be generated at each counter overflow and at each counter underflow or by setting the UG bit in the TIMx_EGR register (by software or by using the slave mode controller) also generates an update event. In this case, the counter restarts counting from 0, as well as the counter of the prescaler.

The UEV update event can be disabled by software by setting the UDIS bit in TIMx_CR1 register. This is to avoid updating the shadow registers while writing new values in the preload registers.

Then no update event occurs until the UDIS bit has been written to 0. However, the counter continues counting up and down, based on the current auto-reload value.

In addition, if the URS bit (update request selection) in TIMx_CR1 register is set, setting the UG bit generates an update event UEV but without setting the UIF flag (thus no interrupt or DMA request is sent). This is to avoid generating both update and capture interrupt when clearing the counter on the capture event.

When an update event occurs, all the registers are updated and the update flag (UIF bit in TIMx_SR register) is set (depending on the URS bit):

- The buffer of the prescaler is reloaded with the preload value (content of the TIMx_PSC register).
- The auto-reload active register is updated with the preload value (content of the TIMx_ARR register).
Note that if the update source is a counter overflow, the auto-reload is updated before the counter is reloaded, so that the next period is the expected one (the counter is loaded with the new value).

The following figures show some examples of the counter behavior for different clock frequencies.

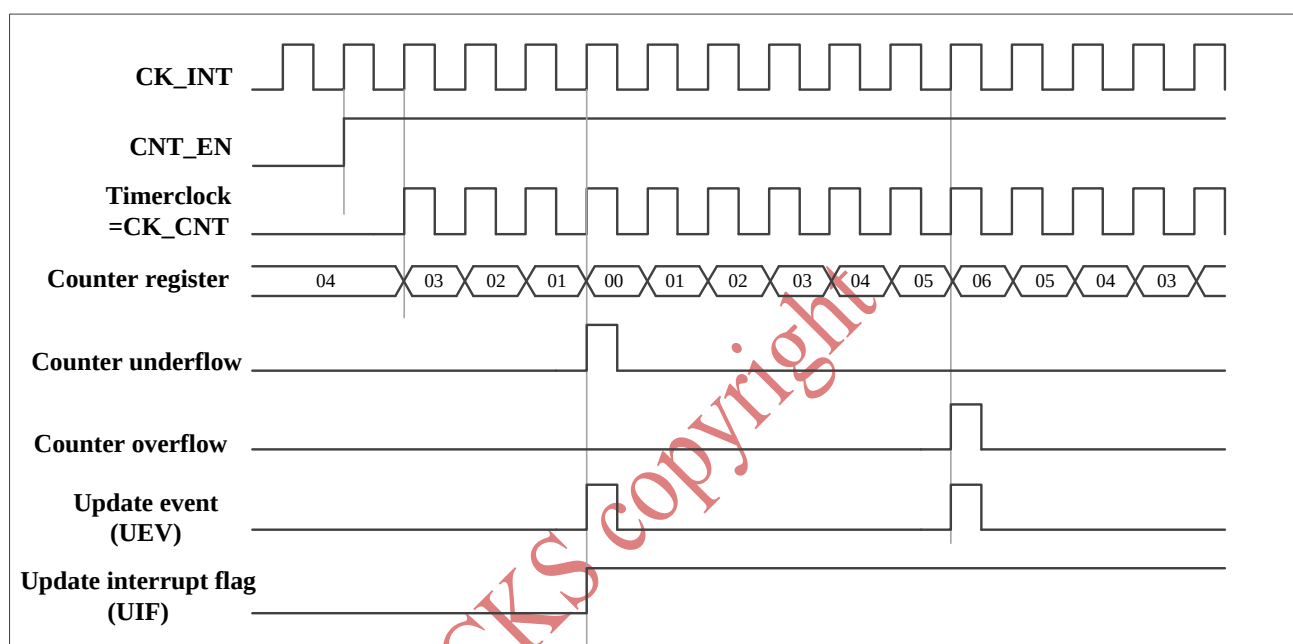


Figure 99 Counter timing diagram, internal clock divided by 1, TIMx_ARR=0x6

1. Here, center-aligned mode 1 is used, for more details refer to Section 12.4.1: TIMx control register 1 (TIMx_CR1).

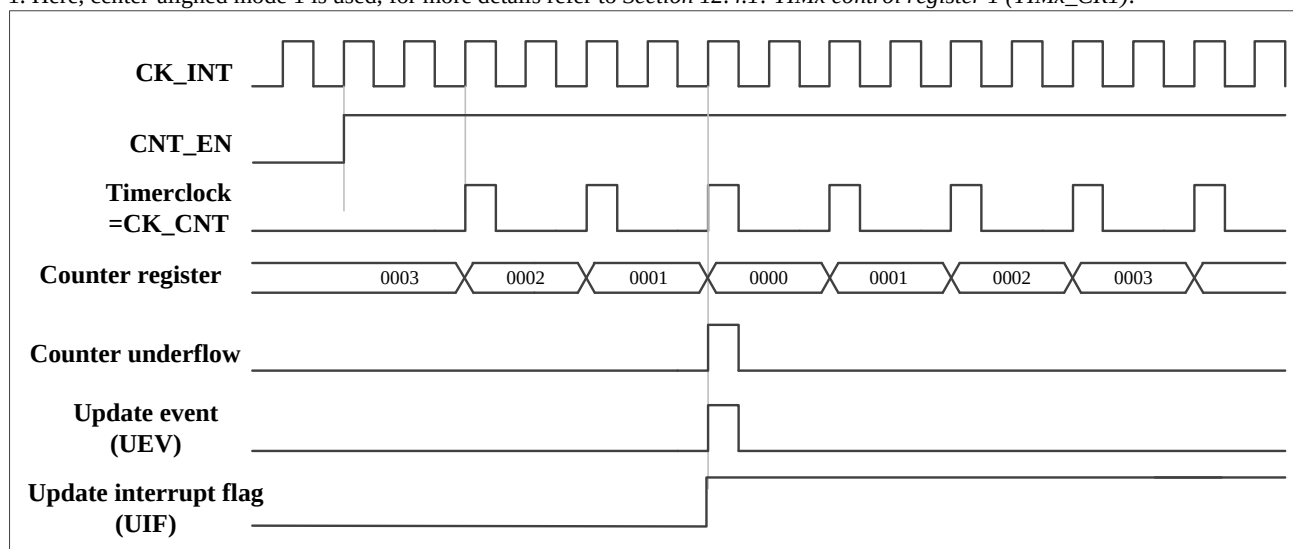


Figure 100 Counter timing diagram, internal clock divided by 2

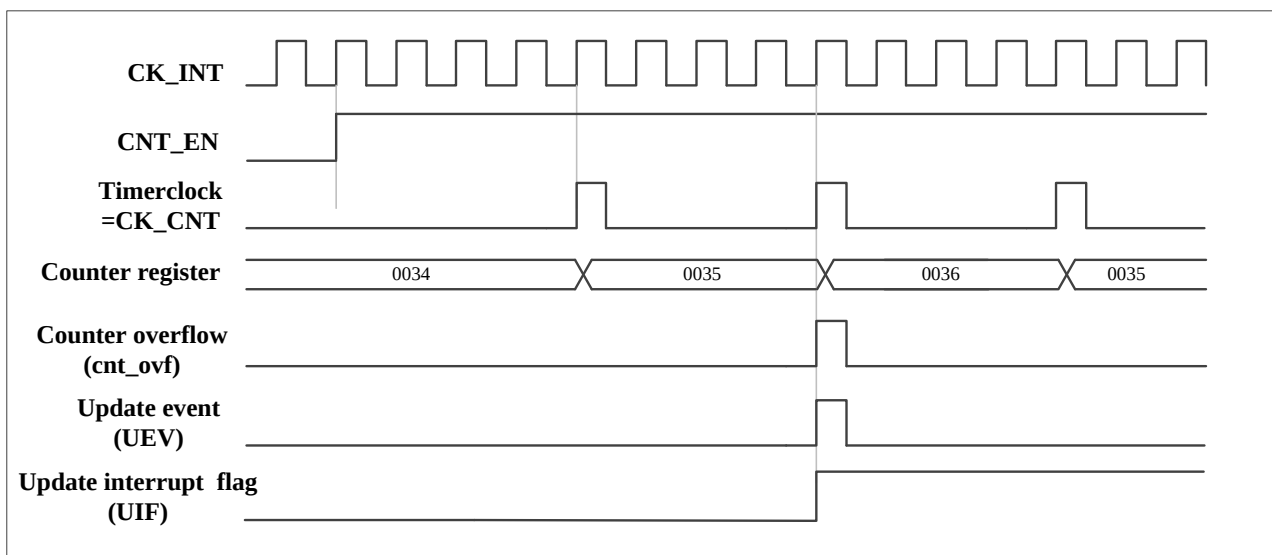


Figure 101 Counter timing diagram, internal clock divided by 4, TIMx_ARR=0x36

- Center-aligned mode 2 or 3 is used with an UIF on overflow.

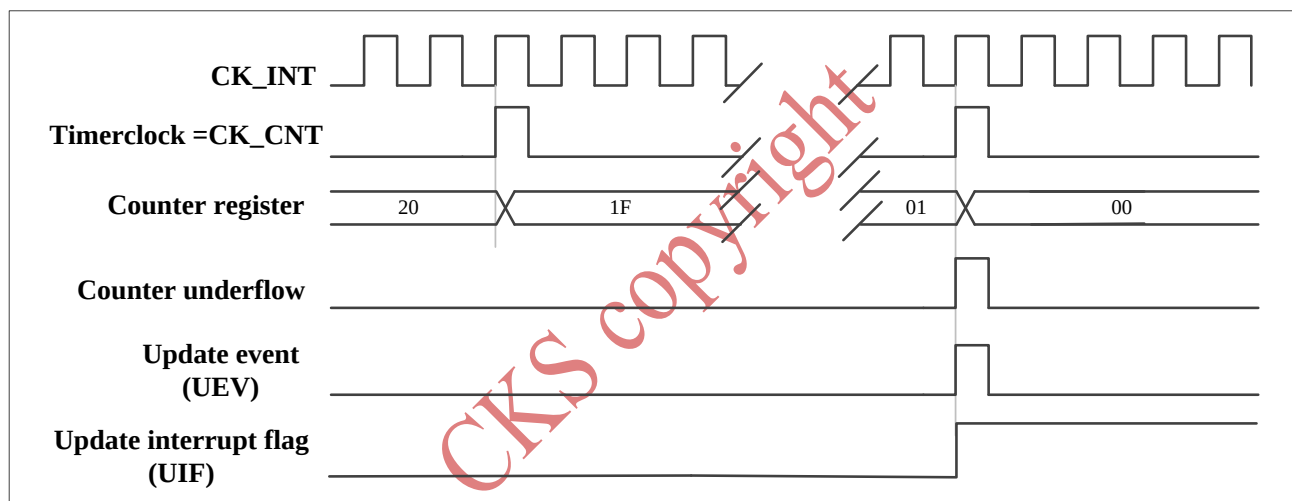


Figure 102 Counter timing diagram, internal clock divided by N

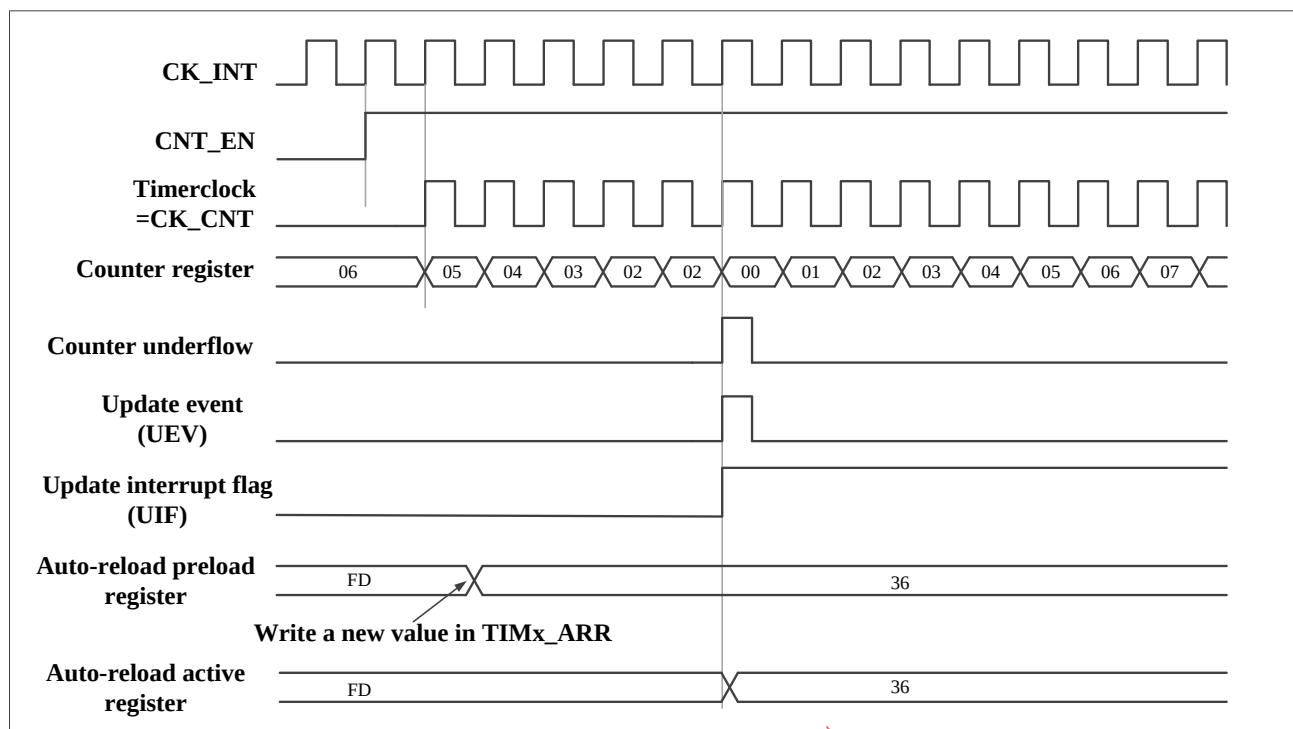


Figure 103 Counter timing diagram, Update event with ARPE=1 (counter underflow)

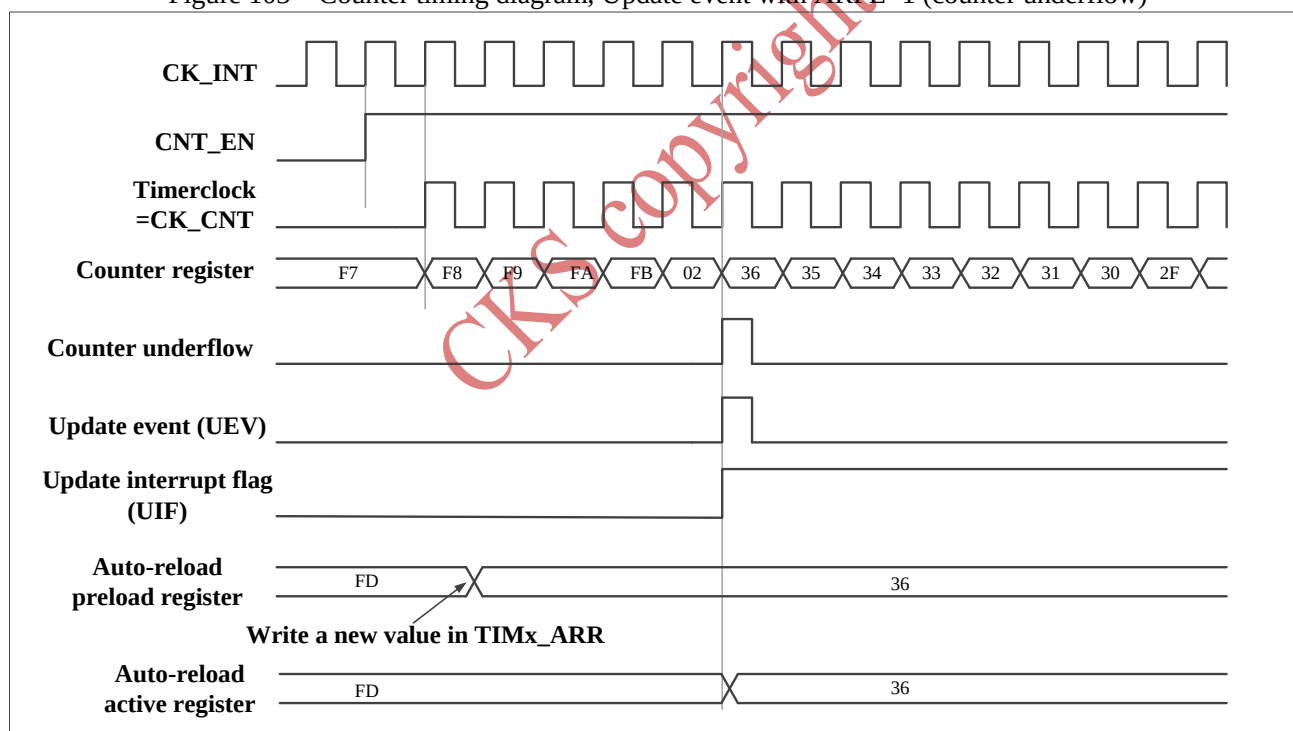


Figure 104 Counter timing diagram, Update event with ARPE=1 (counter overflow)

12.3.3 Clock selection

The counter clock can be provided by the following clock sources:

- Internal clock (CK_INT)
- External clock mode1: external input pin (TIX)
- External clock mode2: external trigger input (ETR).

- Internal trigger inputs (ITRx): using one timer as prescaler for another timer, for example, Timer1 can be configured to act as a prescaler for Timer 2. Refer to *Using one timer as prescaler for another timer* for more details.

Internal clock source (CK_INT)

If the slave mode controller is disabled (SMS=000 in the TIMx_SMCR register), then the CEN, DIR (in the TIMx_CR1 register) and UG bits (in the TIMx_EGR register) are actual control bits and can be changed only by software (except UG which remains cleared automatically). As soon as the CEN bit is written to 1, the prescaler is clocked by the internal clock CK_INT.

Figure 105 shows the behavior of the control circuit and the upcounter in normal mode, without prescaler.

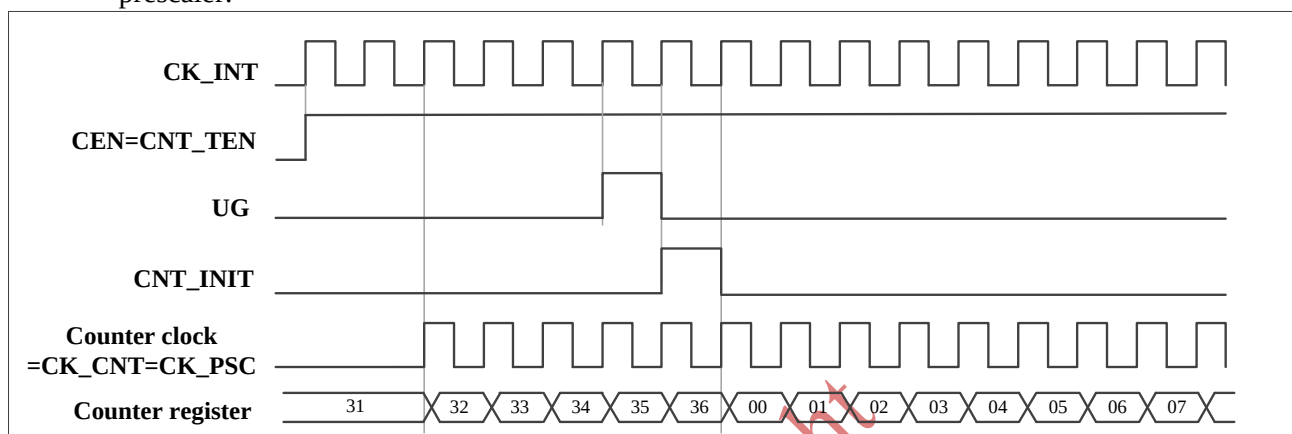


Figure 105 Control circuit in normal mode, internal clock divided by 1

External clock source mode 1

This mode is selected when SMS=111 in the TIMx_SMCR register. The counter can count at each rising or falling edge on a selected input.

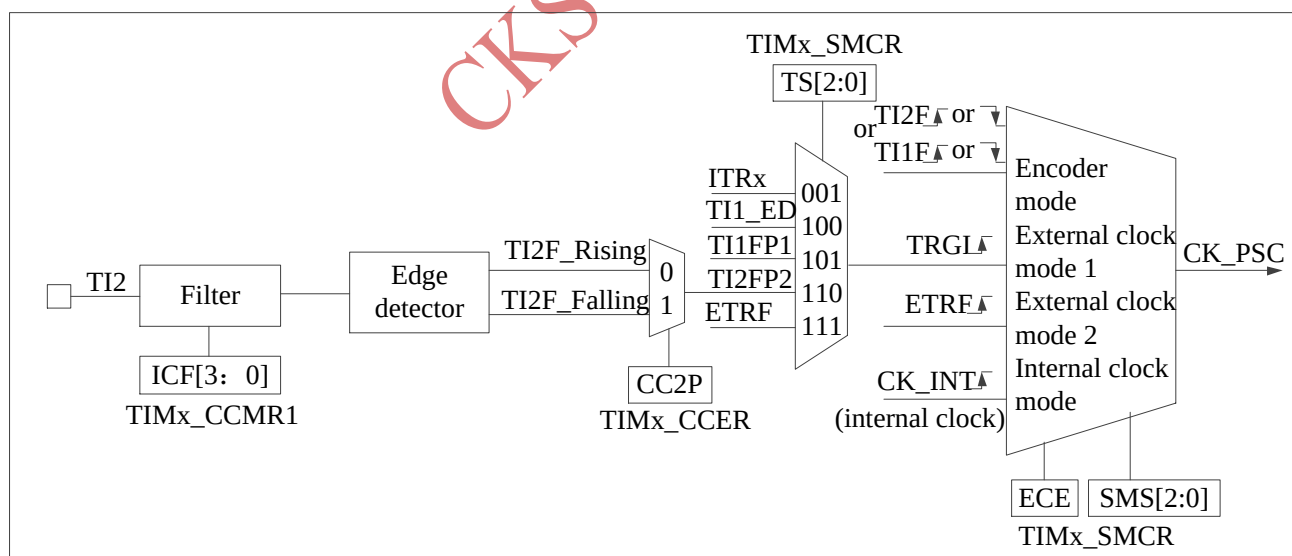


Figure 106 TI2 external clock connection example

For example, to configure the upcounter to count in response to a rising edge on the TI2 input, use the following procedure:

1. Configure channel 2 to detect rising edges on the TI2 input by writing CC2S= '01 in the TIMx_CCMR1 register.
2. Configure the input filter duration by writing the IC2F[3:0] bits in the TIMx_CCMR1 register (if no filter is needed, keep IC2F=0000).

Note: The capture prescaler is not used for triggering, so there's no need to configure it.

3. Select rising edge polarity by writing CC2P=0 in the TIMx_CCER register.
4. Configure the timer in external clock mode 1 by writing SMS=111 in the TIMx_SMCR register.
5. Select TI2 as the input source by writing TS=110 in the TIMx_SMCR register.
6. Enable the counter by writing CEN=1 in the TIMx_CR1 register.

When a rising edge occurs on TI2, the counter counts once and the TIF flag is set.

The delay between the rising edge on TI2 and the actual clock of the counter is due to the resynchronization circuit on TI2 input.

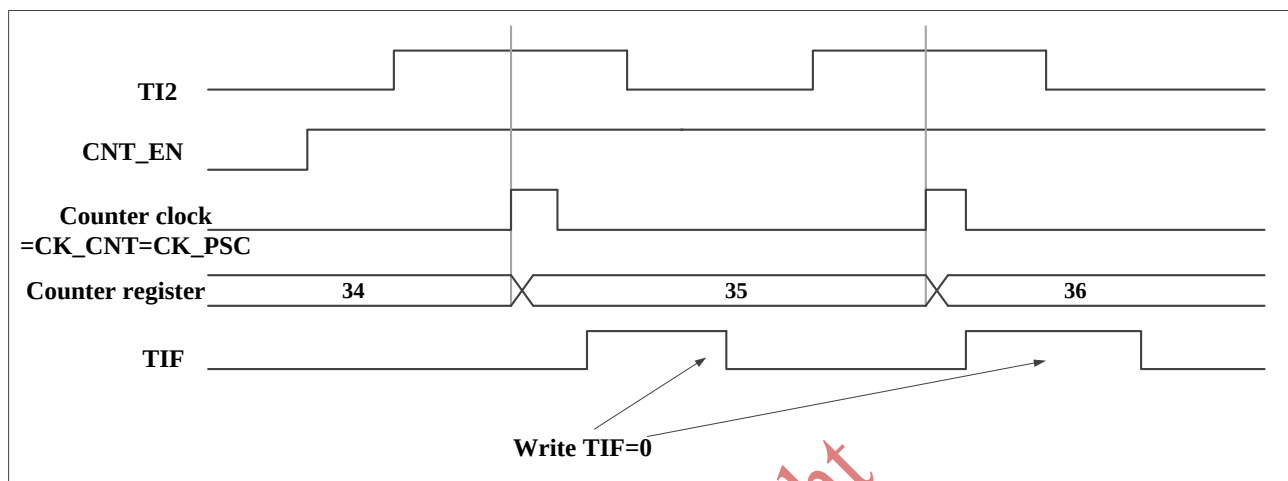


Figure 107 Control circuit in external clock mode 1

External clock source mode 2

This mode is selected by writing ECE=1 in the TIMx_SMCR register.

The counter can count at each rising or falling edge on the external trigger input ETR.

Figure 108 gives an overview of the external trigger input block.

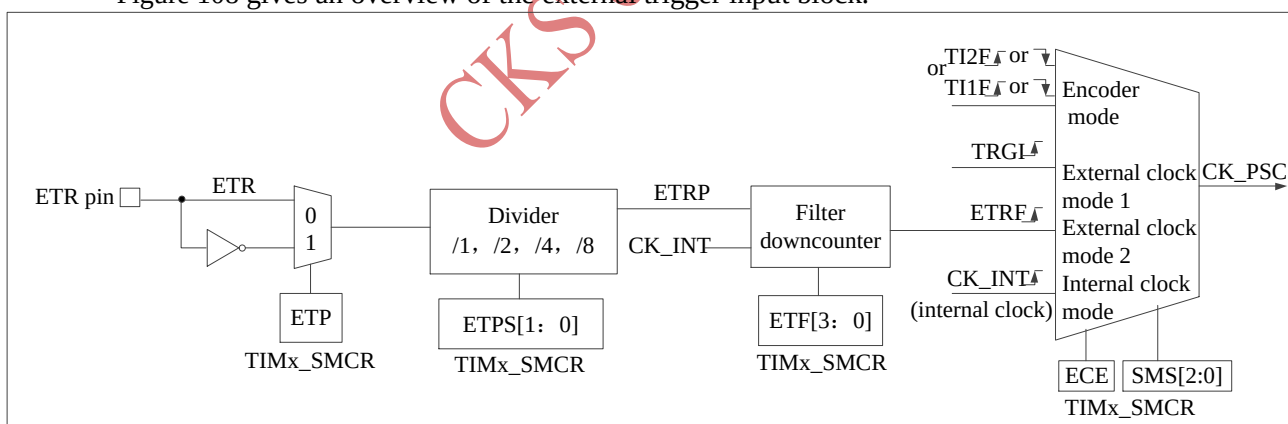


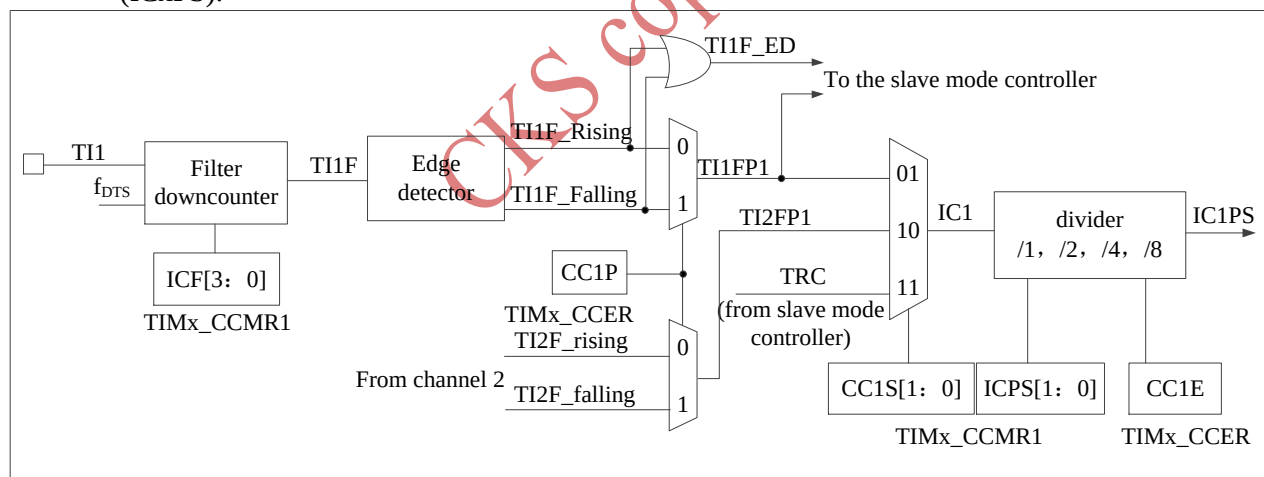
Figure 108 External trigger input block

For example, to configure the upcounter to count each 2 rising edges on ETR, use the following procedure:

1. As no filter is needed in this example, write ETF[3:0]=0000 in the TIMx_SMCR register.
2. Set the prescaler by writing ETPS[1:0]=01 in the TIMx_SMCR register
3. Select rising edge detection on the ETR pin by writing ETP=0 in the TIMx_SMCR register
4. Enable external clock mode 2 by writing ECE=1 in the TIMx_SMCR register.
5. Enable the counter by writing CEN=1 in the TIMx_CR1 register.

The counter counts once each 2 ETR rising edges.

The delay between the rising edge on ETR and the actual clock of the counter is due to the resynchronization circuit on the ETRP signal.



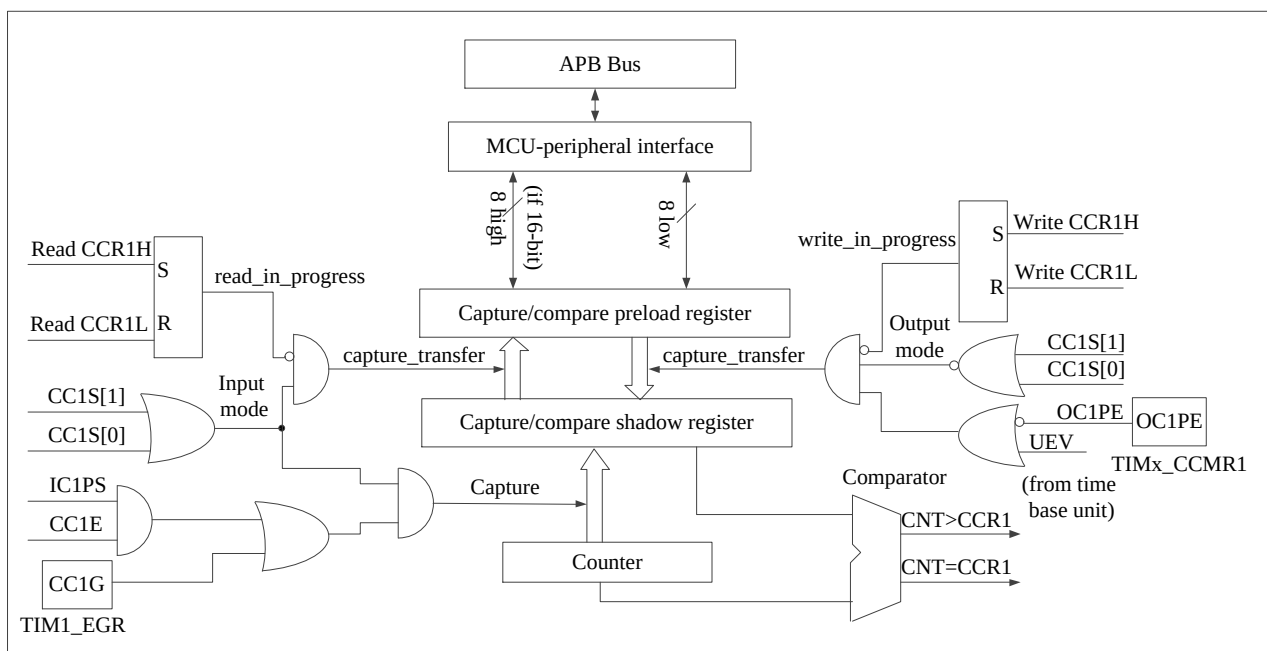


Figure 111 Capture/compare channel 1 main circuit

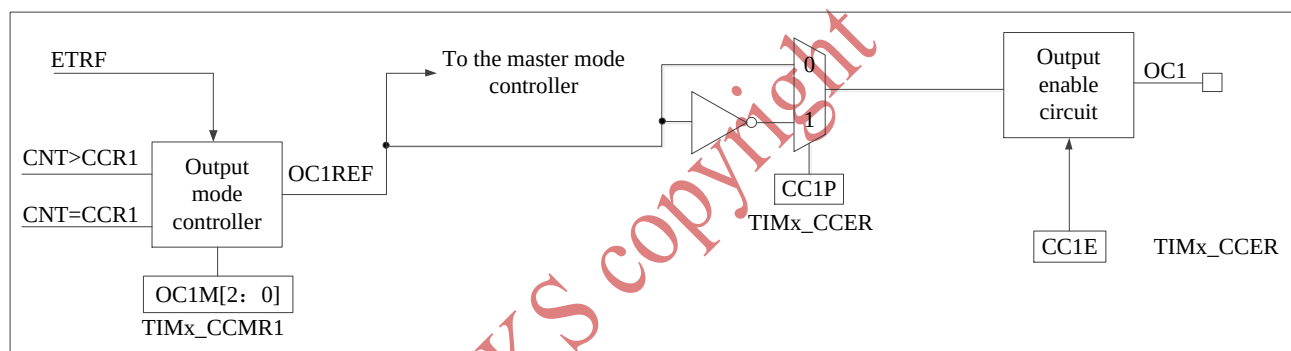


Figure 112 Output stage of capture/compare channel (channel 1)

The capture/compare block is made of one preload register and one shadow register. Write and read always access the preload register.

In capture mode, captures are actually done in the shadow register, which is copied into the preload register.

In compare mode, the content of the preload register is copied into the shadow register which is compared to the counter.

12.3.5 Input capture mode

In Input capture mode, the Capture/Compare Registers (TIMx_CCRx) are used to latch the value of the counter after a transition detected by the corresponding ICx signal. When a capture occurs, the corresponding CCxIF flag (TIMx_SR register) is set and an interrupt or a DMA request can be sent if they are enabled. If a capture occurs while the CCxIF flag was already high, then the over-capture flag CCxOF (TIMx_SR register) is set. CCxIF can be cleared by software by writing it to 0 or by reading the captured data stored in the TIMx_CCRx register. CCxOF is cleared when written to 0.

The following example shows how to capture the counter value in TIMx_CCR1 when TI1 input rises. To do this, use the following procedure:

- Select the active input: TIMx_CCR1 must be linked to the TI1 input, so write the CC1S bits to 01 in the TIMx_CCMR1 register. As soon as CC1S becomes different from 00, the channel is configured in input and the TIMx_CCR1 register becomes read-only.

- Program the needed input filter duration with respect to the signal connected to the timer (by programming the ICxF bits in the TIMx_CCMRx register if the input is one of the TIx inputs). Let's imagine that, when toggling, the input signal is not stable during at most five internal clock cycles. We must program a filter duration longer than these five clock cycles. We can validate a transition on TI1 when eight consecutive samples with the new level have been detected (sampled at f_{DTS} frequency). Then write IC1F bits to 0011 in the TIMx_CCMR1 register.
- Select the edge of the active transition on the TI1 channel by writing the CC1P bit to 0 in the TIMx_CCER register (rising edge in this case).
- Program the input prescaler. In our example, we wish the capture to be performed at each valid transition, so the prescaler is disabled (write IC1PS bits to 00 in the TIMx_CCMR1 register).
- Enable capture from the counter into the capture register by setting the CC1E bit in the TIMx_CCER register.
- If needed, enable the related interrupt request by setting the CC1IE bit in the TIMx_DIER register, and/or the DMA request by setting the CC1DE bit in the TIMx_DIER register.

When an input capture occurs:

- The TIMx_CCR1 register gets the value of the counter on the active transition.
- CC1IF flag is set (interrupt flag). CC1OF is also set if at least two consecutive captures occurred whereas the flag was not cleared.
- An interrupt is generated depending on the CC1IE bit.
- A DMA request is generated depending on the CC1DE bit.

In order to handle the overcapture, it is recommended to read the data before the overcapture flag. This is to avoid missing an overcapture which could happen after reading the flag and before reading the data.

Note: IC interrupt and/or DMA requests can be generated by software by setting the corresponding CCxG bit in the TIMx_EGR register.

12.3.6 PWM input mode

This mode is a particular case of input capture mode. The procedure is the same except:

- Two ICx signals are mapped on the same TIx input.
- These 2 ICx signals are active on edges with opposite polarity.
- One of the two TIxFP signals is selected as trigger input and the slave mode controller is configured in reset mode.

For example, the user can measure the period (in TIMx_CCR1 register) and the duty cycle (in TIMx_CCR2 register) of the PWM applied on TI1 using the following procedure (depending on CK_INT frequency and prescaler value):

- Select the active input for TIMx_CCR1: write the CC1S bits to 01 in the TIMx_CCMR1 register (TI1 selected).
- Select the active polarity for TI1FP1 (used both for capture in TIMx_CCR1 and counter clear): write the CC1P to '0' (active on rising edge).
- Select the active input for TIMx_CCR2: write the CC2S bits to 10 in the TIMx_CCMR1 register (TI1 selected).
- Select the active polarity for TI1FP2 (used for capture in TIMx_CCR2): write the CC2P bit to '1' (active on falling edge).
- Select the valid trigger input: write the TS bits to 101 in the TIMx_SMCR register (TI1FP1 selected).
- Configure the slave mode controller in reset mode: write the SMS bits to 100 in the TIMx_SMCR register.
- Enable the captures: write the CC1E and CC2E bits to '1' in the TIMx_CCER register.

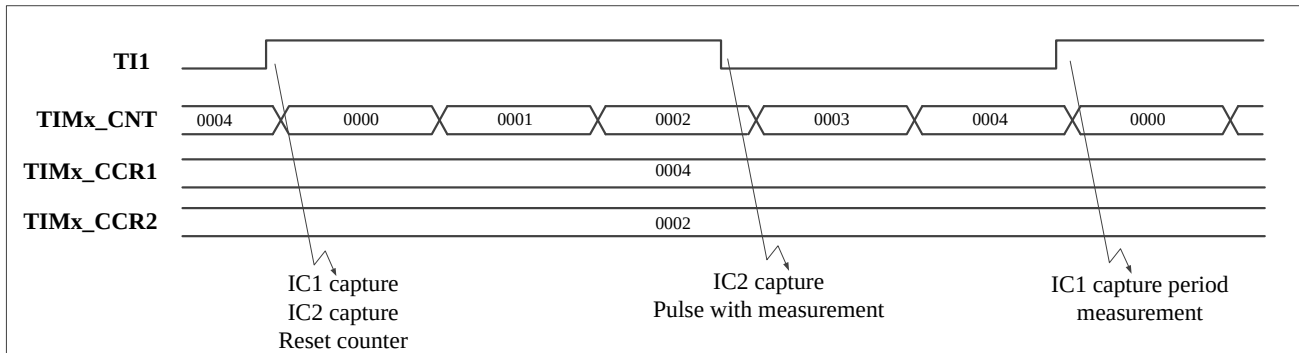


Figure 113 PWM input mode timing

The PWM input mode can be used only with the TIMx_CH1/TIMx_CH2 signals due to the fact that only. TI1FP1 and TI2FP2 are connected to the slave mode controller.

12.3.7 Forced output mode

In output mode (CCxS bits = 00 in the TIMx_CCMRx register), each output compare signal (OCxREF and then OCx) can be forced to active or inactive level directly by software, independently of any comparison between the output compare register and the counter.

To force an output compare signal (ocxref/OCx) to its active level, the user just needs to write 101 in the OCxM bits in the corresponding TIMx_CCMRx register. Thus ocxref is forced high (OCxREF is always active high) and OCx get opposite value to CCxP polarity bit.

e.g.: CCxP=0 (OCx active high) => OCx is forced to high level.

OCxREF signal can be forced low by writing the OCxM bits to 100 in the TIMx_CCMRx register.

Anyway, the comparison between the TIMx_CCRx shadow register and the counter is still performed and allows the flag to be set. Interrupt and DMA requests can be sent accordingly. This is described in the Output Compare Mode section.

12.3.8 Output compare mode

This function is used to control an output waveform or indicating when a period of time has elapsed. When a match is found between the capture/compare register and the counter, the output compare function:

- Assigns the corresponding output pin to a programmable value defined by the output compare mode (OCxM bits in the TIMx_CCMRx register) and the output polarity (CCxP bit in the TIMx_CCER register). The output pin can keep its level (OCxM=000), be set active (OCxM=001), be set inactive (OCxM=010) or can toggle (OCxM=011) on match.
- Sets a flag in the interrupt status register (CCxIF bit in the TIMx_SR register).
- Generates an interrupt if the corresponding interrupt mask is set (CCXIE bit in the TIMx_DIER register).
- Sends a DMA request if the corresponding enable bit is set (CCxDE bit in the TIMx_DIER register, CCDS bit in the TIMx_CR2 register for the DMA request selection).

The TIMx_CCRx registers can be programmed with or without preload registers using the OCxPE bit in the TIMx_CCMRx register.

In output compare mode, the update event UEV has no effect on ocxref and OCx output. The timing resolution is one count of the counter. Output compare mode can also be used to output a single pulse (in One-pulse mode).

Procedure:

1. Select the counter clock (internal, external, prescaler).
2. Write the desired data in the TIMx_ARR and TIMx_CCRx registers.
3. Set the CCXIE and/or CCxDE bits if an interrupt and/or a DMA request is to be generated.

4. Select the output mode. For example, the user must write OCxM=011, OCxPE=0, CCxP=0 and CCxE=1 to toggle OCx output pin when CNT matches CCRx, CCRx preload is not used, OCx is enabled and active high.
 5. Enable the counter by setting the CEN bit in the TIMx_CR1 register.
- The TIMx_CCRx register can be updated at any time by software to control the output waveform, provided that the preload register is not enabled (OCxPE=0, else TIMx_CCRx shadow register is updated only at the next update event UEV). An example is given in Figure 114.

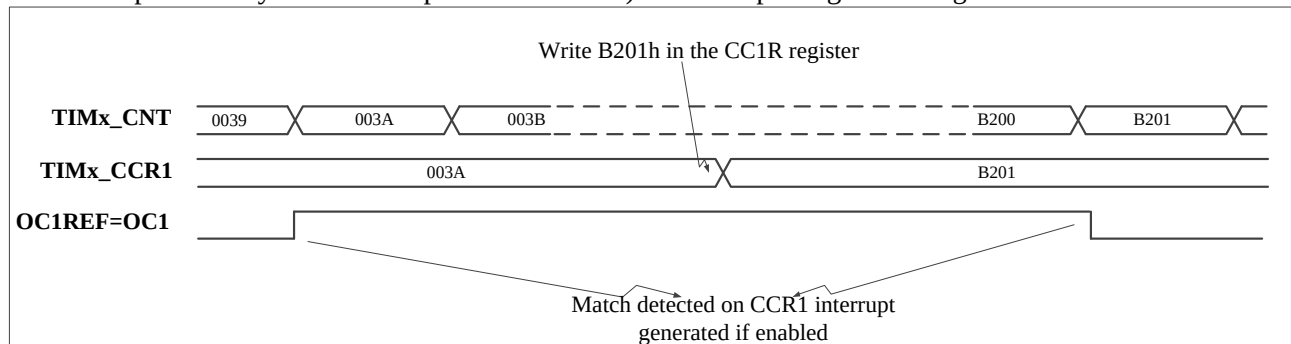


Figure 114 Output compare mode, toggle on OC1

12.3.9 PWM mode

Pulse width modulation mode allows generating a signal with a frequency determined by the value of the TIMx_ARR register and a duty cycle determined by the value of the TIMx_CCRx register.

The PWM mode can be selected independently on each channel (one PWM per OCx output) by writing 110 (PWM mode 1) or '111 (PWM mode 2) in the OCxM bits in the TIMx_CCMRx register. The user must enable the corresponding preload register by setting the OCxPE bit in the TIMx_CCMRx register, and eventually the auto-reload preload register by setting the ARPE bit in the TIMx_CR1 register.

As the preload registers are transferred to the shadow registers only when an update event occurs, before starting the counter, the user has to initialize all the registers by setting the UG bit in the TIMx_EGR register.

OCx polarity is software programmable using the CCxP bit in the TIMx_CCER register. It can be programmed as active high or active low. OCx output is enabled by the CCxE bit in the TIMx_CCER register. Refer to the TIMx_CCERx register description for more details.

In PWM mode (1 or 2), TIMx_CNT and TIMx_CCRx are always compared to determine whether $TIMx_CCRx \leq TIMx_CNT$ or $TIMx_CNT \leq TIMx_CCRx$ (depending on the direction of the counter). However, to comply with the ETRF (OCREF can be cleared by an external event through the ETR signal until the next PWM period), the OCREF signal is asserted only:

- When the result of the comparison changes, or
- When the output compare mode (OCxM bits in TIMx_CCMRx register) switches from the “frozen” configuration (no comparison, OCxM='000) to one of the PWM modes (OCxM='110 or '111).

This forces the PWM by software while the timer is running.

The timer is able to generate PWM in edge-aligned mode or center-aligned mode depending on the CMS bits in the TIMx_CR1 register.

PWM edge-aligned mode

Upcounting configuration

Upcounting is active when the DIR bit in the TIMx_CR1 register is low. Refer to *Upcounting mode*.

In the following example, we consider PWM mode 1. The reference PWM signal OCxREF is high as long as $TIMx_CNT < TIMx_CCRx$ else it becomes low. If the compare value in TIMx_CCRx is greater than the auto-reload value (in TIMx_ARR) then OCxREF is held at '1. If the compare value is 0 then OCxREF is held at '0. Figure 115 shows some edge-aligned PWM waveforms in an example where TIMx_ARR=8.

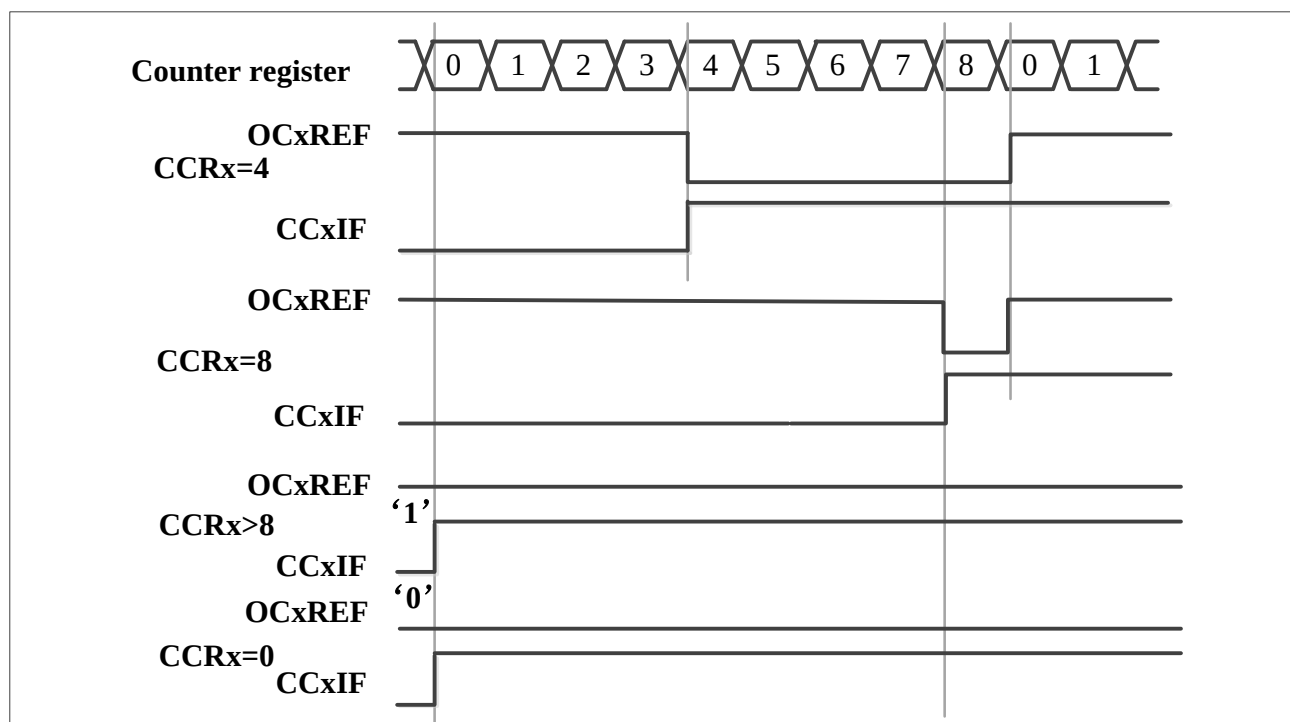


Figure 115 Edge-aligned PWM waveforms (ARR=8)

Downcounting configuration

Downcounting is active when DIR bit in TIMx_CR1 register is high. Refer to *Downcounting mode*.

In PWM mode 1, the reference signal ocxref is low as long as TIMx_CNT > TIMx_CCRx else it becomes high. If the compare value in TIMx_CCRx is greater than the auto-reload value in TIMx_ARR, then ocxref is held at '1'. 0% PWM is not possible in this mode.

PWM center-aligned mode

Center-aligned mode is active when the CMS bits in TIMx_CR1 register are different from '00' (all the remaining configurations having the same effect on the ocxref/OCx signals). The compare flag is set when the counter counts up, when it counts down or both when it counts up and down depending on the CMS bits configuration. The direction bit (DIR) in the TIMx_CR1 register is updated by hardware and must not be changed by software. Refer to *Center-aligned mode (up/down counting)*.

Figure 116 shows some center-aligned PWM waveforms in an example where:

- TIMx_ARR=8,
- PWM mode is the PWM mode 1,
- The flag is set when the counter counts down corresponding to the center-aligned mode 1 selected for CMS=01 in TIMx_CR1 register.

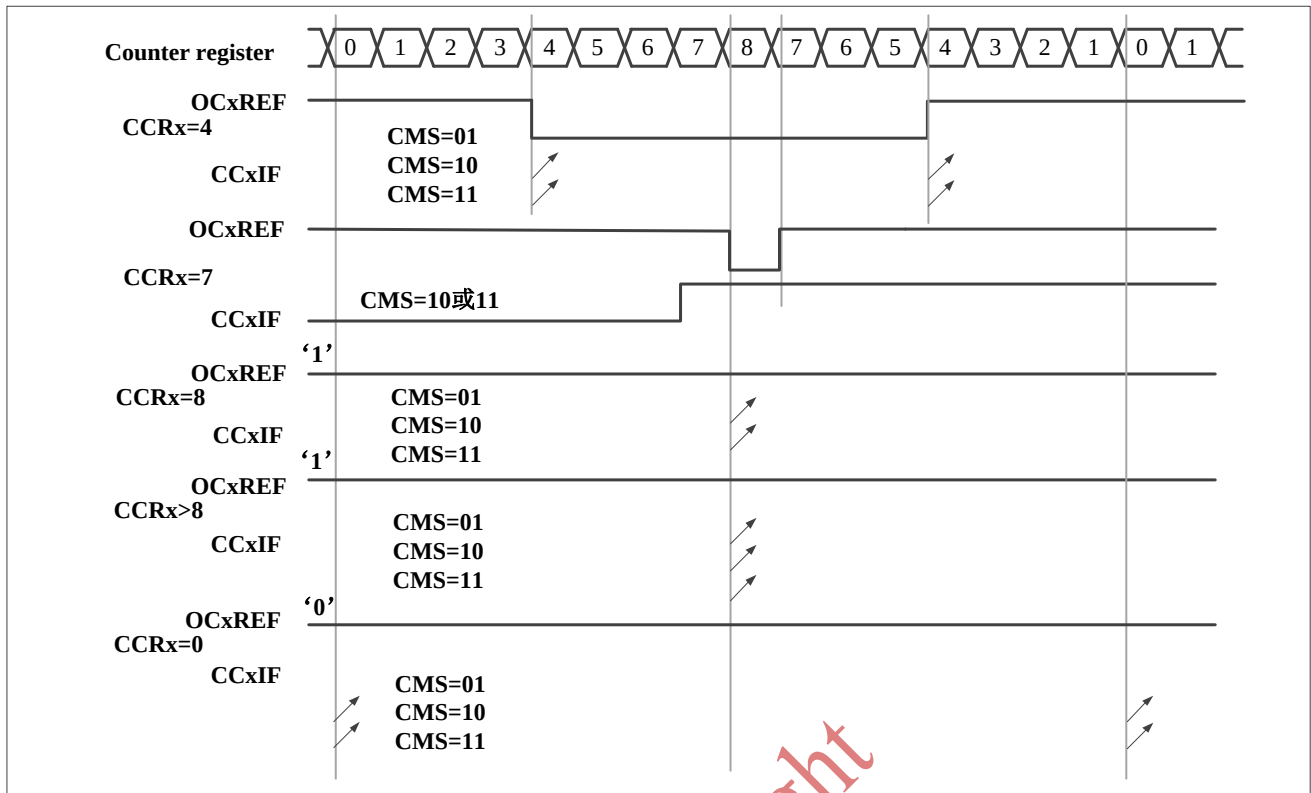


Figure 116 Center-aligned PWM waveforms (ARR=8)

Hints on using center-aligned mode:

- When starting in center-aligned mode, the current up-down configuration is used. It means that the counter counts up or down depending on the value written in the DIR bit in the TIMx_CR1 register. Moreover, the DIR and CMS bits must not be changed at the same time by the software.
- Writing to the counter while running in center-aligned mode is not recommended as it can lead to unexpected results. In particular:
 - The direction is not updated if the user writes a value in the counter that is greater than the auto-reload value ($TIMx_CNT > TIMx_ARR$). For example, if the counter was counting up, it continues to count up.
 - The direction is updated if the user writes 0 or write the TIMx_ARR value in the counter but no Update Event UEV is generated.
- The safest way to use center-aligned mode is to generate an update by software (setting the UG bit in the TIMx_EGR register) just before starting the counter and not to write the counter while it is running.

12.3.10 One-pulse mode

One-pulse mode (OPM) is a particular case of the previous modes. It allows the counter to be started in response to a stimulus and to generate a pulse with a programmable length after a programmable delay.

Starting the counter can be controlled through the slave mode controller. Generating the waveform can be done in output compare mode or PWM mode. Select One-pulse mode by setting the OPM bit in the TIMx_CR1 register. This makes the counter stop automatically at the next update event UEV. A pulse can be correctly generated only if the compare value is different from the counter initial value. Before starting (when the timer is waiting for the trigger), the configuration must be:

- In upcounting: $CNT < CCRx \leq ARR$ (in particular, $0 < CCRx$),
- In downcounting: $CNT > CCRx$.

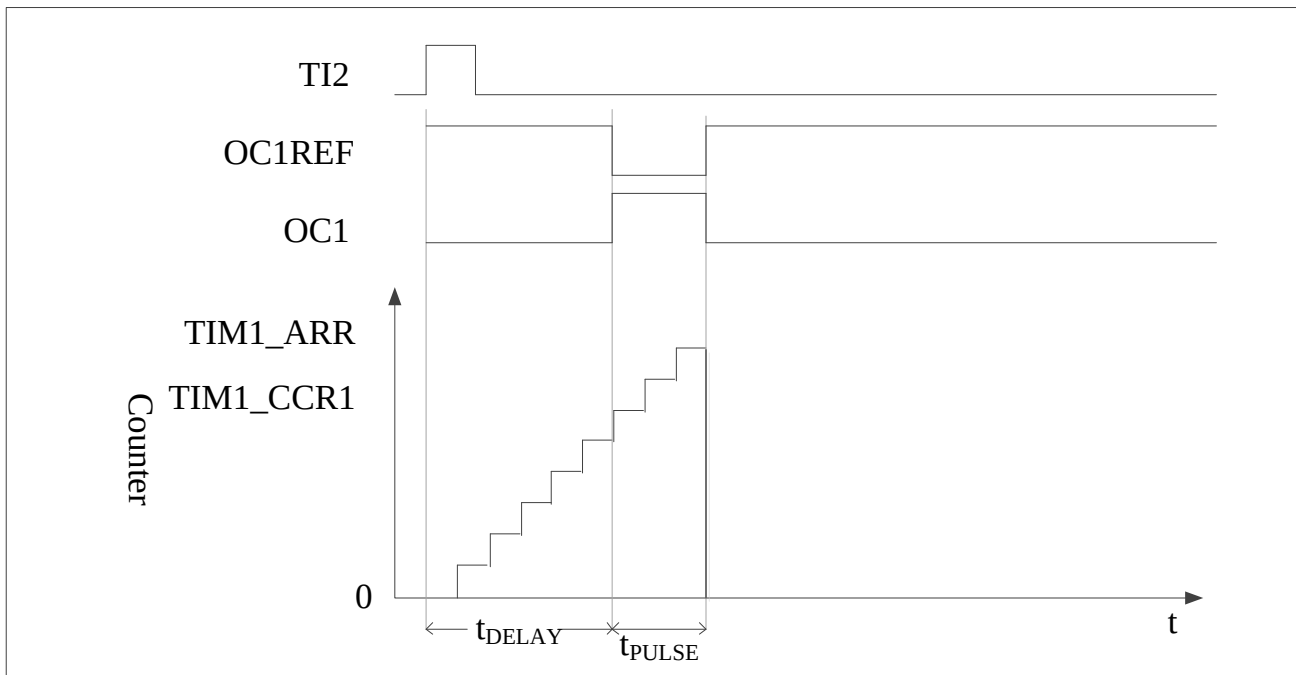


Figure 117 Example of one-pulse mode

For example the user may want to generate a positive pulse on OC1 with a length of t_{PULSE} and after a delay of t_{DELAY} as soon as a positive edge is detected on the TI2 input pin.

Let's use TI2FP2 as trigger 1:

- Map TI2FP2 on TI2 by writing CC2S=01 in the TIMx_CCMR1 register.
- TI2FP2 must detect a rising edge, write CC2P=0 in the TIMx_CCER register.
- Configure TI2FP2 as trigger for the slave mode controller (TRGI) by writing TS=110 in the TIMx_SMCR register.
- TI2FP2 is used to start the counter by writing SMS to '110 in the TIMx_SMCR register (trigger mode).

The OPM waveform is defined by writing the compare registers (taking into account the clock frequency and the counter prescaler).

- The t_{DELAY} is defined by the value written in the TIMx_CCR1 register.
- The t_{PULSE} is defined by the difference between the auto-reload value and the compare value (TIMx_ARR - TIMx_CCR + 1).
- Let us say user wants to build a waveform with a transition from '0 to '1 when a compare match occurs and a transition from '1 to '0 when the counter reaches the auto-reload value. To do this enable PWM mode 2 by writing OC1M=111 in the TIMx_CCMR1 register. The user can optionally enable the preload registers by writing OC1PE=1 in the TIMx_CCMR1 register and ARPE in the TIMx_CR1 register. In this case write the compare value in the TIMx_CCR1 register, the auto-reload value in the TIMx_ARR register, generate an update by setting the UG bit and wait for external trigger event on TI2. CC1P is written to '0 in this example.

In our example, the DIR and CMS bits in the TIMx_CR1 register should be low.

User only wants one pulse (Single mode), so write '1 in the OPM bit in the TIMx_CR1 register to stop the counter at the next update event (when the counter rolls over from the auto-reload value back to 0). When OPM bit in the TIMx_CR1 register is set to '0', so the Repetitive Mode is selected.

Particular case: OCx fast enable:

In One-pulse mode, the edge detection on TIx input set the CEN bit which enables the counter. Then the comparison between the counter and the compare value makes the output toggle. But several clock cycles are needed for these operations and it limits the minimum delay $t_{\text{DELAY min}}$ we can get. To output a waveform with the minimum delay, the user can set the OCxFE bit in the TIMx_CCMRx register. Then OCxRef (and OCx) is forced in response to the stimulus, without taking in account the comparison. Its new level is the same as if a compare match had occurred. OCxFE acts only if the channel is configured in PWM1 or PWM2 mode.

12.3.11 Clearing the OCxREF signal on an external event

The OCxREF signal for a given channel can be driven Low by applying a High level to the ETRF input (OCxCE enable bit of the corresponding TIMx_CCMRx register set to '1'). The OCxREF signal remains Low until the next update event, UEV, occurs.

This function can only be used in output compare and PWM modes, and does not work in forced mode.

For example, the ETR signal can be connected to the output of a comparator to be used for current handling. In this case, ETR must be configured as follows:

1. The external trigger prescaler should be kept off: bits ETPS[1:0] in the TIMx_SMCR register are cleared to 00.
2. The external clock mode 2 must be disabled: bit ECE in the TIM1_SMCR register is cleared to 0.
3. The external trigger polarity (ETP) and the external trigger filter (ETF) can be configured according to the application's needs.

Figure 118 shows the behavior of the OCxREF signal when the ETRF input becomes high, for both values of the OCxCE enable bit. In this example, the timer TIMx is programmed in PWM mode.

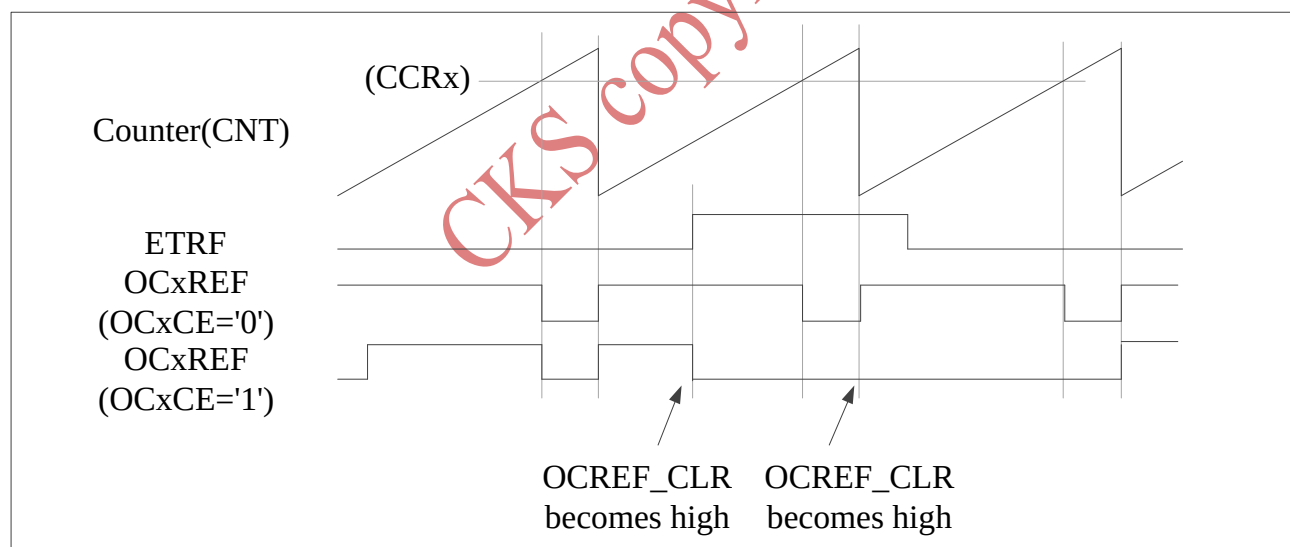


Figure 118 Clearing TIMx OCxREF

12.3.12 Encoder interface mode

To select Encoder Interface mode write SMS='001 in the TIMx_SMCR register if the counter is counting on TI2 edges only, SMS=010 if it is counting on TI1 edges only and SMS=011 if it is counting on both TI1 and TI2 edges.

Select the TI1 and TI2 polarity by programming the CC1P and CC2P bits in the TIMx_CCER register. When needed, program the input filter as well.

The two inputs TI1 and TI2 are used to interface to an incremental encoder. Refer to Table 56. The counter is clocked by each valid transition on TI1FP1 or TI2FP2 (TI1 and TI2 after input filter and

polarity selection, TI1FP1=TI1 if not filtered and not inverted, TI2FP2=TI2 if not filtered and not inverted) assuming that it is enabled (CEN bit in TIMx_CR1 register written to '1'). The sequence of transitions of the two inputs is evaluated and generates count pulses as well as the direction signal. Depending on the sequence the counter counts up or down, the DIR bit in the TIMx_CR1 register is modified by hardware accordingly. The DIR bit is calculated at each transition on any input (TI1 or TI2), whatever the counter is counting on TI1 only, TI2 only or both TI1 and TI2.

Encoder interface mode acts simply as an external clock with direction selection. This means that the counter just counts continuously between 0 and the auto-reload value in the TIMx_ARR register (0 to ARR or ARR down to 0 depending on the direction). So the user must configure TIMx_ARR before starting. In the same way, the capture, compare, prescaler, trigger output features continue to work as normal.

In this mode, the counter is modified automatically following the speed and the direction of the incremental encoder and its content, therefore, always represents the encoder's position. The count direction correspond to the rotation direction of the connected sensor. The table summarizes the possible combinations, assuming TI1 and TI2 don't switch at the same time.

Table 56 Counting direction versus encoder signals

Active edge	Level on opposite signal (TI1FP1 for TI2, TI2FP2 for TI1)	TI1FP1 signal		TI2FP2 signal	
		Rising	Falling	Rising	Falling
Counting on TI1 only	High	Down	Up	No Count	No Count
	Low	Up	Down	No Count	No Count
Counting on TI2 only	High	No Count	No Count	Up	Down
	Low	No Count	No Count	Down	Up
Counting on TI1 and TI2	High	Down	Up	Up	Down
	Low	Up	Down	Down	Up

An external incremental encoder can be connected directly to the MCU without external interface logic. However, comparators are normally be used to convert the encoder's differential outputs to digital signals. This greatly increases noise immunity. The third encoder output which indicate the mechanical zero position, may be connected to an external interrupt input and trigger a counter reset. Figure 119 gives an example of counter operation, showing count signal generation and direction control. It also shows how input jitter is compensated where both edges are selected. This might occur if the sensor is positioned near to one of the switching points. For this example we assume that the configuration is the following:

- CC1S= '01' (TIMx_CCMR1 register, TI1FP1 mapped on TI1)
- CC2S= '01' (TIMx_CCMR2 register, TI2FP2 mapped on TI2)
- CC1P= '0', CC1NP= '0', IC1F= '0000' (TIMx_CCER register, TI1FP1 noninverted, TI1FP1=TI1)
- CC2P= '0', CC2NP= '0', IC2F= '0000' (TIMx_CCER register, TI2FP2 noninverted, TI2FP2=TI2)
- SMS= '011' (TIMx_SMCR register, both inputs are active on both rising and falling edges)
- CEN= 1 (TIMx_CR1 register, Counter is enabled)

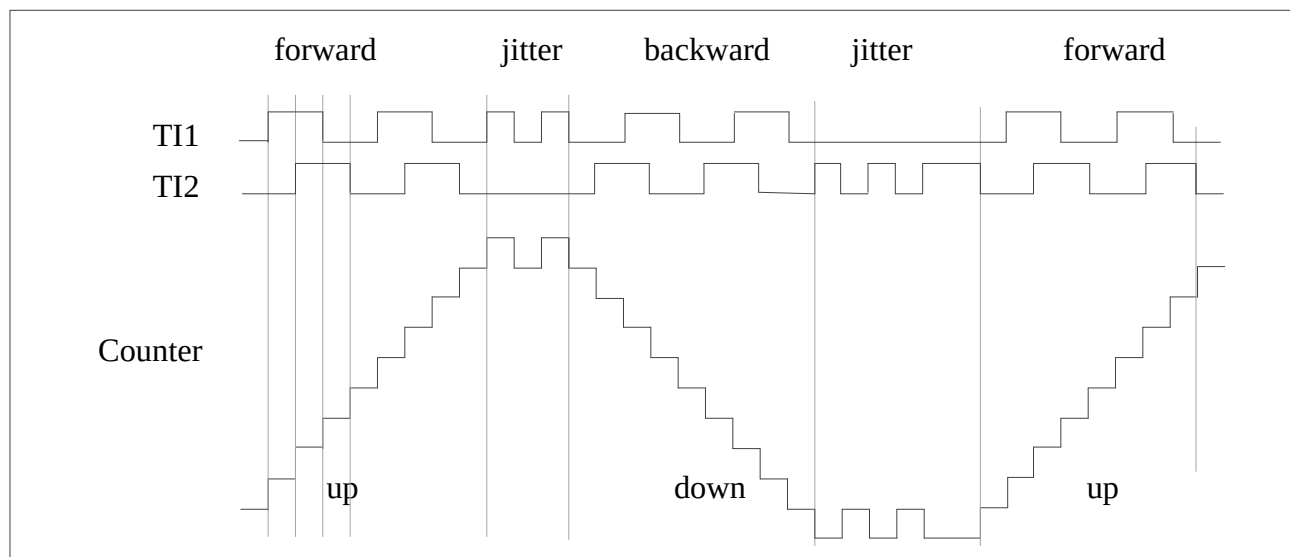


Figure 119 Example of counter operation in encoder interface mode

Figure 120 gives an example of counter behavior when TI1FP1 polarity is inverted (same configuration as above except CC1P=1).

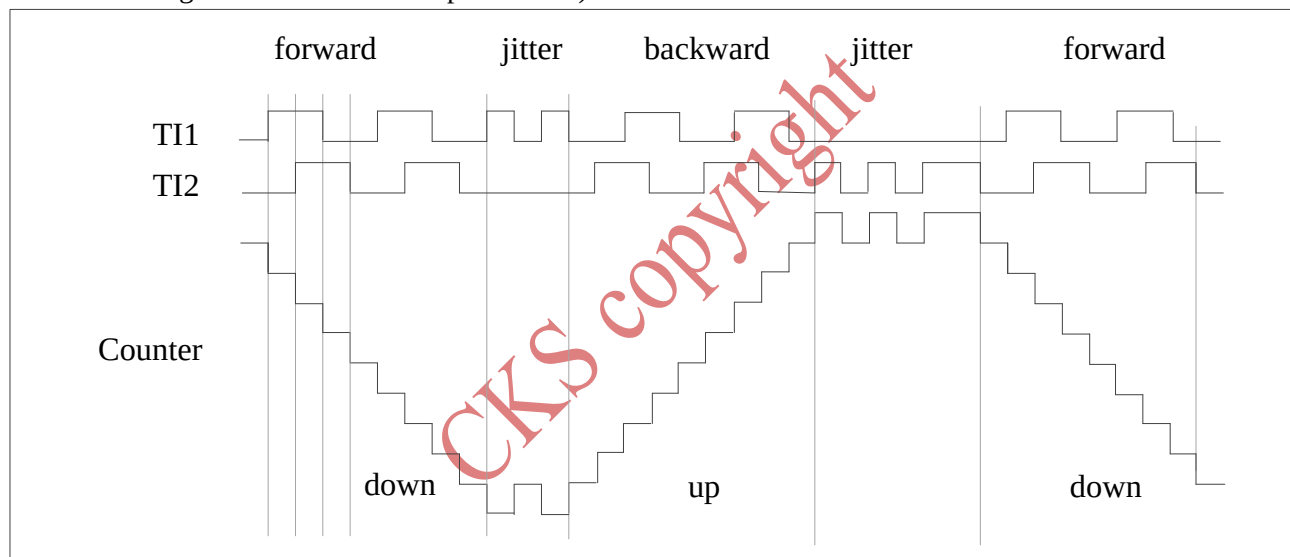


Figure 120 Example of encoder interface mode with TI1FP1 polarity inverted

The timer, when configured in Encoder Interface mode provides information on the sensor's current position. The user can obtain dynamic information (speed, acceleration, deceleration) by measuring the period between two encoder events using a second timer configured in capture mode. The output of the encoder which indicates the mechanical zero can be used for this purpose. Depending on the time between two events, the counter can also be read at regular times. The user can do this by latching the counter value into a third input capture register if available (then the capture signal must be periodic and can be generated by another timer). when available, it is also possible to read its value through a DMA request generated by a Real-Time clock.

12.3.13 Timer input XOR function

The TI1S bit in the TIM1_CR2 register, allows the input filter of channel 1 to be connected to the output of a XOR gate, combining the three input pins TIMx_CH1 to TIMx_CH3.

The XOR output can be used with all the timer input functions such as trigger or input capture. An example of this feature used to interface Hall sensors is given in Section 12.3.18.

12.3.14 Timers and external trigger synchronization

The TIMx Timers can be synchronized with an external trigger in several modes: Reset mode, Gated mode and Trigger mode.

Slave mode: Reset mode

The counter and its prescaler can be reinitialized in response to an event on a trigger input. Moreover, if the URS bit from the TIMx_CR1 register is low, an update event UEV is generated. Then all the preloaded registers (TIMx_ARR, TIMx_CCRx) are updated.

In the following example, the upcounter is cleared in response to a rising edge on TI1 input:

- Configure the channel 1 to detect rising edges on TI1. Configure the input filter duration (in this example, we don't need any filter, so we keep IC1F=0000). The capture prescaler is not used for triggering, so the user does not need to configure it. The CC1S bits select the input capture source only, CC1S = 01 in the TIMx_CCMR1 register. Write CC1P=0 in TIMx_CCER register to validate the polarity (and detect rising edges only).
- Configure the timer in reset mode by writing SMS=100 in TIMx_SMCR register. Select TI1 as the input source by writing TS=101 in TIMx_SMCR register.
- Start the counter by writing CEN=1 in the TIMx_CR1 register.

The counter starts counting on the internal clock, then behaves normally until TI1 rising edge. When TI1 rises, the counter is cleared and restarts from 0. In the meantime, the trigger flag is set (TIF bit in the TIMx_SR register) and an interrupt request, or a DMA request can be sent if enabled (depending on the TIE and TDE bits in TIMx_DIER register).

Figure 121 shows this behavior when the auto-reload register TIMx_ARR=0x36. The delay between the rising edge on TI1 and the actual reset of the counter is due to the resynchronization circuit on TI1 input.

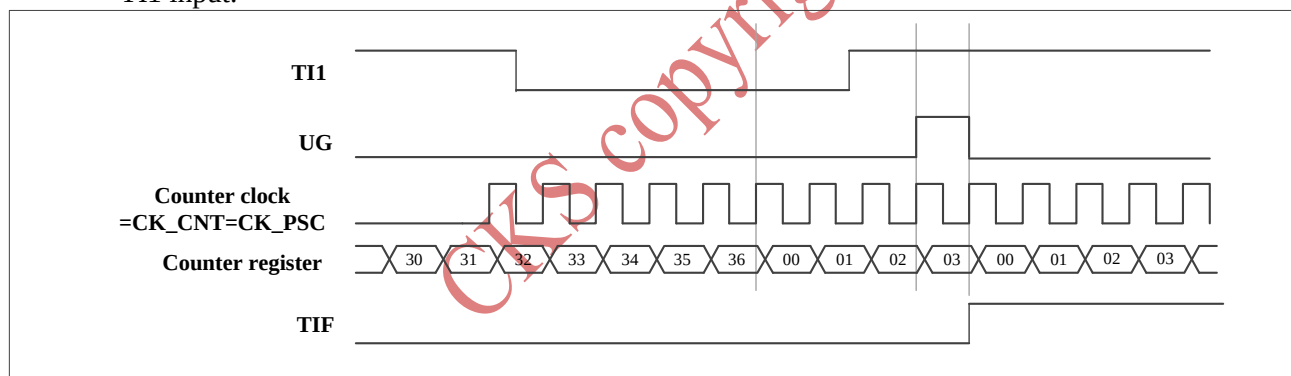


Figure 121 Control circuit in reset mode

Slave mode: Gated mode

The counter can be enabled depending on the level of a selected input.

In the following example, the upcounter counts only when TI1 input is low:

- Configure the channel 1 to detect low levels on TI1. Configure the input filter duration (in this example, we don't need any filter, so we keep IC1F=0000). The capture prescaler is not used for triggering, so the user does not need to configure it. The CC1S bits select the input capture source only, CC1S=01 in TIMx_CCMR1 register. Write CC1P=1 in TIMx_CCER register to validate the polarity (and detect low level only).
- Configure the timer in gated mode by writing SMS=101 in TIMx_SMCR register. Select TI1 as the input source by writing TS=101 in TIMx_SMCR register.
- Enable the counter by writing CEN=1 in the TIMx_CR1 register (in gated mode, the counter doesn't start if CEN=0, whatever is the trigger input level).

The counter starts counting on the internal clock as long as TI1 is low and stops as soon as TI1 becomes high. The TIF flag in the TIMx_SR register is set both when the counter starts or stops. The delay between the rising edge on TI1 and the actual stop of the counter is due to the resynchronization circuit on TI1 input.

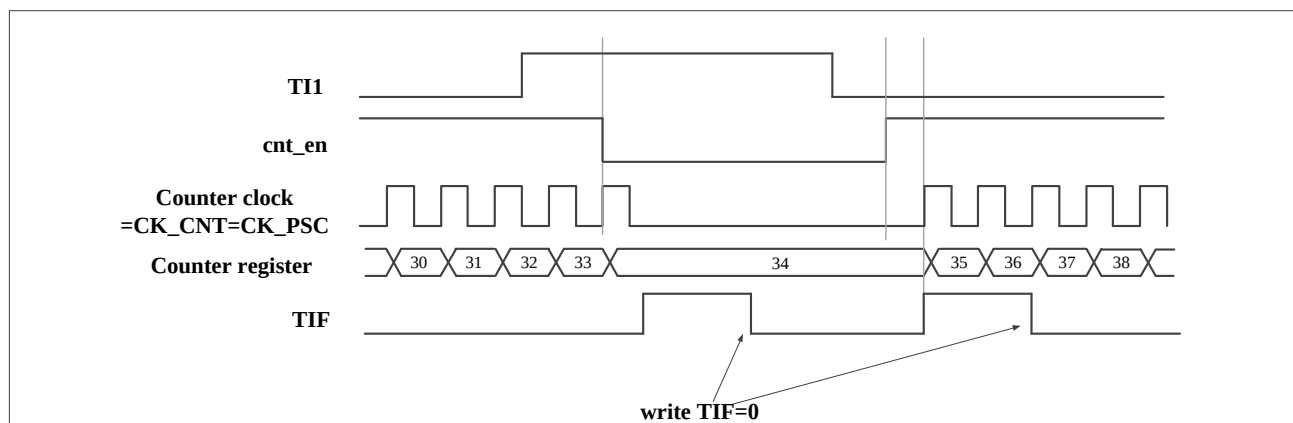


Figure 122 Control circuit in reset mode

Slave mode: Trigger mode

The counter can start in response to an event on a selected input.

In the following example, the upcounter starts in response to a rising edge on TI2 input:

- Configure the channel 2 to detect rising edges on TI2. Configure the input filter duration (in this example, we don't need any filter, so we keep IC2F=0000). The capture prescaler is not used for triggering, so the user does not need to configure it. CC2S bits are selecting the input capture source only, CC2S=01 in TIMx_CCMR1 register. Write CC2P=1 in TIMx_CCER register to validate the polarity (and detect low level only).
- Configure the timer in trigger mode by writing SMS=110 in TIMx_SMCR register. Select TI2 as the input source by writing TS=110 in TIMx_SMCR register.

When a rising edge occurs on TI2, the counter starts counting on the internal clock and the TIF flag is set.

The delay between the rising edge on TI2 and the actual start of the counter is due to the resynchronization circuit on TI2 input.

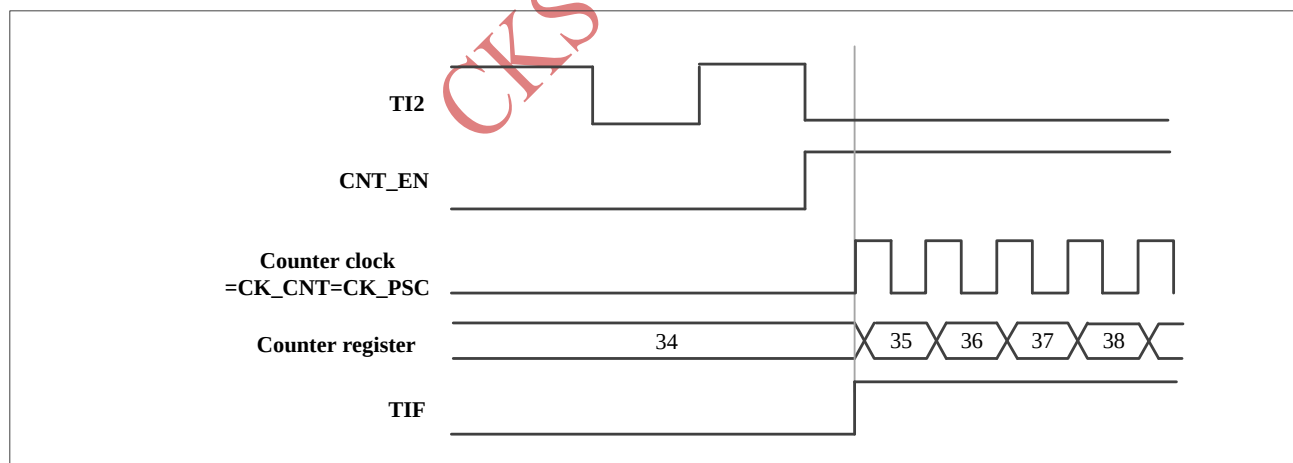


Figure 123 Control circuit in gated mode

Slave mode: External Clock mode 2 + trigger mode

The external clock mode 2 can be used in addition to another slave mode (except external clock mode 1 and encoder mode). In this case, the ETR signal is used as external clock input, and another input can be selected as trigger input when operating in reset mode, gated mode or trigger mode. It is recommended not to select ETR as TRGI through the TS bits of TIMx_SMCR register.

In the following example, the upcounter is incremented at each rising edge of the ETR signal as soon as a rising edge of TI1 occurs:

1. Configure the external trigger input circuit by programming the TIMx_SMCR register as follows:

- ETF = 0000: no filter
 - ETPS = 00: prescaler disabled
 - ETP = 0: detection of rising edges on ETR and ECE=1 to enable the external clock mode 2.
2. Configure the channel 1 as follows, to detect rising edges on TI:
 - IC1F = 0000: no filter.
 - The capture prescaler is not used for triggering and does not need to be configured.
 - CC1S = 01 in TIMx_CCMR1 register to select only the input capture source
 - CC1P = 0 in TIMx_CCER register to validate the polarity (and detect rising edge only).
 3. Configure the timer in trigger mode by writing SMS=110 in TIMx_SMCR register. Select TI1 as the input source by writing TS=101 in TIMx_SMCR register.

A rising edge on TI1 enables the counter and sets the TIF flag. The counter then counts on ETR rising edges.

The delay between the rising edge of the ETR signal and the actual reset of the counter is due to the resynchronization circuit on ETRP input.

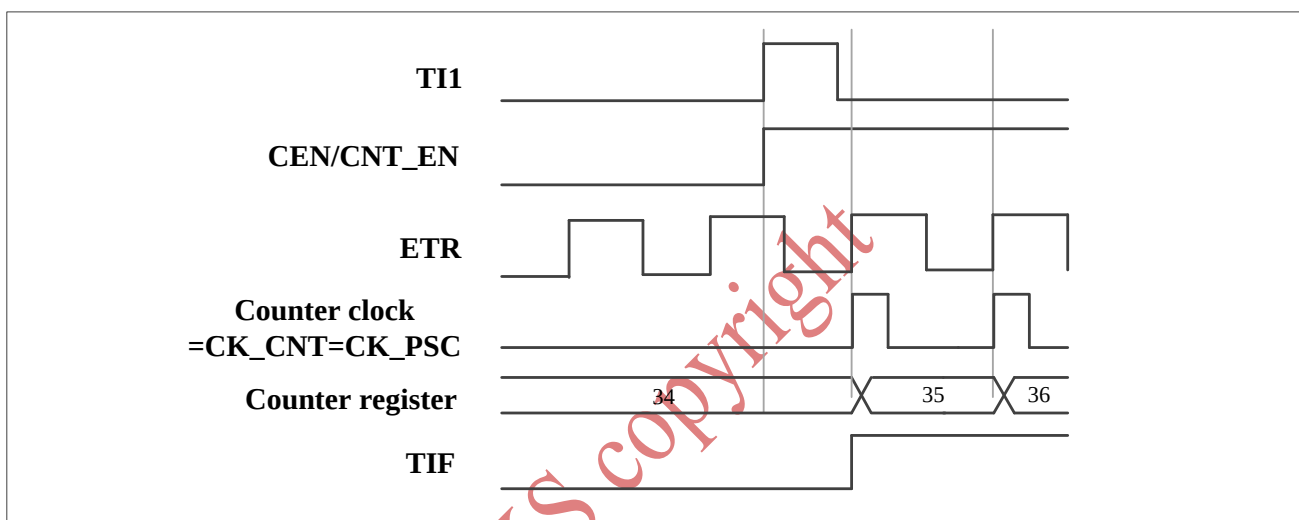


Figure 124 Control circuit in trigger mode

12.3.15 Timer synchronization

The TIMx timers are linked together internally for timer synchronization or chaining. When one Timer is configured in Master Mode, it can reset, start, stop or clock the counter of another Timer configured in Slave Mode.

Figure 125 presents an overview of the trigger selection and the master mode selection blocks.

Note: The clock of the slave timer must be enabled prior to receiving events from the master timer, and must not be changed on-the-fly while triggers are received from the master timer.

Using one timer as prescaler for another timer

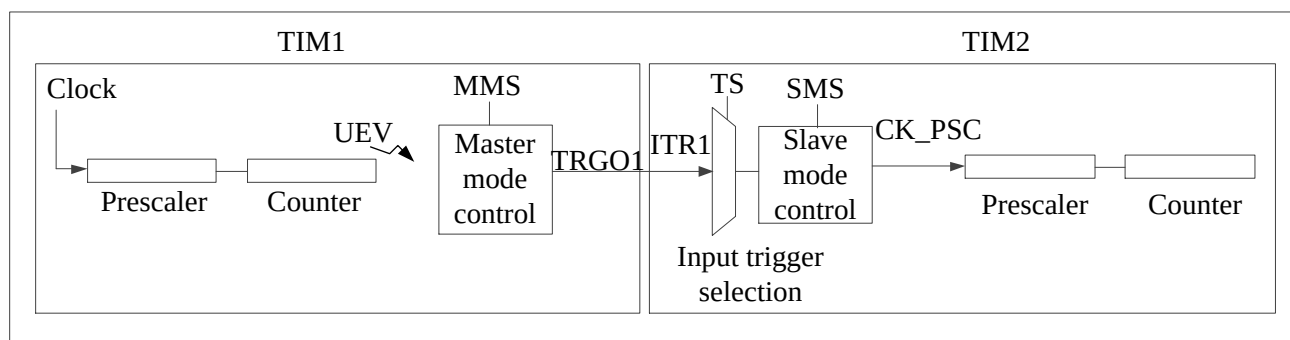


Figure 125 Master/Slave timer example

For example, the user can configure Timer 1 to act as a prescaler for Timer 2 (see Figure 125). To do this:

- Configure Timer 1 in master mode so that it outputs a periodic trigger signal on each update event UEV. If you write MMS=010 in the TIM1_CR2 register, a rising edge is output on TRGO1 each time an update event is generated.
- To connect the TRGO1 output of Timer 1 to Timer 2, Timer 2 must be configured in slave mode using ITR0 as internal trigger. You select this through the TS bits in the TIM2_SMCR register (writing TS=000).
- Then you put the slave mode controller in external clock mode 1 (write SMS=111 in the TIM2_SMCR register). This causes Timer 2 to be clocked by the rising edge of the periodic Timer 1 trigger signal (which correspond to the timer 1 counter overflow).
- Finally both timers must be enabled by setting their respective CEN bits (TIMx_CR1 register).

Note: If OCx is selected on Timer 1 as trigger output (MMS=1xx), its rising edge is used to clock the counter of timer 2.

Using one timer to enable another timer

In this example, we control the enable of Timer 2 with the output compare 1 of Timer 1. Refer to Figure 125 for connections. Timer 2 counts on the divided internal clock only when OC1REF of Timer 1 is high. Both counter clock frequencies are divided by 3 by the prescaler compared to CK_INT ($f_{CK_CNT} = f_{CK_INT}/3$).

- Configure Timer 1 master mode to send its Output Compare 1 Reference (OC1REF) signal as trigger output (MMS=100 in the TIM1_CR2 register).
- Configure the Timer 1 OC1REF waveform (TIM1_CCMR1 register).
- Configure Timer 2 to get the input trigger from Timer 1 (TS=000 in the TIM2_SMCR register).
- Configure Timer 2 in gated mode (SMS=101 in TIM2_SMCR register).
- Enable Timer 2 by writing '1' in the CEN bit (TIM2_CR1 register).
- Start Timer 1 by writing '1' in the CEN bit (TIM1_CR1 register).

Note: The counter 2 clock is not synchronized with counter 1, this mode only affects the Timer 2 counter enable signal.

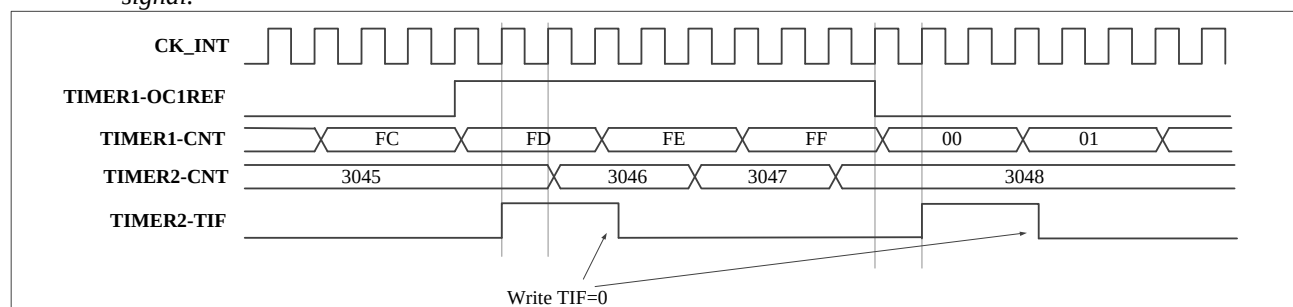


Figure 126 Gating timer 2 with OC1REF of timer 1

In the example in Figure 126, the Timer 2 counter and prescaler are not initialized before being started. So they start counting from their current value. It is possible to start from a given value by resetting both timers before starting Timer 1. You can then write any value you want in the timer counters. The timers can easily be reset by software using the UG bit in the TIMx_EGR registers.

In the next example, we synchronize Timer 1 and Timer 2. Timer 1 is the master and starts from 0. Timer 2 is the slave and starts from 0xE7. The prescaler ratio is the same for both timers. Timer 2 stops when Timer 1 is disabled by writing '0 to the CEN bit in the TIM1_CR1 register:

- Configure Timer 1 master mode to send its Output Compare 1 Reference (OC1REF) signal as trigger output (MMS=100 in the TIM1_CR2 register).
- Configure the Timer 1 OC1REF waveform (TIM1_CCMR1 register).
- Configure Timer 2 to get the input trigger from Timer 1 (TS=000 in the TIM2_SMCR register).
- Configure Timer 2 in gated mode (SMS=101 in TIM2_SMCR register).
- Reset Timer 1 by writing '1 in UG bit (TIM1_EGR register).
- Reset Timer 2 by writing '1 in UG bit (TIM2_EGR register).
- Initialize Timer 2 to 0xE7 by writing '0xE7' in the timer 2 counter (TIM2_CNT).
- Enable Timer 2 by writing '1 in the CEN bit (TIM2_CR1 register).
- Start Timer 1 by writing '1 in the CEN bit (TIM1_CR1 register).
- Stop Timer 1 by writing '0 in the CEN bit (TIM1_CR1 register).

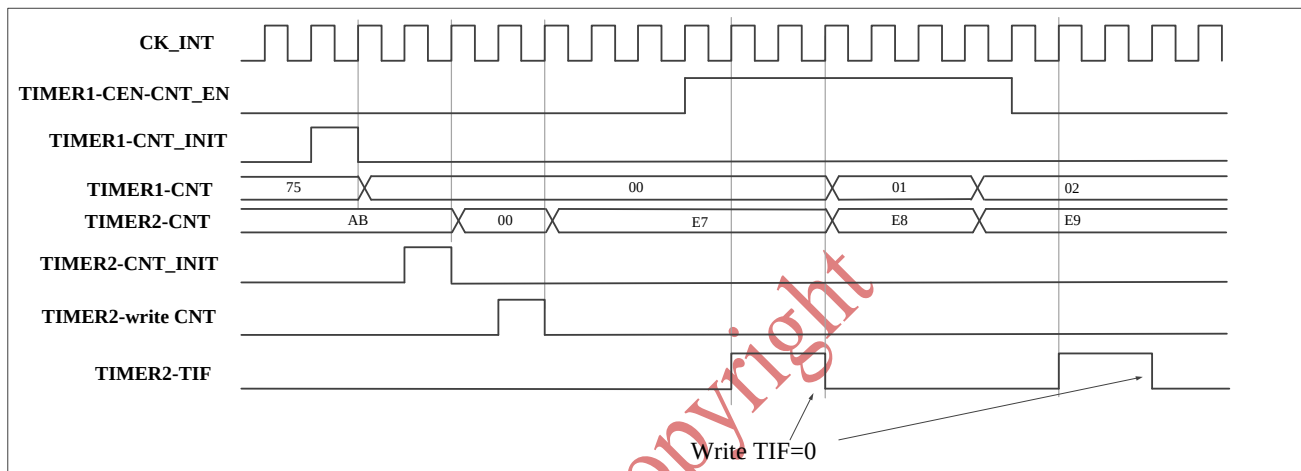


Figure 127 Gating timer 2 with Enable of timer 1

Using one timer to start another timer

In this example, we set the enable of Timer 2 with the update event of Timer 1. Refer to Figure 125 for connections. Timer 2 starts counting from its current value (which can be nonzero) on the divided internal clock as soon as the update event is generated by Timer 1. When Timer 2 receives the trigger signal its CEN bit is automatically set and the counter counts until we write '0 to the CEN bit in the TIM2_CR1 register. Both counter clock frequencies are divided by 3 by the prescaler compared to CK_INT ($f_{CK_CNT} = f_{CK_INT}/3$).

- Configure Timer 1 master mode to send its Update Event (UEV) as trigger output (MMS=010 in the TIM1_CR2 register).
- Configure the Timer 1 period (TIM1_ARR registers).
- Configure Timer 2 to get the input trigger from Timer 1 (TS=000 in the TIM2_SMCR register).
- Configure Timer 2 in trigger mode (SMS=110 in TIM2_SMCR register).
- Start Timer 1 by writing '1 in the CEN bit (TIM1_CR1 register).

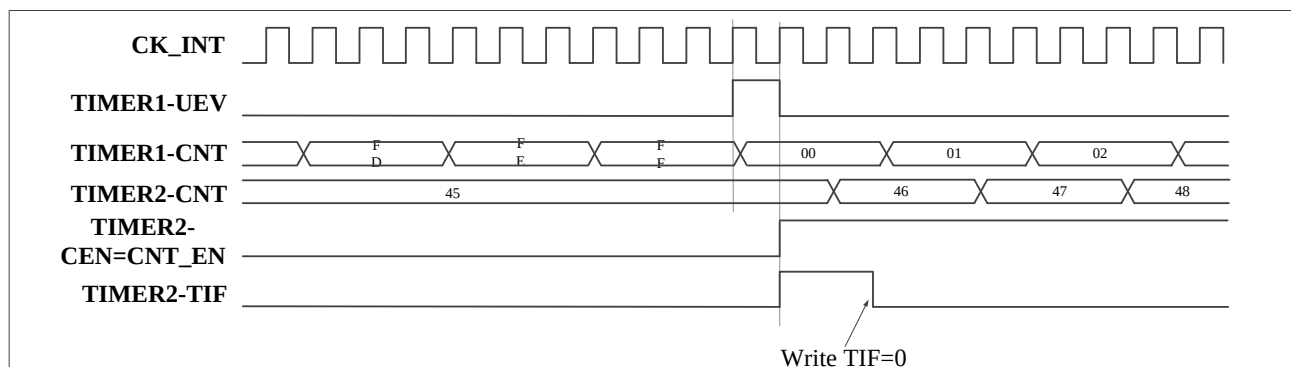


Figure 128 Triggering timer 2 with update of timer 1

As in the previous example, the user can initialize both counters before starting counting. Figure 129 shows the behavior with the same configuration as in Figure 128 but in trigger mode instead of gated mode (SMS=110 in the TIM2_SMCR register)

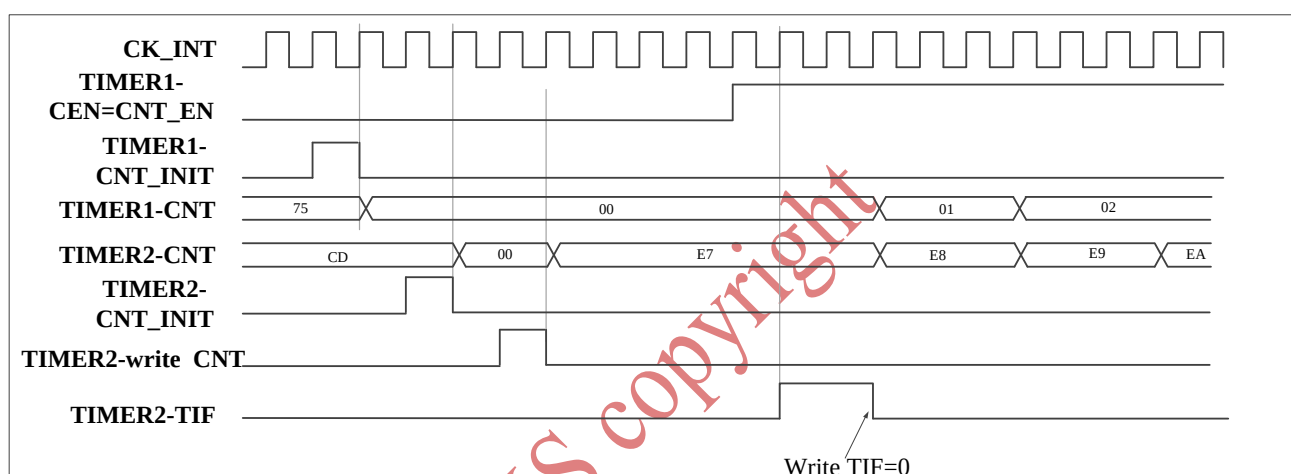


Figure 129 Triggering timer 2 with Enable of timer 1

Starting 2 timers synchronously in response to an external trigger

In this example, we set the enable of timer 1 when its TI1 input rises, and the enable of Timer 2 with the enable of Timer 1. Refer to Figure 125 for connections. To ensure the counters are aligned, Timer 1 must be configured in Master/Slave mode (slave with respect to TI1, master with respect to Timer 2):

- Configure Timer 1 master mode to send its Enable as trigger output (MMS=001 in the TIM1_CR2 register).
- Configure Timer 1 slave mode to get the input trigger from TI1 (TS=100 in the TIM1_SMCR register).
- Configure Timer 1 in trigger mode (SMS=110 in the TIM1_SMCR register).
- Configure the Timer 1 in Master/Slave mode by writing MSM=1 (TIM1_SMCR register).
- Configure Timer 2 to get the input trigger from Timer 1 (TS=000 in the TIM2_SMCR register).
- Configure Timer 2 in trigger mode (SMS=110 in the TIM2_SMCR register).

When a rising edge occurs on TI1 (Timer 1), both counters start counting synchronously on the internal clock and both TIF flags are set.

Note:

In this example both timers are initialized before starting (by setting their respective UG bits). Both counters start from 0, but you can easily insert an offset between them by writing any of the counter registers (TIMx_CNT). You can see that the master/slave mode insert a delay between CNT_EN and CK_PSC on timer 1.

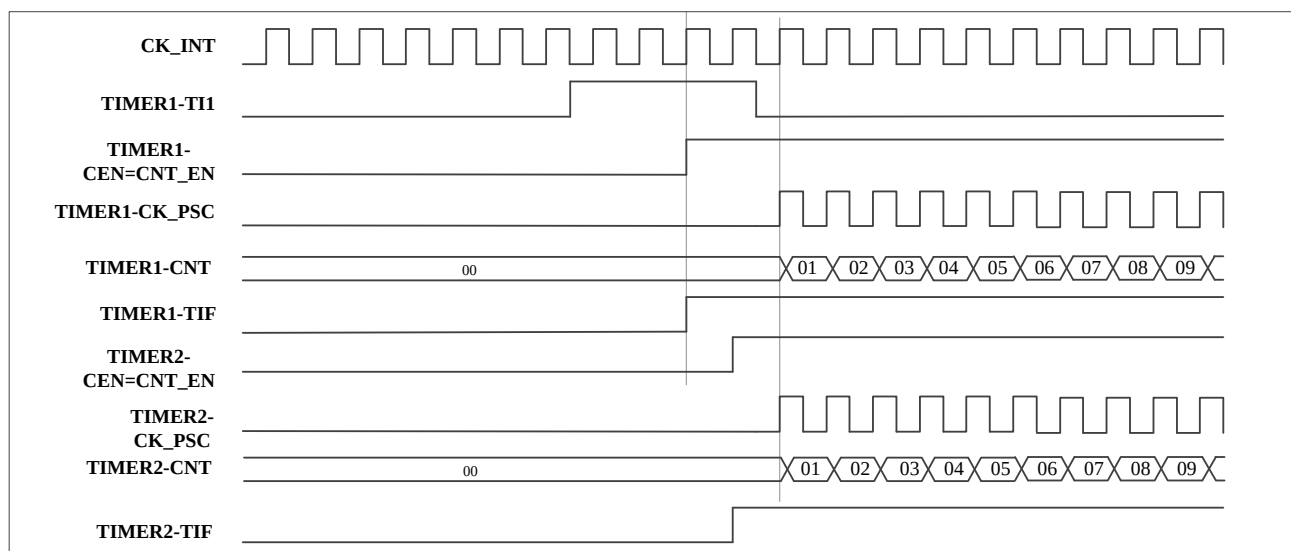


Figure 130 Triggering timer 1 and 2 with timer 1 TI1 input

12.3.16 Debug mode

When the microcontroller enters debug mode (Cortex[®]-M3 core - halted), the TIMx counter either continues to work normally or stops, depending on DBG_TIMx_STOP configuration bit in DBGMCU module. For more details, refer to *Section 22.16.2: Debug support for timers, watchdog, bxCAN and I2C*.

12.4 TIMx registers

Refer to *Section 1.1* for a list of abbreviations used in register descriptions.

The 32-bit peripheral registers have to be written by words (32 bits). All other peripheral registers have to be written by half-words (16 bits) or words (32 bits). Read accesses can be done by bytes (8 bits), half-words (16 bits) or words (32 bits).

12.4.1 TIMx control register 1 (TIMx_CR1)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CKD[1:0]		ARPE	CMS[1:0]		DIR	OPM	URS	UDIS	CEN
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 15: 0		Reserved, must be kept at reset value.													
Bits 9:8		CKD : Clock division This bit-field indicates the division ratio between the timer clock (CK_INT) frequency and sampling clock used by the digital filters (ETR, TIx), 00: $t_{DTS} = t_{CK_INT}$ 01: $t_{DTS} = 2 \times t_{CK_INT}$ 10: $t_{DTS} = 4 \times t_{CK_INT}$ 11: Reserved													
Bit 7		ARPE : Auto-reload preload enable 0: TIMx_ARR register is not buffered 1: TIMx_ARR register is buffered													
Bits 6:5		CMS : Center-aligned mode selection 00: Edge-aligned mode. The counter counts up or down depending on the direction bit (DIR).													

	01: Center-aligned mode 1. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set only when the counter is counting down. 10: Center-aligned mode 2. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set only when the counter is counting up. 11: Center-aligned mode 3. The counter counts up and down alternatively. Output compare interrupt flags of channels configured in output (CCxS=00 in TIMx_CCMRx register) are set both when the counter is counting up or down. <i>Note: It is not allowed to switch from edge-aligned mode to center-aligned mode as long as the counter is enabled (CEN=1)</i>
Bit 4	DIR: Direction 0: Counter used as upcounter 1: Counter used as downcounter <i>Note: This bit is read only when the timer is configured in Center-aligned mode or Encoder mode.</i>
Bit 3	OPM: One-pulse mode 0: Counter is not stopped at update event 1: Counter stops counting at the next update event (clearing the bit CEN)
Bit 2	URS: Update request source This bit is set and cleared by software to select the UEV event sources. 0: Any of the following events generate an update interrupt or DMA request if enabled. These events can be: - Counter overflow/underflow - Setting the UG bit - Update generation through the slave mode controller 1: Only counter overflow/underflow generates an update interrupt or DMA request if enabled.
Bit 1	UDIS: Update disable This bit is set and cleared by software to enable/disable UEV event generation. 0: UEV enabled. The Update (UEV) event is generated by one of the following events: - Counter overflow/underflow - Setting the UG bit - Update generation through the slave mode controller Buffered registers are then loaded with their preload values. 1: UEV disabled. The Update event is not generated, shadow registers keep their value (ARR, PSC, CCRx). However the counter and the prescaler are reinitialized if the UG bit is set or if a hardware reset is received from the slave mode controller.
Bit 0	CEN: Counter enable 0: Counter disabled 1: Counter enabled <i>Note: External clock, gated mode and encoder mode can work only if the CEN bit has been previously set by software. However trigger mode can set the CEN bit automatically by hardware.</i> CEN is cleared automatically in one-pulse mode, when an update event occurs.

12.4.2 TIMx control register 2 (TIMx_CR2)

Address offset: 0x04

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								TI1S	MMS[2:0]		CCDS	Reserved			
								rw	rw	rw	rw	rw			

Bits 15:8	Reserved, must be kept at reset value.
Bit 7	TI1S: TI1 selection 0: The TIMx_CH1 pin is connected to TI1 input 1: The TIMx_CH1, CH2 and CH3 pins are connected to the TI1 input (XOR combination) See also <i>Section 12.3.18: Interfacing with Hall sensors</i>
Bits 6:4	MMS[2:0]: Master mode selection These bits allow to select the information to be sent in master mode to slave timers for synchronization (TRGO). The combination is as follows: 000: Reset - the UG bit from the TIMx_EGR register is used as trigger output (TRGO). If the reset is generated by the trigger input (slave mode controller configured in reset mode) then the signal on TRGO is delayed compared to the actual reset. 001: Enable - the Counter enable signal, CNT_EN, is used as trigger output (TRGO). It is useful to start several timers at the same time or to control a window in which a slave timer is enabled. The Counter

	Enable signal is generated by a logic OR between CEN control bit and the trigger input when configured in gated mode. When the Counter Enable signal is controlled by the trigger input, there is a delay on TRGO, except if the master/slave mode is selected (see the MSM bit description in TIMx_SMCR register). 010: Update - The update event is selected as trigger output (TRGO). For instance a master timer can then be used as a prescaler for a slave timer. 011: Compare Pulse - The trigger output send a positive pulse when the CC1IF flag is to be set (even if it was already high), as soon as a capture or a compare match occurred. (TRGO) 100: Compare - OC1REF signal is used as trigger output (TRGO) 101: Compare - OC2REF signal is used as trigger output (TRGO) 110: Compare - OC3REF signal is used as trigger output (TRGO) 111: Compare - OC4REF signal is used as trigger output (TRGO)
Bit 3	CCDS: Capture/compare DMA selection 0: CCx DMA request sent when CCx event occurs 1: CCx DMA requests sent when update event occurs
Bits 2:0	Reserved, must be kept at reset value.

12.4.3 TIMx slave mode control register (TIMx_SMCR)

Address offset: 0x08

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETP	ECE	ETPS[1:0]	ETF[3:0]			MSM		TS[2:0]			Res		SMS[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit 15	ETP: External trigger polarity This bit selects whether ETR or $\overline{\text{ETR}}$ is used for trigger operations 0: ETR is non-inverted, active at high level or rising edge 1: ETR is inverted, active at low level or falling edge														
Bit 14	ECE: External clock enable This bit enables External clock mode 2. 0: External clock mode 2 disabled 1: External clock mode 2 enabled. The counter is clocked by any active edge on the ETRF signal. Note 1: Setting the ECE bit has the same effect as selecting external clock mode 1 with TRGI connected to ETRF (SMS=111 and TS=111). Note 2: It is possible to simultaneously use external clock mode 2 with the following slave modes: reset mode, gated mode and trigger mode. Nevertheless, TRGI must not be connected to ETRF in this case (TS bits must not be 111). Note 3: If external clock mode 1 and external clock mode 2 are enabled at the same time, the external clock input is ETRF.														
Bits 13:12	ETPS: External trigger prescaler External trigger signal ETRP frequency must be at most 1/4 of CK_INT frequency. A prescaler can be enabled to reduce ETRP frequency. It is useful when inputting fast external clocks. 00: Prescaler OFF 01: ETRP frequency divided by 2 10: ETRP frequency divided by 4 11: ETRP frequency divided by 8														
Bits 11:8	ETF[3:0]: External trigger filter This bit-field then defines the frequency used to sample ETRP signal and the length of the digital filter applied to ETRP. The digital filter is made of an event counter in which N consecutive events are needed to validate a transition on the output: 0000: No filter, sampling is done at f_{DTS} 0001: $f_{\text{SAMPLING}} = f_{\text{CK_INT}}$, N=2 0010: $f_{\text{SAMPLING}} = f_{\text{CK_INT}}$, N=4 0011: $f_{\text{SAMPLING}} = f_{\text{CK_INT}}$, N=8 0100: $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$, N=6 0101: $f_{\text{SAMPLING}} = f_{\text{DTS}}/2$, N=8 0110: $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$, N=6 0111: $f_{\text{SAMPLING}} = f_{\text{DTS}}/4$, N=8 1000: $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$, N=6 1001: $f_{\text{SAMPLING}} = f_{\text{DTS}}/8$, N=8 1010: $f_{\text{SAMPLING}} = f_{\text{DTS}}/16$, N=5														

	1011: $f_{\text{SAMPLING}} = f_{\text{DTS}}/16, N=6$ 1100: $f_{\text{SAMPLING}} = f_{\text{DTS}}/16, N=8$ 1101: $f_{\text{SAMPLING}} = f_{\text{DTS}}/32, N=5$ 1110: $f_{\text{SAMPLING}} = f_{\text{DTS}}/32, N=6$ 1111: $f_{\text{SAMPLING}} = f_{\text{DTS}}/32, N=8$
Bit 7	MSM: Master/Slave mode 0: No action 1: The effect of an event on the trigger input (TRGI) is delayed to allow a perfect synchronization between the current timer and its slaves (through TRGO). It is useful if we want to synchronize several timers on a single external event.
Bits 6:4	TS: Trigger selection This bit-field selects the trigger input to be used to synchronize the counter. 000: Internal Trigger 0 (ITR0). 001: Internal Trigger 1 (ITR1). 010: Internal Trigger 2 (ITR2). 011: Internal Trigger 3 (ITR3). 100: TI1 Edge Detector (TI1F_ED) 101: Filtered Timer Input 1 (TI1FP1) 110: Filtered Timer Input 2 (TI2FP2) 111: External Trigger input (ETRF) See Table 57 for more details on ITRx meaning for each Timer. <i>Note: These bits must be changed only when they are not used (e.g. when SMS=000) to avoid wrong edge detections at the transition.</i>
Bit 3	Reserved, must be kept at reset value.
Bits 2:0	SMS: Slave mode selection When external signals are selected the active edge of the trigger signal (TRGI) is linked to the polarity selected on the external input (see Input Control register and Control Register description). 000: Slave mode disabled - if CEN = '1 then the prescaler is clocked directly by the internal clock. 001: Encoder mode 1 - Counter counts up/down on TI2FP1 edge depending on TI1FP2 level. 010: Encoder mode 2 - Counter counts up/down on TI1FP2 edge depending on TI2FP1 level. 011: Encoder mode 3 - Counter counts up/down on both TI1FP1 and TI2FP2 edges depending on the level of the other input. 100: Reset Mode - Rising edge of the selected trigger input (TRGI) reinitializes the counter and generates an update of the registers. 101: Gated Mode - The counter clock is enabled when the trigger input (TRGI) is high. The counter stops (but is not reset) as soon as the trigger becomes low. Both start and stop of the counter are controlled. 110: Trigger Mode - The counter starts at a rising edge of the trigger TRGI (but it is not reset). Only the start of the counter is controlled. 111: External Clock Mode 1 - Rising edges of the selected trigger (TRGI) clock the counter. <i>Note: The gated mode must not be used if TI1F_ED is selected as the trigger input (TS=100). Indeed, TI1F_ED outputs 1 pulse for each transition on TI1F, whereas the gated mode checks the level of the trigger signal.</i>

Table 57 TIMx Internal trigger connection

Slave TIM	ITR0 (TS = 000)	ITR1 (TS = 001)	ITR2 (TS = 010)	ITR3 (TS = 011)
TIM2	TIM1	-	TIM3	TIM4
TIM3	TIM1	TIM2	-	TIM4
TIM4	TIM1	TIM2	TIM3	-

12.4.4 TIMx DMA/Interrupt enable register (TIMx_DIER)

Address offset: 0x0C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	TDE	Res	CC4DE	CC3DE	CC2DE	CC1DE	UDE	Res	TIE	Res	CC4IE	CC3IE	CC2IE	CC1IE	UIE
	rw		rw	rw	rw	rw	rw		rw		rw	rw	rw	rw	rw
Bit 15		Reserved, must be kept at reset value.													
Bit 14		TDE: Trigger DMA request enable 0: Trigger DMA request disabled. 1: Trigger DMA request enabled.													

Bit 13	Reserved, must be kept at reset value.
Bit 12	CC4DE : Capture/Compare 4 DMA request enable 0: CC4 DMA request disabled. 1: CC4 DMA request enabled.
Bit 11	CC3DE : Capture/Compare 3 DMA request enable 0: CC3 DMA request disabled. 1: CC3 DMA request enabled.
Bit 10	CC2DE : Capture/Compare 2 DMA request enable 0: CC2 DMA request disabled. 1: CC2 DMA request enabled.
Bit 9	CC1DE : Capture/Compare 1 DMA request enable 0: CC1 DMA request disabled. 1: CC1 DMA request enabled.
Bit 8	UDE : Update DMA request enable 0: Update DMA request disabled. 1: Update DMA request enabled.
Bit 7	Reserved, must be kept at reset value.
Bit 6	TIE : Trigger interrupt enable 0: Trigger interrupt disabled. 1: Trigger interrupt enabled.
Bit 5	Reserved, must be kept at reset value.
Bit 4	CC4IE : Capture/Compare 4 interrupt enable 0: CC4 interrupt disabled. 1: CC4 interrupt enabled.
Bit 3	CC3IE : Capture/Compare 3 interrupt enable 0: CC3 interrupt disabled. 1: CC3 interrupt enabled.
Bit 2	CC2IE : Capture/Compare 2 interrupt enable 0: CC2 interrupt disabled. 1: CC2 interrupt enabled.
Bit 1	CC1IE : Capture/Compare 1 interrupt enable 0: CC1 interrupt disabled. 1: CC1 interrupt enabled.
Bit 0	UIE : Update interrupt enable 0: Update interrupt disabled. 1: Update interrupt enabled.

12.4.5 TIMx status register (TIMx_SR)

Address offset: 0x10

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		CC40F	CC30F	CC20F	CC10F	Res	TIF		Res	CC4IF	CC3IF	CC2IF	CC1IF	UIF	
rc w0		rc w0	rc w0	rc w0	rc w0					rc w0		rc w0	rc w0	rc w0	rc w0
Bits15: 13		Reserved, must be kept at reset value.													
Bit 12		CC40F: Capture/Compare 4 overcapture flag refer to CC10F description													
Bit 11		CC30F: Capture/Compare 3 overcapture flag refer to CC10F description													
Bit 10		CC20F: Capture/compare 2 overcapture flag refer to CC10F description													
Bit 9		CC10F: Capture/Compare 1 overcapture flag This flag is set by hardware only when the corresponding channel is configured in input capture mode. It is cleared by software by writing it to ‘0’. 0: No overcapture has been detected. 1: The counter value has been captured in TIMx_CCR1 register while CC1IF flag was already set													
Bits 8:7		Reserved, must be kept at reset value.													
Bit 6		TIF: Trigger interrupt flag This flag is set by hardware on trigger event (active edge detected on TRGI input when the slave mode controller is enabled in all modes but gated mode, both edges in case gated mode is selected). It is cleared by software. 0: No trigger event occurred. 1: Trigger interrupt pending.													
Bit 5		Reserved, must be kept at reset value.													
Bit 4		CC4IF: Capture/Compare 4 interrupt flag refer to CC1IF description													

Bit 3	CC3IF : Capture/Compare 3 interrupt flag refer to CC1IF description
Bit 2	CC2IF : Capture/Compare 2 interrupt flag refer to CC1IF description
Bit 1	CC1IF : Capture/compare 1 interrupt flag If channel CC1 is configured as output: This flag is set by hardware when the counter matches the compare value, with some exception in center-aligned mode (refer to the CMS bits in the TIMx_CR1 register description). It is cleared by software. 0: No match. 1: The content of the counter TIMx_CNT has matched the content of the TIMx_CCR1 register. If channel CC1 is configured as input: This bit is set by hardware on a capture. It is cleared by software or by reading the TIMx_CCR1 register. 0: No input capture occurred. 1: The counter value has been captured in TIMx_CCR1 register (An edge has been detected on IC1 which matches the selected polarity).
Bit 0	UIF : Update interrupt flag This bit is set by hardware on an update event. It is cleared by software. 0: No update occurred. 1: Update interrupt pending. This bit is set by hardware when the registers are updated: At overflow or underflow and if the UDIS=0 in the TIMx_CR1 register. - When CNT is reinitialized by software using the UG bit in TIMx_EGR register, if URS=0 and UDIS=0 in the TIMx_CR1 register. - When CNT is reinitialized by a trigger event (refer to the synchro control register description), if URS=0 and UDIS=0 in the TIMx_CR1 register.

12.4.6 TIMx event generation register (TIMx_EGR)

Address offset: 0x14

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved									TG	Res	CC4G	CC3G	CC2G	CC1G	UG
									w		w	w	w	w	w
Bits 15:7		Reserved, must be kept at reset value.													
Bit 6		TG : Trigger generation This bit is set by software in order to generate an event, it is automatically cleared by hardware. 0: No action 1: The TIF flag is set in TIMx_SR register. Related interrupt or DMA transfer can occur if enabled.													
Bit 5		Reserved, must be kept at reset value.													
Bit 4		CC4G : Capture/compare 4 generation refer to CC1G description													
Bit 3		CC3G : Capture/compare 3 generation refer to CC1G description													
Bit 2		CC2G : Capture/compare 2 generation refer to CC1G description													
Bit 1		CC1G : Capture/compare 1 generation This bit is set by software in order to generate an event, it is automatically cleared by hardware. 0: No action 1: A capture/compare event is generated on channel 1: If channel CC1 is configured as output: CC1IF flag is set, Corresponding interrupt or DMA request is sent if enabled. If channel CC1 is configured as input: The current value of the counter is captured in TIMx_CCR1 register. The CC1IF flag is set, the corresponding interrupt or DMA request is sent if enabled. The CC1OF flag is set if the CC1IF flag was already high.													
Bit 0		UG : Update generation This bit can be set by software, it is automatically cleared by hardware. 0: No action 1: Re-initialize the counter and generates an update of the registers. Note that the prescaler counter is cleared too (anyway the prescaler ratio is not affected). The counter is cleared if the center-aligned mode is selected or if DIR=0 (upcounting), else it takes the auto-reload value (TIMx_ARR) if DIR=1 (downcounting).													

12.4.7 TIMx capture/compare mode register 1 (TIMx_CCMR1)

Address offset: 0x18

Reset value: 0x0000

The channels can be used in input (capture mode) or in output (compare mode). The direction of a channel is defined by configuring the corresponding CCxS bits. All the other bits of this register have a different function in input and in output mode. For a given bit, OCxx describes its function when the channel is configured in output, ICxx describes its function when the channel is configured in input. Take care that the same bit can have a different meaning for the input stage and for the output stage.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2CE	OC2M[2: 0]			OC2PE	OC2FE	CC2S		OC1CE	OC1M[2: 0]			OC1PE	OC1FE	CC1S	
IC2F[3: 0]				IC2PSC[1: 0]		[1: 0]		IC1F[3: 0]				IC1PSC[1: 0]		[1: 0]	
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW

Output compare mode

Bit 15	OC2CE: Output compare 2 clear enable
Bits 14:12	OC2M[2:0]: Output compare 2 mode
Bit 11	OC2PE: Output compare 2 preload enable
Bit 10	OC2FE: Output compare 2 fast enable
Bits 9:8	CC2S[1:0]: Capture/Compare 2 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC2 channel is configured as output 01: CC2 channel is configured as input, IC2 is mapped on TI2 10: CC2 channel is configured as input, IC2 is mapped on TI1 11: CC2 channel is configured as input, IC2 is mapped on TRC. This mode is working only if an internal trigger input is selected through the TS bit (TIMx_SMCR register) <i>Note: CC2S bits are writable only when the channel is OFF (CC2E = 0 in TIMx_CCER).</i>
Bit 7	OC1CE: Output compare 1 clear enable OC1CE: Output Compare 1 Clear Enable 0: OC1Ref is not affected by the ETRF input 1: OC1Ref is cleared as soon as a High level is detected on ETRF input
Bits 6:4	OC1M: Output compare 1 mode These bits define the behavior of the output reference signal OC1REF from which OC1 and OC1N are derived. OC1REF is active high whereas OC1 and OC1N active level depends on CC1P and CC1NP bits. 000: Frozen - The comparison between the output compare register TIMx_CCR1 and the counter TIMx_CNT has no effect on the outputs.(this mode is used to generate a timing base). 001: Set channel 1 to active level on match. OC1REF signal is forced high when the counter TIMx_CNT matches the capture/compare register 1 (TIMx_CCR1). 010: Set channel 1 to inactive level on match. OC1REF signal is forced low when the counter TIMx_CNT matches the capture/compare register 1 (TIMx_CCR1). 011: Toggle - OC1REF toggles when TIMx_CNT=TIMx_CCR1. 100: Force inactive level - OC1REF is forced low. 101: Force active level - OC1REF is forced high. 110: PWM mode 1 - In upcounting, channel 1 is active as long as TIMx_CNT<TIMx_CCR1 else inactive. In downcounting, channel 1 is inactive (OC1REF=0) as long as TIMx_CNT>TIMx_CCR1 else active (OC1REF=1). 111: PWM mode 2 - In upcounting, channel 1 is inactive as long as TIMx_CNT<TIMx_CCR1 else active. In downcounting, channel 1 is active as long as TIMx_CNT>TIMx_CCR1 else inactive. <i>Note: In PWM mode 1 or 2, the OCREF level changes only when the result of the comparison changes or when the output compare mode switches from "frozen" mode to "PWM" mode.</i>
Bit 3	OC1PE: Output compare 1 preload enable 0: Preload register on TIMx_CCR1 disabled. TIMx_CCR1 can be written at anytime, the new value is taken in account immediately. 1: Preload register on TIMx_CCR1 enabled. Read/Write operations access the preload register. TIMx_CCR1 preload value is loaded in the active register at each update event. <i>Note: 1: These bits can not be modified as long as LOCK level 3 has been programmed (LOCK bits in TIMx_BDTR register) and CC1S=00 (the channel is configured in output). 2: The PWM mode can be used without validating the preload register only in one- pulse mode (OPM bit set in TIMx_CR1 register). Else the behavior is not guaranteed.</i>
Bit 2	OC1FE: Output compare 1 fast enable This bit is used to accelerate the effect of an event on the trigger in input on the CC output. 0: CC1 behaves normally depending on counter and CCR1 values even when the trigger is ON. The minimum

	delay to activate CC1 output when an edge occurs on the trigger input is 5 clock cycles. 1: An active edge on the trigger input acts like a compare match on CC1 output. Then, OC is set to the compare level independently from the result of the comparison. Delay to sample the trigger input and to activate CC1 output is reduced to 3 clock cycles. OCFE acts only if the channel is configured in PWM1 or PWM2 mode.
Bits 1:0	CC1S: Capture/Compare 1 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC1 channel is configured as output. 01: CC1 channel is configured as input, IC1 is mapped on TI1. 10: CC1 channel is configured as input, IC1 is mapped on TI2. 11: CC1 channel is configured as input, IC1 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register) <i>Note: CC1S bits are writable only when the channel is OFF (CC1E = 0 in TIMx_CCER).</i>

Input capture mode

Bits 15:12	IC2F : Input capture 2 filter
Bits 11:10	IC2PSC[1:0] : Input capture 2 prescaler
Bits 9:8	CC2S: Capture/compare 2 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC2 channel is configured as output. 01: CC2 channel is configured as input, IC2 is mapped on TI2. 10: CC2 channel is configured as input, IC2 is mapped on TI1 11: CC2 channel is configured as input, IC2 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register) <i>Note: CC2S bits are writable only when the channel is OFF (CC2E = 0 in TIMx_CCER).</i>
Bits 7:4	IC1F: Input capture 1 filter This bit-field defines the frequency used to sample TI1 input and the length of the digital filter applied to TI1. The digital filter is made of an event counter in which N consecutive events are needed to validate a transition on the output: 0000: No filter, sampling is done at f_{DTS} 0001: $f_{SAMPLING} = f_{CK_INT}$, N=2 0010: $f_{SAMPLING} = f_{CK_INT}$, N=4 0011: $f_{SAMPLING} = f_{CK_INT}$, N=8 0100: $f_{SAMPLING} = f_{DTS}/2$, N=6 0101: $f_{SAMPLING} = f_{DTS}/2$, N=8 0110: $f_{SAMPLING} = f_{DTS}/4$, N=6 0111: $f_{SAMPLING} = f_{DTS}/4$, N=8 1000: $f_{SAMPLING} = f_{DTS}/8$, N=6 1001: $f_{SAMPLING} = f_{DTS}/8$, N=8 1010: $f_{SAMPLING} = f_{DTS}/16$, N=5 1011: $f_{SAMPLING} = f_{DTS}/16$, N=6 1100: $f_{SAMPLING} = f_{DTS}/16$, N=8 1101: $f_{SAMPLING} = f_{DTS}/32$, N=5 1110: $f_{SAMPLING} = f_{DTS}/32$, N=6 1111: $f_{SAMPLING} = f_{DTS}/32$, N=8
Bits 3:2	IC1PSC: Input capture 1 prescaler This bit-field defines the ratio of the prescaler acting on CC1 input (IC1). The prescaler is reset as soon as CC1E=0 (TIMx_CCER register). 00: no prescaler, capture is done each time an edge is detected on the capture input 01: capture is done once every 2 events 10: capture is done once every 4 events 11: capture is done once every 8 events
Bits 1:0	CC1S: Capture/Compare 1 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC1 channel is configured as output 01: CC1 channel is configured as input, IC1 is mapped on TI1 10: CC1 channel is configured as input, IC1 is mapped on TI2 11: CC1 channel is configured as input, IC1 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register) <i>Note: CC1S bits are writable only when the channel is OFF (CC1E = 0 in TIMx_CCER).</i>

12.4.8 TIMx capture/compare mode register 2 (TIMx_CCMR2)

Address offset: 0x1C

Reset value: 0x0000

Refer to the above CCMR1 register description.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC4CE	OC4M[2: 0]			OC4PE	OC4FE	CC4S		OC3CE	OC3M[2: 0]			OC3PE	OC3FE	CC3S	
IC4F[3: 0]				IC4PSC[1: 0]			[1: 0]	IC3F[3: 0]				IC3PSC[1: 0]			[1: 0]
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Output compare mode

Bit 15	OC4CE : Output compare 4 clear enable
Bits14:12	OC4M : Output compare 4 mode
Bit 11	OC4PE : Output compare 4 preload enable
Bit 10	OC4FE : Output compare 4 fast enable
Bits 9:8	CC4S : Capture/Compare 4 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC4 channel is configured as output 01: CC4 channel is configured as input, IC4 is mapped on TI4 10: CC4 channel is configured as input, IC4 is mapped on TI3 11: CC4 channel is configured as input, IC4 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register) <i>Note: CC4S bits are writable only when the channel is OFF (CC4E = 0 in TIMx_CCER).</i>
Bit 7	OC3CE : Output compare 3 clear enable
Bits 6:4	OC3M : Output compare 3 mode
Bit 3	OC3PE : Output compare 3 preload enable
Bit 2	OC3FE : Output compare 3 fast enable
Bits 1:0	CC3S : Capture/Compare 3 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC3 channel is configured as output 01: CC3 channel is configured as input, IC3 is mapped on TI3 10: CC3 channel is configured as input, IC3 is mapped on TI4 11: CC3 channel is configured as input, IC3 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register) <i>Note: CC3S bits are writable only when the channel is OFF (CC3E = 0 in TIMx_CCER).</i>

Input capture mode

Bits 15:12	IC4F : Input capture 4 filter
Bits 11:10	IC4PSC : Input capture 4 prescaler
Bits 9:8	CC4S : Capture/Compare 4 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC4 channel is configured as output 01: CC4 channel is configured as input, IC4 is mapped on TI4 10: CC4 channel is configured as input, IC4 is mapped on TI3 11: CC4 channel is configured as input, IC4 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register) <i>Note: CC4S bits are writable only when the channel is OFF (CC4E = 0 in TIMx_CCER).</i>
Bits 7:4	IC3F : Input capture 3 filter
Bits 3:2	IC3PSC : Input capture 3 prescaler
Bits 1:0	CC3S : Capture/Compare 3 selection This bit-field defines the direction of the channel (input/output) as well as the used input. 00: CC3 channel is configured as output 01: CC3 channel is configured as input, IC3 is mapped on TI3 10: CC3 channel is configured as input, IC3 is mapped on TI4 11: CC3 channel is configured as input, IC3 is mapped on TRC. This mode is working only if an internal trigger input is selected through TS bit (TIMx_SMCR register) <i>Note: CC3S bits are writable only when the channel is OFF (CC3E = 0 in TIMx_CCER).</i>

12.4.9 TIMx capture/compare enable register (TIMx_CCER)

Address offset: 0x20

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	CC4P	CC4E	Res	CC3P	CC3E	Res	CC2P	CC2E	Res	CC1P	CC1E				
rw		rw		rw		rw		rw		rw		rw		rw	
Bits 15:14		Reserved, must be kept at reset value.													
Bit 13		CC4P : Capture/Compare 4 output polarity refer to CC1P description													
Bit 12		CC4E : Capture/Compare 4 output enable refer to CC1E description													
Bits 11:10		Reserved, must be kept at reset value.													
Bit 9		CC3P : Capture/Compare 3 output polarity refer to CC1P description													
Bit 8		CC3E : Capture/Compare 3 output enable refer to CC1E description													
Bits 7:6		Reserved, must be kept at reset value.													
Bit 5		CC2P : Capture/Compare 2 output polarity refer to CC1P description													
Bit 4		CC2E : Capture/Compare 2 output enable refer to CC1E description													
Bits 3:2		Reserved, must be kept at reset value.													
Bit 1		CC1P : Capture/Compare 1 output polarity CC1 channel configured as output: 0: OC1 active high. 1: OC1 active low. CC1 channel configured as input: This bit selects whether IC1 or IC1 is used for trigger or capture operations. 0: non-inverted: capture is done on a rising edge of IC1. When used as external trigger, IC1 is non-inverted. 1: inverted: capture is done on a falling edge of IC1. When used as external trigger, IC1 is inverted.													
Bit 0		CC1E : Capture/Compare 1 output enable CC1 channel configured as output: 0: Off - OC1 is not active. 1: On - OC1 signal is output on the corresponding output pin. CC1 channel configured as input: This bit determines if a capture of the counter value can actually be done into the input capture/compare register 1 (TIMx_CCR1) or not. 0: Capture disabled. 1: Capture enabled.													

Table 58 Output control bit for standard OCx channels

CCxE bit	OCx output state
0	Output Disabled (OCx=0, OCx_EN=0)
1	OCx=OCxREF + Polarity, OCx_EN=1

Note: The state of the external IO pins connected to the standard OCx channels depends on the OCx channel state and the GPIO and AFIO registers.

12.4.10 TIMx counter (TIMx_CNT)

Address offset: 0x24

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 15:0			CNT[15:0]: Counter value												

12.4.11 TIMx prescaler (TIMx_PSC)

Address offset: 0x28

Reset value: 0x0000

Reset value: 0x0000															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 15:0			PSC[15:0]: Prescaler value The counter clock frequency CK_CNT is equal to $f_{CK_PSC} / (PSC[15:0] + 1)$.												

	PSC contains the value to be loaded in the active prescaler register at each update event (including when the counter is cleared through UG bit of TIMx_EGR register or through trigger controller when configured in “reset mode”).
--	--

12.4.12 TIMx auto-reload register (TIMx_ARR)

Address offset: 0x2C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15:0		ARR[15:0]: Prescaler value ARR is the value to be loaded in the actual auto-reload register. Refer to the <i>Section 12.3.1: Time-base unit</i> for more details about ARR update and behavior. The counter is blocked while the auto-reload value is null.													

12.4.13 TIMx capture/compare register 1 (TIMx_CCR1)

Address offset: 0x34

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR1[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15:0		CCR1[15:0]: Capture/Compare 1 value If channel CC1 is configured as output: CCR1 is the value to be loaded in the actual capture/compare 1 register (preload value). It is loaded permanently if the preload feature is not selected in the TIMx_CCMR1 register (bit OC1PE). Else the preload value is copied in the active capture/compare 1 register when an update event occurs. The active capture/compare register contains the value to be compared to the counter TIMx_CNT and signaled on OC1 output. If channel CC1 is configured as input: CCR1 is the counter value transferred by the last input capture 1 event (IC1). The TIMx_CCR1 register is read-only and cannot be programmed.													

12.4.14 TIMx capture/compare register 2 (TIMx_CCR2)

Address offset: 0x38

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR2[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15:0		CCR2[15:0]: Capture/Compare 2 value If channel CC2 is configured as output: CCR2 is the value to be loaded in the actual capture/compare 2 register (preload value). It is loaded permanently if the preload feature is not selected in the TIMx_CCMR2 register (bit OC2PE). Else the preload value is copied in the active capture/compare 2 register when an update event occurs. The active capture/compare register contains the value to be compared to the counter TIMx_CNT and signalled on OC2 output. If channel CC2 is configured as input: CCR2 is the counter value transferred by the last input capture 2 event (IC2). The TIMx_CCR2 register is read-only and cannot be programmed.													

12.4.15 TIMx capture/compare register 3 (TIMx_CCR3)

Address offset: 0x3C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR3[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15:0		CCR3[15:0]: Capture/Compare value If channel CC3 is configured as output: CCR3 is the value to be loaded in the actual capture/compare 3 register (preload value). It is loaded permanently if the preload feature is not selected in the TIMx_CCMR3 register (bit OC3PE). Else the preload value is copied in the active capture/compare 3 register when an update event occurs. The active capture/compare register contains the value to be compared to the counter TIMx_CNT and signaled on OC3 output. If channel CC3 is configured as input: CCR3 is the counter value transferred by the last input capture 3 event (IC3). The TIMx_CCR3 register is read-only and cannot be programmed.													

12.4.16 TIMx capture/compare register 4 (TIMx_CCR4)

Address offset: 0x40

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR4[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15:0		CCR4[15:0]: Capture/Compare value if CC4 channel is configured as output (CC4S bits): CCR4 is the value to be loaded in the actual capture/compare 4 register (preload value). It is loaded permanently if the preload feature is not selected in the TIMx_CCMR4 register (bit OC4PE). Else the preload value is copied in the active capture/compare 4 register when an update event occurs. The active capture/compare register contains the value to be compared to the counter TIMx_CNT and signalled on OC4 output. if CC4 channel is configured as input (CC4S bits in TIMx_CCMR4 register): CCR4 is the counter value transferred by the last input capture 4 event (IC4). The TIMx_CCR4 register is read-only and cannot be programmed.													

12.4.17 TIMx DMA control register (TIMx_DCR)

Address offset: 0x48

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Reserved			DBL[4:0]					Reserved			DBA[4:0]					
rw			rw		rw		rw		rw		rw		rw		rw	
Bits 15:13			Reserved, must be kept at reset value.													
Bits 12:8			DBL[4:0]: DMA burst length This 5-bit vector defines the number of DMA transfers (the timer recognizes a burst transfer when a read or a write access is done to the TIMx_DMAR address). 00000: 1 transfer, 00001: 2 transfers 00010: 3 transfers, 10001: 18 transfers.													
Bits 7:5			Reserved, must be kept at reset value.													
Bits 4:0			DBA[4:0]: DMA base address This 5-bit vector defines the base-address for DMA transfers (when read/write access are done through the TIMx_DMAR address). DBA is defined as an offset starting from the address of the TIMx_CR1 register.													

Example: 00000: TIMx_CR1 00001: TIMx_CR2 00010: TIMx_SMCR Example: Let us consider the following transfer: DBL = 7 transfers & DBA = TIMx_CR1. In this case the transfer is done to/from 7 registers starting from the TIMx_CR1 address.
--

12.4.18 TIMx DMA address for full transfer (TIMx_DMAR)

Address offset: 0x4C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DMAB[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 15:0		DMAB[15:0]: DMA register for burst accesses A read or write operation to the DMAR register accesses the register located at the address $(\text{TIMx_CR1 address}) + (\text{DBA} + \text{DMA index}) \times 4$ where TIMx_CR1 address is the address of the control register 1, DBA is the DMA base address configured in TIMx_DCR register, DMA index is automatically controlled by the DMA transfer, and ranges from 0 to DBL (DBL configured in TIMx_DCR).													

12.4.19 TIMx register map

TIMx registers are mapped as described in the table below:

Table 59 TIMx register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																				
000h	TIMx_CR1	Reserved																						CKD [1:0]		ARPE	CMS [1:0]	DIR	OPM	URS	UDIS	CEN																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																					
	Reset value																							0	0								0	0	0	0	0	0	0	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																													
004h	TIMx_CR2	Reserved																							TI1S	MMS [2:0]	CCDS	Reserved																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																									
	Reset value																												0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

00Ch	Reset value	TIMx_DIER	Reserved										0	TDE	Reserved		CC4DE	CC3DE	CC2DE	CC1DE	UDE	Reserved		0	TIE	Reserved		CC4IE	CC3IE	CC2IE	CC1IE	UIE
															0	CC4OE	CC3OE	CC2OE	CC1OE	Reserved		0	TIF	Reserved		0	CC4IF	CC3IF	CC2IF	CC1IF	UIF	
010h	Reset value	TIMx_SR	Reserved										0	CC4OE	CC3OE	CC2OE	CC1OE	Reserved		0	TG	Reserved		0	CC4G	CC3G	CC2G	CC1G	UG			
																				0	Reserved		0	CC4G	CC3G	CC2G	CC1G	UG				
014h	Reset value	TIMx_EGR	Reserved										0	OC2CE	OC2M [2:0]			0	OC2PE	OC2FE	CC2S [1: 0]		0	OC1CE	OC1M [2:0]			0	OC1PE	OC1FE	CC1S [1:0]	
018h	Reset value	TIMx_CCMR1 输入比较模式	Reserved										0	OC2CE	OC2M [2:0]			0	OC2PE	OC2FE	CC2S [1: 0]		0	OC1CE	OC1M [2:0]			0	OC1PE	OC1FE	CC1S [1:0]	
01Ch	Reset value	TIMx_CCMR2 输出比较模式	Reserved										0	OC4CE	OC4M [2:0]			0	OC4PE	OC4FE	CC4S [1: 0]		0	OC3CE	OC3M [2:0]			0	OC3PE	OC3FE	CC3S [1:0]	

	TIMx_CCMR2 输入比较模式	Reserved	IC4F [2:0]				IC4PSC [1: 0]		CC4S [1: 0]		IC3F [3:0]				IC3 PSC [1: 0]		CC3 S [1:0]	
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
020h	TIMx_CCER	Reserved	CC4P				Reserved		CC3P		Reserved		CC2P		Reserved		CC1P	
	Reset value		0	0			0	0	0	0	0	0	0	0	0	0		
024h	TIMx_CNT	Reserved	CNT[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
028h	TIMx_PSC	Reserved	PSC[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
02Ch	TIMx_ARR	Reserved	ARR[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
030h	Reserved																	
034h	TIMx_CCR1	Reserved	CCR1[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

038h	TIMx_CCR2	Reserved	CCR2[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
03Ch	TIMx_CCR3	Reserved	CCR3[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
040h	TIMx_CCR4	Reserved	CCR4[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
044h	Reserved																	
048h	TIMx_DCR	Reserved	DBL[4: 0]				Reserved				DBA[4: 0]							
	Reset value		0	0	0	0					0	0	0	0	0			
04Ch	TIMx_DMAR	Reserved	DMAB[15: 0]															
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Refer to Table 1 for the register boundary addresses.

13 Real-time clock (RTC)

13.1 RTC introduction

The real-time clock is an independent timer. The RTC provides a set of continuously running counters which can be used, with suitable software, to provide a clock-calendar function. The counter values can be written to set the current time/date of the system.

The RTC core and clock configuration (RCC_BDCR register) are in the Backup domain, which means that RTC setting and time are kept after reset or wakeup from Standby mode.

After reset, access to the Backup registers and RTC is disabled and the Backup domain (BKP) is protected against possible parasitic write access. To enable access to the Backup registers and the RTC, proceed as follows:

- enable the power and backup interface clocks by setting the PWREN and BKPEN bits in the RCC_APB1ENR register
- set the DBP bit the Power Control Register (PWR_CR) to enable access to the Backup registers and RTC.

13.2 RTC main features

- Programmable prescaler: division factor up to 2^{20}
- 32-bit programmable counter for long-term measurement
- Two separate clocks: PCLK1 for the APB1 interface and RTC clock (must be at least four times slower than the PCLK1 clock)
- The RTC clock source could be any of the following ones:
 - HSE clock divided by 128
 - LSE oscillator clock
 - LSI oscillator clock (refer to *Section 6.2.8: RTC clock* for details)
- Two separate reset types:
 - The APB1 interface is reset by system reset
 - The RTC Core (Prescaler, Alarm, Counter and Divider) is reset only by a Backup domain reset (see *Section 6.1.3: Backup domain reset*).
- Three dedicated maskable interrupt lines:
 - Alarm interrupt, for generating a software programmable alarm interrupt.
 - Seconds interrupt, for generating a periodic interrupt signal with a programmable period length (up to 1 second).
 - Overflow interrupt, to detect when the internal programmable counter rolls over to zero.

13.3 RTC functional description

13.3.1 Overview

The RTC consists of two main units (see Figure 131). The first one (APB1 Interface) is used to interface with the APB1 bus. This unit also contains a set of 16-bit registers accessible from the APB1 bus in read or write mode (for more information refer to *Section 13.4: RTC registers*). The APB1 interface is clocked by the APB1 bus clock in order to interface with the APB1 bus.

The other unit (RTC Core) consists of a chain of programmable counters made of two main blocks. The first block is the RTC prescaler block, which generates the RTC time base TR_CLK that can be

programmed to have a period of up to 1 second. It includes a 20-bit programmable divider (RTC Prescaler). Every TR_CLK period, the RTC generates an interrupt (Second Interrupt) if it is enabled in the RTC_CR register. The second block is a 32-bit programmable counter that can be initialized to the current system time. The system time is incremented at the TR_CLK rate and compared with a programmable date (stored in the RTC_ALR register) in order to generate an alarm interrupt, if enabled in the RTC_CR control register.

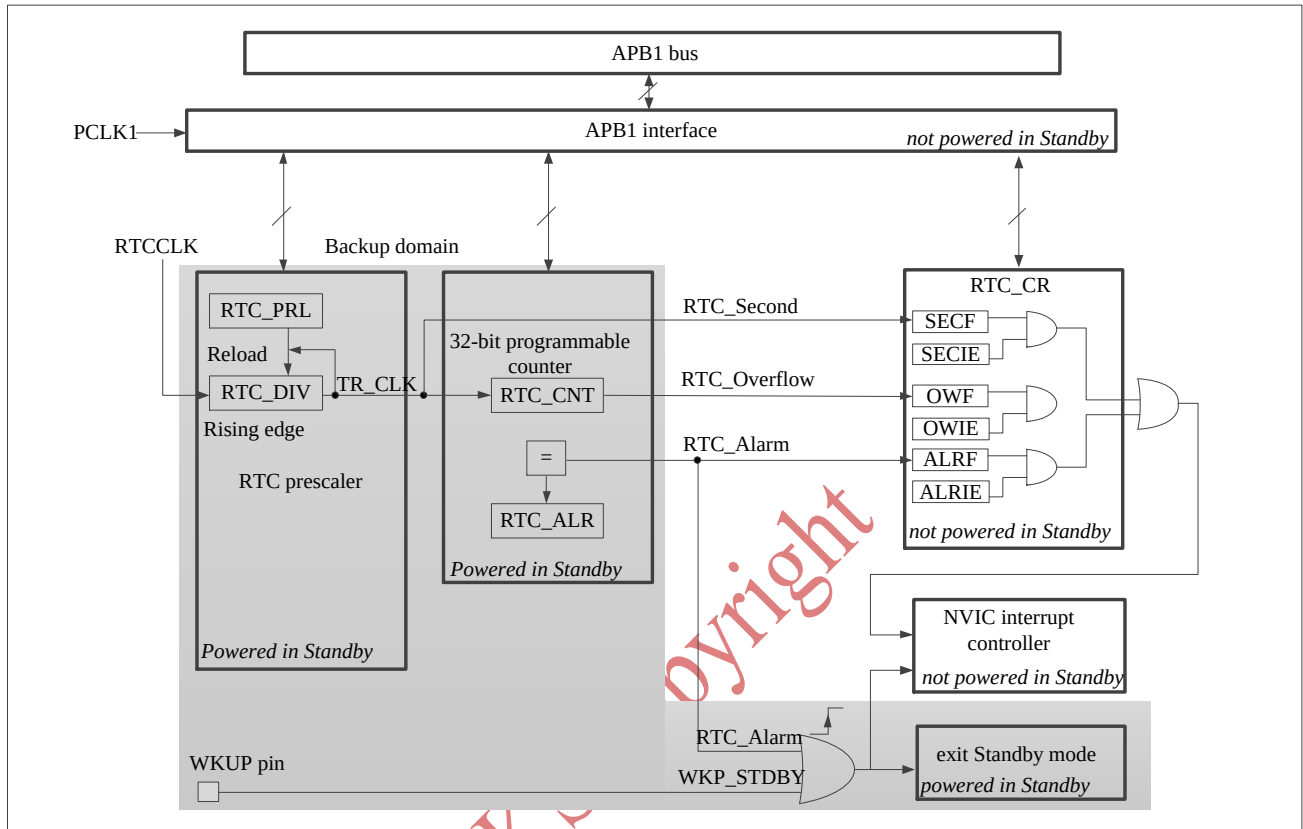


Figure 131 RTC simplified block diagram

13.3.2 Resetting RTC registers

All system registers are asynchronously reset by a System Reset or Power Reset, except for RTC_PRL, RTC_ALR, RTC_CNT, and RTC_DIV.

The RTC_PRL, RTC_ALR, RTC_CNT, and RTC_DIV registers are reset only by a Backup Domain reset. Refer to *Section 6.1.3*.

13.3.3 Reading RTC registers

The RTC core is completely independent from the RTC APB1 interface.

Software accesses the RTC prescaler, counter and alarm values through the APB1 interface but the associated readable registers are internally updated at each rising edge of the RTC clock resynchronized by the RTC APB1 clock. This is also true for the RTC flags.

This means that the first read to the RTC APB1 registers may be corrupted (generally read as 0) if the APB1 interface has previously been disabled and the read occurs immediately after the APB1 interface is enabled but before the first internal update of the registers. This can occur if:

- A system reset or power reset has occurred
- The MCU has just woken up from Standby mode (see *Section 4.3: Low-power modes*)

- The MCU has just woken up from Stop mode (see *Section 4.3: Low-power modes*)

In all the above cases, the RTC core has been kept running while the APB1 interface was disabled (reset, not clocked or unpowered).

Consequently when reading the RTC registers, after having disabled the RTC APB1 interface, the software must first wait for the RSF bit (Register Synchronized Flag) in the RTC_CRL register to be set by hardware.

Note that the RTC APB1 interface is not affected by WFI and WFE low-power modes.

13.3.4 Configuring RTC registers

To write in the RTC_PRL, RTC_CNT, RTC_ALR registers, the peripheral must enter Configuration Mode. This is done by setting the CNF bit in the RTC_CRL register.

In addition, writing to any RTC register is only enabled if the previous write operation is finished. To enable the software to detect this situation, the RTOFF status bit is provided in the RTC_CR register to indicate that an update of the registers is in progress. A new value can be written to the RTC registers only when the RTOFF status bit value is '1'.

Configuration procedure

1. Poll RTOFF, wait until its value goes to '1'
2. Set the CNF bit to enter configuration mode
3. Write to one or more RTC registers
4. Clear the CNF bit to exit configuration mode
5. Poll RTOFF, wait until its value goes to '1' to check the end of the write operation.

The write operation only executes when the CNF bit is cleared; it takes at least three RTCCLK cycles to complete.

13.3.5 RTC flag assertion

The RTC Second flag (SECF) is asserted on each RTC Core clock cycle before the update of the RTC Counter.

The RTC Overflow flag (OWF) is asserted on the last RTC Core clock cycle before the counter reaches 0x0000.

The RTC_Alarm and RTC Alarm flag (ALRF) (see Figure 132) are asserted on the last RTC Core clock cycle before the counter reaches the RTC Alarm value stored in the Alarm register increased by one (RTC_ALR + 1). The write operation in the RTC Alarm and RTC Second flag must be synchronized by using one of the following sequences:

- Use the RTC Alarm interrupt and inside the RTC interrupt routine, the RTC Alarm and/or RTC Counter registers are updated.
- Wait for SECF bit to be set in the RTC Control register. Update the RTC Alarm and/or the RTC Counter register.

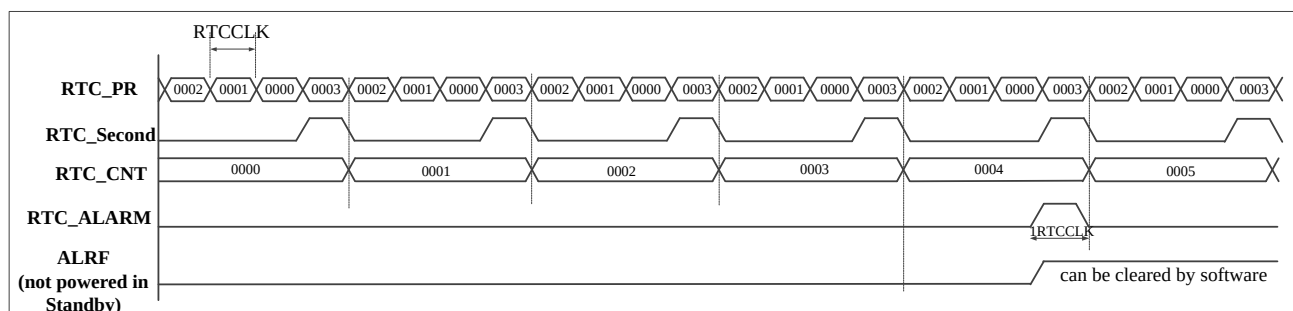


Figure 132 RTC second and alarm waveform example with PR=0003, ALARM=00004

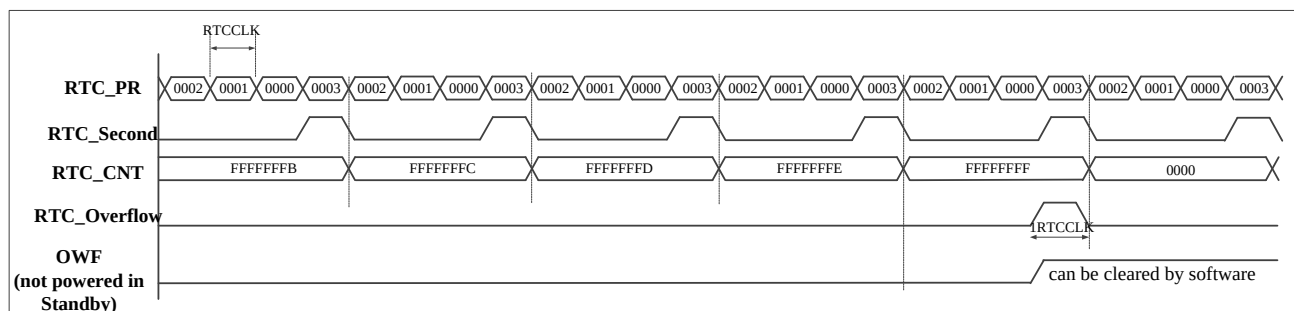


Figure 133 RTC Overflow waveform example with PR=0003

13.4 RTC registers

Refer to *Section 1.1* for a list of abbreviations used in register descriptions.

The peripheral registers can be accessed by half-words (16-bit) or words (32-bit).

13.4.1 RTC control register high (RTC_CRH)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													OWIE	ALRIE	SECIE
													rw	rw	rw
Bits 15:3		Reserved, forced by hardware to 0.													
Bit2		OWIE: Overflow interrupt enable 0: Overflow interrupt is masked. 1: Overflow interrupt is enabled.													
Bit1		ALRIE: Alarm interrupt enable 0: Alarm interrupt is masked 1: Alarm interrupt is enabled.													
Bit0		SECIE: Second interrupt enable 0: Second interrupt is masked. 1: Second interrupt is enabled.													

These bits are used to mask interrupt requests. Note that at reset all interrupts are disabled, so it is possible to write to the RTC registers to ensure that no interrupt requests are pending after initialization. It is not possible to write to the RTC_CRH register when the peripheral is completing a previous write operation (flagged by RTOFF=0, see *Section 13.3.4*).

The RTC functions are controlled by this control register. Some bits must be written using a specific configuration procedure (see *Configuration procedure*).

13.4.2 RTC control register low (RTC_CRL)

Address offset: 0x04

Reset value: 0x0020

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										PTOFF	CNF	RSF	OWF	ALRF	SECF
										r	rw	rc w0	rc w0	rc w0	rc w0
Bits 15:6		Reserved, forced by hardware to 0.													
Bit 5		RTOFF: RTC operation OFF With this bit the RTC reports the status of the last write operation performed on its registers, indicating if it has been completed or not. If its value is '0' then it is not possible to write to any of the RTC registers. This bit is read only.													

	0: Last write operation on RTC registers is still ongoing. 1: Last write operation on RTC registers terminated.
Bit 4	CNF : Configuration flag This bit must be set by software to enter in configuration mode so as to allow new values to be written in the RTC_CNT, RTC_ALR or RTC_PRL registers. The write operation is only executed when the CNF bit is reset by software after has been set. 0: Exit configuration mode (start update of RTC registers). 1: Enter configuration mode.
Bit 3	RSF : Registers synchronized flag This bit is set by hardware at each time the RTC_CNT and RTC_DIV registers are updated and cleared by software. Before any read operation after an APB1 reset or an APB1 clock stop, this bit must be cleared by software, and the user application must wait until it is set to be sure that the RTC_CNT, RTC_ALR or RTC_PRL registers are synchronized. 0: Registers not yet synchronized. 1: Registers synchronized.
Bit 2	OWF : Overflow flag This bit is set by hardware when the 32-bit programmable counter overflows. An interrupt is generated if OWIE=1 in the RTC_CRH register. It can be cleared only by software. Writing '1' has no effect. 0: Overflow not detected 1: 32-bit programmable counter overflow occurred.
Bit 1	ALRF : Alarm flag This bit is set by hardware when the 32-bit programmable counter reaches the threshold set in the RTC_ALR register. An interrupt is generated if ALRIE=1 in the RTC_CRH register. It can be cleared only by software. Writing '1' has no effect. 0: Alarm not detected 1: Alarm detected
Bit 0	SECF : Second flag This bit is set by hardware when the 32-bit programmable prescaler overflows, thus incrementing the RTC counter. Hence this flag provides a periodic signal with a period corresponding to the resolution programmed for the RTC counter (usually one second). An interrupt is generated if SECIE=1 in the RTC_CRH register. It can be cleared only by software. Writing '1' has no effect. 0: Second flag condition not met. 1: Second flag condition met.

The functions of the RTC are controlled by this control register. It is not possible to write to the RTC_CR register while the peripheral is completing a previous write operation (flagged by RTOFF=0, see Section 13.3.4: Configuring RTC registers).

Note: Any flag remains pending until the appropriate RTC_CR request bit is reset by software, indicating that the interrupt request has been granted.

At reset the interrupts are disabled, no interrupt requests are pending and it is possible to write to the RTC registers.

The OWF, ALRF, SECF and RSF bits are not updated when the APB1 clock is not running.

The OWF, ALRF, SECF and RSF bits can only be set by hardware and only cleared by software.

If ALRF = 1 and ALRIE = 1, the RTC global interrupt is enabled. If EXTI Line 17 is also enabled through the EXTI Controller, both the RTC global interrupt and the RTC Alarm interrupt are enabled.

If ALRF = 1, the RTC Alarm interrupt is enabled if EXTI Line 17 is enabled through the EXTI Controller in interrupt mode. When the EXTI Line 17 is enabled in event mode, a pulse is generated on this line (no RTC Alarm interrupt generation).

13.4.3 RTC prescaler load register (RTC_PRLH / RTC_PRL)

The Prescaler Load registers keep the period counting value of the RTC prescaler. They are write-protected by the RTOFF bit in the RTC_CR register, and a write operation is allowed if the RTOFF value is '1'.

RTC prescaler load register high (RTC_PRLH)

Address offset: 0x08

Write only (see Section 13.3.4: Configuring RTC registers)

Reset value: 0x0000

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

Reserved										PRL[19: 16]			
										w	w	w	w
Bits 15: 4	Reserved, forced by hardware to 0.												
Bits 3:0	PRL[19:16]: RTC prescaler reload value high These bits are used to define the counter clock frequency according to the following formula: $f_{TR_CLK} = f_{RTCCLK}/(PRL[19:0]+1)$												

RTC prescaler load register low (RTC_PRL)

Address offset: 0x0C

Write only (see Section 13.3.4: Configuring RTC registers)

Reset value: 0x8000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRL[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
Bits 15: 0	PRL[15:0]: RTC prescaler reload value low These bits are used to define the counter clock frequency according to the following formula: $f_{TR_CLK} = f_{RTCCLK}/(PRL[19:0]+1)$														

Note: If the input clock frequency (f_{RTCCLK}) is 32.768 kHz, write 7FFFh in this register to get a signal period of 1 second.

13.4.4 RTC prescaler divider register (RTC_DIVH / RTC_DIVL)

During each period of TR_CLK, the counter inside the RTC prescaler is reloaded with the value stored in the RTC_PRL register. To get an accurate time measurement it is possible to read the current value of the prescaler counter, stored in the RTC_DIV register, without stopping it. This register is read-only and it is reloaded by hardware after any change in the RTC_PRL or RTC_CNT registers.

RTC prescaler divider register high (RTC_DIVH)

Address offset: 0x10

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										RTC_DIV[19:16]					
										r	r	r	r		
Bits 15: 4	Reserved														
Bits 3:0	RTC_DIV[19:16]: RTC clock divider high														

RTC prescaler divider register low (RTC_DIVL)

Address offset: 0x14

Reset value: 0x8000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTC_DIV[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 15: 0	RTC_DIV[15:0]: RTC clock divider low														

13.4.5 RTC counter register (RTC_CNTH / RTC_CNTL)

The RTC core has one 32-bit programmable counter, accessed through two 16-bit registers; the count rate is based on the TR_CLK time reference, generated by the prescaler. RTC_CNT registers keep the counting value of this counter. They are write-protected by bit RTOFF in the RTC_CR register, and a write operation is allowed if the RTOFF value is '1'. A write operation on the upper (RTC_CNTH) or lower (RTC_CNTL) registers directly loads the corresponding programmable

counter and reloads the RTC Prescaler. When reading, the current value in the counter (system date) is returned.

RTC counter register high (RTC_CNTH)

Address offset: 0x18

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTC_DIV[31:16]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15: 0		RTC_CNT[31:16]: RTC counter high Reading the RTC_CNTH register, the current value of the high part of the RTC Counter register is returned. To write to this register it is necessary to enter configuration mode (see <i>Section 13.3.4: Configuring RTC registers</i>).													

RTC counter register low (RTC_CNTL)

Address offset: 0x1C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTC_CNT[15: 0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15: 0		RTC_CNT[15:0]: RTC counter low Reading the RTC_CNTL register, the current value of the lower part of the RTC Counter register is returned. To write to this register it is necessary to enter configuration mode (see <i>Section 13.3.4: Configuring RTC registers</i>).													

13.4.6 RTC alarm register high (RTC_ALRH/RTC_ALRL)

When the programmable counter reaches the 32-bit value stored in the RTC_ALR register, an alarm is triggered and the RTC_alarmIT interrupt request is generated. This register is write-protected by the RTOFF bit in the RTC_CR register, and a write operation is allowed if the RTOFF value is '1'.

RTC alarm register high (RTC_ALRH)

Address offset: 0x20

Write only (see *Section 13.3.4: Configuring RTC registers*)

Reset value: 0xFFFF

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTC_ALR[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
Bits 15: 0		RTC_ALR[31:16]: RTC alarm high The high part of the alarm time is written by software in this register. To write to this register it is necessary to enter configuration mode (see <i>Section 13.3.4: Configuring RTC registers</i>)													

RTC alarm register low (RTC_ALRL)

Address offset: 0x24

Write only (see *Section 13.3.4: Configuring RTC registers*)

Reset value: 0xFFFF

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTC_ALR[15: 0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
Bits 15: 0		RTC_ALR[15:0]: RTC alarm low The low part of the alarm time is written by software in this register. To write to this register it is necessary to enter configuration mode (see <i>Section 13.3.4: Configuring RTC registers</i>).													

13.4.7 RTC register map

RTC registers are mapped as 16-bit addressable registers as described in the table below:

Table 60 RTC register map and reset values

Offset	000h		004h		008h		00Ch		010h			
Register	Reset value	RTC_CRH	Reset value	RTC_CRL	Reset value	DAC_PRLH	Reset value	RTC_PRLH	Reset value	RTC_DIVH		
31	Reserved											
30												
29												
28												
27												
26												
25												
24												
23												
22												
21												
20												
19												
18												
17												
16												
15	Reserved											
14												
13												
12												
11												
10												
9												
8												
7												
6												
5												
4												
3												
2											0	OWIE
1											0	ALRIE
0											0	SECIE

014h	RTC_DIVL	Reserved	DIV[15:0]													
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
018h	RTC_CNTH	Reserved	CNT[31:16]													
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
01Ch	RTC_CNTL	Reserved	CNT[15:0]													
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
020h	RTC_ALRH	Reserved	ALR[31:16]													
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0
024h	RTC_ALRL	Reserved	ALR [15:0]													
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0

Refer to Table 1 for the register boundary addresses.

14 Independent watchdog (IWDG)

14.1 IWDG introduction

The devices have two embedded watchdog peripherals which offer a combination of high safety level, timing accuracy and flexibility of use. Both watchdog peripherals (Independent and Window) serve to detect and resolve malfunctions due to software failure, and to trigger system reset or an interrupt (window watchdog only) when the counter reaches a given timeout value.

The independent watchdog (IWDG) is clocked by its own dedicated low-speed clock (LSI) and thus stays active even if the main clock fails. The window watchdog (WWDG) clock is prescaled from the APB1 clock and has a configurable time-window that can be programmed to detect abnormally late or early application behavior.

The IWDG is best suited to applications which require the watchdog to run as a totally independent process outside the main application, but have lower timing accuracy constraints. The WWDG is best suited to applications which require the watchdog to react within an accurate timing window. For further information on the window watchdog, refer to *Section 15*.

14.2 IWDG main features

- Free-running downcounter
- clocked from an independent RC oscillator (can operate in Standby and Stop modes)
- Reset (if watchdog activated) when the downcounter value of 0x000 is reached

14.3 IWDG functional description

Figure 134 shows the functional blocks of the independent watchdog module.

When the independent watchdog is started by writing the value 0xCCCC in the Key register (IWDG_KR), the counter starts counting down from the reset value of 0xFFFF. When it reaches the end of count value (0x000) a reset signal is generated (IWDG_RESET).

Whenever the key value 0xAAAA is written in the IWDG_KR register, the IWDG_RLR value is reloaded in the counter and the watchdog reset is prevented.

14.3.1 Hardware watchdog

If the “Hardware watchdog” feature is enabled through the device option bits, the watchdog is automatically enabled at power-on, and will generate a reset unless the Key register is written by the software before the counter reaches end of count.

14.3.2 Register access protection

Write access to the IWDG_PR and IWDG_RLR registers is protected. To modify them, first write the code 0x5555 in the IWDG_KR register. A write access to this register with a different value will

break the sequence and register access will be protected again. This implies that it is the case of the reload operation (writing 0xAAAA).

A status register is available to indicate that an update of the prescaler or the down-counter reload value is on going.

14.3.3 Debug mode

When the microcontroller enters debug mode (Cortex[®]-M3 core halted), the IWDG counter either continues to work normally or stops, depending on DBG_IWDG_STOP configuration bit in DBG module. For more details, refer to *Section 22.16.2: Debug support for timers, watchdog, bxCAN and I2C*.

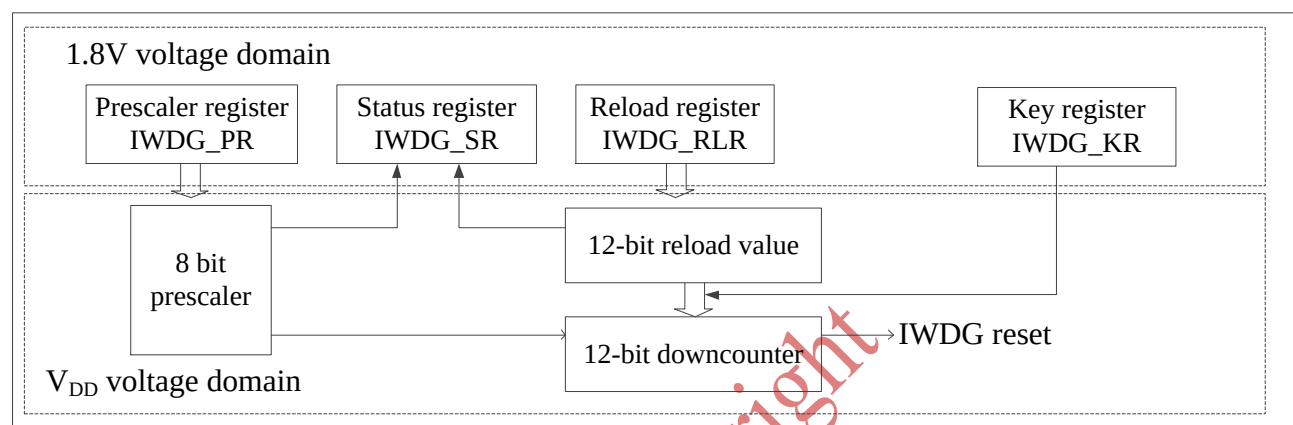


Figure 134 Independent watchdog block diagram

Note: The watchdog function is implemented in the V_{DD} voltage domain, still functional in Stop and Standby modes

Table 61 Min/max IWDG timeout period (in ms) at 40 kHz (LSI)⁽¹⁾

Prescaler divider	PR[2:0] bits	Min timeout RL[11:0]= 0x000	Max timeout RL[11:0]= 0xFFFF
/4	0	0.1	409.6
/8	1	0.2	819.2
/16	2	0.4	1638.4
/32	3	0.8	3276.8
/64	4	1.6	6553.6
/128	5	3.2	13107.2
/256	(6 or 7)	6.4	26214.4

(1) These timings are given for a 40 kHz clock but the microcontroller internal RC frequency can vary. Refer to the LSI oscillator characteristics table in the device datasheet for maximum and minimum values.

The LSI can be calibrated so as to compute the IWDG timeout with an acceptable accuracy. For more details refer to *Section 6.2.5: LSI clock*.

14.4 IWDG registers

Refer to *Section 1.1* for a list of abbreviations used in register descriptions.

The peripheral registers have to be accessed by half-words (16 bits) or words (32 bits).

14.4.1 Key register (IWDG_KR)

Address offset: 0x00

Reset value: 0x0000 0000 (reset by Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
Bits 31:16		Reserved, must be kept at reset value.													
Bits 15:0		KEY[15:0]: Key value (write only, read 0000h) These bits must be written by software at regular intervals with the key value AAAAh, otherwise the watchdog generates a reset when the counter reaches 0. Writing the key value 5555h to enable access to the IWDG_PR and IWDG_RLR registers (see Section 14.3.2)													

14.4.2 Prescaler register (IWDG_PR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													PR[2:0]		
													rw	rw	rw
Bits 31: 3		Reserved, must be kept at reset value.													
Bits 2:0		PR[2:0]: Prescaler divider These bits are write access protected see Section 14.3.2. They are written by software to select the prescaler divider feeding the counter clock. PVU bit of IWDG_SR must be reset in order to be able to change the prescaler divider. 000: divider /4 001: divider /8 010: divider /16 011: divider /32 100: divider /64 101: divider /128 110: divider /256 111: divider /256 <i>Note: Reading this register returns the prescaler value from the VDD voltage domain. This value may not be up to date/valid if a write operation to this register is ongoing. For this reason the value read from this register is valid only when the PVU bit in the IWDG_SR register is reset.</i>													

14.4.3 Reload register (IWDG_RLR)

Address offset: 0x08

Reset value: 0x0000 0FFF (reset by Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				RL[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31: 12		Reserved, must be kept at reset value.													
Bits 11:0		RL[11:0]: Watchdog counter reload value These bits are write access protected see Section 14.3.2. They are written by software to define the value to be loaded in the watchdog counter each time the value AAAAh is written in the IWDG_KR register. The watchdog counter counts down from this value. The timeout period is a function of this value and the clock prescaler. Refer to Table 61. The RVU bit in the IWDG_SR register must be reset in order to be able to change the reload value. <i>Note: Reading this register returns the reload value from the VDD voltage domain. This value may not be up to date/valid if a write operation to this register is ongoing on this register. For this reason the value read from this register is valid only when the RVU bit in the IWDG_SR register is reset.</i>													

14.4.4 Status register (IWDG_SR)

Address offset: 0x0C

Reset value: 0x0000 0000 (not reset by Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														RVU	PVU
														r	r

Bits 31: 2	Reserved, must be kept at reset value.
Bit1	RVU: Watchdog counter reload value update This bit is set by hardware to indicate that an update of the reload value is ongoing. It is reset by hardware when the reload value update operation is completed in the V _{DD} voltage domain (takes up to 5 RC 40 kHz cycles). Reload value can be updated only when RVU bit is reset.
Bit0	PVU: Watchdog prescaler value update This bit is set by hardware to indicate that an update of the prescaler value is ongoing. It is reset by hardware when the prescaler update operation is completed in the V _{DD} voltage domain (takes up to 5 RC 40 kHz cycles). Prescaler value can be updated only when PVU bit is reset.

Note: If several reload values or prescaler values are used by application, it is mandatory to wait until RVU bit is reset before changing the reload value and to wait until PVU bit is reset before changing the prescaler value. However, after updating the prescaler and/or the reload value it is not necessary to wait until RVU or PVU is reset before continuing code execution (even in case of low-power mode entry, the write operation is taken into account and will complete).

14.4.5 IWDG register map

Table 62 IWDG register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
000h	IWDG_KR	Reserved																KEY[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
004h	IWDG_PR	Reserved																										PR[2:0]					
	Reset value																											0	0	0			
008h	IWDG_RLR	Reserved																RL[11:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
00Ch	IWDG_PR	Reserved																										RVU					
	Reset value																											0	PVU				

Refer to Table 1 for the register boundary addresses.

15 Window watchdog (WWDG)

15.1 WWDG introduction

The window watchdog is used to detect the occurrence of a software fault, usually generated by external interference or by unforeseen logical conditions, which causes the application program to abandon its normal sequence. The watchdog circuit generates an MCU reset on expiry of a programmed time period, unless the program refreshes the contents of the downcounter before the T6 bit becomes cleared. An MCU reset is also generated if the 7-bit downcounter value (in the control register) is refreshed before the downcounter has reached the window register value. This implies that the counter must be refreshed in a limited window.

15.2 WWDG main features

- Programmable free-running downcounter
- Conditional reset
 - Reset (if watchdog activated) when the downcounter value becomes less than 0x40
 - Reset (if watchdog activated) if the downcounter is reloaded outside the window (see Figure 136)
- Early wakeup interrupt (EWI): triggered (if enabled and the watchdog activated) when the downcounter is equal to 0x40.

15.3 WWDG functional description

If the watchdog is activated (the WDGA bit is set in the WWDG_CR register) and when the 7-bit downcounter (T[6:0] bits) rolls over from 0x40 to 0x3F (T6 becomes cleared), it initiates a reset. If the software reloads the counter while the counter is greater than the value stored in the window register, then a reset is generated.

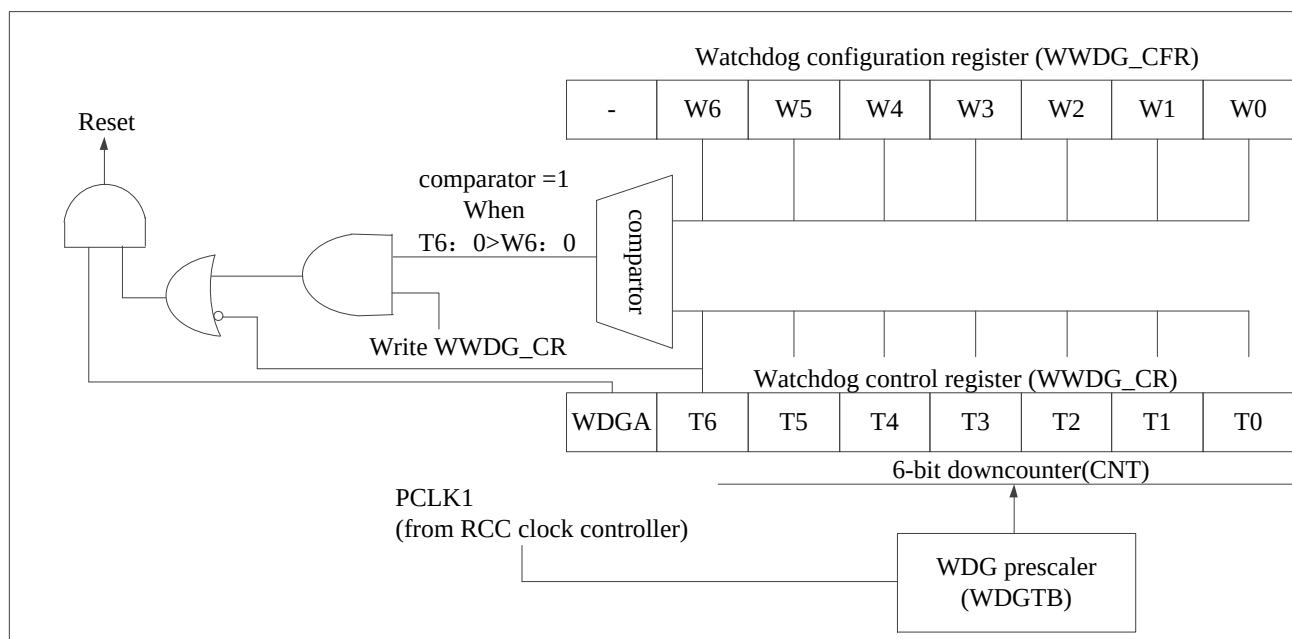


Figure 135 Watchdog block diagram

The application program must write in the WWDG_CR register at regular intervals during normal operation to prevent an MCU reset. This operation must occur only when the counter value is lower than the window register value. The value to be stored in the WWDG_CR register must be between 0xFF and 0xC0.

- Enabling the watchdog

The watchdog is always disabled after a reset. It is enabled by setting the WDGA bit in the WWDG_CR register, then it cannot be disabled again except by a reset.

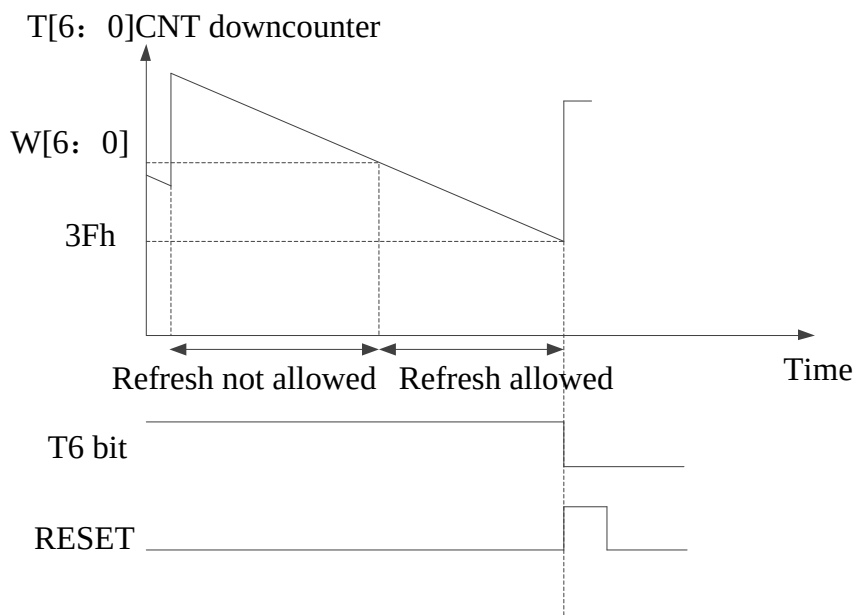
- Controlling the downcounter

This downcounter is free-running, counting down even if the watchdog is disabled. When the watchdog is enabled, the T6 bit must be set to prevent generating an immediate reset. The T[5:0] bits contain the number of increments which represents the time delay before the watchdog produces a reset. The timing varies between a minimum and a maximum value due to the unknown status of the prescaler when writing to the WWDG_CR register (see Figure 136). The Configuration register (WWDG_CFR) contains the high limit of the window: To prevent a reset, the downcounter must be reloaded when its value is lower than the window register value and greater than 0x3F. Figure 136 describes the window watchdog process.

Note: The T6 bit can be used to generate a software reset (the WDGA bit is set and the T6 bit is cleared).

15.4 How to program the watchdog timeout

Warning: When writing to the WWDG_CR register, always write 1 in the T6 bit to avoid generating an immediate reset.



The formula to calculate the WWDG timeout value is given by:

$$T_{\text{WWDG}} = T_{\text{PCLK1}} \times 4096 \times 2^{\text{WDGTB}} \times (T[5:0] + 1); \quad (\text{ms})$$

where:

T_{WWDG} : WWDG timeout

T_{PCLK1} : APB1 clock period measured in ms

WDGTB	min timeout value	max timeout value
0	113 μs	7.28ms
1	227 μs	14.56ms
2	455 μs	29.12ms
3	910 μs	58.25ms

Figure 136 Window watchdog timing diagram

15.5 Debug mode

When the microcontroller enters debug mode (Cortex[®]-M3 core halted), the WWDG counter either continues to work normally or stops, depending on DBG_WWDG_STOP configuration bit in DBG module. For more details, refer to *Section 22.16.2: Debug support for timers, watchdog, bxCAN and I2C*.

15.6 WWDG registers

Refer to *Section 1.1* for a list of abbreviations used in register descriptions.
The peripheral registers have to be accessed by half-words (16 bits) or words (32 bits).

15.6.1 Control register (WWDG_CR)

Address offset: 0x00

Reset value: 0x7F

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								WDGA	T6	T5	T4	T3	T2	T1	T0
								rs	rw	rw	rw	rw	rw	rw	rw
Bits 31:8		Reserved, must be kept at reset value.													
Bit 7		WDGA: Activation bit This bit is set by software and only cleared by hardware after a reset. When WDGA = 1, the watchdog can generate a reset. 0: Watchdog disabled 1: Watchdog enabled													
Bits 6:0		T[6:0]: 7-bit counter (MSB to LSB) These bits contain the value of the watchdog counter. It is decremented every $(4096 \times 2^{\text{WDGTB}[1:0]})$ PCLK1 cycles. A reset is produced when it rolls over from 0x40 to 0x3F (T6 becomes cleared).													

15.6.2 Configuration register (WWDG_CFR)

Address offset: 0x04

Reset value: 0x7F

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								EWI	WDG TB1	WDG TB0	W6	W5	W4	W3	W2
								rs	rw	rw	rw	rw	rw	rw	rw
Bits 31:10		Reserved, must be kept at reset value.													
Bit 9		EWI: Early wakeup interrupt When set, an interrupt occurs whenever the counter reaches the value 0x40. This interrupt is only cleared by hardware after a reset.													
Bits 8:7		WDGTB[1:0]: Timer base The time base of the prescaler can be modified as follows: 00: CK Counter Clock (PCLK1 div 4096) div 1 01: CK Counter Clock (PCLK1 div 4096) div 2 10: CK Counter Clock (PCLK1 div 4096) div 4 11: CK Counter Clock (PCLK1 div 4096) div 8													
Bits 6:0		W[6:0]: 7-bit window value These bits contain the window value to be compared to the downcounter.													

15.6.3 Status register (WWDG_SR)

Address offset: 0x08

Reset value: 0x00

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															EWIF
															rc w0
Bits 31:1		Reserved, must be kept at reset value.													
Bit0		EWIF: Early wakeup interrupt flag													

	This bit is set by hardware when the counter has reached the value 0x40. It must be cleared by software by writing '0'. A write of '1' has no effect. This bit is also set if the interrupt is not enabled.
--	---

CKS copyright

15.6.4 WWDG register map

Table 63 WWDG register map and reset values

offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																									
000h	WWDG_CR	Reserved																								WDGA	T[6: 0]																															
	Reset value																										0	1	1	1	1	1	1	1																								
004h	WWDG_CFR	Reserved																						EWI	WDGTB1	WDGTB0	W[6: 0]																															
	Reset value																										0	0	0	1	1	1	1	1	1																							
	008h																							WWDG_SR	Reserved																																	
																								Reset value																																		

Refer to Table 1 for the register boundary addresses.

16 Universal serial bus full-speed device interface (USB)

16.1 USB introduction

The USB peripheral implements an interface between a full-speed USB 2.0 bus and the APB1 bus. USB suspend/resume are supported which allows to stop the device clocks for low-power consumption.

16.2 USB main features

- USB specification version 2.0 full-speed compliant
- Configurable number of endpoints from 1 to 8
- Cyclic redundancy check (CRC) generation/checking, Non-return-to-zero Inverted (NRZI) encoding/decoding and bit-stuffing
- Isochronous transfers support
- Double-buffered bulk/isochronous endpoint support
- USB Suspend/Resume operations
- Frame locked clock pulse generation

Note: The USB and CAN share a dedicated 512-byte SRAM memory for data transmission and reception, and so they cannot be used concurrently (the shared RAM is accessed through CAN and USB exclusively). The USB and CAN can be used in the same application but not at the same time. Figure 137 shows the block diagram of the USB peripheral.

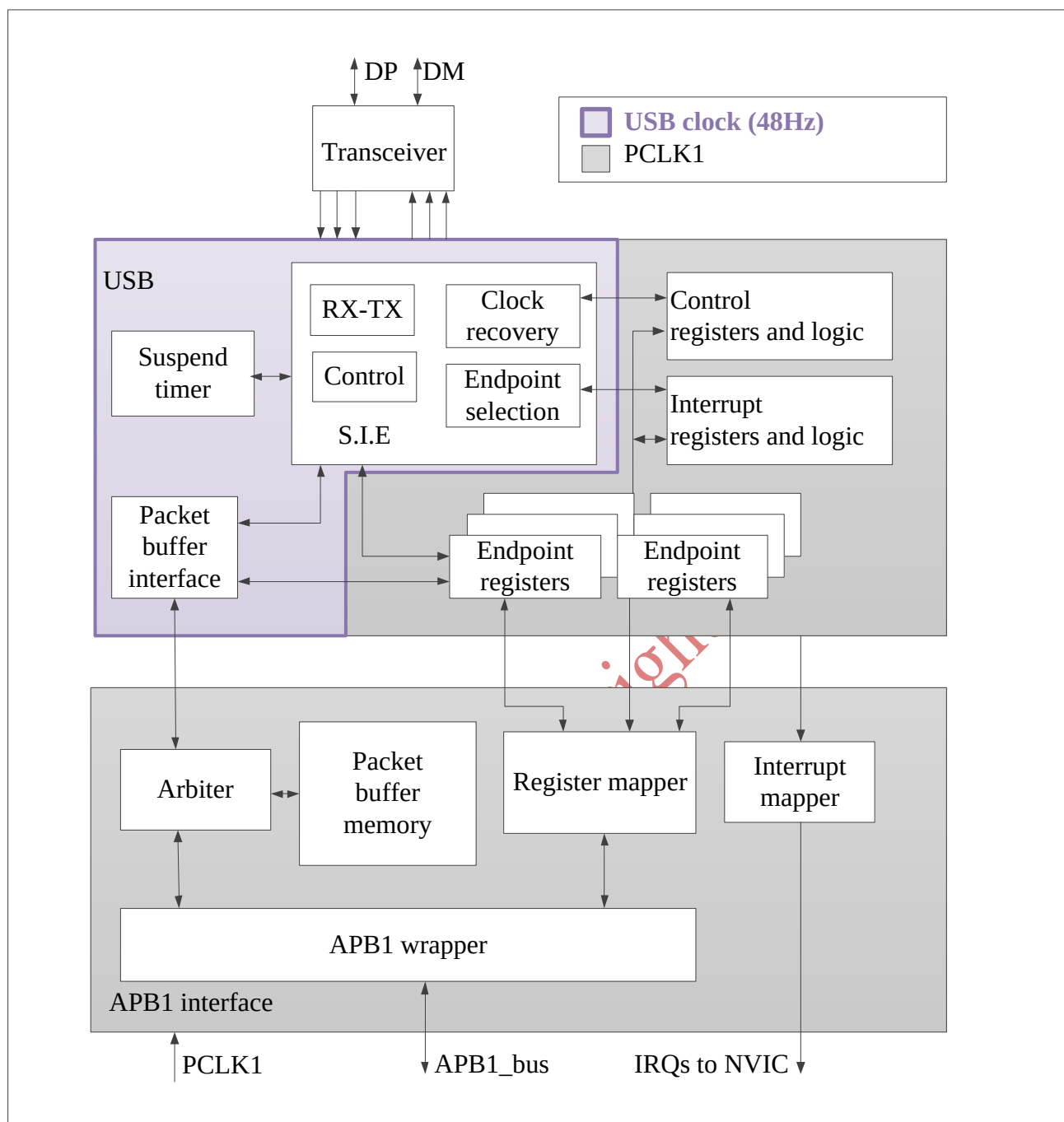


Figure 137 USB peripheral block diagram

16.3 USB functional description

The USB peripheral provides an USB compliant connection between the host PC and the function implemented by the microcontroller. Data transfer between the host PC and the system memory occurs through a dedicated packet buffer memory accessed directly by the USB peripheral. The size of this dedicated buffer memory must be according to the number of endpoints used and the maximum packet size. This dedicated memory is sized to 512 bytes and up to 16 mono-directional or 8 bidirectional endpoints can be used. The USB peripheral interfaces with the USB host, detecting token packets, handling data transmission/reception, and processing handshake packets as required by the USB standard. Transaction formatting is performed by the hardware, including CRC generation and checking.

Each endpoint is associated with a buffer description block indicating where the endpoint related memory area is located, how large it is or how many bytes must be transmitted. When a token for a valid function/endpoint pair is recognized by the USB peripheral, the related data transfer (if required and if the endpoint is configured) takes place. The data buffered by the USB peripheral is loaded in an internal 16 bit register and memory access to the dedicated buffer is performed. When all the data has been transferred, if needed, the proper handshake packet over the USB is generated or expected according to the direction of the transfer.

At the end of the transaction, an endpoint-specific interrupt is generated, reading status registers and/or using different interrupt response routines. The microcontroller can determine:

- Which endpoint has to be served
- Which type of transaction took place, if errors occurred (bit stuffing, format, CRC, protocol, missing ACK, over/underrun, etc.)

Special support is offered to Isochronous transfers and high throughput bulk transfers, implementing a double buffer usage, which allows to always have an available buffer for the USB peripheral while the microcontroller uses the other one.

The unit can be placed in low-power mode (SUSPEND mode), by writing in the control register, whenever required. At this time, all static power dissipation is avoided, and the USB clock can be slowed down or stopped. The detection of activity at the USB inputs, while in low-power mode, wakes the device up asynchronously. A special interrupt source can be connected directly to a wakeup line to allow the system to immediately restart the normal clock generation and/or support direct clock start/stop.

16.3.1 Description of USB blocks

The USB peripheral implements all the features related to USB interfacing, which include the following blocks:

- **Serial Interface Engine (SIE):** The functions of this block include: synchronization pattern recognition, bit-stuffing, CRC generation and checking, PID verification/generation, and handshake evaluation. It must interface with the USB transceivers and uses the virtual buffers provided by the packet buffer interface for local data storage. This unit also generates signals according to USB peripheral events, such as Start of Frame (SOF), USB Reset, Data errors etc. and to Endpoint related events like end of transmission or correct reception of a packet; these signals are then used to generate interrupts.
- **Timer:** This block generates a start-of-frame locked clock pulse and detects a global suspend (from the host) when no traffic has been received for 3 ms.
- **Packet Buffer Interface:** This block manages the local memory implementing a set of buffers in a flexible way, both for transmission and reception. It can choose the proper buffer according to requests coming from the SIE and locate them in the memory addresses pointed by the Endpoint registers. It increments the address after each exchanged word until the end of packet, keeping track of the number of exchanged bytes and preventing the buffer to overrun the maximum capacity.
- **Endpoint-Related Registers:** Each endpoint has an associated register containing the endpoint type and its current status. For mono-directional/single-buffer endpoints, a single register can be used to implement two distinct endpoints. The number of registers is 8, allowing up to 16 mono-directional/single-buffer or up to 7 double-buffer endpoints in any combination. For example the USB peripheral can be programmed to have 4 double buffer endpoints and 8 single-buffer/mono-directional endpoints.
- **Control Registers:** These are the registers containing information about the status of the whole USB peripheral and used to force some USB events, such as resume and power-down.
- **Interrupt Registers:** These contain the Interrupt masks and a record of the events. They can be used to inquire an interrupt reason, the interrupt status or to clear the status of a pending interrupt.

The USB peripheral is connected to the APB1 bus through an APB1 interface, containing the following blocks:

- **Packet Memory:** This is the local memory that physically contains the Packet Buffers. It can be used by the Packet Buffer interface, which creates the data structure and can be accessed directly by the application software. The size of the Packet Memory is 512 bytes, structured as 256 words by 16 bits.
- **Arbiter:** This block accepts memory requests coming from the APB1 bus and from the USB interface. It resolves the conflicts by giving priority to APB1 accesses, while always reserving half of the memory bandwidth to complete all USB transfers. This time-duplex scheme implements a virtual dual-port SRAM

that allows memory access, while an USB transaction is happening. Multiword APB1 transfers of any length are also allowed by this scheme.

- Register Mapper: This block collects the various byte-wide and bit-wide registers of the USB peripheral in a structured 16-bit wide word set addressed by the APB1.
- APB1 Wrapper: This provides an interface to the APB1 for the memory and register. It also maps the whole USB peripheral in the APB1 address space.
- Interrupt Mapper: This block is used to select how the possible USB events can generate interrupts and map them to three different lines of the NVIC:
 - USB low-priority interrupt (Channel 20): Triggered by all USB events (Correct transfer, USB reset, etc.). The firmware has to check the interrupt source before serving the interrupt.
 - USB high-priority interrupt (Channel 19): Triggered only by a correct transfer event for isochronous and double-buffer bulk transfer to reach the highest possible transfer rate.
 - USB wakeup interrupt (Channel 42): Triggered by the wakeup event from the USB Suspend mode.

16.4 Programming considerations

In the following sections, the expected interactions between the USB peripheral and the application program are described, in order to ease application software development.

16.4.1 Generic USB device programming

This part describes the main tasks required of the application software in order to obtain USB compliant behavior. The actions related to the most general USB events are taken into account and paragraphs are dedicated to the special cases of double-buffered endpoints and Isochronous transfers. Apart from system reset, action is always initiated by the USB peripheral, driven by one of the USB events described below.

16.4.2 System and power-on reset

Upon system and power-on reset, the first operation the application software should perform is to provide all required clock signals to the USB peripheral and subsequently de-assert its reset signal so to be able to access its registers. The whole initialization sequence is hereafter described.

As a first step application software needs to activate register macrocell clock and de-assert macrocell specific reset signal using related control bits provided by device clock management logic.

After that, the analog part of the device related to the USB transceiver must be switched on using the PDWN bit in CNTR register, which requires a special handling. This bit is intended to switch on the internal voltage references that supply the port transceiver. This circuit has a defined startup time (t_{STARTUP} specified in the datasheet) during which the behavior of the USB transceiver is not defined. It is thus necessary to wait this time, after setting the PDWN bit in the CNTR register, before removing the reset condition on the USB part (by clearing the FRES bit in the CNTR register). Clearing the ISTR register then removes any spurious pending interrupt before any other macrocell operation is enabled.

At system reset, the microcontroller must initialize all required registers and the packet buffer description table, to make the USB peripheral able to properly generate interrupts and data transfers. All registers not specific to any endpoint must be initialized according to the needs of application software (choice of enabled interrupts, chosen address of packet buffers, etc.). Then the process continues as for the USB reset case (see further paragraph).

USB reset (RESET interrupt)

When this event occurs, the USB peripheral is put in the same conditions it is left by the system reset after the initialization described in the previous paragraph: communication is disabled in all endpoint registers (the USB peripheral will not respond to any packet). As a response to the USB reset event,

the USB function must be enabled, having as USB address 0, implementing only the default control endpoint (endpoint address is 0 too). This is accomplished by setting the Enable Function (EF) bit of the USB_DADDR register and initializing the EP0R register and its related packet buffers accordingly. During USB enumeration process, the host assigns a unique address to this device, which must be written in the ADD[6:0] bits of the USB_DADDR register, and configures any other necessary endpoint.

When a RESET interrupt is received, the application software is responsible to enable again the default endpoint of USB function 0 within 10mS from the end of reset sequence which triggered the interrupt.

Structure and usage of packet buffers

Each bidirectional endpoint may receive or transmit data from/to the host. The received data is stored in a dedicated memory buffer reserved for that endpoint, while another memory buffer contains the data to be transmitted by the endpoint. Access to this memory is performed by the packet buffer interface block, which delivers a memory access request and waits for its acknowledgement. Since the packet buffer memory has to be accessed by the microcontroller also, an arbitration logic takes care of the access conflicts, using half APB1 cycle for microcontroller access and the remaining half for the USB peripheral access. In this way, both the agents can operate as if the packet memory is a dual-port SRAM, without being aware of any conflict even when the microcontroller is performing back-to-back accesses. The USB peripheral logic uses a dedicated clock. The frequency of this dedicated clock is fixed by the requirements of the USB standard at 48 MHz, and this can be different from the clock used for the interface to the APB1 bus. Different clock configurations are possible where the APB1 clock frequency can be higher or lower than the USB peripheral one.

Note: Due to USB data rate and packet memory interface requirements, the APB1 clock frequency must be greater than 8 MHz to avoid data overrun/underrun problems.

Each endpoint is associated with two packet buffers (usually one for transmission and the other one for reception). Buffers can be placed anywhere inside the packet memory because their location and size is specified in a buffer description table, which is also located in the packet memory at the address indicated by the USB_BTABLER register. Each table entry is associated to an endpoint register and it is composed of four 16-bit words so that table start address must always be aligned to an 8-byte boundary (the lowest three bits of USB_BTABLER register are always “000”). Buffer descriptor table entries are described in the *Section 16.5.3*. If an endpoint is unidirectional and it is neither an Isochronous nor a double-buffered bulk, only one packet buffer is required (the one related to the supported transfer direction). Other table locations related to unsupported transfer directions or unused endpoints, are available to the user. Isochronous and double-buffered bulk endpoints have special handling of packet buffers (refer to *Section 16.4.4* and *Section 16.4.3* respectively). The relationship between buffer description table entries and packet buffer areas is depicted in Figure 138.

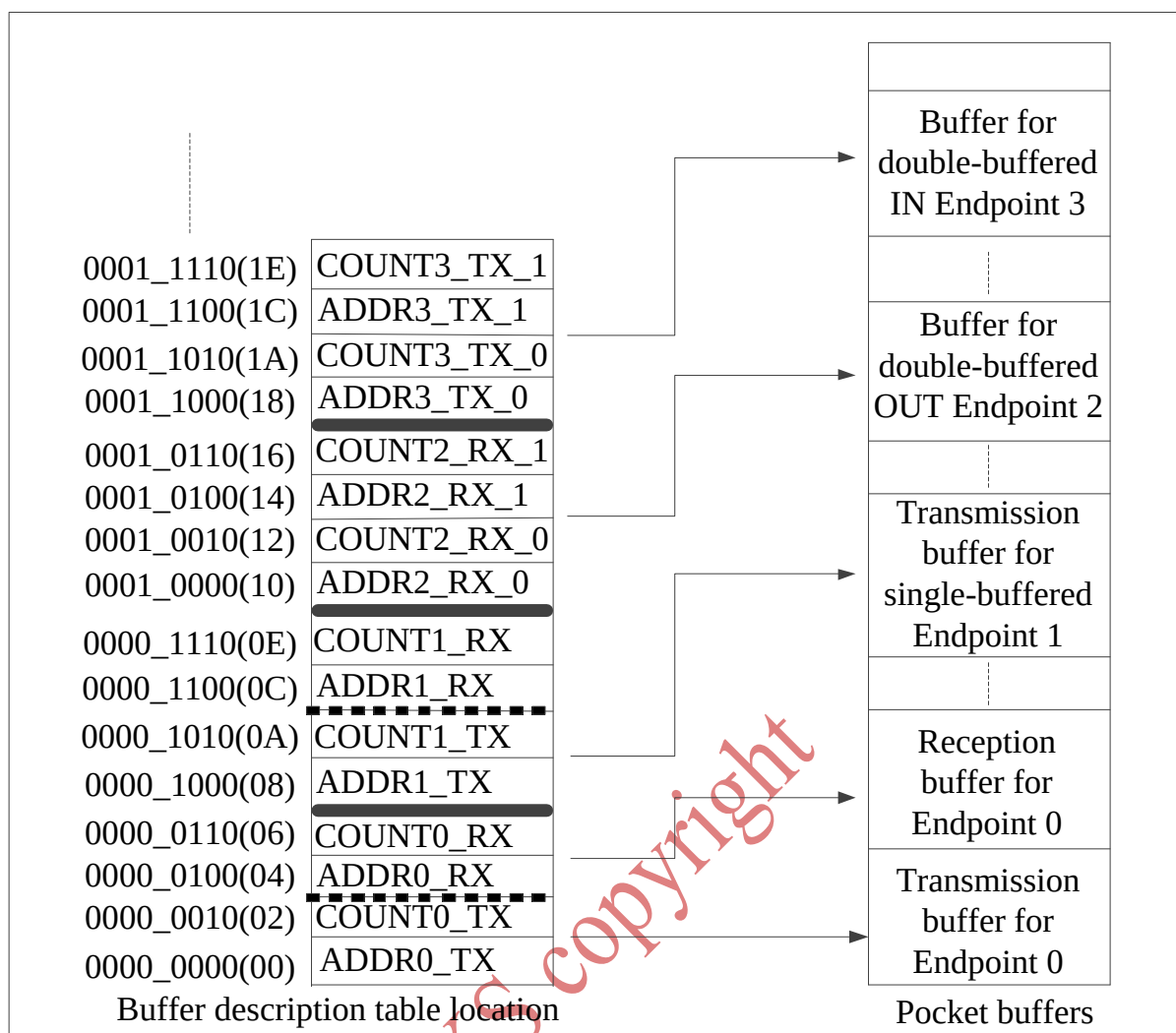


Figure 138 Packet buffer areas with examples of buffer description table locations

Each packet buffer is used either during reception or transmission starting from the bottom. The USB peripheral will never change the contents of memory locations adjacent to the allocated memory buffers; if a packet bigger than the allocated buffer length is received (buffer overrun condition) the data will be copied to the memory only up to the last available location.

Endpoint initialization

The first step to initialize an endpoint is to write appropriate values to the ADDRn_TX/ADDRn_RX registers so that the USB peripheral finds the data to be transmitted already available and the data to be received can be buffered. The EP_TYPE bits in the USB_EPnR register must be set according to the endpoint type, eventually using the EP_KIND bit to enable any special required feature. On the transmit side, the endpoint must be enabled using the STAT_TX bits in the USB_EPnR register and COUNTn_TX must be initialized. For reception, STAT_RX bits must be set to enable reception and COUNTn_RX must be written with the allocated buffer size using the BL_SIZE and NUM_BLOCK fields. Unidirectional endpoints, except Isochronous and double-buffered bulk endpoints, need to initialize only bits and registers related to the supported direction. Once the transmission and/or reception are enabled, register USB_EPnR and locations ADDRn_TX/ADDRn_RX, COUNTn_TX/COUNTn_RX (respectively), should not be modified by the application software, as the hardware can change their value on the fly. When the data transfer operation is completed, notified by a CTR interrupt event, they can be accessed again to re-enable a new operation.

IN packets (data transmission)

When receiving an IN token packet, if the received address matches a configured and valid endpoint one, the USB peripheral accesses the contents of ADDRn_TX and COUNTn_TX locations inside buffer descriptor table entry related to the addressed endpoint. The content of these locations is stored in its internal 16 bit registers ADDR and COUNT (not accessible by software). The packet memory is accessed again to read the first word to be transmitted (refer to *Structure and usage of packet buffers*) and starts sending a DATA0 or DATA1 PID according to USB_EPnR bit DTOG_TX. When the PID is completed, the first byte from the word, read from buffer memory, is loaded into the output shift register to be transmitted on the USB bus. After the last data byte is transmitted, the computed CRC is sent. If the addressed endpoint is not valid, a NAK or STALL handshake packet is sent instead of the data packet, according to STAT_TX bits in the USB_EPnR register.

The ADDR internal register is used as a pointer to the current buffer memory location while COUNT is used to count the number of remaining bytes to be transmitted. Each word read from the packet buffer memory is transmitted over the USB bus starting from the least significant byte. Transmission buffer memory is read starting from the address pointed by ADDRn_TX for COUNTn_TX/2 words. If a transmitted packet is composed of an odd number of bytes, only the lower half of the last word accessed will be used.

On receiving the ACK receipt by the host, the USB_EPnR register is updated in the following way: DTOG_TX bit is toggled, the endpoint is made invalid by setting STAT_TX=10 (NAK) and bit CTR_TX is set. The application software must first identify the endpoint, which is requesting microcontroller attention by examining the EP_ID and DIR bits in the USB_ISTR register. Servicing of the CTR_TX event starts clearing the interrupt bit; the application software then prepares another buffer full of data to be sent, updates the COUNTn_TX table location with the number of byte to be transmitted during the next transfer, and finally sets STAT_TX to '11 (VALID) to re-enable transmissions. While the STAT_TX bits are equal to '10 (NAK), any IN request addressed to that endpoint is NAKed, indicating a flow control condition: the USB host will retry the transaction until it succeeds. It is mandatory to execute the sequence of operations in the above mentioned order to avoid losing the notification of a second IN transaction addressed to the same endpoint immediately following the one which triggered the CTR interrupt.

OUT and SETUP packets (data reception)

These two tokens are handled by the USB peripheral more or less in the same way; the differences in the handling of SETUP packets are detailed in the following paragraph about control transfers. When receiving an OUT/SETUP PID, if the address matches a valid endpoint, the USB peripheral accesses the contents of the ADDRn_RX and COUNTn_RX locations inside the buffer descriptor table entry related to the addressed endpoint. The content of the ADDRn_RX is stored directly in its internal register ADDR. While COUNT is now reset and the values of BL_SIZE and NUM_BLOCK bit fields, which are read within COUNTn_RX content are used to initialize BUF_COUNT, an internal 16 bit counter, which is used to check the buffer overrun condition (all these internal registers are not accessible by software). Data bytes subsequently received by the USB peripheral are packed in words (the first byte received is stored as least significant byte) and then transferred to the packet buffer starting from the address contained in the internal ADDR register while BUF_COUNT is decremented and COUNT is incremented at each byte transfer. When the end of DATA packet is detected, the correctness of the received CRC is tested and only if no errors occurred during the reception, an ACK handshake packet is sent back to the transmitting host.

In case of wrong CRC or other kinds of errors (bit-stuff violations, frame errors, etc.), data bytes are still copied in the packet memory buffer, at least until the error detection point, but ACK packet is not sent and the ERR bit in USB_ISTR register is set. However, there is usually no software action required in this case: the USB peripheral recovers from reception errors and remains ready for the next transaction to come. If the addressed endpoint is not valid, a NAK or STALL handshake packet is sent instead of the ACK, according to bits STAT_RX in the USB_EPnR register and no data is written in the reception memory buffers.

Reception memory buffer locations are written starting from the address contained in the ADDRn_RX for a number of bytes corresponding to the received data packet length, CRC included (i.e. data payload length + 2), or up to the last allocated memory location, as defined by BL_SIZE and NUM_BLOCK, whichever comes first. In this way, the USB peripheral never writes beyond the end of the allocated reception memory buffer area. If the length of the data packet payload (actual

number of bytes used by the application) is greater than the allocated buffer, the USB peripheral detects a buffer overrun condition. In this case, a STALL handshake is sent instead of the usual ACK to notify the problem to the host, no interrupt is generated and the transaction is considered failed.

When the transaction is completed correctly, by sending the ACK handshake packet, the internal COUNT register is copied back in the COUNTn_RX location inside the buffer description table entry, leaving unaffected BL_SIZE and NUM_BLOCK fields, which normally do not require to be re-written, and the USB_EPnR register is updated in the following way: DTOG_RX bit is toggled, the endpoint is made invalid by setting STAT_RX = '10 (NAK) and bit CTR_RX is set. If the transaction has failed due to errors or buffer overrun condition, none of the previously listed actions take place. The application software must first identify the endpoint, which is requesting microcontroller attention by examining the EP_ID and DIR bits in the USB_ISTR register. The CTR_RX event is serviced by first determining the transaction type (SETUP bit in the USB_EPnR register); the application software must clear the interrupt flag bit and get the number of received bytes reading the COUNTn_RX location inside the buffer description table entry related to the endpoint being processed. After the received data is processed, the application software should set the STAT_RX bits to '11 (Valid) in the USB_EPnR, enabling further transactions. While the STAT_RX bits are equal to '10 (NAK), any OUT request addressed to that endpoint is NAKed, indicating a flow control condition: the USB host will retry the transaction until it succeeds. It is mandatory to execute the sequence of operations in the above mentioned order to avoid losing the notification of a second OUT transaction addressed to the same endpoint following immediately the one which triggered the CTR interrupt.

Control transfers

Control transfers are made of a SETUP transaction, followed by zero or more data stages, all of the same direction, followed by a status stage (a zero-byte transfer in the opposite direction). SETUP transactions are handled by control endpoints only and are very similar to OUT ones (data reception) except that the values of DTOG_TX and DTOG_RX bits of the addressed endpoint registers are set to 1 and 0 respectively, to initialize the control transfer, and both STAT_TX and STAT_RX are set to '10 (NAK) to let software decide if subsequent transactions must be IN or OUT depending on the SETUP contents. A control endpoint must check SETUP bit in the USB_EPnR register at each CTR_RX event to distinguish normal OUT transactions from SETUP ones. A USB device can determine the number and direction of data stages by interpreting the data transferred in the SETUP stage, and is required to STALL the transaction in the case of errors. To do so, at all data stages before the last, the unused direction should be set to STALL, so that, if the host reverses the transfer direction too soon, it gets a STALL as a status stage.

While enabling the last data stage, the opposite direction should be set to NAK, so that, if the host reverses the transfer direction (to perform the status stage) immediately, it is kept waiting for the completion of the control operation. If the control operation completes successfully, the software will change NAK to VALID, otherwise to STALL. At the same time, if the status stage will be an OUT, the STATUS_OUT (EP_KIND in the USB_EPnR register) bit should be set, so that an error is generated if a status transaction is performed with not-zero data. When the status transaction is serviced, the application clears the STATUS_OUT bit and sets STAT_RX to VALID (to accept a new command) and STAT_TX to NAK (to delay a possible status stage immediately following the next setup). Since the USB specification states that a SETUP packet cannot be answered with a handshake different from ACK, eventually aborting a previously issued command to start the new one, the USB logic doesn't allow a control endpoint to answer with a NAK or STALL packet to a SETUP token received from the host.

When the STAT_RX bits are set to '01 (STALL) or '10 (NAK) and a SETUP token is received, the USB accepts the data, performing the required data transfers and sends back an ACK handshake. If that endpoint has a previously issued CTR_RX request not yet acknowledged by the application (i.e. CTR_RX bit is still set from a previously completed reception), the USB discards the SETUP transaction and does not answer with any handshake packet regardless of its state, simulating a reception error and forcing the host to send the SETUP token again. This is done to avoid losing the notification of a SETUP transaction addressed to the same endpoint immediately following the transaction, which triggered the CTR_RX interrupt.

16.4.3 Double-buffered endpoints

All different endpoint types defined by the USB standard represent different traffic models, and describe the typical requirements of different kind of data transfer operations. When large portions of data are to be transferred between the host PC and the USB function, the bulk endpoint type is the most suited model. This is because the host schedules bulk transactions so as to fill all the available bandwidth in the frame, maximizing the actual transfer rate as long as the USB function is ready to handle a bulk transaction addressed to it. If the USB function is still busy with the previous transaction when the next one arrives, it will answer with a NAK handshake and the host PC will issue the same transaction again until the USB function is ready to handle it, reducing the actual transfer rate due to the bandwidth occupied by re-transmissions. For this reason, a dedicated feature called ‘double-buffering’ can be used with bulk endpoints.

When ‘double-buffering’ is activated, data toggle sequencing is used to select, which buffer is to be used by the USB peripheral to perform the required data transfers, using both ‘transmission’ and ‘reception’ packet memory areas to manage buffer swapping on each successful transaction in order to always have a complete buffer to be used by the application, while the USB peripheral fills the other one. For example, during an OUT transaction directed to a ‘reception’ double-buffered bulk endpoint, while one buffer is being filled with new data coming from the USB host, the other one is available for the microcontroller software usage (the same would happen with a ‘transmission’ double-buffered bulk endpoint and an IN transaction).

Since the swapped buffer management requires the usage of all 4 buffer description table locations hosting the address pointer and the length of the allocated memory buffers, the USB_EPnR registers used to implement double-buffered bulk endpoints are forced to be used as unidirectional ones. Therefore, only one STAT bit pair must be set at a value different from ‘00 (Disabled): STAT_RX if the double-buffered bulk endpoint is enabled for reception, STAT_TX if the double-buffered bulk endpoint is enabled for transmission. In case it is required to have double-buffered bulk endpoints enabled both for reception and transmission, two USB_EPnR registers must be used.

To exploit the double-buffering feature and reach the highest possible transfer rate, the endpoint flow control structure, described in previous chapters, has to be modified, in order to switch the endpoint status to NAK only when a buffer conflict occurs between the USB peripheral and application software, instead of doing it at the end of each successful transaction. The memory buffer which is currently being used by the USB peripheral is defined by the DTOG bit related to the endpoint direction: DTOG_RX (bit 14 of USB_EPnR register) for ‘reception’ double-buffered bulk endpoints or DTOG_TX (bit 6 of USB_EPnR register) for ‘transmission’ double-buffered bulk endpoints. To implement the new flow control scheme, the USB peripheral should know which packet buffer is currently in use by the application software, so to be aware of any conflict. Since in the USB_EPnR register, there are two DTOG bits but only one is used by USB peripheral for data and buffer sequencing (due to the unidirectional constraint required by double-buffering feature) the other one can be used by the application software to show which buffer it is currently using. This new buffer flag is called SW_BUF. In the following table the correspondence between USB_EPnR register bits and DTOG/SW_BUF definition is explained, for the cases of ‘transmission’ and ‘reception’ double-buffered bulk endpoints.

Table 64 Double-buffering buffer flag definition

Buffer flag	‘Transmission’ endpoint	‘Reception’ endpoint
DTOG	DTOG_TX (USB_EPnRbit 6)	DTOG_RX (USB_EPnRbit 14)
SW_BUF	USB_EPnR bit 14	USB_EPnR bit 6

The memory buffer which is currently being used by the USB peripheral is defined by DTOG buffer flag, while the buffer currently in use by application software is identified by SW_BUF buffer flag. The relationship between the buffer flag value and the used packet buffer is the same in both cases, and it is listed in the following table.

Table 65 Bulk double-buffering memory buffers usage

Endpoint Type	DTOG	SW_BUF	Packet buffer used by USB Peripheral	Packet buffer used by Application Software
IN	0	1	ADDRn_TX_0 / COUNTn_TX_0	ADDRn_TX_1 / COUNTn_TX_1
	1	0	ADDRn_TX_1 / COUNTn_TX_1	ADDRn_TX_0 / COUNTn_TX_0
	0	1	None ⁽¹⁾	ADDRn_TX_0 / COUNTn_TX_0
	1	0	None ⁽¹⁾	ADDRn_TX_0 / COUNTn_TX_0
OUT	0	1	ADDRn_RX_0 / COUNTn_RX_0	ADDRn_RX_1 / COUNTn_RX_1
	1	0	ADDRn_RX_1 / COUNTn_RX_1	ADDRn_RX_0 / COUNTn_RX_0
	0	1	None ⁽¹⁾	ADDRn_RX_0 / COUNTn_RX_0
	1	0	None ⁽¹⁾	ADDRn_RX_0 / COUNTn_RX_0

1. Endpoint in NAK Status

Double-buffering feature for a bulk endpoint is activated by:

- Writing EP_TYPE bit field at '00 in its USB_EPnR register, to define the endpoint as a bulk, and
- Setting EP_KIND bit at '1 (DBL_BUF), in the same register.

The application software is responsible for DTOG and SW_BUF bits initialization according to the first buffer to be used; this has to be done considering the special toggle-only property that these two bits have. The end of the first transaction occurring after having set DBL_BUF, triggers the special flow control of double-buffered bulk endpoints, which is used for all other transactions addressed to this endpoint until DBL_BUF remain set. At the end of each transaction the CTR_RX or CTR_TX bit of the addressed endpoint USB_EPnR register is set, depending on the enabled direction. At the same time, the affected DTOG bit in the USB_EPnR register is hardware toggled making the USB peripheral buffer swapping completely software independent. Unlike common transactions, and the first one after DBL_BUF setting, STAT bit pair is not affected by the transaction termination and its value remains '11 (Valid). However, as the token packet of a new transaction is received, the actual endpoint status will be masked as '10 (NAK) when a buffer conflict between the USB peripheral and the application software is detected (this condition is identified by DTOG and SW_BUF having the same value, see Table 65). The application software responds to the CTR event notification by clearing the interrupt flag and starting any required handling of the completed transaction. When the application packet buffer usage is over, the software toggles the SW_BUF bit, writing '1 to it, to notify the USB peripheral about the availability of that buffer. In this way, the number of NAK-ed transactions is limited only by the application elaboration time of a transaction data: if the elaboration time is shorter than the time required to complete a transaction on the USB bus, no re-transmissions due to flow control will take place and the actual transfer rate will be limited only by the host PC.

16.4.4 Isochronous transfers

The USB standard supports full speed peripherals requiring a fixed and accurate data production/consume frequency, defining this kind of traffic as 'Isochronous'. Typical examples of this data are: audio samples, compressed video streams, and in general any sort of sampled data having strict requirements for the accuracy of delivered frequency. When an endpoint is defined to be 'isochronous' during the enumeration phase, the host allocates in the frame the required bandwidth and delivers exactly one IN or OUT packet each frame, depending on endpoint direction. To limit the bandwidth requirements, no re-transmission of failed transactions is possible for Isochronous traffic; this leads to the fact that an isochronous transaction does not have a handshake phase and no ACK packet is expected or sent after the data packet. For the same reason, Isochronous transfers do not support data toggle sequencing and always use DATA0 PID to start any data packet.

The Isochronous behavior for an endpoint is selected by setting the EP_TYPE bits at '10 in its USB_EPnR register; since there is no handshake phase the only legal values for the STAT_RX/STAT_TX bit pairs are '00 (Disabled) and '11 (Valid), any other value will produce results not compliant to USB standard. Isochronous endpoints implement double-buffering to ease application software development, using both 'transmission' and 'reception' packet memory areas to manage buffer swapping on each successful transaction in order to have always a complete buffer to be used by the application, while the USB peripheral fills the other.

The memory buffer which is currently used by the USB peripheral is defined by the DTOG bit related to the endpoint direction (DTOG_RX for 'reception' isochronous endpoints, DTOG_TX for 'transmission' isochronous endpoints, both in the related USB_EPnR register) according to Table 66.

Table 66 Isochronous memory buffers usage

Endpoint Type	DTOG bit value	Packet buffer used by the USB peripheral	Packet buffer used by the application software
IN	0	ADDRn_TX_0/ COUNTn_TX_0	ADDRn_TX_1/ COUNTn_TX_1
	1	ADDRn_TX_1/ COUNTn_TX_1	ADDRn_TX_0/ COUNTn_TX_0
OUT	0	ADDRn_RX_0/ COUNTn_RX_0	ADDRn_RX_1/ COUNTn_RX_1
	1	ADDRn_RX_1/ COUNTn_RX_1	ADDRn_RX_0/ COUNTn_RX_0

As it happens with double-buffered bulk endpoints, the USB_EPnR registers used to implement Isochronous endpoints are forced to be used as unidirectional ones. In case it is required to have Isochronous endpoints enabled both for reception and transmission, two USB_EPnR registers must be used.

The application software is responsible for the DTOG bit initialization according to the first buffer to be used; this has to be done considering the special toggle-only property that these two bits have. At the end of each transaction, the CTR_RX or CTR_TX bit of the addressed endpoint USB_EPnR register is set, depending on the enabled direction. At the same time, the affected DTOG bit in the USB_EPnR register is hardware toggled making buffer swapping completely software independent. STAT bit pair is not affected by transaction completion; since no flow control is possible for Isochronous transfers due to the lack of handshake phase, the endpoint remains always '11 (Valid). CRC errors or buffer-overflow conditions occurring during Isochronous OUT transfers are anyway considered as correct transactions and they always trigger an CTR_RX event. However, CRC errors will anyway set the ERR bit in the USB_ISTR register to notify the software of the possible data corruption.

16.4.5 Suspend/Resume events

The USB standard defines a special peripheral state, called SUSPEND, in which the average current drawn from the USB bus must not be greater than 2.5 mA. This requirement is of fundamental importance for bus-powered devices, while self-powered devices are not required to comply to this strict power consumption constraint. In suspend mode, the host PC sends the notification to not send any traffic on the USB bus for more than 3mS: since a SOF packet must be sent every mS during normal operations, the USB peripheral detects the lack of 3 consecutive SOF packets as a suspend request from the host PC and set the SUSP bit to '1 in USB_ISTR register, causing an interrupt if enabled. Once the device is suspended, its normal operation can be restored by a so called RESUME sequence, which can be started from the host PC or directly from the peripheral itself, but it is always terminated by the host PC. The suspended USB peripheral must be anyway able to detect a RESET sequence, reacting to this event as a normal USB reset event.

The actual procedure used to suspend the USB peripheral is device dependent since according to the device composition, different actions may be required to reduce the total consumption.

A brief description of a typical suspend procedure is provided below, focused on the USB- related aspects of the application software routine responding to the SUSP notification of the USB peripheral:

1. Set the FSUSP bit in the USB_CNTR register to 1. This action activates the suspend mode within the USB peripheral. As soon as the suspend mode is activated, the check on SOF reception is disabled to avoid any further SUSP interrupts being issued while the USB is suspended.
2. Remove or reduce any static power consumption in blocks different from the USB peripheral.
3. Set LP_MODE bit in USB_CNTR register to 1 to remove static power consumption in the analog USB transceivers but keeping them able to detect resume activity.
4. Optionally turn off external oscillator and device PLL to stop any activity inside the device.

When an USB event occurs while the device is in SUSPEND mode, the RESUME procedure must be invoked to restore nominal clocks and regain normal USB behavior. Particular care must be taken to insure that this process does not take more than 10mS when the waking event is an USB reset sequence (See “Universal Serial Bus Specification” for more details). The start of a resume or reset sequence, while the USB peripheral is suspended, clears the LP_MODE bit in USB_CNTR register asynchronously. Even if this event can trigger an WKUP interrupt if enabled, the use of an interrupt response routine must be carefully evaluated because of the long latency due to system clock restart; to have the shorter latency before re-activating the nominal clock it is suggested to put the resume procedure just after the end of the suspend one, so its code is immediately executed as soon as the system clock restarts. To prevent ESD discharges or any other kind of noise from waking-up the system (the exit from suspend mode is an asynchronous event), a suitable analog filter on data line status is activated during suspend; the filter width is about 70ns.

The following is a list of actions a resume procedure should address:

1. Optionally turn on external oscillator and/or device PLL.
2. Clear FSUSP bit of USB_CNTR register.
3. If the resume triggering event has to be identified, bits RXDP and RXDM in the USB_FNR register can be used according to Table 67, which also lists the intended software action in all the cases. If required, the end of resume or reset sequence can be detected monitoring the status of the above mentioned bits by checking when they reach the “10” configuration, which represent the Idle bus state; moreover at the end of a reset sequence the RESET bit in USB_ISTR register is set to 1, issuing an interrupt if enabled, which should be handled as usual.

Table 67 Resume event detection

[RXDP,RXDM] status	Wakeup event	Required resume software action
00	Root reset	None
01	None (noise on bus)	Go back in Suspend mode
10	Root resume	None
11	Not allowed (noise on bus)	Go back in Suspend mode

A device may require to exit from suspend mode as an answer to particular events not directly related to the USB protocol (e.g. a mouse movement wakes up the whole system). In this case, the resume sequence can be started by setting the RESUME bit in the USB_CNTR register to ‘1 and resetting it to 0 after an interval between 1 mS and 15 mS (this interval can be timed using ESOF interrupts, occurring with a 1mS period when the system clock is running at nominal frequency). Once the RESUME bit is clear, the resume sequence will be completed by the host PC and its end can be monitored again using the RXDP and RXDM bits in the USB_FNR register.

Note: The RESUME bit must be anyway used only after the USB peripheral has been put in suspend mode, setting the FSUSP bit in USB_CNTR register to 1.

16.5 USB registers

The USB peripheral registers can be divided into the following groups:

- Common Registers: Interrupt and Control registers
- Endpoint Registers: Endpoint configuration and status
- Buffer Descriptor Table: Location of packet memory used to locate data buffers

All register addresses are expressed as offsets with respect to the USB peripheral registers base address 0x4000 5C00, except the buffer descriptor table locations, which starts at the address specified by the USB_BTABLER register. Due to the common limitation of APB1 bridges on word

addressability, all register addresses are aligned to 32-bit word boundaries although they are 16-bit wide. The same address alignment is used to access packet buffer memory locations, which are located starting from 0x4000 6000.

Refer to *Section 1.1* for a list of abbreviations used in register descriptions.

The peripheral registers can be accessed by half-words (16-bit) or words (32-bit).

16.5.1 Common registers

These registers affect the general behavior of the USB peripheral defining operating mode, interrupt handling, device address and giving access to the current frame number updated by the host PC.

USB control register (USB_CNTR)

Address offset: 0x40

Reset value: 0x0003

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CTRM	PMA OVRM	ERR M	WK UPM	SU SPM	RES ETM	SOF M	ESO FM	Reserved			RES UME	FSU SP	LP MODE	PDMN	FRES
rw	rw	rw	rw	rw	rw	rw	rw				rw	rw	rw	rw	rw
Bit 15		CTRM: Correct transfer interrupt mask 0: Correct Transfer (CTR) Interrupt disabled. 1: CTR Interrupt enabled, an interrupt request is generated when the corresponding bit in the USB_ISTR register is set.													
Bit 14		PMAOVRM: Packet memory area over / underrun interrupt mask 0: PMAOVR Interrupt disabled. 1: PMAOVR Interrupt enabled, an interrupt request is generated when the corresponding bit in the USB_ISTR register is set.													
Bit 13		ERRM: Error interrupt mask 0: ERR Interrupt disabled. 1: ERR Interrupt enabled, an interrupt request is generated when the corresponding bit in the USB_ISTR register is set.													
Bit 12		WKUPM: Wakeup interrupt mask 0: WKUP Interrupt disabled. 1: WKUP Interrupt enabled, an interrupt request is generated when the corresponding bit in the USB_ISTR register is set.													
Bit 11		SUSPM: Suspend mode interrupt mask 0: Suspend Mode Request (SUSP) Interrupt disabled. 1: SUSP Interrupt enabled, an interrupt request is generated when the corresponding bit in the USB_ISTR register is set.													
Bit 10		RESETM: USB reset interrupt mask 0: RESET Interrupt disabled. 1: RESET Interrupt enabled, an interrupt request is generated when the corresponding bit in the USB_ISTR register is set.													
Bit 9		SOFM: Start of frame interrupt mask 0: SOF Interrupt disabled. 1: SOF Interrupt enabled, an interrupt request is generated when the corresponding bit in the USB_ISTR register is set.													
Bit 8		ESOFM: Expected start of frame interrupt mask 0: Expected Start of Frame (ESOF) Interrupt disabled. 1: ESOF Interrupt enabled, an interrupt request is generated when the corresponding bit in the USB_ISTR register is set.													
Bits [7:5]		Reserved.													
Bit 4		RESUME: Resume request The microcontroller can set this bit to send a Resume signal to the host. It must be activated, according to USB specifications, for no less than 1 mS and no more than 15 mS after which the Host PC is ready to drive the resume sequence up to its end.													
Bit 3		FSUSP: Force suspend Software must set this bit when the SUSP interrupt is received, which is issued when no traffic is received by the USB peripheral for 3 mS 0: No effect. 1: Enter suspend mode. Clocks and static power dissipation in the analog transceiver are left unaffected. If suspend power consumption is a requirement (bus-powered device), the application software should set the LP_MODE bit after FSUSP as explained below.													

Bit 2	LP_MODE: Low-power mode This mode is used when the suspend-mode power constraints require that all static power dissipation is avoided, except the one required to supply the external pull-up resistor. This condition should be entered when the application is ready to stop all system clocks, or reduce their frequency in order to meet the power consumption requirements of the USB suspend condition. The USB activity during the suspend mode (WKUP event) asynchronously resets this bit (it can also be reset by software). 0: No Low-power mode. 1: Enter Low-power mode.
Bit 1	PDWN: Power down This bit is used to completely switch off all USB-related analog parts if it is required to completely disable the USB peripheral for any reason. When this bit is set, the USB peripheral is disconnected from the transceivers and it cannot be used. 0: Exit Power Down. 1: Enter Power down mode.
Bit 0	FRES: Force USB Reset 0: Clear USB reset. 1: Force a reset of the USB peripheral, exactly like a RESET signalling on the USB. The USB peripheral is held in RESET state until software clears this bit. A “USB-RESET” interrupt is generated, if enabled.

USB interrupt status register (USB_ISTR)

Address offset: 0x44

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CTR	PMAOVR	ERR	WKUP	SUSP	RESET	SOF	ESOF		Reserved		DIR				EP_ID[3: 0]
r	rc w0	rc w0	rc w0	rc w0	rc w0	rc w0	rc w0		res		r	r	r	r	r

This register contains the status of all the interrupt sources allowing application software to determine, which events caused an interrupt request.

The upper part of this register contains single bits, each of them representing a specific event. These bits are set by the hardware when the related event occurs; if the corresponding bit in the USB_CNTR register is set, a generic interrupt request is generated. The interrupt routine, examining each bit, will perform all necessary actions, and finally it will clear the serviced bits. If any of them is not cleared, the interrupt is considered to be still pending, and the interrupt line will be kept high again. If several bits are set simultaneously, only a single interrupt will be generated.

Endpoint transaction completion can be handled in a different way to reduce interrupt response latency. The CTR bit is set by the hardware as soon as an endpoint successfully completes a transaction, generating a generic interrupt request if the corresponding bit in USB_CNTR is set. An endpoint dedicated interrupt condition is activated independently from the CTRM bit in the USB_CNTR register. Both interrupt conditions remain active until software clears the pending bit in the corresponding USB_EPnR register (the CTR bit is actually a read only bit). For endpoint-related interrupts, the software can use the Direction of Transaction (DIR) and EP_ID read-only bits to identify, which endpoint made the last interrupt request and called the corresponding interrupt service routine.

The user can choose the relative priority of simultaneously pending USB_ISTR events by specifying the order in which software checks USB_ISTR bits in an interrupt service routine. Only the bits related to events, which are serviced, are cleared. At the end of the service routine, another interrupt will be requested, to service the remaining conditions.

To avoid spurious clearing of some bits, it is recommended to clear them with a load instruction where all bits which must not be altered are written with 1, and all bits to be cleared are written with '0 (these bits can only be cleared by software). Read-modify-write cycles should be avoided because between the read and the write operations another bit could be set by the hardware and the next write will clear it before the microprocessor has the time to serve the event.

Bit 15	CTR: Correct transfer This bit is set by the hardware to indicate that an endpoint has successfully completed a transaction; using DIR and EP_ID bits software can determine which endpoint requested the interrupt. This bit is
--------	--

	read-only.
Bit 14	PMAOVR: Packet memory area over / underrun This bit is set if the microcontroller has not been able to respond in time to an USB memory request. The USB peripheral handles this event in the following way: During reception an ACK handshake packet is not sent, during transmission a bit-stuff error is forced on the transmitted stream; in both cases the host will retry the transaction. The PMAOVR interrupt should never occur during normal operations. Since the failed transaction is retried by the host, the application software has the chance to speed-up device operations during this interrupt handling, to be ready for the next transaction retry; however this does not happen during Isochronous transfers (no isochronous transaction is anyway retried) leading to a loss of data in this case. This bit is read/write but only '0 can be written and writing '1 has no effect.
Bit 13	ERR: Error This flag is set whenever one of the errors listed below has occurred: NANS: No ANSwer. The timeout for a host response has expired. CRC: Cyclic Redundancy Check error. One of the received CRCs, either in the token or in the data, was wrong. BST: Bit Stuffing error. A bit stuffing error was detected anywhere in the PID, data, and/or CRC. FVIO: Framing format Violation. A non-standard frame was received (EOP not in the right place, wrong token sequence, etc.). The USB software can usually ignore errors, since the USB peripheral and the PC host manage retransmission in case of errors in a fully transparent way. This interrupt can be useful during the software development phase, or to monitor the quality of transmission over the USB bus, to flag possible problems to the user (e.g. loose connector, too noisy environment, broken conductor in the USB cable and so on). This bit is read/write but only '0 can be written and writing '1 has no effect.
Bit 12	WKUP: Wakeup This bit is set to 1 by the hardware when, during suspend mode, activity is detected that wakes up the USB peripheral. This event asynchronously clears the LP_MODE bit in the CTLR register and activates the USB_WAKEUP line, which can be used to notify the rest of the device (e.g. wakeup unit) about the start of the resume process. This bit is read/write but only '0 can be written and writing '1 has no effect.
Bit 11	SUSP: Suspend mode request This bit is set by the hardware when no traffic has been received for 3mS, indicating a suspend mode request from the USB bus. The suspend condition check is enabled immediately after any USB reset and it is disabled by the hardware when the suspend mode is active (FSUSP=1) until the end of resume sequence. This bit is read/write but only '0 can be written and writing '1 has no effect.
Bit 10	RESET: USB reset request Set when the USB peripheral detects an active USB RESET signal at its inputs. The USB peripheral, in response to a RESET, just resets its internal protocol state machine, generating an interrupt if RESETM enable bit in the USB_CNTR register is set. Reception and transmission are disabled until the RESET bit is cleared. All configuration registers do not reset: the microcontroller must explicitly clear these registers (this is to ensure that the RESET interrupt can be safely delivered, and any transaction immediately followed by a RESET can be completed). The function address and endpoint registers are reset by an USB reset event. This bit is read/write but only '0 can be written and writing '1 has no effect.
Bit 9	SOF: Start of frame This bit signals the beginning of a new USB frame and it is set when a SOF packet arrives through the USB bus. The interrupt service routine may monitor the SOF events to have a 1 mS synchronization event to the USB host and to safely read the USB_FNR register which is updated at the SOF packet reception (this could be useful for isochronous applications). This bit is read/write but only '0 can be written and writing '1 has no effect.
Bit 8	ESOF: Expected start of frame This bit is set by the hardware when an SOF packet is expected but not received. The host sends an SOF packet each mS, but if the hub does not receive it properly, the Suspend Timer issues this interrupt. If three consecutive ESOF interrupts are generated (i.e. three SOF packets are lost) without any traffic occurring in between, a SUSP interrupt is generated. This bit is set even when the missing SOF packets occur while the Suspend Timer is not yet locked. This bit is read/write but only '0 can be written and writing '1 has no effect.
Bits 7:5	Reserved
Bit 4	DIR: Direction of transaction This bit is written by the hardware according to the direction of the successful transaction, which generated the interrupt request. If DIR bit=0, CTR_TX bit is set in the USB_EPnR register related to the interrupting endpoint. The interrupting transaction is of IN type (data transmitted by the USB peripheral to the host PC). If DIR bit=1, CTR_RX bit or both CTR_TX/CTR_RX are set in the USB_EPnR register related to the interrupting endpoint. The interrupting transaction is of OUT type (data received by the USB peripheral from the host PC) or two pending transactions are waiting to be processed. This information can be used by the application software to access the USB_EPnR bits related to the triggering transaction since it represents the direction having the interrupt pending. This bit is read-only.

Bits 3:0	EP_ID[3:0]: Endpoint Identifier These bits are written by the hardware according to the endpoint number, which generated the interrupt request. If several endpoint transactions are pending, the hardware writes the endpoint identifier related to the endpoint having the highest priority defined in the following way: Two endpoint sets are defined, in order of priority: Isochronous and double-buffered bulk endpoints are considered first and then the other endpoints are examined. If more than one endpoint from the same set is requesting an interrupt, the EP_ID bits in USB_ISTR register are assigned according to the lowest requesting endpoint register, EP0R having the highest priority followed by EP1R and so on. The application software can assign a register to each endpoint according to this priority scheme, so as to order the concurring endpoint requests in a suitable way. These bits are read only.
----------	---

USB frame number register (USB_FNR)

Address offset: 0x48

Reset value: 0x0XXX where X is undefined

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RXDP	RXDM	LCK	LSOF[1: 0]	FN[10: 0]											
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bit 15	RXDP: Receive data + line status This bit can be used to observe the status of received data plus upstream port data line. It can be used during end-of-suspend routines to help determining the wakeup event.														
Bit 14	RXDM: Receive data - line status This bit can be used to observe the status of received data minus upstream port data line. It can be used during end-of-suspend routines to help determining the wakeup event.														
Bit 13	LCK: Locked This bit is set by the hardware when at least two consecutive SOF packets have been received after the end of an USB reset condition or after the end of an USB resume sequence. Once locked, the frame timer remains in this state until an USB reset or USB suspend event occurs.														
Bits 12:11	LSOF[1:0]: Lost SOF These bits are written by the hardware when an ESOF interrupt is generated, counting the number of consecutive SOF packets lost. At the reception of an SOF packet, these bits are cleared.														
Bits 10:0	FN[10:0]: Frame number This bit field contains the 11-bits frame number contained in the last received SOF packet. The frame number is incremented for every frame sent by the host and it is useful for Isochronous transfers. This bit field is updated on the generation of an SOF interrupt.														

USB device address (USB_DADDR)

Address offset: 0x4C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								EF	ADD[6:0]						
								rw	rw	rw	rw	rw	rw	rw	rw
Bits 15: 8	Reserved														
Bit 7	EF: Enable function This bit is set by the software to enable the USB device. The address of this device is contained in the following ADD[6:0] bits. If this bit is at '0 no transactions are handled, irrespective of the settings of USB_EPnR registers.														
Bits 6:0	ADD[6:0]: Device address These bits contain the USB function address assigned by the host PC during the enumeration process. Both this field and the Endpoint Address (EA) field in the associated USB_EPnR register must match with the information contained in a USB token in order to handle a transaction to the required endpoint.														

Buffer table address (USB_BTABLE)

Address offset: 0x50

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BTABLE[15:3]													Reserved		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	res		
Bits 15: 3		BTABLE[15:3]: Buffer table These bits contain the start address of the buffer allocation table inside the dedicated packet memory.													

	This table describes each endpoint buffer location and size and it must be aligned to an 8 byte boundary (the 3 least significant bits are always '0'). At the beginning of every transaction addressed to this device, the USP peripheral reads the element of this table related to the addressed endpoint, to get its buffer start location and the buffer size (Refer to <i>Structure and usage of packet buffers</i>).
Bits 2:0	Reserved, forced by hardware to 0.

16.5.2 Endpoint-specific registers

The number of these registers varies according to the number of endpoints that the USB peripheral is designed to handle. The USB peripheral supports up to 8 bidirectional endpoints. Each USB device must support a control endpoint whose address (EA bits) must be set to 0. The USB peripheral behaves in an undefined way if multiple endpoints are enabled having the same endpoint number value. For each endpoint, an USB_EPnR register is available to store the endpoint specific information.

USB endpoint n register (USB_EPnR), n=[0..7]

Address offset: 0x00 to 0x1C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CTR_RX	DTOG_RX	STAT_RX [1:0]	SET UP	EP TYPE[1:0]	EP_KIN D	CTR_TX	DTOG_TX	STAT_TX [1:0]	EA[3:0]						
rc w0	t	t	t	r	rw	rw	rw	rc w0	t	t	t	rw	rw	rw	rw

They are also reset when an USB reset is received from the USB bus or forced through bit FRES in the CTRLR register, except the CTR_RX and CTR_TX bits, which are kept unchanged to avoid missing a correct packet notification immediately followed by an USB reset event. Each endpoint has its USB_EPnR register where *n* is the endpoint identifier.

Read-modify-write cycles on these registers should be avoided because between the read and the write operations some bits could be set by the hardware and the next write would modify them before the CPU has the time to detect the change. For this purpose, all bits affected by this problem have an 'invariant' value that must be used whenever their modification is not required. It is recommended to modify these registers with a load instruction where all the bits, which can be modified only by the hardware, are written with their 'invariant' value.

Bit 15	CTR_RX: Correct Transfer for reception This bit is set by the hardware when an OUT/SETUP transaction is successfully completed on this endpoint; the software can only clear this bit. If the CTRM bit in USB_CNTR register is set accordingly, a generic interrupt condition is generated together with the endpoint related interrupt condition, which is always activated. The type of occurred transaction, OUT or SETUP, can be determined from the SETUP bit described below. A transaction ended with a NAK or STALL handshake does not set this bit, since no data is actually transferred, as in the case of protocol errors or data toggle mismatches. This bit is read/write but only '0' can be written, writing 1 has no effect.
Bit 14	DTOG_RX: Data Toggle, for reception transfers If the endpoint is not Isochronous, this bit contains the expected value of the data toggle bit (0=DATA0, 1=DATA1) for the next data packet to be received. Hardware toggles this bit, when the ACK handshake is sent to the USB host, following a data packet reception having a matching data PID value; if the endpoint is defined as a control one, hardware clears this bit at the reception of a SETUP PID addressed to this endpoint. If the endpoint is using the double-buffering feature this bit is used to support packet buffer swapping too (Refer to <i>Section 16.4.3: Double-buffered endpoints</i>). If the endpoint is Isochronous, this bit is used only to support packet buffer swapping since no data toggling is used for this sort of endpoints and only DATA0 packet are transmitted (Refer to <i>Section 16.4.4: Isochronous transfers</i>). Hardware toggles this bit just after the end of data packet reception, since no handshake is used for isochronous transfers. This bit can also be toggled by the software to initialize its value (mandatory when the endpoint is not a control one) or to force specific data toggle/packet buffer usage. When the application software writes '0', the value of DTOG_RX remains unchanged, while writing '1' makes the bit value toggle. This bit is read/write but it can be only toggled by writing 1.
Bits 13:12	STAT_RX [1:0]: Status bits, for reception transfers These bits contain information about the endpoint status, which are listed in Table 68. These bits can be toggled by software to initialize their value. When the application software writes '0', the value remains

	unchanged, while writing '1 makes the bit value toggle. Hardware sets the STAT_RX bits to NAK when a correct transfer has occurred (CTR_RX=1) corresponding to a OUT or SETUP (control only) transaction addressed to this endpoint, so the software has the time to elaborate the received data before it acknowledge a new transaction Double-buffered bulk endpoints implement a special transaction flow control, which control the status based upon buffer availability condition (Refer to <i>Section 16.4.3: Double-buffered endpoints</i>) If the endpoint is defined as Isochronous, its status can be only "VALID" or "DISABLED", so that the hardware cannot change the status of the endpoint after a successful transaction. If the software sets the STAT_RX bits to 'STALL' or 'NAK' for an Isochronous endpoint, the USB peripheral behavior is not defined. These bits are read/write but they can be only toggled by writing '1.
Bit 11	SETUP: Setup transaction completed This bit is read-only and it is set by the hardware when the last completed transaction is a SETUP. This bit changes its value only for control endpoints. It must be examined, in the case of a successful receive transaction (CTR_RX event), to determine the type of transaction occurred. To protect the interrupt service routine from the changes in SETUP bits due to next incoming tokens, this bit is kept frozen while CTR_RX bit is at 1; its state changes when CTR_RX is at 0. This bit is read-only.
Bits 10:9	EP_TYPE[1:0]: Endpoint type These bits configure the behavior of this endpoint as described in Table 69. Endpoint 0 must always be a control endpoint and each USB function must have at least one control endpoint which has address 0, but there may be other control endpoints if required. Only control endpoints handle SETUP transactions, which are ignored by endpoints of other kinds. SETUP transactions cannot be answered with NAK or STALL. If a control endpoint is defined as NAK, the USB peripheral will not answer, simulating a receive error, in the receive direction when a SETUP transaction is received. If the control endpoint is defined as STALL in the receive direction, then the SETUP packet will be accepted anyway, transferring data and issuing the CTR interrupt. The reception of OUT transactions is handled in the normal way, even if the endpoint is a control one. Bulk and interrupt endpoints have very similar behavior and they differ only in the special feature available using the EP_KIND configuration bit. The usage of Isochronous endpoints is explained in <i>Section 16.4.4: Isochronous transfers</i>
Bit 8	EP_KIND: Endpoint kind The meaning of this bit depends on the endpoint type configured by the EP_TYPE bits. Table 70 summarizes the different meanings. DBL_BUF: This bit is set by the software to enable the double-buffering feature for this bulk endpoint. The usage of double-buffered bulk endpoints is explained in <i>Section 16.4.3: Double-buffered endpoints</i>. STATUS_OUT: This bit is set by the software to indicate that a status out transaction is expected: in this case all OUT transactions containing more than zero data bytes are answered 'STALL' instead of 'ACK'. This bit may be used to improve the robustness of the application to protocol errors during control transfers and its usage is intended for control endpoints only. When STATUS_OUT is reset, OUT transactions can have any number of bytes, as required.
Bit 7	CTR_TX: Correct Transfer for transmission This bit is set by the hardware when an IN transaction is successfully completed on this endpoint; the software can only clear this bit. If the CTRM bit in the USB_CNTR register is set accordingly, a generic interrupt condition is generated together with the endpoint related interrupt condition, which is always activated. A transaction ended with a NAK or STALL handshake does not set this bit, since no data is actually transferred, as in the case of protocol errors or data toggle mismatches. This bit is read/write but only '0 can be written.
Bit 6	DTOG_TX: Data Toggle, for transmission transfers If the endpoint is non-isochronous, this bit contains the required value of the data toggle bit (0=DATA0, 1=DATA1) for the next data packet to be transmitted. Hardware toggles this bit when the ACK handshake is received from the USB host, following a data packet transmission. If the endpoint is defined as a control one, hardware sets this bit to 1 at the reception of a SETUP PID addressed to this endpoint. If the endpoint is using the double buffer feature, this bit is used to support packet buffer swapping too (Refer to <i>Section 16.4.3: Double-buffered endpoints</i>) If the endpoint is Isochronous, this bit is used to support packet buffer swapping since no data toggling is used for this sort of endpoints and only DATA0 packet are transmitted (Refer to <i>Section 16.4.4: Isochronous transfers</i>). Hardware toggles this bit just after the end of data packet transmission, since no handshake is used for Isochronous transfers. This bit can also be toggled by the software to initialize its value (mandatory when the endpoint is not a control one) or to force a specific data toggle/packet buffer usage. When the application software writes '0, the value of DTOG_TX remains unchanged, while writing '1 makes the bit value toggle. This bit is read/write but it can only be toggled by writing 1.
Bits 5:4	STAT_TX [1:0]: Status bits, for transmission transfers These bits contain the information about the endpoint status, listed in Table 71. These bits can be toggled by the software to initialize their value. When the application software writes '0, the value remains unchanged, while writing '1 makes the bit value toggle. Hardware sets the STAT_TX bits to NAK, when

	a correct transfer has occurred (CTR_TX=1) corresponding to a IN or SETUP (control only) transaction addressed to this endpoint. It then waits for the software to prepare the next set of data to be transmitted. Double-buffered bulk endpoints implement a special transaction flow control, which controls the status based on buffer availability condition (Refer to <i>Section 16.4.3: Double-buffered endpoints</i>). If the endpoint is defined as Isochronous, its status can only be “VALID” or “DISABLED”. Therefore, the hardware cannot change the status of the endpoint after a successful transaction. If the software sets the STAT_TX bits to ‘STALL’ or ‘NAK’ for an Isochronous endpoint, the USB peripheral behavior is not defined. These bits are read/write but they can be only toggled by writing ‘1’.
Bits 3:0	EA[3:0]: Endpoint address Software must write in this field the 4-bit address used to identify the transactions directed to this endpoint. A value must be written before enabling the corresponding endpoint.

Table 68 Reception status encoding

STAT_RX[1:0]	Meaning
00	DISABLED: all reception requests addressed to this endpoint are ignored.
01	STALL: the endpoint is stalled and all reception requests result in a STALL handshake.
10	NAK: the endpoint is naked and all reception requests result in a NAK handshake.
11	VALID: this endpoint is enabled for reception.

Table 69 Endpoint type encoding

EP_TYPE[1:0]	Meaning
00	BULK
01	CONTROL
10	ISO
11	INTERRUPT

Table 70 Endpoint kind meaning

EP_TYPE[1:0]	EP_KIND Meaning
00	BULK DBL_BUF
01	CONTROL STATUS_OUT
10	ISO Not used
11	INTERRUPT Not used

Table 71 Transmission status encoding

STAT_TX[1:0]	Meaning
00	DISABLED: all transmission requests addressed to this endpoint are ignored
01	STALL: the endpoint is stalled and all transmission requests result in a STALL handshake.
10	NAK: the endpoint is naked and all transmission requests result in a NAK handshake.
11	VALID: this endpoint is enabled for transmission.

16.5.3 Buffer descriptor table

Although the buffer descriptor table is located inside the packet buffer memory, its entries can be considered as additional registers used to configure the location and size of the packet buffers used to exchange data between the USB macro cell and the CKS32F103xx. Due to the common APB bridge limitation on word addressability, all packet memory locations are accessed by the APB using 32-bit aligned addresses, instead of the actual memory location addresses utilized by the USB peripheral for the USB_BTABLER register and buffer description table locations.

In the following pages two location addresses are reported: the one to be used by application software while accessing the packet memory, and the local one relative to USB Peripheral access. To obtain the correct CKS32F103xx memory address value to be used in the application software while accessing the packet memory, the actual memory location address must be multiplied by two. The first packet memory location is located at 0x4000 6000. The buffer descriptor table entry associated with the USB_EPnR registers is described below.

A thorough explanation of packet buffers and the buffer descriptor table usage can be found in *Structure and usage of packet buffers*.

Transmission buffer address n (USB_ADDRn_TX)

Address offset: [USB_BTABLE] + n×16

USB local address: [USB_BTABLE] + n×8

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDRn_TX[15:1]															-
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	-
Bits 15:1		ADDRn_TX[15:1]: Transmission buffer address These bits point to the starting address of the packet buffer containing data to be transmitted by the endpoint associated with the USB_EPnR register at the next IN token addressed to it.													
Bit 0		Must always be written as '0 since packet memory is word-wide and all packet buffers must be word-aligned.													

Transmission byte count n (USB_COUNTn_TX)

Address offset: [USB_BTABLE] + n×16 + 4

USB local Address: [USB_BTABLE] + n×8 + 2

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-						COUNTn_TX[9:0]									
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 15:10		These bits are not used since packet size is limited by USB specifications to 1023 bytes. Their value is not considered by the USB peripheral.													
Bits 9:0		COUNTn_TX[9:0]: Transmission byte count These bits contain the number of bytes to be transmitted by the endpoint associated with the USB_EPnR register at the next IN token addressed to it.													

Note: Double-buffered and Isochronous IN Endpoints have two USB_COUNTn_TX registers: named USB_COUNTn_TX_1 and USB_COUNTn_TX_0 with the following content.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
-						COUNTn_TX_1[9:0]									
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-						COUNTn_TX_0[9:0]									
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Reception buffer address n (USB_ADDRn_RX)

Address offset: [USB_BTABLE] + n×16 + 8

USB local Address: [USB_BTABLE] + n×8 + 4

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDRn_RX[15:1]															-
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	-
Bits 15:1		ADDRn_RX[15:1]: Reception buffer address These bits point to the starting address of the packet buffer, which will contain the data received by the endpoint associated with the USB_EPnR register at the next OUT/SETUP token addressed to it.													
Bit 0		This bit must always be written as '0 since packet memory is word-wide and all packet buffers must be word-aligned.													

Reception byte count n (USB_COUNTn_RX)

Address offset: [USB_BTABLE] + n×16 + 12

USB local Address: [USB_BTABLE] + n×8 + 6

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BLSIZE	NUM_BLOCK[4:0]					COUNTn_RX[9:0]									
rw	rw	rw	rw	rw	rw	r	r	r	r	r	r	r	r	r	r

This table location is used to store two different values, both required during packet reception. The most significant bits contains the definition of allocated buffer size, to allow buffer overflow detection, while the least significant part of this location is written back by the USB peripheral at the end of reception to give the actual number of received bytes. Due to the restrictions on the number of available bits, buffer size is represented using the number of allocated memory blocks, where block

size can be selected to choose the trade-off between fine-granularity/small-buffer and coarse-granularity/large-buffer. The size of allocated buffer is a part of the endpoint descriptor and it is normally defined during the enumeration process according to its maxPacketSize parameter value (See “Universal Serial Bus Specification”).

Bit 15	BL_SIZE: BLock size This bit selects the size of memory block used to define the allocated buffer area. – If BL_SIZE=0, the memory block is 2 byte large, which is the minimum block allowed in a word-wide memory. With this block size the allocated buffer size ranges from 2 to 62 bytes. – If BL_SIZE=1, the memory block is 32 byte large, which allows to reach the maximum packet length defined by USB specifications. With this block size the allocated buffer size ranges from 32 to 512 bytes, which is the longest packet size allowed by USB standard specifications.
Bits 14:10	NUM_BLOCK[4:0]: Number of blocks These bits define the number of memory blocks allocated to this packet buffer. The actual amount of allocated memory depends on the BL_SIZE value as illustrated in Table 72.
Bits 9:0	COUNTn_RX[9:0]: Reception byte count These bits contain the number of bytes received by the endpoint associated with the USB_EPnR register during the last OUT/SETUP transaction addressed to it.

Note: Double-buffered and Isochronous IN Endpoints have two USB_COUNTn_TX registers: named USB_COUNTn_TX_1 and USB_COUNTn_TX_0 with the following content

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BLSEI ZE_1	NUM_BLOCK_1[4:0]					COUNTn_RX_1[9:0]									
rw	rw	rw	rw	rw	rw	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BLSEI ZE_0	NUM_BLOCK_0[4:0]					COUNTn_RX_0[9:0]									
rw	rw	rw	rw	rw	rw	r	r	r	r	r	r	r	r	r	r

Table 72 Definition of allocated buffer memory

Value of NUM_BLOCK[4:0]	Memory allocated when BL_SIZE=0	Memory allocated when BL_SIZE=1
00000	Not allowed	32 bytes
00001	2 bytes	64 bytes
00010	4 bytes	96 bytes
00011	6 bytes	128 bytes
...	...	...
01111	30 bytes	512 bytes
10000	32 bytes	Reserved
10001	34 bytes	Reserved
10010	36 bytes	Reserved
...	...	...
11110	60 bytes	Reserved
11111	62 bytes	Reserved

16.5.4 USB register map

Table 73 USB register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
00h	USB_EP0R	Reserved																CTR_RX	DTOG_RX		STAT_RX [1:0]		SETUP		EPTYPE [1:0]		EP_KIND		CTR_TX		DTOG_TX		STAT_TX [1:0]		EA[3:0]			

14h	Reset value	USB_EP5R	Reserved	0Ch	Reset value	USB_EP3R	Reserved	08h	Reset value	USB_EP2R	Reserved	04h	Reset value	USB_EP1R	Reset value
	CTR_RX	0			CTR_RX	0			CTR_RX	0			CTR_RX	0	CTR_RX
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0										

18h	Reset value	USB_EP6R	Reserved	0	CTR_RX	0	DTOG_RX	0	STAT_RX [1:0]	0	SETUP	0	EPTYPE [1:0]	0	EP_KIND	0	CTR_TX	0	DTOG_TX	0	STAT_TX [1:0]	0	0	0	0	EA[3:0]						
1Ch	Reset value	USB_EP7R		Reserved	0	CTR_RX	0	DTOG_RX	0	STAT_RX [1:0]	0	SETUP	0	EPTYPE [1:0]	0	EP_KIND	0	CTR_TX	0	DTOG_TX	0	STAT_TX [1:0]	0	0	0	0	EA[3:0]					
20h~3Fh	Reserved																															
40h	Reset value	USB_CNTR			Reserved	0	CTRM	0	PWAOVRM	0	ERRM	0	WKUPM	0	SUSPM	0	RESETM	0	SOFM	0	ESOFM	Reserved	0	RESUME	0	FSUSP	0	LPMODE	1	PDWN	1	FRES
44h	Reset value	USB_ISTR				Reserved	0	CTR	0	PWAOVR	0	ERR	0	WKUP	0	SUSP	0	RESET	0	SOF	0		ESOF	Reserved	0	DIR	0	EP_ID [3: 0]				0
48h	Reset value	USB_FNR	Reserved				0	RXDP	0	RXDM	0	LCK	0	LSOF [1: 0]	FN[10: 0]														X			
4Ch	USB_DADDR	Reserved		EF			ADD[6: 0]																									

	Reset value											0	0	0	0	0	0	0	0	
50h	USB_BTABLE	Reserved								BTABLE[15: 3]								Reserved		
	Reset value									0	0	0	0	0	0	0	0			

Refer to Table 1 for the register boundary addresses.

CKS copyright

17 Controller area network (bxCAN)

17.1 bxCAN introduction

The **Basic Extended CAN** peripheral, named **bxCAN**, interfaces the CAN network. It supports the CAN protocols version 2.0A and B. It has been designed to manage a high number of incoming messages efficiently with a minimum CPU load. It also meets the priority requirements for transmit messages.

For safety-critical applications, the CAN controller provides all hardware functions for supporting the CAN Time Triggered Communication option.

17.2 bxCAN main features

- Supports CAN protocol version 2.0 A, B Active
- Bit rates up to 1 Mbit/s
- Supports the Time Triggered Communication option

Transmission

- Three transmit mailboxes
- Configurable transmit priority
- Time Stamp on SOF transmission

Reception

- Two receive FIFOs with three stages
- 14 filter banks
- Identifier list feature
- Configurable FIFO overrun
- Time Stamp on SOF reception

Time-triggered communication option

- Disable automatic retransmission mode
- 16-bit free running timer
- Time Stamp sent in last two data bytes

Management

- Maskable interrupts
- Software-efficient mailbox mapping at a unique address space

Dual CAN

- CAN1: Master bxCAN for managing the communication between a Slave bxCAN and the 512-byte SRAM memory
- CAN2: Slave bxCAN, with no direct access to the SRAM memory.
- The two bxCAN cells share the 512-byte SRAM memory

Note: The USB and CAN share a dedicated 512- byte SRAM memory for data transmission and reception, and so they cannot be used concurrently (the shared SRAM is accessed through CAN and USB exclusively). The USB and CAN can be used in the same application but not at the same time.

17.3 bxCAN general description

In today's CAN applications, the number of nodes in a network is increasing and often several networks are linked together via gateways. Typically the number of messages in the system (and thus to be handled by each node) has significantly increased. In addition to the application messages, Network Management and Diagnostic messages have been introduced.

- An enhanced filtering mechanism is required to handle each type of message.

Furthermore, application tasks require more CPU time, therefore real-time constraints caused by message reception have to be reduced.

- A receive FIFO scheme allows the CPU to be dedicated to application tasks for a long time period without losing messages.

The standard HLP (Higher Layer Protocol) based on standard CAN drivers requires an efficient interface to the CAN controller.

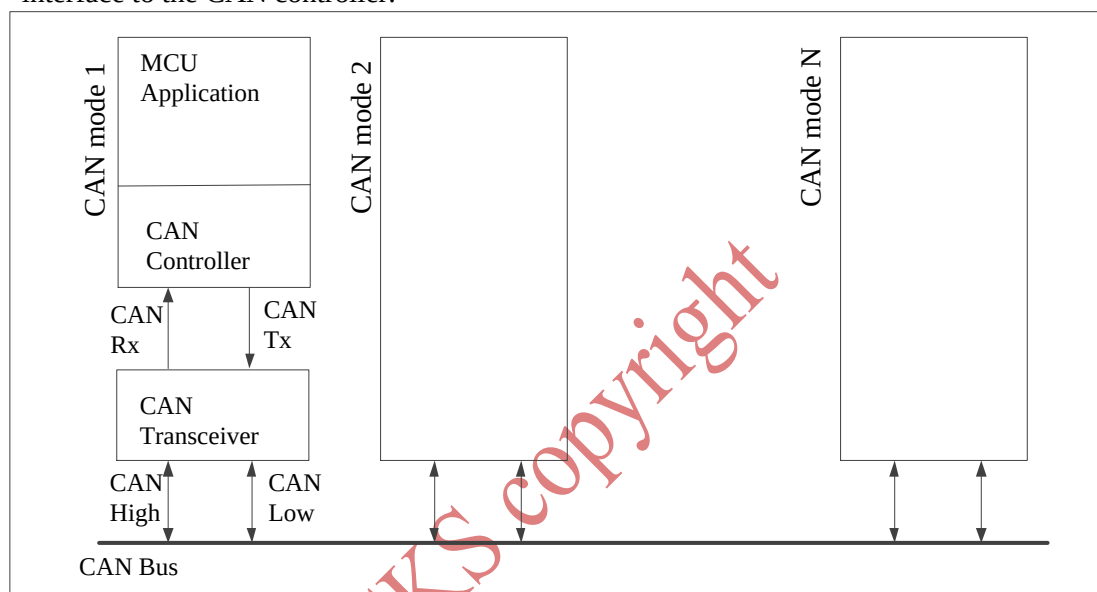


Figure 139 CAN network topology

17.3.1 CAN 2.0B active core

The bxCAN module handles the transmission and the reception of CAN messages fully autonomously. Standard identifiers (11-bit) and extended identifiers (29-bit) are fully supported by hardware.

17.3.2 Control, status and configuration registers

The application uses these registers to:

- Configure CAN parameters, e.g. baud rate
- Request transmissions
- Handle receptions
- Manage interrupts
- Get diagnostic information

17.3.3 Tx mailboxes

Three transmit mailboxes are provided to the software for setting up messages. The transmission Scheduler decides which mailbox has to be transmitted first.

17.3.4 Acceptance filters

In CKS32F103xx, there are 14 scalable/configurable identifier filter banks.

Receive FIFO

Two receive FIFOs are used by hardware to store the incoming messages. Three complete messages can be stored in each FIFO. The FIFOs are managed completely by hardware.

17.4 bxCAN operating modes

bxCAN has three main operating modes: **initialization**, **normal** and **Sleep**. After a hardware reset, bxCAN is in Sleep mode to reduce power consumption and an internal pull-up is active on CANTX. The software requests bxCAN to enter **initialization** or **Sleep** mode by setting the INRQ or SLEEP bits in the CAN_MCR register. Once the mode has been entered, bxCAN confirms it by setting the INAK or SLAK bits in the CAN_MSR register and the internal pull-up is disabled. When neither INAK nor SLAK are set, bxCAN is in **normal** mode. Before entering **normal** mode bxCAN always has to **synchronize** on the CAN bus. To synchronize, bxCAN waits until the CAN bus is idle, this means 11 consecutive recessive bits have been monitored on CANRX.

17.4.1 Initialization mode

The software initialization can be done while the hardware is in Initialization mode. To enter this mode the software sets the INRQ bit in the CAN_MCR register and waits until the hardware has confirmed the request by setting the INAK bit in the CAN_MSR register.

To leave Initialization mode, the software clears the INQR bit. bxCAN has left Initialization mode once the INAK bit has been cleared by hardware.

While in Initialization Mode, all message transfers to and from the CAN bus are stopped and the status of the CAN bus output CANTX is recessive (high).

Entering Initialization Mode does not change any of the configuration registers.

To initialize the CAN Controller, software has to set up the Bit Timing (CAN_BTR) and CAN options (CAN_MCR) registers.

To initialize the registers associated with the CAN filter banks (mode, scale, FIFO assignment, activation and filter values), software has to set the FINIT bit (CAN_FMR). Filter initialization also can be done outside the initialization mode.

*Note: When FINIT=1, CAN reception is deactivated.
The filter values also can be modified by deactivating the associated filter activation bits (in the CAN_FA1R register).
If a filter bank is not used, it is recommended to leave it non active (leave the corresponding FACT bit cleared).*

17.4.2 Normal mode

Once the initialization is complete, the software must request the hardware to enter Normal mode to be able to synchronize on the CAN bus and start reception and transmission. The request to enter Normal mode is issued by clearing the INRQ bit in the CAN_MCR register. The bxCAN enters Normal mode and is ready to take part in bus activities when it has synchronized with the data transfer on the CAN bus. This is done by waiting for the occurrence of a sequence of 11 consecutive recessive bits (Bus Idle state). The switch to Normal mode is confirmed by the hardware by clearing the INAK bit in the CAN_MSR register.

The initialization of the filter values is independent from Initialization Mode but must be done while the filter is not active (corresponding FACTx bit cleared). The filter scale and mode configuration must be configured before entering Normal Mode.

17.4.3 Sleep mode (low power)

To reduce power consumption, bxCAN has a low-power mode called Sleep mode. This mode is entered on software request by setting the SLEEP bit in the CAN_MCR register. In this mode, the bxCAN clock is stopped, however software can still access the bxCAN mailboxes.

If software requests entry to **initialization** mode by setting the INRQ bit while bxCAN is in **Sleep** mode, it must also clear the SLEEP bit.

bxCAN can be woken up (exit Sleep mode) either by software clearing the SLEEP bit or on detection of CAN bus activity.

On CAN bus activity detection, hardware automatically performs the wakeup sequence by clearing the SLEEP bit if the AWUM bit in the CAN_MCR register is set. If the AWUM bit is cleared, software has to clear the SLEEP bit when a wakeup interrupt occurs, in order to exit from Sleep mode.

Note: If the wakeup interrupt is enabled (WKUIE bit set in CAN_IER register) a wakeup interrupt will be generated on detection of CAN bus activity, even if the bxCAN automatically performs the wakeup sequence.

After the SLEEP bit has been cleared, Sleep mode is exited once bxCAN has synchronized with the CAN bus, refer to Figure 140. The Sleep mode is exited once the SLAK bit has been cleared by hardware.

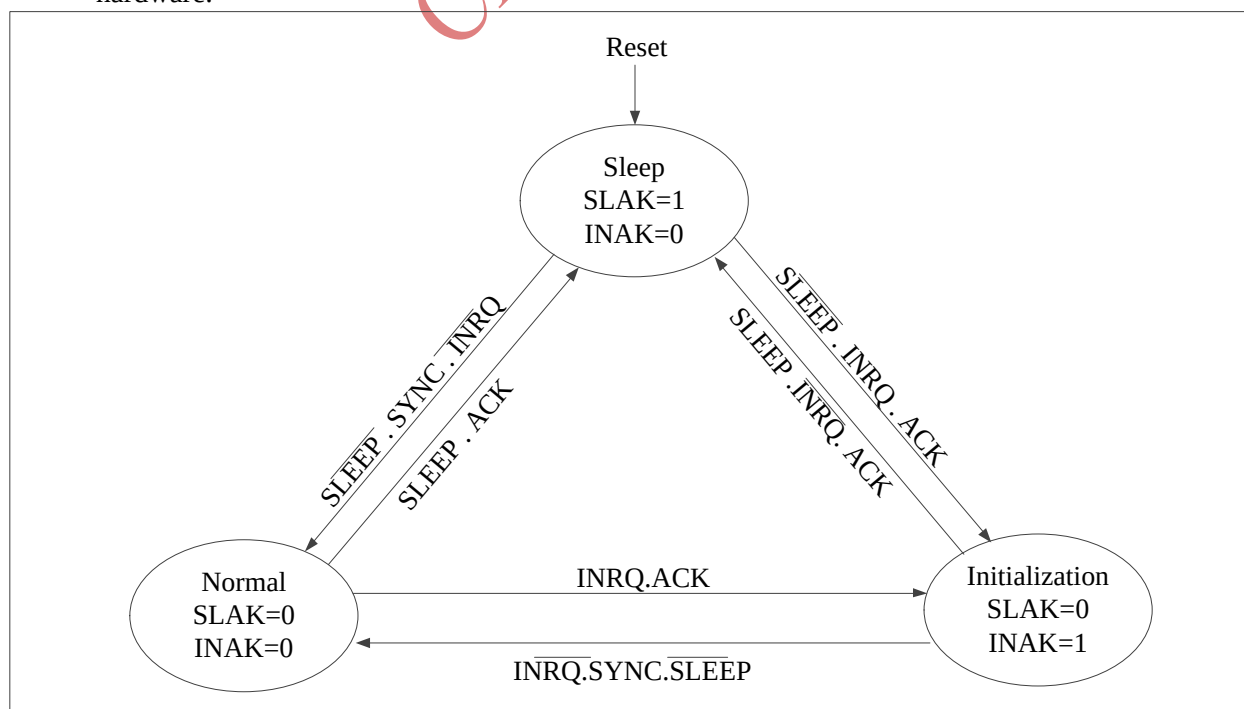


Figure 140 bxCAN operating modes

Note:

- 1.ACK = The wait state during which hardware confirms a request by setting the INAK or SLAK bits in the CAN_MSR register
- 2.SYNC = The state during which bxCAN waits until the CAN bus is idle, meaning 11 consecutive recessive bits have been monitored on CANRX

17.5 Test mode

Test mode can be selected by the SILM and LBKM bits in the CAN_BTR register. These bits must be configured while bxCAN is in Initialization mode. Once test mode has been selected, the INRQ bit in the CAN_MCR register must be reset to enter Normal mode.

17.5.1 Silent mode

The bxCAN can be put in Silent mode by setting the SILM bit in the CAN_BTR register. In Silent mode, the bxCAN is able to receive valid data frames and valid remote frames, but it sends only recessive bits on the CAN bus and it cannot start a transmission. If the bxCAN has to send a dominant bit (ACK bit, overload flag, active error flag), the bit is rerouted internally so that the CAN Core monitors this dominant bit, although the CAN bus may remain in recessive state. Silent mode can be used to analyze the traffic on a CAN bus without affecting it by the transmission of dominant bits (Acknowledge Bits, Error Frames).

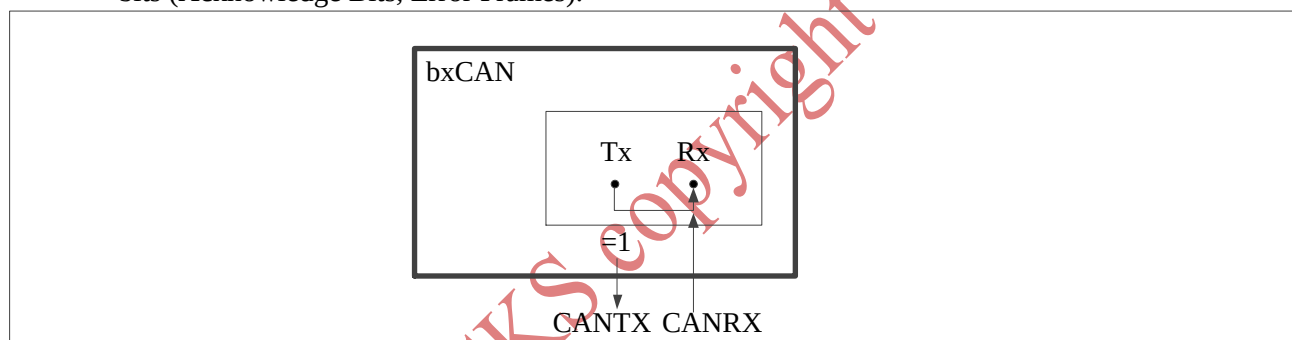


Figure 141 bxCAN in silent mode

17.5.2 Loop back mode

The bxCAN can be set in Loop Back Mode by setting the LBKM bit in the CAN_BTR register. In Loop Back Mode, the bxCAN treats its own transmitted messages as received messages and stores them (if they pass acceptance filtering) in a Receive mailbox.

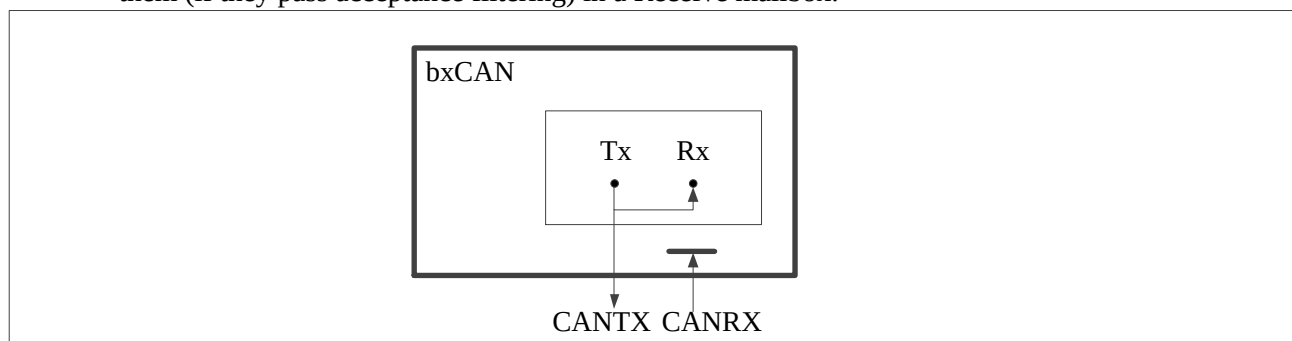


Figure 142 bxCAN in loop back mode

This mode is provided for self-test functions. To be independent of external events, the CAN Core ignores acknowledge errors (no dominant bit sampled in the acknowledge slot of a data/remote frame) in Loop Back Mode. In this mode, the bxCAN performs an internal feedback from its Tx output to its Rx input. The actual value of the CANRX input pin is disregarded by the bxCAN. The transmitted messages can be monitored on the CANTX pin.

17.5.3 Loop back combined with silent mode

It is also possible to combine Loop Back mode and Silent mode by setting the LBKM and SILM bits in the CAN_BTR register. This mode can be used for a “Hot Selftest”, meaning the bxCAN can be tested like in Loop Back mode but without affecting a running CAN system connected to the CANTX and CANRX pins. In this mode, the CANRX pin is disconnected from the bxCAN and the CANTX pin is held recessive.

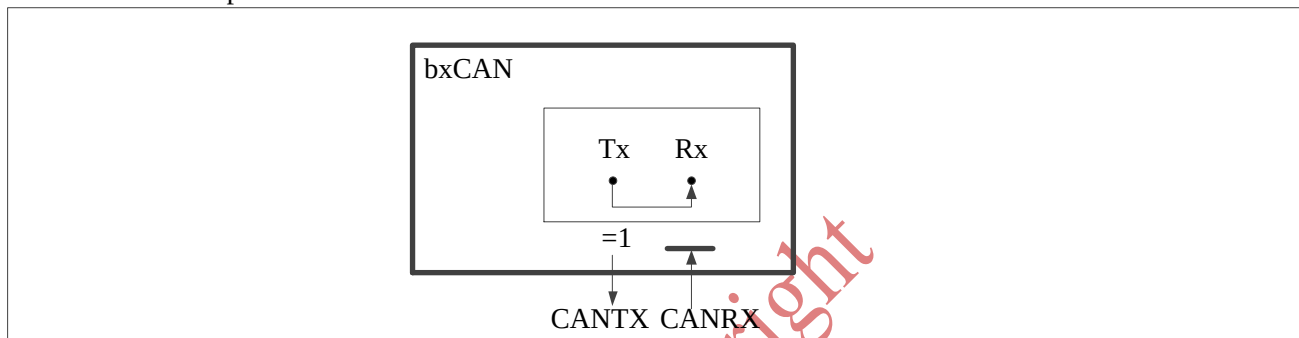


Figure 143 bxCAN in combined mode

17.6 Debug mode

When the microcontroller enters the debug mode (Cortex[®]-M3 core halted), the bxCAN continues to work normally or stops, depending on:

- the DBG_CAN1_STOP bit for CAN1 or the DBG_CAN2_STOP bit for CAN2 in the DBG module. For more details, refer to *Section 22.16.2: Debug support for timers, watchdog, bxCAN and I2C*.
- the DBF bit in CAN_MCR. For more details, refer to *Section 17.9.2: CAN control and status registers*.

17.7 bxCAN functional description

17.7.1 Transmission handling

In order to transmit a message, the application must select one **empty** transmit mailbox, set up the identifier, the data length code (DLC) and the data before requesting the transmission by setting the corresponding TXRQ bit in the CAN_TiRxR register. Once the mailbox has left **empty** state, the software no longer has write access to the mailbox registers. Immediately after the TXRQ bit has been set, the mailbox enters **pending** state and waits to become the highest priority mailbox, see *Transmit Priority*. As soon as the mailbox has the highest priority it will be **scheduled** for transmission. The transmission of the message of the scheduled mailbox will start (enter **transmit** state) when the CAN bus becomes idle. Once the mailbox has been successfully transmitted, it will become **empty** again. The hardware indicates a successful transmission by setting the RQCP and TXOK bits in the CAN_TSR register.

If the transmission fails, the cause is indicated by the ALST bit in the CAN_TSR register in case of an Arbitration Lost, and/or the TERR bit, in case of transmission error detection.

Transmit priority

By identifier

When more than one transmit mailbox is pending, the transmission order is given by the identifier of the message stored in the mailbox. The message with the lowest identifier value has the highest priority according to the arbitration of the CAN protocol. If the identifier values are equal, the lower mailbox number will be scheduled first.

By transmit request order

The transmit mailboxes can be configured as a transmit FIFO by setting the TXFP bit in the CAN_MCR register. In this mode the priority order is given by the transmit request order.

This mode is very useful for segmented transmission.

Abort

A transmission request can be aborted by the user setting the ABRQ bit in the CAN_TSR register. In **pending** or **scheduled** state, the mailbox is aborted immediately. An abort request while the mailbox is in **transmit** state can have two results. If the mailbox is transmitted successfully the mailbox becomes **empty** with the TXOK bit set in the CAN_TSR register. If the transmission fails, the mailbox becomes **scheduled**, the transmission is aborted and becomes **empty** with TXOK cleared. In all cases the mailbox will become **empty** again at least at the end of the current transmission.

Nonautomatic retransmission mode

This mode has been implemented in order to fulfil the requirement of the Time Triggered Communication option of the CAN standard. To configure the hardware in this mode the NART bit in the CAN_MCR register must be set.

In this mode, each transmission is started only once. If the first attempt fails, due to an arbitration loss or an error, the hardware will not automatically restart the message transmission.

At the end of the first transmission attempt, the hardware considers the request as completed and sets the RQCP bit in the CAN_TSR register. The result of the transmission is indicated in the CAN_TSR register by the TXOK, ALST and TERR bits.

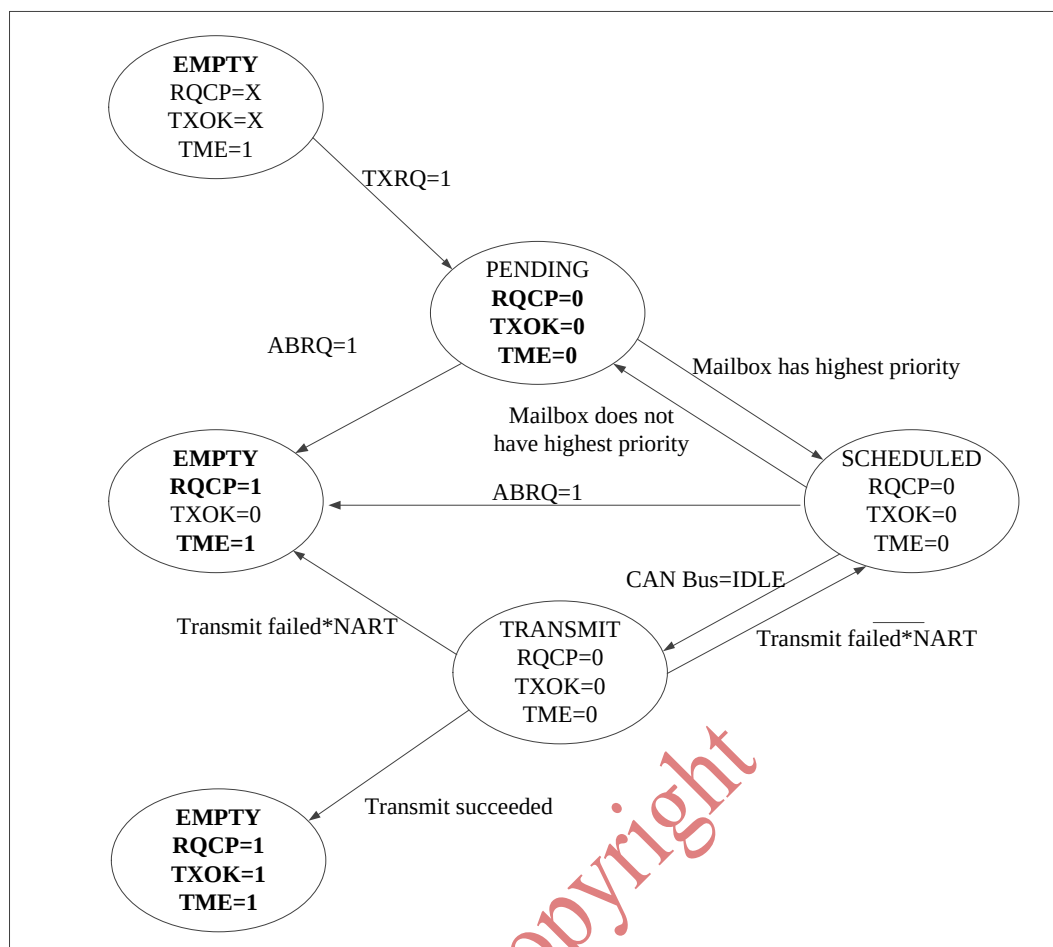


Figure 144 Transmit mailbox states

17.7.2 Time triggered communication mode

In this mode, the internal counter of the CAN hardware is activated and used to generate the Time Stamp value stored in the CAN_RDTxR/CAN_TDTxR registers, respectively (for Rx and Tx mailboxes). The internal counter is incremented each CAN bit time (refer to *Section 17.7.7: Bit timing*). The internal counter is captured on the sample point of the Start Of Frame bit in both reception and transmission.

17.7.3 Reception handling

For the reception of CAN messages, three mailboxes organized as a FIFO are provided. In order to save CPU load, simplify the software and guarantee data consistency, the FIFO is managed completely by hardware. The application accesses the messages stored in the FIFO through the FIFO output mailbox.

Valid message

A received message is considered as valid **when** it has been received correctly according to the CAN protocol (no error until the last but one bit of the EOF field) **and** It passed through the identifier filtering successfully, see *Section 17.7.4: Identifier filtering*.

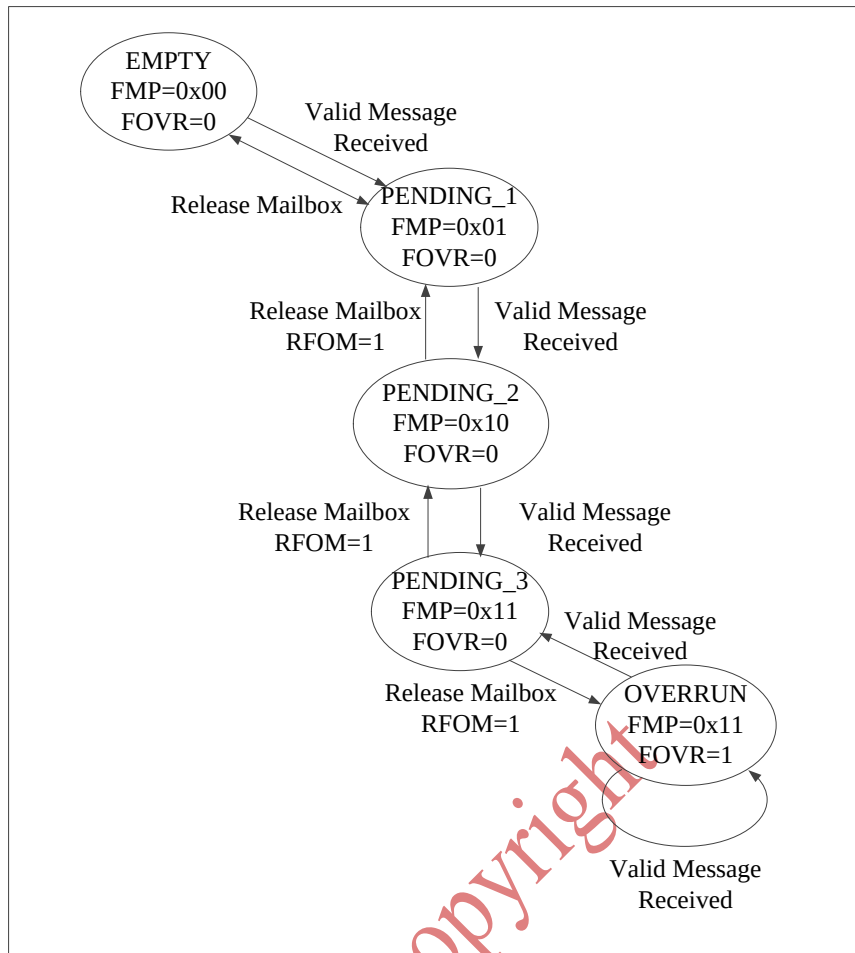


Figure 145 Receive FIFO states

FIFO management

Starting from the **empty** state, the first valid message received is stored in the FIFO which becomes **pending_1**. The hardware signals the event setting the FMP[1:0] bits in the CAN_RFR register to the value 01b. The message is available in the FIFO output mailbox. The software reads out the mailbox content and releases it by setting the RFOM bit in the CAN_RFR register. The FIFO becomes **empty** again. If a new valid message has been received in the meantime, the FIFO stays in **pending_1** state and the new message is available in the output mailbox.

If the application does not release the mailbox, the next valid message will be stored in the FIFO which enters **pending_2** state (FMP[1:0] = 10b). The storage process is repeated for the next valid message putting the FIFO into **pending_3** state (FMP[1:0] = 11b). At this point, the software must release the output mailbox by setting the RFOM bit, so that a mailbox is free to store the next valid message. Otherwise the next valid message received will cause a loss of message.

Refer also to *Section 17.7.5: Message storage*

Overrun

Once the FIFO is in **pending_3** state (i.e. the three mailboxes are full) the next valid message reception will lead to an **overrun** and a message will be lost. The hardware signals the overrun condition by setting the FOVR bit in the CAN_RFR register. Which message is lost depends on the configuration of the FIFO:

- If the FIFO lock function is disabled (RFLM bit in the CAN_MCR register cleared) the last message stored in the FIFO will be overwritten by the new incoming message. In this case the latest messages will be always available to the application.
- If the FIFO lock function is enabled (RFLM bit in the CAN_MCR register set) the most recent message will be discarded and the software will have the three oldest messages in the FIFO available.

Reception related interrupts

Once a message has been stored in the FIFO, the FMP[1:0] bits are updated and an interrupt request is generated if the FMPIE bit in the CAN_IER register is set.

When the FIFO becomes full (i.e. a third message is stored) the FULL bit in the CAN_RFR register is set and an interrupt is generated if the FFIE bit in the CAN_IER register is set.

On overrun condition, the FOVR bit is set and an interrupt is generated if the FOVIE bit in the CAN_IER register is set.

17.7.4 Identifier filtering

In the CAN protocol the identifier of a message is not associated with the address of a node but related to the content of the message. Consequently a transmitter broadcasts its message to all receivers. On message reception a receiver node decides - depending on the identifier value - whether the software needs the message or not. If the message is needed, it is copied into the SRAM. If not, the message must be discarded without intervention by the software.

To fulfill this requirement, the bxCAN Controller provides 14 configurable and scalable filter banks (13-0) to the application in order to receive only the messages the software needs. This hardware filtering saves CPU resources which would be otherwise needed to perform filtering by software. Each filter bank x consists of two 32-bit registers, CAN_FxR0 and CAN_FxR1.

Scalable width

To optimize and adapt the filters to the application needs, each filter bank can be scaled independently. Depending on the filter scale a filter bank provides:

- One 32-bit filter for the STDID[10:0], EXTID[17:0], IDE and RTR bits.
- Two 16-bit filters for the STDID[10:0], RTR, IDE and EXTID[17:15] bits.

Refer to *Figure 146*.

Furthermore, the filters can be configured in mask mode or in identifier list mode.

Mask mode

In **mask** mode the identifier registers are associated with mask registers specifying which bits of the identifier are handled as “must match” or as “don’t care”.

Identifier list mode

In **identifier list** mode, the mask registers are used as identifier registers. Thus instead of defining an identifier and a mask, two identifiers are specified, doubling the number of single identifiers. All bits of the incoming identifier must match the bits specified in the filter registers.

Filter bank scale and mode configuration

The filter banks are configured by means of the corresponding CAN_FMR register. To configure a filter bank it must be deactivated by clearing the FACT bit in the CAN_FAR register. The filter scale is configured by means of the corresponding FSCx bit in the CAN_FS1R register, refer to *Figure 146*. The **identifier list** or **identifier mask** mode for the corresponding Mask/Identifier registers is configured by means of the FBMx bits in the CAN_FMR register.

To filter a group of identifiers, configure the Mask/Identifier registers in mask mode.

To select single identifiers, configure the Mask/Identifier registers in identifier list mode.

Filters not used by the application should be left deactivated.

Each filter within a filter bank is numbered (called the *Filter Number*) from 0 to a maximum dependent on the mode and the scale of each of the filter banks.

Concerning the filter configuration, refer to *Figure 146*.

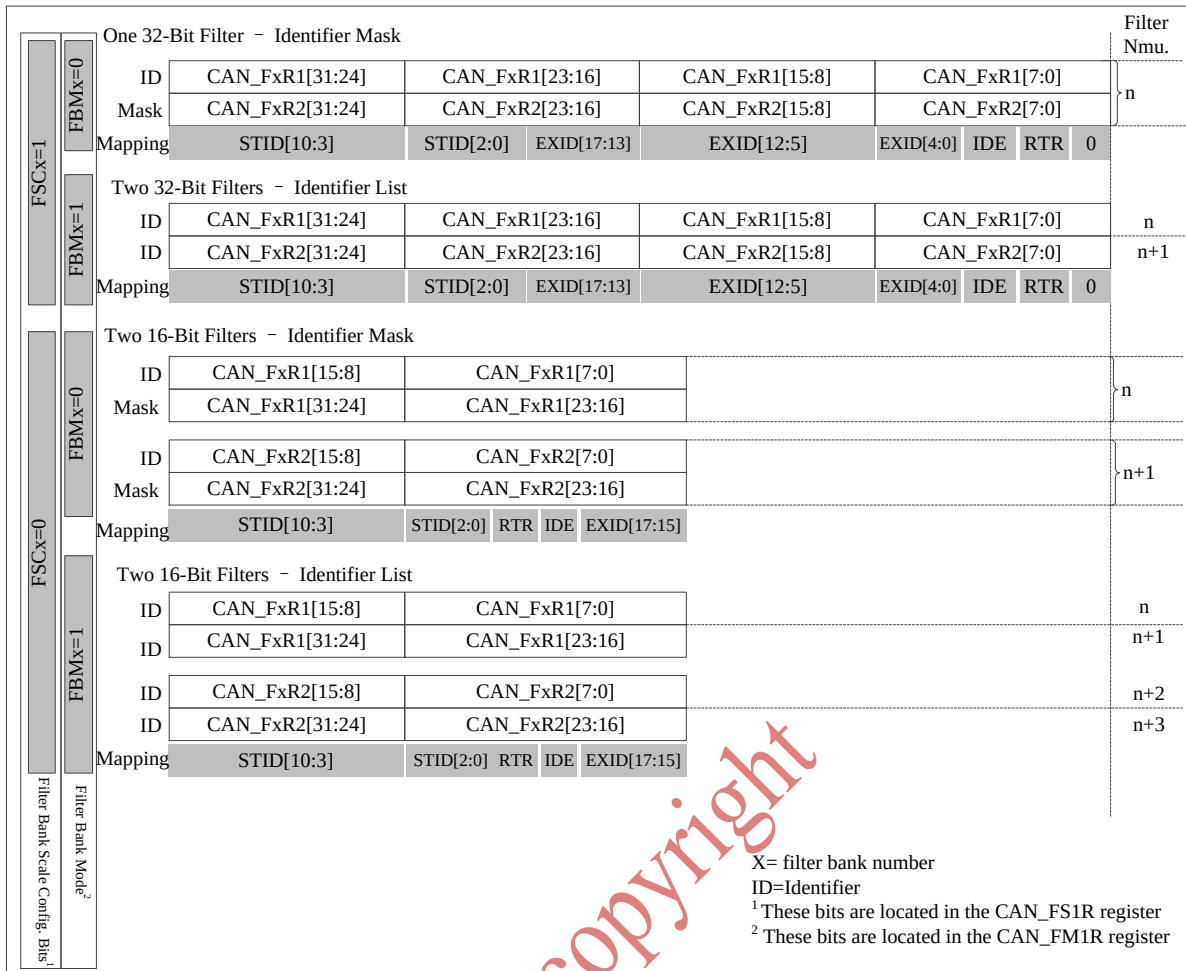


Figure 146 Filter bank scale configuration - register organization

Filter match index

Once a message has been received in the FIFO it is available to the application. Typically, application data is copied into SRAM locations. To copy the data to the right location the application has to identify the data by means of the identifier. To avoid this, and to ease the access to the SRAM locations, the CAN controller provides a Filter Match Index.

This index is stored in the mailbox together with the message according to the filter priority rules. Thus each received message has its associated filter match index.

The Filter Match index can be used in two ways:

- Compare the Filter Match index with a list of expected values.
- Use the Filter Match Index as an index on an array to access the data destination location.

For nonmasked filters, the software no longer has to compare the identifier.

If the filter is masked the software reduces the comparison to the masked bits only.

The index value of the filter number does not take into account the activation state of the filter banks. In addition, two independent numbering schemes are used, one for each FIFO. Refer to Figure 147 for an example.

Filter Bank	FIFO0	Filter Num.	Filter Bank	FIFO1	Filter Num.
0	ID List(32-bit)	0 1	2	ID Mask(16-bit)	0 1
1	ID Mask(32-bit)	2	4	ID List(32-bit)	2 3
3	ID List(16-bit)	3 4 5 6	7	Deactivated ID Mask(32-bit)	4 5
5	Deactivated ID List(32-bit)	7 8	8	ID Mask(16-bit)	6 7
6	ID Mask(16-bit)	9 10	10	Deactivated ID List(16-bit)	8 9 10 11
9	ID List(32-bit)	11 12	11	ID List(32-bit)	12 13
13	ID Mask(32-bit)	13	12	ID Mask(16-bit)	14

Figure 147 Example of filter numbering

Filter priority rules

Depending on the filter combination it may occur that an identifier passes successfully through several filters. In this case the filter match value stored in the receive mailbox is chosen according to the following priority rules:

- A 32-bit filter takes priority over a 16-bit filter.
- For filters of equal scale, priority is given to the Identifier List mode over the Identifier Mask mode
- For filters of equal scale and mode, priority is given by the filter number (the lower the number, the higher the priority).

Example of 3 filter banks in 32-bit Unidentified List mode and remaining in 32-bit Identifier Mask mode

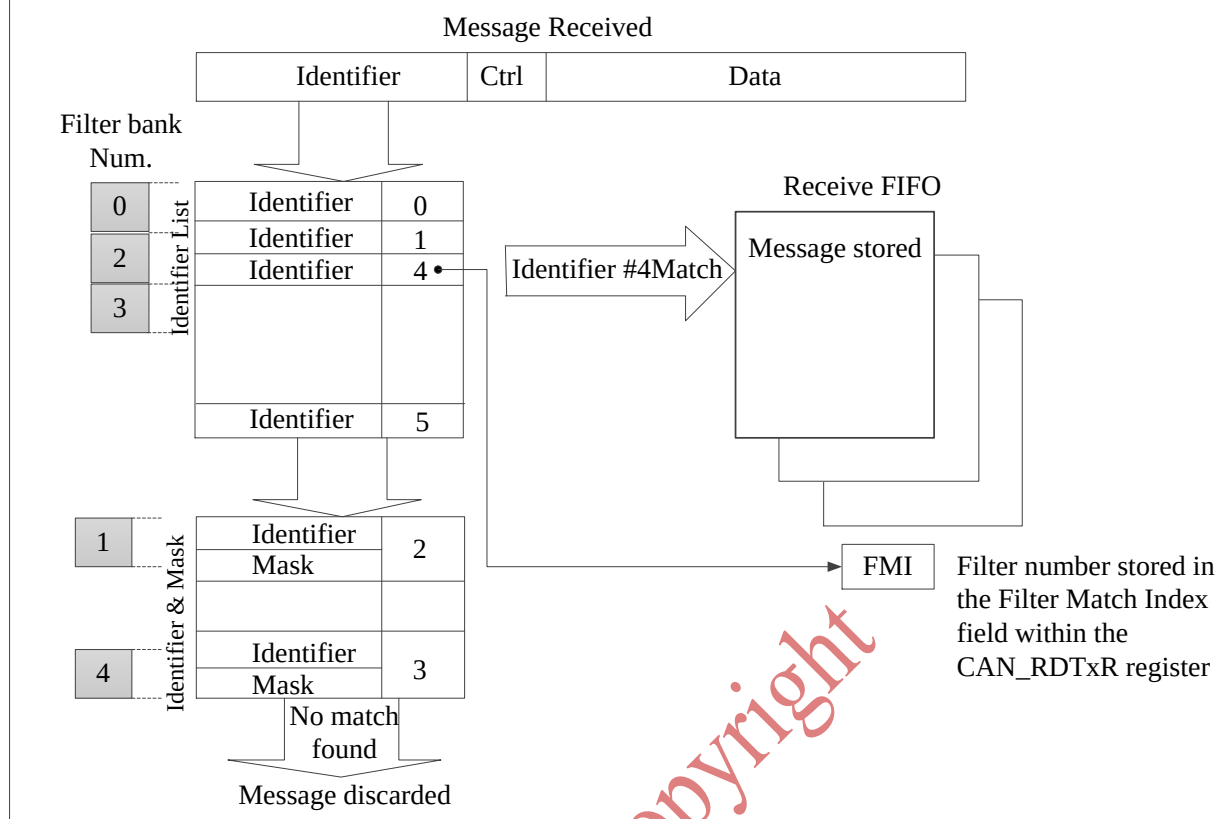


Figure 148 Filtering mechanism - example

The example above shows the filtering principle of the bxCAN. On reception of a message, the identifier is compared first with the filters configured in identifier list mode. If there is a match, the message is stored in the associated FIFO and the index of the matching filter is stored in the Filter Match Index. As shown in the example, the identifier matches with Identifier #2 thus the message content and FMI 2 is stored in the FIFO.

If there is no match, the incoming identifier is then compared with the filters configured in mask mode.

If the identifier does not match any of the identifiers configured in the filters, the message is discarded by hardware without disturbing the software.

17.7.5 Message storage

The interface between the software and the hardware for the CAN messages is implemented by means of mailboxes. A mailbox contains all information related to a message; identifier, data, control, status and time stamp information.

Transmit mailbox

The software sets up the message to be transmitted in an empty transmit mailbox. The status of the transmission is indicated by hardware in the CAN_TSR register.

Table 74 Transmit mailbox mapping

Offset to transmit mailbox base address	Register name
0	CAN_TlRxR
4	CAN_TDTxR

8	CAN_TDLxR
12	CAN_TDHxR

Receive mailbox (FIFO)

When a message has been received, it is available to the software in the FIFO output mailbox. Once the software has handled the message (e.g. read it) the software must release the FIFO output mailbox by means of the RFOM bit in the CAN_RFR register to make the next incoming message available. The filter match index is stored in the MFMI field of the CAN_RDTxR register. The 16-bit time stamp value is stored in the TIME[15:0] field of CAN_RDTxR.

Table 75 Receive mailbox mapping

Offset to receive mailbox base address (bytes)	Register name
0	CAN_RIxR
4	CAN_RDTxR
8	CAN_RDLxR
12	CAN_RDHxR

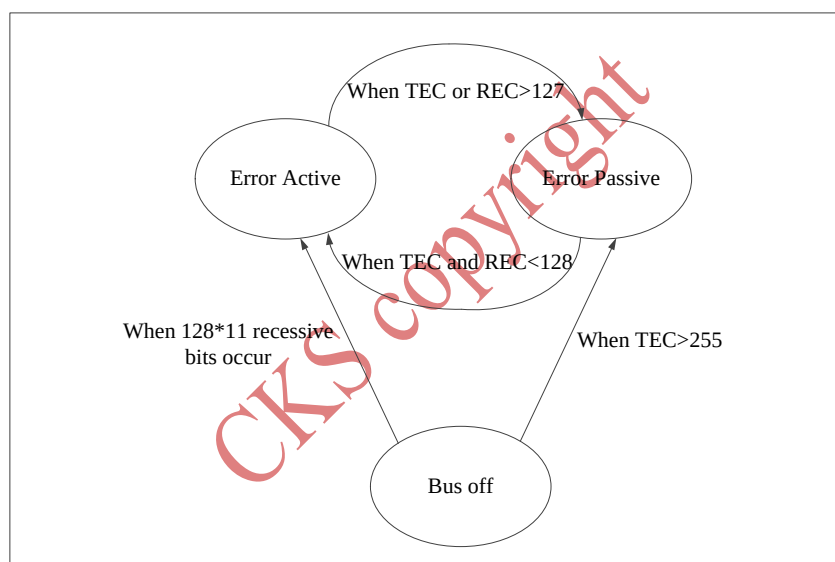


Figure 149 CAN error state diagram

17.7.6 Error management

The error management as described in the CAN protocol is handled entirely by hardware using a Transmit Error Counter (TEC value, in CAN_ESR register) and a Receive Error Counter (REC value, in the CAN_ESR register), which get incremented or decremented according to the error condition. For detailed information about TEC and REC management, refer to the CAN standard. Both of them may be read by software to determine the stability of the network. Furthermore, the CAN hardware provides detailed information on the current error status in CAN_ESR register. By means of the CAN_IER register (ERRIE bit, etc.), the software can configure the interrupt generation on error detection in a very flexible way.

Bus-Off recovery

The Bus-Off state is reached when TEC is greater than 255, this state is indicated by BOFF bit in CAN_ESR register. In Bus-Off state, the bxCAN is no longer able to transmit and receive messages.

Depending on the ABOM bit in the CAN_MCR register bxCAN will recover from Bus-Off (become error active again) either automatically or on software request. But in both cases the bxCAN has to wait at least for the recovery sequence specified in the CAN standard (128 occurrences of 11 consecutive recessive bits monitored on CANRX).

If ABOM is cleared, the software must initiate the recovering sequence by requesting bxCAN to enter and to leave initialization mode.

Note: In initialization mode, bxCAN does not monitor the CANRX signal, therefore it cannot complete the recovery sequence. To recover, bxCAN must be in normal mode.

17.7.7 Bit timing

The bit timing logic monitors the serial bus-line and performs sampling and adjustment of the sample point by synchronizing on the start-bit edge and resynchronizing on the following edges. Its operation may be explained simply by splitting nominal bit time into three segments as follows:

- **Synchronization segment (SYNC_SEG):** a bit change is expected to occur within this time segment. It has a fixed length of one time quantum ($1 \times t_q$).
- **Bit segment 1 (BS1):** defines the location of the sample point. It includes the PROP_SEG and PHASE_SEG1 of the CAN standard. Its duration is programmable between 1 and 16 time quanta but may be automatically lengthened to compensate for positive phase drifts due to differences in the frequency of the various nodes of the network.
- **Bit segment 2 (BS2):** defines the location of the transmit point. It represents the PHASE_SEG2 of the CAN standard. Its duration is programmable between 1 and 8 time quanta but may also be automatically shortened to compensate for negative phase drifts.

The resynchronization Jump Width (SJW) defines an upper bound to the amount of lengthening or shortening of the bit segments. It is programmable between 1 and 4 time quanta.

A valid edge is defined as the first transition in a bit time from dominant to recessive bus level provided the controller itself does not send a recessive bit.

If a valid edge is detected in BS1 instead of SYNC_SEG, BS1 is extended by up to SJW so that the sample point is delayed.

Conversely, if a valid edge is detected in BS2 instead of SYNC_SEG, BS2 is shortened by up to SJW so that the transmit point is moved earlier.

As a safeguard against programming errors, the configuration of the Bit Timing Register (CAN_BTR) is only possible while the device is in Standby mode.

Note: For a detailed description of the CAN bit timing and resynchronization mechanism, refer to the ISO 11898 standard.

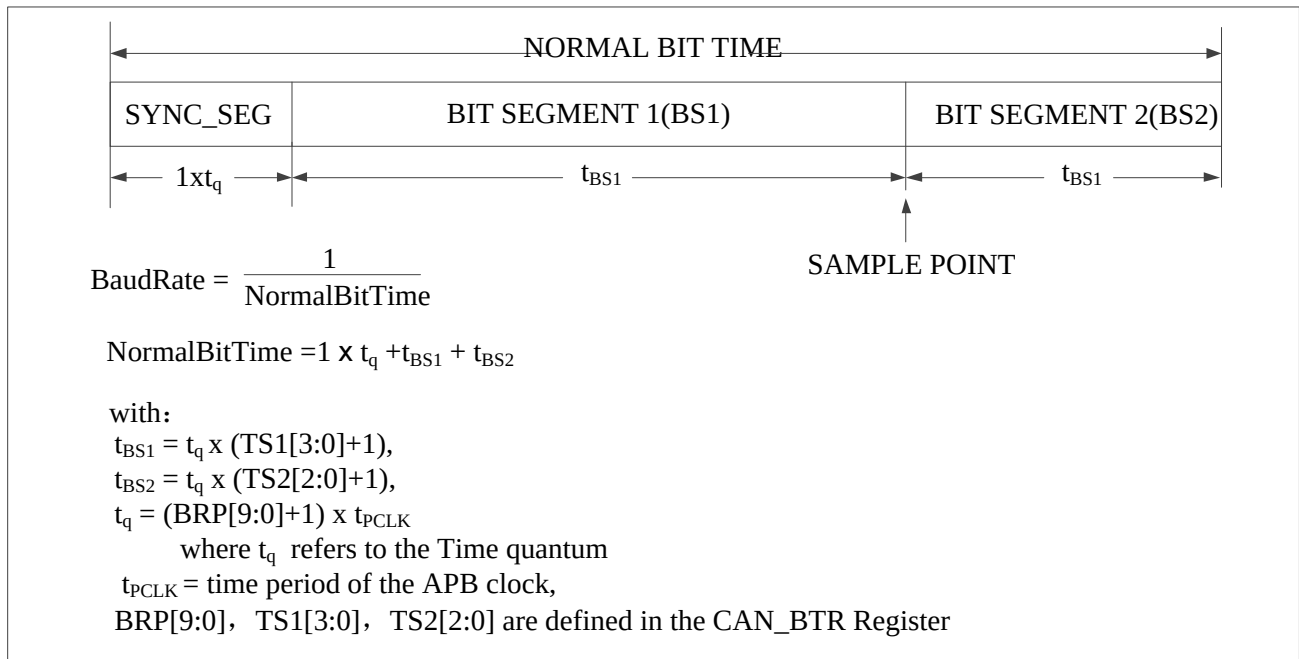


Figure 150 Bit timing

CKS copyright

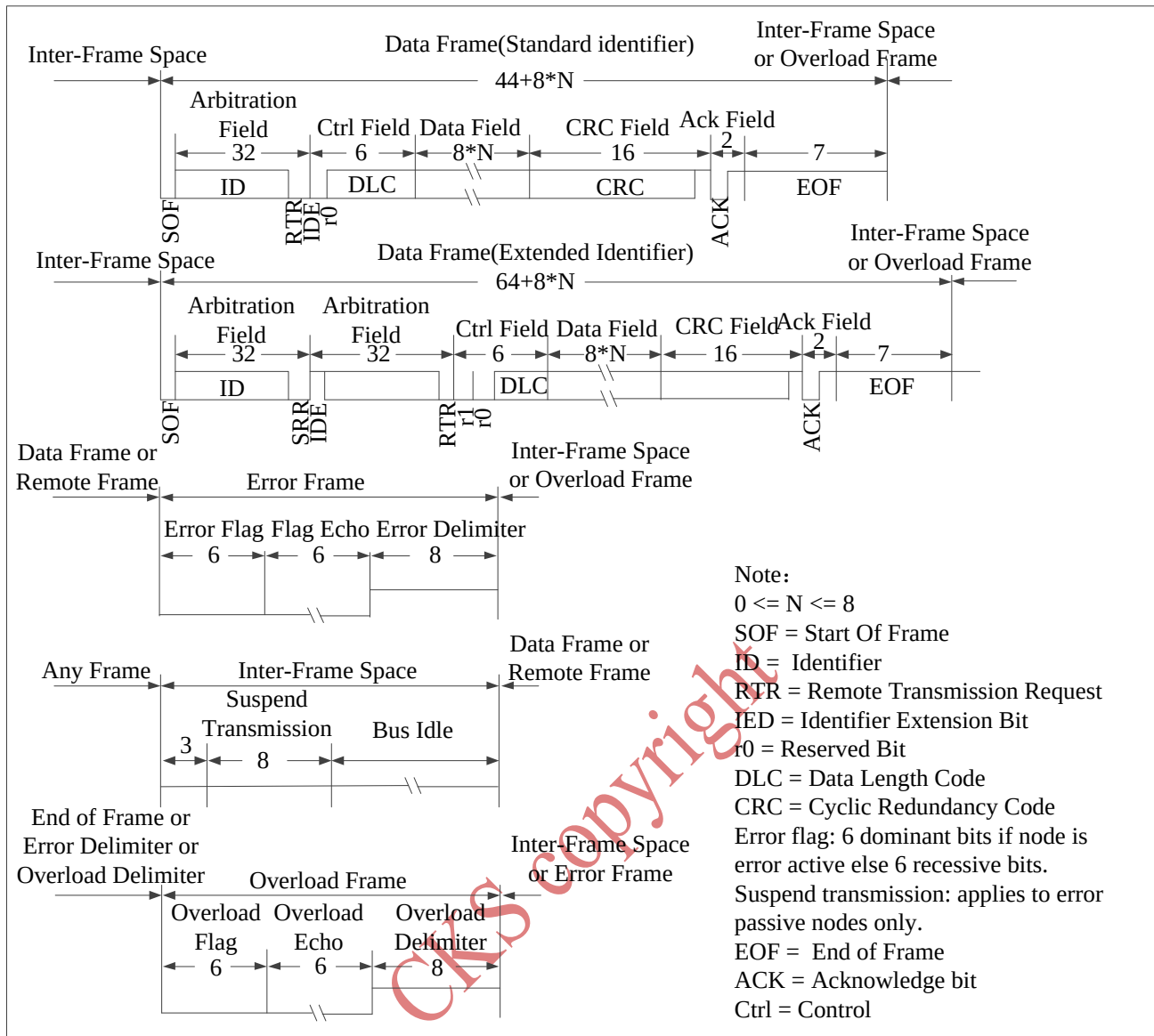


Figure 151 CAN frames

17.8 bxCAN interrupts

Four interrupt vectors are dedicated to bxCAN. Each interrupt source can be independently enabled or disabled by means of the CAN Interrupt Enable Register (CAN_IER).

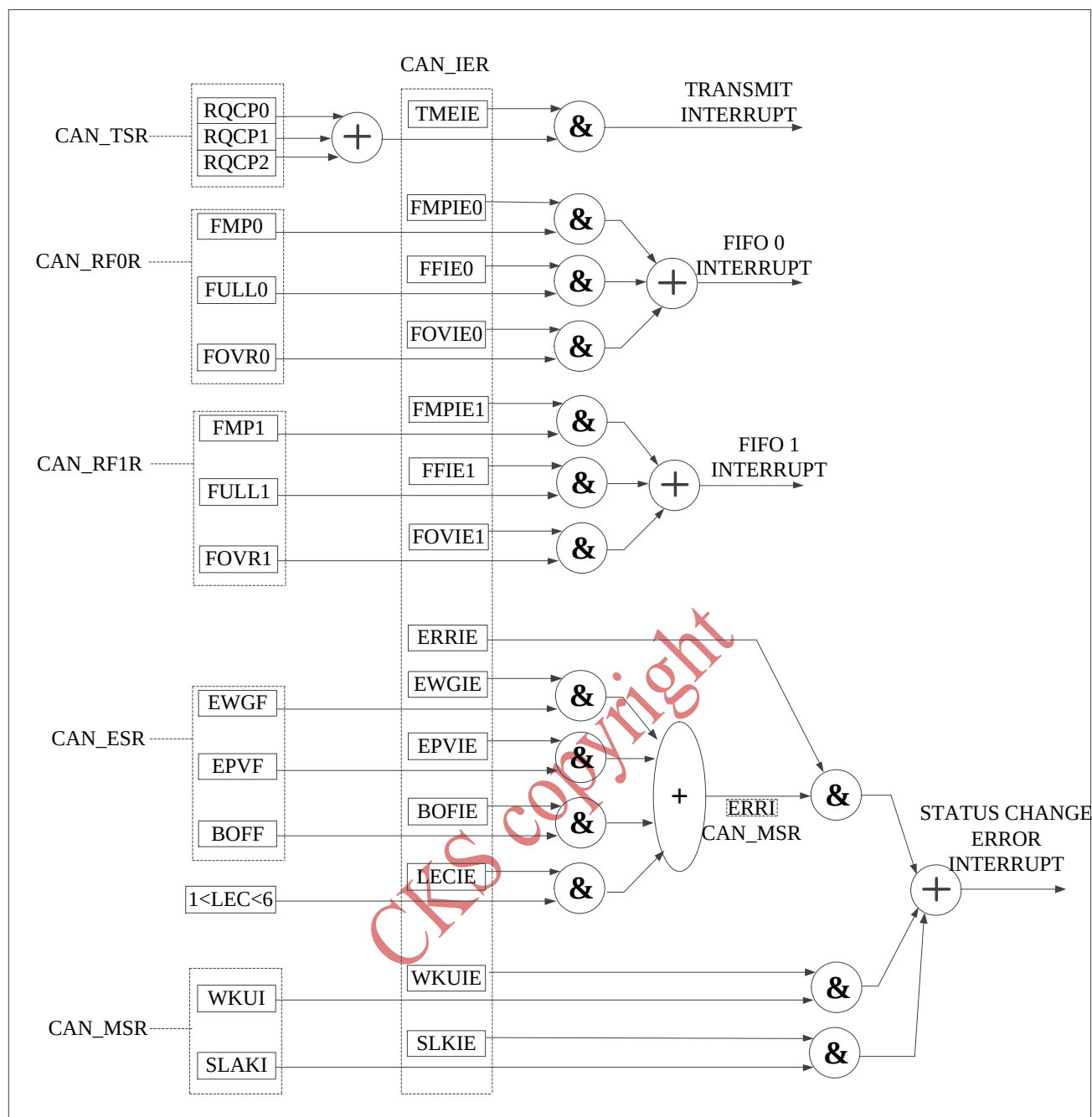


Figure 152 Event flags and interrupt generation

- The **transmit interrupt** can be generated by the following events:
 - Transmit mailbox 0 becomes empty, RQCP0 bit in the CAN_TSR register set.
 - Transmit mailbox 1 becomes empty, RQCP1 bit in the CAN_TSR register set.
 - Transmit mailbox 2 becomes empty, RQCP2 bit in the CAN_TSR register set.
- The **FIFO 0 interrupt** can be generated by the following events:
 - Reception of a new message, FMP0 bits in the CAN_RF0R register are not '00'.
 - FIFO0 full condition, FULL0 bit in the CAN_RF0R register set.
 - FIFO0 overrun condition, FOVR0 bit in the CAN_RF0R register set.
- The **FIFO 1 interrupt** can be generated by the following events:
 - Reception of a new message, FMP1 bits in the CAN_RF1R register are not '00'.
 - FIFO1 full condition, FULL1 bit in the CAN_RF1R register set.
 - FIFO1 overrun condition, FOVR1 bit in the CAN_RF1R register set.

- The **error and status change interrupt** can be generated by the following events:
 - Error condition, for more details on error conditions refer to the CAN Error Status register (CAN_ESR).
 - Wakeup condition, SOF monitored on the CAN Rx signal.
 - Entry into Sleep mode.

17.9 CAN registers

The peripheral registers have to be accessed by words (32 bits).

17.9.1 Register access protection

Erroneous access to certain configuration registers can cause the hardware to temporarily disturb the whole CAN network. Therefore the CAN_BTR register can be modified by software only while the CAN hardware is in initialization mode.

Although the transmission of incorrect data will not cause problems at the CAN network level, it can severely disturb the application. A transmit mailbox can be only modified by software while it is in empty state, refer to Figure 144.

The filter values can be modified either deactivating the associated filter banks or by setting the FINIT bit. Moreover, the modification of the filter configuration (scale, mode and FIFO assignment) in CAN_FMR, CAN_FSR and CAN_FFR registers can only be done when the filter initialization mode is set (FINIT=1) in the CAN_FMR register.

17.9.2 CAN control and status registers

Refer to *Section 1.1* for a list of abbreviations used in register descriptions.

CAN master control register (CAN_MCR)

Address offset: 0x00

Reset value: 0x0001 0002

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															DBF
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESET	Reserved							TTCM	ABOM	AWUM	NART	RFLM	TXEP	SLEEP	INRQ
rs	res							rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:15	Reserved, must be kept at reset value.														
Bit 16	DBF: Debug freeze 0: CAN working during debug 1: CAN reception/transmission frozen during debug. Reception FIFOs can still be accessed/controlled normally.														
Bit 15	RESET: bxCAN software master reset 0: Normal operation. 1: Force a master reset of the bxCAN -> Sleep mode activated after reset (FMP bits and CAN_MCR register are initialized to the reset values). This bit is automatically reset to 0.														
Bits 14:8	Reserved, must be kept at reset value.														
Bit 7	TTCM: Time triggered communication mode 0: Time Triggered Communication mode disabled. 1: Time Triggered Communication mode enabled <i>Note: For more information on Time Triggered Communication mode refer to Section 17.7.2: Time triggered communication mode.</i>														
Bit 6	ABOM: Automatic bus-off management This bit controls the behavior of the CAN hardware on leaving the Bus-Off state. 0: The Bus-Off state is left on software request, once 128 occurrences of 11 recessive bits have been														

	monitored and the software has first set and cleared the INRQ bit of the CAN_MCR register. 1: The Bus-Off state is left automatically by hardware once 128 occurrences of 11 recessive bits have been monitored. For detailed information on the Bus-Off state refer to <i>Section 17.7.6: Error management</i> .
Bit 5	AWUM: Automatic wakeup mode This bit controls the behavior of the CAN hardware on message reception during Sleep mode. 0: The Sleep mode is left on software request by clearing the SLEEP bit of the CAN_MCR register. 1: The Sleep mode is left automatically by hardware on CAN message detection. The SLEEP bit of the CAN_MCR register and the SLAK bit of the CAN_MSR register are cleared by hardware.
Bit 4	NART: No automatic retransmission 0: The CAN hardware will automatically retransmit the message until it has been successfully transmitted according to the CAN standard. 1: A message will be transmitted only once, independently of the transmission result (successful, error or arbitration lost).
Bit 3	RFLM: Receive FIFO locked mode 0: Receive FIFO not locked on overrun. Once a receive FIFO is full the next incoming message will overwrite the previous one. 1: Receive FIFO locked against overrun. Once a receive FIFO is full the next incoming message will be discarded.
Bit 2	TXFP: Transmit FIFO priority This bit controls the transmission order when several mailboxes are pending at the same time. 0: Priority driven by the identifier of the message 1: Priority driven by the request order (chronologically)
Bit 1	SLEEP: Sleep mode request This bit is set by software to request the CAN hardware to enter the Sleep mode. Sleep mode will be entered as soon as the current CAN activity (transmission or reception of a CAN frame) has been completed. This bit is cleared by software to exit Sleep mode. This bit is cleared by hardware when the AWUM bit is set and a SOF bit is detected on the CAN Rx signal. This bit is set after reset - CAN starts in Sleep mode.
Bit 0	INRQ: Initialization request The software clears this bit to switch the hardware into normal mode. Once 11 consecutive recessive bits have been monitored on the Rx signal the CAN hardware is synchronized and ready for transmission and reception. Hardware signals this event by clearing the INAK bit in the CAN_MSR register. Software sets this bit to request the CAN hardware to enter initialization mode. Once software has set the INRQ bit, the CAN hardware waits until the current CAN activity (transmission or reception) is completed before entering the initialization mode. Hardware signals this event by setting the INAK bit in the CAN_MSR register.

CAN master status register (CAN_MSR)

Address offset: 0x04

Reset value: 0x0000 0C02

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16			
Reserved																		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
Reserved				RX	SAMP	RXM	TXM	Reserved			SLAKI	WKUI	ERRI	SLAK	INAK			
				r	r	r	r				rc	wl	rc	wl	rc	wl	r	r
Bits31:12		Reserved, must be kept at reset value.																
Bit11		RX: CAN Rx signal Monitors the actual value of the CAN_RX Pin.																
Bit10		SAMP: Last sample point The value of RX on the last sample point (current received bit value).																
Bit9		RXM: Receive mode The CAN hardware is currently receiver.																
Bit8		TXM: Transmit mode The CAN hardware is currently transmitter.																
Bits 7:5		Reserved, must be kept at reset value.																
Bit4		SLAKI: Sleep acknowledge interrupt When SLKIE=1, this bit is set by hardware to signal that the bxCAN has entered Sleep Mode. When																

	set, this bit generates a status change interrupt if the SLKIE bit in the CAN_IER register is set. This bit is cleared by software or by hardware, when SLAK is cleared. <i>Note: When SLKIE=0, no polling on SLAKI is possible. In this case the SLAK bit can be polled.</i>
Bit3	WKUI: Wakeup interrupt This bit is set by hardware to signal that a SOF bit has been detected while the CAN hardware was in Sleep mode. Setting this bit generates a status change interrupt if the WKUIE bit in the CAN_IER register is set. This bit is cleared by software.
Bit2	ERRI: Error interrupt This bit is set by hardware when a bit of the CAN_ESR has been set on error detection and the corresponding interrupt in the CAN_IER is enabled. Setting this bit generates a status change interrupt if the ERRIE bit in the CAN_IER register is set. This bit is cleared by software.
Bit1	SLAK: Sleep acknowledge This bit is set by hardware and indicates to the software that the CAN hardware is now in Sleep mode. This bit acknowledges the Sleep mode request from the software (set SLEEP bit in CAN_MCR register). This bit is cleared by hardware when the CAN hardware has left Sleep mode (to be synchronized on the CAN bus). To be synchronized the hardware has to monitor a sequence of 11 consecutive recessive bits on the CAN RX signal. <i>Note: The process of leaving Sleep mode is triggered when the SLEEP bit in the CAN_MCR register is cleared. Refer to the AWUM bit of the CAN_MCR register description for detailed information for clearing SLEEP bit</i>
Bit0	INAK: Initialization acknowledge This bit is set by hardware and indicates to the software that the CAN hardware is now in initialization mode. This bit acknowledges the initialization request from the software (set INRQ bit in CAN_MCR register). This bit is cleared by hardware when the CAN hardware has left the initialization mode (to be synchronized on the CAN bus). To be synchronized the hardware has to monitor a sequence of 11 consecutive recessive bits on the CAN RX signal.

CAN transmit status register (CAN_TSR)

Address offset: 0x08

Reset value: 0x1C00 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOW 2	LO W 1	LO W 0	TM E 2	TME 1	TME 0	CODE [1: 0]	ABR Q 2	Reserved				TERR 2	ALST 2	TXO K 2	RQCP 2
r	r	r	r	r	r	r	r	rs				rc wl	rc wl	rc wl	rc wl
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AB RQ1	Reserved		TE RR1	AL ST1	TX OK1	RQ CP1	ABR Q0	Reserved				TER R0	AL ST0	TX OK0	RQ CP0
rs			rc wl	rc wl	rc wl	rc wl	rs					rc wl	rc wl	rc wl	rc wl

Bit 31	LOW2: Lowest priority flag for mailbox 2 This bit is set by hardware when more than one mailbox are pending for transmission and mailbox 2 has the lowest priority.
Bit 30	LOW1: Lowest priority flag for mailbox 1 This bit is set by hardware when more than one mailbox are pending for transmission and mailbox 1 has the lowest priority.
Bit 29	LOW0: Lowest priority flag for mailbox 0 This bit is set by hardware when more than one mailbox are pending for transmission and mailbox 0 has the lowest priority. <i>Note: The LOW[2:0] bits are set to zero when only one mailbox is pending.</i>
Bit 28	TME2: Transmit mailbox 2 empty This bit is set by hardware when no transmit request is pending for mailbox 2.
Bit 27	TME1: Transmit mailbox 1 empty This bit is set by hardware when no transmit request is pending for mailbox 1.
Bit 26	TME0: Transmit mailbox 0 empty This bit is set by hardware when no transmit request is pending for mailbox 0.
Bits 25:24	CODE[1:0]: Mailbox code In case at least one transmit mailbox is free, the code value is equal to the number of the next transmit mailbox free. In case all transmit mailboxes are pending, the code value is equal to the number of the transmit

	mailbox with the lowest priority.
Bit 23	ABRQ2: Abort request for mailbox 2 Set by software to abort the transmission request for the corresponding mailbox. Cleared by hardware when the mailbox becomes empty. Setting this bit has no effect when the mailbox is not pending for transmission.
Bits 22:20	Reserved, must be kept at reset value.
Bit 19	TERR2: Transmission error of mailbox 2 This bit is set when the previous TX failed due to an error.
Bit 18	ALST2: Arbitration lost for mailbox 2 This bit is set when the previous TX failed due to an arbitration lost.
Bit 17	TXOK2: Transmission OK of mailbox 2 The hardware updates this bit after each transmission attempt. 0: The previous transmission failed 1: The previous transmission was successful This bit is set by hardware when the transmission request on mailbox 2 has been completed successfully. Refer to Figure 144
Bit 16	RQCP2: Request completed mailbox2 Set by hardware when the last request (transmit or abort) has been performed. Cleared by software writing a “1” or by hardware on transmission request (TXRQ2 set in CAN_TMD2R register). Clearing this bit clears all the status bits (TXOK2, ALST2 and TERR2) for Mailbox 2.
Bit 15	ABRQ1: Abort request for mailbox 1 Set by software to abort the transmission request for the corresponding mailbox. Cleared by hardware when the mailbox becomes empty. Setting this bit has no effect when the mailbox is not pending for transmission.
Bits 14:12	Reserved, must be kept at reset value.
Bit 11	TERR1: Transmission error of mailbox1 This bit is set when the previous TX failed due to an error.
Bit 10	ALST1: Arbitration lost for mailbox1 This bit is set when the previous TX failed due to an arbitration lost.
Bit 9	TXOK1: Transmission OK of mailbox1 The hardware updates this bit after each transmission attempt. 0: The previous transmission failed 1: The previous transmission was successful This bit is set by hardware when the transmission request on mailbox 1 has been completed successfully. Refer to Figure 144
Bit 8	RQCP1: Request completed mailbox1 Set by hardware when the last request (transmit or abort) has been performed. Cleared by software writing a “1” or by hardware on transmission request (TXRQ1 set in CAN_TI1R register). Clearing this bit clears all the status bits (TXOK1, ALST1 and TERR1) for Mailbox 1.
Bit 7	ABRQ0: Abort request for mailbox0 Set by software to abort the transmission request for the corresponding mailbox. Cleared by hardware when the mailbox becomes empty. Setting this bit has no effect when the mailbox is not pending for transmission.
Bits 6:4	Reserved, must be kept at reset value.
Bit 3	TERR0: Transmission error of mailbox0 This bit is set when the previous TX failed due to an error.
Bit 2	ALST0: Arbitration lost for mailbox0 This bit is set when the previous TX failed due to an arbitration lost.
Bit 1	TXOK0: Transmission OK of mailbox0 The hardware updates this bit after each transmission attempt. 0: The previous transmission failed 1: The previous transmission was successful This bit is set by hardware when the transmission request on mailbox 1 has been completed successfully. Refer to Figure 144
Bit 0	RQCP0: Request completed mailbox0 Set by hardware when the last request (transmit or abort) has been performed. Cleared by software writing a “1” or by hardware on transmission request (TXRQ0 set in CAN_TI0R register). Clearing this bit clears all the status bits (TXOK0, ALST0 and TERR0) for Mailbox 0.

CAN receive FIFO 0 register (CAN_RF0R)

Address offset: 0x0C

Reset value: 0x00

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										RFOM0	FOVR0	FULL0	Res	FMP0[1: 0]	
										rs	rc	wl	rc	wl	r
Bits31:16		Reserved, must be kept at reset value.													
Bit 5		RFOM0: Release FIFO 0 output mailbox Set by software to release the output mailbox of the FIFO. The output mailbox can only be released when at least one message is pending in the FIFO. Setting this bit when the FIFO is empty has no effect. If at least two messages are pending in the FIFO, the software has to release the output mailbox to access the next message. Cleared by hardware when the output mailbox has been released.													
Bit 4		FOVR0: FIFO 0 overrun This bit is set by hardware when a new message has been received and passed the filter while the FIFO was full. This bit is cleared by software.													
Bit 3		FULL0: FIFO 0 full Set by hardware when three messages are stored in the FIFO. This bit is cleared by software.													
Bit 2		Reserved, must be kept at reset value.													
Bits 1:0		FMP0[1:0]: FIFO 0 message pending These bits indicate how many messages are pending in the receive FIFO. FMP is increased each time the hardware stores a new message in to the FIFO. FMP is decreased each time the software releases the output mailbox by setting the RFOM0 bit.													

CAN receive FIFO 1 register (CAN_RF1R)

Address offset: 0x10

Reset value: 0x00

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										RFOM1	FOVR1	FULL1	Res	FMP1[1: 0]	
										rs	rc	wl	rc	wl	r
Bits31:16		Reserved, must be kept at reset value.													
Bit 5		RFOM1: Release FIFO 1 output mailbox Set by software to release the output mailbox of the FIFO. The output mailbox can only be released when at least one message is pending in the FIFO. Setting this bit when the FIFO is empty has no effect. If at least two messages are pending in the FIFO, the software has to release the output mailbox to access the next message. Cleared by hardware when the output mailbox has been released.													
Bit 4		FOVR1: FIFO 1 overrun This bit is set by hardware when a new message has been received and passed the filter while the FIFO was full. This bit is cleared by software.													
Bit 3		FULL1: FIFO 1 full Set by hardware when three messages are stored in the FIFO. This bit is cleared by software.													
Bit 2		Reserved, must be kept at reset value.													
Bits 1:0		FMP1[1:0]: FIFO 1 message pending These bits indicate how many messages are pending in the receive FIFO1. FMP1 is increased each time the hardware stores a new message in to the FIFO1. FMP1 is decreased each time the software releases the output mailbox by setting the RFOM1 bit.													

CAN interrupt enable register (CAN_IER)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														SLK	WKU

													IE	IE	
													rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ERR IE	Reserved			IEC IE	BOF IE	EPV IE	EWG IE	Res	FOV IE1	FF IE1	FMP IE1	FOV IE0	FF IE0	FMP IE0	TME IE
rw				rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
Bits31:18		Reserved, must be kept at reset value.													
Bit17		SLKIE : Sleep interrupt enable 0: No interrupt when SLAKI bit is set. 1: Interrupt generated when SLAKI bit is set.													
Bit16		WKUIE : Wakeup interrupt enable 0: No interrupt when WKUI is set. 1: Interrupt generated when WKUI bit is set.													
Bit 15		ERRIE : Error interrupt enable 0: No interrupt will be generated when an error condition is pending in the CAN_ESR. 1: An interrupt will be generation when an error condition is pending in the CAN_ESR.													
Bits14:12		Reserved, must be kept at reset value.													
Bit 11		LECIE : Last error code interrupt enable 0: ERRI bit will not be set when the error code in LEC[2:0] is set by hardware on error detection. 1: ERRI bit will be set when the error code in LEC[2:0] is set by hardware on error detection.													
Bit 10		BOFIE : Bus-off interrupt enable 0: ERRI bit will not be set when BOFF is set. 1: ERRI bit will be set when BOFF is set.													
Bit 9		EPVIE : Error passive interrupt enable 0: ERRI bit will not be set when EPVF is set. 1: ERRI bit will be set when EPVF is set.													
Bit 8		EWGIE : Error warning interrupt enable 0: ERRI bit will not be set when EWGF is set. 1: ERRI bit will be set when EWGF is set.													
Bit 7		Reserved, must be kept at reset value.													
Bit 6		FOVIE1 : FIFO overrun interrupt enable 0: No interrupt when FOVR is set. 1: Interrupt generation when FOVR is set.													
Bit 5		FFIE1 : FIFO full interrupt enable 0: No interrupt when FULL bit is set. 1: Interrupt generated when FULL bit is set.													
Bit 4		FMPIE1 : FIFO message pending interrupt enable 0: No interrupt generated when state of FMP[1:0] bits are not 00b. 1: Interrupt generated when state of FMP[1:0] bits are not 00b.													
Bit 3		FOVIE0 : FIFO overrun interrupt enable 0: No interrupt when FOVR bit is set. 1: Interrupt generated when FOVR bit is set.													
Bit 2		FFIE0 : FIFO full interrupt enable 0: No interrupt when FULL bit is set. 1: Interrupt generated when FULL bit is set.													
Bit 1		FMPIE0 : FIFO message pending interrupt enable 0: No interrupt generated when state of FMP[1:0] bits are not 00b. 1: Interrupt generated when state of FMP[1:0] bits are not 00b.													
Bit 0		TMEIE : Transmit mailbox empty interrupt enable 0: No interrupt when RQCPx bit is set. 1: Interrupt generated when RQCPx bit is set. Note: Refer to Section 17.8: bxCAN interrupts.													

CAN error status register (CAN_ESR)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
REC[7:0]								TEC[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								LEC[2:0]		Res	BOFF	EPVF	WEGF		
								rw	rw	rw		r	r		r

Bits 31:24	REC[7:0]: Receive error counter The implementing part of the fault confinement mechanism of the CAN protocol. In case of an error during reception, this counter is incremented by 1 or by 8 depending on the error condition as defined by the CAN standard. After every successful reception the counter is decremented by 1 or reset to 120 if its value was higher than 128. When the counter value exceeds 127, the CAN controller enters the error passive state.
Bits 23:16	TEC[7:0]: Least significant byte of the 9-bit transmit error counter The implementing part of the fault confinement mechanism of the CAN protocol.
Bits 15:7	Reserved, must be kept at reset value
Bits 6:4	LEC[2:0]: Last error code This field is set by hardware and holds a code which indicates the error condition of the last error detected on the CAN bus. If a message has been transferred (reception or transmission) without error, this field will be cleared to '0' The LEC[2:0] bits can be set to value 0b111 by software. They are updated by hardware to indicate the current communication status. 000: No Error 001: Stuff Error 010: Form Error 011: Acknowledgment Error 100: Bit recessive Error 101: Bit dominant Error 110: CRC Error 111: Set by software
Bit 3	Reserved, must be kept at reset value
Bit 2	BOFF: Bus-off flag This bit is set by hardware when it enters the bus-off state. The bus-off state is entered on TEC overflow, greater than 255, refer to <i>Section 18.7.6: Error management</i> .
Bit 1	EPVF: Error passive flag This bit is set by hardware when the Error Passive limit has been reached (Receive Error Counter or Transmit Error Counter > 127).
Bit 0	EWGF: Error warning flag This bit is set by hardware when the warning limit has been reached (Receive Error Counter or Transmit Error Counter ≥ 96).

CAN bit timing register (CAN_BTR)

Address offset: 0x1C

Reset value: 0x0123 0000

This register can only be accessed by the software when the CAN hardware is in initialization mode.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SILM	LBKM	Reserved				SJW[1:0]		Res	TS2[2:0]			TS1[3:0]			
rw	rw					rw	rw	res	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						BRP[9:0]									
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31	SILM: Silent mode (debug) 0: Normal operation 1: Silent Mode
Bit 30	LBKM: Loop back mode (debug) 0: Loop Back Mode disabled 1: Loop Back Mode enabled
Bits 29:26	Reserved, must be kept at reset value.
Bits 25:24	SJW[1:0]: Resynchronization jump width These bits define the maximum number of time quanta the CAN hardware is allowed to lengthen or shorten a bit to perform the resynchronization. $t_{RJW} = t_q \times (SJW[1:0] + 1)$
Bit 23	Reserved, must be kept at reset value.
Bits 22:20	TS2[2:0]: Time segment 2 These bits define the number of time quanta in Time Segment 2. $t_{BS2} = t_{CAN} \times (TS2[2:0] + 1)$
Bits 19:16	TS1[3:0]: Time segment 1 These bits define the number of time quanta in Time Segment 1.

	$t_{BS1} = t_{CAN} \times (TS1[3:0] + 1)$ For more information on bit timing refer to <i>Section 17.7.7: Bit timing</i> .
Bits 15:10	Reserved, must be kept at reset value.
Bits 9:0	BRP[9:0] : Baud rate prescaler These bits define the length of a time quanta. $t_q = (BRP[9:0] + 1) \times t_{CLK}$

17.9.3 CAN mailbox registers

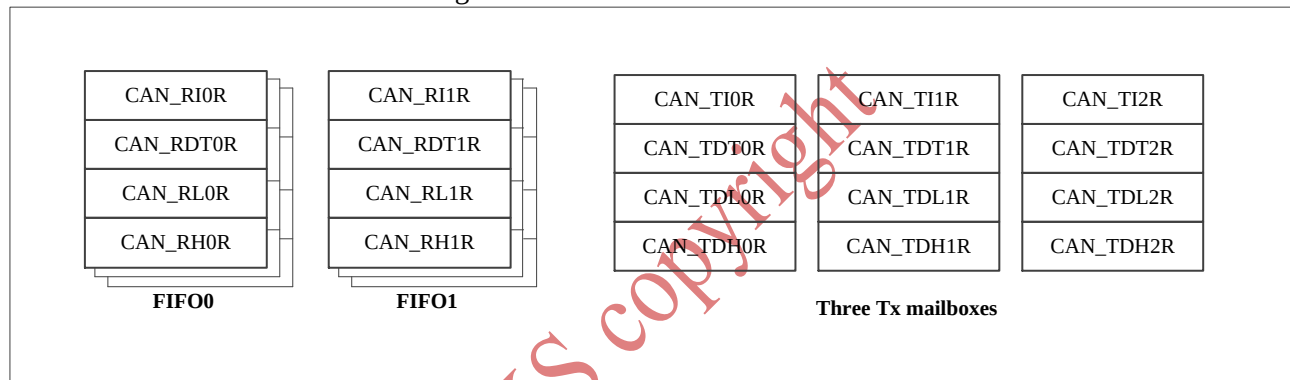
This chapter describes the registers of the transmit and receive mailboxes. Refer to *Section 17.7.5: Message storage* for detailed register mapping.

Transmit and receive mailboxes have the same registers except:

- The FMI field in the CAN_RDTxR register
- A receive mailbox is always write protected
- A transmit mailbox is write-enabled only while empty, corresponding TME bit in the CAN_TSR register set.

There are 3 TX Mailboxes and 2 RX Mailboxes. Each RX Mailbox allows access to a 3 level depth FIFO, the access being offered only to the oldest received message in the FIFO.

Each mailbox consist of 4 registers.



CAN TX mailbox identifier register (CAN_TIxR) (x=0..2)

Address offsets: 0x180, 0x190, 0x1A0

Reset value: 0xFFFF XXXX (except bit 0, TXRQ = 0)

All TX registers are write protected when the mailbox is pending transmission (TMEx reset).

This register also implements the TX request control (bit 0) - reset value 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
STID[10:0]/EXID[28:18]											EXID[17:13]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXID[12:0]													IDE	RTR	TXRQ
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:21		STID[10:0]/EXID[28:18] : Standard identifier or extended identifier The standard identifier or the MSBs of the extended identifier (depending on the IDE bit value).													
Bits 20:3		EXID[17:0] : Extended identifier The LSBs of the extended identifier.													
Bit 2		IDE : Identifier extension This bit defines the identifier type of message in the mailbox. 0: Standard identifier. 1: Extended identifier													
Bit 1		RTR : Remote transmission request 0: Data frame 1: Remote frame													
Bit 0		TXRQ : Transmit mailbox request Set by software to request the transmission for the corresponding mailbox. Cleared by hardware when the mailbox becomes empty.													

CAN mailbox data length control and time stamp register (CAN_TDTxR) (x=0..2)

All bits of this register are write protected when the mailbox is not in empty state.

Address offsets: 0x184, 0x194, 0x1A4

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TIME[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							TGT	Reserved				DLC[3:0]			
rwrwrw															
Bits 31:16		TIME[15:0]: Message time stamp This field contains the 16-bit timer value captured at the SOF transmission.													
Bits 15:9		Reserved, must be kept at reset value.													
Bit 8		TGT: Transmit global time This bit is active only when the hardware is in the Time Trigger Communication mode, TTCM bit of the CAN_MCR register is set. 0: Time stamp TIME[15:0] is not sent. 1: Time stamp TIME[15:0] value is sent in the last two data bytes of the 8-byte message: TIME[7:0] in data byte 7 and TIME[15:8] in data byte 6, replacing the data written in CAN_TDHxR[31:16] register (DATA6[7:0] and DATA7[7:0]). DLC must be programmed as 8 in order these two bytes to be sent over the CAN bus.													
Bits 7:4		Reserved, must be kept at reset value.													
Bits 3:0		DLC[3:0]: Data length code This field defines the number of data bytes a data frame contains or a remote frame request. A message can contain from 0 to 8 data bytes, depending on the value in the DLC field.													

CAN mailbox data low register (CAN_TDLxR) (x=0..2)

All bits of this register are write protected when the mailbox is not in empty state

Address offsets: 0x188, 0x198, 0x1A8

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATA3[7:0]								DATA2[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA1[7:0]								DATA0[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:24		DATA3[7:0]: Data byte 3 Data byte 3 of the message.													
Bits 23:16		DATA2[7:0]: Data byte 2 Data byte 2 of the message.													
Bits 15:8		DATA1[7:0]: Data byte 1 Data byte 1 of the message.													
Bits 7:0		DATA0[7:0]: Data byte 0 Data byte 0 of the message. A message can contain from 0 to 8 data bytes and starts with byte 0.													

CAN mailbox data high register (CAN_TDHxR) (x=0..2)

All bits of this register are write protected when the mailbox is not in empty state.

Address offsets: 0x18C, 0x19C, 0x1AC

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATA7[7:0]								DATA6[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA5[7:0]								DATA4[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:24		DATA7[7:0]: Data byte 7 Data byte 7 of the message.													

	<i>Note: If TGT of this message and TTCM are active, DATA7 and DATA6 will be replaced by the TIME stamp value.</i>
Bits 23:16	DATA6[7:0] : Data byte 6 Data byte 6 of the message.
Bits 15:8	DATA5[7:0] : Data byte 5 Data byte 5 of the message.
Bits 7:0	DATA4[7:0] : Data byte 4 Data byte 4 of the message.

CAN receive FIFO mailbox identifier register (CAN_RIxR) (x=0..1)

Address offsets: 0x1B0, 0x1C0

Reset value: 0xFFFF XXXX

All RX registers are write protected.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
STID[10:0]/EXID[28:18]											EXID[17:13]				
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXID[12:0]													IDE	RTR	Res
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 31:21	STID[10:0]/EXID[28:18] : Standard identifier or extended identifier The standard identifier or the MSBs of the extended identifier (depending on the IDE bit value).														
Bits 20:3	EXID[17:0] : Extended identifier The LSBs of the extended identifier.														
Bit 2	IDE : Identifier extension This bit defines the identifier type of message in the mailbox. 0: Standard identifier. 1: Extended identifier.														
Bit 1	RTR : Remote transmission request 0: Data frame 1: Remote frame														
Bit 0	Reserved, must be kept at reset value.														

CAN receive FIFO mailbox data length control and time stamp register (CAN_RDTxR) (x=0..1)

Address offsets: 0x1B4, 0x1C4

Reset value: 0xFFFF XXXX

All RX registers are write protected.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TIME[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FMI[7:0]								Resvered				DLC[3:0]			
r	r	r													
		Bits 31:16	TIME[15:0]: Message time stamp This field contains the 16-bit timer value captured at the SOF detection.												
		Bits 15:8	FMI[7:0]: Filter match index This register contains the index of the filter the message stored in the mailbox passed through. For more details on identifier filtering refer to <i>Section 17.7.4: Identifier filtering - Filter Match Index</i> paragraph.												
		Bits 7:4	Reserved, must be kept at reset value												
		Bits 3:0	DLC[3:0]: Data length code This field defines the number of data bytes a data frame contains (0 to 8). It is 0 in the case of a remote frame request.												

CAN receive FIFO mailbox data low register (CAN_RDLxR) (x=0..1)

All bits of this register are write protected when the mailbox is not in empty state.

Address offsets: 0x1B8, 0x1C8

Reset value: 0xFFFF XXXX

All RX registers are write protected.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATA3[7:0]								DATA2[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA1[7:0]								DATA0[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 31:24		DATA3[7:0] : Data Byte 3 Data byte 3 of the message.													
Bits 23:16		DATA2[7:0] : Data Byte 2 Data byte 2 of the message													
Bits 15:8		DATA1[7:0] : Data Byte 1 Data byte 1 of the message.													
Bits 7:0		DATA0[7:0] : Data Byte 0 Data byte 0 of the message. A message can contain from 0 to 8 data bytes and starts with byte 0													

CAN receive FIFO mailbox data high register (CAN_RDHxR) (x=0..1)

Address offsets: 0x1BC, 0x1CC

Reset value: 0xFFFF XXXX

All RX registers are write protected.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATA7[7:0]								DATA6[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA5[7:0]								DATA4[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 31:24		DATA7[7:0] : Data Byte 7 Data byte 7 of the message.													
Bits 23:16		DATA6[7:0] : Data Byte 6 Data byte 6 of the message													
Bits 15:8		DATA5[7:0] : Data Byte 5 Data byte 5 of the message.													
Bits 7:0		DATA4[7:0] : Data Byte 4 Data byte 4 of the message.													

17.9.4 CAN filter registers

CAN filter master register (CAN_FMR)

Address offset: 0x200

Reset value: 0x2A1C 0E01

All bits of this register are set and cleared by software.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															FINIT
rw															
Bits 31:1		Reserved, must be kept at reset value.													
Bit 0		FINIT : Filter init mode Initialization mode for filter banks 0: Active filters mode. 1: Initialization mode for the filters.													

CAN filter mode register (CAN_FM1R)

Address offset: 0x204

Reset value: 0x0000 0000

This register can be written only when the filter initialization mode is set (FINIT=1) in the CAN_FMR register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	FBM	FBM	FBM	FBM	FBM	FBM	FBM	FBM	FBM	FBM	FBM	FBM	FBM	FBM	FBM
	13	12	11	10	9	8	7	6	5	4	3	2	1	0	0
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Note:

Refer to figure 146

Bits 31:14	Reserved, must be kept at reset value.
Bits 13:0	FBMx : Filter mode Mode of the registers of Filter x. 0: Two 32-bit registers of filter bank x are in Identifier Mask mode. 1: Two 32-bit registers of filter bank x are in Identifier List mode.

CAN filter scale register (CAN_FS1R)

Address offset: 0x20C

Reset value: 0x0000 0000

This register can be written only when the filter initialization mode is set (FINIT=1) in the CAN_FMR register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	FSC	FSC	FSC	FSC	FSC	FSC	FSC	FSC	FSC	FSC	FSC	FSC	FSC	FSC	FSC
	13	12	11	10	9	8	7	6	5	4	3	2	1	0	0
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Note:

Refer to figure 146

Bits 31:14	Reserved, must be kept at reset value.
Bits 13:0	FSCx : Filter scale configuration These bits define the scale configuration of Filters 13-0 0: Dual 16-bit scale configuration 1: Single 32-bit scale configuration

CAN filter FIFO assignment register (CAN_FFA1R)

Address offset: 0x214

Reset value: 0x0000 0000

This register can be written only when the filter initialization mode is set (FINIT=1) in the CAN_FMR register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	FFA	FFA	FFA	FFA	FFA	FFA	FFA	FFA	FFA	FFA	FFA	FFA	FFA	FFA	FFA
	13	12	11	10	9	8	7	6	5	4	3	2	1	0	0
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:14	Reserved, must be kept at reset value.
Bits 13:0	FFAx : Filter FIFO assignment for filter x The message passing through this filter will be stored in the specified FIFO. 0: Filter assigned to FIFO 0 1: Filter assigned to FIFO 1

CAN filter activation register (CAN_FA1R)

Address offset: 0x21C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	FAC T13	FAC T12	FAC T11	FAC T10	FAC T9	FAC T8	FAC T7	FAC T6	FAC T5	FAC T4	FAC T3	FAC T2	FAC T1	FAC T0	
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:14		Reserved, must be kept at reset value.													
Bits 13:0		FACTx : Filter active The software sets this bit to activate Filter x. To modify the Filter x registers (CAN_FxR[0:7]), the FACTx bit must be cleared or the FINIT bit of the CAN_FMR register must be set. 0: Filter x is not active 1: Filter x is active													

Filter bank i register x (CAN_FiRx) (i=0..27, x=1, 2)

Address offsets: 0x240h..0x31C

Reset value: 0xFFFF XXXX

There are 14 filter banks $i = 0 \dots 13$. Each filter bank i is composed of two 32-bit registers, CAN_FiR[2:1].

This register can only be modified when the FACTx bit of the CAN_FAxR register is cleared or when the FINIT bit of the CAN_FMR register is set.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	FB 13	FB 12	FB 11	FB 10	FB 9	FB 8	FB 7	FB 6	FB 5	FB 4	FB 3	FB 2	FB 1	FB 0	
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:14		Reserved, must be kept at reset value.													
Bits 13:0		FB[31:0] : Filter bits Identifier Each bit of the register specifies the level of the corresponding bit of the expected identifier. 0: Dominant bit is expected 1: Recessive bit is expected Mask Each bit of the register specifies whether the bit of the associated identifier register must match with the corresponding bit of the expected identifier or not. 0: Don't care, the bit is not used for the comparison 1: Must match, the bit of the incoming identifier must have the same level as specified in the corresponding identifier register of the filter.													

Note: Depending on the scale and mode configuration of the filter the function of each register can differ. For the filter mapping, functions description and mask registers association, refer to Section 17.7.4: Identifier filtering.

A Mask/Identifier register in **mask mode** has the same bit mapping as in **identifier list mode**.

For the register mapping/addresses of the filter banks refer to Table 76.

17.9.5 bxCAN register map

Refer to *Section 1.1* for the register boundary addresses.

Table 76 bxCAN register map and reset values

offset		Register		
000h	Reset value	CAN_MCR	31	
	Reserved		30	
			29	
			28	
			27	
			26	
			25	
			24	
			23	
			22	
			21	
			20	
			19	
			18	
			17	
			16	
	15			
004h	Reset value	CAN_MSR	14	
	Reserved		13	
			12	
			11	
			10	
			9	
			8	
			7	
			6	
			5	
			4	
			3	
			2	
			1	
			0	
		008h	Reset value	CAN_TSR
Reserved			RESET	
00Ch	Reset value	CAN_RF0R	1	
	Reserved		RX	
010h	Reset value	CAN_RF1F	1	
	Reserved		SAMP	

014h	Reset value	CAN_IER	Reserved										SLKIE	WKUIE	ERRIE	Reserved	LECIE	BOFIE	EPVIE	EWGIE	Reserved	FOVIE	FFIE1	FMPIE	FOVIE	FFIE0	FMPIE	TMEIE				
													0	0	0		0	0	0	0		0	0	0	0	0	0	0				
018h	Reset value	CAN_ESR	REC[7:0]							TEC[7:0]							Reserved							LEC [2: 0]		Reserved	BOFF	EPVF	EWGF			
			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				0	0	0		0		0	0				
01Ch	Reset value	CAN_BTR	SILM	LBKM	Reserved			SJW [1: 0]	Reserved	TS2 [2: 0]		TS1 [3: 0]		Reserved				BPR[9: 0]														
			0	0				0		0	0	1	0	0	0	1	1		0	0	0	0	0	0	0	0	0	0				
020h ~ 17Fh	Reserved																															
180h	Reset value	CAN_TTOR	STID[10:0]/EXID[28:18]														EXID[17:3]										IDE				RTR	TXRQ
			X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	0				
184h	Reset value	CAN_TDTOR	TIME[15:0]														Reserved				TGT	Reserved				DLC[3:0]						
			X	X	X	X	X	X	X	X	X	X	X	X	X	X				X					X	X	X	X				
188h	CAN_TDLOR	DATA3[7:0]							DATA2[7:0]							DATA1[7:0]							DATA0[7:0]									

[illegible]

1A0h	CAN_TI2R	STID[10:0]/EXID[28:18]												EXID[17:3]												IDE			RTR	TXR			
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	0					
1A4h	CAN_TDT2R	TIME[15:0]															Reserved						TGT	Reserved						DLC[3:0]			
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X					
1A8h	CAN_TDL2R	DATA3[7:0]							DATA2[7:0]							DATA1[7:0]							DATA0[7:0]										
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X					
1ACh	CAN_TDH2R	DATA7[7:0]							DATA6[7:0]							DATA5[7:0]							DATA4[7:0]										
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X					
1B0h	CAN_RI0R	STID[10:0]/EXID[28:18]												EXID[17:3]												IDE			RTR	Reserved			
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X						
1B4h	CAN_RDT0R	TIME[15:0]															FMI[7: 0]						Reserved						DLC[3:0]				

	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X		X	X	X	X			
1B8h	CAN_RDL0R	DATA3[7:0]								DATA2[7:0]								DATA1[7:0]								DATA0[7:0]							
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X			
1BCh	CAN_RDH0R	DATA7[7:0]								DATA6[7:0]								DATA5[7:0]								DATA4[7:0]							
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X			
1C0h	CAN_R11R	STID[10:0]/EXID[28:18]										EXID[17:3]																IDE		RTR		Reserved	
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X			
1C4h	CAN_RDT1R	TIME[15:0]														FMI[7: 0]								Reserved				DLC[3:0]					
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X				X	X	X	X		
1C8h	CAN_RDL0R	DATA3[7:0]								DATA2[7:0]								DATA1[7:0]								DATA0[7:0]							
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X		

1CCh	CAN_RDH0R	DATA7[7:0]								DATA6[7:0]								DATA5[7:0]								DATA4[7:0]							
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X		
1D0h ~ 1FFh	Reserved																																
200h	CAN_FMR	Reserved														CAN2SB[5:0]						Reserved								FINIT			
	Reset value															0	0	1	1	1	0										0		
204h	CAN_FM1R	Reserved	FBM[27:0]																														
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
208h	Reserved																																
20Ch	CAN_FS1R	Reserved	FSC[27:0]																														
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
210h	Reserved																																
214h	CAN_FFA1R	Reserved	FFA[27:0]																														
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

218h	Reserved																															
21Ch	CAN_FA1R	Reserved	FACT[27:0]																													
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
220h	Reserved																															
224h ~ 23Fh	Reserved																															
240h	CAN_F0R1	FB[31:0]																														
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
244h	CAN_F0R2	FB[31:0]																														
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
240h	CAN_F1R1	FB[31:0]																														
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
244h	CAN_F1R2	FB[31:0]																														
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
...	...																															

318h	CAN_F27R2	FB[31:0]																																	
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
31Ch	CAN_F27R2	FB[31:0]																																	
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

CKS copyright

18 Serial peripheral interface (SPI)

18.1 SPI introduction

The serial peripheral interface (SPI) allows half/ full-duplex, synchronous, serial communication with external devices. The interface can be configured as the master and in this case it provides the communication clock (SCK) to the external slave device. The interface is also capable of operating in multimaster configuration.

It may be used for a variety of purposes, including simplex synchronous transfers on two lines with a possible bidirectional data line or reliable communication using CRC checking.

18.2 SPI main features

18.2.1 SPI features

- Full-duplex synchronous transfers on three lines
- Simplex synchronous transfers on two lines with or without a bidirectional data line
- 8- or 16-bit transfer frame format selection
- Master or slave operation
- Multimaster mode capability
- 8 master mode baud rate prescalers ($f_{PCLK}/2$ max.)
- Slave mode frequency ($f_{PCLK}/2$ max.)
- Faster communication for both master and slave
- NSS management by hardware or software for both master and slave: dynamic change of master/slave operations
- Programmable clock polarity and phase
- Programmable data order with MSB-first or LSB-first shifting
- Dedicated transmission and reception flags with interrupt capability
- SPI bus busy status flag
- Hardware CRC feature for reliable communication:
 - CRC value can be transmitted as last byte in Tx mode
 - Automatic CRC error checking for last received byte
- Master mode fault, overrun and CRC error flags with interrupt capability
- 1-byte transmission and reception buffer with DMA capability: Tx and Rx requests

18.3 SPI functional description

18.3.1 General description

The block diagram of the SPI is shown in Figure 153.

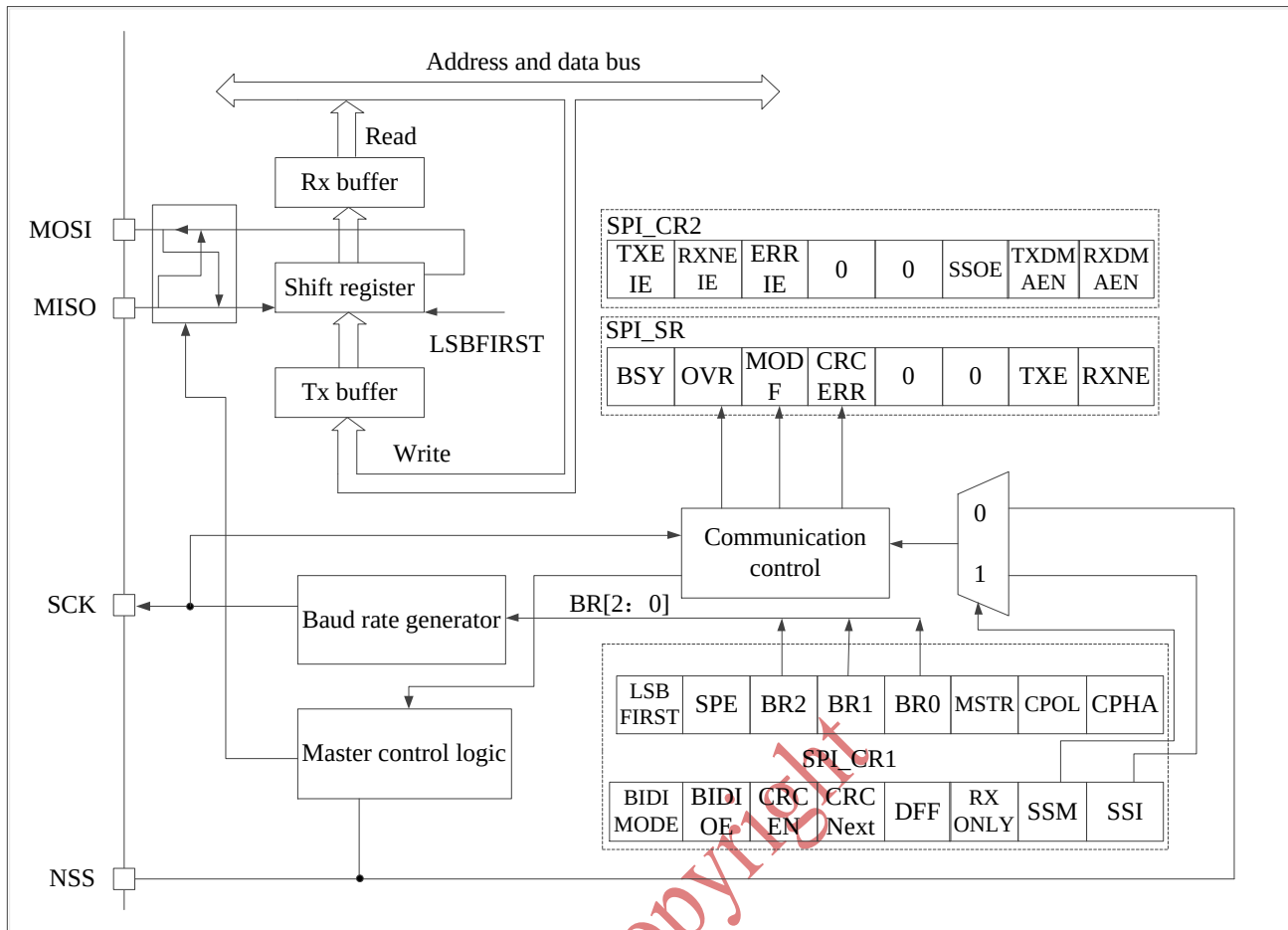


Figure 153 SPI block diagram

Usually, the SPI is connected to external devices through four pins:

- MISO: Master In / Slave Out data. This pin can be used to transmit data in slave mode and receive data in master mode.
- MOSI: Master Out / Slave In data. This pin can be used to transmit data in master mode and receive data in slave mode.
- SCK: Serial Clock output for SPI masters and input for SPI slaves.
- NSS: Slave select. This is an optional pin to select a slave device. This pin acts as a 'chip select' to let the SPI master communicate with slaves individually and to avoid contention on the data lines. Slave NSS inputs can be driven by standard IO ports on the master device. The NSS pin may also be used as an output if enabled (SSOE bit) and driven low if the SPI is in master configuration. In this manner, all NSS pins from devices connected to the Master NSS pin see a low level and become slaves when they are configured in NSS hardware mode. When configured in master mode with NSS configured as an input (MSTR=1 and SSOE=0) and if NSS is pulled low, the SPI enters the master mode fault state: the MSTR bit is automatically cleared and the device is configured in slave mode (refer to *Section 18.3.10: Error flags*).

A basic example of interconnections between a single master and a single slave is illustrated in Figure 154.

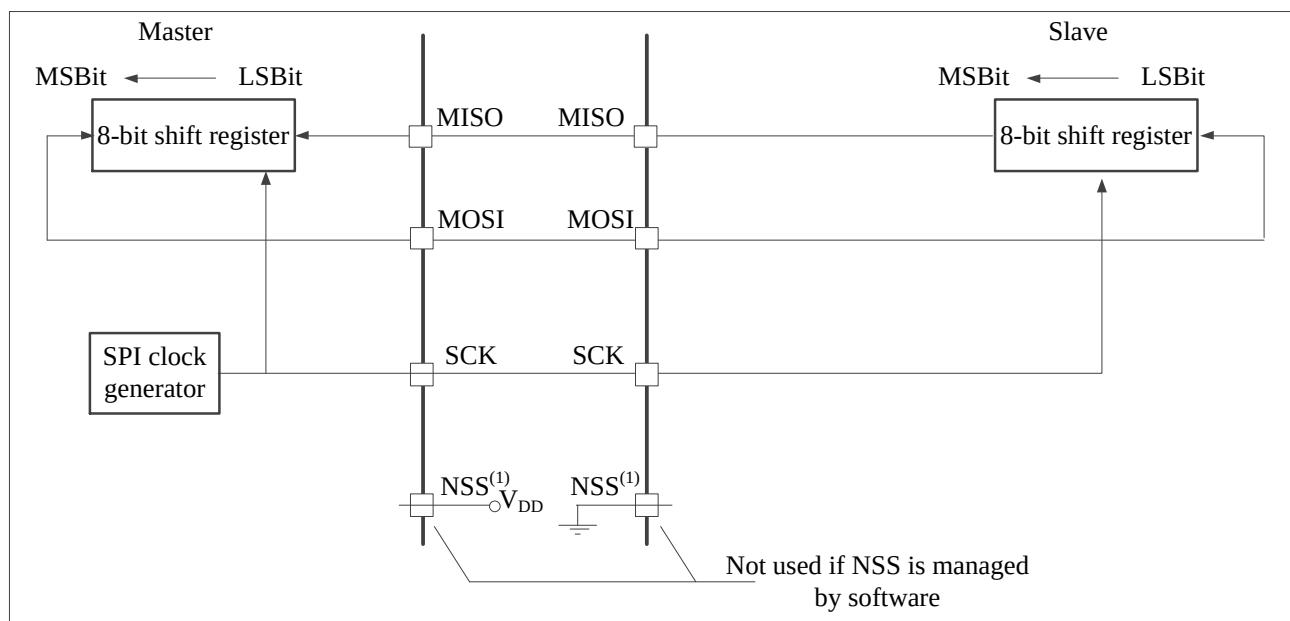


Figure 154 Single master/single slave application

1. Here, the NSS pin is configured as an input.

The MOSI pins are connected together and the MISO pins are connected together. In this way data is transferred serially between master and slave (most significant bit first).

The communication is always initiated by the master. When the master device transmits data to a slave device via the MOSI pin, the slave device responds via the MISO pin. This implies full-duplex communication with both data out and data in synchronized with the same clock signal (which is provided by the master device via the SCK pin).

Slave select (NSS) pin management

There are two NSS modes:

- Software NSS management (SSM = 1)

The slave select information is driven internally by the value of the SSI bit in the SPI_CR1 register. The external NSS pin remains free for other application uses.
- Hardware NSS management (SSM = 0)
 - NSS output enabled (SSM = 0, SSOE = 1)

This configuration is used only when CKS32F103xx operates in master mode. The NSS signal is driven low when the master starts the communication and is kept low until the SPI is disabled.
 - NSS output disabled (SSM = 0, SSOE = 0)

Clock phase and clock polarity

Four possible timing relationships may be chosen by software, using the CPOL and CPHA bits in the SPI_CR1 register. The CPOL (clock polarity) bit controls the steady state value of the clock when no data is being transferred. This bit affects both master and slave modes. If CPOL is reset, the SCK pin has a low-level idle state. If CPOL is set, the SCK pin has a high-level idle state.

If the CPHA (clock phase) bit is set, the second edge on the SCK pin (falling edge if the CPOL bit is reset, rising edge if the CPOL bit is set) is the MSBit capture strobe. Data are latched on the occurrence of the second clock transition. If the CPHA bit is reset, the first edge on the SCK pin (falling edge if CPOL bit is set, rising edge if CPOL bit is reset) is the MSBit capture strobe. Data are latched on the occurrence of the first clock transition.

The combination of the CPOL (clock polarity) and CPHA (clock phase) bits selects the data capture clock edge.

Figure 155 shows an SPI transfer with the four combinations of the CPHA and CPOL bits. The diagram may be interpreted as a master or slave timing diagram where the SCK pin, the MISO pin, the MOSI pin are directly connected between the master and the slave device.

Note: Prior to changing the CPOL/CPHA bits the SPI must be disabled by resetting the SPE bit. Master and slave must be programmed with the same timing mode. The idle state of SCK must correspond to the polarity selected in the SPI_CR1 register (by pulling up SCK if CPOL=1 or pulling down SCK if CPOL=0). The Data Frame Format (8- or 16-bit) is selected through the DFF bit in SPI_CR1 register, and determines the data length during transmission/reception.

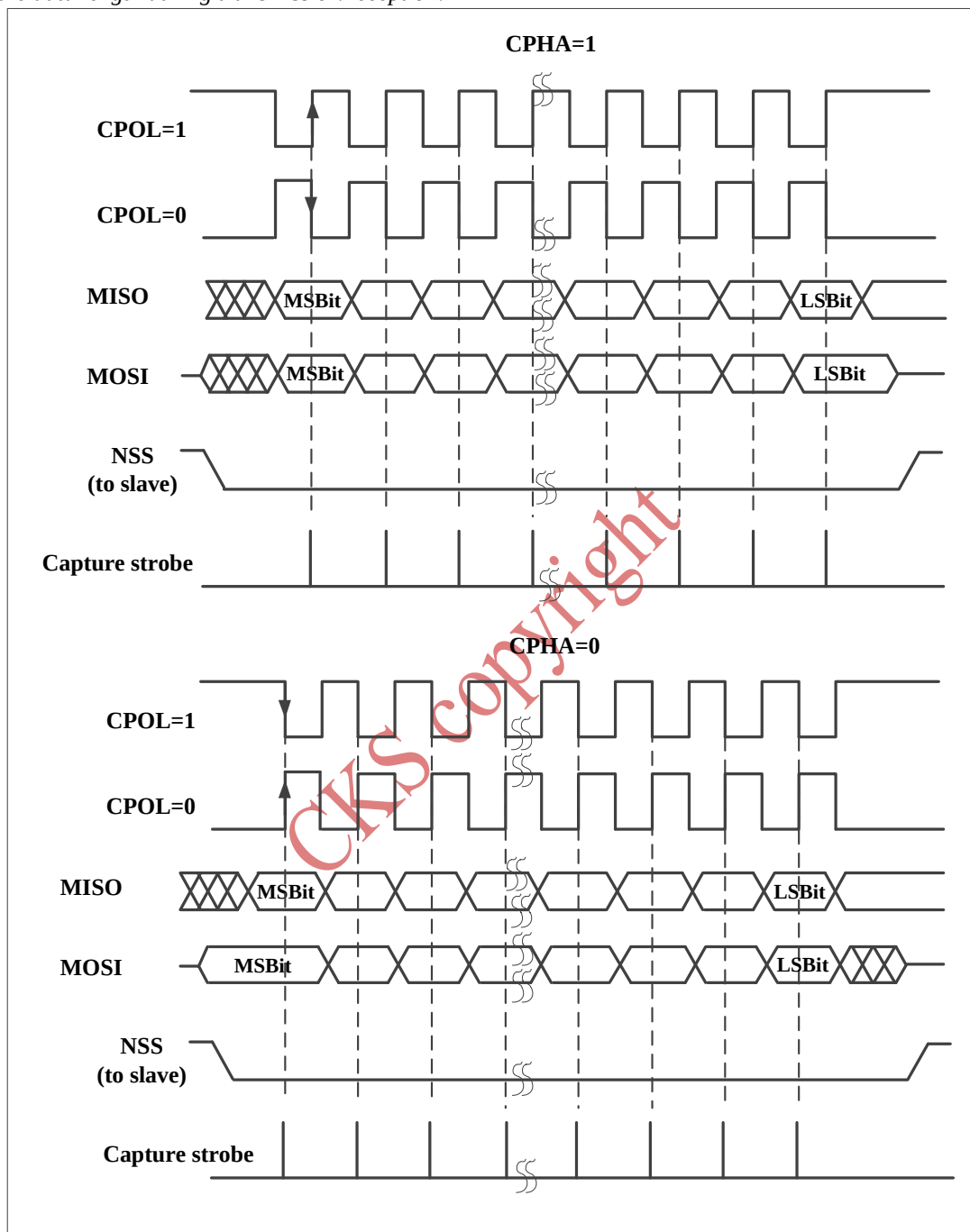


Figure 155 Data clock timing diagram

1. These timings are shown with the LSBFIRST bit reset in the SPI_CR1 register.

Data frame format

Data can be shifted out either MSB-first or LSB-first depending on the value of the LSBFIRST bit in the SPI_CR1 Register.

Each data frame is 8 or 16 bits long depending on the size of the data programmed using the DFF bit in the SPI_CR1 register. The selected data frame format is applicable for transmission and/or reception.

18.3.2 Configuring the SPI in slave mode

In the slave configuration, the serial clock is received on the SCK pin from the master device. The value set in the BR[2:0] bits in the SPI_CR1 register, does not affect the data transfer rate.

Note: It is recommended to enable the SPI slave before the master sends the clock. If not, undesired data transmission might occur. The data register of the slave needs to be ready before the first edge of the communication clock or before the end of the ongoing communication. It is mandatory to have the polarity of the communication clock set to the steady state value before the slave and the master are enabled.

Follow the procedure below to configure the SPI in slave mode:

Procedure

1. Set the DFF bit to define 8- or 16-bit data frame format
2. Select the CPOL and CPHA bits to define one of the four relationships between the data transfer and the serial clock (see Figure 155). For correct data transfer, the CPOL and CPHA bits must be configured in the same way in the slave device and the master device.
3. The frame format (MSB-first or LSB-first depending on the value of the LSBFIRST bit in the SPI_CR1 register) must be the same as the master device.
4. In Hardware mode (refer to *Slave select (NSS) pin management*), the NSS pin must be connected to a low level signal during the complete byte transmit sequence. In NSS software mode, set the SSM bit and clear the SSI bit in the SPI_CR1 register.
5. Clear the MSTR bit and set the SPE bit (both in the SPI_CR1 register) to assign the pins to alternate functions.

In this configuration the MOSI pin is a data input and the MISO pin is a data output.

Transmit sequence

The data byte is parallel-loaded into the Tx buffer during a write cycle.

The transmit sequence begins when the slave device receives the clock signal and the most significant bit of the data on its MOSI pin. The remaining bits (the 7 bits in 8-bit data frame format, and the 15 bits in 16-bit data frame format) are loaded into the shift-register. The TXE flag in the SPI_SR register is set on the transfer of data from the Tx Buffer to the shift register and an interrupt is generated if the TXEIE bit in the SPI_CR2 register is set.

Receive sequence

For the receiver, when data transfer is complete:

- The Data in shift register is transferred to Rx Buffer and the RXNE flag (SPI_SR register) is set
- An Interrupt is generated if the RXNEIE bit is set in the SPI_CR2 register.

After the last sampling clock edge the RXNE bit is set, a copy of the data byte received in the shift register is moved to the Rx buffer. When the SPI_DR register is read, the SPI peripheral returns this buffered value.

Clearing of the RXNE bit is performed by reading the SPI_DR register.

18.3.3 Configuring the SPI in master mode

In the master configuration, the serial clock is generated on the SCK pin.

Procedure

1. Select the BR[2:0] bits to define the serial clock baud rate (see SPI_CR1 register).
2. Select the CPOL and CPHA bits to define one of the four relationships between the data transfer and the serial clock (see Figure 155).
3. Set the DFF bit to define 8- or 16-bit data frame format
4. Configure the LSBFIRST bit in the SPI_CR1 register to define the frame format.

5. If the NSS pin is required in input mode, in hardware mode, connect the NSS pin to a high-level signal during the complete byte transmit sequence. In NSS software mode, set the SSM and SSI bits in the SPI_CR1 register. If the NSS pin is required in output mode, the SSOE bit only should be set.
6. The MSTR and SPE bits must be set (they remain set only if the NSS pin is connected to a high-level signal).

In this configuration the MOSI pin is a data output and the MISO pin is a data input.

Transmit sequence

The transmit sequence begins when a byte is written in the Tx Buffer.

The data byte is parallel-loaded into the shift register (from the internal bus) during the first bit transmission and then shifted out serially to the MOSI pin MSB first or LSB first depending on the LSBFIRST bit in the SPI_CR1 register. The TXE flag is set on the transfer of data from the Tx Buffer to the shift register and an interrupt is generated if the TXEIE bit in the SPI_CR1 register is set.

Receive sequence

For the receiver, when data transfer is complete:

- The data in the shift register is transferred to the Rx Buffer and the RXNE flag is set
- An interrupt is generated if the RXNEIE bit is set in the SPI_CR2 register

At the last sampling clock edge the RXNE bit is set, a copy of the data byte received in the shift register is moved to the Rx buffer. When the SPI_DR register is read, the SPI peripheral returns this buffered value.

Clearing the RXNE bit is performed by reading the SPI_DR register.

A continuous transmit stream can be maintained if the next data to be transmitted is put in the Tx buffer once the transmission is started. Note that TXE flag should be '1' before any attempt to write the Tx buffer is made.

Note: When a master is communicating with SPI slaves which need to be de-selected between transmissions, the NSS pin must be configured as GPIO or another GPIO must be used and toggled by software.

18.3.4 Configuring the SPI for half-duplex communication

The SPI is capable of operating in half-duplex mode in 2 configurations.

- 1 clock and 1 bidirectional data wire
- 1 clock and 1 data wire (receive-only or transmit-only)

1 clock and 1 bidirectional data wire (BIDIMODE = 1)

This mode is enabled by setting the BIDIMODE bit in the SPI_CR1 register. In this mode SCK is used for the clock and MOSI in master or MISO in slave mode is used for data communication. The transfer direction (Input/Output) is selected by the BIDIOE bit in the SPI_CR1 register. When this bit is 1, the data line is output otherwise it is input.

1 clock and 1 unidirectional data wire (BIDIMODE = 0)

In this mode, the application can use the SPI either in transmit-only mode or in receive-only mode.

- Transmit-only mode is similar to full-duplex mode (BIDIMODE=0, RXONLY=0): the data are transmitted on the transmit pin (MOSI in master mode or MISO in slave mode) and the receive pin (MISO in master mode or MOSI in slave mode) can be used as a general-purpose IO. In this case, the application just needs to ignore the Rx buffer (if the data register is read, it does not contain the received value).
- In receive-only mode, the application can disable the SPI output function by setting the RXONLY bit in the SPI_CR1 register. In this case, it frees the transmit IO pin (MOSI in master mode or MISO in slave mode), so it can be used for other purposes.

To start the communication in receive-only mode, configure and enable the SPI:

- In master mode, the communication starts immediately and stops when the SPE bit is cleared and the current reception stops. There is no need to read the BSY flag in this mode. It is always set when an SPI communication is ongoing.
- In slave mode, the SPI continues to receive as long as the NSS is pulled down (or the SSI bit is cleared in NSS software mode) and the SCK is running.

18.3.5 Data transmission and reception procedures

Rx and Tx buffers

In reception, data are received and then stored into an internal Rx buffer while In transmission, data are first stored into an internal Tx buffer before being transmitted.

A read access of the SPI_DR register returns the Rx buffered value whereas a write access to the SPI_DR stores the written data into the Tx buffer.

Start sequence in master mode

- In full-duplex (BIDIMODE=0 and RXONLY=0)
 - The sequence begins when data are written into the SPI_DR register (Tx buffer).
 - The data are then parallel loaded from the Tx buffer into the 8-bit shift register during the first bit transmission and then shifted out serially to the MOSI pin.
 - At the same time, the received data on the MISO pin is shifted in serially to the 8-bit shift register and then parallel loaded into the SPI_DR register (Rx buffer).
- In unidirectional receive-only mode (BIDIMODE=0 and RXONLY=1)
 - The sequence begins as soon as SPE=1
 - Only the receiver is activated and the received data on the MISO pin are shifted in serially to the 8-bit shift register and then parallel loaded into the SPI_DR register (Rx buffer).
- In bidirectional mode, when transmitting (BIDIMODE=1 and BIDIOE=1)
 - The sequence begins when data are written into the SPI_DR register (Tx buffer).
 - The data are then parallel loaded from the Tx buffer into the 8-bit shift register during the first bit transmission and then shifted out serially to the MOSI pin.
 - No data are received.
- In bidirectional mode, when receiving (BIDIMODE=1 and BIDIOE=0)
 - The sequence begins as soon as SPE=1 and BIDIOE=0.
 - The received data on the MOSI pin are shifted in serially to the 8-bit shift register and then parallel loaded into the SPI_DR register (Rx buffer).
 - The transmitter is not activated and no data are shifted out serially to the MOSI pin.

Start sequence in slave mode

- In full-duplex mode (BIDIMODE=0 and RXONLY=0)
 - The sequence begins when the slave device receives the clock signal and the first bit of the data on its MOSI pin. The 7 remaining bits are loaded into the shift register.
 - At the same time, the data are parallel loaded from the Tx buffer into the 8-bit shift register during the first bit transmission, and then shifted out serially to the MISO pin. The software must have written the data to be sent before the SPI master device initiates the transfer.
- In unidirectional receive-only mode (BIDIMODE=0 and RXONLY=1)
 - The sequence begins when the slave device receives the clock signal and the first bit of the data on its MOSI pin. The 7 remaining bits are loaded into the shift register.
 - The transmitter is not activated and no data are shifted out serially to the MISO pin
- In bidirectional mode, when transmitting (BIDIMODE=1 and BIDIOE=1)
 - The sequence begins when the slave device receives the clock signal and the first bit in the Tx buffer is transmitted on the MISO pin.
 - The data are then parallel loaded from the Tx buffer into the 8-bit shift register during the first bit transmission and then shifted out serially to the MISO pin. The software must have written the data to be sent before the SPI master device initiates the transfer.
 - No data are received.
- In bidirectional mode, when receiving (BIDIMODE=1 and BIDIOE=0)
 - The sequence begins when the slave device receives the clock signal and the first bit of the data on its MISO pin.
 - The received data on the MISO pin are shifted in serially to the 8-bit shift register and then parallel loaded into the SPI_DR register (Rx buffer).
 - The transmitter is not activated and no data are shifted out serially to the MISO pin.

Handling data transmission and reception

The TXE flag (Tx buffer empty) is set when the data are transferred from the Tx buffer to the shift register. It indicates that the internal Tx buffer is ready to be loaded with the next data. An interrupt

can be generated if the TXEIE bit in the SPI_CR2 register is set. Clearing the TXE bit is performed by writing to the SPI_DR register.

Note: The software must ensure that the TXE flag is set to 1 before attempting to write to the Tx buffer. Otherwise, it overwrites the data previously written to the Tx buffer.

The RXNE flag (Rx buffer not empty) is set on the last sampling clock edge, when the data are transferred from the shift register to the Rx buffer. It indicates that data are ready to be read from the SPI_DR register. An interrupt can be generated if the RXNEIE bit in the SPI_CR2 register is set. Clearing the RXNE bit is performed by reading the SPI_DR register.

For some configurations, the BSY flag can be used during the last data transfer to wait until the completion of the transfer.

Full-duplex transmit and receive procedure in master or slave mode (BIDIMODE=0 and RXONLY=0)

The software has to follow this procedure to transmit and receive data (see Figure 156 and Figure 157):

1. Enable the SPI by setting the SPE bit to 1.
2. Write the first data item to be transmitted into the SPI_DR register (this clears the TXE flag).
3. Wait until TXE=1 and write the second data item to be transmitted. Then wait until RXNE=1 and read the SPI_DR to get the first received data item (this clears the RXNE bit). Repeat this operation for each data item to be transmitted/received until the n-1 received data.
4. Wait until RXNE=1 and read the last received data.
5. Wait until TXE=1 and then wait until BSY=0 before disabling the SPI.

This procedure can also be implemented using dedicated interrupt subroutines launched at each rising edges of the RXNE or TXE flag.

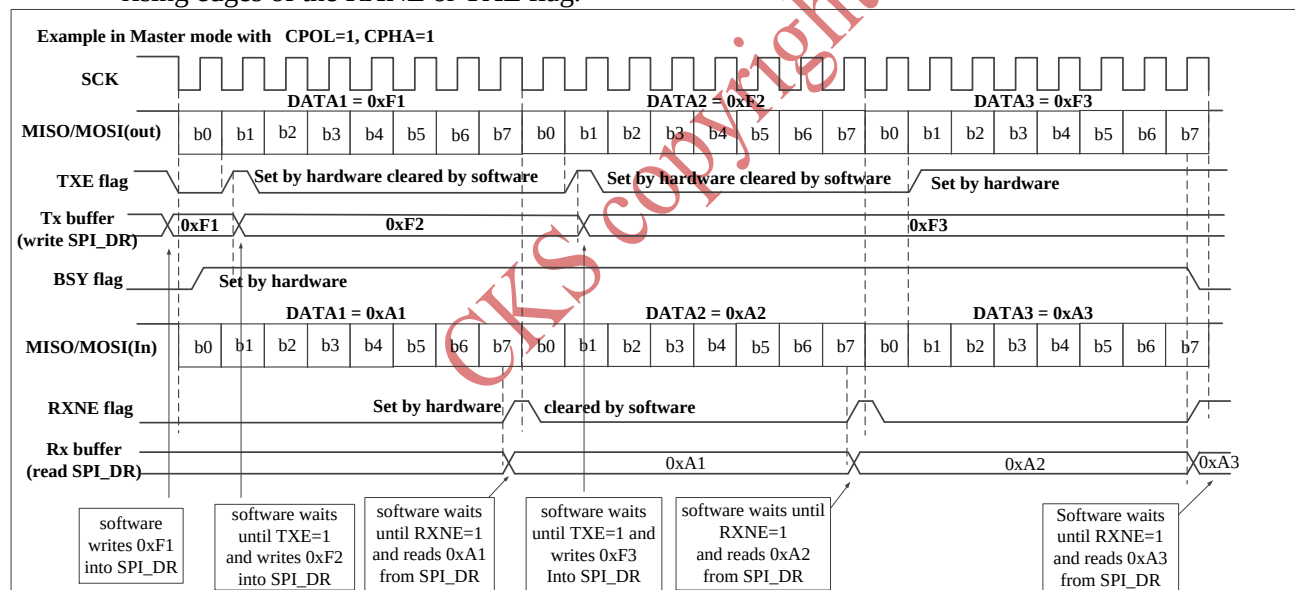


Figure 156 TXE/RXNE/BSY behavior in Master / full-duplex mode (BIDIMODE=0 and RXONLY=0) in case of continuous transfers

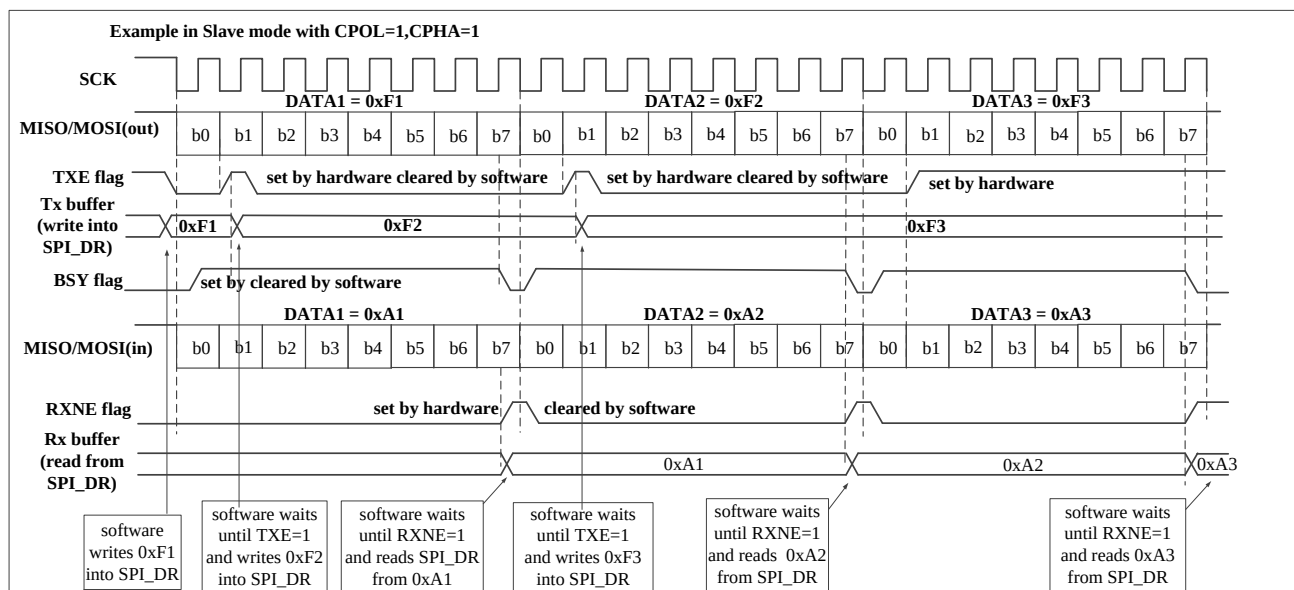


Figure 157 TXE/RXNE/BSY behavior in Slave / full-duplex mode
(BIDIMODE=0, RXONLY=0) in case of continuous transfers

Transmit-only procedure (BIDIMODE=0 RXONLY=0)

In this mode, the procedure can be reduced as described below and the BSY bit can be used to wait until the completion of the transmission (see Figure 158 and Figure 159).

1. Enable the SPI by setting the SPE bit to 1.
2. Write the first data item to send into the SPI_DR register (this clears the TXE bit).
3. Wait until TXE=1 and write the next data item to be transmitted. Repeat this step for each data item to be transmitted.
4. After writing the last data item into the SPI_DR register, wait until TXE=1, then wait until BSY=0, this indicates that the transmission of the last data is complete.

This procedure can be also implemented using dedicated interrupt subroutines launched at each rising edge of the TXE flag.

Note: During discontinuous communications, there is a 2 APB clock period delay between the write operation to SPI_DR and the BSY bit setting. As a consequence, in transmit-only mode, it is mandatory to wait first until TXE is set and then until BSY is cleared after writing the last data. After transmitting two data items in transmit-only mode, the OVR flag is set in the SPI_SR register since the received data are never read.

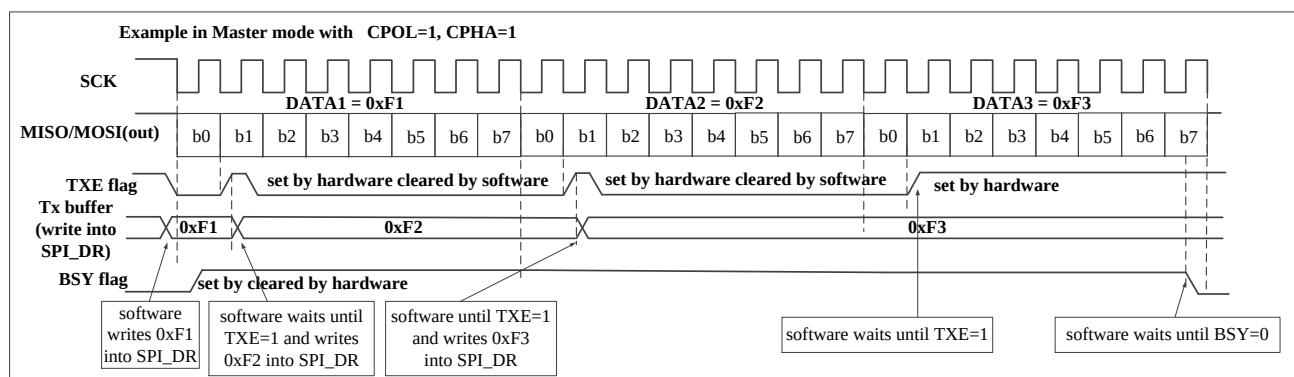


Figure 158 TXE/BSY behavior in Master transmit-only mode
(BIDIMODE=0 and RXONLY=0) in case of continuous transfers

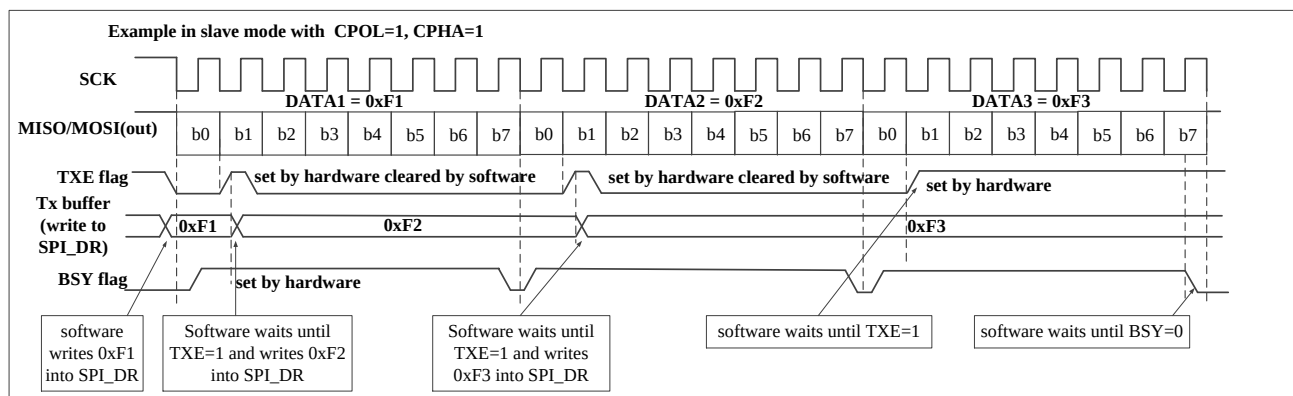


Figure 159 TXE/BSY in Slave transmit-only mode (BIDIMODE=0 and RXONLY=0) in case of continuous transfers

Bidirectional transmit procedure (BIDIMODE=1 and BIDIOE=1)

In this mode, the procedure is similar to the procedure in Transmit-only mode except that the BIDIMODE and BIDIOE bits both have to be set in the SPI_CR2 register before enabling the SPI.

Unidirectional receive-only procedure (BIDIMODE=0 and RXONLY=1)

In this mode, the procedure can be reduced as described below (see Figure 160):

1. Set the RXONLY bit in the SPI_CR1 register.
2. Enable the SPI by setting the SPE bit to 1:
 - a) In master mode, this immediately activates the generation of the SCK clock, and data are serially received until the SPI is disabled (SPE=0).
 - b) In slave mode, data are received when the SPI master device drives NSS low and generates the SCK clock.
3. Wait until RXNE=1 and read the SPI_DR register to get the received data (this clears the RXNE bit). Repeat this operation for each data item to be received.

This procedure can also be implemented using dedicated interrupt subroutines launched at each rising edge of the RXNE flag.

Note: If it is required to disable the SPI after the last transfer, follow the recommendation described in Section 18.3.8: Disabling the SPI.

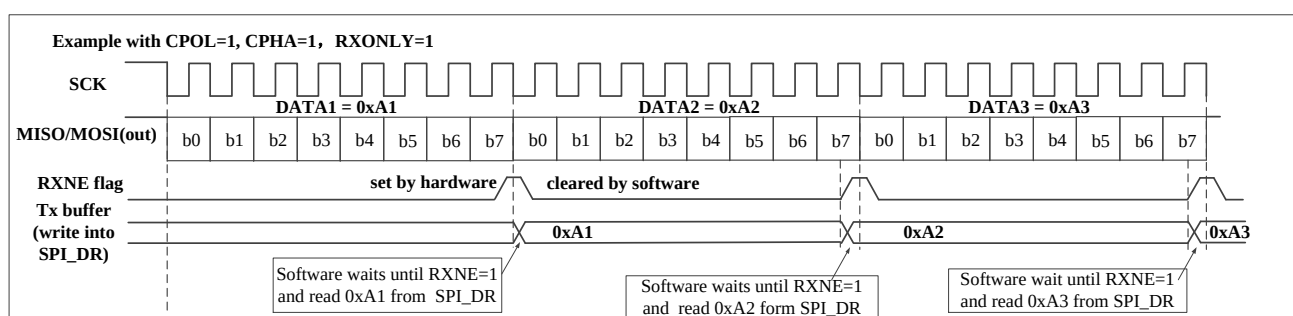


Figure 160 RXNE behavior in receive-only mode (BIDIRMODE=0 and RXONLY=1) in case of continuous transfers

Bidirectional receive procedure (BIDIMODE=1 and BIDIOE=0)

In this mode, the procedure is similar to the Receive-only mode procedure except that the BIDIMODE bit has to be set and the BIDIOE bit cleared in the SPI_CR2 register before enabling the SPI.

Continuous and discontinuous transfers

When transmitting data in master mode, if the software is fast enough to detect each rising edge of TXE (or TXE interrupt) and to immediately write to the SPI_DR register before the ongoing data transfer is complete, the communication is said to be continuous. In this case, there is no discontinuity in the generation of the SPI clock between each data item and the BSY bit is never cleared between each data transfer.

On the contrary, if the software is not fast enough, this can lead to some discontinuities in the communication. In this case, the BSY bit is cleared between each data transmission (see Figure 161).

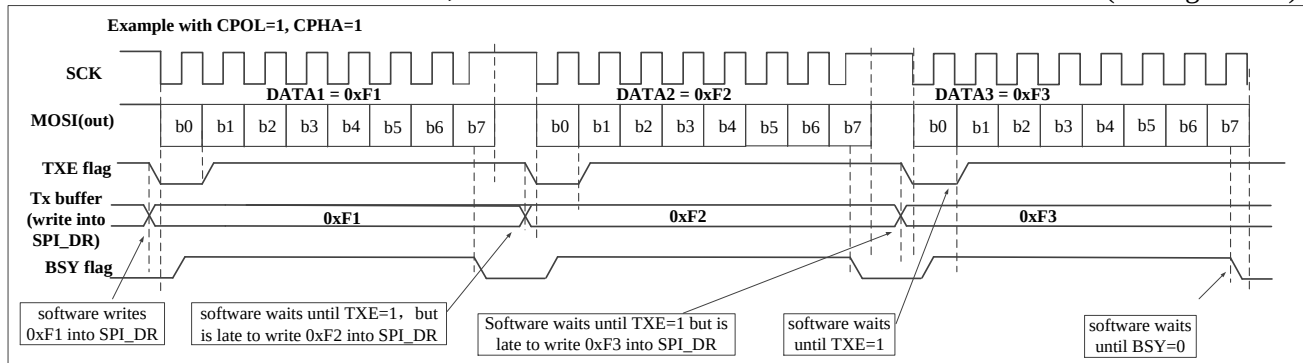


Figure 161 TXE/BSY behavior when transmitting (BIDIRMODE=0 and RXONLY=0) in case of discontinuous transfers

18.3.6 CRC calculation

A CRC calculator has been implemented for communication reliability. Separate CRC calculators are implemented for transmitted data and received data. The CRC is calculated using a programmable polynomial serially on each bit. It is calculated on the sampling clock edge defined by the CPHA and CPOL bits in the SPI_CR1 register.

Note: This SPI offers two kinds of CRC calculation standard which depend directly on the data frame format selected for the transmission and/or reception: 8-bit data (CR8) and 16-bit data (CRC16).

CRC calculation is enabled by setting the CRCEN bit in the SPI_CR1 register. This action resets the CRC registers (SPI_RXCRCR and SPI_TXCRCR). In full duplex or transmitter only mode, when the transfers are managed by the software (CPU mode), it is necessary to write the bit CRCNEXT immediately after the last data to be transferred is written to the SPI_DR. At the end of this last data transfer, the SPI_TXCRCR value is transmitted.

In receive only mode and when the transfers are managed by software (CPU mode), it is necessary to write the CRCNEXT bit after the second last data has been received. The CRC is received just after the last data reception and the CRC check is then performed.

At the end of data and CRC transfers, the CRCERR flag in the SPI_SR register is set if corruption occurs during the transfer.

If data are present in the TX buffer, the CRC value is transmitted only after the transmission of the data byte. During CRC transmission, the CRC calculator is switched off and the register value remains unchanged.

SPI communication using the CRC is possible through the following procedure:

- Program the CPOL, CPHA, LSBFirst, BR, SSM, SSI and MSTR values.
- Program the polynomial in the SPI_CRCPR register.
- Enable the CRC calculation by setting the CRCEN bit in the SPI_CR1 register. This also clears the SPI_RXCRCR and SPI_TXCRCR registers.
- Enable the SPI by setting the SPE bit in the SPI_CR1 register.
- Start the communication and sustain the communication until all but one byte or half-word have been transmitted or received.
- After the transfer of the last byte or half word, the SPI enters the CRC transfer and check phase. In full duplex mode or receiver-only mode, the received CRC is compared to the SPI_RXCRCR value. If the value does not match, the CRCERR flag in SPI_SR is set and an interrupt can be generated when the ERRRIE bit in the SPI_CR2 register is set.

Note: When the SPI is in slave mode, be careful to enable CRC calculation only when the clock is stable, that is, when the clock is in the steady state. If not, a wrong CRC calculation may be done. In fact, the CRC is sensitive to the SCK slave input clock as soon as CRCEN is set, and this, whatever the value of the SPE bit.

With high bitrate frequencies, be careful when transmitting the CRC. As the number of used CPU cycles has to be as low as possible in the CRC transfer phase, it is forbidden to call software functions in the CRC transmission sequence to avoid errors in the last data and CRC reception. In fact, CRCNEXT bit has to be written before the end of the transmission/reception of the last data.

For high bit rate frequencies, it is advised to use the DMA mode to avoid the degradation of the SPI speed performance due to CPU accesses impacting the SPI bandwidth.

When the devices are configured as slaves and the NSS hardware mode is used, the NSS pin needs to be kept low between the data phase and the CRC phase.

When the SPI is configured in slave mode with the CRC feature enabled, CRC calculation takes place even if a high level is applied on the NSS pin. This may happen for example in case of a multislave environment where the communication master addresses slaves alternately.

Between a slave deselection (high level on NSS) and a new slave selection (low level on NSS), the CRC value should be cleared on both master and slave sides in order to resynchronize the master and slave for their respective CRC calculation.

To clear the CRC, follow the procedure below:

1. Disable SPI (SPE = 0)
2. Clear the CRCEN bit
3. Set the CRCEN bit
4. Enable the SPI (SPE = 1)

18.3.7 Status flags

Three status flags are provided for the application to completely monitor the state of the SPI bus.

Tx buffer empty flag (TXE)

When it is set, this flag indicates that the Tx buffer is empty and the next data to be transmitted can be loaded into the buffer. The TXE flag is cleared when writing to the SPI_DR register.

Rx buffer not empty (RXNE)

When set, this flag indicates that there are valid received data in the Rx buffer. It is cleared when SPI_DR is read.

BUSY flag

This BSY flag is set and cleared by hardware (writing to this flag has no effect). The BSY flag indicates the state of the communication layer of the SPI.

When BSY is set, it indicates that the SPI is busy communicating. There is one exception in master mode / bidirectional receive mode (MSTR=1 and BDM=1 and BDOE=0) where the BSY flag is kept low during reception.

The BSY flag is useful to detect the end of a transfer if the software wants to disable the SPI and enter Halt mode (or disable the peripheral clock). This avoids corrupting the last transfer. For this, the procedure described below must be strictly respected.

The BSY flag is also useful to avoid write collisions in a multimaster system.

The BSY flag is set when a transfer starts, with the exception of master mode / bidirectional receive mode (MSTR=1 and BDM=1 and BDOE=0).

It is cleared:

- when a transfer is finished (except in master mode if the communication is continuous)
- when the SPI is disabled
- when a master mode fault occurs (MODF=1)

When communication is not continuous, the BSY flag is low between each communication

When communication is continuous:

- in master mode, the BSY flag is kept high during all the transfers
- in slave mode, the BSY flag goes low for one SPI clock cycle between each transfer

Note: Do not use the BSY flag to handle each data transmission or reception. It is better to use the TXE and RXNE flags instead.

18.3.8 Disabling the SPI

When a transfer is terminated, the application can stop the communication by disabling the SPI peripheral. This is done by clearing the SPE bit.

For some configurations, disabling the SPI and entering the Halt mode while a transfer is ongoing can cause the current transfer to be corrupted and/or the BSY flag might become unreliable.◦

To avoid any of those effects, it is recommended to respect the following procedure when disabling the SPI:

In master or slave full-duplex mode (BIDIMODE=0, RXONLY=0)

1. Wait until RXNE=1 to receive the last data
2. Wait until TXE=1
3. Then wait until BSY=0
4. Disable the SPI (SPE=0) and, eventually, enter the Halt mode (or disable the peripheral clock)

In master or slave unidirectional transmit-only mode (BIDIMODE=0, RXONLY=0) or bidirectional transmit mode (BIDIMODE=1, BIDIOE=1)

After the last data is written into the SPI_DR register:

1. Wait until TXE=1
2. Then wait until BSY=0
3. Disable the SPI (SPE=0) and, eventually, enter the Halt mode (or disable the peripheral clock)

In master unidirectional receive-only mode (MSTR=1, BIDIMODE=0, RXONLY=1) or bidirectional receive mode (MSTR=1, BIDIMODE=1, BIDIOE=0)

This case must be managed in a particular way to ensure that the SPI does not initiate a new transfer:

1. Wait for the second to last occurrence of RXNE=1 (n-1)
2. Then wait for one SPI clock cycle (using a software loop) before disabling the SPI (SPE=0)
3. Then wait for the last RXNE=1 before entering the Halt mode (or disabling the peripheral clock)

Note: In master bidirectional receive mode (MSTR=1 and BDM=1 and BDOE=0), the BSY flag is kept low during transfers.

In slave receive-only mode (MSTR=0, BIDIMODE=0, RXONLY=1) or bidirectional receive mode (MSTR=0, BIDIMODE=1, BDOE=0)

1. You can disable the SPI (write SPE=1) at any time: the current transfer will complete before the SPI is effectively disabled
2. Then, if you want to enter the Halt mode, you must first wait until BSY = 0 before entering the Halt mode (or disabling the peripheral clock).

18.3.9 SPI communication using DMA (direct memory addressing)

To operate at its maximum speed, the SPI needs to be fed with the data for transmission and the data received on the Rx buffer should be read to avoid overrun. To facilitate the transfers, the SPI features a DMA capability implementing a simple request/acknowledge protocol.

A DMA access is requested when the enable bit in the SPI_CR2 register is enabled. Separate requests must be issued to the Tx and Rx buffers (see Figure 162 and Figure 163):◦

- In transmission, a DMA request is issued each time TXE is set to 1. The DMA then writes to the SPI_DR register (this clears the TXE flag).
- In reception, a DMA request is issued each time RXNE is set to 1. The DMA then reads the SPI_DR register (this clears the RXNE flag).

When the SPI is used only to transmit data, it is possible to enable only the SPI Tx DMA channel. In this case, the OVR flag is set because the data received are not read.

When the SPI is used only to receive data, it is possible to enable only the SPI Rx DMA channel.

In transmission mode, when the DMA has written all the data to be transmitted (flag TCIF is set in the DMA_ISR register), the BSY flag can be monitored to ensure that the SPI communication is complete. This is required to avoid corrupting the last transmission before disabling the SPI or entering the Stop mode. The software must first wait until TXE=1 and then until BSY=0.

Note: During discontinuous communications, there is a 2 APB clock period delay between the write operation to SPI_DR and the BSY bit setting. As a consequence, it is mandatory to wait first until TXE=1 and then until BSY=0 after writing the last data.

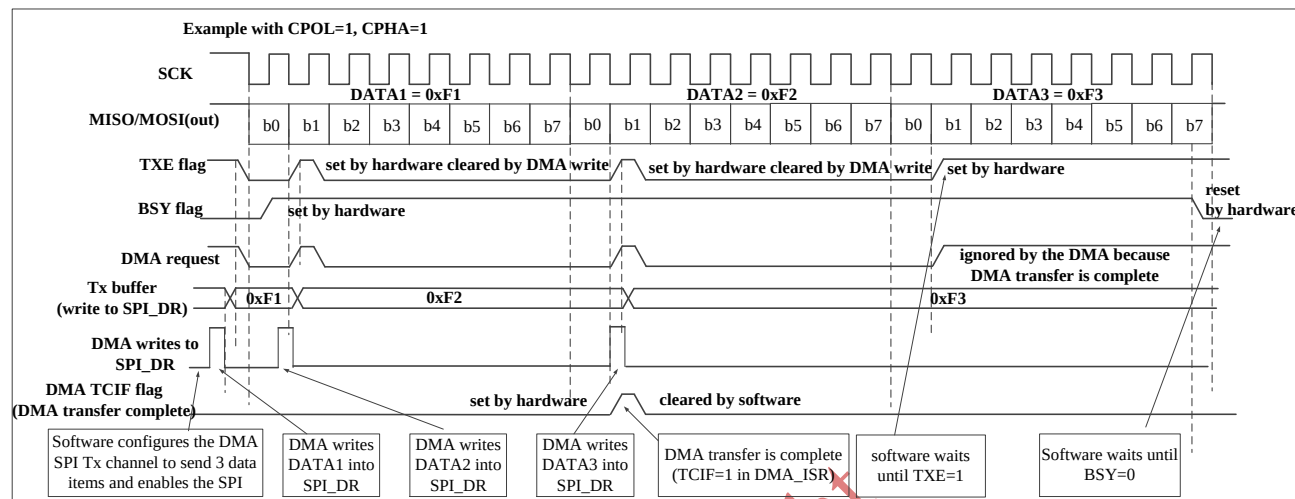


Figure 162 Transmission using DMA

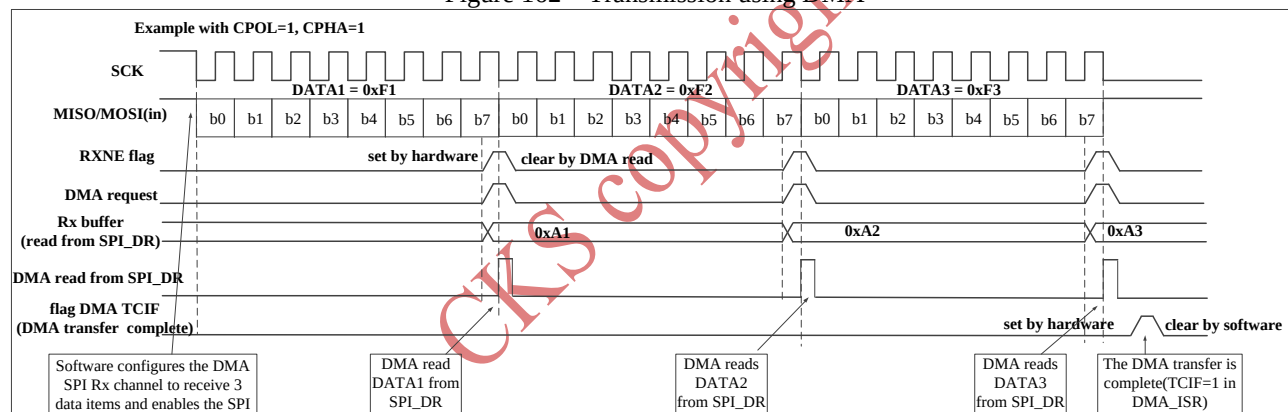


Figure 163 Reception using DMA

DMA capability with CRC

When SPI communication is enabled with CRC communication and DMA mode, the transmission and reception of the CRC at the end of communication are automatic that is without using the bit CRCNEXT. After the CRC reception, the CRC must be read in the SPI_DR register to clear the RXNE flag.

At the end of data and CRC transfers, the CRCERR flag in SPI_SR is set if corruption occurs during the transfer.

18.3.10 Error flags

Master mode fault (MODF)

Master mode fault occurs when the master device has its NSS pin pulled low (in NSS hardware mode) or SSI bit low (in NSS software mode), this automatically sets the MODF bit. Master mode fault affects the SPI peripheral in the following ways:

- The MODF bit is set and an SPI interrupt is generated if the ERRIE bit is set.
- The SPE bit is cleared. This blocks all output from the device and disables the SPI interface.

- The MSTR bit is cleared, thus forcing the device into slavemode.

Use the following software sequence to clear the MODF bit:

1. Make a read or write access to the SPI_SR register while the MODF bit is set.
2. Then write to the SPI_CR1 register.

To avoid any multiple slave conflicts in a system comprising several MCUs, the NSS pin must be pulled high during the MODF bit clearing sequence. The SPE and MSTR bits can be restored to their original state after this clearing sequence.

As a security, hardware does not allow the setting of the SPE and MSTR bits while the MODF bit is set.

In a slave device the MODF bit cannot be set. However, in a multimaster configuration, the device can be in slave mode with this MODF bit set. In this case, the MODF bit indicates that there might have been a multimaster conflict for system control. An interrupt routine can be used to recover cleanly from this state by performing a reset or returning to a default state.

Overrun condition

An overrun condition occurs when the master device has sent data bytes and the slave device has not cleared the RXNE bit resulting from the previous data byte transmitted. When an overrun condition occurs:

- the OVR bit is set and an interrupt is generated if the ERRIE bit is set.

In this case, the receiver buffer contents will not be updated with the newly received data from the master device. A read from the SPI_DR register returns this byte. All other subsequently transmitted bytes are lost.

Clearing the OVR bit is done by a read from the SPI_DR register followed by a read access to the SPI_SR register.

CRC error

This flag is used to verify the validity of the value received when the CRCEN bit in the SPI_CR1 register is set. The CRCERR flag in the SPI_SR register is set if the value received in the shift register does not match the receiver SPI_RXCRCR value.

18.3.11 SPI interrupts

Table 77 SPI interrupt requests

Interrupt event	Event flag	Enable Control bit
Transmit buffer empty flag	TXE	TXEIE
Receive buffer not empty flag	RXNE	RXNEIE
Master Mode fault event	MODF	ERRIE
Overrun error	OVR	
CRC error flag	CRCERR	

18.4 SPI registers

The peripheral registers have to be accessed by half-words (16 bits) or words (32 bits).

18.4.1 SPI control register 1 (SPI_CR1)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDI MODE	BIDI OE	CRC EN	CRC NEXT	DFR	RX ONLY	SSM	SSI	LSB FIRST	SPE	BR[2: 0]		MSTR	CPOL	CPHA	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit 15		BIDIMODE: Bidirectional data mode enable 0: 2-line unidirectional data mode selected													

	1: 1-line bidirectional data mode selected
Bit 14	BIDIOE: Output enable in bidirectional mode This bit combined with the BIDImode bit selects the direction of transfer in bidirectional mode 0: Output disabled (receive-only mode) 1: Output enabled (transmit-only mode) <i>In master mode, the MOSI pin is used while the MISO pin is used in slave mode.</i>
Bit 13	CRCEN: Hardware CRC calculation enable 0: CRC calculation disabled 1: CRC calculation enabled <i>Note: This bit should be written only when SPI is disabled (SPE = '0') for correct operation. It is not used in I²S mode.</i>
Bit 12	CRCNEXT: CRC transfer next 0: Data phase (no CRC phase) 1: Next transfer is CRC (CRC phase) <i>Note: When the SPI is configured in full duplex or transmitter only modes, CRCNEXT must be written as soon as the last data is written to the SPI_DR register.</i>
Bit 11	DFF: Data frame format 0: 8-bit data frame format is selected for transmission/reception 1: 16-bit data frame format is selected for transmission/reception <i>Note: This bit should be written only when SPI is disabled (SPE = '0') for correct operation.</i>
Bit 10	RXONLY: Receive only This bit combined with the BIDImode bit selects the direction of transfer in 2-line unidirectional mode. This bit is also useful in a multislave system in which this particular slave is not accessed, the output from the accessed slave is not corrupted. 0: Full duplex (Transmit and receive) 1: Output disabled (Receive-only mode)
Bit 9	SSM: Software slave management When the SSM bit is set, the NSS pin input is replaced with the value from the SSI bit. 0: Software slave management disabled 1: Software slave management enabled
Bit 8	SSI: Internal slave select This bit has an effect only when the SSM bit is set. The value of this bit is forced onto the NSS pin and the IO value of the NSS pin is ignored.
Bit 7	LSBFIRST: Frame format 0: MSB transmitted first 1: LSB transmitted first <i>Note: This bit should not be changed when communication is ongoing.</i>
Bit 6	SPE: SPI enable 0: Peripheral disabled 1: Peripheral enabled <i>Note: When disabling the SPI, follow the procedure described in Section 18.3.8: Disabling the SPI.</i>
Bits 5:3	BR[2:0]: Baud rate control 000: f _{PCLK} /2 001: f _{PCLK} /4 010: f _{PCLK} /8 011: f _{PCLK} /16 100: f _{PCLK} /32 101: f _{PCLK} /64 110: f _{PCLK} /128 111: f _{PCLK} /256 <i>Note: These bits should not be changed when communication is ongoing.</i>
Bit 2	MSTR: Master selection 0: Slave configuration 1: Master configuration <i>Note: This bit should not be changed when communication is ongoing.</i>
Bit 1	CPOL: Clock polarity 0: CK to 0 when idle 1: CK to 1 when idle <i>Note: This bit should not be changed when communication is ongoing.</i>
Bit 0	CPHA: Clock phase 0: The first clock transition is the first data capture edge 1: The second clock transition is the first data capture edge <i>Note: This bit should not be changed when communication is ongoing.</i>

18.4.2 SPI control register 2 (SPI_CR2)

Address offset:0x04

Reset value:0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
----	----	----	----	----	----	---	---	---	---	---	---	---	---	---	---

Reserved	TXE IE	RXNE IE	ERR IE	Reserved	SSOE	TXDMA EN	RXDMA EN
	rw	rw	rw	res	rw	rw	rw
Bits 15:8	Reserved, must be kept at reset value.						
Bit 7	TXEIE : Tx buffer empty interrupt enable 0: TXE interrupt masked 1: TXE interrupt not masked. Used to generate an interrupt request when the TXE flag is set.						
Bit 6	RXNEIE : RX buffer not empty interrupt enable 0: RXNE interrupt masked 1: RXNE interrupt not masked. Used to generate an interrupt request when the RXNE flag is set.						
Bit 5	ERRIE : Error interrupt enable This bit controls the generation of an interrupt when an error condition occurs (CRCERR, OVR, MODF in SPI mode and UDR, OVR in I ² S mode). 0: Error interrupt is masked 1: Error interrupt is enabled						
Bits 4:3	Reserved, must be kept at reset value.						
Bit 2	SSOE : SS output enable 0: SS output is disabled in master mode and the cell can work in multimaster configuration 1: SS output is enabled in master mode and when the cell is enabled. The cell cannot work in a multimaster environment.						
Bit 1	TXDMAEN : Tx buffer DMA enable When this bit is set, the DMA request is made whenever the TXE flag is set. 0: Tx buffer DMA disabled 1: Tx buffer DMA enabled						
Bit 0	RXDMAEN : Rx buffer DMA enable When this bit is set, the DMA request is made whenever the RXNE flag is set. 0: Rx buffer DMA disabled 1: Rx buffer DMA enabled						

18.4.3 SPI status register (SPI_SR)

Address offset: 0x08

Reset value: 0x0002

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							BSY	OVR	MODF	CRC ERR	Reserved	TXE	RXNE		
							r	r	r	rc w0	r	r	r	r	
Bits 15:8	Reserved, must be kept at reset value.														
Bit 7	BSY: Busy flag 0: SPI not busy 1: SPI is busy in communication or Tx buffer is not empty This flag is set and cleared by hardware. <i>Note: BSY flag must be used with caution: refer to Section 18.3.7: Status flags and Section 18.3.8: Disabling the SPI.</i>														
Bit 6	OVR: Overrun flag 0: No overrun occurred 1: Overrun occurred This flag is set by hardware and reset by a software sequence.														
Bit 5	MODF: Mode fault 0: No mode fault occurred 1: Mode fault occurred This flag is set by hardware and reset by a software sequence. Refer to <i>Section 18.3.10: Error flags</i> for the software sequence.														
Bit 4	CRCERR: CRC error flag 0: CRC value received matches the SPI_RXCRCR value 1: CRC value received does not match the SPI_RXCRCR value This flag is set by hardware and cleared by software writing 0.														
Bits 3:2	Reserved														
Bit 1	TXE: Transmit buffer empty 0: Tx buffer not empty 1: Tx buffer empty														
Bit 0	RXNE: Receive buffer not empty														

	0: Rx buffer empty 1: Rx buffer not empty
--	--

18.4.4 SPI data register (SPI_DR)

Address offset: 0x0C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DR[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15:0		DR[15:0]: Data register Data received or to be transmitted. The data register is split into 2 buffers - one for writing (Transmit Buffer) and another one for reading (Receive buffer). A write to the data register will write into the Tx buffer and a read from the data register will return the value held in the Rx buffer. <i>Note: These notes apply to SPI mode:</i> Depending on the data frame format selection bit (DFF in SPI_CR1 register), the data sent or received is either 8-bit or 16-bit. This selection has to be made before enabling the SPI to ensure correct operation. For an 8-bit data frame, the buffers are 8-bit and only the LSB of the register (SPI_DR[7:0]) is used for transmission/reception. When in reception mode, the MSB of the register (SPI_DR[15:8]) is forced to 0. For a 16-bit data frame, the buffers are 16-bit and the entire register, SPI_DR[15:0] is used for transmission/reception.													

18.4.5 SPI CRC polynomial register (SPI_CRCPR)

Address offset: 0x10

Reset value: 0x0007

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRCPOLY[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
Bits 15:0		CRCPOLY[15:0]: CRC polynomial register This register contains the polynomial for the CRC calculation. The CRC polynomial (0007h) is the reset value of this register. Another polynomial can be configured as required.													

18.4.6 SPI RX CRC register (SPI_RXCRCR)

Address offset: 0x14

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RxCRC[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 15:0		RxCRC[15:0]: Rx CRC register When CRC calculation is enabled, the RxCRC[15:0] bits contain the computed CRC value of the subsequently received bytes. This register is reset when the CRCEN bit in SPI_CR1 register is written to 1. The CRC is calculated serially using the polynomial programmed in the SPI_CRCPR register. Only the 8 LSB bits are considered when the data frame format is set to be 8-bit data (DFF bit of SPI_CR1 is cleared). CRC calculation is done based on any CRC8 standard. The entire 16-bits of this register are considered when a 16-bit data frame format is selected (DFF bit of the SPI_CR1 register is set). CRC calculation is done based on any CRC16 standard. <i>Note: A read to this register when the BSY Flag is set could return an incorrect value.</i>													

18.4.7 SPI TX CRC register (SPI_TXCRCR)

Address offset: 0x18

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TxCRC[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 15:0			TXCRC[15:0]: Tx CRC register When CRC calculation is enabled, the TxCRC[7:0] bits contain the computed CRC value of the subsequently transmitted bytes. This register is reset when the CRCEN bit of SPI_CR1 is written to 1. The CRC is calculated serially using the polynomial programmed in the SPI_CRCPR register. Only the 8 LSB bits are considered when the data frame format is set to be 8-bit data (DFF bit of SPI_CR1 is cleared). CRC calculation is done based on any CRC8 standard. The entire 16-bits of this register are considered when a 16-bit data frame format is selected (DFF bit of the SPI_CR1 register is set). CRC calculation is done based on any CRC16 standard. <i>Note: A read to this register when the BSY flag is set could return an incorrect value.</i>												

18.4.8 SPI register map

Table 78 SPI register map and reset values

offset	000h		004h		008h		00Ch
register	Reset value	SPI_CR1	Reset value	SPI_CR2	Reset value	SPI_SR	SPI_DR
31	Reserved		Reserved		Reserved		Reserved
30							
29							
28							
27							
26							
25							
24							
23							
22							
21							
20							
19							
18							
17							
16							
15	0	BIDIMODE					
14	0	BIDIOE					
13	0	CRCEN					
12	0	CRCNEXT					
11	0	DFF					
10	0	RXOnly					
9	0	SSM					
8	0	SSI					
7	0	LSBFIRST					
6	0	SPE					
5	0	BR[2:0]					
4	0						
3	0						
2	0	MSTR					
1	0	CPOL					
0	0	CPHA					

	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
010h	SPI_CRCPR	Reserved	CRCPOLY[15:0]																
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
014h	SPI_RXCRCR	Reserved	RxCRC[15:0]																
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
018h	SPI_TXCRCR	Reserved	TxCRC[15:0]																
	Reset value		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Refer to Table 1 for the register boundary addresses.

19 Inter-integrated circuit (I2C) interface

19.1 I²C introduction

I²C (inter-integrated circuit) bus Interface serves as an interface between the microcontroller and the serial I²C bus. It provides multimaster capability, and controls all I²C bus-specific sequencing, protocol, arbitration and timing.

It may be used for a variety of purposes, including CRC generation and verification, SMBus (system management bus) and PMBus (power management bus).

Depending on specific device implementation DMA capability can be available for reduced CPU overload.

19.2 I²C main features

- Parallel-bus/I²C protocol converter
- Multimaster capability: the same interface can act as Master or Slave
- I²C Master features:
 - Clock generation
 - Start and Stop generation
- I²C Slave features:
 - Programmable I²C Address detection
 - Dual Addressing Capability to acknowledge 2 slave addresses
 - Stop bit detection
- Generation and detection of 7-bit/10-bit addressing and General Call
- Supports different communication speeds:
 - Standard Speed (up to 100 kHz)
 - Fast Speed (up to 400 kHz)
- Status flags:
 - Transmitter/Receiver mode flag
 - End-of-Byte transmission flag
 - I²C busy flag
- Error flags:
 - Arbitration lost condition for master mode
 - Acknowledgment failure after address/ data transmission
 - Detection of misplaced start or stop condition
 - Overrun/Underrun if clock stretching is disabled
- 2 Interrupt vectors:
 - 1 Interrupt for successful address/ data communication
 - 1 Interrupt for error condition
- Optional clock stretching
- 1-byte buffer with DMA capability
- Configurable PEC (packet error checking) generation or verification:
 - PEC value can be transmitted as last byte in Tx mode
 - PEC error checking for last received byte
- SMBus 2.0 Compatibility:
 - 25 ms clock low timeout delay

- 10 ms master cumulative clock low extend time
- 25 ms slave cumulative clock low extend time
- Hardware PEC generation/verification with ACK control
- Address Resolution Protocol (ARP) supported
- PMBus Compatibility

19.3 I²C functional description

In addition to receiving and transmitting data, this interface converts it from serial to parallel format and vice versa. The interrupts are enabled or disabled by software. The interface is connected to the I²C bus by a data pin (SDA) and by a clock pin (SCL). It can be connected with a standard (up to 100 kHz) or fast (up to 400 kHz) I²C bus.

19.3.1 Mode selection

The interface can operate in one of the four following modes:

- Slave transmitter
- Slave receiver
- Master transmitter
- Master receiver

By default, it operates in slave mode. The interface automatically switches from slave to master, after it generates a START condition and from master to slave, if an arbitration loss or a Stop generation occurs, allowing multimaster capability.

Communication flow

In Master mode, the I²C interface initiates a data transfer and generates the clock signal. A serial data transfer always begins with a start condition and ends with a stop condition. Both start and stop conditions are generated in master mode by software.

In Slave mode, the interface is capable of recognizing its own addresses (7 or 10-bit), and the General Call address. The General Call address detection may be enabled or disabled by software. Data and addresses are transferred as 8-bit bytes, MSB first. The first byte(s) following the start condition contain the address (one in 7-bit mode, two in 10-bit mode). The address is always transmitted in Master mode.

A 9th clock pulse follows the 8 clock cycles of a byte transfer, during which the receiver must send an acknowledge bit to the transmitter. Refer to *Figure 164*.

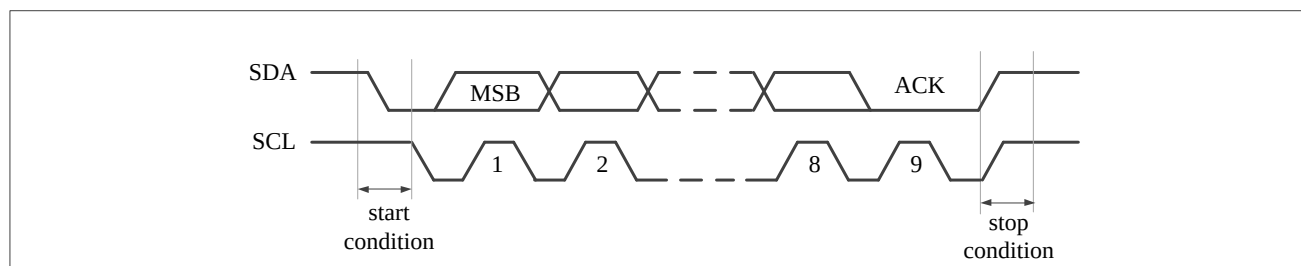


Figure 164 I²C bus protocol

Acknowledge may be enabled or disabled by software. The I²C interface addresses (dual addressing 7-bit/ 10-bit and/or general call address) can be selected by software.

The block diagram of the I²C interface is shown in Figure 165

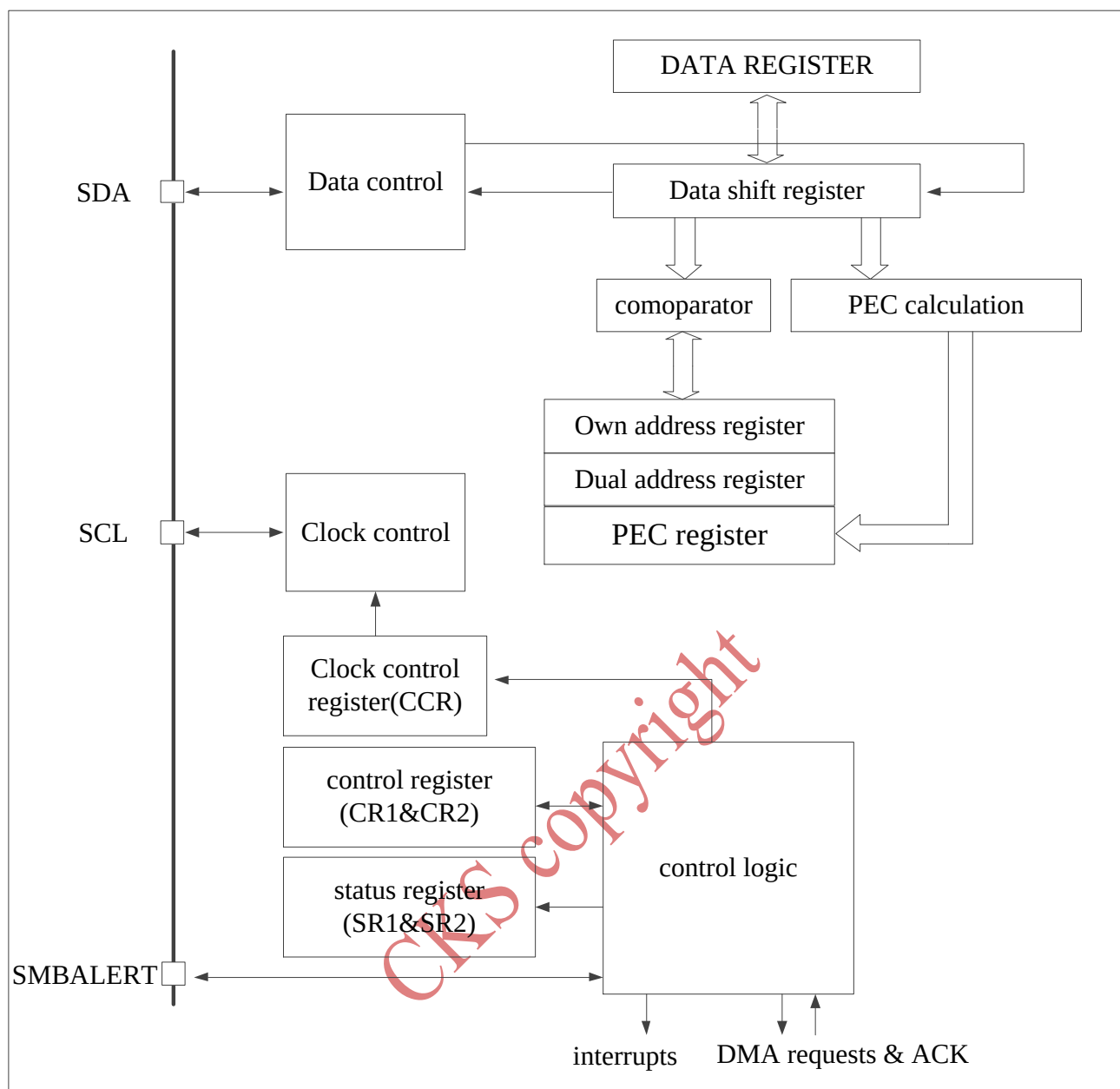


Figure 165 I²C block diagram

Note: SMBA is an optional signal in SMBus mode. This signal is not applicable if SMBus is disabled.

19.3.2 I²C slave mode

By default the I²C interface operates in Slave mode. To switch from default Slave mode to Master mode a Start condition generation is needed.

The peripheral input clock must be programmed in the I2C_CR2 register in order to generate correct timings. The peripheral input clock frequency must be at least:

- 2 MHz in Sm mode
- 4 MHz in Fm mode

As soon as a start condition is detected, the address is received from the SDA line and sent to the shift register. Then it is compared with the address of the interface (OAR1) and with OAR2 (if ENDUAL=1) or the General Call address (if ENGCG = 1).

Note: In 10-bit addressing mode, the comparison includes the header sequence (11110xx0), where xx denotes the two most significant bits of the address.

Header or address not matched: the interface ignores it and waits for another Start condition.

Header matched (10-bit mode only): the interface generates an acknowledge pulse if the ACK bit is set and waits for the 8-bit slave address.

Address matched: the interface generates in sequence:

- An acknowledge pulse if the ACK bit is set
- The ADDR bit is set by hardware and an interrupt is generated if the ITEVFEN bit is set.
- If ENDUAL=1, the software has to read the DUALF bit to check which slave address has been acknowledged.

In 10-bit mode, after receiving the address sequence the slave is always in Receiver mode. It will enter Transmitter mode on receiving a repeated Start condition followed by the header sequence with matching address bits and the least significant bit set (11110xx1).

The TRA bit indicates whether the slave is in Receiver or Transmitter mode.

Slave transmitter

Following the address reception and after clearing ADDR, the slave sends bytes from the DR register to the SDA line via the internal shift register.

The slave stretches SCL low until ADDR is cleared and DR filled with the data to be sent (see Figure 166 Transfer sequencing EV1 EV3).

When the acknowledge pulse is received:

- The TxE bit is set by hardware with an interrupt if the ITEVFEN and the ITBUFEN bits are set.

If TxE is set and some data were not written in the I2C_DR register before the end of the next data transmission, the BTF bit is set and the interface waits until BTF is cleared by a read to I2C_SR1 followed by a write to the I2C_DR register, stretching SCL low.

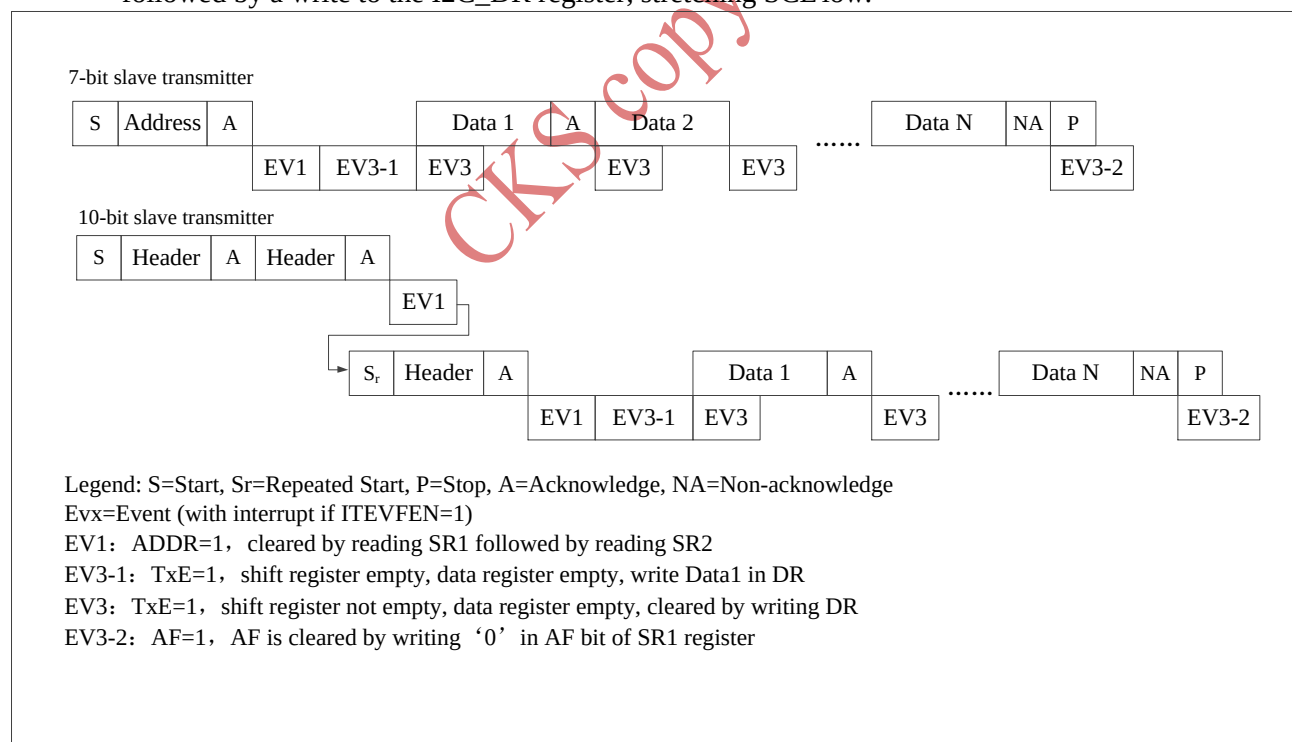


Figure 166 Transfer sequence diagram for slave transmitter

Slave receiver

Following the address reception and after clearing ADDR, the slave receives bytes from the SDA line into the DR register via the internal shift register. After each byte the interface generates in sequence:

- An acknowledge pulse if the ACK bit is set
 - The RxNE bit is set by hardware and an interrupt is generated if the ITEVFEN and ITBUFEN bit is set.
- If RxNE is set and the data in the DR register is not read before the end of the next data reception, the BTF bit is set and the interface waits until BTF is cleared by a read from I2C_SR1 followed by a read from the I2C_DR register, stretching SCL low (see Figure 167 Transfer sequencing)

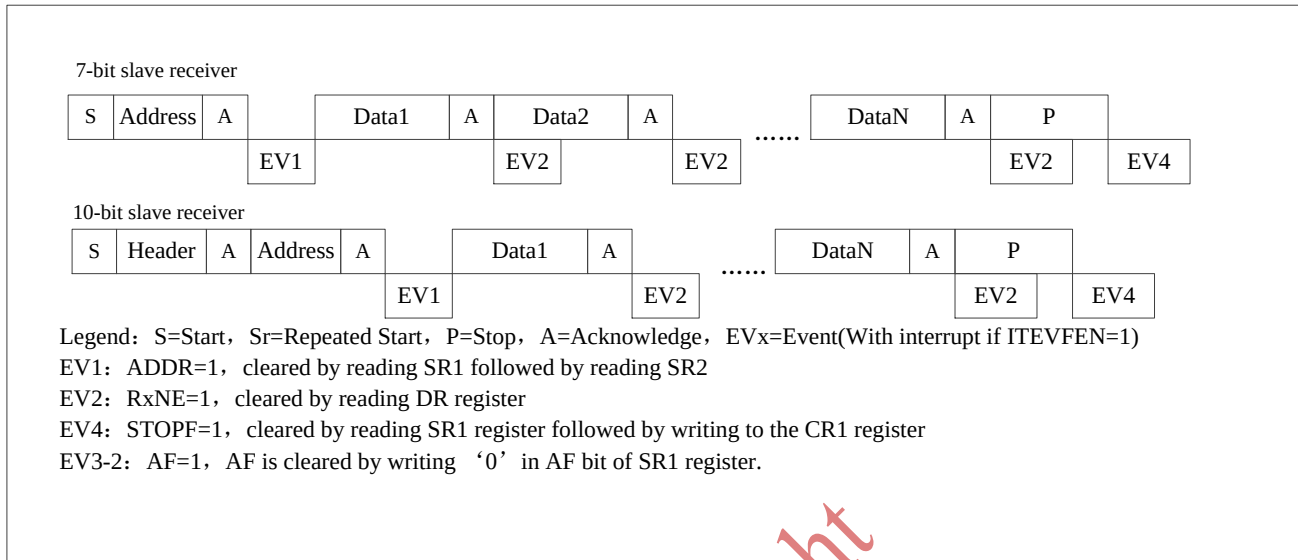


Figure 167 Transfer sequence diagram for slave receiver

Closing slave communication

After the last data byte is transferred a Stop Condition is generated by the master. The interface detects this condition and sets:

- The STOPF bit and generates an interrupt if the ITEVFEN bit is set.

The STOPF bit is cleared by a read of the SR1 register followed by a write to the CR1 register (see EV4 in Figure 167).

19.3.3 I²C master mode

In Master mode, the I²C interface initiates a data transfer and generates the clock signal. A serial data transfer always begins with a Start condition and ends with a Stop condition. Master mode is selected as soon as the Start condition is generated on the bus with a START bit.

The following is the required sequence in master mode.

- Program the peripheral input clock in I2C_CR2 Register in order to generate correct timings
- Configure the clock control registers
- Configure the rise time register
- Program the I2C_CR1 register to enable the peripheral
- Set the START bit in the I2C_CR1 register to generate a Start condition

The peripheral input clock frequency must be at least:

- 2 MHz in Sm mode
- 4 MHz in Fm mode

Start condition

Setting the START bit causes the interface to generate a Start condition and to switch to Master mode (MSL bit set) when the BUSY bit is cleared.

Note: In master mode, setting the START bit causes the interface to generate a ReStart condition at the end of the current byte transfer.

Once the Start condition is sent:

- The SB bit is set by hardware and an interrupt is generated if the ITEVFEN bit is set.

Then the master waits for a read of the SR1 register followed by a write in the DR register with the Slave address (see Figure 168 and Figure 169 Transfer sequencing EV5).

Slave address transmission

Then the slave address is sent to the SDA line via the internal shift register.

- In 10-bit addressing mode, sending the header sequence causes the following event:
 - The ADD10 bit is set by hardware and an interrupt is generated if the ITEVFEN bit is set.
 Then the master waits for a read of the SR1 register followed by a write in the DR register with the second address byte (see Figure 168 and Figure 169 Transfer sequencing).
- In 7-bit addressing mode, one address byte is sent.
 - As soon as the address byte is sent
 - The ADDR bit is set by hardware and an interrupt is generated if the ITEVFEN bit is set.
 - Then the master waits for a read of the SR1 register followed by a read of the SR2 register (see Figure 168 and Figure 169 Transfer sequencing).

The master can decide to enter Transmitter or Receiver mode depending on the LSB of the slave address sent.

- In 7-bit addressing mode
 - To enter Transmitter mode, a master sends the slave address with LSB reset.
 - To enter Receiver mode, a master sends the slave address with LSB set.
- In 10-bit addressing mode
 - To enter Transmitter mode, a master sends the header (11110xx0) and then the slave address, (where xx denotes the two most significant bits of the address).
 - To enter Receiver mode, a master sends the header (11110xx0) and then the slave address. Then it should send a repeated Start condition followed by the header (11110xx1), (where xx denotes the two most significant bits of the address).

The TRA bit indicates whether the master is in Receiver or Transmitter mode.

Master transmitter

Following the address transmission and after clearing ADDR, the master sends bytes from the DR register to the SDA line via the internal shift register.

The master waits until the first data byte is written into I2C_DR (see Figure 168 Transfer sequencing EV8).

When the acknowledge pulse is received

- the TxE bit is set by hardware and an interrupt is generated if the ITEVFEN and ITBUFEN bits are set.

If TxE is set and a data byte was not written in the DR register before the end of the last data transmission, BTF is set and the interface waits until BTF is cleared by a read from I2C_SR1 followed by a write to I2C_DR, stretching SCL low.

Closing the communication

After the last byte is written to the DR register, the STOP bit is set by software to generate a Stop condition (see Figure 168 Transfer sequencing EV8_2). The interface automatically goes back to slave mode (MSL bit cleared).

Note: Stop condition should be programmed during EV8_2 event, when either TxE or BTF is set.

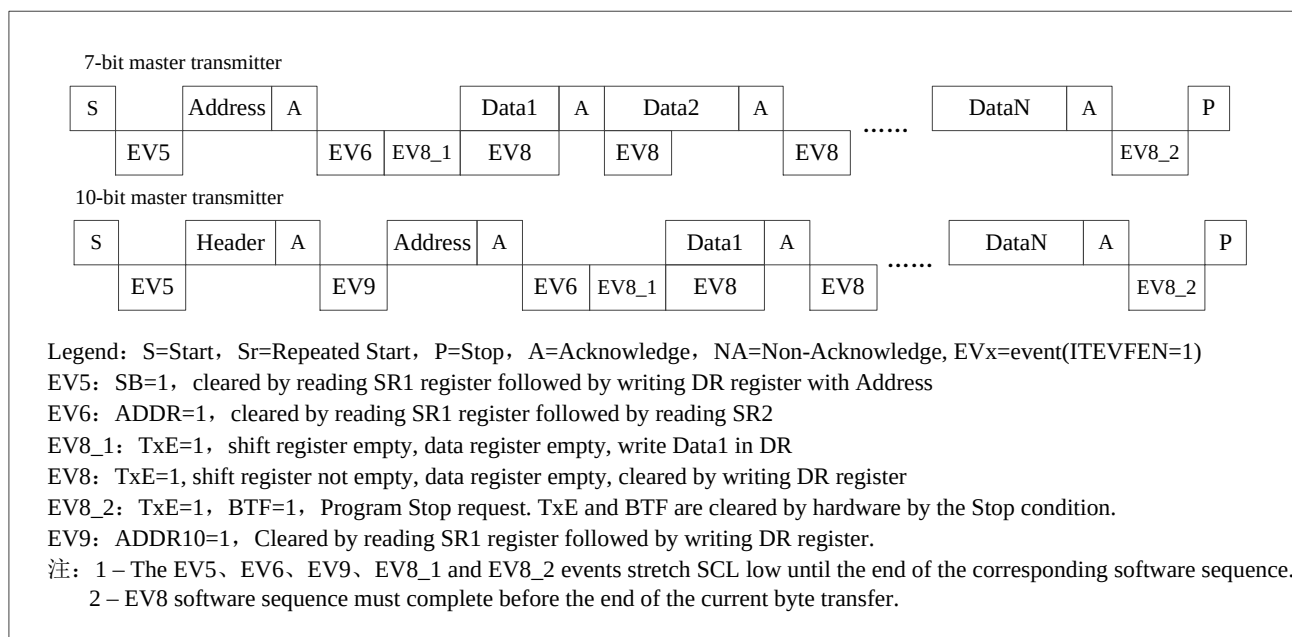


Figure 168 Transfer sequence diagram for master transmitter

Master receiver

Following the address transmission and after clearing ADDR, the I²C interface enters Master Receiver mode. In this mode the interface receives bytes from the SDA line into the DR register via the internal shift register. After each byte the interface generates in sequence:

- The RxNE bit is set and an interrupt is generated if the ITEVFEN and ITBUFEN bits are set (see Figure 169 Transfer sequencing EV7).

If the RxNE bit is set and the data in the DR register is not read before the end of the last data reception, the BTF bit is set by hardware and the interface waits until BTF is cleared by a read in the SR1 register followed by a read in the DR register, stretching SCL low.

Closing the communication

The master sends a NACK for the last byte received from the slave. After receiving this NACK, the slave releases the control of the SCL and SDA lines. Then the master can send a Stop/Restart condition.

- To generate the nonacknowledge pulse after the last received data byte, the ACK bit must be cleared just after reading the second last data byte (after second last RxNE event).
- To generate the Stop/Restart condition, software must set the STOP/START bit just after reading the second last data byte (after the second last RxNE event).
- In case a single byte has to be received, the Acknowledge disable and the Stop condition generation are made just after EV6 (in EV6_1, just after ADDR is cleared).

After the Stop condition generation, the interface goes automatically back to slave mode (MSL bit cleared).

An overrun error can occur in slave mode when clock stretching is disabled and the I²C interface is receiving data. The interface has received a byte (RxNE=1) and the data in DR has not been read, before the next byte is received by the interface. In this case,

- The last received byte is lost
- In case of Overrun error, software should clear the RxNE bit and the transmitter should re-transmit the last received byte.

Underrun error can occur in slave mode when clock stretching is disabled and the I²C interface is transmitting data. The interface has not updated the DR with the next byte (TxNE=1), before the clock comes for the next byte. In this case

- The same byte in the DR register will be sent again
- The user should make sure that data received on the receiver side during an underrun error are discarded and that the next bytes are written within the clock low time specified in the I²C bus standard.

For the first byte to be transmitted, the DR must be written after ADDR is cleared and before the first SCL rising edge. If not possible, the receiver must discard the first data.

19.3.5 SDA/SCL line control

- If clock stretching is enabled:
 - Transmitter mode: If TxNE=1 and BTF=1: the interface holds the clock line low before transmission to wait for the microcontroller to read SR1 and then write the byte in the Data Register (both buffer and shift register are empty).
 - Receiver mode: If RxNE=1 and BTF=1: the interface holds the clock line low after reception to wait for the microcontroller to read SR1 and then read the byte in the Data Register (both buffer and shift register are full).
- If clock stretching is disabled in Slave mode:
 - Overrun Error in case of RxNE=1 and no read of DR has been done before the next byte is received. The last received byte is lost.
 - Underrun Error in case TxNE=1 and no write into DR has been done before the next byte must be transmitted. The same byte will be sent again.
 - Write Collision not managed.

19.3.6 SMBus

Introduction

The System Management Bus (SMBus) is a two-wire interface through which various devices can communicate with each other and with the rest of the system. It is based on I²C principles of operation. SMBus provides a control bus for system and power management related tasks. A system may use SMBus to pass messages to and from devices instead of toggling individual control lines.

The System Management Bus Specification refers to three types of devices. A *slave* is a device that is receiving or responding to a command. A *master* is a device that issues commands, generates the clocks, and terminates the transfer. A *host* is a specialized master that provides the main interface to the system's CPU. A host must be a master-slave and must support the SMBus host notify protocol. Only one host is allowed in a system.

Similarities between SMBus and I²C

- 2 wire bus protocol (1 Clk, 1 Data) + SMBus Alert line optional
- Master-slave communication, Master provides clock
- Multi master capability

- SMBus data format similar to I²C 7-bit addressing format (*Figure 164*).

Differences between SMBus and I²C

The following table describes the differences between SMBus and I²C.

Table 79 SMBus vs I²C

SMBus	I ² C
Max. speed 100 kHz	Max. speed 400 kHz
Min. clock speed 10 kHz	No minimum clock speed
35 ms clock low timeout	No timeout
Logic levels are fixed	Logic levels are V _{DD} dependent
Different address types (reserved, dynamic etc.)	7-bit, 10-bit and general call slave address types
Different bus protocols (quick command, process call etc.)	No bus protocols

SMBus application usage

With System Management Bus, a device can provide manufacturer information, tell the system what its model/part number is, save its state for a suspend event, report different types of errors, accept control parameters, and return its status. SMBus provides a control bus for system and power management related tasks.

Device identification

Any device that exists on the System Management Bus as a slave has a unique address called the Slave Address. For the list of reserved slave addresses, refer to the SMBus specification version. 2.0 (<http://smbus.org/>).

Bus protocols

The SMBus specification supports up to nine bus protocols. For more details of these protocols and SMBus address types, refer to SMBus specification version. 2.0. These protocols should be implemented by the user software.

Address resolution protocol (ARP)

SMBus slave address conflicts can be resolved by dynamically assigning a new unique address to each slave device. The Address Resolution Protocol (ARP) has the following attributes:

- Address assignment uses the standard SMBus physical layer arbitration mechanism
- Assigned addresses remain constant while device power is applied; address retention through device power loss is also allowed
- No additional SMBus packet overhead is incurred after address assignment. (i.e. subsequent accesses to assigned slave addresses have the same overhead as accesses to fixed address devices.)
- Any SMBus master can enumerate the bus

Unique device identifier (UDID)

In order to provide a mechanism to isolate each device for the purpose of address assignment, each device must implement a unique device identifier (UDID).

For the details on 128-bit UDID and more information on ARP, refer to SMBus specification version 2.0.

SMBus alert mode

SMBus Alert is an optional signal with an interrupt line for devices that want to trade their ability to master for a pin. SMBA is a wired-AND signal just as the SCL and SDA signals are. SMBA is used in conjunction with the SMBus General Call Address. Messages invoked with the SMBus are two bytes long.

A slave-only device can signal the host through SMBA that it wants to talk by setting ALERT bit in I2C_CR1 register. The host processes the interrupt and simultaneously accesses all SMBA devices through the *Alert Response Address* (known as ARA having a value 0001 100X). Only the device(s)

which pulled SMBA low will acknowledge the Alert Response Address. This status is identified using SMBALERT Status flag in I2C_SR1 register. The host performs a modified Receive Byte operation. The 7 bit device address provided by the slave transmit device is placed in the 7 most significant bits of the byte. The eighth bit can be a zero or one.

If more than one device pulls SMBA low, the highest priority (lowest address) device will win communication rights via standard arbitration during the slave address transfer. After acknowledging the slave address the device must disengage its SMBA pull-down. If the host still sees SMBA low when the message transfer is complete, it knows to read the ARA again.

A host which does not implement the SMBA signal may periodically access the ARA.

For more details on SMBus Alert mode, refer to SMBus specification version 2.0.

Timeout error

There are differences in the timing specifications between I²C and SMBus.

SMBus defines a clock low timeout, TIMEOUT of 35 ms. Also SMBus specifies TLOW: SEXT as the cumulative clock low extend time for a slave device. SMBus specifies TLOW: MEXT as the cumulative clock low extend time for a master device. For more details on these timeouts, refer to SMBus specification version 2.0.

The status flag Timeout or Tlow Error in I2C_SR1 shows the status of this feature.

How to use the interface in SMBus mode

To switch from I²C mode to SMBus mode, the following sequence should be performed.

- Set the SMBus bit in the I2C_CR1 register
- Configure the SMBTYPE and ENARP bits in the I2C_CR1 register as required for the application

If you want to configure the device as a master, follow the Start condition generation procedure in *Section 19.3.3: I2C master mode*. Otherwise, follow the sequence in *Section 19.3.2: I2C slave mode*.

The application has to control the various SMBus protocols by software.

- SMB Device Default Address acknowledged if ENARP=1 and SMBTYPE=0
- SMB Host Header acknowledged if ENARP=1 and SMBTYPE=1
- SMB Alert Response Address acknowledged if SMBALERT=1

19.3.7 DMA requests

DMA requests (when enabled) are generated only for data transfer. DMA requests are generated by Data Register becoming empty in transmission and Data Register becoming full in reception. The DMA must be initialized and enabled before the I2C data transfer. When the number of data transfers which has been programmed for the corresponding DMA stream is reached, the DMA controller sends an End of Transfer EOT signal to the I²C interface and generates a Transfer Complete interrupt if enabled:

- Master transmitter: In the interrupt routine after the EOT interrupt, disable DMA requests then wait for a BTF event before programming the Stop condition.
- Master receiver: when the number of bytes to be received is equal to or greater than two, the DMA controller sends a hardware signal, EOT_1, corresponding to the last but one data byte (number_of_bytes – 1). If, in the I2C_CR2 register, the LAST bit is set, I²C automatically sends a NACK after the next byte following EOT_1. The user can generate a Stop condition in the DMA Transfer Complete interrupt routine if enabled.

Transmission using DMA

DMA mode can be enabled for transmission by setting the DMAEN bit in the I2C_CR2 register. Data will be loaded from a Memory area configured using the DMA peripheral (refer to the DMA specification) to the I2C_DR register whenever the TxE bit is set. To map a DMA stream x for I²C transmission (where x is the stream number), perform the following sequence:

1. Set the I2C_DR register address in the DMA_SxPAR register. The data will be moved to this address from the memory after each TxE event.

2. Set the memory address in the DMA_SxMA0R register (and in DMA_SxMA1R register in the case of a double buffer mode). The data will be loaded into I2C_DR from this memory after each TxEvent.
3. Configure the total number of bytes to be transferred in the DMA_SxNDTR register. After each TxEvent, this value will be decremented.
4. Configure the DMA stream priority using the PL[0:1] bits in the DMA_SxCR register
5. Set the DIR bit in the DMA_SxCR register and configure interrupts after half transfer or full transfer depending on application requirements.
6. Activate the stream by setting the EN bit in the DMA_SxCR register.

When the number of data transfers which has been programmed in the DMA Controller registers is reached, the DMA controller sends an End of Transfer EOT/ EOT_1 signal to the I²C interface and the DMA generates an interrupt, if enabled, on the DMA stream interrupt vector.

Note: Do not enable the ITBUFEN bit in the I2C_CR2 register if DMA is used for transmission.

Reception using DMA

DMA mode can be enabled for reception by setting the DMAEN bit in the I2C_CR2 register. Data will be loaded from the I2C_DR register to a Memory area configured using the DMA peripheral (refer to the DMA specification) whenever a data byte is received. To map a DMA stream x for I²C reception (where x is the stream number), perform the following sequence:

1. Set the I2C_DR register address in DMA_SxPAR register. The data will be moved from this address to the memory after each RxNE event.
2. Set the memory address in the DMA_SxMA0R register (and in DMA_SxMA1R register in the case of a double buffer mode). The data will be loaded from the I2C_DR register to this memory area after each RxNE event.
3. Configure the total number of bytes to be transferred in the DMA_SxNDTR register. After each RxNE event, this value will be decremented.
4. Configure the stream priority using the PL[0:1] bits in the DMA_SxCR register
5. Reset the DIR bit and configure interrupts in the DMA_SxCR register after half transfer or full transfer depending on application requirements.
6. Activate the stream by setting the EN bit in the DMA_SxCR register.

When the number of data transfers which has been programmed in the DMA Controller registers is reached, the DMA controller sends an End of Transfer EOT/ EOT_1 signal to the I²C interface and DMA generates an interrupt, if enabled, on the DMA stream interrupt vector.

Note: Do not enable the ITBUFEN bit in the I2C_CR2 register if DMA is used for reception.

19.3.8 Packet error checking (PEC)

A PEC calculator has been implemented to improve the reliability of communication. The PEC is calculated by using the $C(x) = x^8 + x^2 + x + 1$ CRC-8 polynomial serially on each bit.

- PEC calculation is enabled by setting the ENPEC bit in the I2C_CR1 register. PEC is a CRC-8 calculated on all message bytes including addresses and R/W bits.
 - In transmission: set the PEC transfer bit in the I2C_CR1 register after the TxEvent corresponding to the last byte. The PEC will be transferred after the last transmitted byte.
 - In reception: set the PEC bit in the I2C_CR1 register after the RxNE event corresponding to the last byte so that the receiver sends a NACK if the next received byte is not equal to the internally calculated PEC. In case of Master- Receiver, a NACK must follow the PEC whatever the check result. The PEC must be set before the ACK pulse of the current byte reception.
- A PECERR error flag/interrupt is also available in the I2C_SR1 register.
- If DMA and PEC calculation are both enabled:
 - In transmission: when the I²C interface receives an EOT signal from the DMA controller, it automatically sends a PEC after the last byte.
 - In reception: when the I²C interface receives an EOT_1 signal from the DMA controller, it will automatically consider the next byte as a PEC and will check it. A DMA request is generated after PEC reception.

- To allow intermediate PEC transfers, a control bit is available in the I2C_CR2 register (LAST bit) to determine if it is really the last DMA transfer or not. If it is the last DMA request for a master receiver, a NACK is automatically sent after the last received byte.
- PEC calculation is corrupted by an arbitration loss.

19.4 I²C interrupts

The table below gives the list of I²C interrupt requests.

Table 80 I²C Interrupt requests

Interrupt event	Event flag	Enable control bit
Start bit sent (Master)	SB	ITEVFEN
Address sent (Master) or Address matched (Slave)	ADDR	
10-bit header sent (Master)	ADD10	
Stop received (Slave)	STOPF	
Data byte transfer finished	BTF	
Receive buffer not empty	RxNE	ITEVFEN and ITBUFEN
Transmit buffer empty	TxE	
Bus error	BERR	ITERREN
Arbitration loss (Master)	ARLO	
Acknowledge failure	AF	
Overrun/Underrun	OVR	
PEC error	PECERR	
Timeout/Tlow error	TIMEOUT	
SMBus Alert	SMBALERT	

Note: SB, ADDR, ADD10, STOPF, BTF, RxNE and TxE are logically OR-ed on the same interrupt channel
BERR, ARLO, AF, OVR, PECERR, TIMEOUT and SMBALERT are logically OR-ed on the same interrupt channel.

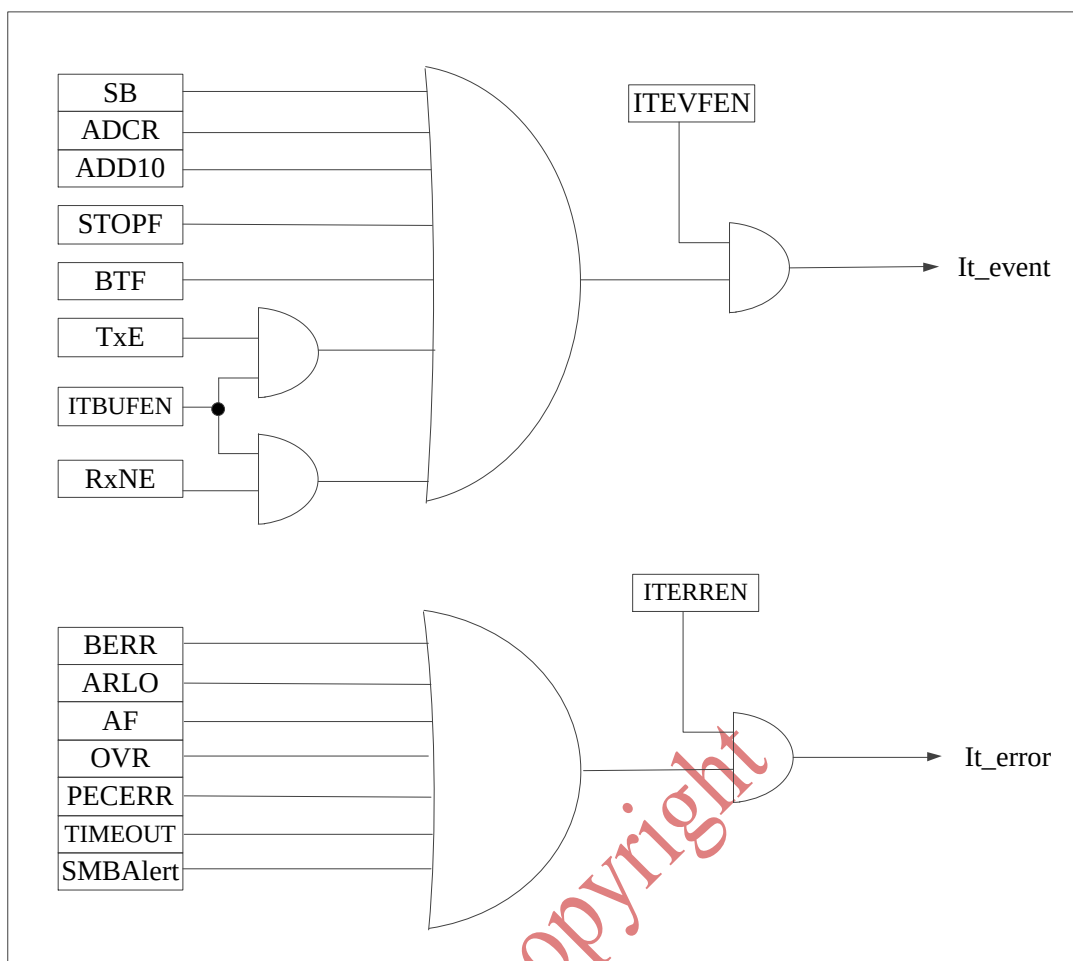


Figure 170 I²C interrupt mapping diagram

19.5 I²C debug mode

When the microcontroller enters the debug mode (Cortex[®]-M3 core halted), the SMBUS timeout either continues to work normally or stops, depending on the DBG_I2Cx_SMBUS_TIMEOUT configuration bits in the DBG module. For more details, refer to *Section 22.16.2: Debug support for timers, watchdog, bxCAN and I2C*.

19.6 I²C registers

Refer to for a list of abbreviations used in register descriptions.

The peripheral registers have to be accessed by half-words (16 bits) or words (32 bits).

19.6.1 I²C Control register 1 (I2C_CR1)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWRST	Res	ALERT	PEC	POS	ACK	STOP	START	NOSTRETCH	ENG	ENPEC	ENARP	SMBTYPE	Res	SMBUS	PE

rw	res	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	res	rw	rw
	Bit 15	SWRST: Software reset When set, the I2C is under reset state. Before resetting this bit, make sure the I2C lines are released and the bus is free. 0: I ² C Peripheral not under reset 1: I ² C Peripheral under reset state <i>Note: This bit can be used to reinitialize the peripheral after an error or a locked state. As an example, if the BUSY bit is set and remains locked due to a glitch on the bus, the SWRST bit can be used to exit from this state.</i>													
	Bit 14	Reserved, must be kept at reset value													
	Bit 13	ALERT: SMBus alert This bit is set and cleared by software, and cleared by hardware when PE=0. 0: Releases SMBA pin high. Alert Response Address Header followed by NACK 1: Drives SMBA pin low. Alert Response Address Header followed by ACK.													
	Bit 12	PEC: Packet error checking This bit is set and cleared by software, and cleared by hardware when PEC is transferred or by a START or Stop condition or when PE=0. 0: No PEC transfer 1: PEC transfer (in Tx or Rx mode) <i>Note: PEC calculation is corrupted by an arbitration loss.</i>													
	Bit 11	POS: Acknowledge/PEC Position (for data reception) This bit is set and cleared by software and cleared by hardware when PE=0. 0: ACK bit controls the (N)ACK of the current byte being received in the shift register. The PEC bit indicates that current byte in shift register is a PEC. 1: ACK bit controls the (N)ACK of the next byte which will be received in the shift register. The PEC bit indicates that the next byte in the shift register is a PEC <i>Note: The POS bit is used when the procedure for reception of 2 bytes (see Method 2: transfer sequence diagram for master receiver when N=2) is followed. It must be configured before data reception starts. In this case, to NACK the 2nd byte, the ACK bit must be cleared just after ADDR is cleared. To check the 2nd byte as PEC, the PEC bit must be set during the ADDR stretch event after configuring the POS bit.</i>													
	Bit 10	ACK: Acknowledge enable This bit is set and cleared by software and cleared by hardware when PE=0. 0: No acknowledge returned 1: Acknowledge returned after a byte is received (matched address or data)													
	Bit 9	STOP: Stop generation The bit is set and cleared by software, cleared by hardware when a Stop condition is detected, set by hardware when a timeout error is detected. In Master Mode: 0: No Stop generation. 1: Stop generation after the current byte transfer or after the current Start condition is sent. In Slave mode: 0: No Stop generation. 1: Release the SCL and SDA lines after the current byte transfer.													
	Bit 8	START: Start generation This bit is set and cleared by software and cleared by hardware when start is sent or PE=0. In Master Mode: 0: No Start generation 1: Repeated start generation In Slave mode: 0: No Start generation 1: Start generation when the bus is free													
	Bit 7	NOSTRETCH: Clock stretching disable (Slave mode) This bit is used to disable clock stretching in slave mode when ADDR or BTF flag is set, until it is reset by software. 0: Clock stretching enabled 1: Clock stretching disabled													
	Bit 6	ENGCG: General call enable 0: General call disabled. Address 00h is NACKed 1: General call enabled. Address 00h is ACKed													
	Bit 5	ENPEC: PEC enable 0: PEC calculation disabled 1: PEC calculation enabled													
	Bit 4	ENARP: ARP enable 0: ARP disable 1: ARP enable													

	SMBus Device default address recognized if SMBTYPE=0 SMBus Host address recognized if SMBTYPE=1
Bit 3	SMBTYPE : SMBus type 0: SMBus Device 1: SMBus Host
Bit 2	Reserved, must be kept at reset value
Bit 1	SMBUS : SMBus mode 0: I ² C mode 1: SMBus mode
Bit 0	PE : Peripheral enable 0: Peripheral disable 1: Peripheral enable <i>Note: If this bit is reset while a communication is on going, the peripheral is disabled at the end of the current communication, when back to IDLE state. All bit resets due to PE=0 occur at the end of the communication. In master mode, this bit must not be reset before the end of the communication.</i>

19.6.2 I²C Control register 2 (I2C_CR2)

Address offset: 0x04

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			LAST	DMAEN	ITBUF EN	ITEVT EN	ITERR EN	Reserved		FREQ[5:0]					
rw			rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw
Bits 15: 13			Reserved, must be kept at reset value												
Bit12			LAST: DMA last transfer 0: Next DMA EOT is not the last transfer 1: Next DMA EOT is the last transfer <i>Note: This bit is used in master receiver mode to permit the generation of a NACK on the last received data.</i>												
Bit11			DMAEN: DMA requests enable 0: DMA requests disabled 1: DMA request enabled when TxE=1 or RxNE =1												
Bit10			ITBUFEN: Buffer interrupt enable 0: TxE = 1 or RxNE = 1 does not generate any interrupt. 1: TxE = 1 or RxNE = 1 generates Event Interrupt (whatever the state of DMAEN)												
Bit 9			ITEVTEN: Event interrupt enable 0: Event interrupt disabled 1: Event interrupt enabled This interrupt is generated when: – SB = 1 (Master); – ADDR = 1 (Master/Slave); – ADD10= 1 (Master); – STOPF = 1 (Slave); – BTF = 1 with no TxE or RxNE event –TxE event to 1 if ITBUFEN = 1 –RxNE event to 1if ITBUFEN = 1												
Bit 8			ITERREN: Error interrupt enable 0: Error interrupt disabled 1: Error interrupt enabled This interrupt is generated when: – BERR = 1; – ARLO = 1; – AF = 1; – OVR = 1; – PECERR = 1; – TIMEOUT = 1; – SMBAlert = 1.												
Bits 7:6			Reserved, must be kept at reset value												
Bits 5:0			FREQ[5:0]: Peripheral clock frequency												

	The FREQ bits must be configured with the APB clock frequency value, The minimum allowed frequency is 2 MHz, the maximum frequency is 36 MHz. 000000: Not allowed 000001: Not allowed 000010: 2MHz ... 100100: 36MHz Higher than 100100: Not allowed
--	--

19.6.3 I²C Own address register 1 (I2C_OAR1)

Address offset: 0x08

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD MODE	Reserved					ADD[9: 8]		ADD[7: 1]							ADD0
rw	rw					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit 15	ADDMODE Addressing mode (slave mode) 0: 7-bit slave address (10-bit address not acknowledged) 1: 10-bit slave address (7-bit address not acknowledged)														
Bit 14	Should always be kept at 1 by software														
Bits 13:10	Reserved, must be kept at reset value														
Bits 9:8	ADD[9:8] : Interface address 7-bit addressing mode: don't care 10-bit addressing mode: bits9:8 of address														
Bits 7:1	ADD[7:1] : Interface address bits 7:1 of address														
Bit 0	ADD0 : Interface address 7-bit addressing mode: don't care 10-bit addressing mode: bit 0 of address														

19.6.4 I²C Own address register 2 (I2C_OAR2)

Address offset: 0x0C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								ADD2[7:1]							ENDUAL
rw								rw	rw	rw	rw	rw	rw	rw	rw
Bits 15:8	Reserved, must be kept at reset value														
Bits 7:1	ADD2[7:1] : Interface address bits 7:1 of address in dual addressing mode														
Bit 0	ENDUAL : Dual addressing mode enable 0: Only OAR1 is recognized in 7-bit addressing mode 1: Both OAR1 and OAR2 are recognized in 7-bit addressing mode														

19.6.5 I²C Data register (I2C_DR)

Address offset : 0x10

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								DR[7:0]							
rw								rw	rw	rw	rw	rw	rw	rw	rw
Bits 15:8	Reserved, must be kept at reset value														
Bits 7:0	DR[7:0] 8-bit data register Byte received or to be transmitted to the bus. – Transmitter mode: Byte transmission starts automatically when a byte is written in the DR														

	register. A continuous transmit stream can be maintained if the next data to be transmitted is put in DR once the transmission is started (TxE=1) Receiver mode: Received byte is copied into DR (RxNE=1). A continuous transmit stream can be maintained if DR is read before the next data byte is received (RxNE=1). <i>Note: In slave mode, the address is not copied into DR.</i> <i>Write collision is not managed (DR can be written if TxE=0).</i> <i>If an ARLO event occurs on ACK pulse, the received byte is not copied into DR and so cannot be read.</i>
--	--

19.6.6 I²C Status register 1 (I2C_SR1)

Address offset: 0x14

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMB ALERT	TIME OUT	Res	PEC ERR	OVR	AF	ARLO	BERR	Tx E	Rx NE	Res	STO PF	ADD 10	BTF	ADDR	SB
rc w0	rc w0	res	rc w0	rc w0	rc w0	rc w0	rc w0	r	r	res	r	r	r	r	r
Bit 15		SMBALERT: SMBus alert In SMBus host mode: 0: no SMBALERT 1: SMBALERT event occurred on pin In SMBus slave mode: 0: no SMBALERT response address header 1: SMBALERT response address header to SMBALERT LOW received –Cleared by software writing 0, or by hardware when PE=0.													
Bit 14		TIMEOUT: Timeout or Tlow error 0: No timeout error 1: SCL remained LOW for 25 ms (Timeout) or Master cumulative clock low extend time more than 10 ms (Tlow:mext) or Slave cumulative clock low extend time more than 25 ms (Tlow:sext) –When set in slave mode: slave resets the communication and lines are released by hardware –When set in master mode: Stop condition sent by hardware –Cleared by software writing 0, or by hardware when PE=0.													
Bit 13		Reserved, must be kept at reset value													
Bit 12		PECERR: PEC Error in reception 0: no PEC error: receiver returns ACK after PEC reception (if ACK=1) 1: PEC error: receiver returns NACK after PEC reception (whatever ACK) <i>Note:</i> Cleared by software writing 0, or by hardware when PE=0.													
Bit 11		OVR: Overrun/Underrun 0: No overrun/underrun 1: Overrun or underrun – Set by hardware in slave mode when NOSTRETCH=1 and: –In reception when a new byte is received (including ACK pulse) and the DR register has not been read yet. New received byte is lost. –In transmission when a new byte should be sent and the DR register has not been written yet. The same byte is sent twice. –Cleared by software writing 0, or by hardware when PE=0. <i>Note: If the DR write occurs very close to SCL rising edge, the sent data is unspecified and a hold timing error occurs</i>													
Bit 10		AF: Acknowledge failure 0: No acknowledge failure 1: Acknowledge failure –Set by hardware when no acknowledge is returned. –Cleared by software writing 0, or by hardware when PE=0.													
Bit 9		ARLO: Arbitration lost (master mode) 0: No Arbitration Lost detected 1: Arbitration Lost detected Set by hardware when the interface loses the arbitration of the bus to another master –Cleared by software writing 0, or by hardware when PE=0. After an ARLO event the interface switches back automatically to Slave mode (MSL=0). <i>Note: In SMBUS, the arbitration on the data in slave mode occurs only during the data phase, or the acknowledge transmission (not on the address acknowledge).</i>													
Bit 8		BERR: Bus error 0: No misplaced Start or Stop condition 1: Misplaced Start or Stop condition													

	–Set by hardware when the interface detects an SDA rising or falling edge while SCL is high, occurring in a non-valid position during a byte transfer. –Cleared by software writing 0, or by hardware when PE=0.
Bit 7	TxE: Data register empty (transmitters) 0: Data register not empty 1: Data register empty –Set when DR is empty in transmission. TxE is not set during address phase. –Cleared by software writing to the DR register or by hardware after a start or a stop condition or when PE=0. TxE is not set if either a NACK is received, or if next byte to be transmitted is PEC (PEC=1) <i>Note: TxE is not cleared by writing the first data being transmitted, or by writing data when BTF is set, as in both cases the data register is still empty.</i>
Bit 6	RxNE: Data register not empty (receivers) 0: Data register empty 1: Data register not empty –Set when data register is not empty in receiver mode. RxNE is not set during address phase. –Cleared by software reading or writing the DR register or by hardware when PE=0. RxNE is not set in case of ARLO event. <i>Note: RxNE is not cleared by reading data when BTF is set, as the data register is still full.</i>
Bit 5	Reserved, must be kept at reset value
Bit 4	STOPF: Stop detection (slave mode) 0: No Stop condition detected 1: Stop condition detected –Set by hardware when a Stop condition is detected on the bus by the slave after an acknowledge (if ACK=1). –Cleared by software reading the SR1 register followed by a write in the CR1 register, or by hardware when PE=0 <i>Note: The STOPF bit is not set after a NACK reception.</i>
Bit 3	ADD10: 10-bit header sent (Master mode) 0: No ADD10 event occurred. 1: Master has sent first address byte (header). –Set by hardware when the master has sent the first byte in 10-bit address mode. –Cleared by software reading the SR1 register followed by a write in the DR register of the second address byte, or by hardware when PE=0. <i>Note: ADD10 bit is not set after a NACK reception</i>
Bit 2	BTF: Byte transfer finished 0: Data byte transfer not done 1: Data byte transfer succeeded Set by hardware when NOSTRETCH=0 and: –In reception when a new byte is received (including ACK pulse) and DR has not been read yet (RxNE=1). –In transmission when a new byte should be sent and DR has not been written yet (TxNE=1). –Cleared by software reading SR1 followed by either a read or write in the DR register or by hardware after a start or a stop condition in transmission or when PE=0. <i>Note: The BTF bit is not set after a NACK reception</i> <i>The BTF bit is not set if next byte to be transmitted is the PEC (TRA=1 in I2C_SR2 register and PEC=1 in I2C_CR1 register)</i>
Bit 1	ADDR: Address sent (master mode)/matched (slave mode) This bit is cleared by software reading SR1 register followed reading SR2, or by hardware when PE=0. Address matched (Slave) 0: Address mismatched or not received 1: Received address matched. –Set by hardware as soon as the received slave address matched with the OAR registers content or a general call or a SMBus Device Default Address or SMBus Host or SMBus Alert is recognized. (when enabled depending on configuration). Address sent (Master) 0: No end of address transmission 1: End of address transmission –For 10-bit addressing, the bit is set after the ACK of the 2nd byte. –For 7-bit addressing, the bit is set after the ACK of the byte. <i>Note: ADDR is not set after a NACK reception</i>
Bit 0	SB: Start bit (Master mode) 0: No Start condition 1: Start condition generated. –Set when a Start condition generated. –Cleared by software by reading the SR1 register followed by writing the DR register, or by hardware

	when PE=0
--	-----------

19.6.7 I²C Status register 2 (I2C_SR2)

Address offset: 0x18

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PEC[7:0]								DUALF	SMB HOST	SMB DEFAULT	GEN CALL	Res	TRA	BUSY	MSL
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 15:8		PEC[7:0] Packet error checking register This register contains the internal PEC when ENPEC=1.													
Bit 7		DUALF : Dual flag (Slave mode) 0: Received address matched with OAR1 1: Received address matched with OAR2 –Cleared by hardware after a Stop condition or repeated Start condition, or when PE=0.													
Bit 6		SMBHOST : SMBus host header (Slave mode) 0: No SMBus Host address 1: SMBus Host address received when SMBTYPE=1 and ENARP=1. –Cleared by hardware after a Stop condition or repeated Start condition, or when PE=0.													
Bit 5		SMBDEFAULT : SMBus device default address (Slave mode) 0: No SMBus Device Default address 1: SMBus Device Default address received when ENARP=1 –Cleared by hardware after a Stop condition or repeated Start condition, or when PE=0.													
Bit 4		GENCALL : General call address (Slave mode) 0: No General Call 1: General Call Address received when ENGC=1 –Cleared by hardware after a Stop condition or repeated Start condition, or when PE=0.													
Bit 3		Reserved, must be kept at reset value													
Bit 2		TRA : Transmitter/receiver 0: Data bytes received 1: Data bytes transmitted This bit is set depending on the R/W bit of the address byte, at the end of total address phase. It is also cleared by hardware after detection of Stop condition (STOPF=1), repeated Start condition, loss of bus arbitration (ARLO=1), or when PE=0.													
Bit 1		BUSY : Bus busy 0: No communication on the bus 1: Communication ongoing on the bus –Set by hardware on detection of SDA or SCL low –cleared by hardware on detection of a Stop condition. It indicates a communication in progress on the bus. This information is still updated when the interface is disabled (PE=0).													
Bit 0		MSL : Master/slave 0: Slave Mode 1: Master Mode –Set by hardware as soon as the interface is in Master mode (SB=1). –Cleared by hardware after detecting a Stop condition on the bus or a loss of arbitration (ARLO=1), or by hardware when PE=0.													

19.6.8 I²C Clock control register (I2C_CCR)

Address offset: 0x1C

Reset value: 0x0000

Note: f_{PCLK1} must be at least 2 MHz to achieve Sm mode I²C frequencies. It must be at least 4 MHz to achieve Fm mode I²C frequencies. It must be a multiple of 10MHz to reach the 400 kHz maximum I²C Fm mode clock.
The CCR register must be configured only when the I2C is disabled (PE = 0).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
F/S	DUTY	Reserved		CCR[11: 0]											
rw	rw			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bit 15		F/S: I2C master mode selection 0: Sm mode I2C 1: Fm mode I2C													
Bit 14		DUTY: Fm mode duty cycle 0: Fm mode $t_{low}/t_{high} = 2$ 1: Fm mode $t_{low}/t_{high} = 16/9$ (see CCR)													
Bits 13:12		Reserved, must be kept at reset value													
Bits 11:0		CCR[11:0]: Clock control register in Fm/Sm mode (Master mode) Controls the SCL clock in master mode. <u>Sm mode or SMBus:</u> $T_{high} = CCR * T_{PCLK1}$ $T_{low} = CCR * T_{PCLK1}$ <u>Fm mode:</u> If DUTY = 0: $T_{high} = CCR * T_{PCLK1}$ $T_{low} = 2 * CCR * T_{PCLK1}$ If DUTY = 1: (to reach 400 kHz) $T_{high} = 9 * CCR * T_{PCLK1}$ $T_{low} = 16 * CCR * T_{PCLK1}$ For instance: in Sm mode, to generate a 100 kHz SCL frequency: If FREQR = 08, $T_{PCLK1} = 125$ ns so CCR must be programmed with 0x28 (0x28 <=> 40d x 125 ns = 5000 ns.) <i>Note: The minimum allowed value is 0x04, except in FAST DUTY mode where the minimum allowed value is 0x01</i> $t_{high} = t_{r(SCL)} + t_{w(SCLH)}$. See device datasheet for the definitions of parameters. $t_{low} = t_{f(SCL)} + t_{w(SCLL)}$. See device datasheet for the definitions of parameters. I2C communication speed, $f_{SCL} \sim 1/(t_{high} + t_{low})$. The real frequency may differ due to the analog noise filter input delay. <i>The CCR register must be configured only when the I²C is disabled (PE = 0).</i>													

19.6.9 I²C TRISE register (I2C_TRISE)

Address offset: 0x20

Reset value: 0x0002

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										TRISE[5:0]					
										rw	rw	rw	rw	rw	rw
Bits 15: 6		Reserved, must be kept at reset value													
Bits 5: 0		TRISE[5:0]: Maximum rise time in Fm/Sm mode (Master mode) These bits should provide the maximum duration of the SCL feedback loop in master mode. The purpose is to keep a stable SCL frequency whatever the SCL rising edge duration. These bits must be programmed with the maximum SCL rise time given in the I ² C bus specification, incremented by 1. For instance: in Sm mode, the maximum allowed SCL rise time is 1000 ns. If, in the I2C_CR2 register, the value of FREQ[5:0] bits is equal to 0x08 and $T_{PCLK1} = 125$ ns therefore the TRISE[5:0] bits must be programmed with 09h. (1000 ns / 125 ns = 8 + 1) The filter value can also be added to TRISE[5:0]. If the result is not an integer, TRISE[5:0] must be programmed with the integer part, in order to respect the t_{HIGH} parameter. <i>Note: TRISE[5:0] must be configured only when the I2C is disabled (PE = 0).</i>													

19.6.10 I²C register map

Table 81 I²C register map and reset values

offset	register		31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0							
0x00	Reset value	I2C_CR1	Reserved																0	SWRST	Reserved	0	ALERT	PEC	POS	ACK	STOP	START	0	NOSTRETCH	0	ENG	ENPEC	ENARP	SMBTYPE	Reserved	0	SMBUS	PE	0	
		I2C_CR2	Reserved																LAST		0	DMAEN	ITBUFEN	ITEVTEN	ITERREN	Reserved	FREQ[5:0]						0	0	0	0	0	0			
0x08	Reset value	I2C_OAR1	Reserved																0	ADDMODE	Reserved						ADD [9:8]		0	0	ADD[7:1]						ADD0		0		
		I2C_OAR2	Reserved																Reserved						ADD2[7:1]						0	0	0	0	0	0	0	0	0		
0x0C	Reset value	I2C_OAR2	Reserved																Reserved						ADD2[7:1]						0	0	0	0	0	0	0	0	0	0	0
		I2C_DR	Reserved																Reserved						DR[7:0]						0	0	0	0	0	0	0	0	0	0	

0x14	I2C_SR1	Reset value	Reserved	0	SMBALERT
		0		TIMEOUT	
		Reserved			
	0	PECERR			
	0	OVR			
	0	AF			
	0	ARLO			
	0	BERR			
	0	TxE			
	0	RxNE			
		Reserved			
	0	STOPF			
	0	ADD10			
	0	BTF			
	0	ADDR			
	0	SB			
0x18	I2C_SR2	Reset value	Reserved	0	PEC[7:0]
		0			
	0				
	0				
	0				
	0				
	0				
	0				
	0	DUALF			
	0	SMBHOST			
	0	SMBDEFAU			
	0	GENCALL			
		Reserved			
	0	TRA			
	0	BUSY			
	0	MSL			
0x1C	I2C_CCR	Reset value	Reserved	0	F/S
		0		DUTY	
		Reserved			
	0	CCR[11:0]			
	0				
	0				
	0				
	0				
	0				
	0				
	0				
	0				
	0				
	0				
	0				
	0				
	0				
0x20	I2C_TRISE	Reset value	Reserved	0	TRISE[5:0]
		0			
	0				
	0				
	0				
	1				
	0				

Refer to Table 1 for the register boundary addresses.

20 Universal synchronous asynchronous receiver transmitter (USART)

20.1 USART introduction

The universal synchronous asynchronous receiver transmitter (USART) offers a flexible means of full-duplex data exchange with external equipment requiring an industry standard NRZ asynchronous serial data format. The USART offers a very wide range of baud rates using a fractional baud rate generator.

It supports synchronous one-way communication and half-duplex single wire communication. It also supports the LIN (local interconnection network), Smartcard Protocol and IrDA (infrared data association) SIR ENDEC specifications, and modem operations (CTS/RTS). It allows multiprocessor communication.

High speed data communication is possible by using the DMA for multibuffer configuration.

20.2 USART main features

- Full duplex, asynchronous communications
- NRZ standard format (Mark/Space)
- Fractional baud rate generator systems
 - A common programmable transmit and receive baud rates up to 4.5 MBits/s
- Programmable data word length (8 or 9 bits)
- Configurable stop bits - support for 1 or 2 stop bits
- LIN Master Synchronous Break send capability and LIN slave break detection capability
 - 13-bit break generation and 10/11 bit break detection when USART is hardware configured for LIN
- Transmitter clock output for synchronous transmission
- IrDA SIR Encoder Decoder
 - Support for 3/16 bit duration for normal mode
- Smartcard Emulation Capability
 - The Smartcard interface supports the asynchronous protocol Smartcards as defined in ISO 7816-3 standards
 - 0.5, 1.5 Stop Bits for Smartcard operation
- Single wire half duplex communication
- Configurable multibuffer communication using DMA (direct memory access)
 - Buffering of received/transmitted bytes in reserved SRAM using centralized DMA
- Separate enable bits for Transmitter and Receiver
- Transfer detection flags:
 - Receive buffer full
 - Transmit buffer empty
 - End of Transmission flags
- Parity control:
 - Transmits parity bit
 - Checks parity of received data byte
- Four error detection flags:
 - Overrun error

- Noise error
- Frame error
- Parity error
- Ten interrupt sources with flags:
 - CTS changes
 - LIN break detection
 - Transmit data register empty
 - Transmission complete
 - Receive data register full
 - Idle line received
 - Overrun error
 - Framing error
 - Noise error
 - Parity error
- Multiprocessor communication - enter into mute mode if address match does not occur
- Wake up from mute mode (by idle line detection or address mark detection)
- Two receiver wakeup modes: Address bit (MSB, 9th bit), Idle line

20.3 USART functional description

The interface is externally connected to another device by three pins (see Figure 171). Any USART bidirectional communication requires a minimum of two pins: Receive Data In (RX) and Transmit Data Out (TX):

RX: Receive Data Input is the serial data input. Oversampling techniques are used for data recovery by discriminating between valid incoming data and noise.

TX: Transmit Data Output. When the transmitter is disabled, the output pin returns to its IO port configuration. When the transmitter is enabled and nothing is to be transmitted, the TX pin is at high level. In single-wire and smartcard modes, this IO is used to transmit and receive the data (at USART level, data are then received on SW_RX).

Through these pins, serial data is transmitted and received in normal USART mode as frames comprising:

- An Idle Line prior to transmission or reception
- A start bit
- A data word (8 or 9 bits) least significant bit first
- 0.5, 1, 1.5, 2 Stop bits indicating that the frame is complete
- This interface uses a fractional baud rate generator - with a 12-bit mantissa and 4-bit fraction
- A status register (USART_SR)
- Data register (USART_DR)
- A baud rate register (USART_BRR) - 12-bit mantissa and 4-bit fraction.
- A Guardtime Register (USART_GTPR) in case of Smartcard mode.

Refer to *Section 20.6: USART registers* for the definition of each bit.

The following pin is required to interface in synchronous mode:

- **CK:** Transmitter clock output. This pin outputs the transmitter data clock for synchronous transmission corresponding to SPI master mode (no clock pulses on start bit and stop bit, and a software option to send a clock pulse on the last data bit). In parallel data can be received synchronously on RX. This can be used to control peripherals that have shift registers (e.g. LCD drivers). The clock phase and polarity are software programmable. In Smartcard mode, CK can provide the clock to the smartcard.

The following pins are required in IrDA mode:

- **IrDA_RDI:** data input in IrDA mode.
- **IrDA_TDO:** data output in IrDA mode

The following pins are required in Hardware flow control mode:

- **nCTS:** Clear To Send blocks the data transmission at the end of the current transfer when high
- **nRTS:** Request to send indicates that the USART is ready to receive a data (when low).

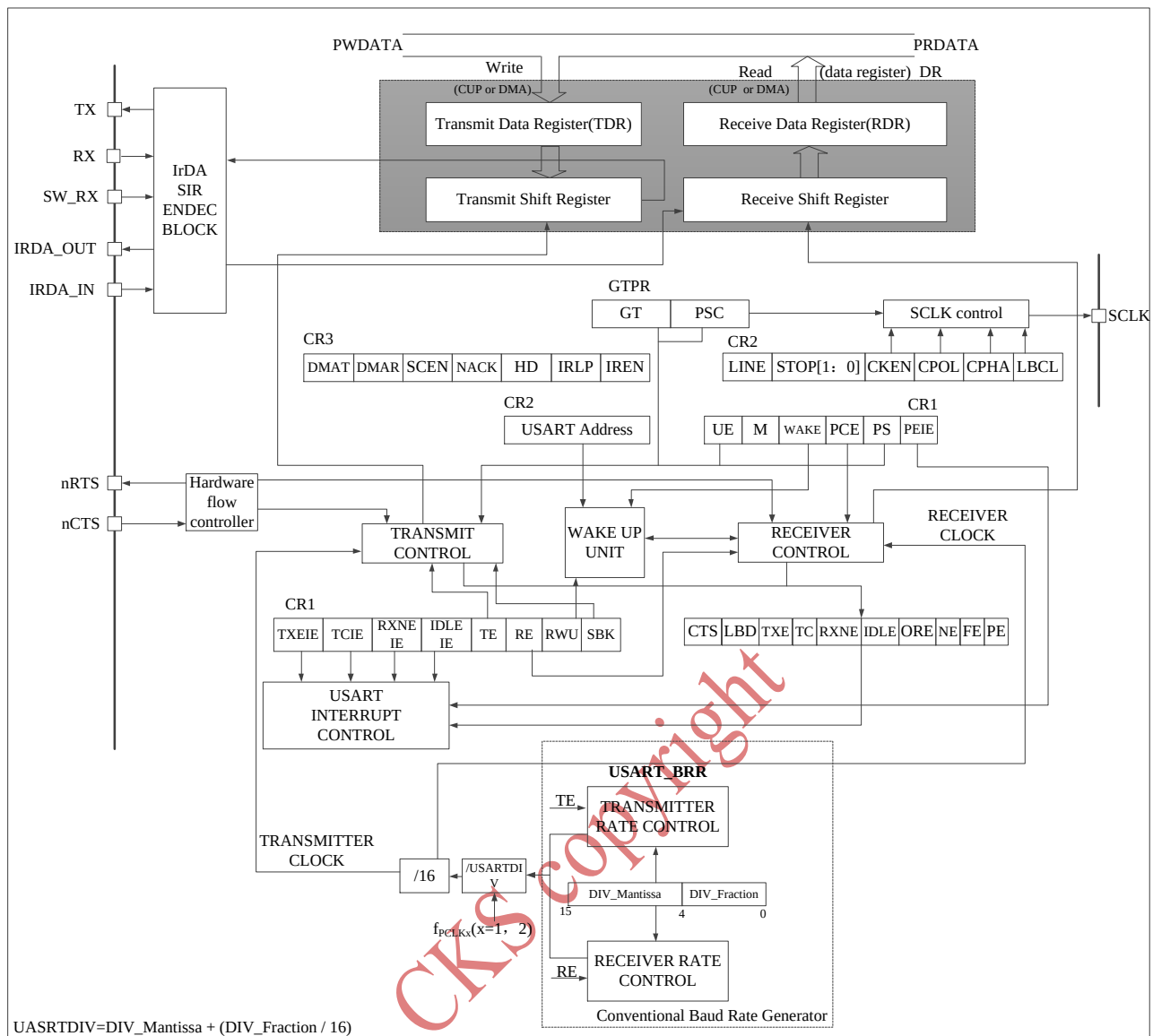


Figure 171 USART block diagram

20.3.1 USART character description

Word length may be selected as being either 8 or 9 bits by programming the M bit in the USART_CR1 register (see Figure 172). The TX pin is in low state during the start bit. It is in high state during the stop bit.

An **Idle character** is interpreted as an entire frame of “1”s followed by the start bit of the next frame which contains data (The number of “1”s will include the number of stop bits).

A **Break character** is interpreted on receiving “0”s for a frame period. At the end of the break frame the transmitter inserts either 1 or 2 stop bits (logic “1” bit) to acknowledge the start bit.

Transmission and reception are driven by a common baud rate generator, the clock for each is generated when the enable bit is set respectively for the transmitter and receiver.

The details of each block is given below.

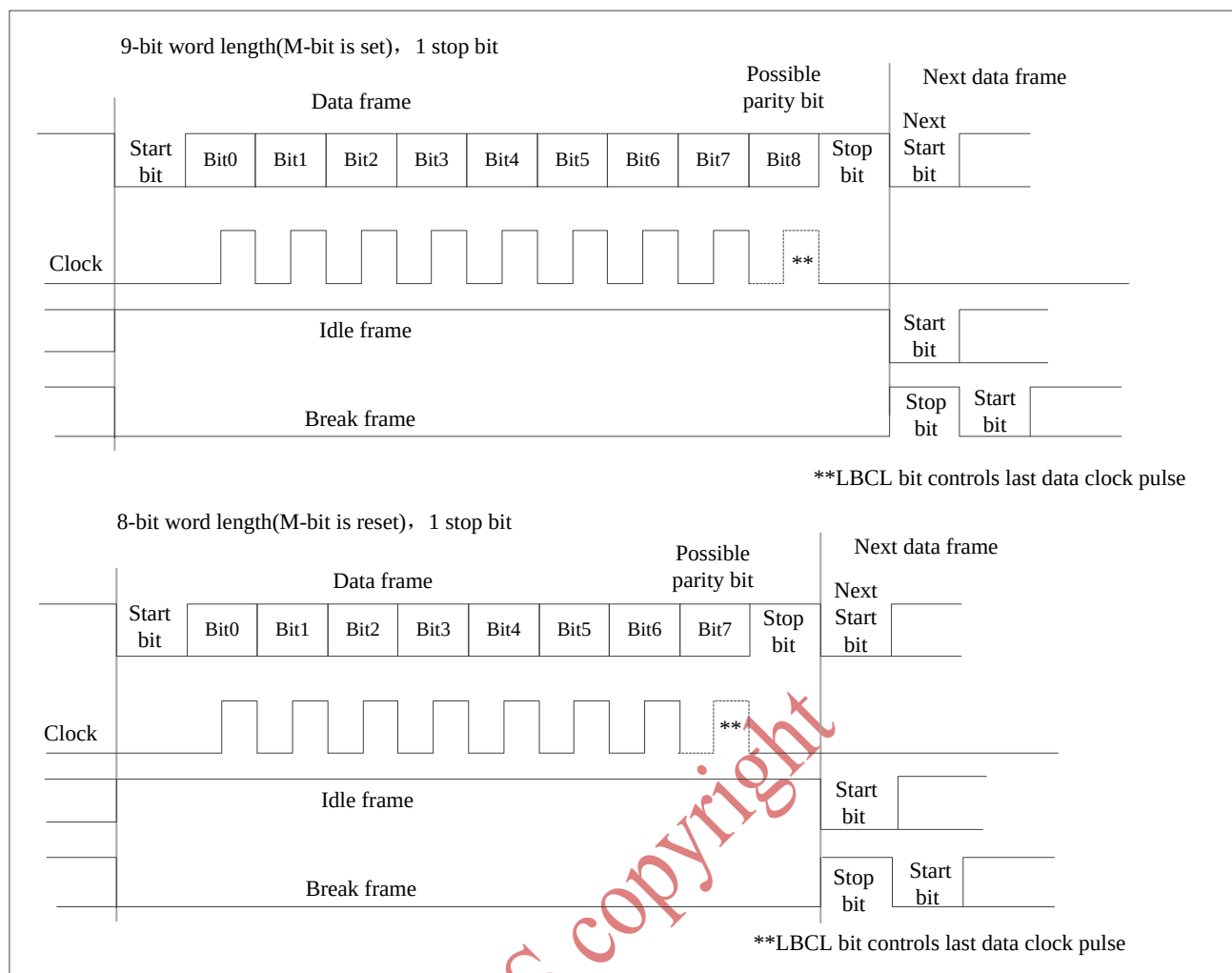


Figure 172 Word length programming

20.3.2 Transmitter

The transmitter can send data words of either 8 or 9 bits depending on the M bit status. When the transmit enable bit (TE) is set, the data in the transmit shift register is output on the TX pin and the corresponding clock pulses are output on the CK pin.

Character transmission

During a USART transmission, data shifts out least significant bit first on the TX pin. In this mode, the USART_DR register consists of a buffer (TDR) between the internal bus and the transmit shift register.

Every character is preceded by a start bit, which is a logic level low for one bit period. The character is terminated by a configurable number of stop bits.

The following stop bits are supported by USART: 0.5, 1, 1.5 and 2 stop bits.

Note: The TE bit should not be reset during transmission of data. Resetting the TE bit during the transmission will corrupt the data on the TX pin as the baud rate counters will get frozen. The current data being transmitted will be lost.

An idle frame will be sent after the TE bit is enabled.

Configurable stop bits

The number of stop bits to be transmitted with every character can be programmed in Control register 2, bits 13,12.

1. **1stop bit:** This is the default value.
2. **2 stop bits:** This is supported by normal USART, single-wire and modem modes.

3. **0.5 stop bit:** To be used when receiving data in Smartcard mode.

4. **1.5 stop bit:** To be used when transmitting and receiving data in Smartcard mode.

An idle frame transmission will include the stop bits.

A break transmission will be 10 low bits followed by the configured number of stop bits (when $m = 0$) and 11 low bits followed by the configured number of stop bits (when $m = 1$). It is not possible to transmit long breaks (break of length greater than 10/11 low bits).

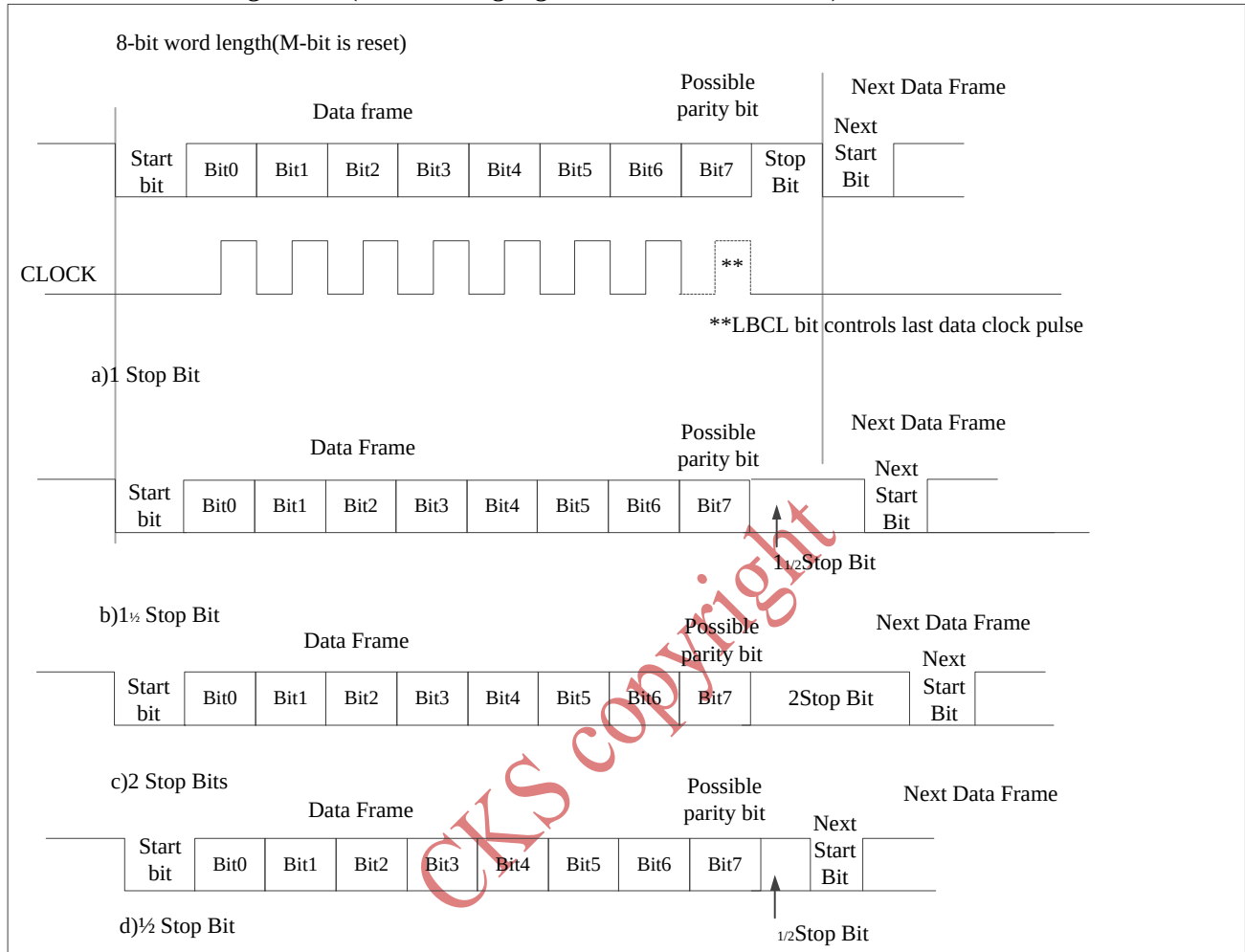


Figure 173 Configurable stop bits

Procedure:

1. Enable the USART by writing the UE bit in USART_CR1 register to 1.
2. Program the M bit in USART_CR1 to define the word length.
3. Program the number of stop bits in USART_CR2.
4. Select DMA enable (DMAT) in USART_CR3 if Multi buffer Communication is to take place. Configure the DMA register as explained in multibuffer communication.
5. Select the desired baud rate using the USART_BRR register.
6. Set the TE bit in USART_CR1 to send an idle frame as first transmission.
7. Write the data to send in the USART_DR register (this clears the TXE bit). Repeat this for each data to be transmitted in case of single buffer.
8. After writing the last data into the USART_DR register, wait until TC=1. This indicates that the transmission of the last frame is complete. This is required for instance when the USART is disabled or enters the Halt mode to avoid corrupting the last transmission.

Single byte communication

The TXE bit is always cleared by a write to the data register. The TXE bit is set by hardware and it indicates:

- The data has been moved from TDR to the shift register and the data transmission has started.

- The TDR register is empty.
- The next data can be written in the USART_DR register without overwriting the previous data. This flag generates an interrupt if the TXEIE bit is set.

When a transmission is taking place, a write instruction to the USART_DR register stores the data in the TDR register and which is copied in the shift register at the end of the current transmission.

When no transmission is taking place, a write instruction to the USART_DR register places the data directly in the shift register, the data transmission starts, and the TXE bit is immediately set.

If a frame is transmitted (after the stop bit) and the TXE bit is set, the TC bit goes high. An interrupt is generated if the TCIE bit is set in the USART_CR1 register.

After writing the last data into the USART_DR register, it is mandatory to wait for TC=1 before disabling the USART or causing the microcontroller to enter the low-power mode (see Figure 174).

The TC bit is cleared by the following software sequence:

1. A read from the USART_SR register
2. A write to the USART_DR register

Note: The TC bit can also be cleared by writing a '0' to it. This clearing sequence is recommended only for Multibuffer communication.

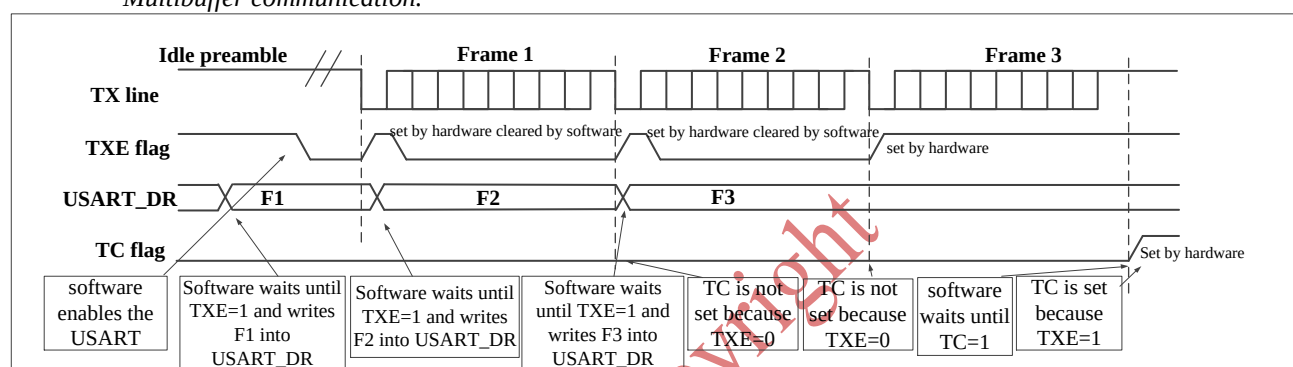


Figure 174 TC/TXE behavior when transmitting

Break characters

Setting the SBK bit transmits a break character. The break frame length depends on the M bit (see Figure 172).

If the SBK bit is set to '1' a break character is sent on the TX line after completing the current character transmission. This bit is reset by hardware when the break character is completed (during the stop bit of the break character). The USART inserts a logic 1 bit at the end of the last break frame to guarantee the recognition of the start bit of the next frame.

Note: If the software resets the SBK bit before the commencement of break transmission, the break character will not be transmitted. For two consecutive breaks, the SBK bit should be set after the stop bit of the previous break.

Idle characters

Setting the TE bit drives the USART to send an idle frame before the first data frame.

20.3.3 Receiver

The USART can receive data words of either 8 or 9 bits depending on the M bit in the USART_CR1 register.

Start bit detection

In the USART, the start bit is detected when a specific sequence of samples is recognized. This sequence is: 1 1 1 0 X 0 X 0 0 0 0.

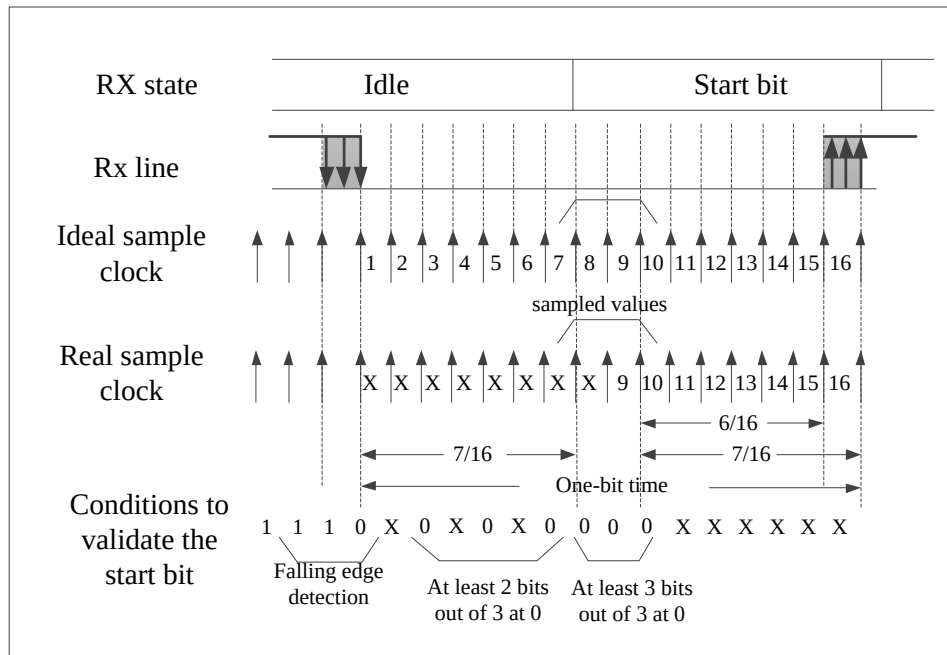


Figure 175 Start bit detection

Note: If the sequence is not complete, the start bit detection aborts and the receiver returns to the idle state (no flag is set) where it waits for a falling edge

The start bit is confirmed (RXNE flag set, interrupt generated if RXNEIE=1) if the 3 sampled bits are at 0 (first sampling on the 3rd, 5th and 7th bits finds the 3 bits at 0 and second sampling on the 8th, 9th and 10th bits also finds the 3 bits at 0).

The start bit is validated (RXNE flag set, interrupt generated if RXNEIE=1) but the NE noise flag is set if, for both samplings, at least 2 out of the 3 sampled bits are at 0 (sampling on the 3rd, 5th and 7th bits and sampling on the 8th, 9th and 10th bits). If this condition is not met, the start detection aborts and the receiver returns to the idle state (no flag is set).

If, for one of the samplings (sampling on the 3rd, 5th and 7th bits or sampling on the 8th, 9th and 10th bits), 2 out of the 3 bits are found at 0, the start bit is validated but the NE noise flag bit is set.

Character reception

During a USART reception, data shifts in least significant bit first through the RX pin. In this mode, the USART_DR register consists of a buffer (RDR) between the internal bus and the received shift register.

Procedure:

1. Enable the USART by writing the UE bit in USART_CR1 register to 1.
2. Program the M bit in USART_CR1 to define the word length.
3. Program the number of stop bits in USART_CR2
4. Select DMA enable (DMAR) in USART_CR3 if multibuffer communication is to take place. Configure the DMA register as explained in multibuffer communication. STEP 3
5. Select the desired baud rate using the baud rate register USART_BRR
6. Set the RE bit USART_CR1. This enables the receiver which begins searching for a start bit.

When a character is received

- The RXNE bit is set. It indicates that the content of the shift register is transferred to the RDR. In other words, data has been received and can be read (as well as its associated error flags).
- An interrupt is generated if the RXNEIE bit is set.
- The error flags can be set if a frame error, noise or an overrun error has been detected during reception.
- In multibuffer, RXNE is set after every byte received and is cleared by the DMA read to the Data Register.
- In single buffer mode, clearing the RXNE bit is performed by a software read to the USART_DR register. The RXNE flag can also be cleared by writing a zero to it. The RXNE bit must be cleared before the end of the reception of the next character to avoid an overrun error.

Note: The RE bit should not be reset while receiving data. If the RE bit is disabled during reception, the reception of the current byte will be aborted.

Break character

When a break character is received, the USART handles it as a framing error.

Idle character

When an idle frame is detected, there is the same procedure as a data received character plus an interrupt if the IDLEIE bit is set.

Overrun error

An overrun error occurs when a character is received when RXNE has not been reset. Data can not be transferred from the shift register to the RDR register until the RXNE bit is cleared. The RXNE flag is set after every byte received. An overrun error occurs if RXNE flag is set when the next data is received or the previous DMA request has not been serviced.

When an overrun error occurs:

- The ORE bit is set.
- The RDR content will not be lost. The previous data is available when a read to USART_DR is performed
- The shift register will be overwritten. After that point, any data received during overrun is lost.
- An interrupt is generated if either the RXNEIE bit is set or both the EIE and DMAR bits are set.
- The ORE bit is reset by a read to the USART_SR register followed by a USART_DR register read operation.

Note: The ORE bit, when set, indicates that at least 1 data has been lost. There are two possibilities:

- if RXNE=1, then the last valid data is stored in the receive register RDR and can be read
- if RXNE=0, then it means that the last valid data has already been read and thus there is nothing to be read in the RDR. This case can occur when the last valid data is read in the RDR at the same time as the new (and lost) data is received. It may also occur when the new data is received during the reading sequence (between the USART_SR register read access and the USART_DR read access).

Noise error

Over-sampling techniques are used (except in synchronous mode) for data recovery by discriminating between valid incoming data and noise.

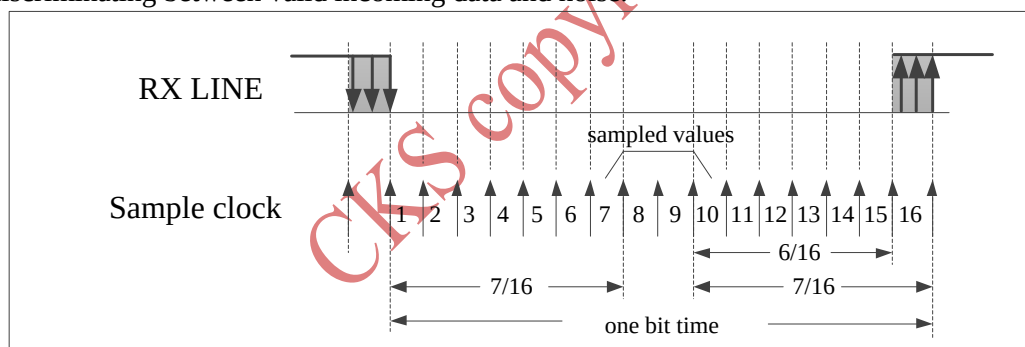


Figure 176 Data sampling for noise detection

Table 82 Noise detection from sampled data

Sampled value	NE status	Received bit value	Data validity
000	0	0	Valid
001	1	0	Not Valid
010	1	0	Not Valid
011	1	1	Not Valid
100	1	0	Not Valid
101	1	1	Not Valid
110	1	1	Not Valid
111	0	1	Valid

When noise is detected in a frame:

- The NE is set at the rising edge of the RXNE bit
- The invalid data is transferred from the Shift register to the USART_DR register.

- No interrupt is generated in case of single byte communication. However this bit rises at the same time as the RXNE bit which itself generates an interrupt. In case of multibuffer communication an interrupt will be issued if the EIE bit is set in the USART_CR3 register.

The NE bit is reset by a USART_SR register read operation followed by a USART_DR register read operation.

Framing error

A framing error is detected when:

The stop bit is not recognized on reception at the expected time, following either a de-synchronization or excessive noise.

When the framing error is detected:

- The FE bit is set by hardware
- The invalid data is transferred from the Shift register to the USART_DR register.
- No interrupt is generated in case of single byte communication. However this bit rises at the same time as the RXNE bit which itself generates an interrupt. In case of multibuffer communication an interrupt will be issued if the EIE bit is set in the USART_CR3 register.

The FE bit is reset by a USART_SR register read operation followed by a USART_DR register read operation.

Configurable stop bits during reception

The number of stop bits to be received can be configured through the control bits of Control Register 2 - it can be either 1 or 2 in normal mode and 0.5 or 1.5 in Smartcard mode.

1. **0.5 stop bit (reception in Smartcard mode):** No sampling is done for 0.5 stop bit. As a consequence, no framing error and no break frame can be detected when 0.5 stop bit is selected.
2. **1 stop bit:** Sampling for 1 stop Bit is done on the 8th, 9th and 10th samples.
3. **1.5 stop bits (Smartcard mode):** When transmitting in Smartcard mode, the device must check that the data is correctly sent. Thus the receiver block must be enabled (RE =1 in the USART_CR1 register) and the stop bit is checked to test if the smartcard has detected a parity error. In the event of a parity error, the smartcard forces the data signal low during the sampling - NACK signal-, which is flagged as a framing error. Then, the FE flag is set with the RXNE at the end of the 1.5 stop bit. Sampling for 1.5 stop bits is done on the 16th, 17th and 18th samples (1 baud clock period after the beginning of the stop bit). The 1.5 stop bit can be decomposed into 2 parts: one 0.5 baud clock period during which nothing happens, followed by 1 normal stop bit period during which sampling occurs halfway through. Refer to Section 25.3.11 for more details.
4. **2 stop bits:** Sampling for 2 stop bits is done on the 8th, 9th and 10th samples of the first stop bit. If a framing error is detected during the first stop bit the framing error flag will be set. The second stop bit is not checked for framing error. The RXNE flag will be set at the end of the first stop bit.

20.3.4 Fractional baud rate generation

The baud rate for the receiver and transmitter (Rx and Tx) are both set to the same value as programmed in the Mantissa and Fraction values of USARTDIV.

$$Tx/Rx \text{ baud} = f_{CK}/(16 \times USARTDIV)$$

legend: f_{CK} - Input clock to the peripheral (PCLK1 for USART2, 3, 4, 5 or PCLK2 for USART1)

USARTDIV is an unsigned fixed point number that is coded on the USART_BRR register.

Note: The baud counters are updated with the new value of the Baud registers after a write to USART_BRR. Hence the Baud rate register value should not be changed during communication.

How to derive USARTDIV from USART_BRR register values

Example 1:

If DIV_Mantissa = 27, DIV_Fraction = 12 (USART_BRR=0x1BC),
then
Mantissa (USARTDIV) = 27
Fraction (USARTDIV) = 12/16 = 0.75
Therefore USARTDIV = 27.75

Example 2:

To program USARTDIV = 25.62,

This leads to:

DIV_Fraction = $16 \times 0.62 = 9.92$

The nearest real number is: 10 = 0x0A

DIV_Mantissa = mantissa (25.620) = 25 = 0x19

Then, USART_BRR = 0x19A

Example 3:

To program USARTDIV = 50.99

This leads to:

DIV_Fraction = $16 \times 0.99 = 15.84$

The nearest real number is: 16 = 0x10 => DIV_frac[3:0] overflow => carry must be added up to the mantissa

DIV_Mantissa = mantissa (50.990 + carry) = 51 = 0x33

Then: USART_BRR = 0x330, USARTDIV=51

Table 83 Error calculation for programmed baud rates

Baud rate		f _{PCLK} = 36MHz			f _{PCLK} = 72MHz		
No.	Kbps	Actual	Value programmed in the Baud Rate register	Error%	Actual	Value programmed in the Baud Rate register	Error%
1	2.4	2.400	937.5	0%	2.4	1875	0%
2	9.6	9.600	234.375	0%	9.6	468.75	0%
3	19.2	19.2	117.1875	0%	19.2	234.375	0%
4	57.6	57.6	39.0625	0%	57.6	78.125	0%
5	115.2	115.384	19.5	0.15%	115.2	39.0625	0%
6	230.4	230.769	9.75	0.16%	230.769	19.5	0.16%
7	460.8	461.538	4.875	0.16%	461.538	9.75	0.16%
8	921.6	923.076	2.4375	0.16%	923.076	4.875	0.16%
9	2250	2250	1	0%	2250	2	0%
10	4500	NA	NA	NA	4500	1	0%

Note: The lower the CPU clock the lower will be the accuracy for a particular Baud rate. The upper limit of the achievable baud rate can be fixed with this data.

Only USART1 is clocked with PCLK2 (72 MHz max). Other USARTs are clocked with PCLK1 (36 MHz max)

20.3.5 USART receiver's tolerance to clock deviation

The USART's asynchronous receiver works correctly only if the total clock system deviation is smaller than the USART receiver tolerance. The causes that contribute to the total deviation are:

- DTRA: Deviation due to the transmitter error (which also includes the deviation of the transmitter local oscillator)
- DQUANT: Error due to the baud rate quantization of the receiver
- DREC: Deviation of the receiver's local oscillator
- DTCL: Deviation due to the transmission line (generally due to the transceivers that can introduce an asymmetry between the low-to-high transition timing and the high-to-low transition timing)

$DTRA + DQUANT + DREC + DTCL < \text{USART receiver tolerance}$

The USART receiver tolerance to properly receive data is equal to the maximum tolerated deviation and depends on the following choices:

- 10- or 11-bit character length defined by the M bit in the USART_CR1 register
- use of fractional baud rate or not

Table 84 USART receiver tolerance when DIV_Fraction is 0

M bit	NF is an error	NF is don't care
0	3.75%	4.375%
1	3.41%	3.97%

Table 85 USART receiver tolerance when DIV_Fraction is different from 0

M bit	NF is an error	NF is don't care
0	3.33%	3.88%
1	3.03%	3.53%

Note: The figures specified in Table 84 and Table 85 may slightly differ in the special case when the received frames contain some Idle frames of exactly 10-bit times when M=0 (11-bit times when M=1).

20.3.6 Multiprocessor communication

There is a possibility of performing multiprocessor communication with the USART (several USARTs connected in a network). For instance, one of the USARTs can be the master, its TX output is connected to the RX input of the other USART. The others are slaves, their respective TX outputs are logically ANDed together and connected to the RX input of the master.

In multiprocessor configurations it is often desirable that only the intended message recipient should actively receive the full message contents, thus reducing redundant USART service overhead for all non addressed receivers.

The non addressed devices may be placed in mute mode by means of the muting function. In mute mode:

- None of the reception status bits can be set
- All the receive interrupts are inhibited.
- The RWU bit in USART_CR1 register is set to 1. RWU can be controlled automatically by hardware or written by the software under certain conditions.

The USART can enter or exit from mute mode using one of two methods, depending on the WAKE bit in the USART_CR1 register:

- Idle Line detection if the WAKE bit is reset
- Address Mark detection if the WAKE bit is set

Idle line detection (WAKE=0)

The USART enters mute mode when the RWU bit is written to 1. It wakes up when an Idle frame is detected. Then the RWU bit is cleared by hardware but the IDLE bit is not set in the USART_SR register. RWU can also be written to 0 by software. An example of mute mode behavior using idle line detection is given in Figure 177.

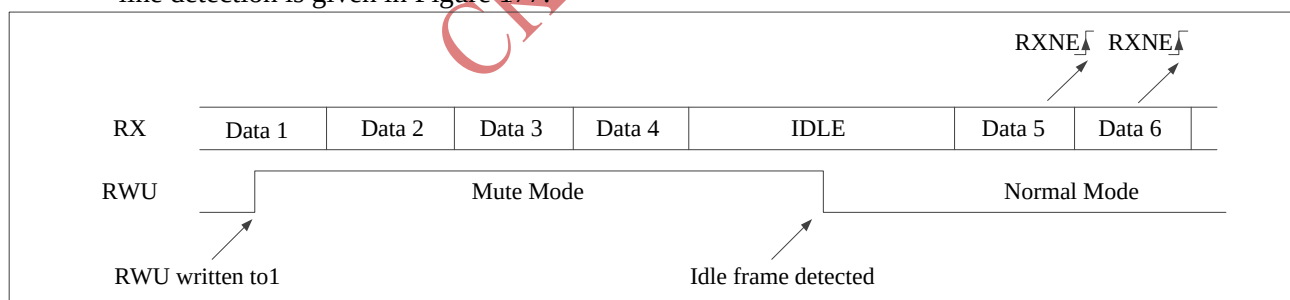


Figure 177 Mute mode using Idle line detection

Address mark detection (WAKE=1)

In this mode, bytes are recognized as addresses if their MSB is a '1' else they are considered as data. In an address byte, the address of the targeted receiver is put on the 4 LSB. This 4-bit word is compared by the receiver with its own address which is programmed in the ADD bits in the USART_CR2 register.

The USART enters mute mode when an address character is received which does not match its programmed address. In this case, the RWU bit is set by hardware. The RXNE flag is not set for this address byte and no interrupt nor DMA request is issued as the USART would have entered mute mode.

It exits from mute mode when an address character is received which matches the programmed address. Then the RWU bit is cleared and subsequent bytes are received normally. The RXNE bit is set for the address character since the RWU bit has been cleared.

The RWU bit can be written to as 0 or 1 when the receiver buffer contains no data (RXNE=0 in the USART_SR register). Otherwise the write attempt is ignored.

An example of mute mode behavior using address mark detection is given in Figure 178.

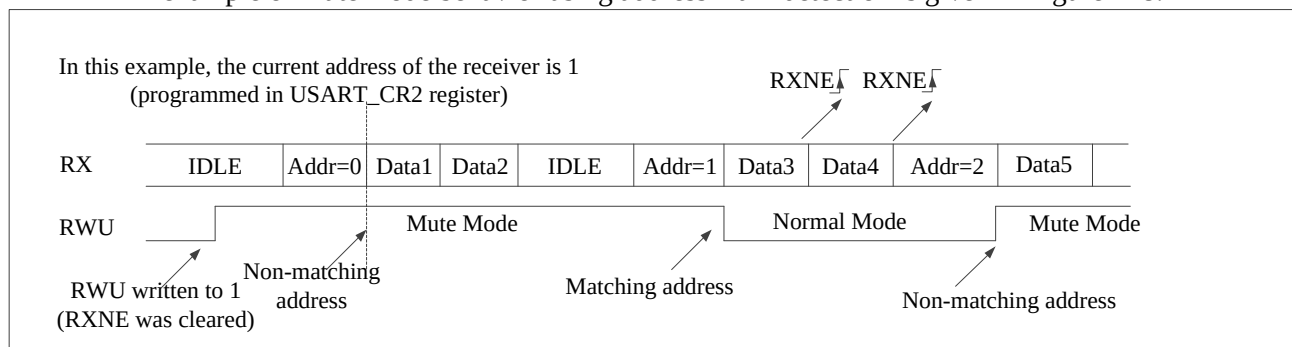


Figure 178 Mute mode using address mark detection

20.3.7 Parity control

Parity control (generation of parity bit in transmission and parity checking in reception) can be enabled by setting the PCE bit in the USART_CR1 register. Depending on the frame length defined by the M bit, the possible USART frame formats are as listed in Table 86.

Table 86 Frame formats

M bit	PCE bit	USART frame
0	0	SB 8-bit data STB
0	1	SB 7-bit data PB STB
1	0	SB 9-bit data STB
1	1	SB 8-bit data PB STB

Note: In case of wake up by an address mark, the MSB bit of the data is taken into account and not the parity bit.

Even parity: the parity bit is calculated to obtain an even number of “1s” inside the frame made of the 7 or 8 LSB bits (depending on whether M is equal to 0 or 1) and the parity bit.

Ex: data=00110101; 4 bits set => parity bit will be 0 if even parity is selected (PS bit in USART_CR1 = 0).

Odd parity: the parity bit is calculated to obtain an odd number of “1s” inside the frame made of the 7 or 8 LSB bits (depending on whether M is equal to 0 or 1) and the parity bit.

Ex: data=00110101; 4 bits set => parity bit will be 1 if odd parity is selected (PS bit in USART_CR1 = 1).

Transmission mode: If the PCE bit is set in USART_CR1, then the MSB bit of the data written in the data register is transmitted but is changed by the parity bit (even number of “1s” if even parity is selected (PS=0) or an odd number of “1s” if odd parity is selected (PS=1)). If the parity check fails, the PE flag is set in the USART_SR register and an interrupt is generated if PEIE is set in the USART_CR1 register.

20.3.8 LIN (local interconnection network) mode

The LIN mode is selected by setting the LINEN bit in the USART_CR2 register. In LIN mode, the following bits must be kept cleared:

- STOP[1:0], CLKEN in the USART_CR2 register
- SCEN, HDSEL and IREN in the USART_CR3 register.

LIN transmission

The same procedure explained in Section 20.3.2 has to be applied for LIN Master transmission than for normal USART transmission with the following differences:

- Clear the M bit to configure 8-bit word length.

- Set the LINEN bit to enter LIN mode. In this case, setting the SBK bit sends 13 '0' bits as a break character. Then a bit of value '1' is sent to allow the next start detection.

LIN reception

A break detection circuit is implemented in the USART. The detection is totally independent from the normal USART receiver. A break can be detected whenever it occurs, during idle state or during a frame.

When the receiver is enabled (RE=1 in USART_CR1), the circuit looks at the RX input for a start signal. The method for detecting start bits is the same when searching break characters or data. After a start bit has been detected, the circuit samples the next bits exactly like for the data (on the 8th, 9th and 10th samples). If 10 (when the LBDL = 0 in USART_CR2) or 11 (when LBDL=1 in USART_CR2) consecutive bits are detected as '0', and are followed by a delimiter character, the LBD flag is set in USART_SR. If the LBDIE bit=1, an interrupt is generated. Before validating the break, the delimiter is checked for as it signifies that the RX line has returned to a high level.

If a '1' is sampled before the 10 or 11 have occurred, the break detection circuit cancels the current detection and searches for a start bit again.

If the LIN mode is disabled (LINEN=0), the receiver continues working as normal USART, without taking into account the break detection.

If the LIN mode is enabled (LINEN=1), as soon as a framing error occurs (i.e. stop bit detected at '0', which will be the case for any break frame), the receiver stops until the break detection circuit receives either a '1', if the break word was not complete, or a delimiter character if a break has been detected.

The behavior of the break detector state machine and the break flag is shown on the Figure 179.

Examples of break frames are given on Figure 180

CKS copyright

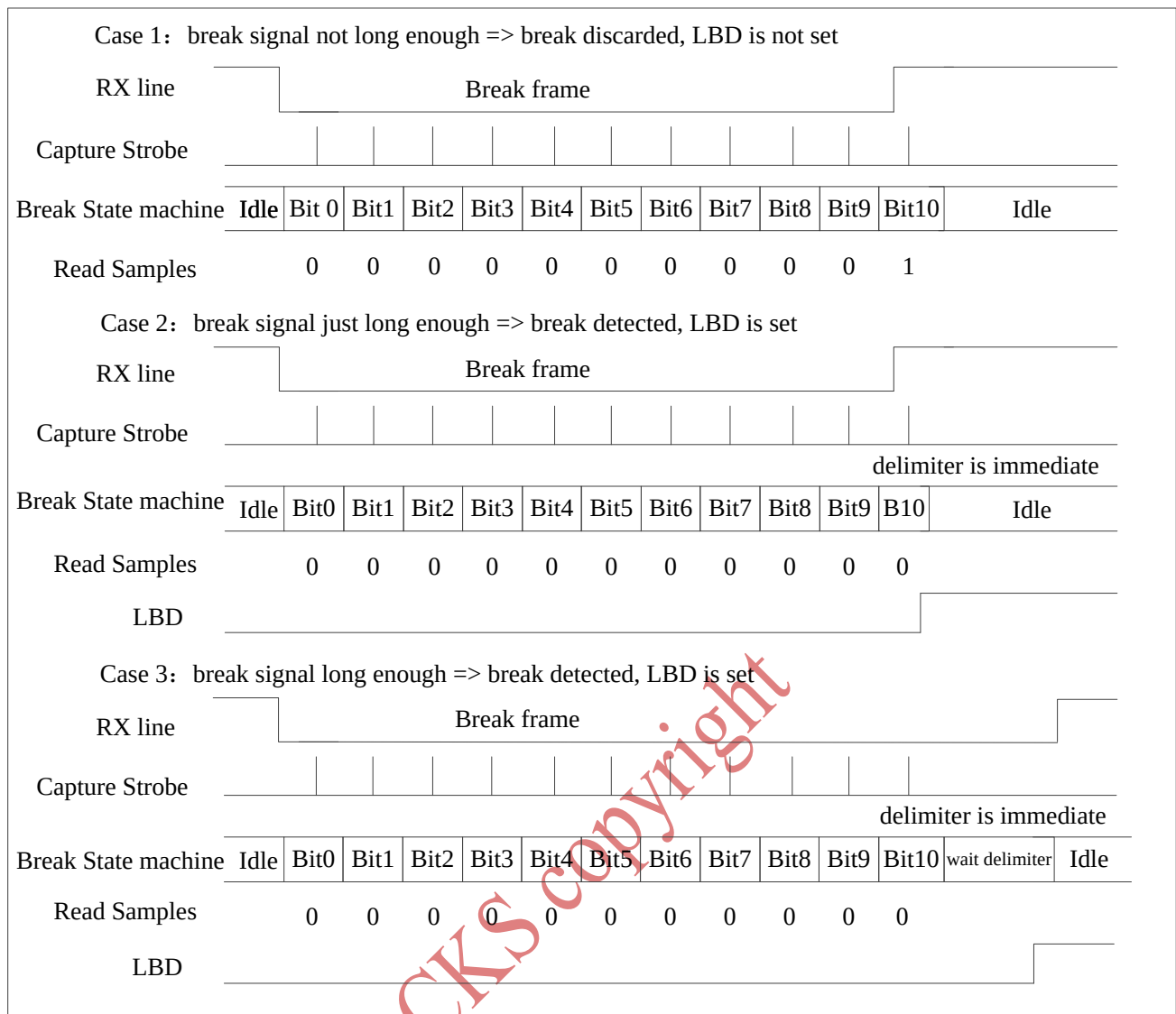
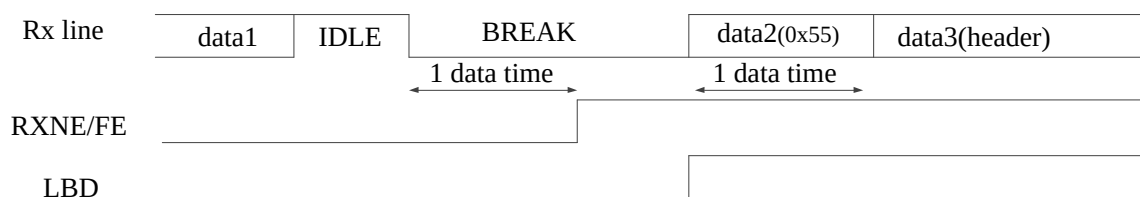


Figure 179 Break detection in LIN mode (11-bit break length - LBDL bit is set)

In these examples, we suppose that LBDL=1(11-bit break length), M=0(8-bit data)

Case 1: break occurring after an Idle



Case 2: break occurring while a data is being received

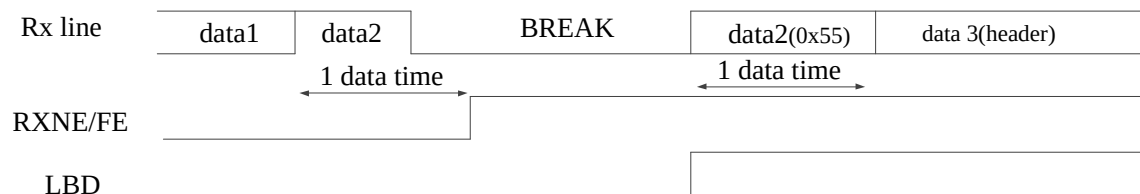


Figure 180 Break detection in LIN mode vs. Framing error detection

20.3.9 USART synchronous mode

The synchronous mode is selected by writing the CLKEN bit in the USART_CR2 register to 1. In synchronous mode, the following bits must be kept cleared:

- LINEN bit in the USART_CR2 register,
- SCEN, HDSEL and IREN bits in the USART_CR3 register.

The USART allows the user to control a bidirectional synchronous serial communications in master mode. The CK pin is the output of the USART transmitter clock. No clock pulses are sent to the CK pin during start bit and stop bit. Depending on the state of the LBCL bit in the USART_CR2 register clock pulses will or will not be generated during the last valid data bit (address mark). The CPOL bit in the USART_CR2 register allows the user to select the clock polarity, and the CPHA bit in the USART_CR2 register allows the user to select the phase of the external clock (see Figure 181, Figure 182 and Figure 183).

During idle, preamble and send break, the external CK clock is not activated.

In synchronous mode the USART transmitter works exactly like in asynchronous mode. But as CK is synchronized with TX (according to CPOL and CPHA), the data on TX is synchronous.

In this mode the USART receiver works in a different manner compared to the asynchronous mode. If RE=1, the data is sampled on CK (rising or falling edge, depending on CPOL and CPHA), without any oversampling. A setup and a hold time must be respected (which depends on the baud rate: 1/16 bit time).

Note: The CK pin works in conjunction with the TX pin. Thus, the clock is provided only if the transmitter is enabled (TE=1) and a data is being transmitted (the data register USART_DR has been written). This means that it is not possible to receive a synchronous data without transmitting data.

The LBCL, CPOL and CPHA bits have to be selected when both the transmitter and the receiver are disabled (TE=RE=0) to ensure that the clock pulses function correctly. These bits should not be changed while the transmitter or the receiver is enabled.

It is advised that TE and RE are set in the same instruction to minimize the setup and the hold time of the receiver.

The USART supports master mode only: it cannot receive or send data related to an input clock (CK is always an output).

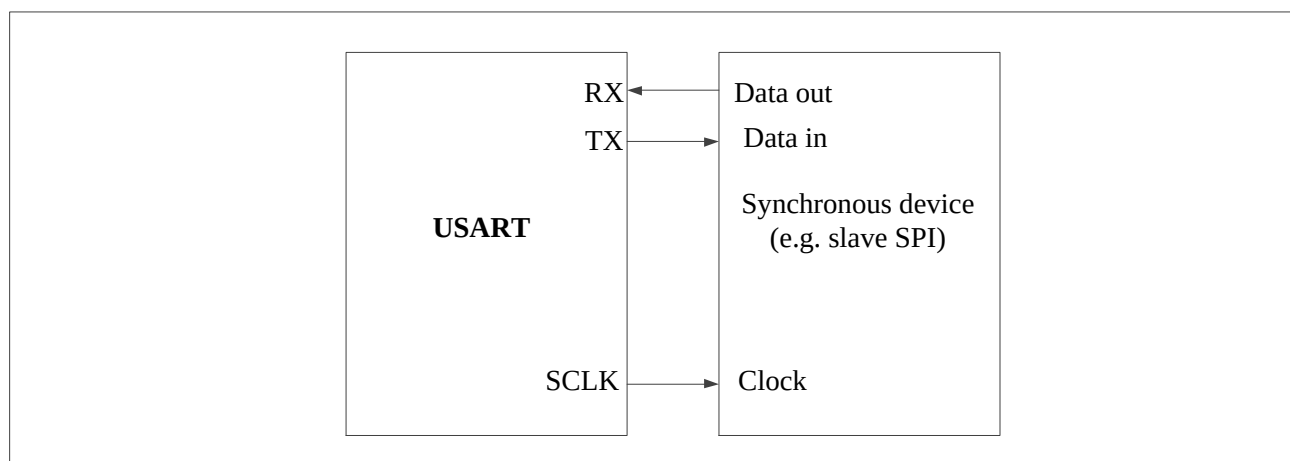


Figure 181 USART example of synchronous transmission

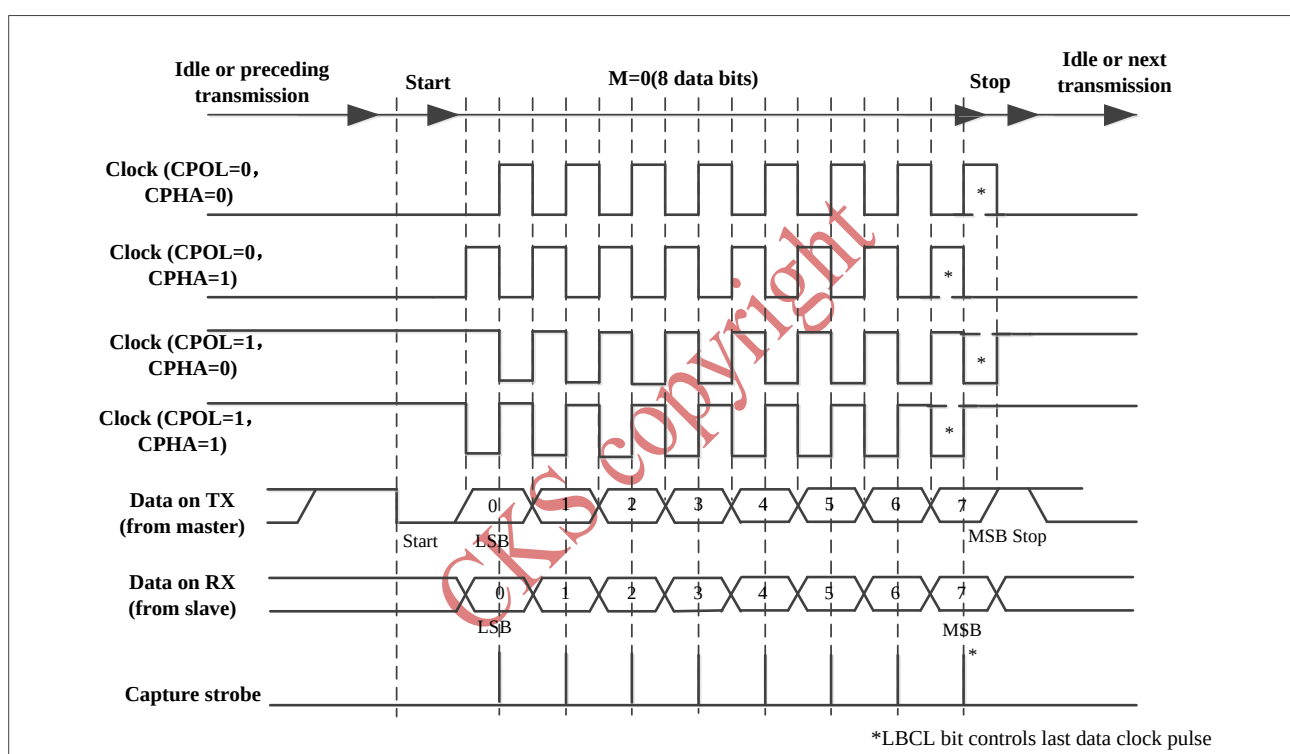


Figure 182 USART data clock timing diagram (M=0)

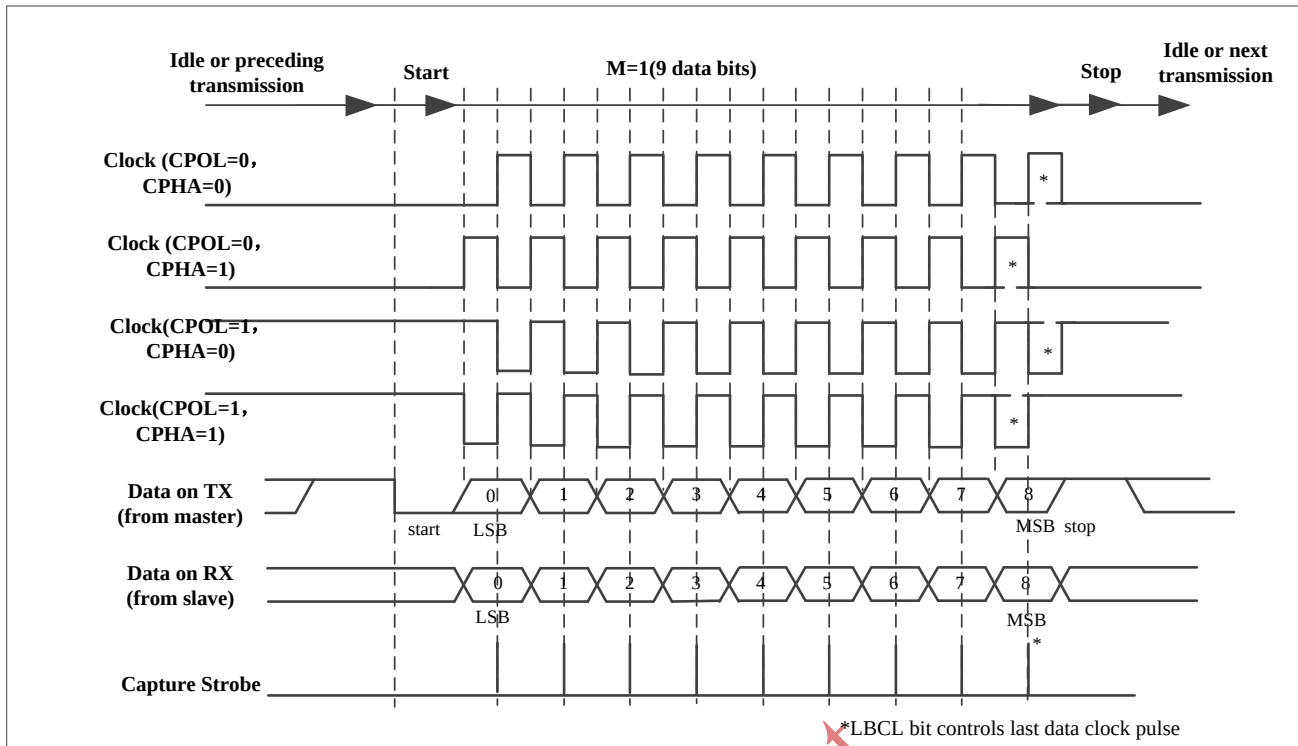


Figure 183 USART data clock timing diagram (M=1)

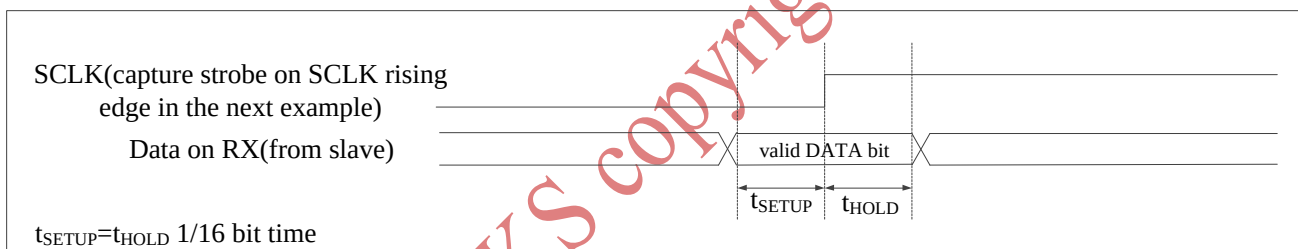


Figure 184 RX data setup/hold time

Note: The function of CK is different in Smartcard mode. Refer to the Smartcard mode section for more details.

20.3.10 Single-wire half-duplex communication

The single-wire half-duplex mode is selected by setting the HDSEL bit in the USART_CR3 register. In this mode, the following bits must be kept cleared:

- LINEN and CLKEN bits in the USART_CR2 register,
- SCEN and IREN bits in the USART_CR3 register.

The USART can be configured to follow a single-wire half-duplex protocol. In single-wire half-duplex mode, the TX and RX pins are connected internally. The selection between half- and full-duplex communication is made with a control bit 'HALF DUPLEX SEL' (HDSEL in USART_CR3).

As soon as HDSEL is written to 1:

- RX is no longer used
- TX is always released when no data is transmitted. Thus, it acts as a standard IO in idle or in reception. It means that the IO must be configured so that TX is configured as floating input (or output high open-drain) when not driven by the USART.

Apart from this, the communications are similar to what is done in normal USART mode. The conflicts on the line must be managed by the software (by the use of a centralized arbiter, for instance). In particular, the transmission is never blocked by hardware and continue to occur as soon as a data is written in the data register while the TE bit is set.

20.3.11 Smartcard

The Smartcard mode is selected by setting the SCEN bit in the USART_CR3 register. In Smartcard mode, the following bits must be kept cleared:

- LINEN bit in the USART_CR2 register
- HDSEL and IREN bits in the USART_CR3 register.

Moreover, the CLKEN bit may be set in order to provide a clock to the smartcard.

The Smartcard interface is designed to support asynchronous protocol Smartcards as defined in the ISO 7816-3 standard. The USART should be configured as:

- 8 bits plus parity: where M=1 and PCE=1 in the USART_CR1 register
- 1.5 stop bits when transmitting and receiving : where STOP='11' in the USART_CR2 register.

Note: It is also possible to choose 0.5 stop bit for receiving but it is recommended to use 1.5 stop bits for both transmitting and receiving to avoid switching between the two configurations.

Figure 185 shows examples of what can be seen on the data line with and without parity error.

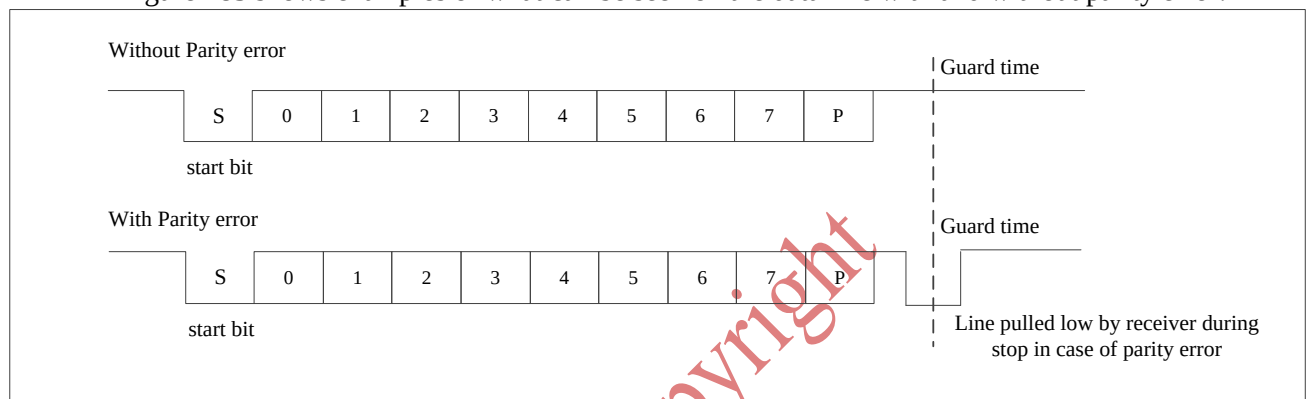


Figure 185 ISO 7816-3 asynchronous protocol

When connected to a smartcard, the USART TX output drives a bidirectional line that is also driven by the smartcard. The TX pin must be configured as open drain.

Smartcard is a single wire half duplex communication protocol.

- Transmission of data from the transmit shift register is guaranteed to be delayed by a minimum of 1/2 baud clock. In normal operation a full transmit shift register will start shifting on the next baud clock edge. In Smartcard mode this transmission is further delayed by a guaranteed 1/2 baud clock.
- If a parity error is detected during reception of a frame programmed with a 0.5 or 1.5 stop bit period, the transmit line is pulled low for a baud clock period after the completion of the receive frame. This is to indicate to the Smartcard that the data transmitted to USART has not been correctly received. This NACK signal (pulling transmit line low for 1 baud clock) will cause a framing error on the transmitter side (configured with 1.5 stop bits). The application can handle re-sending of data according to the protocol. A parity error is 'NACK'ed by the receiver if the NACK control bit is set, otherwise a NACK is not transmitted.
- The assertion of the TC flag can be delayed by programming the Guard Time register. In normal operation, TC is asserted when the transmit shift register is empty and no further transmit requests are outstanding. In Smartcard mode an empty transmit shift register triggers the guard time counter to count up to the programmed value in the Guard Time register. TC is forced low during this time. When the guard time counter reaches the programmed value TC is asserted high.
- The de-assertion of TC flag is unaffected by Smartcard mode.
- If a framing error is detected on the transmitter end (due to a NACK from the receiver), the NACK will not be detected as a start bit by the receive block of the transmitter. According to the ISO protocol, the duration of the received NACK can be 1 or 2 baud clock periods.
- On the receiver side, if a parity error is detected and a NACK is transmitted the receiver will not detect the NACK as a start bit.

Note: A break character is not significant in Smartcard mode. A 0x00 data with a framing error will be treated as data and not as a break.

No IDLE frame is transmitted when toggling the TE bit. The IDLE frame (as defined for the other configurations) is not defined by the ISO protocol.

Figure 186 details how the NACK signal is sampled by the USART. In this example, the USART transmits a data and is configured with 1.5 stop bits. The receiver part of the USART is enabled in order to check the integrity of the data and the NACK signal.

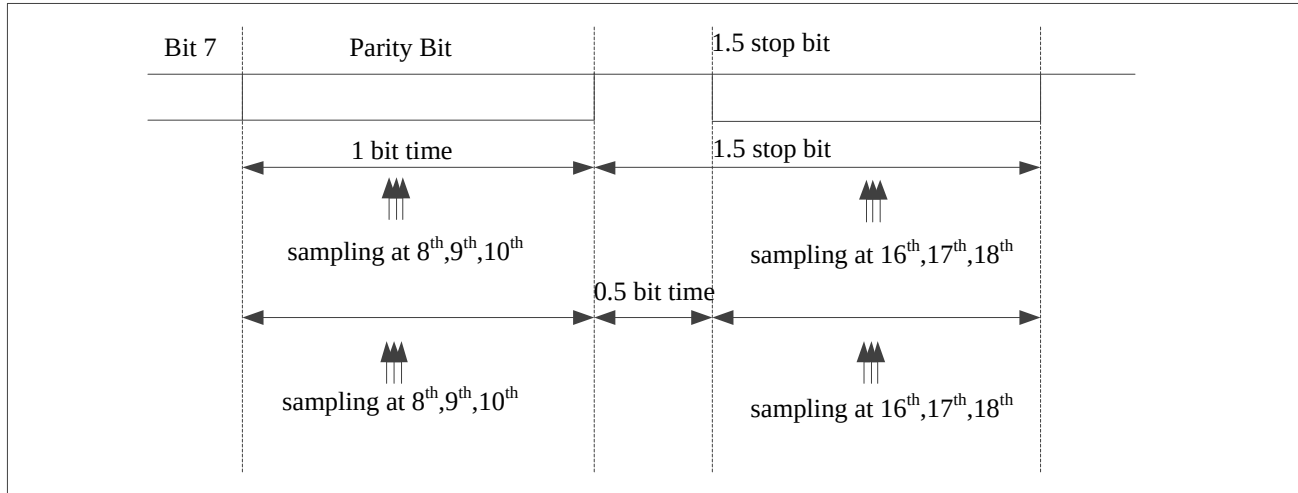


Figure 186 Parity error detection using the 1.5 stop bits

The USART can provide a clock to the smartcard through the CK output. In Smartcard mode, CK is not associated to the communication but is simply derived from the internal peripheral input clock through a 5-bit prescaler. The division ratio is configured in the prescaler register USART_GTPR. CK frequency can be programmed from $f_{CK}/2$ to $f_{CK}/62$, where f_{CK} is the peripheral input clock.

20.3.12 IrDA SIR ENDEC block

The IrDA mode is selected by setting the IREN bit in the USART_CR3 register. In IrDA mode, the following bits must be kept cleared:

- LINEN, STOP and CLKEN bits in the USART_CR2 register
- SCEN and HDSEL bits in the USART_CR3 register.

The IrDA SIR physical layer specifies use of a Return to Zero, Inverted (RZI) modulation scheme that represents logic 0 as an infrared light pulse (see Figure 187). The SIR Transmit encoder modulates the Non Return to Zero (NRZ) transmit bit stream output from USART. The output pulse stream is transmitted to an external output driver and infrared LED. USART supports only bit rates up to 115.2Kbps for the SIR ENDEC. In normal mode the transmitted pulse width is specified as 3/16 of a bit period.

The SIR receive decoder demodulates the return-to-zero bit stream from the infrared detector and outputs the received NRZ serial bit stream to USART. The decoder input is normally HIGH (marking state) in the idle state. The transmit encoder output has the opposite polarity to the decoder input. A start bit is detected when the decoder input is low.

- IrDA is a half duplex communication protocol. If the Transmitter is busy (i.e. the USART sends data to the IrDA encoder), any data on the IrDA receive line is ignored by the IrDA decoder and if the Receiver is busy (USART receives decoded data from the USART), data on the TX from the USART to IrDA is not encoded by IrDA. While receiving data, transmission should be avoided as the data to be transmitted could be corrupted.
- A '0' is transmitted as a high pulse and a '1' is transmitted as a '0'. The width of the pulse is specified as 3/16th of the selected bit period in normal mode (see Figure 188).
- The SIR decoder converts the IrDA compliant receive signal into a bit stream for USART
- The SIR receive logic interprets a high state as a logic one and low pulses as logic zeros.

- The transmit encoder output has the opposite polarity to the decoder input. The SIR output is in low state when idle.
- The IrDA specification requires the acceptance of pulses greater than 1.41 μ s. The acceptable pulse width is programmable. Glitch detection logic on the receiver end filters out pulses of width less than 2 PSC periods (PSC is the prescaler value programmed in the IrDA low-power Baud Register, USART_GTPR). Pulses of width less than 1 PSC period are always rejected, but those of width greater than one and less than two periods may be accepted or rejected, those greater than 2 periods will be accepted as a pulse. The IrDA encoder/decoder doesn't work when PSC=0.
- The receiver can communicate with a low-power transmitter.
- In IrDA mode, the STOP bits in the USART_CR2 register must be configured to "1 stop bit".

IrDA low-power mode

Transmitter

In low-power mode the pulse width is not maintained at 3/16 of the bit period. Instead, the width of the pulse is 3 times the low-power baud rate which can be a minimum of 1.42 MHz. Generally this value is 1.8432 MHz ($1.42 \text{ MHz} < \text{PSC} < 2.12 \text{ MHz}$). A low-power mode programmable divisor divides the system clock to achieve this value.

Receiver

Receiving in low-power mode is similar to receiving in normal mode. For glitch detection, the USART should discard pulses of duration shorter than $1/\text{PSC}$. A valid low is accepted only if its duration is greater than 2 periods of the IrDA low-power Baud clock (PSC value in USART_GTPR).

Note:

A pulse of width less than two and greater than one PSC period(s) may or may not be rejected.

The receiver set up time should be managed by software. The IrDA physical layer specification specifies a minimum of 10 ms delay between transmission and reception (IrDA is a half duplex protocol).

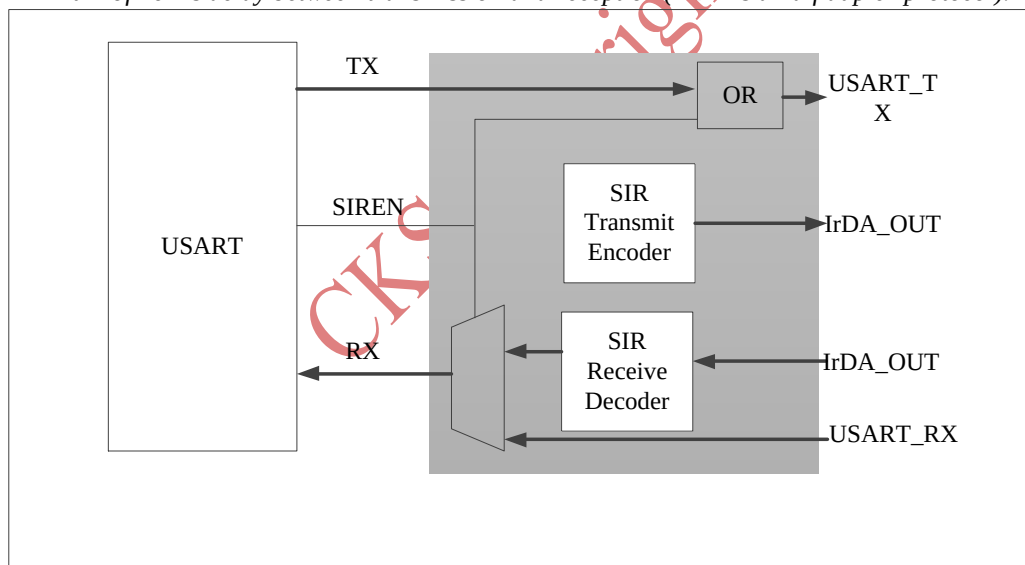


Figure 187 IrDA SIR ENDEC- block diagram

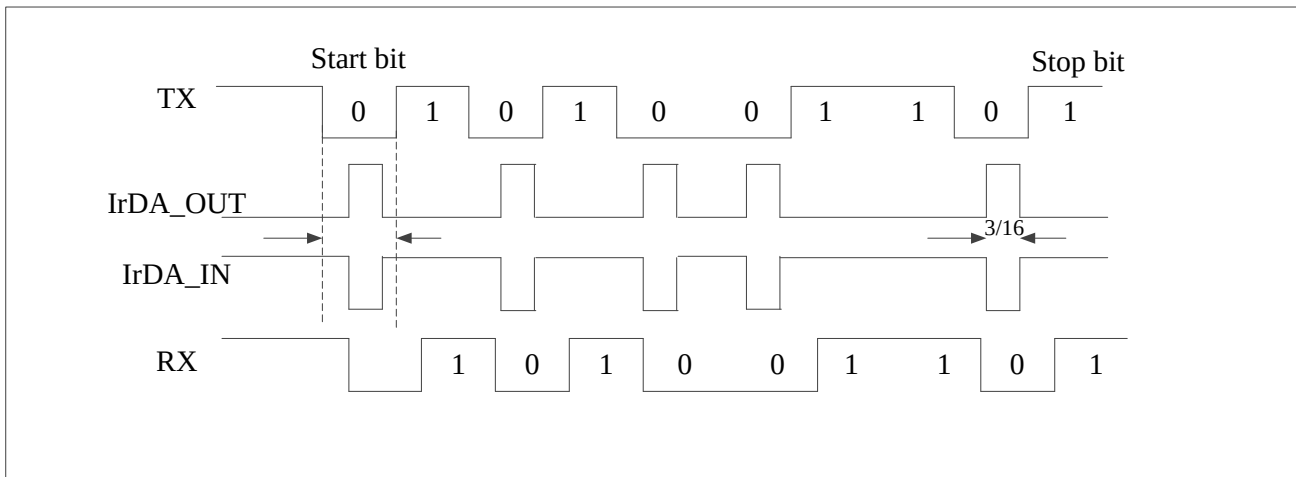


Figure 188 IrDA data modulation (3/16) -normal mode

20.3.13 Continuous communication using DMA

The USART is capable of continuing communication using the DMA. The DMA requests for Rx buffer and Tx buffer are generated independently.

Note: User should refer to product specs for availability of the DMA controller. If DMA is not available in the product, you should use the USART as explained in Section 20.3.2 or 20.3.3. In the USART_SR register, user can clear the TXE/ RXNE flags to achieve continuous communication.

Transmission using DMA

DMA mode can be enabled for transmission by setting DMAT bit in the USART_CR3 register. Data is loaded from a SRAM area configured using the DMA peripheral (refer to the DMA specification) to the USART_DR register whenever the TXE bit is set. To map a DMA channel for USART transmission, use the following procedure (x denotes the channel number):

1. Write the USART_DR register address in the DMA control register to configure it as the destination of the transfer. The data will be moved to this address from memory after each TXE event.
2. Write the memory address in the DMA control register to configure it as the source of the transfer. The data will be loaded into the USART_DR register from this memory area after each TXE event.
3. Configure the total number of bytes to be transferred to the DMA control register.
4. Configure the channel priority in the DMA register
5. Configure DMA interrupt generation after half/ full transfer as required by the application.
6. Activate the channel in the DMA register.

When the number of data transfers programmed in the DMA Controller is reached, the DMA controller generates an interrupt on the DMA channel interrupt vector.

In transmission mode, once the DMA has written all the data to be transmitted (the TCIF flag is set in the DMA_ISR register), the TC flag can be monitored to make sure that the USART communication is complete. This is required to avoid corrupting the last transmission before disabling the USART or entering the Stop mode. The software must wait until TC=1. The TC flag remains cleared during all data transfers and it is set by hardware at the last frame's end of transmission.

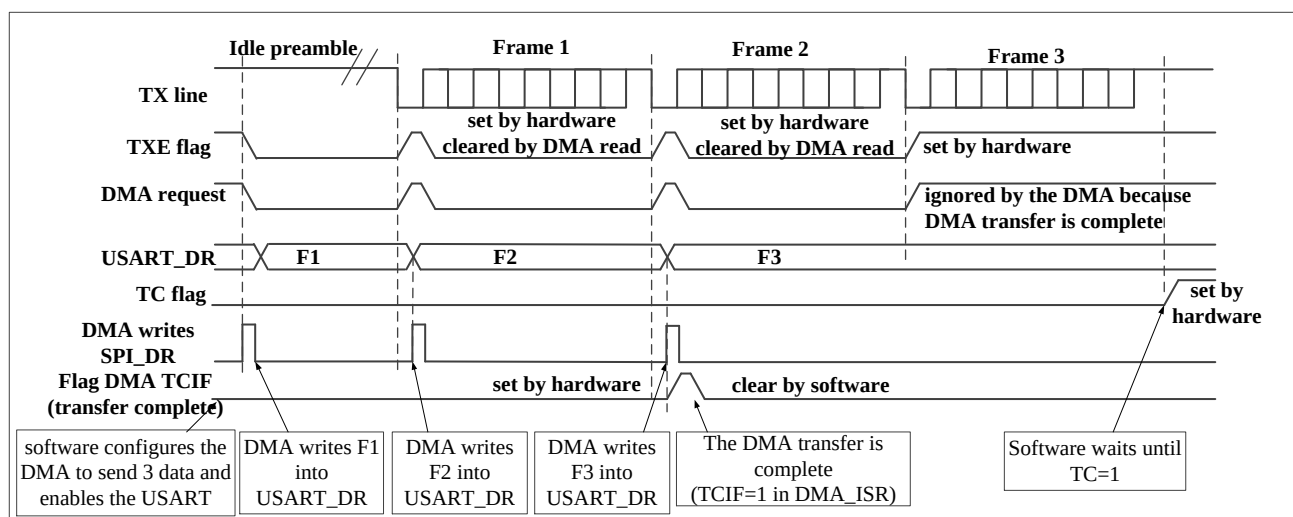


Figure 189 Transmission using DMA

Reception using DMA

DMA mode can be enabled for reception by setting the DMAR bit in USART_CR3 register. Data is loaded from the USART_DR register to a SRAM area configured using the DMA peripheral (refer to the DMA specification) whenever a data byte is received. To map a DMA channel for USART reception, use the following procedure:

1. Write the USART_DR register address in the DMA control register to configure it as the source of the transfer. The data will be moved from this address to the memory after each RXNE event.
2. Write the memory address in the DMA control register to configure it as the destination of the transfer. The data will be loaded from USART_DR to this memory area after each RXNE event.
3. Configure the total number of bytes to be transferred in the DMA control register.
4. Configure the channel priority in the DMA control register
5. Configure interrupt generation after half/ full transfer as required by the application.
6. Activate the channel in the DMA control register.

When the number of data transfers programmed in the DMA Controller is reached, the DMA controller generates an interrupt on the DMA channel interrupt vector.

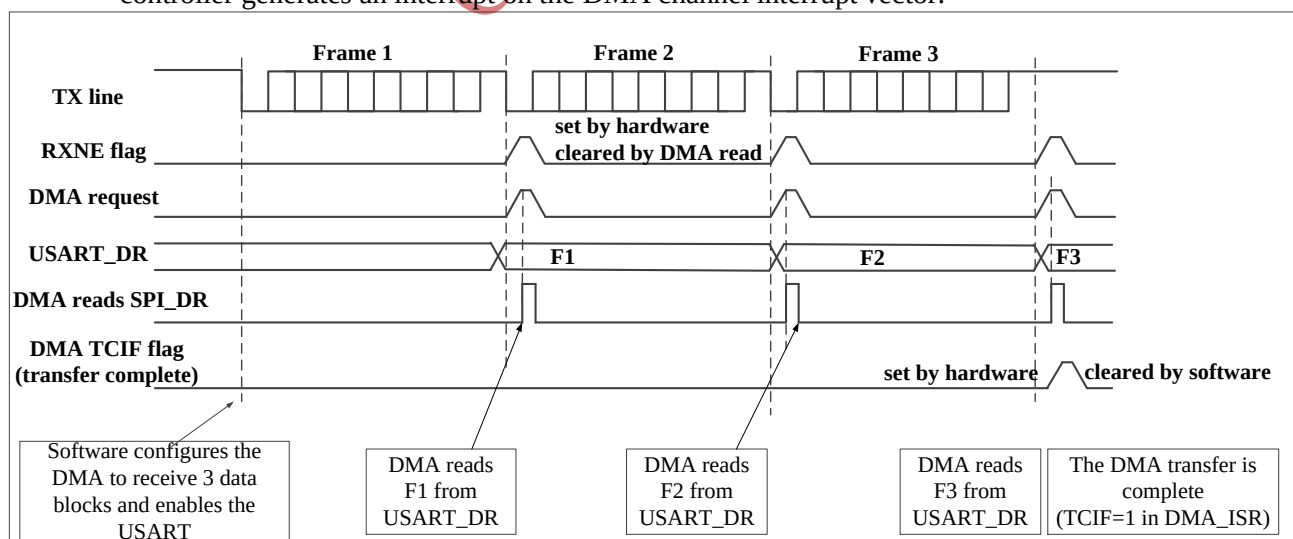


Figure 190 Reception using DMA

Error flagging and interrupt generation in multibuffer communication

In case of multibuffer communication if any error occurs during the transaction the error flag will be asserted after the current byte. An interrupt will be generated if the interrupt enable flag is set. For framing error, overrun error and noise flag which are asserted with RXNE in case of single byte reception, there will be separate error flag interrupt enable bit (EIE bit in the USART_CR3 register), which if set will issue an interrupt after the current byte with either of these errors.

20.3.14 Hardware flow control

It is possible to control the serial data flow between two devices by using the CTS input and the RTS output. Figure 191 shows how to connect two devices in this mode:

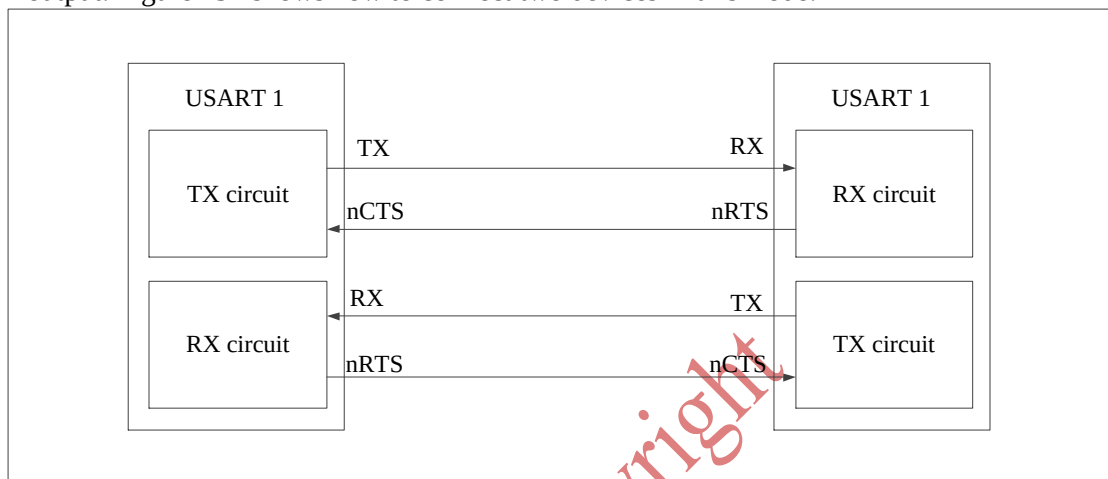


Figure 191 Hardware flow control between two USARTs

RTS and CTS flow control can be enabled independently by writing respectively RTSE and CTSE bits to 1 (in the USART_CR3 register).

RTS flow control

If the RTS flow control is enabled (RTSE=1), then RTS is asserted (tied low) as long as the USART receiver is ready to receive new data. When the receive register is full, RTS is deasserted, indicating that the transmission is expected to stop at the end of the current frame. Figure 192 shows an example of communication with RTS flow control enabled.

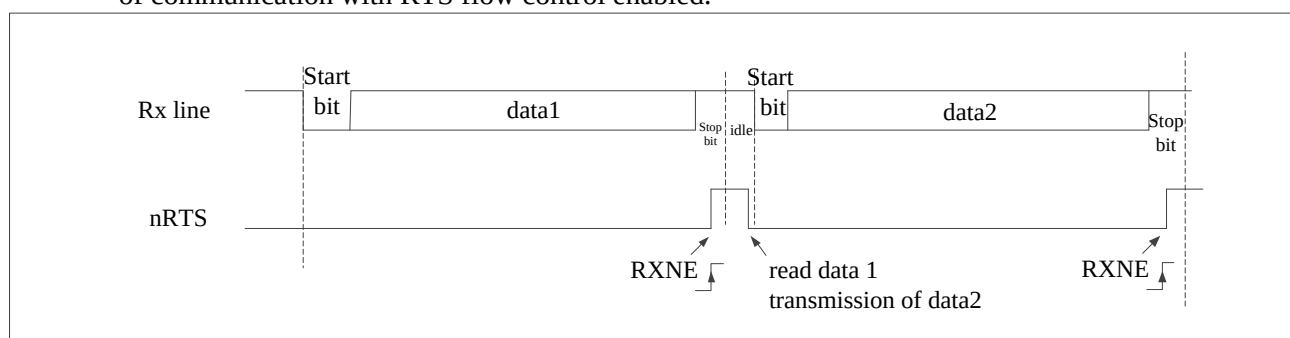


Figure 192 RTS flow control

CTS flow control

If the CTS flow control is enabled (CTSE=1), then the transmitter checks the CTS input before transmitting the next frame. If CTS is asserted (tied low), then the next data is transmitted (assuming that a data is to be transmitted, in other words, if TXE=0), else the transmission does not occur. When CTS is deasserted during a transmission, the current transmission is completed before the transmitter stops.

When CTSE=1, the CTSIF status bit is automatically set by hardware as soon as the CTS input toggles. It indicates when the receiver becomes ready or not ready for communication. An interrupt is generated if the CTSIE bit in the USART_CR3 register is set. The figure below shows an example of communication with CTS flow control enabled.

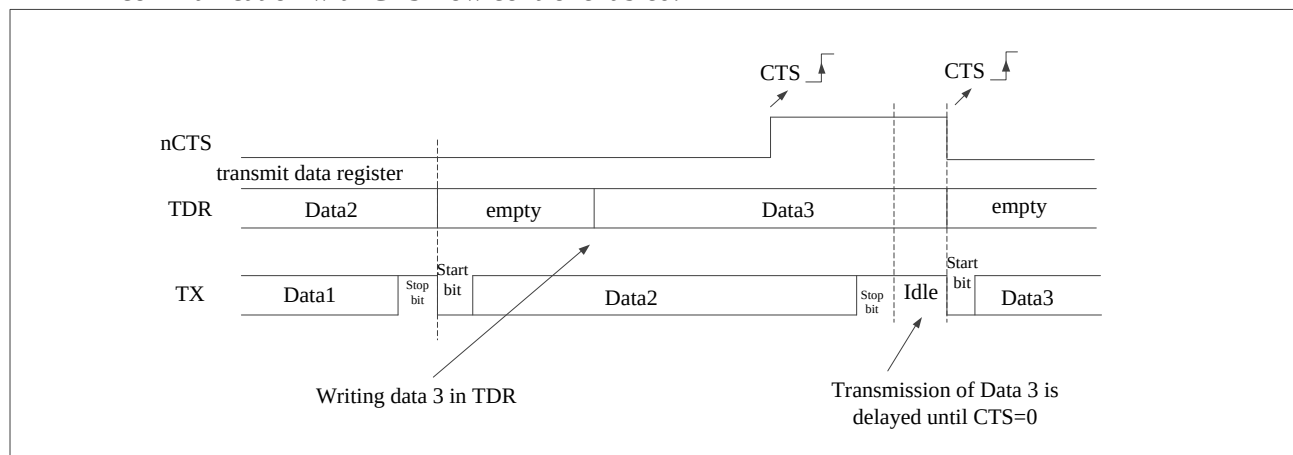


Figure 193 CTS flow control

20.4 USART interrupts

Table 87 USART interrupt requests

Interrupt event	Event flag	Enable Control bit
Transmit data register empty	TXE	TXEIE
CTS flag	CTS	CTSIE
Transmission complete	TC	TCIE
Received data ready to be read	TXNE	TXNEIE
Overrun error detected	ORE	
Idle line detected	IDLE	IDLEIE
Parity error	PE	PEIE
Break flag	LBD	LBDIE
Noise flag, Overrun error and Framing error in multibuffer communication	NE or ORT or FE	EIE ⁽¹⁾

1. This bit is used only when data reception is performed by DMA.

The USART interrupt events are connected to the same interrupt vector (see Figure 194).

- During transmission: Transmission Complete, Clear to Send or Transmit Data Register empty interrupt.
- While receiving: Idle Line detection, Overrun error, Receive Data register not empty, Parity error, LIN break detection, Noise Flag (only in multi buffer communication) and Framing Error (only in multi buffer communication)

These events generate an interrupt if the corresponding Enable Control Bit is set.

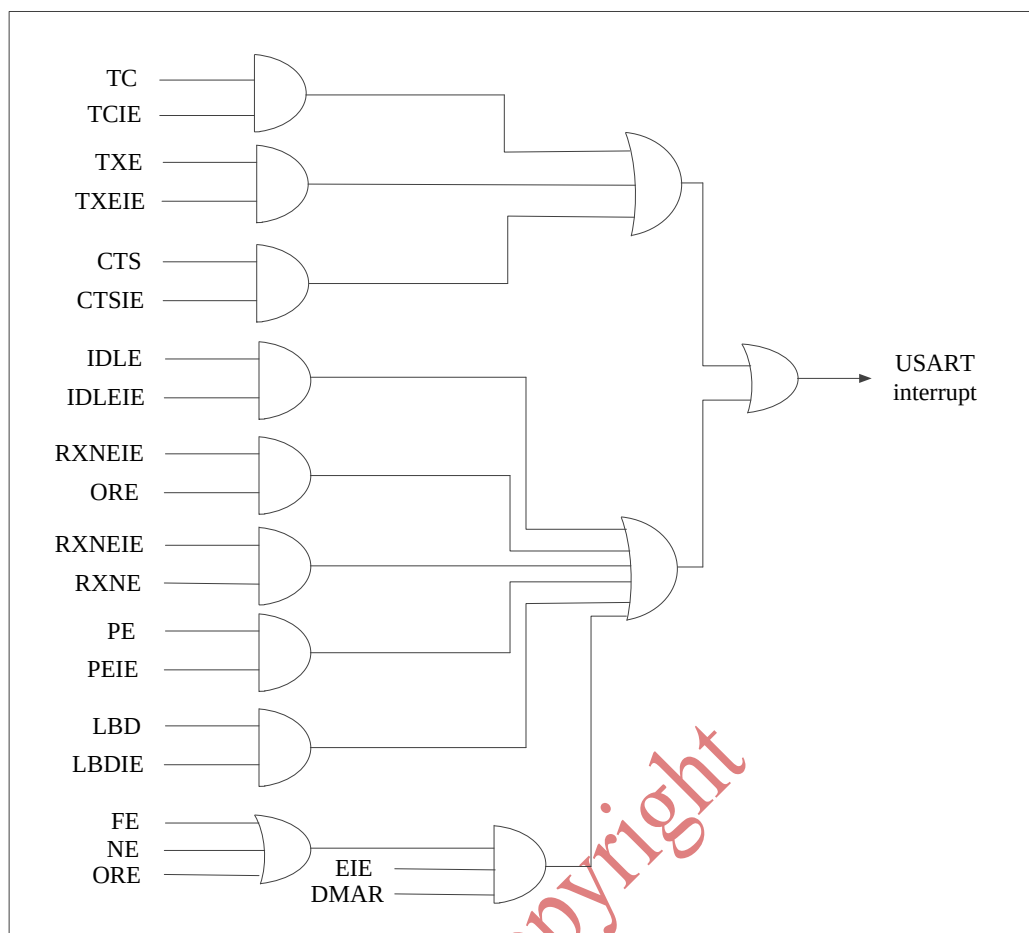


Figure 194 USART interrupt mapping diagram

20.5 USART mode configuration

Table 88 USART mode configuration⁽¹⁾

USART modes	USART1	USART2	USART3	UART4	UART5
Asynchronous mode	X	X	X	X	X
Hardware Flow Control	X	X	X	NA	NA
Multibuffer Communication (DMA)	X	X	X	X	X
Multiprocessor Communication	X	X	X	X	X
Synchronous	X	X	X	NA	NA
Smartcard	X	X	X	NA	NA
Half-Duplex (Single-Wire mode)	X	X	X	X	X
IrDA	X	X	X	X	X
LIN	X	X	X	X	X

(1) X = supported; NA = not applicable.

20.6 USART registers

Refer to *Section 1.1* for a list of abbreviations used in register descriptions.

The peripheral registers can be accessed by half-words (16-bit) or words (32-bit).

20.6.1 Status register (USART_SR)

Address offset: 0x00

Reset value: 0x00C0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
						rc w0	rc w0	r	rc w0	rc w0	r	r	r	r	r
Bits 31: 10		Reserved, forced by hardware to 0.													
Bit 9		CTS: CTS flag This bit is set by hardware when the CTS input toggles, if the CTSE bit is set. It is cleared by software (by writing it to 0). An interrupt is generated if CTSIE=1 in the USART_CR3 register. 0: No change occurred on the CTS status line 1: A change occurred on the CTS status line This bit is not available for UART4 & UART5													
Bit 8		LBD: LIN break detection flag This bit is set by hardware when the LIN break is detected. It is cleared by software (by writing it to 0). An interrupt is generated if LBDIE = 1 in the USART_CR2 register. 0: LIN Break not detected 1: LIN break detected <i>Note: An interrupt is generated when LBD=1 if LBDIE=1</i>													
Bit 7		TXE: Transmit data register empty This bit is set by hardware when the content of the TDR register has been transferred into the shift register. An interrupt is generated if the TXEIE bit =1 in the USART_CR1 register. It is cleared by a write to the USART_DR register. 0: Data is not transferred to the shift register 1: Data is transferred to the shift register <i>Note: This bit is used during single buffer transmission.</i>													
Bit 6		TC: Transmission complete This bit is set by hardware if the transmission of a frame containing data is complete and if TXE is set. An interrupt is generated if TCIE=1 in the USART_CR1 register. It is cleared by a software sequence (a read from the USART_SR register followed by a write to the USART_DR register). The TC bit can also be cleared by writing a '0' to it. This clearing sequence is recommended only for multibuffer communication. 0: Transmission is not complete 1: Transmission is complete													
Bit 5		RXNE: Read data register not empty This bit is set by hardware when the content of the RDR shift register has been transferred to the USART_DR register. An interrupt is generated if RXNEIE=1 in the USART_CR1 register. It is cleared by a read to the USART_DR register. The RXNE flag can also be cleared by writing a zero to it. This clearing sequence is recommended only for multibuffer communication. 0: Data is not received 1: Received data is ready to be read.													
Bit 4		IDLE: IDLE line detected This bit is set by hardware when an Idle Line is detected. An interrupt is generated if the IDLEIE=1 in the USART_CR1 register. It is cleared by a software sequence (an read to the USART_SR register followed by a read to the USART_DR register). 0: No Idle Line is detected 1: Idle Line is detected <i>Note: The IDLE bit will not be set again until the RXNE bit has been set itself (i.e. a new idle line occurs).</i>													
Bit 3		ORE: Overrun error													

	This bit is set by hardware when the word currently being received in the shift register is ready to be transferred into the RDR register while RXNE=1. An interrupt is generated if RXNEIE=1 in the USART_CR1 register. It is cleared by a software sequence (an read to the USART_SR register followed by a read to the USART_DR register). 0: No Overrun error 1: Overrun error is detected <i>Note: When this bit is set, the RDR register content will not be lost but the shift register will be overwritten. An interrupt is generated on ORE flag in case of Multi Buffer communication if the EIE bit is set.</i>
Bit 2	NE: Noise error flag This bit is set by hardware when noise is detected on a received frame. It is cleared by a software sequence (an read to the USART_SR register followed by a read to the USART_DR register). 0: No noise is detected 1: Noise is detected <i>Note: This bit does not generate interrupt as it appears at the same time as the RXNE bit which itself generates an interrupt. An interrupt is generated on NE flag in case of Multi Buffer communication if the EIE bit is set.</i>
Bit 1	FE: Framing error This bit is set by hardware when a de-synchronization, excessive noise or a break character is detected. It is cleared by a software sequence (an read to the USART_SR register followed by a read to the USART_DR register). 0: No Framing error is detected 1: Framing error or break character is detected <i>Note: This bit does not generate interrupt as it appears at the same time as the RXNE bit which itself generates an interrupt. If the word currently being transferred causes both frame error and overrun error, it will be transferred and only the ORE bit will be set. An interrupt is generated on FE flag in case of Multi Buffer communication if the EIE bit is set.</i>
Bit 0	PE: Parity error This bit is set by hardware when a parity error occurs in receiver mode. It is cleared by a software sequence (a read to the status register followed by a read to the USART_DR data register). The software must wait for the RXNE flag to be set before clearing the PE bit. An interrupt is generated if PEIE = 1 in the USART_CR1 register. 0: No parity error 1: Parity error

20.6.2 Data register (USART_DR)

Address offset: 0x04

Reset value: Undefined

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								DR[8:0]							
								rW	rW	rW	rW	rW	rW	rW	rW
Bits 31: 9		Reserved, forced by hardware to 0													
Bits 8: 0		DR[8:0]: Data value Contains the Received or Transmitted data character, depending on whether it is read from or written to. The Data register performs a double function (read and write) since it is composed of two registers, one for transmission (TDR) and one for reception (RDR) The TDR register provides the parallel interface between the internal bus and the output shift register (see Figure 1). The RDR register provides the parallel interface between the input shift register and the internal bus. When transmitting with the parity enabled (PCE bit set to 1 in the USART_CR1 register), the value written in the MSB (bit 7 or bit 8 depending on the data length) has no effect because it is replaced by the parity.													

	When receiving with the parity enabled, the value read in the MSB bit is the received parity bit.
--	---

20.6.3 Baud rate register (USART_BRR)

Note: The baud counters stop counting if the TE or RE bits are disabled respectively.

Address offset: 0x08

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIV_Mantissa[11:0]												DIV_Fraction[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31: 16		Reserved, forced by hardware to 0													
Bits 15:4		DIV_Mantissa[11:0]: mantissa of USARTDIV These 12 bits define the mantissa of the USART Divider (USARTDIV)													
Bits 3:0		DIV_Fraction[3:0]: fraction of USARTDIV These 4 bits define the fraction of the USART Divider (USARTDIV)													

20.6.4 Control register 1 (USART_CR1)

Address offset: 0x0C

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	UE	M	WAKE	PCE	PS	PEIE	TXE IE	TCIE	RXNE IE	IDLE IE	TE	RE	RWU	SBK	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits31: 14		Reserved, forced by hardware to 0													
Bit13		UE: USART enable When this bit is cleared the USART prescalers and outputs are stopped and the end of the current byte transfer in order to reduce power consumption. This bit is set and cleared by software. 0: USART prescaler and outputs disabled 1: USART enabled													
Bit12		M: Word length This bit determines the word length. It is set or cleared by software. 0: 1 Start bit, 8 Data bits, n Stop bit 1: 1 Start bit, 9 Data bits, n Stop bit Note: The M bit must not be modified during a data transfer (both transmission and reception)													
Bit11		WAKE: Wakeup method This bit determines the USART wakeup method, it is set or cleared by software. 0: Idle Line 1: Address Mark													
Bit10		PCE: Parity control enable This bit selects the hardware parity control (generation and detection). When the parity control is enabled, the computed parity is inserted at the MSB position (9th bit if M=1; 8th bit if M=0) and parity is checked on the received data. This bit is set and cleared by software. Once it is set, PCE is active after the current byte (in reception and in transmission). 0: Parity control disabled 1: Parity control enabled													
Bit9		PS: Parity selection This bit selects the odd or even parity when the parity generation/detection is enabled (PCE bit set). It is set and cleared by software. The parity will be selected after the current byte.													

	0: Even parity 1: Odd parity
Bit8	PEIE : PE interrupt enable This bit is set and cleared by software. 0: Interrupt is inhibited 1: A USART interrupt is generated whenever PE=1 in the USART_SR register
Bit7	TXEIE : TXE interrupt enable This bit is set and cleared by software. 0: Interrupt is inhibited 1: A USART interrupt is generated whenever TXE=1 in the USART_SR register
Bit6	TCIE : Transmission complete interrupt enable This bit is set and cleared by software. 0: Interrupt is inhibited 1: A USART interrupt is generated whenever TC=1 in the USART_SR register
Bit5	RXNEIE : RXNE interrupt enable This bit is set and cleared by software. 0: Interrupt is inhibited 1: A USART interrupt is generated whenever ORE=1 or RXNE=1 in the USART_SR register
Bit4	IDLEIE : IDLE interrupt enable This bit is set and cleared by software. 0: Interrupt is inhibited 1: A USART interrupt is generated whenever IDLE=1 in the USART_SR register
Bit3	TE : Transmitter enable This bit enables the transmitter. It is set and cleared by software. 0: Transmitter is disabled 1: Transmitter is enabled <i>Note: 1: During transmission, a “0” pulse on the TE bit (“0” followed by “1”) sends a preamble (idle line) after the current word, except in Smartcard mode. 2: When TE is set there is a 1-bit-time delay before the transmission starts.</i>
Bit2	RE : Receiver enable This bit enables the receiver. It is set and cleared by software. 0: Receiver is disabled 1: Receiver is enabled and begins searching for a start bit
Bit1	RWU : Receiver wakeup This bit determines if the USART is in mute mode or not. It is set and cleared by software and can be cleared by hardware when a wakeup sequence is recognized. 0: Receiver in active mode 1: Receiver in mute mode <i>Note: 1: Before selecting Mute mode (by setting the RWU bit) the USART must first receive a data byte, otherwise it cannot function in Mute mode with wakeup by Idle line detection. 2: In Address Mark Detection wakeup configuration (WAKE bit=1) the RWU bit cannot be modified by software while the RXNE bit is set.</i>
Bit0	SBK : Send break This bit set is used to send break characters. It can be set and cleared by software. It should be set by software, and will be reset by hardware during the stop bit of break. 0: No break character is transmitted 1: Break character will be transmitted

20.6.5 Control register 2 (USART_CR2)

Address offset: 0x10

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	LINEN	STOP[1:0]	CLKEN	CPOL	CPHA	LBCL	Res	LBDIE	LBDL	Res	ADD[3:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:15		Reserved, forced by hardware to 0.													
Bit 14		LINEN: LIN mode enable This bit is set and cleared by software. 0: LIN mode disabled 1: LIN mode enabled The LIN mode enables the capability to send LIN Synch Breaks (13 low bits) using the SBK bit in the USART_CR1 register, and to detect LIN Sync breaks.													
Bits 13:12		STOP: STOP bits These bits are used for programming the stop bits. 00: 1 Stop bit 01: 0.5 Stop bit 10: 2 Stop bits 11: 1.5 Stop bit The 0.5 Stop bit and 1.5 Stop bit are not available for UART4 & UART5.													
Bit 11		CLKEN: Clock enable This bit allows the user to enable the CK pin. 0: CK pin disabled 1: CK pin enabled This bit is not available for UART4 & UART5.													
Bit 10		CPOL: Clock polarity This bit allows the user to select the polarity of the clock output on the CK pin in synchronous mode. It works in conjunction with the CPHA bit to produce the desired clock/data relationship 0: Steady low value on CK pin outside transmission window. 1: Steady high value on CK pin outside transmission window. This bit is not available for UART4 & UART5.													
Bit 9		CPHA: Clock phase This bit allows the user to select the phase of the clock output on the CK pin in synchronous mode. It works in conjunction with the CPOL bit to produce the desired clock/data relationship (see Figure 182 to Figure 183) 0: The first clock transition is the first data capture edge. 1: The second clock transition is the first data capture edge. This bit is not available for UART4 & UART5.													
Bit 8		LBCL: Last bit clock pulse This bit allows the user to select whether the clock pulse associated with the last data bit transmitted (MSB) has to be output on the CK pin in synchronous mode. 0: The clock pulse of the last data bit is not output to the CK pin 1: The clock pulse of the last data bit is output to the CK pin The last bit is the 8th or 9th data bit transmitted depending on the 8 or 9 bit format selected by the M bit in the USART_CR1 register. This bit is not available for UART4 & UART5.													
Bit 7		Reserved, forced by hardware to 0.													
Bit 6		LBDIE: LIN break detection interrupt enable Break interrupt mask (break detection using break delimiter). 0: Interrupt is inhibited 1: An interrupt is generated whenever LBD=1 in the USART_SR register													
Bit 5		LBDL: lin break detection length This bit is for selection between 11 bit or 10 bit break detection. 0: 10 bit break detection 1: 11 bit break detection													

Bit 4	Reserved, forced by hardware to 0
Bits 3:0	ADD[3:0] : Address of the USART node This bit-field gives the address of the USART node. This is used in multiprocessor communication during mute mode, for wake up with address mark detection.

20.6.6 Control register 3 (USART_CR3)

Address offset: 0x14

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved					CTSIE	CTSE	RTSE	DMAR	DMAR	SCEN	NACK	HDSEL	IRLP	IREN	EIE
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31: 11					Reserved, forced by hardware to 0.										
Bit 10					CTSIE : CTS interrupt enable 0: Interrupt is inhibited 1: An interrupt is generated whenever CTS=1 in the USART_SR register This bit is not available for UART4 & UART5.										
Bit 9					CTSE : CTS enable 0: CTS hardware flow control disabled 1: CTS mode enabled, data is only transmitted when the CTS input is asserted (tied to 0). If the CTS input is deasserted while a data is being transmitted, then the transmission is completed before stopping. If a data is written into the data register while CTS is deasserted, the transmission is postponed until CTS is asserted. This bit is not available for UART4 & UART5.										
Bit 8					RTSE : RTS enable 0: RTS hardware flow control disabled 1: RTS interrupt enabled, data is only requested when there is space in the receive buffer. The transmission of data is expected to cease after the current character has been transmitted. The RTS output is asserted (tied to 0) when a data can be received. This bit is not available for UART4 & UART5.										
Bit 7					DMAT : DMA enable transmitter This bit is set/reset by software 1: DMA mode is enabled for transmission 0: DMA mode is disabled for transmission This bit is not available for UART5.										
Bit 6					DMAR : DMA enable receiver This bit is set/reset by software 1: DMA mode is enabled for reception 0: DMA mode is disabled for reception This bit is not available for UART5.										
Bit 5					SCEN : Smartcard mode enable This bit is used for enabling Smartcard mode. 0: Smartcard Mode disabled 1: Smartcard Mode enabled This bit is not available for UART4 & UART5.										
Bit 4					NACK : Smartcard NACK enable 0: NACK transmission in case of parity error is disabled 1: NACK transmission during parity error is enabled This bit is not available for UART4 & UART5.										
Bit 3					HDSEL : Half-duplex selection Selection of Single-wire Half-duplex mode 0: Half duplex mode is not selected 1: Half duplex mode is selected										
Bit 2					IRLP : IrDA low-power										

	This bit is used for selecting between normal and low-power IrDA modes 0: Normal mode 1: Low-power mode
Bit 1	IREN: IrDA mode enable This bit is set and cleared by software. 0: IrDA disabled 1: IrDA enabled
Bit 0	EIE: Error interrupt enable Error Interrupt Enable Bit is required to enable interrupt generation in case of a framing error, overrun error or noise error (FE=1 or ORE=1 or NE=1 in the USART_SR register) in case of Multi Buffer Communication (DMAR=1 in the USART_CR3 register). 0: Interrupt is inhibited 1: An interrupt is generated whenever DMAR=1 in the USART_CR3 register and FE=1 or ORE=1 or NE=1 in the USART_SR register.

20.6.7 Guard time and prescaler register (USART_GTPR)

Address offset: 0x18

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
GT[7:0]								PSC[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31:16			Reserved, forced by hardware to 0												
Bits 15:8			GT[7:0]: Guard time value This bit-field gives the Guard time value in terms of number of baud clocks. This is used in Smartcard mode. The Transmission Complete flag is set after this guard time value. This bit is not available for UART4 & UART5.												
Bits 7:0			PSC[7:0]: Prescaler value – In IrDA Low-power mode: PSC[7:0] = IrDA Low-Power Baud Rate Used for programming the prescaler for dividing the system clock to achieve the low-power frequency: The source clock is divided by the value given in the register (8 significant bits): 00000000: Reserved - do not program this value 00000001: divides the source clock by 1 00000010: divides the source clock by 2 – In normal IrDA mode: PSC must be set to 00000001. – In Smartcard mode: PSC[4:0]: Prescaler value Used for programming the prescaler for dividing the system clock to provide the smartcard clock. The value given in the register (5 significant bits) is multiplied by 2 to give the division factor of the source clock frequency: 00000: Reserved - do not program this value 00001: divides the source clock by 2 00010: divides the source clock by 4 00011: divides the source clock by 6 <i>Note: Bits [7:5] have no effect if Smartcard mode is used.</i> This bit is not available for UART4 & UART5.												

20.6.8 USART register map

The table below gives the USART register map and reset values.

Table 89 USART register map and reset values

offset	register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
000h	USART_SR	Reserved																						CTS	LBD	TXEIE	TC	RXNE	IDLE	ORE	NE	FE	PE
	Reset value																							0	0	1	1	0	0	0	0	0	0
004h	USART_DR	Reserved																						DR[8:0]									
	Reset value																							0	0	0	0	0	0	0	0	0	0
008h	USART_BRR	Reserved												DIV_Mantissa[15:4]										DIV_Fraction [3:0]									
	Reset value													0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
00Ch	USART_CR1	Reserved												UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK						
	Reset value													0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					
010h	USART_CR2	Reserved												LIEN	STOP [1:0]	CLKEN	CPOL	CPHA	LBCL	Reserved	LBDIE	LBDL	Reserved	ADD[3:0]									

	0	0	0	0
--	---	---	---	---

	0	0	0	0
--	---	---	---	---

21 Device electronic signature

The electronic signature is stored in the System memory area in the Flash memory module, and can be read using the JTAG/SWD or the CPU. It contains factory-programmed identification data that allow the user firmware or other external devices to automatically match its interface to the characteristics of the CKS32F103xx microcontroller.

21.1 Memory size registers

21.1.1 Flash size register

Base address: 0x1FFF F7E0

Read only = 0xFFFF where X is factory-programmed

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
F_SIZE															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 15: 0		F_SIZE: Flash memory size This field value indicates the Flash memory size of the device in Kbytes. Example: 0x0080 = 128 Kbytes.													

21.2 Unique device ID register (96 bits)

The unique device identifier is ideally suited:

- for use as serial numbers (for example USB string serial numbers or other end applications)
- for use as security keys in order to increase the security of code in Flash memory while using and combining this unique ID with software cryptographic primitives and protocols before programming the internal Flash memory
- to activate secure boot processes, etc.

The 96-bit unique device identifier provides a reference number which is unique for any device and in any context. These bits can never be altered by the user.

The 96-bit unique device identifier can also be read in single bytes/half-words/words in different ways and then be concatenated using a custom algorithm.

Base address: 0x1FFF F7E8

Address offset: 0x00

Read only = 0xFFFF where X is factory-programmed

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
U_ID[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 15: 0		U_ID[15:0]: 15:0 unique ID bits													

Address offset: 0x02

Read only = 0xFFFF where X is factory-programmed

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
U_ID[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 15: 0		U_ID(31:16): 31:16 unique ID bits This field value is also reserved for a future feature.													

Address offset: 0x04

Read only = 0xXXXX XXXX where X is factory-programmed

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
U_ID[63:48]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
U_ID[47:32]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 31: 0		U_ID(63:32): 63:32 unique ID bits													

Address offset: 0x08

Read only = 0xXXXX XXXX where X is factory-programmed

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
U_ID[95:80]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
U_ID[79:64]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
Bits 31: 0		U_ID(95:64): 95:64 Unique ID bits.													

CKS copyright

22 Debug support (DBG)

22.1 Overview

The CKS32F103xx are built around a Cortex®-M3 core which contains hardware extensions for advanced debugging features. The debug extensions allow the core to be stopped either on a given instruction fetch (breakpoint) or data access (watchpoint). When stopped, the core's internal state and the system's external state may be examined. Once examination is complete, the core and the system may be restored and program execution resumed.

The debug features are used by the debugger host when connecting to and debugging the CKS32F103xx MCUs.

Two interfaces for debug are available:

- Serial wire
- JTAG debug port

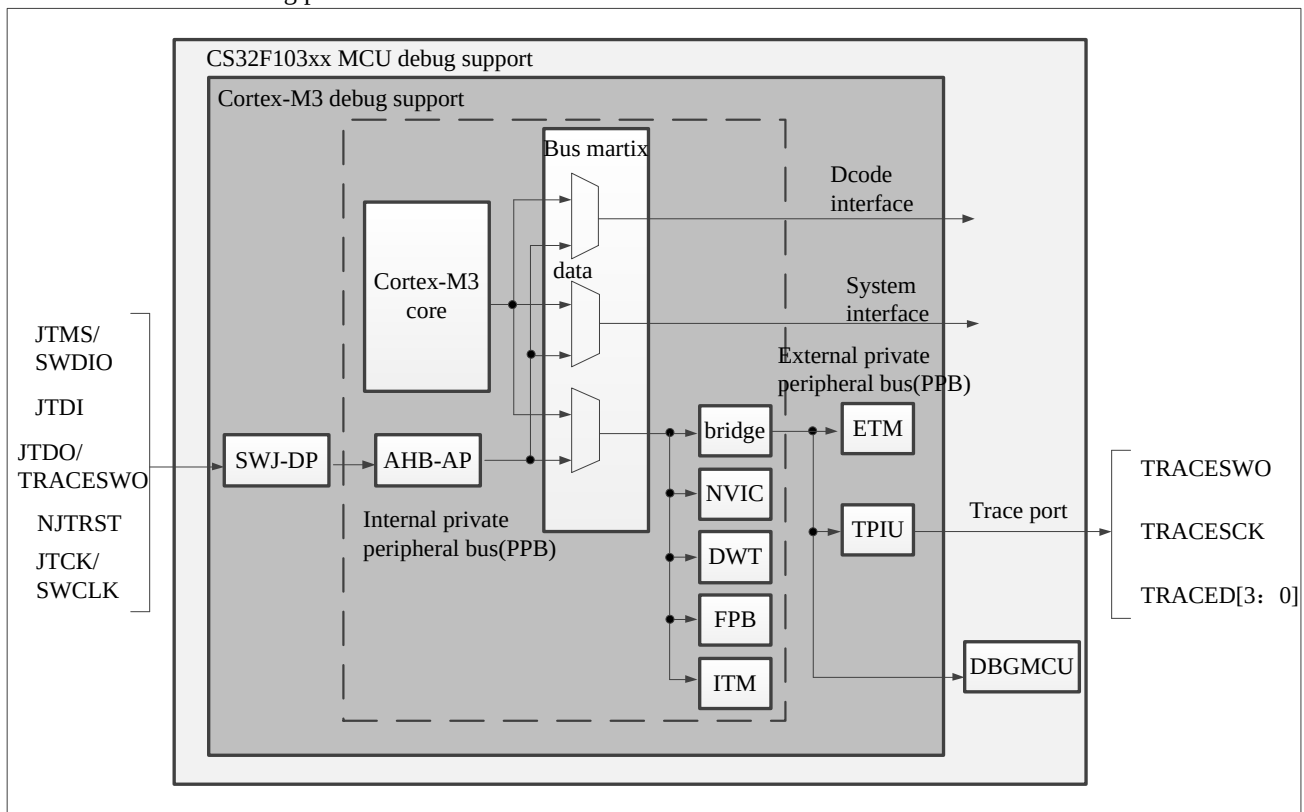


Figure 195 Block diagram of CKS32F103xx MCU and Cortex®-M3-level debug support

Note: The debug features embedded in the Cortex®-M3 core are a subset of the Arm® CoreSight Design Kit.

The Arm® Cortex®-M3 core provides integrated on-chip debug support. It is comprised of:

- SWJ-DP: Serial wire / JTAG debug port
- AHP-AP: AHB access port
- ITM: Instrumentation trace macrocell
- FPB: Flash patch breakpoint
- DWT: Data watchpoint trigger
- TPUI: Trace port unit interface (available on larger packages, where the corresponding pins are mapped)

- ETM: Embedded Trace Macrocell (available on larger packages, where the corresponding pins are mapped)

It also includes debug features dedicated to the CKS32F103xx:

- Flexible debug pinout assignment
- MCU debug box (support for low-power modes, control over peripheral clocks, etc.)

Note: For further information on debug functionality supported by the Arm® Cortex®-M3 core, refer to the Cortex®-M3 -r1p1 Technical Reference Manual and to the CoreSight Design Kit-r1p0 TRM

22.2 Reference Arm® documentation

- Cortex®-M3 r1p1 Technical Reference Manual (TRM)
It is available from: <http://infocenter.arm.com/>
- Arm® Debug Interface V5
- Arm® CoreSight Design Kit revision r1p1 Technical Reference Manual

22.3 SWJ debug port (serial wire and JTAG)

The core of the CKS32F103xx integrates the Serial Wire / JTAG Debug Port (SWJ-DP). It is an Arm® standard CoreSight debug port that combines a JTAG-DP (5-pin) interface and a SW-DP (2-pin) interface.

- The JTAG Debug Port (JTAG-DP) provides a 5-pin standard JTAG interface to the AHP-AP port.
 - The Serial Wire Debug Port (SW-DP) provides a 2-pin (clock + data) interface to the AHP-AP port.
- In the SWJ-DP, the two JTAG pins of the SW-DP are multiplexed with some of the five JTAG pins of the JTAG-DP.

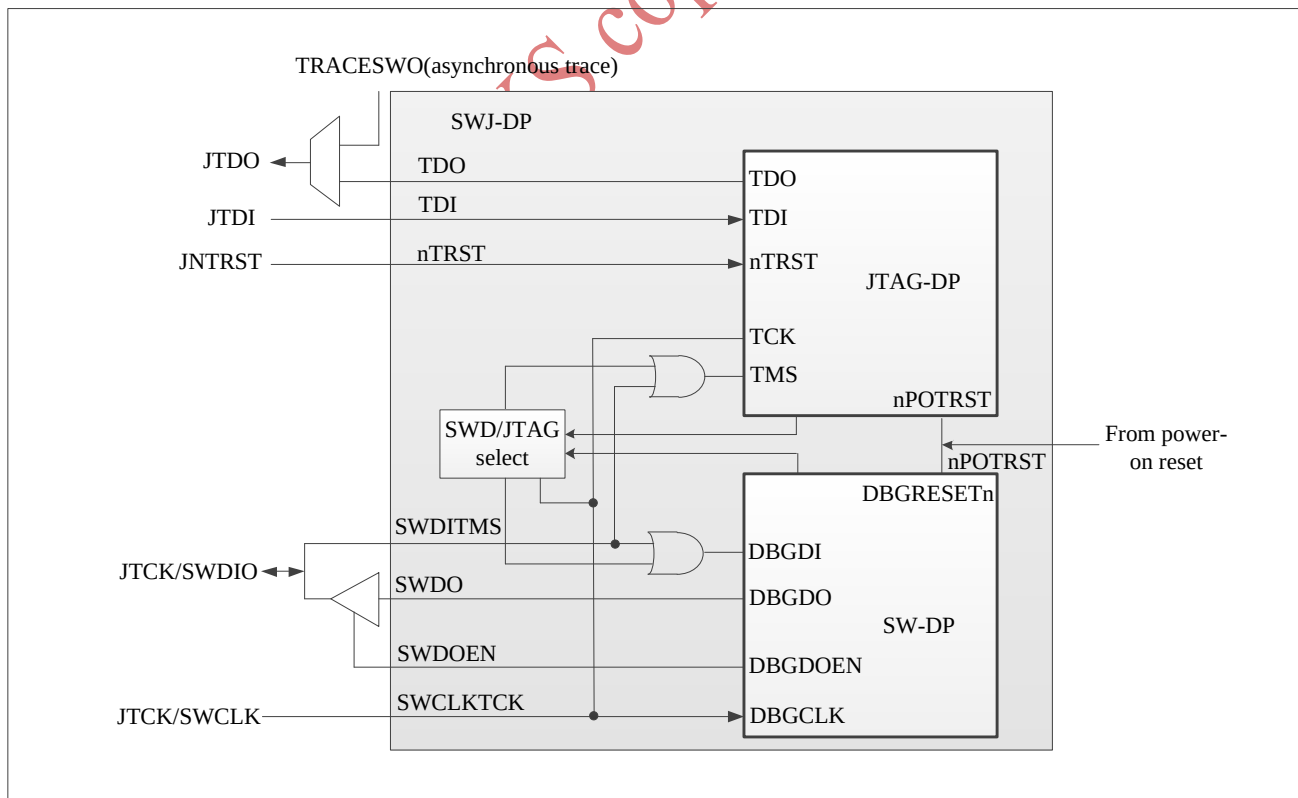


Figure 196 SWJ debug port

Figure 196 shows that the asynchronous TRACE output (TRACESWO) is multiplexed with TDO. This means that the asynchronous trace can only be used with SW-DP, not JTAG-DP.

22.3.1 Mechanism to select the JTAG-DP or the SW-DP

By default, the JTAG-Debug Port is active.

If the debugger host wants to switch to the SW-DP, it must provide a dedicated JTAG sequence on TMS/TCK (respectively mapped to SWDIO and SWCLK) which disables the JTAG-DP and enables the SW-DP. This way it is possible to activate the SWDP using only the SWCLK and SWDIO pins. This sequence is:

1. Send more than 50 TCK cycles with TMS (SWDIO) =1
2. Send the 16-bit sequence on TMS (SWDIO) = 0111100111100111 (MSB transmitted first)
3. Send more than 50 TCK cycles with TMS (SWDIO) =1

22.4 Pinout and debug port pins

The CKS32F103xx MCUs are available in various packages with different numbers of available pins. As a result, some functionality (ETM) related to pin availability may differ between packages.

22.4.1 SWJ debug port pins

Five pins are used as outputs from the CKS32F103xx for the SWJ-DP as *alternate functions* of general-purpose I/Os. These pins are available on all packages.

Table 90 SWJ debug port pins

SWJ-DP pin name	JTAG debug port		SW debug port		Pin assignment
	Type	Description	Type	Debug assignment	
JTMS/SWDIO	I	JTAG Test Mode Selection	IO	Serial Wire Data Input/Output	PA13
JTCK/SWCLK	I	JTAG Test Clock	I	Serial Wire Clock	PA14
JTDI	I	JTAG Test Data Input	-	-	PA15
JTDO/TRACESWO	O	JTAG Test Data Output	-	TRACESWO if async trace is enabled	PB3
JNTRST	I	JTAG Test nReset	-	-	PB4

22.4.2 Flexible SWJ-DP pin assignment

After RESET (SYSRESETn or PORESETn), all five pins used for the SWJ-DP are assigned as dedicated pins immediately usable by the debugger host (note that the trace outputs are not assigned except if explicitly programmed by the debugger host).

However, the CKS32F10xxx MCU implements the *AF remap and debug I/O configuration register* (AFIO_MAPR) register to disable some part or all of the SWJ-DP port and so releases the associated pins for General Purpose I/Os usage. This register is mapped on an APB bridge connected to the Cortex®-M3 System Bus. Programming of this register is done by the user software program and not the debugger host.

Three control bits allow the configuration of the SWJ-DP pin assignments. These bits are reset by the System Reset.

- AFIO_MAPR (@ 0x40010004 in the CKS32F10xxx MCU)
 - READ: APB - No Wait State
 - WRITE: APB - 1 Wait State if the write buffer of the AHB-APB bridge is full.

Bit 26:24= **SWJ_CFG[2:0]**

Set and cleared by software.

These bits are used to configure the number of pins assigned to the SWJ debug port. The goal is to release as much as possible the number of pins to be used as General Purpose I/Os if using a small size for the debug port.

The default state after reset is “000” (whole pins assigned for a full JTAG-DP connection). Only one of the 3 bits can be set (it is forbidden to set more than one bit).

Table 91 Flexible SWJ-DP pin assignment

SWJ- CFG[2:0]	Available debug ports	SWJ IO pin assigned				
		PA13/ JTMS/ SWDIO	PA14/ JTCK/ SWCLK	PA15/ JTDI	PB3/ JTDO	PB4/ JNTRST
000	Full SWJ (JTAG-DP + SW-DP) - Reset State	X	X	X	X	X
001	Full SWJ (JTAG-DP + SW-DP) but without NJTRST	X	X	X	X	Released
010	JTAG-DP Disabled and SW-DP Enabled	X	X	Released		
100	JTAG-DP Disabled and SW-DP Disabled	Released				
其他	Disabled					

Note: When the APB bridge write buffer is full, it takes one extra APB cycle when writing the AFIO_MAPR register. This is because the deactivation of the JTAGSW pins is done in two cycles to guarantee a clean level on the nTRST and TCK input signals of the core.

- Cycle 1: the JTAGSW input signals to the core are tied to 1 or 0 (to 1 for nTRST, TDI and TMS, to 0 for TCK)
- Cycle 2: the GPIO controller takes the control signals of the SWJTAG IO pins (like controls of direction, pull-up/down, Schmitt trigger activation, etc.).

22.4.3 Internal pull-up and pull-down on JTAG pins

It is necessary to ensure that the JTAG input pins are not floating since they are directly connected to flip-flops to control the debug mode features. Special care must be taken with the SWCLK/TCK pin which is directly connected to the clock of some of these flip-flops.

To avoid any uncontrolled IO levels, the device embeds internal pull-ups and pull-downs on the JTAG input pins:

- JNTRST: Internal pull-up
- JTDI: Internal pull-up
- JTMS/SWDIO: Internal pull-up
- TCK/SWCLK: Internal pull-down

Once a JTAG IO is released by the user software, the GPIO controller takes control again. The reset states of the GPIO control registers put the I/Os in the equivalent state:

- JNTRST: Input pull-up
- JTDI: Input pull-up
- JTMS/SWDIO: Input pull-down
- JICK/SWCLK: Input pull-down
- JTDO: AF output floating Input floating

The software can then use these I/Os as standard GPIOs.

Note: The JTAG IEEE standard recommends to add pull-ups on TDI, TMS and nTRST but there is no special recommendation for TCK. However, for JTCK, the device needs an integrated pull-down.

Having embedded pull-ups and pull-downs removes the need to add external resistors.

22.4.4 Using serial wire and releasing the unused debug pins as GPIOs

To use the serial wire DP to release some GPIOs, the user software must set SWJ_CFG=010 just after reset. This releases PA15, PB3 and PB4 which now become available as GPIOs.

When debugging, the host performs the following actions:

- Under system reset, all SWJ pins are assigned (JTAG-DP + SW-DP).
- Under system reset, the debugger host sends the JTAG sequence to switch from the JTAG-DP to the SW-DP.
- Still under system reset, the debugger sets a breakpoint on vector reset.
- The system reset is released and the Core halts.
- All the debug communications from this point are done using the SW-DP. The other JTAG pins can then be reassigned as GPIOs by the user software.

Note: For user software designs, note that:

To release the debug pins, remember that they will be first configured either in input-pull-up (nTRST, TMS, TDI) or pull-down (TCK) or output tristate (TDO) for a certain duration after reset until the instant when the user software releases the pins.

When debug pins (JTAG or SW or TRACE) are mapped, changing the corresponding IO pin configuration in the IOPORT controller has no effect.

22.5 CKS32F103xx JTAG TAP connection

The CKS32F103xx MCUs integrate two serially connected JTAG TAPs, the boundary scan TAP (IR is 5-bit wide) and the Cortex®-M3 TAP (IR is 4-bit wide).

To access the TAP of the Cortex®-M3 for debug purposes:

1. First, it is necessary to shift the BYPASS instruction of the boundary scan TAP.
2. Then, for each IR shift, the scan chain contains 9 bits (=5+4) and the unused TAP instruction must be shifted in using the BYPASS instruction.
3. For each data shift, the unused TAP, which is in BYPASS mode, adds 1 extra data bit in the data scan chain.

Note: **Important:** Once Serial-Wire is selected using the dedicated Arm® JTAG sequence, the boundary scan TAP is automatically disabled (JTMS forced high).

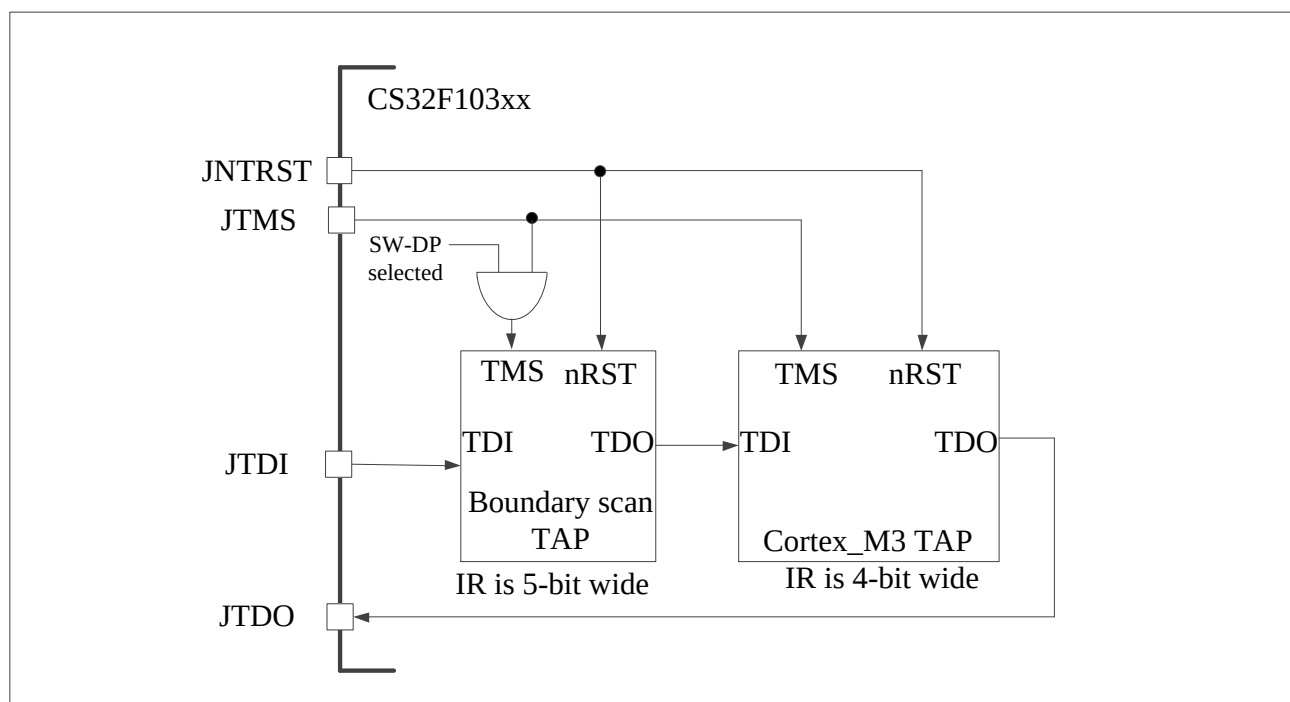


Figure 197 JTAG TAP connections

22.6 ID codes and locking mechanism

There are several ID codes inside the CKS32F103xx MCUs. It is recommended tools designers to lock their debuggers using the MCU DEVICE ID code located in the external PPB memory map at address 0xE0042000.

22.6.1 MCU device ID code

The CKS32F10xxx MCUs integrate an MCU ID code. This ID identifies the ST MCU part- number and the die revision. It is part of the DBG_MCU component and is mapped on the external PPB bus (see Section 22.16). This code is accessible using the JTAG debug port (four to five pins) or the SW debug port (two pins) or by the user software. It is even accessible while the MCU is under system reset.

Only the DEV_ID[11:0] should be used for identification by the debugger/programmer tools.

DBGMCU_IDCODE

Address: 0xE004 2000

Only 32-bits access supported. Read-only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
REV_ID															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DEV_ID											
r r r r r r r r r r r r r															
Bits 31: 16		REV_ID[15: 0]: Revision identifier 0x2003 = Revision 1													
Bits 15: 12		Reserved, initial value is 6													
Bits 11 : 0		DEV_ID[11:0]: Device identifier This field indicates the device ID. the device ID is 0x410													

22.6.2 Boundary scan TAP

JTAG ID code

The TAP of the CKS32F103xx BSC (boundary scan) integrates a JTAG ID code equal to

- 0x06410041 = Revision A
- 0x16410041 = Revision B and Revision Z

22.6.3 Cortex®-M3 TAP

The TAP of the Arm® Cortex®-M3 integrates a JTAG ID code. This ID code is the Arm® default one and has not been modified. This code is only accessible by the JTAG Debug Port. This code is 0x3BA00477 (corresponds to Cortex®-M3 r1p1-01rel0).

22.6.4 Cortex-M3 JEDEC-106 ID code

The Arm® Cortex®-M3 integrates a JEDEC-106 ID code. It is located in the 4KB ROM table mapped on the internal PPB bus at address 0xE00FF000_0xE00FFFFF.

22.7 JTAG debug port

A standard JTAG state machine is implemented with a 4-bit instruction register (IR) and five data registers (for full details, refer to the Cortex®-M3 r1p1 *Technical Reference Manual (TRM)*).

Table 92 JTAG debug port data registers

IR(3:0)	Data register	Details
1111	BYPASS[1 bit]	
1110	IDCODE[32 bits]	ID CODE 0x4BA00477(ARM Cortex-M3 r2p1-01rel0 ID Code)
1010	DPACC [32 bits]	Debug port access register This initiates a debug port and allows access to a debug port register. - When transferring data IN: Bits34:3=DATA[31:0]: 32-bit data to transfer for a write request Bits2:1=A[3:2]: 2-bit address of a debug port register. Bit0=RnW: Read request (1) or write request (0). - When transferring data OUT: Bits34:3=DATA[31:0]: 32-bit data which is read following a read request Bits2:0=ACK[2:0]: 3-bit Acknowledge: 010=OK/FAULT 001=WAIT OTHER = reserved Refer to Table 93 for a description of the A[3:2] bits
1011	APACC [35 bits]	Access port access register Initiates an access port and allows access to an access port register. - When transferring data IN: Bits34:3=DATA[31:0]: 32-bit data to shift in for a write request Bits2:1=A[3:2]: 2-bit address (sub-address AP registers). Bit0=RnW: Read request (1) or write request (0). - When transferring data OUT: Bits34:3=DATA[31:0]: 32-bit data which is read following a read request Bits2:0=ACK[2:0]: 3-bit Acknowledge 010=OK/FAULT

		001= WAIT OTHER = reserved There are many AP Registers (see AHB-AP) addressed as the combination of: - The shifted value A[3:2] - The current value of the DP SELECT register
1000	ABORT [35 bits]	Abort register - Bits 31:1 Reserved - Bit 0=DAPABORT: write 1 to generate a DAP abort.

Table 93 32-bit debug port registers addressed through the shifted value A[3:2]

Address	A(3:2) value	Description
0x0	00	Reserved, must be kept at reset value.
0x4	01	DP CTRL/STAT register. Used to: - Request a system or debug power-up - Configure the transfer operation for AP accesses - Control the pushed compare and pushed verify operations. - Read some status flags (overrun, power-up acknowledges)
0x8	10	DP SELECT register: Used to select the current access port and the active 4-words register window. - Bits31:24: APSEL: select the current AP - Bits23:8: reserved - Bits7:4: APBANKSEL: select the active 4-words register window on the current AP - Bits3:0: reserved
0xC	11	DP RDBUFF register: Used to allow the debugger to get the final result after a sequence of operations (without requesting new JTAG-DP operation)

22.8 SW debug port

22.8.1 SW protocol introduction

This synchronous serial protocol uses two pins:

- SWCLK: clock from host to target
- SWDIO: bidirectional

The protocol allows two banks of registers (DPACC registers and APACC registers) to be read and written to.

Bits are transferred LSB-first on the wire.

For SWDIO bidirectional management, the line must be pulled-up on the board (100 K Ω recommended by Arm[®]).

Each time the direction of SWDIO changes in the protocol, a turnaround time is inserted where the line is not driven by the host nor the target. By default, this turnaround time is one bit time, however this can be adjusted by configuring the SWCLK frequency.

22.8.2 SW protocol sequence

Each sequence consist of three phases:

1. Packet request (8 bits) transmitted by the host
2. Acknowledge response (3 bits) transmitted by the target
3. Data transfer phase (33 bits) transmitted by the host or the target

Table 94 Packet request (8-bits)

Bit	Name	Description
0	Start	Must be "1"
1	APnDP	0: DP Access 1: AP Access

2	RnW	0: Write Request 1: Read Request
4:3	A(3:2)	Address field of the DP or AP registers (refer to Table 93)
5	Parity	Single bit parity of preceding bits
6	Stop	0
7	Park	Not driven by the host. Must be read as “1” by the target because of the pull-up

Refer to the Cortex[®]-M3 r1p1 *TRM* for a detailed description of DPACC and APACC registers. The packet request is always followed by the turnaround time (default 1 bit) where neither the host nor target drive the line.

Table 95 ACK response (3 bits)

Bit	Name	Description
0..2	ACK	001: FAULT 010: WAIT 100: OK

The ACK Response must be followed by a turnaround time only if it is a READ transaction or if a WAIT or FAULT acknowledge has been received.

Table 96 DATA transfer (33 bits)

Bit	Name	Description
0..31	WDATA/ RDATA	Write or Read data
32	Parity	Single parity of the 32 data bits

The DATA transfer must be followed by a turnaround time only if it is a READ transaction.

22.8.3 SW-DP state machine (reset, idle states, ID code)

The State Machine of the SW-DP has an internal ID code which identifies the SW-DP. It follows the JEP-106 standard. This ID code is the default Arm[®] one and is set to 0x1BA01477 (corresponding to Cortex[®]-M3 r1p1).

Note: Note that the SW-DP state machine is inactive until the target reads this ID code.

- The SW-DP state machine is in RESET STATE either after power-on reset, or after the DP has switched from JTAG to SWD or after the line is high for more than 50 cycles
- The SW-DP state machine is in IDLE STATE if the line is low for at least two cycles after RESET state.
- After RESET state, it is **mandatory** to first enter into an IDLE state AND to perform a READ access of the DP-SW ID CODE register. Otherwise, the target will issue a FAULT acknowledge response on another transactions.

Further details of the SW-DP state machine can be found in the Cortex[®]-M3 r1p1 *TRM* and the *CoreSight Design Kit r1p0 TRM*.

22.8.4 DP and AP read/write accesses

- Read accesses to the DP are not posted: the target response can be immediate (if ACK=OK) or can be delayed (if ACK=WAIT).
- Read accesses to the AP are posted. This means that the result of the access is returned on the next transfer. If the next access to be done is NOT an AP access, then the DP-RDBUFF register must be read to obtain the result.
- The READOK flag of the DP-CTRL/STAT register is updated on every AP read access or RDBUFF read request to know if the AP read access was successful.
- The SW-DP implements a write buffer (for both DP or AP writes), that enables it to accept a write operation even when other transactions are still outstanding. If the write buffer is full, the target acknowledge response is “WAIT”. With the exception of IDCODE read or CTRL/STAT read or ABORT write which are accepted even if the write buffer is full.

- Because of the asynchronous clock domains SWCLK and HCLK, two extra SWCLK cycles are needed after a write transaction (after the parity bit) to make the write effective internally. These cycles should be applied while driving the line low (IDLE state). This is particularly important when writing the CTRL/STAT for a power-up request. If the next transaction (requiring a power-up) occurs immediately, it will fail.

22.8.5 SW-DP registers

Access to these registers are initiated when APnDP=0

Table 97 SW-DP registers

A(3:2)	R/W	CTRLSEL bit of SELECT register	Register	Notes
00	Read		IDCODE	0x1BA01477 (identifies the SW-DP)
00	Write		ABORT	
01	Read / Write	0	DP-CTRL/STAT	- request a system or debug power-up - configure the transfer operation for AP accesses - control the pushed compare and pushed verify operations. - read some status flags (overrun, power-up acknowledges)
01	Read / Write	1	WIRE CONTROL	Purpose is to configure the physical serial port protocol (like the duration of the turnaround time)
10	Read		READ RESEND	Enables recovery of the read data from a corrupted debugger transfer, without repeating the original AP transfer.
10	Write		SELECT	The purpose is to select the current access port and the active 4-words register window
11	Read / Write		READ BUFFER	This read buffer is useful because AP accesses are posted (the result of a read AP request is available on the next AP transaction). This read buffer captures data from the AP, presented as the result of a previous read, without initiating a new transaction

22.8.6 SW-AP registers

Access to these registers are initiated when APnDP=1

There are many AP Registers (see AHB-AP) addressed as the combination of:

- The shifted value A[3:2]
- The current value of the DP SELECT register

22.9 AHB-AP (AHB access port) - valid for both JTAG-DP and SW-DP

Features:

- System access is independent of the processor status.
- Either SW-DP or JTAG-DP accesses AHB-AP.
- The AHB-AP is an AHB master into the Bus Matrix. Consequently, it can access all the data buses (Dcode Bus, System Bus, internal and external PPB bus) but the ICode bus
- Bitband transactions are supported.
- AHB-AP transactions bypass the FPB.

The address of the 32-bits AHP-AP registers are 6-bits wide (up to 64 words or 256 bytes) and consists of:

a) Bits [7:4] = the bits [7:4] APBANKSEL of the DP SELECT register

b) Bits [3:2] = the 2 address bits of A[3:2] of the 35-bit packet request for SW-DP.

The AHB-AP of the Cortex[®]-M3 includes 9 x 32-bits registers:

Table 98 Cortex[®]-M3 AHB-AP registers

Address offset	Register name	Notes
0x00	AHB-AP Control and Status Word	Configures and controls transfers through the AHB interface (size, hprot, status on current transfer, address increment type)
0x04	AHB-AP Transfer Address	
0x0C	AHB-AP Data Read/Write	
0x10	AHB-AP Banked Data 0	Directly maps the 4 aligned data words without rewriting the Transfer Address Register.
0x14	AHB-AP Banked Data 1	
0x18	AHB-AP Banked Data 2	
0x1C	AHB-AP Banked Data 3	
0xF8	AHB-AP Debug ROM Address	Base Address of the debug interface
0xFC	AHB-AP ID Register	

Refer to the Cortex[®]-M3 r1p1 TRM for further details.

22.10 Core debug

Core debug is accessed through the core debug registers. Debug access to these registers is by means of the *Advanced High-performance Bus* (AHB-AP) port. The processor can access these registers directly over the internal *Private Peripheral Bus* (PPB).

It consists of 4 registers:

Table 99 Core debug registers

Register	Description
DHCSR	The 32-bit Debug Halting Control and Status Register This provides status information about the state of the processor enable core debug halt and step the processor
DCRSR	The 17-bit Debug Core Register Selector Register: This selects the processor register to transfer data to or from.
DCRDR	The 32-bit Debug Core Register Data Register: This holds data for reading and writing registers to and from the processor selected by the DCRSR (Selector) register.
DEMCR	The 32-bit Debug Exception and Monitor Control Register: This provides Vector Catching and Debug Monitor Control. This register contains a bit named TRCENA which enable the use of a TRACE.

Note: **Important:** these registers are not reset by a system reset. They are only reset by a power-on reset.

Refer to the Cortex[®]-M3 r1p1 TRM for further details.

To Halt on reset, it is necessary to:

- enable the bit0 (VC_CORRESET) of the Debug and Exception Monitor Control Register
- enable the bit0 (C_DEBUGEN) of the Debug Halting Control and Status Register.

22.11 Capability of the debugger host to connect under system reset

The reset system of the CKS32F103xx MCU comprises the following reset sources:

- POR (power-on reset) which asserts a RESET at each power-up.
- Internal watchdog reset
- Software reset
- External reset

The Cortex[®]-M3 differentiates the reset of the debug part (generally PORRESETn) and the other one (SYSRESETn). This way, it is possible for the debugger to connect under System Reset,

programming the Core Debug Registers to halt the core when fetching the reset vector. Then the host can release the system reset and the core will immediately halt without having executed any instructions. In addition, it is possible to program any debug features under System Reset.

Note: It is highly recommended for the debugger host to connect (set a breakpoint in the reset vector) under system reset.

22.12 FPB (Flash patch breakpoint)

The FPB unit:

- implements hardware breakpoints
- patches code and data from code space to system space. This feature gives the possibility to correct software bugs located in the Code Memory Space.

The use of a Software Patch or a Hardware Breakpoint is exclusive.

The FPB consists of:

- 2 literal comparators for matching against literal loads from Code Space and remapping to a corresponding area in the System Space.
- 6 instruction comparators for matching against instruction fetches from Code Space. They can be used either to remap to a corresponding area in the System Space or to generate a Breakpoint Instruction to the core.

22.13 DWT (data watchpoint trigger)

The DWT unit consists of four comparators. They are configurable as:

- a hardware watchpoint or
- a trigger to an ETM or
- a PC sampler or
- a data address sampler

The DWT also provides some means to give some profiling informations. For this, some counters are accessible to give the number of:

- Clock cycle
- Folded instructions
- Load store unit (LSU) operations
- Sleep cycles
- CPI (clock per instructions)
- Interrupt overhead

22.14 ITM (instrumentation trace macrocell)

22.14.1 General description

The ITM is an application-driven trace source that supports *printf* style debugging to trace *Operating System* (OS) and application events, and emits diagnostic system information. The ITM emits trace information as packets which can be generated as:

- **Software trace.** Software can write directly to the ITM stimulus registers to emit packets.
- **Hardware trace.** The DWT generates these packets, and the ITM emits them.
- **Time stamping.** Timestamps are emitted relative to packets. The ITM contains a 21-bit counter to generate the timestamp. The Cortex[®]-M3 clock or the bit clock rate of the *Serial Wire Viewer* (SWV) output clocks the counter.

The packets emitted by the ITM are output to the TPIU (Trace Port Interface Unit). The formatter of the TPIU adds some extra packets (refer to TPIU) and then output the complete packets sequence to the debugger host.

The bit TRCEN of the Debug Exception and Monitor Control Register must be enabled before programming or using the ITM.

22.14.2 Time stamp packets, synchronization and overflow packets

Time stamp packets encode time stamp information, generic control and synchronization. It uses a 21-bit timestamp counter (with possible prescalers) which is reset at each time stamp packet emission. This counter can be either clocked by the CPU clock or the SWV clock.

A synchronization packet consists of 6 bytes equal to 0x80_00_00_00_00_00 which is emitted to the TPIU as 00 00 00 00 00 80 (LSB emitted first).

A synchronization packet is a timestamp packet control. It is emitted at each DWT trigger.

For this, the DWT must be configured to trigger the ITM: the bit CYCCNTENA (bit0) of the DWT Control Register must be set. In addition, the bit2 (SYNCENA) of the ITM Trace Control Register must be set.

Note: If the SYNENA bit is not set, the DWT generates Synchronization triggers to the TPIU which will send only TPIU synchronization packets and not ITM synchronization packets.

An overflow packet consists is a special timestamp packets which indicates that data has been written but the FIFO was full.

Table 100 Main ITM registers

Address	Register	Details
@E0000FB0	ITM Lock Access	Write 0xC5ACCE55 to unlock Write Access to the other ITM registers
@E0000E80	ITM Trace Control	Bits 31:24 = Always 0 Bits 23 = Busy Bits 22-16 = 7-bits ATB ID which identifies the source of the trace data. Bits 15-10 = Always 0 Bits 9:8 = TSPrescale = Time Stamp Prescaler Bits 7-5 = Reserved Bit 4 = SWOENA = Enable SWV behavior (to clock the timestamp counter by the SWV clock) Bit 3 = DWTENA: Enable the DWT Stimulus Bit 2 = SYNCENA: this bit must be to 1 to enable the DWT to generate synchronization triggers so that the TPIU can then emit the synchronization packets. Bit 1 = TSENA (Timestamp Enable) Bit 0 = ITMENA: Global Enable Bit of the ITM
@E0000E40	ITM Trace Privilege	Bit 3: mask to enable tracing ports31:24 Bit 2: mask to enable tracing ports23:16 Bit 1: mask to enable tracing ports15:8 Bit 0: mask to enable tracing ports7:0
@E0000E00	ITM Trace Enable	Each bit enables the corresponding Stimulus port to generate trace.
@E0000000— E000007C	Stimulus Port Registers 0-31	Write the 32-bits data on the selected Stimulus Port (32 available) to be traced out.

Example of configuration

To output a simple value to the TPIU:

- Configure the TPIU and assign TRACE I/Os by configuring the DBGMCU_CR (refer to Section 22.17.2: TRACE pin assignment and Section 22.16.3: Debug MCU configuration register)
- Write 0xC5ACCE55 to the ITM Lock Access Register to unlock the write access to the ITM registers
- Write 0x00010005 to the ITM Trace Control Register to enable the ITM with Sync enabled and an ATB ID different from 0x00

- Write 0x1 to the ITM Trace Enable Register to enable the Stimulus Port 0
- Write 0x1 to the ITM Trace Privilege Register to unmask stimulus ports 7:0
- Write the value to output in the Stimulus Port Register 0: this can be done by software (using a printf function)

22.15 ETM (Embedded trace macrocell)

22.15.1 ETM general description

The ETM enables the reconstruction of program execution. Data are traced using the Data Watchpoint and Trace (DWT) component or the Instruction Trace Macrocell (ITM) whereas instructions are traced using the Embedded Trace Macrocell (ETM).

The ETM transmits information as packets and is triggered by embedded resources. These resources must be programmed independently and the trigger source is selected using the Trigger Event Register (0xE0041008). An event could be a simple event (address match from an address comparator) or a logic equation between 2 events. The trigger source is one of the fourth comparators of the DWT module, The following events can be monitored:

- Clock cycle matching
- Data address matching

For more informations on the trigger resources refer to *Section 22.13: DWT (data watchpoint trigger)*.

The packets transmitted by the ETM are output to the TPIU (Trace Port Interface Unit). The formatter of the TPIU adds some extra packets (refer to *Section 22.17: TPIU (trace port interface unit)*) and then outputs the complete packet sequence to the debugger host.

22.15.2 ETM signal protocol and packet types

This part is described in the chapter 7 ETMv3 Signal Protocol of the Arm® IHI 0014N document.

22.15.3 Main ETM registers

For more information on registers refer to the chapter 3 of the Arm® IHI 0014N specification.

Table 101 Main ETM registers

Address	Register	Details
0xE0041FB0	ETM Lock Access	Write 0xC5ACCE55 to unlock the write access to the other ETM registers.
0xE0041000	ETM Control	This register controls the general operation of the ETM, for instance how tracing is enabled.
0xE0041010	ETM Status	This register provides information about the current status of the trace and trigger logic.
0xE0041008	ETM Trigger Event	This register defines the event that will control trigger.
0xE004101C	ETM Trace Enable Control	This register defines which comparator is selected.
0xE0041020	ETM Trace Enable Event	This register defines the trace enabling event.
0xE0041024	ETM Trace Start/Stop	This register defines the traces used by the trigger source to start and stop the trace, respectively.

22.15.4 ETM configuration example

To output a simple value to the TPIU:

- Configure the TPIU and enable the I/O_TRACEN to assign TRACE I/Os in the XL- and high-density device debug configuration register.
- Write 0xC5AC CE55 to the ETM Lock Access Register to unlock the write access to the ITM registers
- Write 0x0000 1D1E to the ETM control register (configure the trace)
- Write 0x0000 406F to the ETM Trigger Event register (define the trigger event)
- Write 0x0000 006F to the ETM Trace Enable Event register (define an event to start/stop)
- Write 0x0000 0001 to the ETM Trace Start/stop register (enable the trace)
- Write 0x0000 191E to the ETM Control Register (end of configuration)

22.16 MCU debug component (DBGMCU)

The MCU debug component helps the debugger provide support for:

- Low-power modes
- Clock control for timers, watchdog, I2C and bxCAN during a breakpoint
- Control of the trace pins assignment

22.16.1 Debug support for low-power modes

To enter low-power mode, the instruction WFI or WFE must be executed.

The MCU implements several low-power modes which can either deactivate the CPU clock or reduce the power of the CPU.

The core does not allow FCLK or HCLK to be turned off during a debug session. As these are required for the debugger connection, during a debug, they must remain active. The MCU integrates special means to allow the user to debug software in low-power modes.

For this, the debugger host must first set some debug configuration registers to change the low-power mode behavior:

- In Sleep mode, DBG_SLEEP bit of DBGMCU_CR register must be previously set by the debugger. This will feed HCLK with the same clock that is provided to FCLK (system clock previously configured by the software).
- In Stop mode, the bit DBG_STOP must be previously set by the debugger. This will enable the internal RC oscillator clock to feed FCLK and HCLK in STOP mode.

22.16.2 Debug support for timers, watchdog, bxCAN and I²C

During a breakpoint, it is necessary to choose how the counter of timers and watchdog should behave:

- They can continue to count inside a breakpoint. This is usually required when a PWM is controlling a motor, for example
- They can stop to count inside a breakpoint. This is required for watchdog purposes.

For the bxCAN, the user can choose to block the update of the receive register during a breakpoint.

For the I²C, the user can choose to block the SMBUS timeout during a breakpoint.

22.16.3 Debug MCU configuration register

This register allows the configuration of the MCU under DEBUG. This concerns:

- Low-power mode support
- Timer and watchdog counter support
- bxCAN communication support
- Trace pin assignment

This DBGMCU_CR is mapped on the External PPB bus at address 0xE0042004

It is asynchronously reset by the PORESET (and not the system reset). It can be written by the debugger under system reset.

If the debugger host does not support these features, it is still possible for the user software to write to these registers.

DBGMCU_CR

Address: 0xE0042004

Only 32-bit access supported

POR Reset: 0x0000 0000 (not reset by system reset)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															DBG_I2C2_SMBUS_TIMEOUT
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DBG_I2C1_SMBUS_TIMEOUT	DBG_CAN1_STOP	DBG_TIM4_STOP	DBG_TIM3_STOP	DBG_TIM2_STOP	DBG_TIM1_STOP	DBG_WWDG_STOP	DBG_IWDG_STOP	TRACE_MODE [1:0]	TRACE_IOEN	Reserved	DBG_STANDBY	DBG_STOP	DBG_SLEEP		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
Bits 31: 17		Reserved, must be kept at reset value.													
Bit16		DBG_I2C2_SMBUS_TIMEOUT: SMBUS timeout mode stopped when Core is halted 0: Same behavior as in normal mode 1: The SMBUS timeout is frozen													
Bit15		DBG_I2C1_SMBUS_TIMEOUT: SMBUS timeout mode stopped when Core is halted 0: Same behavior as in normal mode 1: The SMBUS timeout is frozen													
Bit14		DBG_CAN1_STOP: Debug CAN1 stopped when Core is halted 0: Same behavior as in normal mode 1: CAN1 receive registers are frozen													
Bits 13:10		DBG_TIMx_STOP: TIMx counter stopped when core is halted (x=4..1) 0: The clock of the involved Timer Counter is fed even if the core is halted 1: The clock of the involved Timer counter is stopped when the core is halted													
Bit 9		DBG_WWDG_STOP: Debug window watchdog stopped when core is halted 0: The window watchdog counter clock continues even if the core is halted 1: The window watchdog counter clock is stopped when the core is halted													
Bit 8		DBG_IWDG_STOP: Debug independent watchdog stopped when core is halted 0: The watchdog counter clock continues even if the core is halted 1: The watchdog counter clock is stopped when the core is halted													
Bits 7:5		TRACE_MODE[1:0] and TRACE_IOEN: Trace pin assignment control - With TRACE_IOEN=0: TRACE_MODE=xx: TRACE pins not assigned (default state) - With TRACE_IOEN=1: TRACE_MODE=00: TRACE pin assignment for Asynchronous Mode TRACE_MODE=01: TRACE pin assignment for Synchronous Mode with a TRACEDATA size of 1 TRACE_MODE=10: TRACE pin assignment for Synchronous Mode with a TRACEDATA size of 2 TRACE_MODE=11: TRACE pin assignment for Synchronous Mode with a TRACEDATA size of 4													
Bits 4:3		Reserved, must be kept at reset value.													

Bit2	DBG_STANDBY: Debug Standby mode 0: (FCLK=Off, HCLK=Off) The whole digital part is unpowered. From software point of view, exiting from Standby is identical than fetching reset vector (except a few status bit indicated that the MCU is resuming from Standby) 1: (FCLK=On, HCLK=On) In this case, the digital part is not unpowered and FCLK and HCLK are provided by the internal RC oscillator which remains active. In addition, the MCU generate a system reset during Standby mode so that exiting from Standby is identical than fetching from reset
Bit1	DBG_STOP: Debug Stop mode 0: (FCLK=Off, HCLK=Off) In STOP mode, the clock controller disables all clocks (including HCLK and FCLK). When exiting from STOP mode, the clock configuration is identical to the one after RESET (CPU clocked by the 8 MHz internal RC oscillator (HSI)). Consequently, the software must reprogram the clock controller to enable the PLL, the Xtal, etc. 1: (FCLK=On, HCLK=On) In this case, when entering STOP mode, FCLK and HCLK are provided by the internal RC oscillator which remains active in STOP mode. When exiting STOP mode, the software must reprogram the clock controller to enable the PLL, the Xtal, etc. (in the same way it would do in case of DBG_STOP=0)
Bit0	DBG_SLEEP: Debug Sleep mode 0: (FCLK=On, HCLK=Off) In Sleep mode, FCLK is clocked by the system clock as previously configured by the software while HCLK is disabled. In Sleep mode, the clock controller configuration is not reset and remains in the previously programmed state. Consequently, when exiting from Sleep mode, the software does not need to reconfigure the clock controller. 1: (FCLK=On, HCLK=On) In this case, when entering Sleep mode, HCLK is fed by the same clock that is provided to FCLK (system clock as previously configured by the software).

22.17 TPIU (trace port interface unit)

22.17.1 Introduction

The TPIU acts as a bridge between the on-chip trace data from the ITM and the ETM.
The output data stream encapsulates the trace source ID, that is then captured by a *trace port analyzer* (TPA).
The core embeds a simple TPIU, especially designed for low-cost debug (consisting of a special version of the CoreSight TPIU).

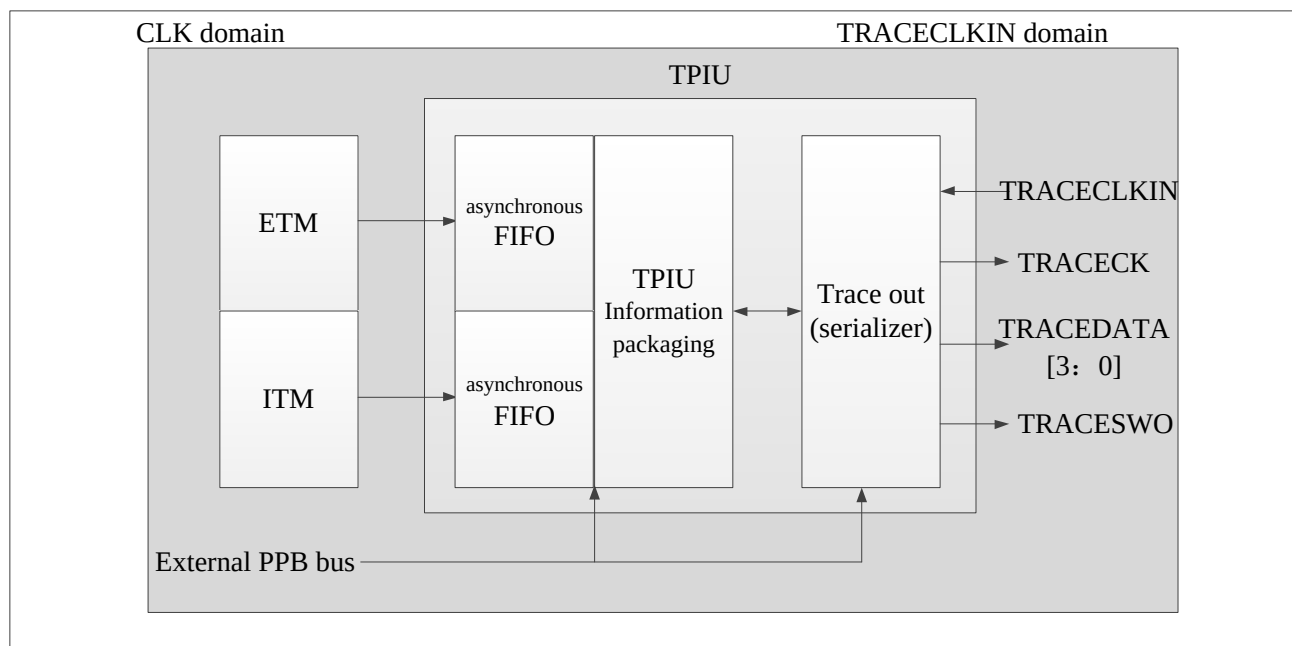


Figure 198 TPIU block diagram

22.17.2 TRACE pin assignment

- Asynchronous mode

The asynchronous mode requires 1 extra pin and is available on all packages. It is only available if using Serial Wire mode (not in JTAG mode).

Table 102 Asynchronous TRACE pin assignment

TPUI pin name	Trace synchronous mode		CKS32F103xx pin assignment
	Type	Description	
TRACESWO	O	TRACE Async Data Output	PB3

- Synchronous mode

The synchronous mode requires from 2 to 6 extra pins depending on the data trace size and is only available in the larger packages. In addition it is available in JTAG mode and in Serial Wire mode and provides better bandwidth output capabilities than asynchronous trace.

Table 103 Synchronous TRACE pin assignment

TPUI pin name	Trace synchronous mode		CKS32F103xxpin assignment
	Type	Description	
TRACECK	O	TRACE Clock	PE2
TRACED[3:0]	O	TRACE Sync Data Outputs Can be 1, 2 or 4.	PE[6:3]

TPUI TRACE pin assignment

By default, these pins are NOT assigned. They can be assigned by setting the TRACE_IOEN and TRACE_MODE bits in the **MCU Debug component configuration register**. This configuration has to be done by the debugger host.

In addition, the number of pins to assign depends on the trace configuration (asynchronous or synchronous).

- Asynchronous mode:** 1 extra pin is needed

- **Synchronous mode:** from 2 to 5 extra pins are needed depending on the size of the data trace port register (1, 2 or 4):
 - TRACECK
 - TRACED(0) if port size is configured to 1, 2 or 4
 - TRACED(1) if port size is configured to 2 or 4
 - TRACED(2) if port size is configured to 4
 - TRACED(3) if port size is configured to 4

To assign the TRACE pin, the debugger host must program the bits TRACE_IOEN and TRACE_MODE[1:0] of the Debug MCU configuration Register (DBGMCU_CR). By default the TRACE pins are not assigned.

This register is mapped on the external PPB and is reset by the PORESET (and not by the SYSTEM reset). It can be written by the debugger under SYSTEM reset.

Table 104 Flexible TRACE pin assignment

DBGMCU_CR register		Pins assigned for:	TRACE IO pin assigned					
TRACE_IOEN	TRACE_MODE[1:0]		PB3/JTDO/TRACES WO	PE2/TRACE CK	PE3/TRACE D[0]	PE4/TRACE D[1]	PE5/TRACE D[2]	PE6/TRACE D[3]
0	XX	No Trace (default state)	Released ⁽¹⁾	Released (usable as GPIO)				
1	00	Asynchronous Trace	TRACES WO					
1	01	Synchronous Trace 1 bit	Released ⁽¹⁾	TRACE CK	TRACE D[0]	Released (usable as GPIO)		
1	10	Synchronous Trace 2 bit		TRACE CK	TRACE D[0]	TRACE D[1]	Released (usable as GPIO)	
1	11	Synchronous Trace 4 bit		TRACE CK	TRACE D[0]	TRACE D[1]	TRACE D[2]	TRACE D[3]

Note: When Serial Wire mode is used, it is released. But when JTAG is used, it is assigned to JTDO

Note: By default, the TRACECLKIN input clock of the TPIU is tied to GND. It is assigned to HCLK two clock cycles after the bit TRACE_IOEN has been set.

The debugger must then program the Trace Mode by writing the PROTOCOL[1:0] bits in the SPP_R (Selected Pin Protocol) register of the TPIU.

- PROTOCOL=00: Trace Port Mode (synchronous)
 - PROTOCOL=01 or 10: Serial Wire (Manchester or NRZ) Mode (asynchronous mode). Default state is 01
- It then also configures the TRACE port size by writing the bits [3:0] in the CPSPS_R (Current Sync Port Size Register) of the TPIU:
- 0x1 for 1 pin (default state)
 - 0x2 for 2 pins
 - 0x8 for 4 pins

22.17.3 TPUI formatter

The formatter protocol outputs data in 16-byte frames:

- seven bytes of data
- eight bytes of mixed-use bytes consisting of:
 - 1 bit (LSB) to indicate it is a DATA byte ('0') or an ID byte ('1').
 - 7 bits (MSB) which can be data or change of source ID trace.
- one byte of auxiliary bits where each bit corresponds to one of the eight mixed-use bytes:
 - if the corresponding byte was a data, this bit gives bit0 of the data.
 - if the corresponding byte was an ID change, this bit indicates when that ID change takes effect.

Note: Refer to the Arm® CoreSight Architecture Specification v1.0 (Arm® IHI 0029B) for further information

22.17.4 TPUI frame synchronization packets

The TPUI can generate two types of synchronization packets:

- The Frame Synchronization packet (or Full Word Synchronization packet)
It consists of the word: 0x7F_FF_FF_FF (LSB emitted first). This sequence can not occur at any other time provided that the ID source code 0x7F has not been used.
It is output periodically **between** frames.
In continuous mode, the TPA must discard all these frames once a synchronization frame has been found.
- The Half-Word Synchronization packet
It consists of the half word: 0x7F_FF (LSB emitted first).
It is output periodically **between or within** frames.
These packets are only generated in continuous mode and enable the TPA to detect that the TRACE port is in IDLE mode (no TRACE to be captured). When detected by the TPA, it must be discarded.

22.17.5 Transmission of the synchronization frame packet

There is no Synchronization Counter register implemented in the TPIU of the core. Consequently, the synchronization trigger can only be generated by the **DWT**. Refer to the registers DWT Control Register (bits SYNCTAP[11:10]) and the DWT Current PC Sampler Cycle Count Register.

The TPUI Frame synchronization packet (0x7F_FF_FF_FF) is emitted:

- after each TPIU reset release. This reset is synchronously released with the rising edge of the TRACECLKIN clock. This means that this packet is transmitted when the TRACE_IOEN bit in the DBGMCU_CFG register is set. In this case, the word 0x7F_FF_FF_FF is not followed by any formatted packet.
- at each DWT trigger (assuming DWT has been previously configured). Two cases occur:
 - If the bit SYNENA of the ITM is reset, only the word 0x7F_FF_FF_FF is emitted without any formatted stream which follows.
 - If the bit SYNENA of the ITM is set, then the ITM synchronization packets will follow (0x80_00_00_00_00_00), formatted by the TPUI (trace source ID added).

22.17.6 Synchronous mode

The trace data output size can be configured to 4, 2 or 1 pin: TRACED(3:0)

The output clock is output to the debugger (TRACECK)

Here, TRACECLKIN is driven internally and is connected to HCLK only when TRACE is used.

Note: In this synchronous mode, it is not required to provide a stable clock frequency.

The TRACE I/Os (including TRACECK) are driven by the rising edge of TRACCLKIN (equal to HCLK). Consequently, the output frequency of TRACECK is equal to HCLK/2.

22.17.7 Asynchronous mode

This is a low cost alternative to output the trace using only 1 pin: this is the asynchronous output pin TRACESWO. Obviously there is a limited bandwidth.

TRACESWO is multiplexed with JTDO when using the SW-DP pin. This way, this functionality is available in all CKS32F10xxx packages.

This asynchronous mode requires a constant frequency for TRACECLKIN. For the standard UART (NRZ) capture mechanism, 5% accuracy is needed. The Manchester encoded version is tolerant up to 10%.

22.17.8 TRACECLKIN connection inside the CKS32F103xx

In the CKS32F103xx, this TRACECLKIN input is internally connected to HCLK. This means that when in asynchronous trace mode, the application is restricted to use to time frames where the CPU frequency is stable.

Note: ***Important:** when using asynchronous trace: it is important to be aware that:*

The default clock of the CKS32F10xxx MCUs is the internal RC oscillator. Its frequency under reset is different from the one after reset release. This is because the RC calibration is the default one under system reset and is updated at each system reset release.

Consequently, the trace port analyzer (TPA) should not enable the trace (with the TRACE_IOEN bit) under system reset, because a Synchronization Frame Packet will be issued with a different bit time than trace packets which will be transmitted after reset release.

22.17.9 TPIU registers

The TPIU APB registers can be read and written only if the bit TRCENA of the Debug Exception and Monitor Control Register (DEMCR) is set. Otherwise, the registers are read as zero (the output of this bit enables the PCLK of the TPIU).

Table 105 Important TPIU registers

Address	Register	Description
0xE0040004	Current port size	Allows the trace port size to be selected: Bit 0: Port size = 1 Bit 1: Port size = 2 Bit 2: Port size = 3, not supported Bit 3: Port Size = 4 Only 1 bit must be set. By default, the port size is one bit. (0x00000001)
0xE00400F0	Selected pin protocol	Allows the Trace Port Protocol to be selected: Bit1:0= 00: Sync Trace Port Mode 01: Serial Wire Output - manchester (default value) 10: Serial Wire Output - NRZ 11: reserved
0xE0040304	Formatter and flush control	Bits 31-9 = always '0 Bit 8 = TrgIn = always '1 to indicate that triggers are indicated Bits 7-4 = always 0 Bits 3-2 = always 0 Bit 1 = EnFCont. In Sync Trace mode (Select_Pin_Protocol register bit1:0=00), this bit is forced to '1: the formatter is automatically enabled in continuous mode. In asynchronous mode (Select_Pin_Protocol register bit1:0 <> 00), this bit can be written to activate or not the formatter. Bit 0 = always 0 The resulting default value is 0x102 Note: In synchronous mode, because the TRACECTL pin is not mapped outside the chip, the formatter is always enabled in continuous mode -this way the formatter inserts some control packets to identify the source of the trace packets).
0xE0040300	Formatter and flush status	Not used in Cortex [®] -M3, always read as 0x00000008

22.17.10 Example of configuration

- Set the bit TRCENA in the Debug Exception and Monitor Control Register (DEMCR)
- Write the TPIU Current Port Size Register to the desired value (default is 0x1 for a 1-bit port size)
- Write TPIU Formatter and Flush Control Register to 0x102 (default value)
- Write the TPIU Select Pin Protocol to select the sync or async mode. Example: 0x2 for async NRZ mode (UART like)
- Write the DBGMCU control register to 0x20 (bit IO_TRACEN) to assign TRACE I/Os for async mode. A TPIU Sync packet is emitted at this time (FF_FF_FF_7F)
- Configure the ITM and write the ITM Stimulus register to output a value

22.18 DBG register map

The following table summarizes the Debug registers.

Table 106 DBG register map and reset values

0xE004 2004		DBGMCU_CR	Reset value
Reserved			
			0
0	DBG_I2C1_SMBUS_TIMEOUT		
0	DBG_CAN1_STOP		
0	DBG_TIM4_STP		
0	DBG_TIM3_STP		
0	DBG_TIM2_STP		
0	DBG_TIM1_STP		
0	DBG_WWDG_STOP		
0	DBG_IWDG_STOP		
0	TRACE_MODE [1:0]		
0			
0	TRACE_IOEN		
Reserved			
0	DBG_STANDBY		
0	DBG_STOP		
0	DBG_SLEEP		

Addr.	register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0xE004 2000	DBGMCU_IDCODE	REV_ID	X															Reserved																	
	X																																		
	X																																		
	X																																		
	X																																		
	X																																		
	X																																		
	X																																		
	X																																		
	X																																		
	X																																		
	X																																		
	X																																		
	X																																		
			DEV_ID	X																	Reserved														
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			
X																																			

(1) The reset value is product dependent. For more information, refer to Section 22.6.1: MCU device ID code